

EXPERT INSIGHT



Modern Time Series Forecasting with Python

Industry-ready machine learning and deep learning
time series analysis with PyTorch and pandas

Foreword by:
Christoph Bergmeir
Senior Fellow, University of Granada

Second Edition



Manu Joseph
Jeffrey Tackes

<packt>

Modern Time Series Forecasting with Python

Second Edition

Industry-ready machine learning and deep learning time series analysis with PyTorch and pandas

Manu Joseph
Jeffrey Tackes



Packt and this book are not officially connected with Python. This book is an effort from the Python community of experts to help more developers.

Modern Time Series Forecasting with Python

Second Edition

Copyright © 2024 Packt Publishing

All rights reserved. No part of this book may be reproduced, stored in a retrieval system, or transmitted in any form or by any means, without the prior written permission of the publisher, except in the case of brief quotations embedded in critical articles or reviews.

Every effort has been made in the preparation of this book to ensure the accuracy of the information presented. However, the information contained in this book is sold without warranty, either express or implied. Neither the authors, nor Packt Publishing or its dealers and distributors, will be held liable for any damages caused or alleged to have been caused directly or indirectly by this book.

Packt Publishing has endeavored to provide trademark information about all of the companies and products mentioned in this book by the appropriate use of capitals. However, Packt Publishing cannot guarantee the accuracy of this information.

Senior Publishing Product Manager: Bhavesh Amin

Acquisition Editor – Peer Reviews: Jane D’Souza

Project Editor: Parvathy Nair

Content Development Editor: Deepayan Bhattacharjee

Copy Editor: Safis Editor

Technical Editor: Karan Sonawane

Proofreader: Safis Editor

Indexer: Hemangini Bari

Presentation Designer: Pranit Padwal

Developer Relations Marketing Executive: Anamika Singh

First published: November 2022

Second edition: October 2024

Production reference: 2300625

Published by Packt Publishing Ltd.

Grosvenor House

11 St Paul’s Square

Birmingham

B3 1RB, UK.

ISBN 978-1-83588-318-1

www.packt.com

Foreword

Forecasting as a discipline has evolved significantly. For decades, the field was dominated by simple models that often outperformed more complex ones. Machine learning methods, in various competitions, were repeatedly shown to be uncompetitive or, at best, to add little value. This period, during which I began my work in forecasting as a PhD student, has been termed the *forecasting winter* by some.

Since then, much has changed, and we now live in a different world in forecasting. With developments like the global modeling paradigm and the availability of more data and data with higher frequencies, machine learning methods have become highly competitive in many forecasting situations, and forecasting research is now driven by these approaches. Similarly, on the practitioner side, forecasting is often carried out by data scientists with a machine learning background but limited specialized training in forecasting. Their preferred programming tool is usually Python.

Manu's book is the first and most comprehensive resource reflecting this profound shift that I am aware of. It brings together many concepts and ideas into a coherent and understandable format that data scientists would otherwise struggle to acquire and understand. It should be the go-to resource for any data science practitioner working in forecasting.

This second edition acknowledges the rapid developments in the field. It offers an update using the most established software frameworks, such as the NIXTLAverse, introduces some of the latest models, and also covers methods for probabilistic forecasting, most notably conformal prediction.

Manu and his co-author for this edition, Jeffrey, both have extensive practical experience and clearly know their subject. Even I learned a few new things while reading the book.

Christoph Bergmeir

María Zambrano (Senior) Fellow

Department of Computer Science and AI

University of Granada

Spain

Contributors

About the authors

Manu Joseph is a self-made data scientist with 15 years of experience working with many Fortune 500 companies enabling digital and AI transformations, specifically in machine-learning-based Demand Forecasting. He is considered an expert, thought leader, and strong voice in the world of time series forecasting. Currently, Manu is a Staff Data Scientist at Walmart and the developer of an open-source library—PyTorch Tabular. Originally from Trivandrum, India, Manu currently resides in Bangalore, India, with his wife and son.

I extend my heartfelt gratitude to my lovely wife for her unwavering support through the years, and to Siddharth Roy for taking a chance on a newbie to the world of data science and enabling me to climb the ladder of knowledge. Even if I have not named you, thanks to everyone who has supported me and helped me on my journey.

Jeff Tackes is a seasoned data scientist specializing in Demand Forecasting with over a decade of industry experience. Currently, he is at Kraft Heinz, where he leads the research team in charge of demand forecasting. He has pioneered the development of best-in-class forecasting systems utilized by leading Fortune 500 companies. Jeff's approach combines a robust data-driven methodology with innovative strategies, enhancing forecasting models and business outcomes significantly. Leading cross-functional teams, Jeff has designed and implemented Demand Forecasting systems that have markedly improved forecast accuracy, inventory optimization, and customer satisfaction. Jeff actively contributes to open-source communities, notably to PyTimeTK, where he develops tools that enhance time series analysis capabilities. He currently resides in Chicago, IL with his wife and son.

I would like to express my deepest gratitude to my wife, whose unwavering support, patience, and encouragement have been my constant source of strength throughout this journey. Without her belief in me, this work would not have been possible. I also extend my heartfelt thanks to the many mentors, colleagues, and friends who have shared their knowledge and provided invaluable opportunities that have shaped my path. To everyone who has contributed to my growth, both personal and professional, I am deeply grateful. This achievement is as much yours as it is mine.

About the reviewers

Greg Rafferty is the author of *Forecasting Time Series Data with Prophet*. He is a data scientist in the San Francisco Bay Area with over a decade of experience working with many of the top firms in tech, including Google, Meta, and IBM. Greg has also worked as a consultant for companies in the retail sector, such as The Gap and Albertsons/Safeway, taught business analytics on Coursera, and has led face-to-face workshops with industry professionals in data science and analytics.

Raghurami Etukuru, PhD, is the originator of the concept of *Complexity-Conscious Prediction*, a novel approach that recognizes and integrates the inherent complexity of data by quantifying the complexity of input data and designing AI models tailored to this complexity.

He is an AI scientist with over 25 years of industry experience, excelling in data science and AI, with a track record of impactful AI research and model development. He is the author of several books, including *AI-Driven Time Series Forecasting: Complexity-Conscious Prediction and Decision-Making*.

Join our community on Discord

Join our community's Discord space for discussions with authors and other readers:

<https://packt.link/mts>



Table of Contents

Preface	xxiii
Making the Most Out of This Book – Get to Know Your Free Benefits	xxxiii
Part 1: Getting Familiar with Time Series	1
Chapter 1: Introducing Time Series	3
Technical requirements	3
What is a time series?	4
Types of time series • 4	
Main areas of application for time series analysis • 4	
Data-generating process (DGP)	5
Generating synthetic time series • 7	
<i>White and red noise</i> • 7	
<i>Cyclical or seasonal signals</i> • 10	
<i>Autoregressive signals</i> • 12	
<i>Mix and match</i> • 13	
Stationary and non-stationary time series • 14	
<i>Change in mean over time</i> • 14	
<i>Change in variance over time</i> • 16	
What can we forecast?	17
Forecasting terminology	18
Summary	19
Further reading	19

Chapter 2: Acquiring and Processing Time Series Data	21
Technical requirements	21
Understanding the time series dataset	22
Preparing a data model	23
pandas datetime operations, indexing, and slicing—a refresher	25
Converting the date columns into pd.Timestamp/DatetimeIndex • 25	
Using the .dt accessor and datetime properties • 27	
Indexing and slicing • 27	
Creating date sequences and managing date offsets • 28	
Handling missing data	29
Converting the half-hourly block-level data (hbblock) into time series data • 34	
Compact, expanded, and wide forms of data • 34	
Enforcing regular intervals in time series • 35	
Converting the London Smart Meters dataset into a time series format • 36	
Expanded form • 36	
Compact form • 37	
Mapping additional information	37
Saving and loading files to disk	38
Handling longer periods of missing data	39
Imputing with the previous day • 43	
Hourly average profile • 44	
The hourly average for each weekday • 45	
Seasonal interpolation • 46	
Summary	48
Chapter 3: Analyzing and Visualizing Time Series Data	49
Technical requirements	49
Components of a time series	50
The trend component • 50	
The seasonal component • 51	
The cyclical component • 51	
The irregular component • 51	
Visualizing time series data	52
Line charts • 52	
Seasonal plots • 55	
Seasonal box plots • 56	

Calendar heatmaps • 58	
Autocorrelation plot • 58	
Decomposing a time series	60
Detrending • 60	
Moving averages • 60	
LOESS • 60	
Deseasonalizing • 61	
Period adjusted averages • 61	
Fourier series • 61	
Implementations • 63	
seasonal_decompose from statsmodel • 63	
Seasonality and trend decomposition using LOESS (STL) • 64	
Fourier decomposition • 66	
Multiple seasonality decomposition using LOESS (MSTL) • 68	
Detecting and treating outliers	71
Standard deviation • 71	
IQR • 72	
Isolation Forest • 72	
Extreme studentized deviate (ESD) and seasonal ESD (S-ESD) • 73	
Treating outliers • 74	
Summary	74
References	74
Further reading	75
 Chapter 4: Setting a Strong Baseline Forecast	 77
Technical requirements	77
Setting up a test harness	78
Creating holdout (test) and validation datasets • 78	
Choosing an evaluation metric • 79	
Generating strong baseline forecasts	80
Naïve forecast • 82	
Moving average forecast • 83	
Seasonal naive forecast • 84	
Exponential smoothing • 85	
AutoRegressive Integrated Moving Average (ARIMA) • 88	
Theta forecast • 91	
TBATS • 93	

<i>Box-Cox transformation</i> • 94	
<i>Exponentially smoothed trend</i> • 96	
<i>Seasonal decomposition using Fourier series (trigonometric seasonality)</i> • 96	
ARMA • 97	
<i>Parameter optimization</i> • 98	
Multiple Seasonal-Trend decomposition using LOESS (MSTL) • 99	
Evaluating the baseline forecasts • 101	
Assessing the forecastability of a time series	103
Coefficient of variation • 103	
Residual variability • 104	
Entropy-based measures • 105	
<i>Spectral entropy</i> • 106	
Kaboudan metric • 108	
Summary	109
References	110
Further reading	110

Part 2: Machine Learning for Time Series **113**

Chapter 5: Time Series Forecasting as Regression **115**

Understanding the basics of machine learning	115
Supervised machine learning tasks • 118	
Overfitting and underfitting • 118	
Hyperparameters and validation sets • 121	
Time series forecasting as regression	122
Time delay embedding • 123	
Temporal embedding • 124	
Global forecasting models—a paradigm shift	124
Summary	127
References	127
Further reading	128

Chapter 6: Feature Engineering for Time Series Forecasting **129**

Technical requirements	129
Understanding feature engineering	130
Avoiding data leakage	131

Setting a forecast horizon	132
Time delay embedding	132
Lags or backshift • 133	
Rolling window aggregations • 134	
Seasonal rolling window aggregations • 136	
Exponentially weighted moving average (EWMA) • 139	
Temporal embedding	141
Calendar features • 142	
Time elapsed • 142	
Fourier terms • 143	
Summary	146
 Chapter 7: Target Transformations for Time Series Forecasting	 147
Technical requirements	147
Detecting non-stationarity in time series	148
Detecting and correcting for unit roots	150
Unit roots • 151	
The Augmented Dickey-Fuller (ADF) test • 152	
Differencing transform • 152	
Detecting and correcting for trends	154
Deterministic and stochastic trends • 154	
Kendall's Tau • 156	
Mann-Kendall test (M-K test) • 157	
Detrending transform • 160	
Detecting and correcting for seasonality	161
Detecting seasonality • 161	
Deseasonalizing transform • 163	
Detecting and correcting for heteroscedasticity	165
Detecting heteroscedasticity • 165	
Log transform • 166	
Box-Cox transformation • 167	
AutoML approach to target transformation	169
Summary	173
References	173
Further reading	173

Chapter 8: Forecasting Time Series with Machine Learning Models	175
Technical requirements	175
Training and predicting with machine learning models	176
Generating single-step forecast baselines	176
Standardized code to train and evaluate machine learning models	177
FeatureConfig • 177	
MissingValueConfig • 179	
ModelConfig • 180	
MLForecast • 181	
<i>The fit function</i> • 181	
<i>The predict function</i> • 181	
<i>The feature_importance function</i> • 182	
Helper functions for evaluating models • 182	
Linear regression • 183	
Regularized linear regression • 186	
<i>Regularization—a geometric perspective</i> • 187	
Decision trees • 194	
Random forest • 199	
Gradient boosting decision trees • 202	
Training and predicting for multiple households	208
Using AutoStationaryTransformer • 210	
Summary	211
References	211
Further reading	212
Chapter 9: Ensembling and Stacking	213
Technical requirements	213
Combining forecasts	214
Best fit • 215	
Measures of central tendency • 216	
Simple hill climbing • 218	
Stochastic hill climbing • 221	
Simulated annealing • 222	
Optimal weighted ensemble • 226	
Stacking and blending	228
Summary	232

References	232
Further reading	233
Chapter 10: Global Forecasting Models	235
Technical requirements	235
Why Global Forecasting Models?	236
Sample size • 236	
Cross-learning • 237	
Multi-task learning • 238	
Engineering complexity • 238	
Creating GFMs	239
Strategies to improve GFMs	241
Increasing memory • 242	
<i>Adding more lag features • 243</i>	
<i>Adding rolling features • 243</i>	
<i>Adding EWMA features • 243</i>	
Using time series meta-features • 243	
<i>Ordinal encoding and one-hot encoding • 244</i>	
<i>Frequency encoding • 245</i>	
<i>Target mean encoding • 247</i>	
<i>LightGBM's native handling of categorical features • 249</i>	
Tuning hyperparameters • 251	
<i>Grid search • 252</i>	
<i>Random search • 253</i>	
<i>Bayesian optimization • 254</i>	
Partitioning • 258	
<i>Random partition • 259</i>	
<i>Judgmental partitioning • 259</i>	
<i>Algorithmic partitioning • 260</i>	
Interpretability	265
Summary	266
References	266
Further reading	267

Part 3: Deep Learning for Time Series	269
Chapter 11: Introduction to Deep Learning	271
Technical requirements	271
What is deep learning and why now?	271
Why now? • 272	
<i>Increase in compute availability • 272</i>	
<i>Increase in data availability • 273</i>	
What is deep learning? • 275	
Perceptron, the first neural network • 276	
Components of a deep learning system	281
Representation learning • 282	
Linear transformation • 284	
Activation functions • 284	
<i>Sigmoid • 285</i>	
<i>Hyperbolic tangent (tanh) • 286</i>	
<i>Rectified linear units and variants • 286</i>	
Output activation functions • 289	
<i>Softmax • 289</i>	
Loss function • 290	
Forward and backward propagation • 290	
<i>Gradient descent • 291</i>	
Summary	298
References	298
Further reading	299
Chapter 12: Building Blocks of Deep Learning for Time Series	301
Technical requirements	301
Understanding the encoder-decoder paradigm	302
Feed-forward networks	303
Recurrent neural networks	308
RNN architecture • 308	
RNN in PyTorch • 310	
Long short-term memory (LSTM) networks	313
LSTM architecture • 313	
LSTM in PyTorch • 315	

Gated recurrent unit (GRU)	316
GRU architecture • 316	
GRU in PyTorch • 318	
Convolution networks	318
Convolution • 318	
Padding, stride, and dilations • 320	
Convolution in PyTorch • 323	
Summary	324
References	325
Further reading	326
 Chapter 13: Common Modeling Patterns for Time Series	 327
Technical requirements	327
Tabular regression	328
Single-step-ahead recurrent neural networks	332
Sequence-to-sequence (Seq2Seq) models	343
RNN-to-fully connected network • 343	
RNN-to-RNN • 345	
Summary	354
Reference	354
Further reading	354
 Chapter 14: Attention and Transformers for Time Series	 355
Technical requirements	355
What is attention?	356
The generalized attention model	358
Alignment functions • 360	
<i>Dot product</i> • 360	
<i>Scaled dot product attention</i> • 361	
<i>General attention</i> • 362	
<i>Additive/concat attention</i> • 362	
The distribution function • 363	
Forecasting with sequence-to-sequence models and attention	364
Transformers—Attention is all you need	367
Attention is all you need • 368	
<i>Self-attention</i> • 368	
<i>Multi-headed attention</i> • 370	

<i>Positional encoding</i> • 372	
<i>Position-wise feed-forward layer</i> • 375	
<i>Encoder</i> • 376	
<i>Decoder</i> • 379	
Transformers in time series • 380	
Forecasting with Transformers	381
Summary	385
References	386
Further reading	387
 Chapter 15: Strategies for Global Deep Learning Forecasting Models	 389
Technical requirements	389
Creating global deep learning forecasting models	390
Preprocessing the data • 391	
Understanding TimeSeriesDataset from PyTorch Forecasting • 394	
<i>Initializing TimeSeriesDataset</i> • 395	
<i>Creating the dataloader</i> • 396	
<i>Visualizing how the dataloader works</i> • 397	
Building the first global deep learning forecasting model • 398	
<i>Defining our first RNN model</i> • 398	
<i>Initializing the RNN model</i> • 399	
<i>Training the RNN model</i> • 399	
<i>Forecasting with the trained model</i> • 400	
Using time-varying information	402
Using static/meta information	404
One-hot encoding and why it is not ideal • 404	
Embedding vectors and dense representations • 405	
Defining a model with categorical features • 405	
Using the scale of the time series	407
Balancing the sampling procedure	408
Visualizing the data distribution • 409	
Tweaking the sampling procedure • 410	
Using and visualizing the dataloader with WeightedRandomSampler • 411	
Summary	413
Further reading	413

Chapter 16: Specialized Deep Learning Architectures for Forecasting	415
Technical requirements	416
The need for specialized architectures	416
Introduction to NeuralForecast	417
Common parameters and configurations • 417	
“Auto” models • 419	
Exogenous features • 419	
Neural Basis Expansion Analysis for Interpretable Time Series Forecasting (N-BEATS)	420
The architecture of N-BEATS • 420	
Blocks • 421	
Stacks • 422	
The overall architecture • 422	
Basis functions and interpretability • 423	
Forecasting with N-BEATS • 424	
Interpreting N-BEATS forecasting • 425	
Neural Basis Expansion Analysis for Interpretable Time Series	
Forecasting with Exogenous Variables (N-BEATSx)	426
Handling exogenous variables • 427	
Exogenous blocks • 427	
Neural Hierarchical Interpolation for Time Series Forecasting (N-HiTS)	428
The Architecture of N-HiTS • 428	
Multi-rate data sampling • 429	
Hierarchical interpolation • 429	
Synchronizing the input sampling and output interpolation • 430	
Forecasting with N-HiTS • 431	
Autoformer	431
The architecture of the Autoformer model • 432	
Uniform Input Representation • 432	
Generative-style decoder • 433	
Decomposition architecture • 434	
Auto-correlation mechanism • 436	
Forecasting with Autoformer • 438	
LTSF-Linear family of models	439
Linear • 439	
D-Linear • 440	
N-Linear • 440	
Forecasting with the LTSF-Linear family • 441	

Patch Time Series Transformer (PatchTST)	442
The architecture of the PatchTST model • 443	
<i>Patching</i> • 443	
<i>Channel independence</i> • 444	
Forecasting with PatchTST • 445	
iTransformer	446
The architecture of iTransformer • 446	
Forecasting with iTransformer • 447	
Temporal Fusion Transformer (TFT)	448
The architecture of TFT • 448	
<i>Locality Enhancement Seq2Seq layer</i> • 451	
<i>Temporal fusion decoder</i> • 452	
<i>Gated residual networks</i> • 453	
<i>Variable selection networks</i> • 454	
Forecasting with TFT • 454	
Interpreting TFT • 455	
TSMixer	456
The architecture of the TSMixer model • 457	
<i>Mixer Layer</i> • 458	
<i>Temporal Projection Layer</i> • 459	
<i>TSMixerx—TSMixer with auxiliary information</i> • 460	
Forecasting with TSMixer and TSMixerx • 461	
Time Series Dense Encoder (TiDE)	462
The architecture of the TiDE model • 462	
<i>Residual block</i> • 462	
<i>Encoder</i> • 465	
<i>Decoder</i> • 465	
Forecasting with TiDE • 466	
Summary	467
References	467
Further reading	469
 Chapter 17: Probabilistic Forecasting and More	 471
Probabilistic forecasting	472
Types of Predictive Uncertainty • 473	
What are probabilistic forecasts and Prediction Intervals? • 474	
Confidence levels, error rates, and quantiles • 475	

Measuring the goodness of prediction intervals • 476	
Probability Density Function (PDF) • 478	
<i>Forecasting with PDF—machine learning models</i> • 480	
<i>Forecasting with PDF—deep learning models</i> • 486	
Quantile function • 491	
<i>Forecasting with quantile loss (machine learning)</i> • 494	
<i>Forecasting with quantile loss (deep learning)</i> • 498	
Monte Carlo Dropout • 502	
<i>Creating a custom model in neuralforecast</i> • 505	
<i>Forecasting with MC Dropout (neuralforecast)</i> • 508	
Conformal Prediction • 514	
<i>Conformal Prediction for classification</i> • 515	
<i>Conformal Prediction for regression</i> • 516	
<i>Conformalized Quantile Regression</i> • 517	
<i>Conformalizing uncertainty estimates</i> • 519	
<i>Exchangeability in Conformal Prediction and time series forecasting</i> • 520	
<i>Forecasting with Conformal Prediction</i> • 523	
Road less traveled in time series forecasting	538
Intermittent or sparse demand forecasting • 538	
Interpretability • 539	
Cold-start forecasting • 540	
Hierarchical forecasting • 541	
Summary	541
References	541
Further reading	543

Part 4: Mechanics of Forecasting **545**

Chapter 18: Multi-Step Forecasting	547
Why multi-step forecasting?	547
Standard notation	548
Recursive strategy	549
Training regime • 549	
Forecasting regime • 550	

Direct strategy	552
Training regime • 552	
Forecasting regime • 553	
The Joint strategy	554
Training regime • 555	
Forecasting regime • 555	
Hybrid strategies	555
DirRec strategy • 555	
<i>Training regime • 556</i>	
<i>Forecasting regime • 556</i>	
Iterative block-wise direct strategy • 557	
<i>Training regime • 558</i>	
<i>Forecasting regime • 558</i>	
Rectify strategy • 560	
<i>Training regime • 561</i>	
<i>Forecasting regime • 561</i>	
RecJoint • 562	
<i>Training regime • 562</i>	
<i>Forecasting regime • 562</i>	
How to choose a multi-step forecasting strategy	563
Summary	565
References	565
 Chapter 19: Evaluating Forecast Errors—A Survey of Forecast Metrics	 567
Technical requirements	567
Taxonomy of forecast error measures	568
Intrinsic metrics • 569	
<i>Absolute error • 570</i>	
<i>Squared error • 571</i>	
<i>Percent error • 572</i>	
<i>Symmetric error • 572</i>	
<i>Other intrinsic metrics • 573</i>	
Extrinsic metrics • 574	
<i>Relative error • 574</i>	
<i>Scaled error • 575</i>	
<i>Other extrinsic metrics • 576</i>	

Investigating the error measures	576
Loss curves and complementarity • 576	
<i>Absolute error • 576</i>	
<i>Squared error • 578</i>	
<i>Percent error • 579</i>	
<i>Symmetric error • 580</i>	
<i>Extrinsic errors • 581</i>	
Bias toward over- or under-forecasting • 582	
Experimental study of the error measures	584
Using Spearman's rank correlation • 584	
Guidelines for choosing a metric	587
Summary	589
References	589
Further reading	589
 Chapter 20: Evaluating Forecasts—Validation Strategies	 591
Technical requirements	591
Model validation	592
Holdout strategies	593
Window strategy • 594	
Calibration strategy • 595	
Sampling strategy • 595	
Cross-validation strategies	599
Choosing a validation strategy	601
Validation strategies for datasets with multiple time series	602
Summary	602
References	603
Further reading	603
 Other Books You May Enjoy	 607
 Index	 611

Preface

Mankind has always sought the ability to predict the future. Since the earliest civilizations, people have tried to predict the future. Shamans, oracles, and prophets used anything ranging from astrology and palmistry to numerology to satisfy the human need to see into the future. In the last century, with the developments in IT, the mantle of predicting the future landed on data analysts and data scientists. And how do we predict the future? It's not by examining the lines and creases on our hands or the positions of the stars anymore but by using data that has been generated in the past. And instead of prophecies, we now have forecasts.

Time, being the fourth dimension in our world, makes all the data generated in the world time series data. All the data that is generated in the real world has an element of time associated with it. Whether the temporal aspect is relevant to the problem or not is another question altogether. However, to be more concrete and immediate, we can find time series forecasting use cases in many industries, such as retail, energy, healthcare, and finance. We might want to know how many units of a particular product are to be dispatched to a particular store, or we might want to know how much electricity is to be produced to meet demand.

In this book, using a real-world dataset, you will learn how to handle and visualize time series data using pandas and plotly, generate baseline forecasts using darts, and use machine learning and deep learning for forecasting, using popular Python libraries such as scikit-learn and PyTorch. We conclude the book with a few chapters that cover seldom-touched aspects, such as multi-step forecasting, forecast metrics and cross validation for time series.

The book will enable you to build real-world time series forecasting systems that scale to millions of time series by mastering and applying modern concepts in machine learning and deep learning.

Who this book is for

The book is ideal for data scientists, data analysts, machine learning engineers, and Python developers who want to build industry-ready time series models. Since the book explains most concepts from the ground up, basic proficiency in Python is all you need. A prior understanding of machine learning or forecasting would help speed up the learning. For seasoned practitioners in machine learning and forecasting, the book has a lot to offer in terms of advanced techniques and traversing the latest research frontiers in time series forecasting.

What this book covers

Part 1—Getting Familiar with Time Series

Chapter 1, Introducing Time Series, is all about introducing you to the world of time series. We lay down a definition of time series and talk about how it is related to a **Data Generating Process (DGP)**. We will also talk about the limits of forecasting and talk about what we cannot forecast, and then we finish off the chapter by laying down some terminology that will help you understand the rest of the book.

Chapter 2, Acquiring and Processing Time Series Data, covers how you can process time series data. You will understand how different forms of time series data can be represented in a tabular form. You will learn different date-time-related functionalities in pandas and learn how to fill in missing data using techniques suited for time series. Finally, using a real-world dataset, you will go through a step-by-step journey in processing time series data using pandas.

Chapter 3, Analyzing and Visualizing Time Series Data, furthers your introduction to time series by learning how to visualize and analyze time series. You will learn different visualizations that are commonly used for time series data and then learn how to go one level deeper by decomposing time series into its components. To wrap it up, you will also look at ways to identify and treat outliers in time series data.

Chapter 4, Setting a Strong Baseline Forecast, gets right to the topic of time series forecasting as we use tried and tested methods from econometrics, such as *ARIMA* and *exponential smoothing*, to generate strong baselines. These efficient forecasting methods will provide strong baselines so that we can go beyond these classical techniques and learn modern techniques, such as machine learning. You will also get an introduction to another key topic—assessing forecastability using techniques such as *spectral entropy* and *coefficient of variation*.

Part 2—Machine Learning for Time Series

Chapter 5, Time Series Forecasting as Regression, starts our journey into using machine learning for forecasting. A short introduction to machine learning lays down the foundations of what is to come in the next chapters. You will also understand, conceptually, how we can cast a time series problem as a regression problem so that we can use machine learning for it. To close off the chapter, we tease you with the possibility of global forecasting models.

Chapter 6, Feature Engineering for Time Series Forecasting, shifts gear into a more practical lesson. Using a real-world dataset, you will learn about different feature engineering techniques, such as *lag features*, *rolling features*, and *Fourier terms*, which help us formulate a time series problem as a regression problem.

Chapter 7, Target Transformations for Time Series Forecasting, continues the practice of exploring different target transformations to accommodate non-stationarity in time series. You will learn techniques such as the *augmented Dickey–Fuller test* and *Mann–Kendall test* to identify and treat non-stationarity.

Chapter 8, Forecasting Time Series with Machine Learning Models, continues from where the last chapter left off to start training machine learning models on the dataset we have been working on. Using the standard code framework present in the book, you will train models such as *linear regression*, *random forest*, and *gradient-boosted decision trees* on our dataset.

Chapter 9, Ensembling and Stacking, takes a step back and explores how we can use multiple forecasts and combine them to create a better forecast. You will explore popular techniques such as *best fit*, different versions of the *hill-climbing algorithm*, *simulated annealing*, and *stacking* to combine the different forecasts we have generated to get a better one.

Chapter 10, Global Forecasting Models, concludes your guided journey into machine learning-enabled forecasting to an exciting and new paradigm—global forecasting models. You will learn how to use global forecasting models and industry-proven techniques to improve their performance, which finally lets you develop scalable and efficient machine learning forecasting systems for thousands of time series.

Part 3—Deep Learning for Time Series

Chapter 11, Introduction to Deep Learning, we switch tracks and start with a specific type of machine learning—deep learning. In this chapter, we lay the foundations of deep learning by looking at different topics such as *representation learning*, *linear transformations*, *activation functions*, and *gradient descent*.

Chapter 12, Building Blocks of Deep Learning for Time Series, continues the journey into deep learning by making it specific to time series. Keeping in mind the compositionality of deep learning systems, you will learn about different building blocks with which you can construct a deep learning architecture. The chapter starts off by establishing the *encoder-decoder architecture* and then talks about different blocks such as *feed forward networks*, *recurrent neural networks*, and *convolutional neural networks*.

Chapter 13, Common Modeling Patterns for Time Series, strengthens the encoder-decoder architecture that you saw in the previous chapter by showing you a few concrete and common patterns in which you can arrange building blocks to generate forecasts. This is a hands-on chapter where you will be creating forecasts using deep learning-based *tabular regression* and different *sequence-to-sequence models*.

Chapter 14, Attention and Transformers for Time Series, covers the contemporary topic of using attention to improve deep learning models. The chapter starts off by talking about a generalized attention model with which you will learn different types of attention schemes, such as *scaled dot product* and *additive*. You will also tweak the sequence-to-sequence models from the previous chapter to include attention and then train those models to generate a forecast. The chapter then talks about *transformer* models, which is a deep learning architecture that relies solely on attention, and then you will use that to generate forecasts as well.

Chapter 15, Strategies for Global Deep Learning Forecasting Models, tackles yet another important aspect of deep learning-based forecasting. Although the book talked about global forecasting models earlier, there are some differences in how it is implemented for deep learning models. In this chapter, you will learn how to implement global deep learning models and techniques on how to make those models better. You will also see them working in the hands-on section, where we will be generating forecasts using the real-world dataset we have been working with.

Chapter 16, Specialized Deep Learning Architectures for Forecasting, concludes your journey into deep learning-based time series forecasting by talking about a few popular, specialized deep learning architectures for time series forecasting. Using the concepts and building blocks you have learned through the previous chapters, this chapter takes you to the cutting edge of research and exposes the leading state-of-the-art models in time series forecasting such as *N-BEATS*, *N-HiTS*, *Informer*, *Autoformer*, and *Temporal Fusion Transformer*. In addition to understanding them, you will also learn how to use these models to generate forecasts using a real-world dataset.

Chapter 17, Probabilistic Forecasting and More, take you into the realm of probabilistic forecasting with techniques like Conformal Prediction, Monte Carlo Dropout, Quantile Functions, and Probability Density Functions and enable you to practically implement these using popular open-source frameworks. The chapter also takes the road less travelled in time series forecasting and talk about Intermittent Forecasting, Interpretability, Cold Start Forecasting, and Hierarchical forecasting at a high level to serve as a starting point in your journey into those.

Part 4—Mechanics of Forecasting

Chapter 18, Multi-Step Forecasting, tackles the rarely talked-about but highly relevant topic of multi-step forecasting. You will learn about different strategies for generating forecasts for more than one time step into the future, such as *Recursive*, *Direct*, *DirRec*, *RecJoint*, and *Rectify*. The book also talks about the merits and demerits of each of them and helps you choose the right strategy for your problem.

Chapter 19, Evaluating Forecast Errors—A Survey of Forecast Metrics, traverses yet another topic that is rarely talked about and rife with controversy, with many opinions from different quarters. You will learn about different ways to measure the goodness of a forecast and through experiments, which you can run, expose the strengths and weaknesses of different metrics. The chapter concludes by laying down some guidelines that can help you choose the correct metric for your problem.

Chapter 20, Evaluating Forecasts—Validation Strategies, concludes the evaluation of forecasts and the book by talking about different validation strategies we can use for time series. You will learn different validation strategies such as hold-out, cross-validation, and their variations. The chapter also touches upon aspects to keep in mind while designing validation strategies for global settings as well. At the conclusion of the chapter, you will come across a few guidelines for choosing your validation strategies and answers to questions such as *can we use cross-validation for time series?*

To get the most out of this book

The book has to be considered bundled with the notebooks and associated code base in GitHub and is best used when both are used together. There are three levels of learning that you can do with the book. The first one is enabled by the text in the book alone and it will take you through the theory, build you intuitions and go through basic codes to get something implemented fast. To solidify the learning, we recommend you take the next step and use the provided notebooks and experiment with them. In most notebooks, we show how to do one thing in a particular way. But there are many levers, like hyperparameters, that you can tweak and run to understand how the output changes with these changes. This level of learning will make you understand the code and have a deeper understanding of the concepts you learning from the book.

And lastly, we have abstracted some code into `src` folder in the repository which is used in the notebooks. Your last and final level of learning is to go through and understand those so that you know how it works under the hood. Most of the code is well commented so that its easier for you to understand what's happening under the hood of the functions and classes you used in the notebooks. This will elevate your learning to a level where you can confidently apply these techniques to other use-cases like a boss.

You should have basic familiarity with Python programming, as the entire code that we use for the practical sections is in Python. Familiarity with major libraries in Python, such as `pandas` and `scikit-learn`, are not essential (because the book covers some basics) but will help you get through the book much faster. Familiarity with `PyTorch`, the framework the book uses for deep learning, is also not essential but would accelerate your learning. Any of the software requirements shouldn't stop you because, in today's internet-enabled world, the only thing that is standing between you and a world of knowledge is the search bar in your favorite search engine.

Setting up an environment

Setting up an environment, preferably a separate one, for the book is highly recommended. There are two main ways we suggest to create the environment—Anaconda/Mamba or a Python virtual environment.

Using Anaconda/Miniconda/Mamba

The easiest way to set up an environment is by using Anaconda, a distribution of Python for scientific computing. You can use Miniconda, a minimal installer for Conda, as well if you do not want the pre-installed packages that come with Anaconda. And you can also use Mamba, a reimplementaion of the conda package manager in C++. It is much faster than conda and is a drop-in replacement for conda. Mamba is the recommended way because it has much less chances of getting stuck at the dreaded *Resolving dependencies...* screen in Anaconda. If you are using Anaconda version 23.10 or above, then you need not worry about Mamba that much because the fast and efficient package resolver is part of anaconda by default.

1. **Install Anaconda/Miniconda/Mamba/MicroMamba:** Anaconda can be installed from <https://www.anaconda.com/products/distribution>. Depending on your operating system, choose the corresponding file and follow the instructions. Alternatively, you can install Miniconda from here: <https://docs.anaconda.com/miniconda/>. You can install Mamba and MicroMamba from here: <https://mamba.readthedocs.io/en/latest/>. If you are using Mamba, in all the instructions below replace “conda” with “mamba”.
2. **Open conda prompt:** To open Anaconda Prompt (or Terminal on Linux or macOS), do the following:
 - **Windows:** Open the Anaconda Prompt ([Start | Anaconda Prompt](#))
 - **macOS:** Open Launchpad and then open Terminal. Type `conda activate`.
 - **Linux:** Open Terminal. Type `conda activate`.

3. **Create a new environment:** Use the following command to create a new environment of your choice. For instance, to create an environment named `modern_ts_2E` with Python 3.10 (*recommended to use 3.10 or above*), use the following command:

```
conda create -n modern_ts_2E python=3.10
```

4. **Activate the environment:** Use the following command to activate the environment:

```
conda activate modern_ts_2E
```

5. **Install PyTorch from the official website:** PyTorch is best installed from the official website. Go to <https://pytorch.org/get-started/locally/> and select the appropriate options for your system. You can replace `conda` with `mamba` if you want to use Mamba to install.
6. **Navigate to the downloaded code:** Use operating system-specific commands to navigate to the folder where you have downloaded the code. For instance, in Windows, use `cd`.
7. **Install the required libraries:** Use the provided `anaconda_env.yml` file to install all the required libraries. Use the following command:

```
conda env update --file anaconda_env.yml
```

This will install all the required libraries in the environment. This can take a while.

8. **Checking the installation:** We can check if all the libraries required for the book is installed properly by executing a script in the downloaded code folder, `python test_installation.py`. If the GPU is not showing up, install PyTorch again on top of the environment.
9. **Activating the environment and Running Notebooks:** Every time you want to run the notebooks, first activate the environment using the `conda activate modern_ts_2E` command and then use Jupyter Notebook (`jupyter notebook`) or Jupyter Lab (`jupyter lab`) according to your preference.

Using Python Virtual environments and pip

If you prefer to stick to native Python for environment management, we have provided an alternate `requirements.txt` as well, which should help you with that:

1. **Install Python:** You can download Python from <https://www.python.org/downloads/>. *Recommended to use 3.10 or above.*
2. **Create a virtual environment:** Use the following command to create a virtual environment named `modern_ts_2E`:

```
python -m venv modern_ts_2E
```

3. **Activate the environment:** Use the following command to activate the environment:
 - Windows: `modern_ts_2E\Scripts\activate`
 - macOS/Linux: `source modern_ts_2E/bin/activate`
4. **Install PyTorch:** PyTorch is best installed from the official website. Go to <https://pytorch.org/get-started/locally/> and select the appropriate options for your system.

5. **Navigate to the downloaded code:** Use operating-system-specific commands to navigate to the folder where you have downloaded the code. For instance, in Windows, use `cd`.
6. **Install the required libraries:** Use the provided `requirements.txt` file to install all the required libraries. Use the following command:

```
pip install -r requirements.txt
```

This will install all the required libraries in the environment. This can take a while.

7. **Checking the installation:** We can check if all the libraries required for the book is installed properly by executing a script in the downloaded code folder

```
python test_installation.py
```

8. **Activating the environment and Running Notebooks:** Every time you want to run the notebooks, first activate the environment using the command `modern_ts_2E\Scripts\activate` (Windows) or `source modern_ts_2E/bin/activate` (macOS/Linux) and then use Jupyter Notebook (`jupyter notebook`) or Jupyter Lab (`jupyter lab`) according to your preference.

What to do when environment creation throws an error?

Considering the wide variety of computers around the world and ever-changing library dependencies, there is a very real chance that the environment setup we are providing with the book (which is tested and working as of Sept 2024) may not stand the test of time. In such cases, we recommend you open the `requirements.txt` and see which libraries we are installing and try to figure out if downgrading or upgrading versions will help you along.

Another way to tackle this will be to just install the packages that you need for a notebook when you are using those. Mostly conflicts occur when we try to install different libraries into a same environment. For instance, one of the libraries needs `PyTorch < 1.0.0`, but some other library needs `PyTorch > 1.0.0`. In such cases, it makes sense to separate the two libraries into two environments or find another version of the library that is compatible with others.

Your best friend in such situations is a Google search and subsequent trawling of GitHub comments that complain of the same issue. As a last resort, you can also raise an issue in the book repository, and we can try to help you out.

Download the data

You are going to be using a single dataset throughout the book. The book uses London Smart Meters dataset from Kaggle for this purpose. Many of the notebooks from early chapters are dependencies for some of the later chapters. As such, to remove this dependency if you want to run the notebooks out of order, we have included a `data.zip` file with all the required datasets.

To set up, follow these steps:

1. Download the data from AWS: <https://packt-modern-time-series-py.s3.eu-west-1.amazonaws.com/data.zip>.
2. Unzip the content.

3. Copy over the data folder to the Modern-Time-Series-Forecasting-with-Python-2E folder you pull from GitHub.

That's it! You are now ready to start running the code.

If you are using the digital version of this book, we advise you to type the code yourself or access the code from the book's GitHub repository. Doing so will help you avoid any potential errors related to the copying and pasting of code.

The code that is provided along with the book is in no way a library but more of a guide for you to start experimenting on. The amount of learning you can derive from the book and code is directly proportional to how much you experiment with the code and stray outside your comfort zone. So, go ahead and start experimenting and putting the skills you pick up in the book to good use.

Download the example code files

You can download the example code files for this book from GitHub at <https://github.com/PacktPublishing/Modern-Time-Series-Forecasting-with-Python-2E>. If there's an update to the code, it will be updated in the GitHub repository.

We also have other code bundles from our rich catalog of books and videos available at <https://github.com/PacktPublishing/>. Check them out!

Download the color images

We also provide a PDF file that has color images of the screenshots and diagrams used in this book. You can download it here: <https://packt.link/gbp/9781835883181>.

Conventions used

There are a number of text conventions used throughout this book.

Code in text: Indicates code words in text, database table names, folder names, filenames, file extensions, pathnames, dummy URLs, user input, and Twitter handles. Here is an example: “statsmodels.tsa.seasonal has a function called seasonal_decompose.”

A block of code is set as follows:

```
#Does not support missing values, so using imputed ts instead
res = seasonal_decompose(ts, period=7*48, model="additive", extrapolate_
trend="freq")
```

Any command-line input or output is written as follows:

```
conda env create -f anaconda_env.yml
```

Bold: Indicates a new term, an important word, or words that you see onscreen. For instance, words in menus or dialog boxes appear in **bold**. Here is an example: “But if you look at the **Time Elapsed** column, it stands out.”



IMPORTANT NOTES Appear like this.



Tips Appear like this.

Get in touch

Feedback from our readers is always welcome.

General feedback: If you have questions about any aspect of this book, email us at customer care@packtpub.com and mention the book title in the subject of your message.

Errata: Although we have taken every care to ensure the accuracy of our content, mistakes do happen. If you have found a mistake in this book, we would be grateful if you would report this to us. Please visit www.packtpub.com/support/errata and fill in the form.

Piracy: If you come across any illegal copies of our works in any form on the internet, we would be grateful if you would provide us with the location address or website name. Please contact us at copyright@packt.com with a link to the material.

If you are interested in becoming an author: If there is a topic that you have expertise in and you are interested in either writing or contributing to a book, please visit authors.packtpub.com.

Share your thoughts

Once you've read *Modern Time Series Forecasting with Python, Second Edition*, we'd love to hear your thoughts! Please [click here](#) to go straight to the Amazon review page for this book and share your feedback.

Your review is important to us and the tech community and will help us make sure we're delivering excellent quality content.

Making the Most Out of This Book – Get to Know Your Free Benefits

Unlock exclusive free benefits that come with your purchase, thoughtfully crafted to supercharge your learning journey and help you learn without limits.

UNLOCK NOW



Note: Have your purchase invoice ready before you begin. <https://www.packtpub.com/unlock/9781835883181>



Figure 1.1: Next-Gen Reader, AI Assistant (Beta), and Free PDF access

Enhanced reading experience with our Next-gen Reader:

- 🔄 Multi-device progress sync: Learn from any device with seamless progress sync.
- 📖 Highlighting and Notetaking: Turn your reading into lasting knowledge.
- 🔖 Bookmarking: Revisit your most important learnings anytime.
- 🌙 Dark mode: Focus with minimal eye strain by switching to dark or sepia modes.

Learn smarter using our AI assistant (Beta):

- 💡 Summarize it: Summarize key sections or an entire chapter.
- 💡 AI code explainers: In Packt Reader, click the “Explain” button above each code block for AI-powered code explanations.

Note: AI Assistant is part of next-gen Packt Reader and is still in beta.

Learn anytime, anywhere:

- 📖 Access your content offline with DRM-free PDF and ePub versions—compatible with your favorite e-readers.

Unlock Your Book's Exclusive Benefits

Your copy of this book comes with the following exclusive benefits:

 Next-gen Packt Reader

 AI assistant (beta)

 DRM-free PDF/ePub downloads

Use the following guide to unlock them if you haven't already. The process takes just a few minutes and needs to be done only once.

How to unlock these benefits in three easy steps

Step 1

Have your purchase invoice for this book ready, as you'll need it in *Step 3*. If you received a physical invoice, scan it on your phone and have it ready as either a PDF, JPG, or PNG.

For more help on finding your invoice, visit <https://www.packtpub.com/unlock-benefits/help>.



Note: Bought this book directly from Packt? You don't need an invoice. After completing Step 2, you can jump straight to your exclusive content.

Step 2

Scan the following QR code or visit <https://www.packtpub.com/unlock/9781835883181>:



Step 3

Sign in to your Packt account or create a new one for free. Once you're logged in, upload your invoice. It can be in PDF, PNG, or JPG format and must be no larger than 10 MB. Follow the rest of the instructions on the screen to complete the process.

Need help?

If you get stuck and need help, visit <https://www.packtpub.com/unlock-benefits/help> for a detailed FAQ on how to find your invoices and more. The following QR code will take you to the help page directly:



Note: If you are still facing issues, reach out to customer@packt.com.

Part 1

Getting Familiar with Time Series

We dip our toes into time series forecasting by understanding what a time series is, how to process and manipulate time series data, and how to analyze and visualize time series data. This part also covers classical time series forecasting methods, such as ARIMA, to serve as strong baselines.

This part comprises the following chapters:

- *Chapter 1, Introducing Time Series*
- *Chapter 2, Acquiring and Processing Time Series Data*
- *Chapter 3, Analyzing and Visualizing Time Series Data*
- *Chapter 4, Setting a Strong Baseline Forecast*

1

Introducing Time Series

Welcome to *Modern Time Series Forecasting with Python*! This book is intended for data scientists or **machine learning (ML)** engineers who want to level up their time series analysis skills by learning new and advanced techniques from the ML world. **Time series analysis** is something that is commonly overlooked in regular ML books, courses, and so on. They typically start with classification, touch upon regression, and then move on. But it is also something that is immensely valuable and ubiquitous in business. We look at the world from a three-dimensional perspective. Time is the hidden dimension that we rarely think about, but is all-pervasive. And as long as time is one of the four dimensions in the world we live in, time series data is all-pervasive too.

Analyzing time series data unlocks a lot of value for a business. Time series analysis isn't new—it's been around since the 1920s. But in the current age of data, the time series that are collected by businesses are growing larger and wider by the minute. Combined with an explosion in the quantum of data collected and the renewed interest in ML, the landscape of time series analysis also changed considerably. This book attempts to take you beyond classical statistical methods such as **AutoRegressive Integrated Moving Average (ARIMA)** and introduce to you the latest techniques from the ML world in time series analysis.

We are going to start with some fundamental concepts and quickly scale up to more complex topics. In this chapter, we're going to cover the following main topics:

- What is a time series?
- Data-generating process (DGP)
- What can we forecast?
- Forecasting terminology and notation

Technical requirements

You will need to set up the **Anaconda** environment by following the instructions in the *Preface* of the book to get a working environment with all the libraries and datasets required for the code in this book. Any additional library will be installed while running the notebooks.

The associated code for the chapter can be found at <https://github.com/PacktPublishing/Modern-Time-Series-Forecasting-with-Python-2E/tree/main/notebooks/Chapter01>.

What is a time series?

To keep it simple, a **time series** is a set of observations taken sequentially in time. The focus is on the word *time*. If we keep taking the same observation at different points in time, we will get a time series. For example, if you keep recording the number of bars of chocolate you have in a month, you'll end up with a time series of your chocolate consumption. Suppose you are recording your weight at the beginning of every month. You get another time series of your weight. Is there any relation between the two time series? Most likely, yeah. But we will be able to analyze that scientifically by the end of this book.

A few other examples of time series are the weekly closing price of a stock that you follow, daily rainfall or snowfall in your city, and hourly readings of your pulse rate from your smartwatch.

Types of time series

There are two types of time series data based on time intervals, as outlined here:

- **Regular time series:** This is the most common type of time series, where we have observations coming in at regular intervals of time, such as every hour or every month. For example, if we take a time series of temperature in a city, we will get the time series in a regular interval (whichever frequency we choose for observation).
- **Irregular time series:** There are a few time series where we do not have observations at regular intervals of time. For example, consider we have a sequence of readings from lab tests of a patient. We see an observation in the time series only when the patient heads to the clinic and carries out the lab test, and this may not happen at regular intervals.



This book only focuses on regular time series, which are evenly spaced in time. Irregular time series are slightly more advanced and require specialized techniques to handle them. A couple of survey papers on the topic is a good way to get started on irregular time series, and you can find them in the *Further reading* section of this chapter.

Main areas of application for time series analysis

There are broadly three important areas of application for time series analysis, outlined as follows:

- **Time series forecasting:** Predicting the future values of a time series, given the past values—for example, predict the next day's temperature using the last 5 years of temperature data. This use case is one of the most popular and important ones because any kind of planning we need to do needs some visibility into the future. For instance, planning how many chocolates to produce next month needs a forecast of expected demand.

- **Time series classification:** Sometimes, instead of predicting the future value of the time series, we may also want to predict an action based on past values. For example, given historical measurements from an **electroencephalogram** (EEG; tracking electrical activity in the brain) or an **electrocardiogram** (EKG; tracking electrical activity in the heart), we need to predict whether the result of an EEG or an EKG is normal or abnormal.
- **Outlier detection:** There are some situations where we only want to detect if something is going wrong or if something is out of the ordinary. In such cases, we need to use classification or forecasting, but instead, we can do outlier detection. For instance, the wearable tech on your body records accelerometer readings across time and can use outlier detection to identify falls or accidents.
- **Interpretation and causality:** You can use time-series analysis to understand the whats and whys of the time series based on past values, understand the relationships between several related time series, or derive causal inferences based on time series data. For example, we have a time series of market share for a brand and another time series of advertising spend. Using interpretation and causality techniques, we can start to understand how much advertising investment is affecting the market share and possibly take appropriate action.



The focus of this book is predominantly on *time series forecasting*, but the techniques that you learn will help you approach *time series classification* problems also, with minimal changes in the approach. *Interpretation* is also addressed, although only briefly, but *causality* is an area that this book does not address because it warrants a whole different approach.

Now that we have an overview of the time series landscape, let's build a mental model of how time series data is generated.

Data-generating process (DGP)

We have seen that time series data is a collection of observations made sequentially along the time dimension. Any time series is, in turn, generated by some kind of *mechanism*. For example, time series data of daily shipments of your favorite chocolate from the manufacturing plant is affected by a lot of factors, such as the time of the year (holiday season, for example), the availability of cocoa, the uptime of the machines working on the plant, and so on. In statistics, this underlying process that generates the time series is referred to as the **DGP**. Time series data is produced by stochastic and deterministic processes. The deterministic processes involve quantities that evolve in a predictable manner over time. An example of this is the radioactive decay of an element, where the remaining quantity diminishes according to a precise mathematical formula, leading to a consistent reduction over time. But most of the interesting time series (from a forecasting perspective) are generated by a stochastic process. A stochastic process is a way to describe how things change over time in a random but somewhat predictable manner, like how the weather changes daily with some patterns and probabilities involved. So, let's discuss more about time series generated from stochastic processes.

If we had complete and perfect knowledge of reality, all we would need to do would be to put this DGP together in a mathematical form and you would get the most accurate forecast possible. But sadly, nobody has complete and perfect knowledge of reality. So, what we try to do is approximate the DGP, mathematically, as much as possible so that our imitation of the DGP gives us the best possible forecast (or any other output we want from the analysis). This imitation is called a **model** that provides a useful approximation of the DGP.

But we must remember that the model is not the DGP, but a representation of some essential aspects of reality. For example, let's consider an aerial view of Bengaluru and a map of Bengaluru, as represented here:



Figure 1.1: An aerial view of Bengaluru (left) and a map of Bengaluru (right)

The map of Bengaluru is certainly useful—we can use it to go from point A to point B. But a map of Bengaluru is not the same as a photo of Bengaluru. It doesn't showcase the bustling nightlife or the insufferable traffic. A map is just a model that represents some useful features of a location, such as roads and places. The following diagram might help us internalize the concept and remember it:

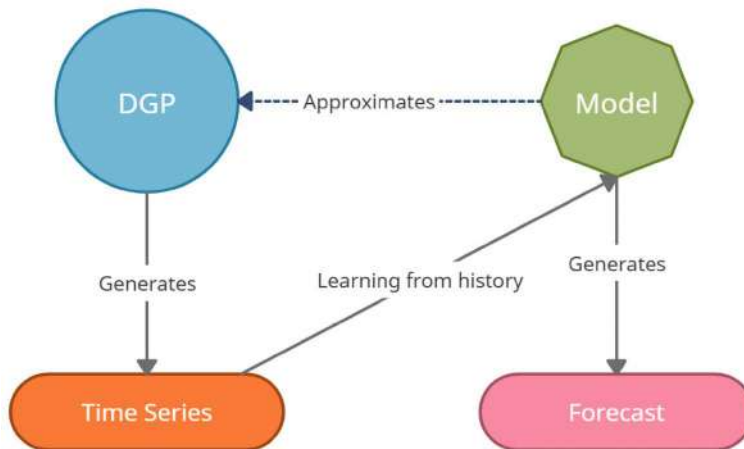


Figure 1.2: DGP, model, and time series

Naturally, the next question would be this: *Do we have a useful model?* Every model has limitations and challenges. As we have seen, a map of Bengaluru does not perfectly represent Bengaluru. But if our purpose is to navigate Bengaluru, then a map is a very useful model. What if we want to understand the culture? A map doesn't give you a flavor of that. So, now, the same model that was useful is utterly useless in the new context.

Different kinds of models are required in different situations and for different objectives. For example, the best model for forecasting may not be the same as the best model for making a causal inference.

We can use the concept of DGPs to generate multiple synthetic time series of varying degrees of complexity.

Generating synthetic time series

Synthetic time series, or artificial time series, are excellent tools with which you can understand the time series space, experiment with different techniques, and even test new models or modeling setups. These time series are designed to be predictable, even though a bit challenging. Let's take a look at a few practical examples where we can generate a few time series using a set of fundamental building blocks. You can get creative and mix and match any of these components, or even add them together to generate a time series of arbitrary complexity.

White and red noise

An extreme case of a stochastic process that generates a time series is a **white noise** process. It has a sequence of random numbers with zero mean and constant variance. This is also one of the most popular assumptions of noise in a time series.

Let's see how we can generate such a time series and plot it:

```
# Generate the time axis with sequential numbers upto 200
time = np.arange(200)
# Sample 200 hundred random values
values = np.random.randn(200)*100
plot_time_series(time, values, "White Noise")
```

💡 **Quick tip:** Enhance your coding experience with the **AI Code Explainer** and **Quick Copy** features. Open this book in the next-gen Packt Reader. Click the **Copy** button (1) to quickly copy code into your coding environment, or click the **Explain** button (2) to get the AI assistant to explain a block of code to you.

```
function calculate(a, b) {  
  return {sum: a + b};  
};
```

Copy

Explain

1

2



🔒 The next-gen Packt Reader is included for free with the purchase of this book. Unlock it by scanning the QR code below or visiting <https://www.packtpub.com/unlock/9781835883181>.



Here is the output:

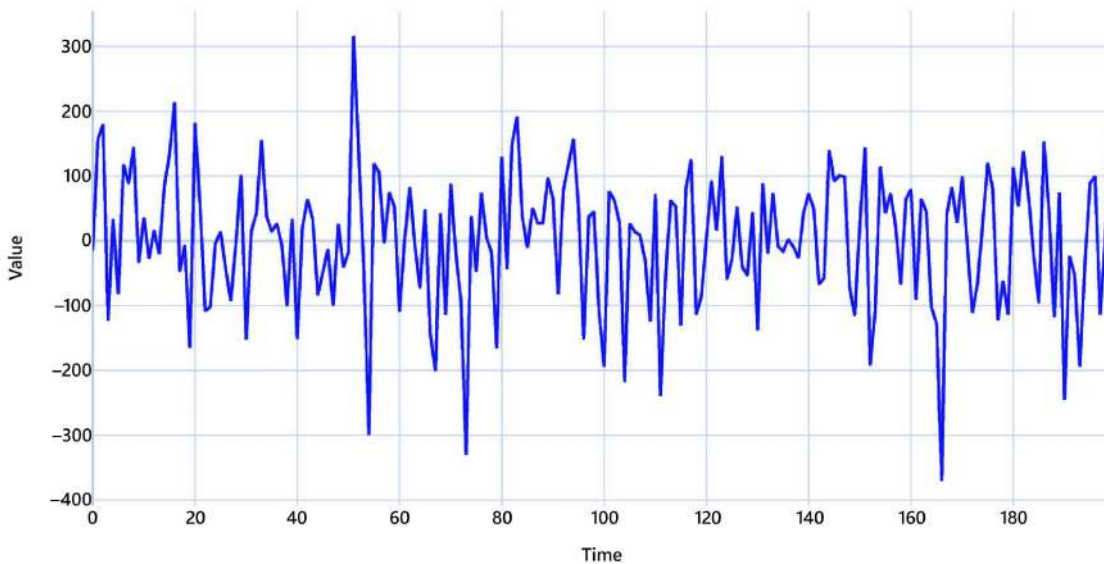


Figure 1.3: White noise process

Red noise, on the other hand, has zero mean and constant variance but is serially correlated in time. This serial correlation or redness is parameterized by a correlation coefficient r , such that:

$$x_{j+1} = r \cdot x_j + (1 - r^2)^{\frac{1}{2}} \cdot w$$

where w is a random sample from a white noise distribution.

Let's see how we can generate that, as follows:

```
# Setting the correlation coefficient
r = 0.4
# Generate the time axis
time = np.arange(200)
# Generate white noise
white_noise = np.random.randn(200)*100
# Create Red Noise by introducing correlation between subsequent values in the
white noise
values = np.zeros(200)
for i, v in enumerate(white_noise):
    if i==0:
        values[i] = v
    else:
        values[i] = r*values[i-1]+ np.sqrt((1-np.power(r,2))) *v
plot_time_series(time, values, "Red Noise Process")
```

Here is the output:

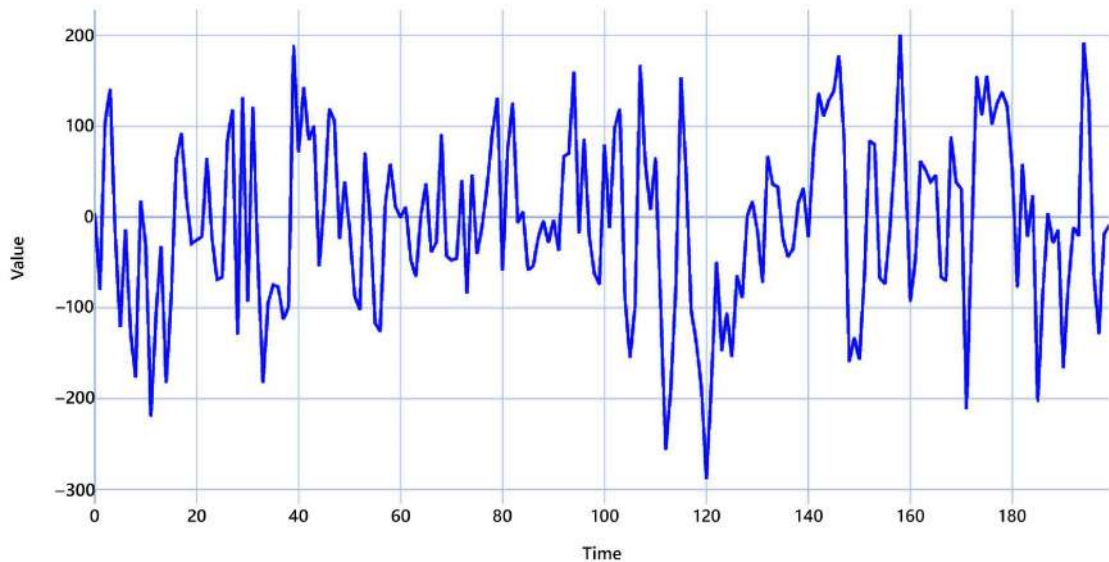


Figure 1.4: Red noise process

Cyclical or seasonal signals

Among the most common signals you see in time series are seasonal or cyclical signals. Therefore, you can introduce seasonality into your generated series in a few ways.

Let's take the help of a very useful library to generate the rest of the time series—TimeSynth. For more information, refer to <https://github.com/TimeSynth/TimeSynth>.

This is a useful library for generating time series. It has all kinds of DGPs that you can mix and match and create an authentic synthetic time series.



Notebook alert:

For the exact code and usage, please refer to the associated Jupyter notebooks.

Let's see how we can use a sinusoidal function to create cyclicity. There is a helpful function in TimeSynth called `generate_timeseries` that helps us combine signals and generate time series. Have a look at the following code snippet:

```
#Sinusoidal Signal with Amplitude=1.5 & Frequency=0.25
signal_1 = ts.signals.Sinusoidal(amplitude=1.5, frequency=0.25)
#Sinusoidal Signal with Amplitude=1 & Frequency=0.5
signal_2 = ts.signals.Sinusoidal(amplitude=1, frequency=0.5)
#Generating the time series
samples_1, regular_time_samples, signals_1, errors_1 = generate_
timeseries(signal=signal_1)
samples_2, regular_time_samples, signals_2, errors_2 = generate_
timeseries(signal=signal_2)
plot_time_series(regular_time_samples,
                  [samples_1, samples_2],
                  "Sinusoidal Waves",
                  legends=["Amplitude = 1.5 | Frequency = 0.25", "Amplitude = 1
| Frequency = 0.5"])
```

Here is the output:

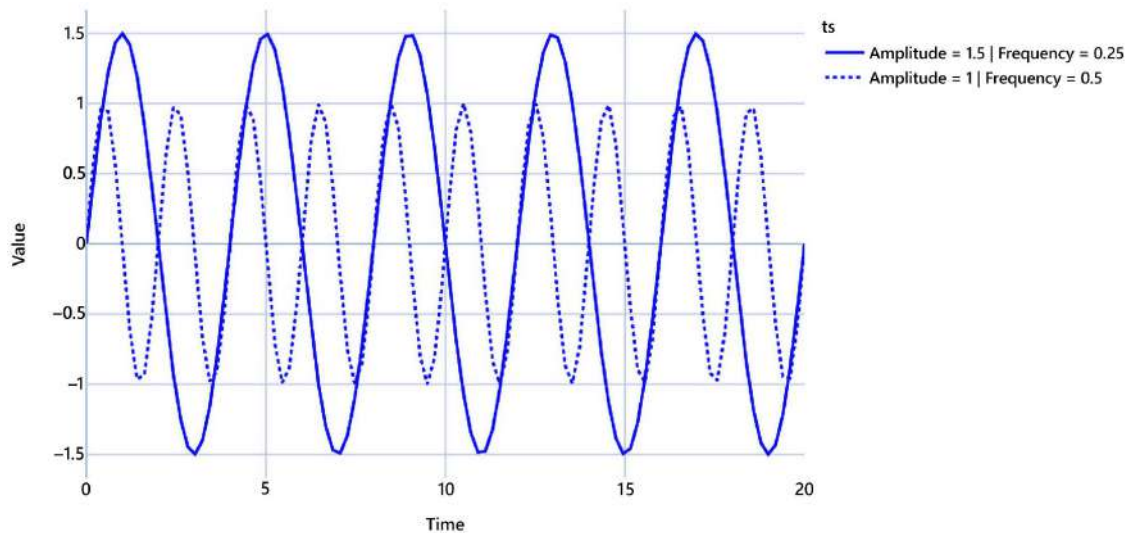


Figure 1.5: Sinusoidal waves

Note the two sinusoidal waves are different with respect to the frequency (how fast the time series crosses zero) and amplitude (how far away from zero the time series travels).

TimeSynth also has another signal called PseudoPeriodic. This is like the Sinusoidal class, but the frequency and amplitude have some stochasticity. We can see in the following code snippet that this is more realistic than the vanilla sine and cosine waves from the Sinusoidal class:

```
# PseudoPeriodic signal with Amplitude=1 & Frequency=0.25
signal = ts.signals.PseudoPeriodic(amplitude=1, frequency=0.25)
#Generating Timeseries
samples, regular_time_samples, signals, errors = generate_
timeseries(signal=signal)
plot_time_series(regular_time_samples,
                 samples,
                 "Pseudo Periodic")
```

Here is the output:

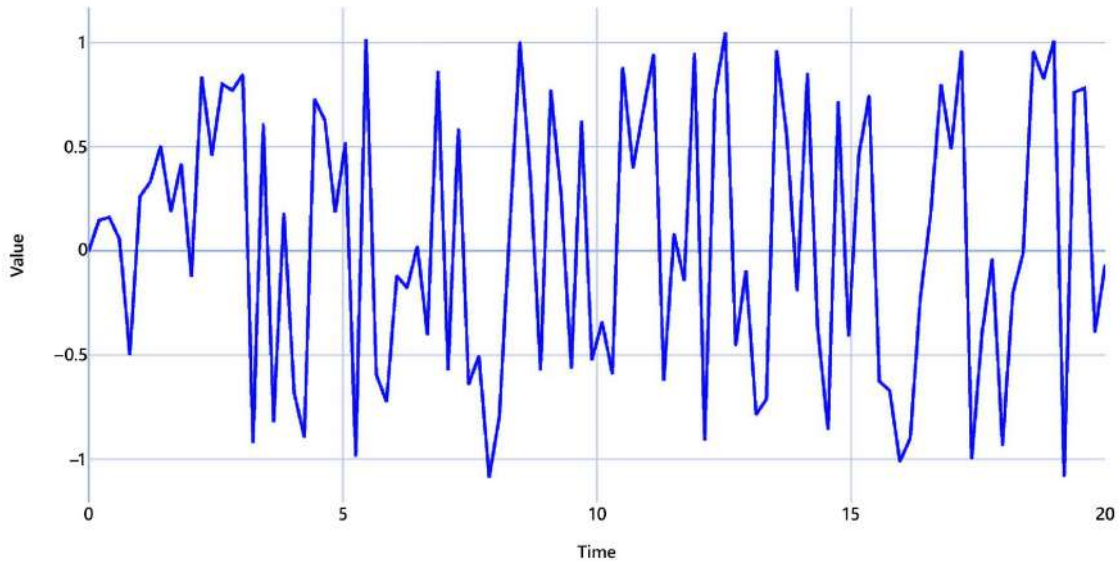


Figure 1.6: Pseudo-periodic signal

Autoregressive signals

Another very popular signal in the real world is an **autoregressive (AR) signal**. We will go into this in more detail in *Chapter 4, Setting a Strong Baseline Forecast*, but for now, an AR signal refers to when the value of a time series for the current timestep is dependent on the values of the time series in the previous timesteps. This serial correlation is a key property of the AR signal, and it is parametrized by a few parameters, outlined as follows:

- Order of serial correlation—or, in other words, the number of previous timesteps the signal is dependent on
- Coefficients to combine the previous timesteps

Let's see how we can generate an AR signal and see what it looks like, as follows:

```
# We have re-implemented the class in src because of a bug in TimeSynth
from src.synthetic_ts.autoregressive import Autoregressive
# Autoregressive signal with parameters 1.5 and -0.75
#  $y(t) = 1.5*y(t-1) - 0.75*y(t-2)$ 
signal= Autoregressive(ar_param=[1.5, -0.75])
#Generate Timeseries
samples, regular_time_samples, signals, errors = generate_
timeseries(signal=signal)
plot_time_series(regular_time_samples,
                 samples,
                 "Auto Regressive")
```

Here is the output:

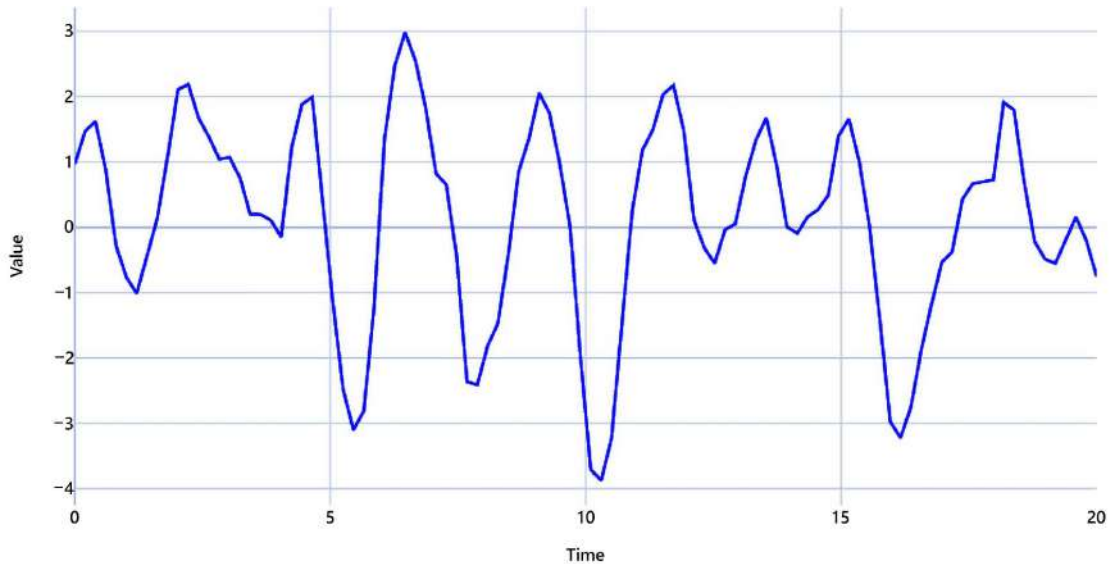


Figure 1.7: AR signal

Mix and match

There are many more components that you can use to create your DGP and thereby generate a time series, but let's quickly look at how we can combine the components we have already seen to generate a realistic time series.

Let's use a pseudo-periodic signal with white noise and combine it with an AR signal, as follows:

```
#Generating Pseudo Periodic Signal
pseudo_samples, regular_time_samples, _, _ = generate_timeseries(signal=ts.
signals.PseudoPeriodic(amplitude=1, frequency=0.25), noise=ts.noise.
GaussianNoise(std=0.3))

# Generating an Autoregressive Signal
ar_samples, regular_time_samples, _, _ = generate_timeseries(signal=
AutoRegressive(ar_param=[1.5, -0.75]))

# Combining the two signals using a mathematical equation
ts = pseudo_samples*2+ar_samples
plot_time_series(regular_time_samples,
                 ts,
                 "Pseudo Periodic with AutoRegression and White Noise")
```

Here is the output:

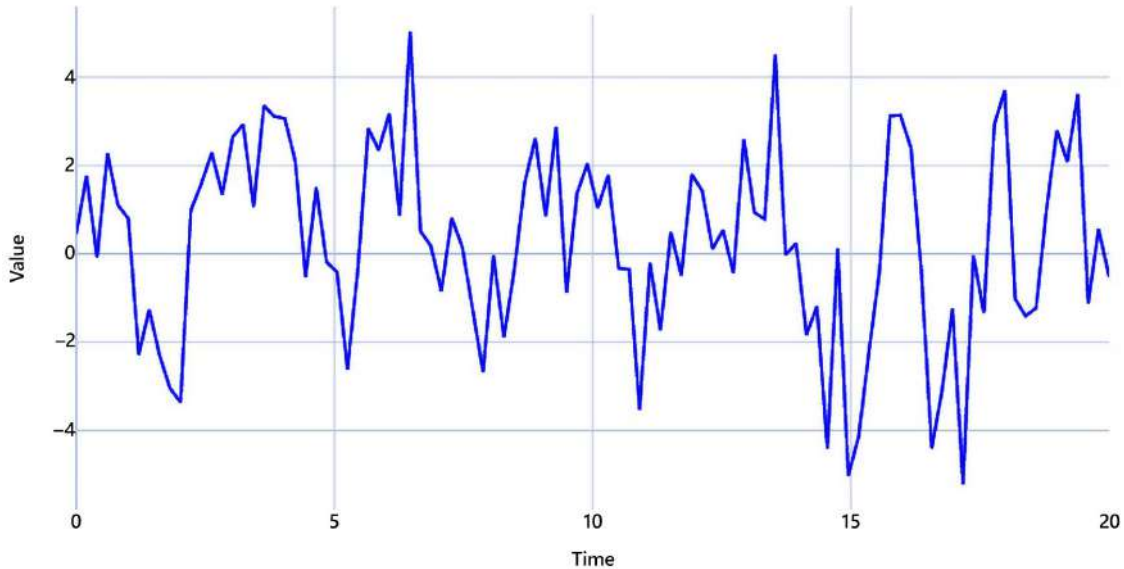


Figure 1.8: Pseudo-periodic signal with AR and white noise

Stationary and non-stationary time series

In time series, **stationarity** is of great significance and is a key assumption in many modeling approaches. Ironically, many (if not most) real-world time series are non-stationary. So, let's understand what a stationary time series is from a layman's point of view.

There are multiple ways to look at stationarity, but one of the clearest and most intuitive ways is to think of the probability distribution or the data distribution of a time series. We call a time series stationary when the probability distribution remains the same at every point in time. In other words, if you pick different windows in time, the data distribution across all those windows should be the same.

A standard Gaussian distribution is defined by two parameters—the mean and the variance. So, there are two ways the stationarity assumption can be broken, as outlined here:

- Change in mean over time
- Change in variance over time

Let's look at these assumptions in detail and understand them better.

Change in mean over time

This is the most popular way a non-stationary time series presents itself. If there is an upward/downward trend in the time series, the mean across two windows of time would not be the same.

Another way non-stationarity manifests itself is in the form of seasonality. Suppose we are looking at the time series of average temperature measurements per month for the last 5 years. From our experience, we know that temperature peaks during summer and falls in winter. So, when we take the mean temperature of winter and the mean temperature of summer, they will be different.

Let's generate a time series with trend and seasonality and see how it manifests:

```
# Sinusoidal Signal with Amplitude=1 & Frequency=0.25
signal=ts.signals.Sinusoidal(amplitude=1, frequency=0.25)
# White Noise with standard deviation = 0.3
noise=ts.noise.GaussianNoise(std=0.3)
# Generate the time series
sinusoidal_samples, regular_time_samples, _, _ = generate_
timeseries(signal=signal, noise=noise)
# Regular_time_samples is a linear increasing time axis and can be used as a
trend
trend = regular_time_samples*0.4
# Combining the signal and trend
ts = sinusoidal_samples+trend
plot_time_series(regular_time_samples,
                 ts,
                 "Sinusoidal with Trend and White Noise")
```

Here is the output:



Figure 1.9: Sinusoidal signal with trend and white noise

If you examine the time series in *Figure 1.9*, you will be able to see a definite trend and the seasonality, which together make the mean of the data distribution change wildly across different windows of time.

Change in variance over time

Non-stationarity can also present itself in the fluctuating variance of a time series. If the time series starts off with low variance and as time progresses, the variance keeps getting bigger and bigger, we have a non-stationary time series. In statistics, there is a scary name for this phenomenon—**heteroscedasticity**. The Air Passengers dataset, which is the “iris dataset” of time series (the most popular, over-used, and useless) is a classic example of a heteroscedastic time series. Let’s look at the plot:

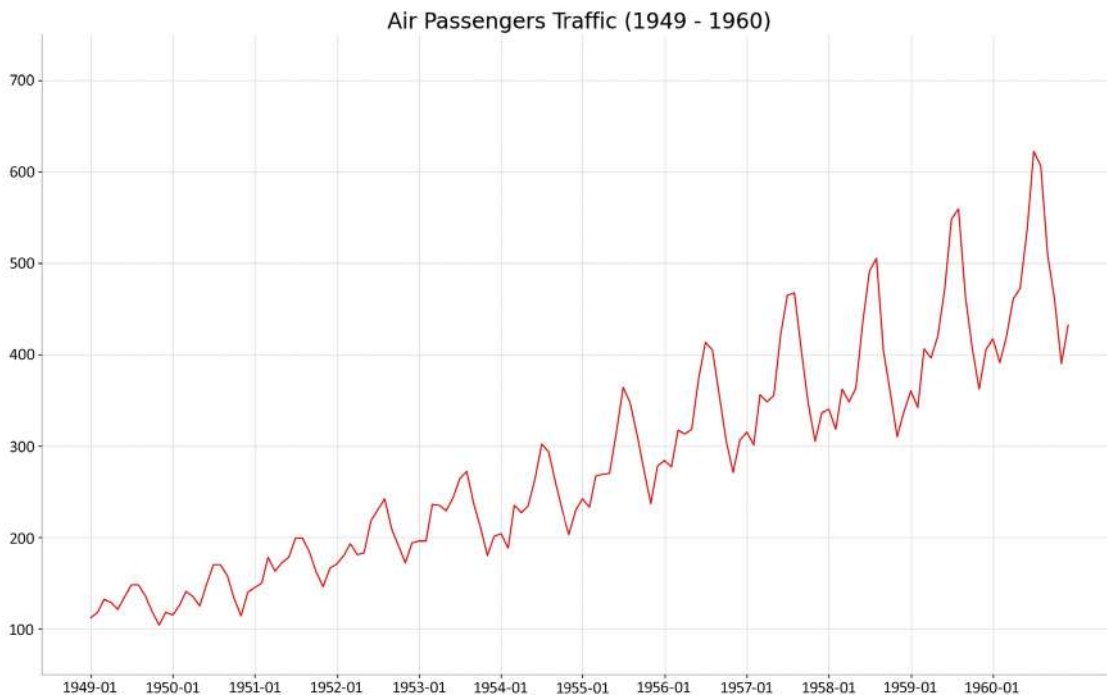


Figure 1.10: Air Passengers dataset—Example of a heteroscedastic time series

In the figure, you can see that the seasonal peaks keep getting wider and wider as we move through time, and this is a classic sign that the time series is heteroscedastic. But not all heteroscedastic time series are easy to spot. We have statistical tests to check for each of the stationarity cases, which we will cover in *Chapter 7, Target Transformations for Time Series Forecasting*.

This book just tries to give you an understanding of stationary and non-stationary time series. There is a lot of statistical theory and depth in this discussion that we are skipping to keep our focus on the practical aspects of time series.

Armed with the mental model of the DGP, we are at the right place to think about another important question: *what can we forecast?*

What can we forecast?

Before we move ahead, there is another aspect of time series forecasting that we have to understand—the *predictability of a time series*. The most basic assumption when we forecast a time series is that the future depends on the past. But not all time series are equally predictable.

Let's take a look at a few examples and try to rank these in order of predictability (from easiest to hardest), as follows:

- High tide next Monday
- Lottery numbers next Sunday
- The stock price of Tesla next Friday

Intuitively, it is very easy for us to rank them. High tide next Monday is going to be the easiest to predict because it is so predictable, the stock price of Tesla next Friday is going to be difficult to predict, but not impossible, and the lottery numbers are going to be very hard to predict because they are pretty much random.



However, for people thinking that they can forecast stock prices with the advanced techniques covered in the book and get rich, that (most likely) won't happen. Although it is worthy of a lengthy discussion, we can summarize the key points in a short paragraph.

Share prices are not a function of their past values but an anticipation of their future values, and this thereby violates our first assumption while forecasting. And if that is not bad enough, financial stock prices typically have a very low signal-to-noise ratio. The final wrench in the process is the **efficient market hypothesis (EMH)**. This seemingly innocent hypothesis proclaims that all known information about a stock price is already factored into the price of the stock. The implication of the hypothesis is that if you can forecast accurately, many others will also be able to do that, and thereby the market price of the stock already reflects the change in price that this forecast brought about.

The M6 competition chose to tackle this problem head-on to evaluate if the EMH holds true by conducting a year-long forecasting and investment strategy competition. Although not conclusive, the results show that the EMH holds true for the vast majority of the participants, barring a few top teams. And even in that, they found out that in the top teams, there was no significant correlation between forecasting accuracy and the selection of stocks into the portfolio, i.e. the teams weren't choosing stocks which they were able to forecast better (a link to the full report is provided in the *Further reading* section).

Coming back to the topic at hand—predictability—three main factors form a mental model for this, as follows:

- **Understanding the DGP:** The better you understand the DGP, the higher the predictability of a time series.
- **Amount of data:** The more data you have, the better your predictability is.

- **Adequately repeating pattern:** For any mathematical model to work well, there should be an adequately repeating pattern in your time series. The more repeatable the pattern is, the better your predictability is.

Even though you have a mental model of how to think about predictability, we will look at more concrete ways of assessing the predictability of time series in *Chapter 3, Analyzing and Visualizing Time Series Data*, but the key takeaway is that not all time series are equally predictable.

In order to fully follow the discussion in the coming chapters, we need to establish a standard notation and learn the terminology that is specific to time series analysis.

Forecasting terminology

There are a few terms that will help you understand this book as well as other literature on time series. These terms are described in more detail here:

- **Forecasting**

Forecasting is the prediction of future values of a time series using the known past values of the time series and/or some other related variables. This is very similar to prediction in ML, where we use a model to predict unseen data.

- **Multivariate forecasting**

Multivariate time series consist of more than one time series variable that is not only dependent on its past values but also has some dependency on the other variables. For example, a set of macroeconomic indicators, such as **gross domestic product (GDP)** and inflation, of a particular country can be considered a multivariate time series. The aim of multivariate forecasting is to come up with a model that captures the interrelationship between the different variables along with its relationship with its past and forecast all the time series together in the future.

- **Explanatory forecasting**

In addition to the past values of a time series, we might use some other information to predict the future values of a time series. For example, when predicting retail store sales, information regarding promotional offers (both historical and future ones) is usually helpful. This type of forecasting, which uses information other than its own history, is called explanatory forecasting.

- **Backtesting**

Setting aside a validation set from your training data to evaluate your models is a practice that is common in the ML world. Backtesting is the time series equivalent of validation, whereby you use the history to evaluate a trained model. We will cover the different ways of doing validation and cross-validation for time series data later.

- **In-sample and out-sample**

Again drawing parallels with ML, in-sample refers to training data and out-sample refers to unseen or testing data. When you hear in-sample metrics, this refers to metrics calculated on training data, and out-sample metrics refers to metrics calculated on testing data.

- **Exogenous and endogenous variables**

Exogenous variables are parallel time series variables that are not modeled directly for output but used to help us model the time series that we are interested in. Typically, exogenous variables are not affected by other variables in the system. Endogenous variables are variables that are affected by other variables in the system. A purely endogenous variable is a variable that is entirely dependent on the other variables in the system. Relaxing the strict assumptions a bit, we can consider the target variable as the endogenous variable and the explanatory regressors we include in the model as exogenous variables.

- **Forecast combination**

Forecast combinations in the time series world are similar to ensembles from the ML world. Forecast combination is a process by which we combine multiple forecasts by using a function, either learned or heuristic-based, such as a simple average of three forecast models.

There are a lot more terms that are specific to time series, some of which we will be covering throughout the book. But these terms should be a good starting point to give you basic familiarity in the field.

Summary

In this chapter, we had our first look at time series as we discussed the different types of time series, looked at how a DGP generates a time series, and saw how we can think about the important question: *how well can we forecast a time series?* We also had a quick review of the terminology required to understand the rest of the book. In the next chapter, we will be getting our hands dirty and will learn how to acquire and process time series data. If you have not set up the environment yet, take a break and put some time into doing that.

Further reading

- *A Survey on Principles, Models and Methods for Learning from Irregularly Sampled Time Series: From Discretization to Attention and Invariance* by S.N. Shukla and B.M. Marlin (2020): <https://arxiv.org/abs/2012.00168>
- *Learning from Irregularly-Sampled Time Series: A Missing Data Perspective* by S.C. Li and B.M. Marlin (2020), ICML: <https://arxiv.org/abs/2008.07599>
- *The M6 forecasting competition: Bridging the gap between forecasting and investment decisions* by Spyros Makridakis et al. (2023): <https://arxiv.org/abs/2310.13357>

Join our community on Discord

Join our community's Discord space for discussions with authors and other readers:

<https://packt.link/mts>



2

Acquiring and Processing Time Series Data

In the previous chapter, we learned what a time series is and established some standard notation and terminology. Now, let's switch tracks from theory to practice. In this chapter, we are going to get our hands dirty and start working with data. Although we said time series data is everywhere, we are yet to start working with a few time series datasets. We are going to start working on the dataset we will use throughout this book, process it in the right way, and learn about a few techniques to deal with missing values.

In this chapter, we will cover the following topics:

- Understanding the time series dataset
- pandas datetime operations, indexing, and slicing—a refresher
- Handling missing data
- Mapping additional information
- Saving and loading files to disk
- Handling longer periods of missing data

Technical requirements

You will need to set up the **Anaconda** environment, following the instructions in the *Preface* of the book, to get a working environment with all the libraries and datasets required for the code in this book. Any additional library will be installed while running the notebooks.

The code for this chapter can be found at <https://github.com/PacktPublishing/Modern-Time-Series-Forecasting-with-Python-2E/tree/main/notebooks/Chapter02>.

Handling time series data is like handling other tabular datasets, only with a focus on the temporal dimension. As with any tabular dataset, pandas is perfectly equipped to handle time series data as well.

Let's start getting our hands dirty and work through a dataset from the beginning. We are going to use the *London Smart Meters* dataset throughout this book. If you have not downloaded the data already as part of the environment setup, go to the *Preface* and do that now.

Understanding the time series dataset

This is the key first step in any new dataset you come across, even before **Exploratory Data Analysis (EDA)**, which we will cover in *Chapter 3, Analyzing and Visualizing Time Series Data*. Understanding where the data comes from, the data generating process behind it, and the source domain is essential to having a good understanding of the dataset.

London Data Store, a free and open data-sharing portal, provided this dataset, which was collected and enriched by Jean-Michel D and uploaded on Kaggle.

The dataset contains energy consumption readings for a sample of 5,567 London households that took part in the UK Power Networks-led Low Carbon London project between November 2011 and February 2014. Readings were taken at half-hourly intervals. Some metadata about the households is also available as part of the dataset. Let's look at what metadata is available as part of the dataset:

- CACI UK segmented the UK's population into demographic types, called Acorn. For each household in the data, we have the corresponding Acorn classification. The Acorn classes (Lavish Lifestyles, City Sophisticates, Student Life, and so on) are grouped into parent classes (Affluent Achievers, Rising Prosperity, Financially Stretched, and so on). A full list of Acorn classes can be found in *Table 2.1*. The complete documentation detailing each class is available at <https://acorn.caci.co.uk/downloads/Acorn-User-guide.pdf>.
- The dataset contains two groups of customers—one group who was subjected to **dynamic time-of-use (dToU)** energy prices throughout 2013, and another group who were on flat-rate tariffs. The tariff prices for the dToU were given a day ahead, via the smart meter IHD or text message.
- Jean-Michel D also enriched the dataset with weather and UK bank holiday data.

The following table shows the Acorn classes:

Acorn Group	Acorn Class
Affluent Achievers	A-Lavish Lifestyles
	B-Executive Wealth
	C-Mature Money
Rising Prosperity	D-City Sophisticates
	E-Career Climbers

Comfortable Communities	F-Countryside Communities
	G-Successful Suburbs
	H-Steady Neighborhoods
	I-Comfortable Seniors
	J-Starting Out
Financially Stretched	K-Student Life
	L-Modest Means
	M-Striving Families
	N-Poorer Pensioners
Urban Adversity	O-Young Hardship
	P-Struggling Estates
	Q-Difficult Circumstances

Table 2.1: Acorn classification



The Kaggle dataset also preprocesses the time series data daily and combines all the separate files. Here, we will ignore those files and start with the raw files, which can be found in the `hbblock_dataset` folder. Learning to work with raw files is an integral part of working with real-world datasets in the industry.

Preparing a data model

Once we understand where the data comes from, we can look at it, understand the information present in the different files, and figure out a mental model of how to relate the different files. You may call it old-school, but Microsoft Excel is an excellent tool for gaining this first-level understanding. If the file is too big to open in Excel, we can also read it in Python, save a sample of the data to an Excel file, and open it. However, keep in mind that Excel sometimes messes with the format of the data, especially dates, so we need to take care to not save the file and write back the formatting changes Excel made. If you are allergic to Excel, you can do it in Python as well, albeit with a lot more keystrokes. The purpose of this exercise is to see what the different data files contain, explore the relationship between the different files, and so on.

We can make this more formal and explicit by drawing a data model, similar to the one shown in the following diagram:

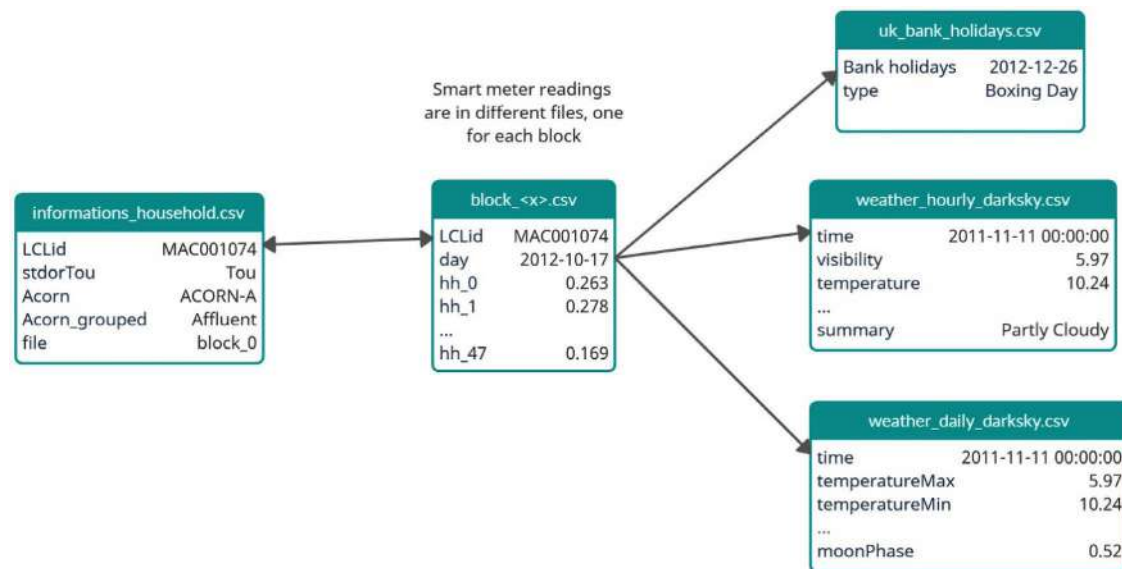


Figure 2.1: Data model of the London Smart Meters dataset

The data model is more for us to understand the data rather than any data engineering purpose. Therefore, it only contains bare-minimum information, such as the key columns on the left and the sample data on the right. We also have arrows connecting different files, with keys used to link the files.

Let's look at a few key column names and their meanings:

- **LCLid:** The unique consumer ID for a household
- **stdorTou:** Whether the household has dToU or standard tariff
- **Acorn:** The ACORN class
- **Acorn_grouped:** The ACORN group
- **file:** The block number

Each **LCLid** has a unique time series attached to it. The time series file is formatted in a slightly tricky format—each day, there will be 48 observations at a half-hourly frequency in the columns of the file.



Notebook alert:

To follow along with the complete code, use the `01-Pandas_Refresher_&_Missing_Values_Treatment.ipynb` notebook in the `Chapter01` folder.

Before we start working with our dataset, there are a few concepts we need to establish. One of them is a concept in pandas DataFrames, which is of utmost importance—the pandas datetime properties and index. Let's quickly look at a few pandas concepts that will be useful.



If you are familiar with the datetime manipulations in pandas, feel free to skip ahead to the next section.

pandas datetime operations, indexing, and slicing—a refresher

Instead of using our dataset, which is slightly complex, let's pick an easy, well-formatted stock exchange price dataset from the UCI Machine Learning Repository and look at the functionality of pandas:

```
# Skipping first row cause it doesn't have any data
df = pd.read_excel("https://archive.ics.uci.edu/ml/machine-learning-
databases/00247/data_akbilgic.xlsx", skiprows=1)
```

The DataFrame that we read looks as follows:

	date	ISE	ISE.1	SP	DAX	FTSE	NIKKEI	BOVESPA	EU	EM
0	2009-01-05	0.035754	0.038376	-0.004679	0.002193	0.003894	0.000000	0.031190	0.012698	0.028524
1	2009-01-06	0.025426	0.031813	0.007787	0.008455	0.012866	0.004162	0.018920	0.011341	0.008773
2	2009-01-07	-0.028862	-0.026353	-0.030469	-0.017833	-0.028735	0.017293	-0.035899	-0.017073	-0.020015
3	2009-01-08	-0.062208	-0.084716	0.003391	-0.011726	-0.000466	-0.040061	0.028283	-0.005561	-0.019424
4	2009-01-09	0.009860	0.009658	-0.021533	-0.019873	-0.012710	-0.004474	-0.009764	-0.010989	-0.007802

Figure 2.2: The DataFrame with stock exchange prices

Now that we have read the DataFrame, let's start manipulating it.

Converting the date columns into `pd.Timestamp/DatetimeIndex`

First, we must convert the date column (which may not always be parsed as dates automatically by pandas) into pandas datetime format. For that, pandas has a handy function called `pd.to_datetime`. It infers the datetime format automatically and converts the input into a `pd.Timestamp`, if the input is a string, or into a `DatetimeIndex`, if the input is a list of strings. So if we pass a single date as a string, `pd.to_datetime` converts it into `pd.Timestamp`, while if we pass a list of dates, it converts it into `DatetimeIndex`. Let's also use a handy function, `strftime`, which formats the date representation into a format we specify. It uses `strftime` conventions to specify the format of the data. For instance, `%d` means a zero-padded date, `%B` means a month's full name, and `%Y` means a year in four digits. A full list of `strftime` conventions can be found at <https://strftime.org/>:

```
>>> pd.to_datetime("13-4-1987").strftime("%d, %B %Y")
'13, April 1987'
```


Now, let's look at a case where the automatic parsing fails. The date is January 4, 1987. Let's see what happens when we pass the string to the function:

```
>>> pd.to_datetime("4-1-1987").strftime("%d, %B %Y")
'01, April 1987'
```

Well, that wasn't expected, right? But if you think about it, anyone can make that mistake because we are not telling the computer whether the month or the day comes first, and pandas assumes the month comes first. Let's rectify that:

```
>>> pd.to_datetime("4-1-1987", dayfirst=True).strftime("%d, %B %Y")
'04, January 1987'
```

Another case where automatic date parsing fails is when the date string is in a non-standard form. In that case, we can provide a `strftime`-formatted string to help pandas parse the dates correctly:

```
>>> pd.to_datetime("4|1|1987", format="%d|%m|%Y").strftime("%d, %B %Y")
'04, January 1987'
```

A full list of `strftime` conventions can be found at <https://strftime.org/>.

Practioner's tip:

Because of the wide variety of data formats, pandas may infer the time incorrectly. While reading a file, pandas will try to parse dates automatically and create an error. There are many ways we can control this behavior: we can use the `parse_dates` flag to turn off date parsing, the `date_parser` argument to pass in a custom date parser, and `year_first` and `day_first` to easily denote two popular formats of dates. From version 2.0, pandas supports `date_format`, which can be used to pass in the exact format of the date as a Python dictionary, with the column name as the key.



Out of all these options, I prefer to use `date_format`, if using pandas ≥ 2.0 . We can keep `parse_dates=True` and then pass in the exact date format, using `strftime` conventions. This ensures that the date is parsed in the way we want it to be.

If working with pandas < 2.0 , then I prefer to keep `parse_dates=False` in both `pd.read_csv` and `pd.read_excel` to make sure that pandas does not parse the data automatically. After that, you can convert the date using the `format` parameter, which lets you explicitly set the date format of the column using `strftime` conventions. There are two other parameters in `pd.to_datetime` that will also make inferring dates less error-prone—`yearfirst` and `dayfirst`. If you don't provide an explicit date format, at least provide one of these.

Now, let's convert the date column in our stock prices dataset into datetime:

```
df['date'] = pd.to_datetime(df['date'], yearfirst=True)
```

Now, the 'date' column, dtype, should be either `datetime64[ns]` or `<M8[ns]`, which are both pandas/NumPy-native datetime formats. But why do we need to do this?

It's because of the wide range of additional functionalities this unlocks. The traditional `min()` and `max()` functions will start working because pandas knows it is a datetime column:

```
>>> df.date.min(),df.date.max()
(Timestamp('2009-01-05 00:00:00'), Timestamp('2011-02-22 00:00:00'))
```

Let's look at a few cool features that the datetime format gives us.

Using the .dt accessor and datetime properties

Since the column is now in date format, all the semantic information that is encoded in the date can be used through pandas datetime properties. We can access many datetime properties, such as month, day_of_week, day_of_year, and so on, using the `.dt` accessor:

```
>>> print(f"""
    Date: {df.date.iloc[0]}
    Day of year: {df.date.dt.day_of_year.iloc[0]}
    Day of week: {df.date.dt.dayofweek.iloc[0]}
    Month: {df.date.dt.month.iloc[0]}
    Month Name: {df.date.dt.month_name().iloc[0]}
    Quarter: {df.date.dt.quarter.iloc[0]}
    Year: {df.date.dt.year.iloc[0]}
    ISO Week: {df.date.dt.isocalendar().week.iloc[0]}
    """)
Date: 2009-01-05 00:00:00
Day of year: 5
Day of week: 0
Month: 1
Month Name: January
Quarter: 1
Year: 2009
ISO Week: 2
```

As of pandas 1.1.0, `week_of_year` has been deprecated because of the inconsistencies it produces at the end/start of the year. Instead, the ISO calendar standards (which are commonly used in government and business) have been adopted, and we can access the ISO calendar to get the ISO weeks.

Indexing and slicing

The real fun starts when we make the date column the index of the DataFrame. By doing this, you can use all the fancy slicing operations that pandas supports but on the datetime axis. Let's take a look at a few of them:

```
# Setting the index as the datetime column
df.set_index("date", inplace=True)
# Select all data after 2010-01-04(inclusive)
```

```
df["2010-01-04":]
# Select all data between 2010-01-04 and 2010-02-06(exclusive)
df["2010-01-04": "2010-02-06"]
# Select data 2010 and before
df[: "2010"]
# Select data between 2010-01 and 2010-06(both including)
df["2010-01": "2010-06"]
```

In addition to the semantic information and intelligent indexing and slicing, pandas also provide tools to create and manipulate date sequences.

Creating date sequences and managing date offsets

If you are familiar with `range` in Python and `np.arange` in NumPy, then you will know they help us create integer/float sequences by providing a start point and an end point. pandas has something similar for datetime—`pd.date_range`. The function accepts start and end dates, along with a frequency (daily, monthly, and so on), and creates the sequence of dates in between. Let's look at a couple of ways to create a sequence of dates:

```
# Specifying start and end dates with frequency
pd.date_range(start="2018-01-20", end="2018-01-23", freq="D").astype(str).tolist()
# Output: ['2018-01-20', '2018-01-21', '2018-01-22', '2018-01-23']
# Specifying start and number of periods to generate in the given frequency
pd.date_range(start="2018-01-20", periods=4, freq="D").astype(str).tolist()
# Output: ['2018-01-20', '2018-01-21', '2018-01-22', '2018-01-23']
# Generating a date sequence with every 2 days
pd.date_range(start="2018-01-20", periods=4, freq="2D").astype(str).tolist()
# Output: ['2018-01-20', '2018-01-22', '2018-01-24', '2018-01-26']
# Generating a date sequence every month. By default it starts with Month end
pd.date_range(start="2018-01-20", periods=4, freq="M").astype(str).tolist()
# Output: ['2018-01-31', '2018-02-28', '2018-03-31', '2018-04-30']
# Generating a date sequence every month, but month start
pd.date_range(start="2018-01-20", periods=4, freq="MS").astype(str).tolist()
# Output: ['2018-02-01', '2018-03-01', '2018-04-01', '2018-05-01']
```

We can also add or subtract days, months, and other values to/from dates using `pd.TimeDelta`:

```
# Add four days to the date range
(pd.date_range(start="2018-01-20", end="2018-01-23", freq="D") +
 pd.Timedelta(4, unit="D")).astype(str).tolist()
# Output: ['2018-01-24', '2018-01-25', '2018-01-26', '2018-01-27']
# Add four weeks to the date range
(pd.date_range(start="2018-01-20", end="2018-01-23", freq="D") +
 pd.Timedelta(4, unit="W")).astype(str).tolist()
# Output: ['2018-02-17', '2018-02-18', '2018-02-19', '2018-02-20']
```

There are a lot of these aliases in pandas, including W, W-MON, MS, and others. The full list can be found at https://pandas.pydata.org/docs/user_guide/timeseries.html#timeseries-offset-aliases.

In this section, we looked at a few useful features and operations we can perform on datetime indices and know how to manipulate DataFrames with datetime columns. Now, let's review a few techniques we can use to deal with missing data.

Handling missing data

While dealing with large datasets in the wild, you are bound to encounter missing data. If it is not part of the time series, it may be part of the additional information you collect and map. Before we jump the gun and fill it with a mean value or drop those rows, let's consider a few aspects:

- The first consideration should be whether the missing data we are worried about is missing or not. For that, we need to think about the **Data Generating Process (DGP)** (the process that generates the time series). As an example, let's look at sales at a local supermarket. You have been given the **point-of-sale (POS)** transactions for the last 2 years, and you are processing the data into a time series. While analyzing the data, you found that there are a few products where there aren't any transactions for a few days. Now, what you need to think about is whether the missing data is missing or whether there is some information that this missingness gives you. If you don't have any transactions for a particular product for a day, it will appear as missing data while you are processing it, even though it is not missing. What that tells us is that there were no sales for that item and that you should fill such missing data with zeros.
- Now, what if you see that, every Sunday, the data is missing—that is, there is a pattern to the missingness? This becomes tricky because how you fill in such gaps depends on the model that you intend to use. If you fill in such gaps with zeros, a model that looks at the immediate past to predict the future might be thrown off, especially for Monday. However, if you tell the model that the previous day was Sunday, then the model still can learn to tell the difference.
- Lastly, what if you see zero sales on one of the best-selling products that always gets sold? This can happen because of something such as a POS machine malfunction, a data entry mistake, or an out-of-stock situation. These types of missing values can be imputed with a few techniques.

Let's look at an Air Quality dataset published by the ACT Government, Canberra, Australia, under the CC by Attribution 4.0 International License (<https://www.data.act.gov.au/Environment/Air-Quality-Monitoring-Data/94a5-zqnn>) and see how we can impute such values using pandas (there are more sophisticated techniques available, all of which will be covered later in this chapter).



Practitioner's tip:

When reading data using a method such as `read_csv`, pandas provides a few handy ways to handle missing values. pandas treats values such as #N/A, null, and so on as NaN by default. We can control this list of allowable NaN values using the `na_values` and `keep_default_na` parameters.

We have chosen region **Monash** and **PM2.5** readings, and artificially introduced some missing values, as shown in the following diagram:

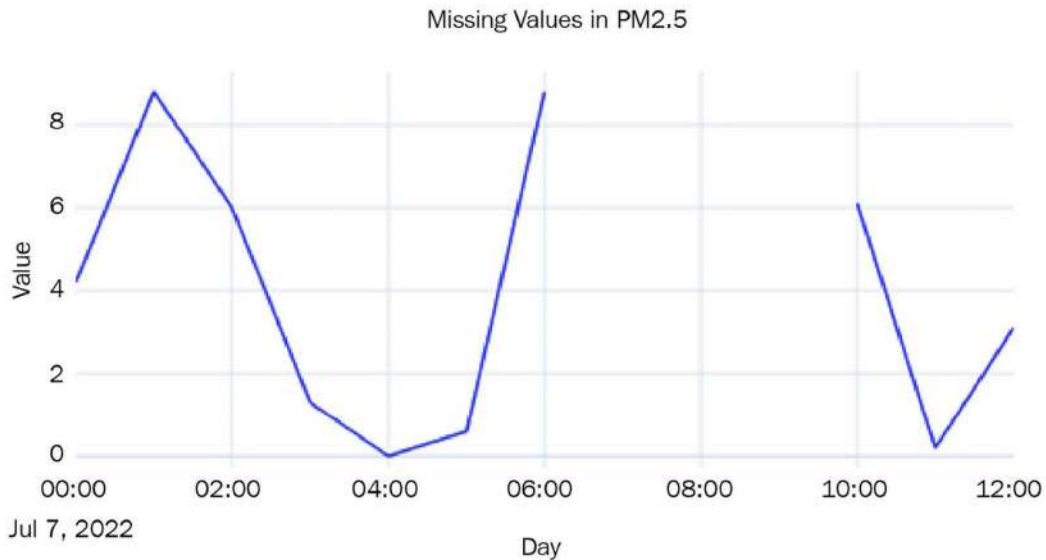


Figure 2.3: Missing values in the Air Quality dataset

Now, let's look at a few simple techniques we can use to fill in the missing values:

- **Last Observation Carried Forward or Forward Fill:** This imputation technique takes the last observed value, using that to fill in all the missing values until it finds the next observation. This is also called forward fill. We can do this like so:

```
df['pm2_5_1_hr'].ffill()
```

- **Next Observation Carried Backward or Backward Fill:** This imputation technique takes the next observation and backtracks to fill in all the missing values with this value. This is also called backward fill. Let's see how we can do this in pandas:

```
df['pm2_5_1_hr'].bfill()
```

- **Mean Value Fill:** This imputation technique is also pretty simple. We calculate the mean of the entire series, and wherever we find missing values, we fill it with the mean value:

```
df['pm2_5_1_hr'].fillna(df['pm2_5_1_hr'].mean())
```

Let's plot the imputed lines we get from using these three techniques:

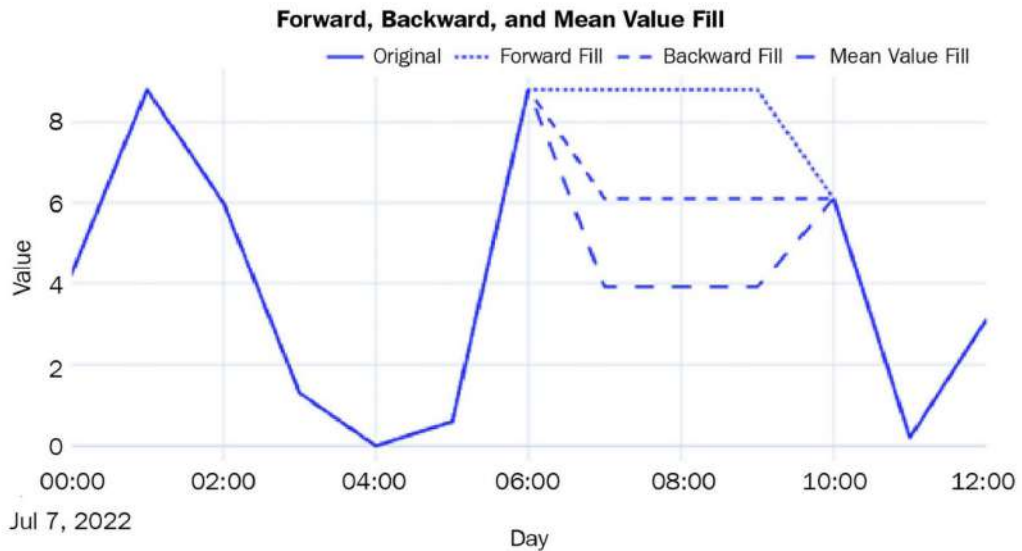


Figure 2.4: Imputed missing values using forward, backward, and mean value fill

Another family of imputation techniques covers interpolation:

- **Linear Interpolation:** Linear interpolation is just like drawing a line between the two observed points and filling in the missing values so that they lie on this line. This is how we do it:

```
df['pm2_5_1_hr'].interpolate(method="linear")
```

- **Nearest Interpolation:** This is intuitively like a combination of the forward and backward fill. For each missing value, the closest observed value is found and used to fill in the missing value:

```
df['pm2_5_1_hr'].interpolate(method="nearest")
```

Let's plot the two interpolated lines:

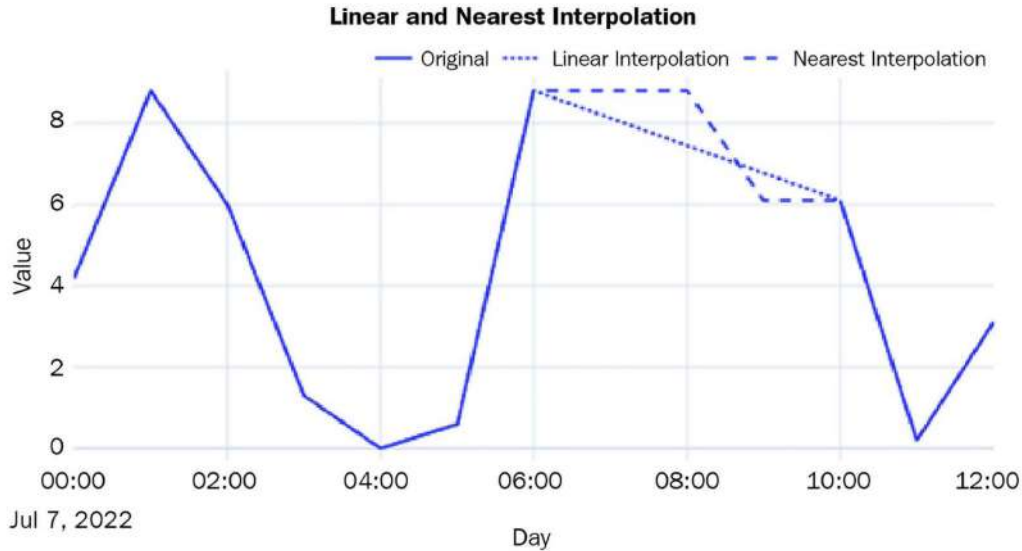


Figure 2.5: Imputed missing values using linear and nearest interpolation

There are a few non-linear interpolation techniques as well:

- **Spline, Polynomial, and Other Interpolations:** In addition to linear interpolation, pandas also supports non-linear interpolation techniques that call a SciPy routine at the backend. Spline and polynomial interpolations are similar. They fit a spline/polynomial of a given order to the data and use that to fill in missing values. While using `spline` or `polynomial` as the method in `interpolate`, we should always provide order as well. The higher the order, the more flexible the function that is used to fit the observed points will be. Let's see how we can use spline and polynomial interpolation:

```
df['pm2_5_1_hr'].interpolate(method="spline", order=2)
df['pm2_5_1_hr'].interpolate(method="polynomial", order=5)
```

Let's plot these two non-linear interpolation techniques:

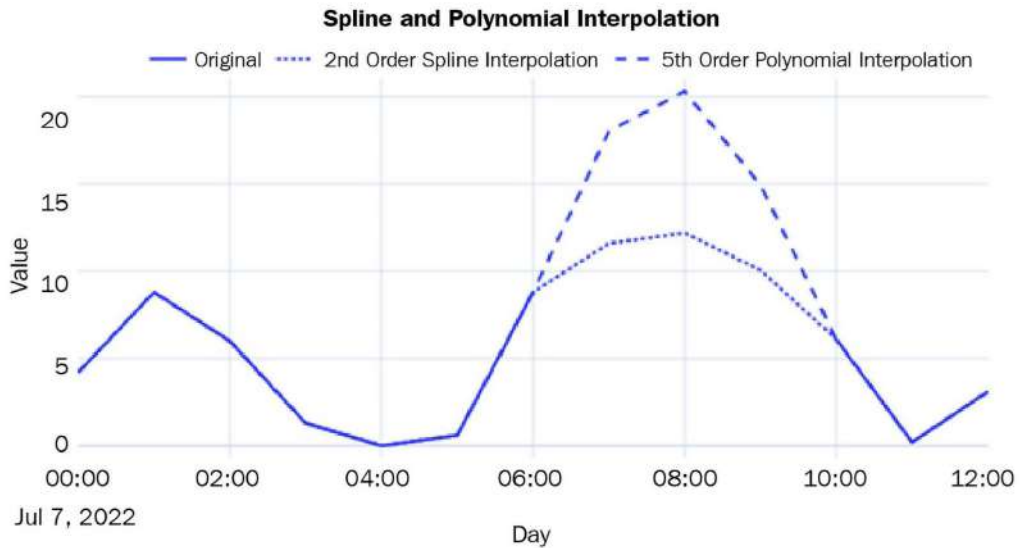


Figure 2.6: Imputed missing values using spline and polynomial interpolation

For a complete list of interpolation techniques supported by `interpolate`, go to <https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.Series.interpolate.html> and <https://docs.scipy.org/doc/scipy/reference/generated/scipy.interpolate.interp1d.html#scipy.interpolate.interp1d>.

Now that we are more comfortable with the way pandas manages datetime, let's go back to our dataset and convert the data into a more manageable form.



Notebook alert:

To follow along with the complete code for pre-processing, use the `02-Preprocessing_London_Smart_Meter_Dataset.ipynb` notebook in the `Chapter02` folder.

Converting the half-hourly block-level data (hhblock) into time series data

Before we start processing, let's understand a few general categories of information we will find in a time series dataset:

- **Time Series Identifiers:** These are identifiers for a particular time series. It can be a name, an ID, or any other unique feature—for example, the SKU name or the ID of a retail sales dataset or the consumer ID in the energy dataset that we are working with are all time series identifiers.
- **Metadata or Static Features:** This information does not vary with time. An example of this is the ACORN classification of the household in our dataset.
- **Time-Varying Features:** This information varies with time—for example, the weather information. For each point in time, we have a different value for weather, unlike the Acorn classification.

Next, let's discuss formatting of a dataset.

Compact, expanded, and wide forms of data

There are many ways to format a time series dataset, especially a dataset with many related time series, like the one we have now. A standard way of doing this is **wide** data. This is where the date column becomes sort of an index and each time series occupies a different column. And if there are a million time series, it will have a million and one columns (hence the term wide). Apart from the standard **wide** data, we can also look at two non-standard ways to format time series data. Although there is no standard nomenclature for them, we will refer to them as **compact** and **expanded** in this book. The expanded form is also referred to as **long** in some literature.

Compact-form data is when any particular time series occupies only a single row in the pandas DataFrame—that is, the time dimension is managed as an array within a DataFrame row. The time series identifiers and the metadata occupy the columns with scalar values and then the time series values; other time-varying features occupy the columns with an array. Two additional columns are included to extrapolate time—`start_datetime` and `frequency`. If we know the start datetime and the frequency of the time series, we can easily construct the time and recover the time series from the DataFrame. This only works for regularly sampled time series. The advantage is that the DataFrames take up much less memory and are easy and faster to work with:

LCLid	start_datetime	frequency	energy_consumption	series_length
MAC000002	2012-10-13	30min	[0.263, 0.26899999999999999, 0.275, 0.256, 0.21...	24144
MAC000246	2011-12-04	30min	[0.175, 0.098, 0.144, 0.065, 0.071, 0.037, 0.0...	39216
MAC000450	2012-03-23	30min	[0.337, 1.426, 0.996, 0.971, 0.994, 0.952, 0.8...	33936
MAC001074	2012-05-09	30min	[0.18, 0.086, 0.106, 0.173, 0.146, 0.223, 0.21...	31680
MAC003223	2012-09-18	30min	[0.076, 0.079, 0.123, 0.109, 0.051, 0.069, 0.0...	25344

Figure 2.7: Compact-form data

The expanded form is when the time series is expanded along the rows of a DataFrame. If there are n steps in the time series, it occupies n rows in the DataFrame. The time series identifiers and the metadata get repeated along all the rows. The time-varying features also get expanded along the rows. Also, instead of the start date and frequency, we have the timestamp as a column:

LCLid	energy_consumption	series_length	timestamp	frequency
MAC000002	0.263	24144	2012-10-13 00:00:00	30min
MAC000002	0.269	24144	2012-10-13 00:30:00	30min
MAC000002	0.275	24144	2012-10-13 01:00:00	30min
MAC000002	0.256	24144	2012-10-13 01:30:00	30min
MAC000002	0.211	24144	2012-10-13 02:00:00	30min

Figure 2.8: Expanded-form data

If the compact form had a time series identifier as the key, the time series identifier and the datetime column would be combined and become the key.

Wide-format data is more common in traditional time series literature. It can be considered a legacy format, which is limiting in many ways. Do you remember the stock data we saw earlier (*Figure 2.2*)? We have the date as an index or one of the columns, and the different time series as different columns of the DataFrame. As the number of time series increases, they become wider and wider, hence the name. This data format does not allow us to include any metadata about the time series. For instance, in our data, we have information about whether a particular household is under standard or dynamic pricing. There is no way for us to include such metadata in the wide format. From an operational perspective, the wide format also does not play well with relational databases because we have to keep adding columns to a table when we get new time series. We won't be using this format in this book.

Enforcing regular intervals in time series

One of the first things you should check and correct is whether the regularly sampled time series data that you have has equal intervals of time. In practice, even regularly sampled time series have some samples missing in between, due to some data collection error or some other peculiar way data is collected. So while working with the data, we will make sure we enforce regular intervals in the time series.



Best practice:

While working with datasets with multiple time series, it is best practice to check the end dates of all the time series. If they are not uniform, we can align them with the latest date across all the time series in the dataset.

In our smart meters dataset, some `LCLid` columns end much earlier than the rest. Maybe the household opted out of the program, or they moved out and left the house empty; the reason could be anything. However, we need to handle that while we enforce regular intervals.

We will learn how to convert the dataset into a time series format in the next section. The code for this process can be found in the `02-Preprocessing_London_Smart_Meter_Dataset.ipynb` notebook.

Converting the London Smart Meters dataset into a time series format

For each dataset that you come across, the steps you would have to take to convert it into either a compact or expanded form would be different. It depends on how the original data is structured. Here, we will look at how the London Smart Meters dataset can be transformed so that we can transfer those learnings to other datasets.

There are two steps we need to do before we can start processing the data into either compact or expanded form:

1. **Find the Global End Date:** We must find the maximum date across all the block files so that we know the global end date of the time series.
2. **Basic Preprocessing:** If you remember how `hhblock_dataset` is structured, you will remember that each row had a date and that, along the columns, we have half-hourly blocks. We need to reshape that into a long form, where each row has a date and a single half-hourly block. It's easier to handle that way.

Now, let's define separate functions to convert data into compact and expanded forms and apply those functions to each of the `LCLid` columns. We will do this for each `LCLid` separately, since the start date for each `LCLid` is different.

Expanded form

The function to convert data into the expanded form does the following:

1. Finds the start date.
2. Create a standard `DataFrame` using the start date and the global end date.
3. Left-merges the `DataFrame` for `LCLid` with the standard `DataFrame`, leaving the missing data as `np.nan`.
4. Returns the merged `DataFrame`.

Once we have all the `LCLid` `DataFrames`, we must perform a couple of additional steps to complete the expanded form processing:

1. Concatenate all the `DataFrames` into a single `DataFrame`.
2. Create a column called `offset`, which is the numerical representation of the half-hour blocks; for example, `hh_3` $\rightarrow$ 3.
3. Create a timestamp by adding a 30-minute offset to the day and dropping the unnecessary columns.

For one block, this representation takes up ~47 MB of memory.

Compact form

The function for converting into compact form does the following:

1. Finds the start date and time series identifiers.
2. Creates a standard DataFrame using the start date and the global end date.
3. Left merges the DataFrame for `LCLid` to the standard DataFrame, leaving the missing data as `np.nan`.
4. Sorts the values on the date.
5. Returns the time series array, along with the time series identifier, start date, and the length of the time series.

Once we have this information for each `LCLid`, we can compile it into a DataFrame and add 30min as the frequency.

For one block, this representation takes up only ~0.002 MB of memory.

We are going to use the compact form because it is easy to work with and much less resource-hungry.

Mapping additional information

From the data model that we prepared earlier, we know that there are three key files that we have to map: *Household Information*, *Weather*, and *Bank Holidays*.

The `informations_households.csv` file contains metadata about the household. There are static features that are not dependent on time. For this, we just need to left-merge `informations_households.csv` with the compact form based on `LCLid`, which is the time series identifier.

Best practice:



While doing a pandas merge, one of the most common and unexpected outcomes is that the number of rows before and after the operation is not the same (even if you are doing a left merge). This typically happens because there are duplicates in the keys on which you are merging. As a best practice, you can use the `validate` parameter in the pandas merge, which takes in inputs such as `one_to_one` and `many_to_one` so that this check is done while merging and will throw an error if the assumption is not met. For more information, go to <https://pandas.pydata.org/docs/reference/api/pandas.merge.html>.

Bank Holidays and Weather, on the other hand, are time-varying features and should be dealt with accordingly. The most important aspect to keep in mind is that while we map this information, it should perfectly align with the time series that we have already stored as an array.

`uk_bank_holidays.csv` is a file that contains the dates of the holidays and the kind of holiday. The holiday information is quite important here because the energy consumption patterns would be different on a holiday when the family members are at home spending time with each other, watching television, and so on. Follow these steps to process this file:

1. Convert the date column into the datetime format and set it as the index of the DataFrame.
2. Using the `resample` function we saw earlier, we must ensure that the index is resampled every 30 minutes, which is the frequency of the times series.
3. Forward fill the holidays within a day and fill in the rest of the NaN values with `NO_HOLIDAY`.

Now, we have converted the holiday file into a DataFrame that has a row for each 30-minute interval. On each row, we have a column that specifies whether that day was a holiday or not.

`weather_hourly_darksky.csv` is a file that is, once again, at the daily frequency. We need to downsample it to a 30-minute frequency because the data that we need to map to this is at a half-hourly frequency. If we don't do this, the weather will only be mapped to the hourly timestamps, leaving the half-hourly timestamps empty.

The steps we must follow to process this file are also similar to the way we processed holidays:

1. Convert the date column into the datetime format and set it as the index of the DataFrame.
2. Using the `resample` function, we must ensure that the index is resampled every 30 minutes, which is the frequency of the times series.
3. Forward fill the weather features to fill in the missing values that were created while resampling.

Now that you have made sure the alignment between the time series and the time-varying features is ensured, you can loop over each of the time series and extract the weather and bank holiday array before storing it in the corresponding row of the DataFrame.

Saving and loading files to disk

The fully merged DataFrame in its compact form takes up only ~10 MB. However, saving this file requires a little bit of engineering. If we try to save the file in the CSV format, it will not work because of the way we have stored arrays in pandas columns (since the data is in its compact form). We can save it in pickle or parquet format, or any of the binary forms of file storage. This can work, depending on the size of the RAM available on our machines. Although the fully merged DataFrame is just ~10 MB, saving it in pickle format will make the size explode to ~15 GB.

What we can do is save this as a text file while making a few tweaks to accommodate the column names, column types, and other metadata that is required to read the file back into memory. The resulting file size on disk still comes out to ~15 GB, but since we are doing it as an I/O operation, we do not keep all that data in our memory. We call this the time series (`.ts`) format. The functions to save a compact form in the `.ts` format, read the `.ts` format, and convert the compact form into the expanded form are available in this book's GitHub repository under `src/data_utils.py`.

If you don't need to store all of the DataFrame in a single file, you can split it into multiple chunks and save them individually in a binary format, such as parquet. For our datasets, let's follow this route and split the whole DataFrame into chunks of blocks and save them as parquet files. This is the best route for us for a few reasons:

- It leverages the compression that comes with the format
- It reads in parts of the whole data for quick iteration and experimentation
- The data types are retained between the read and write operations, leading to less ambiguity



For very large datasets, we can use some pandas alternatives, which makes it easier to process datasets that are out of memory. Polars is a great library that has lazy loading and is very fast. And for truly huge datasets, PySpark with a distributed cluster might be the right choice.

Now that we have processed the dataset and stored it on disk, let's read it back into memory and look at a few more techniques to handle missing data.

Handling longer periods of missing data

We saw some techniques to handle missing data earlier—forward and backward filling, interpolation, and so on. Those techniques usually work if there are one or two missing data points. But if a large section of data is missing, then these simple techniques fall short.



Notebook alert:

To follow along with the complete code for missing data imputation, use the 03-Handling_Missing_Data_(Long_Gaps).ipynb notebook in the Chapter02 folder.

Let's read blocks 0-7 parquet from memory:

```
block_df = pd.read_parquet("data/london_smart_meters/preprocessed/london_smart_meters_merged_block_0-7.parquet")
```

The data that we have saved is in the compact form. We need to convert it into the expanded form because it is easier to work with time series data in that form. Since we only need a subset of the time series (for faster demonstration purposes), we will just extract one block from these seven blocks. To convert the compact form into the expanded form, we can use a helpful function in `src/utils/data_utils.py` called `compact_to_expanded`:

```
#Converting to expanded form
exp_block_df = compact_to_expanded(block_df[block_df.file=="block_7"],
timeseries_col = 'energy_consumption',
```

```
static_cols = ["frequency", "series_length", "stdorToU", "Acorn", "Acorn_
grouped", "file"],
time_varying_cols = ['holidays', 'visibility', 'windBearing', 'temperature',
'dewPoint',
    'pressure', 'apparentTemperature', 'windSpeed', 'precipType', 'icon',
    'humidity', 'summary'],
ts_identifier = "LCLid")
```

One of the best ways to visualize the missing data in a group of related time series is by using a very helpful package called `missingno`:

```
# Pivot the data to set the index as the datetime and the different time series
along the columns
plot_df = pd.pivot_table(exp_block_df, index="timestamp", columns="LCLid",
values="energy_consumption")
# Generate Plot. Since we have a datetime index, we can mention the frequency
to decide what do we want on the X axis
msno.matrix(plot_df, freq="M")
```

The preceding code produces the following output:

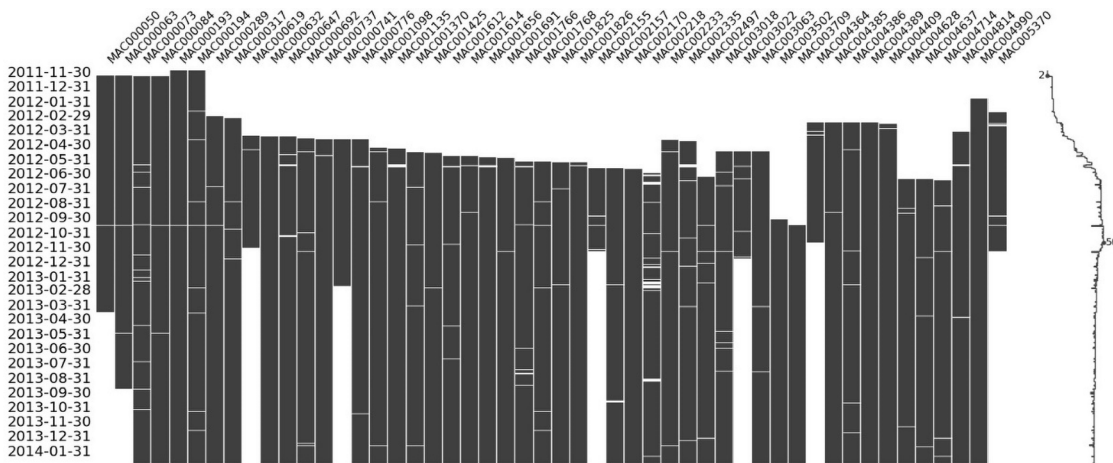


Figure 2.9: Visualization of the missing data in block 7



Quick tip: Need to see a high-resolution version of this image? Open this book in the next-gen Packt Reader or view it in the PDF/ePub copy.

The next-gen Packt Reader and a free PDF/ePub copy of this book are included with your purchase. Unlock them by scanning the QR code below or visiting <https://www.packtpub.com/unlock/9781835883181>.



Only attempt the missingno visualization on related time series where there are less than 25 time series. If you have a dataset that contains thousands of time series (such as in our full dataset), applying this visualization will give us an illegible plot and a frozen computer.

This visualization tells us a lot of things at a single glance. The Y-axis contains the dates that we are plotting the visualization for, while the X-axis contains the columns, which in this case are the different households. We know that all the time series are not perfectly aligned—that is, not all of them start at the same time and end at the same time. The big white gaps we can see at the beginning of many of the time series show that data collection for those consumers started later than the others. We can also see that a few time series finish earlier than the rest, which means either they stopped being consumers or the measurement phase stopped. There are also a few smaller white lines in many time series, which are real missing values. We can also notice a sparkline to the right, which is a compact representation of the number of missing columns for each row. If there are no missing values (all time series have some value), then the sparkline would be at the far right. Finally, if there are a lot of missing values, the line will be to the left.

Just because there are missing values, we are not going to fill/impute them because the decision of whether to impute missing data or not comes later in the workflow. For some models, we do not need to do the imputation, while for others, we do. There are multiple ways of imputing missing data, and which one to choose is another decision we cannot make beforehand.

So for now, let's pick one LCLid and dig deeper. We already know that there are some missing values between 2012-09-30 and 2012-10-31. Let's visualize that period:

```
# Taking a single time series from the block
ts_df = exp_block_df[exp_block_df.LCLid=="MAC000193"].set_index("timestamp")
msno.matrix(ts_df["2012-09-30": "2012-10-31"], freq="D")
```


The preceding code produces the following output:

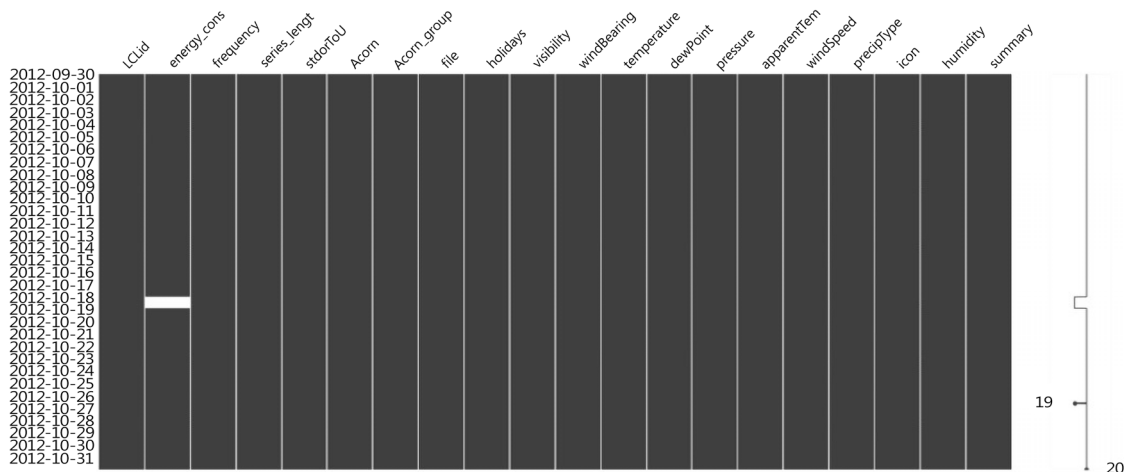


Figure 2.10: Visualization of missing data of MAC000193 between 2012-09-30 and 2012-10-31

Here, we can see that the missing data is between 2012-10-18 and 2012-10-19. Normally, we would go ahead and impute the missing data in this period, but since we are looking at this with an academic lens, we will take a slightly different route. Let's introduce an artificial missing data section, see how the different techniques we are going to look at impute the missing data, and compute a metric to see how close we are to the real time series (We are going to use a metric called **Mean Absolute Error (MAE)** to do the comparison, and it's nothing but the average of the absolute error across the time steps. Just understand that it is a lower-the-better metric that we will talk about in detail later in the book.):

```
# The dates between which we are nulling out the time series
window = slice("2012-10-07", "2012-10-08")

# Creating a new column and artificially creating missing values
ts_df['energy_consumption_missing'] = ts_df.energy_consumption
ts_df.loc[window, "energy_consumption_missing"] = np.nan
```

Now, let's plot the missing area in the time series:

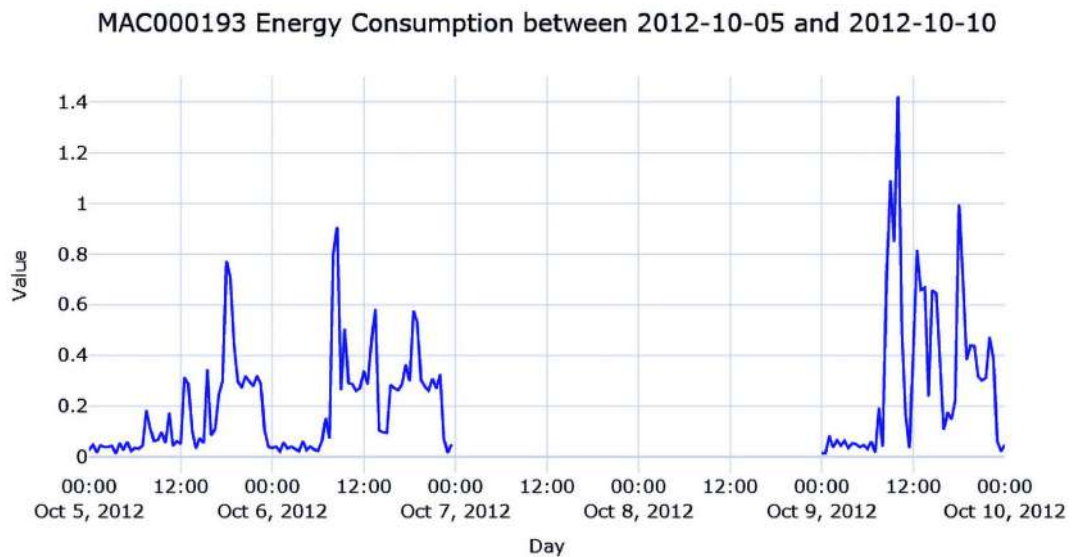


Figure 2.11: The energy consumption of MAC000193 between 2012-10-05 and 2012-10-10

We are missing 2 whole days of energy consumption readings, which means there are 96 missing data points (half-hourly). If we use one of the techniques we saw earlier, such as interpolation, we will see that it will mostly be a straight line because none of the methods are complex enough to capture the pattern over a long time.

There are a few techniques that we can use to fill in such large missing gaps in data. We will cover these now.

Imputing with the previous day

Since this is a half-hourly time series of energy consumption, it stands to reason that there might be a pattern that repeats day after day. The energy consumption between 9:00 A.M. and 10:00 A.M. might be higher as everybody gets ready to go to the office, slumping during the day when most houses may be empty. So the simplest way to fill in the missing data would be to use the previous day's energy readings so that the energy reading at 10:00 A.M, 2012-10-18, can be filled with the energy reading at 10:00 A.M, 2012-10-17:

```
#Shifting 48 steps to get previous day
ts_df["prev_day"] = ts_df['energy_consumption'].shift(48)
#Using the shifted column to fill missing
ts_df['prev_day_imputed'] = ts_df['energy_consumption_missing']
ts_df.loc[null_mask, "prev_day_imputed"] = ts_df.loc[null_mask, "prev_day"]
mae = mean_absolute_error(ts_df.loc>window, "prev_day_imputed", ts_
df.loc>window, "energy_consumption")
```

Let's see what the imputation looks like:

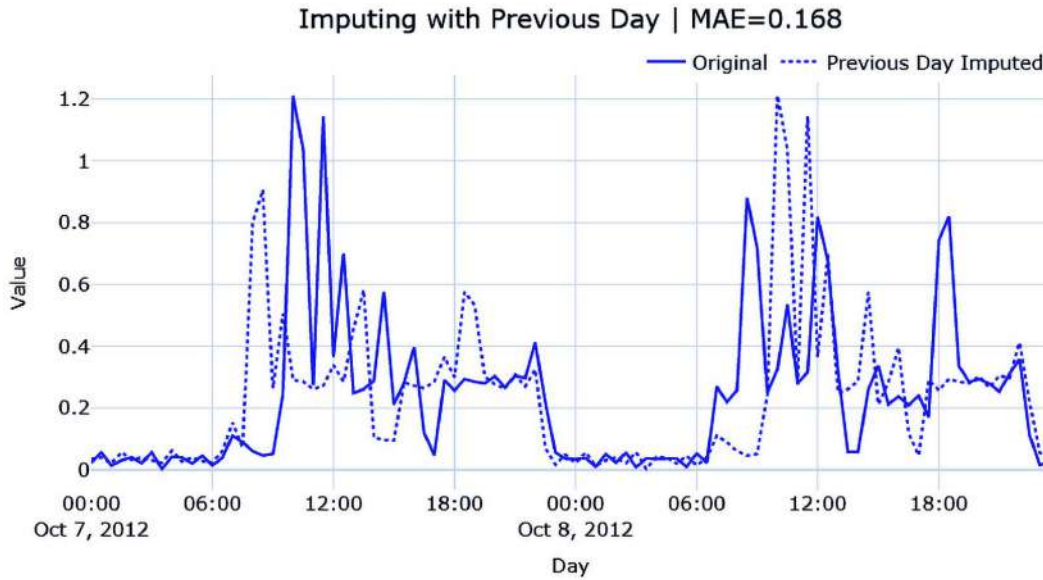


Figure 2.12: Imputing with the previous day

While this looks better, this is also very brittle. When we copy the previous day, we also assume that any kind of variation or anomalous behavior is also repeated. We can already see that the patterns for the day before and the day after are not the same.

Hourly average profile

A better approach would be to calculate an hourly profile from the data—the mean consumption for every hour—and use the average to fill in the missing data:

```
#Create a column with the Hour from timestamp
ts_df["hour"] = ts_df.index.hour

#Calculate hourly average consumption
hourly_profile = ts_df.groupby(['hour'])['energy_consumption'].mean().reset_index()

hourly_profile.rename(columns={"energy_consumption": "hourly_profile"},
inplace=True)

#Saving the index because it gets lost in merge
idx = ts_df.index

#Merge the hourly profile dataframe to ts dataframe
ts_df = ts_df.merge(hourly_profile, on=['hour'], how='left', validate="many_to_one")

ts_df.index = idx

#Using the hourly profile to fill missing
```

```
ts_df['hourly_profile_imputed'] = ts_df['energy_consumption_missing']
ts_df.loc[null_mask, "hourly_profile_imputed"] = ts_df.loc[null_mask, "hourly_
profile"]
mae = mean_absolute_error(ts_df.loc>window, "hourly_profile_imputed"), ts_
df.loc>window, "energy_consumption"])
```

Let's see if this is better:

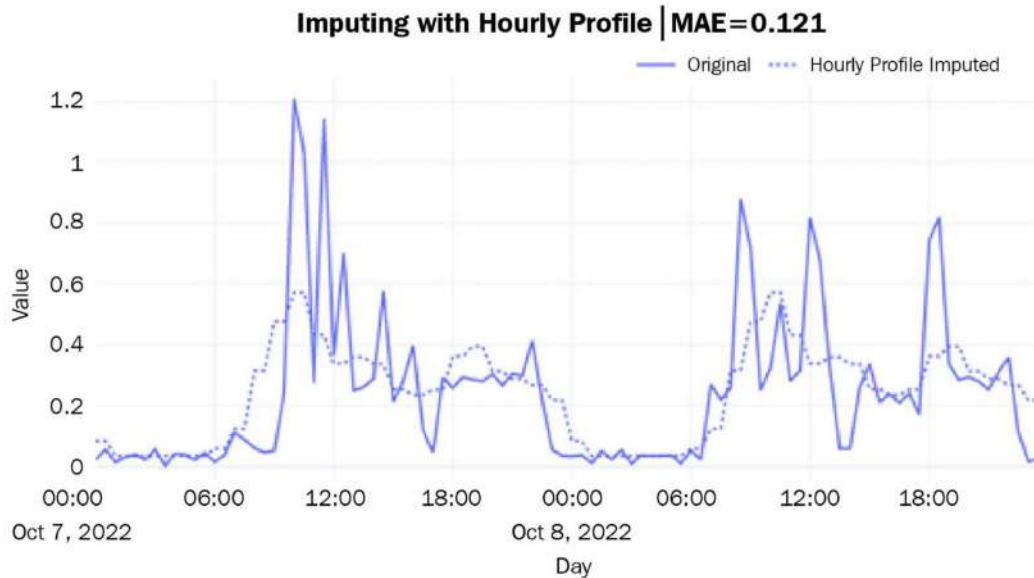


Figure 2.13: Imputing with an hourly profile

This gives us a much more generalized curve that does not have the spikes that we saw for the individual days. The hourly ups and downs have also been captured as per our expectations. The MAE is also lower than before.

The hourly average for each weekday

We can further refine this rule by introducing a specific profile for each weekday. It stands to reason that the usage pattern on a weekday is not going to be the same on a weekend. Hence, we can calculate the average hourly consumption for each weekday separately so that we have one profile for Monday, another for Tuesday, and so on:

```
#Create a column with the weekday from timestamp
ts_df["weekday"] = ts_df.index.weekday
#Calculate weekday-hourly average consumption
day_hourly_profile = ts_df.groupby(['weekday', 'hour'])['energy_consumption'].
mean().reset_index()
day_hourly_profile.rename(columns={"energy_consumption": "day_hourly_profile"},
inplace=True)
#Saving the index because it gets lost in merge
```

```

idx = ts_df.index
#Merge the day-hourly profile dataframe to ts dataframe
ts_df = ts_df.merge(day_hourly_profile, on=['weekday', 'hour'], how='left',
validate="many_to_one")
ts_df.index = idx
#Using the day-hourly profile to fill missing
ts_df['day_hourly_profile_imputed'] = ts_df['energy_consumption_missing']
ts_df.loc[null_mask, "day_hourly_profile_imputed"] = ts_df.loc[null_mask, "day_
hourly_profile"]
mae = mean_absolute_error(ts_df.loc>window, "day_hourly_profile_imputed"], ts_
df.loc>window, "energy_consumption"))

```

Let's see what this looks like:

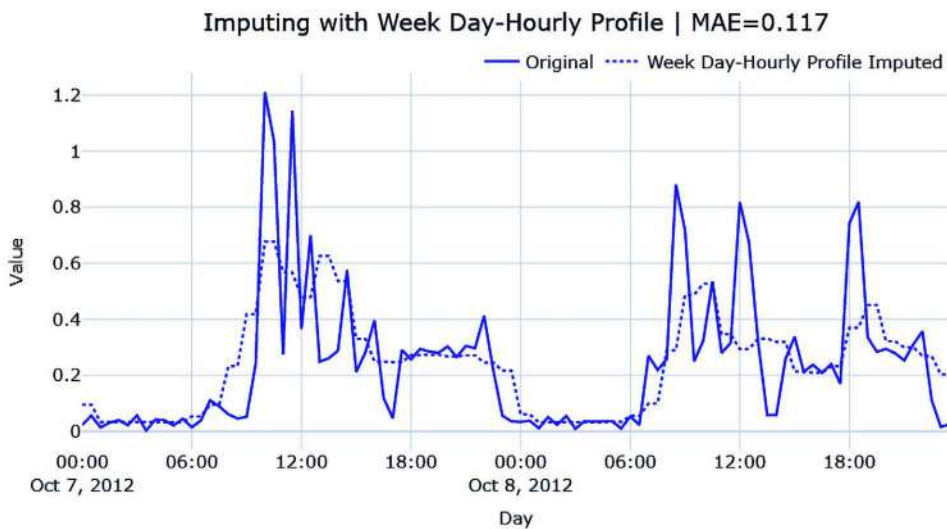


Figure 2.14: Imputing the hourly average for each weekday

This looks very similar to the other one, but this is because the day we are imputing is a weekday and the weekday profiles are similar. The MAE is also lower than the day profile. The weekend profile is slightly different, which you can see in the associated Jupyter notebook.

Seasonal interpolation

Although calculating seasonal profiles and using them to impute works well, there are instances, especially when there is a trend in the time series, where such a simple technique falls short. The simple seasonal profile doesn't capture the trend at all and ignores it completely. For such cases, we can do the following:

1. Calculate the seasonal profile, similar to how we calculated the averages earlier.
2. Subtract the seasonal profile and apply any of the interpolation techniques we saw earlier.
3. Return the seasonal profile to the interpolated series.

This process has been implemented in this book's GitHub repository in the `src/imputation/interpolation.py` file. We can use it as follows:

```
from src.imputation.interpolation import SeasonalInterpolation
# Seasonal interpolation using 48*7 as the seasonal period.
recovered_matrix_seas_interp_weekday_half_hour =
SeasonalInterpolation(seasonal_period=48*7,decomposition_strategy="additive",
interpolation_strategy="spline", interpolation_args={"order":3}, min_value=0).
fit_transform(ts_df.energy_consumption_missing.values.reshape(-1,1))
ts_df['seas_interp_weekday_half_hour_imputed'] = recovered_matrix_seas_interp_
weekday_half_hour
```

The key parameter here is `seasonal_period`, which tells the algorithm to look for patterns that repeat every `seasonal_period`. If we mention `seasonal_period=48`, it will look for patterns that repeat every 48 data points. In our case, they are after each day (because we have 48 half-hour timesteps in a day). In addition to this, we need to specify what kind of interpolation we need to perform.



Additional information:

Internally, we use something called seasonal decomposition (`statsmodels.tsa.seasonal.seasonal_decompose`), which will be covered in *Chapter 3, Analyzing and Visualizing Time Series Data*, to isolate the seasonality component.

Here, we have done seasonal interpolation using 48 (half-hourly) and 48*7 (weekday to half-hourly) and plotted the resulting imputation:

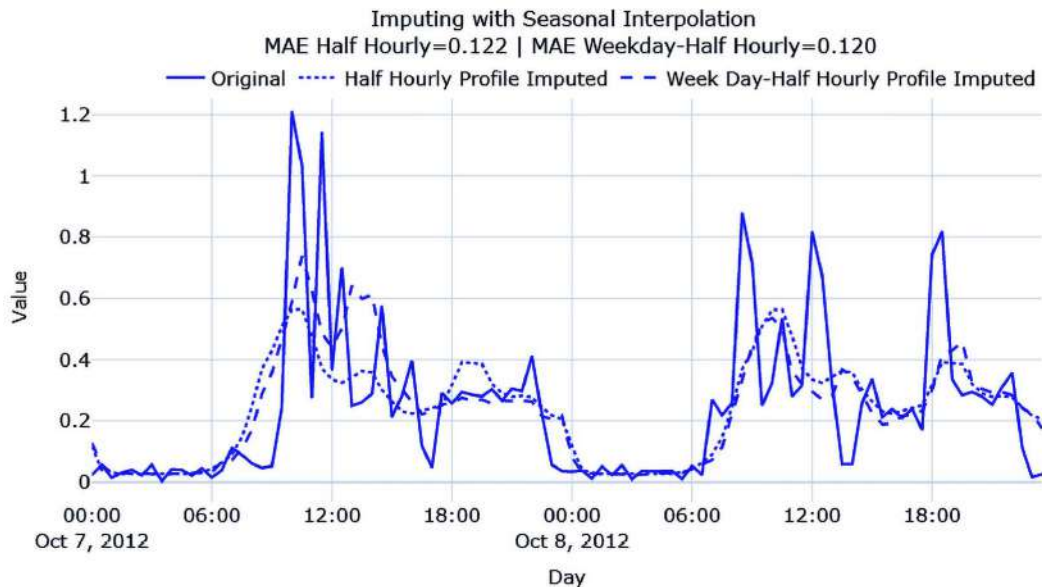


Figure 2.15: Imputing with seasonal interpolation

Here, we can see that both have captured the seasonality patterns, but the half-hourly profile every weekday has captured the peaks of the first day better, so they have a lower MAE. There is no improvement in terms of hourly averages, mostly because there are no strong increasing or decreasing patterns in the time series.

With this, we have come to the end of this chapter. We are now officially into the nitty-gritty of juggling, cleaning, and processing time series data. Congratulations on finishing this chapter!

Summary

In this chapter, after a short refresher on pandas DataFrames, especially on the datetime manipulations and simple techniques to handle missing data, we learned about the two forms of storing and working with time series data—compact and expanded. With all this knowledge, we took our raw dataset and built a pipeline to convert it into the compact form. If you have run the accompanying notebook, you should have the preprocessed dataset saved on disk. We also had an in-depth look at some techniques to handle long gaps of missing data.

Now that we have the processed datasets, in the next chapter, we will learn how to visualize and analyze a time series dataset.

Unlock this book's exclusive benefits now

This book comes with additional benefits designed to elevate your learning experience.



Note: Have your purchase invoice ready before you begin. <https://www.packtpub.com/unlock/9781835883181>

3

Analyzing and Visualizing Time Series Data

In the previous chapter, we learned where to obtain time series datasets, as well as how to manipulate time series data using pandas, handle missing values, and so on. Now that we have the processed time series data, it's time to understand the dataset, which data scientists call **Exploratory Data Analysis (EDA)**. It is a process by which the data scientist analyzes the data by looking at aggregate statistics, feature distributions, visualizations, and so on to try and uncover patterns in the data that they can leverage in modeling. In this chapter, we will look at a couple of ways to analyze a time series dataset, a few specific techniques that are tailor-made for time series, and review some of the visualization techniques for time series data.

In this chapter, we will cover the following topics:

- Components of a time series
- Visualizing time series data
- Decomposing a time series
- Detecting and treating outliers

Technical requirements

You will need to set up the **Anaconda** environment following the instructions in the *Preface* of the book to get a working environment with all the libraries and datasets required for the code in this book. Any additional library will be installed while running the notebooks.

You will need to run the `02-Preprocessing_London_Smart_Meter_Dataset.ipynb` notebook from Chapter02 folder.

The code for this chapter can be found at <https://github.com/PacktPublishing/Modern-Time-Series-Forecasting-with-Python-2E/tree/main/notebooks/Chapter03>.

Components of a time series

Before we start analyzing and visualizing time series, we need to understand the structure of a time series. Any time series can contain some or all of the following components:

- Trend
- Seasonal
- Cyclical
- Irregular

These components can be mixed in different ways, but two very commonly assumed ways are *additive* ($Y = \text{Trend} + \text{Seasonal} + \text{Cyclical} + \text{Irregular}$) and *multiplicative* ($Y = \text{Trend} * \text{Seasonal} * \text{Cyclical} * \text{Irregular}$), where Y is the time series.

The trend component

The **trend** is a long-term change in the mean of a time series. It is the smooth and steady movement of a time series in a particular direction. When the time series moves upward, we say there is an *upward or increasing trend*, while when it moves downward, we say there is a *downward or decreasing trend*. At the time of writing, if we think about the revenue of Tesla over the years, as shown in the following figure, we can see that it has been increasing consistently for the last few years:

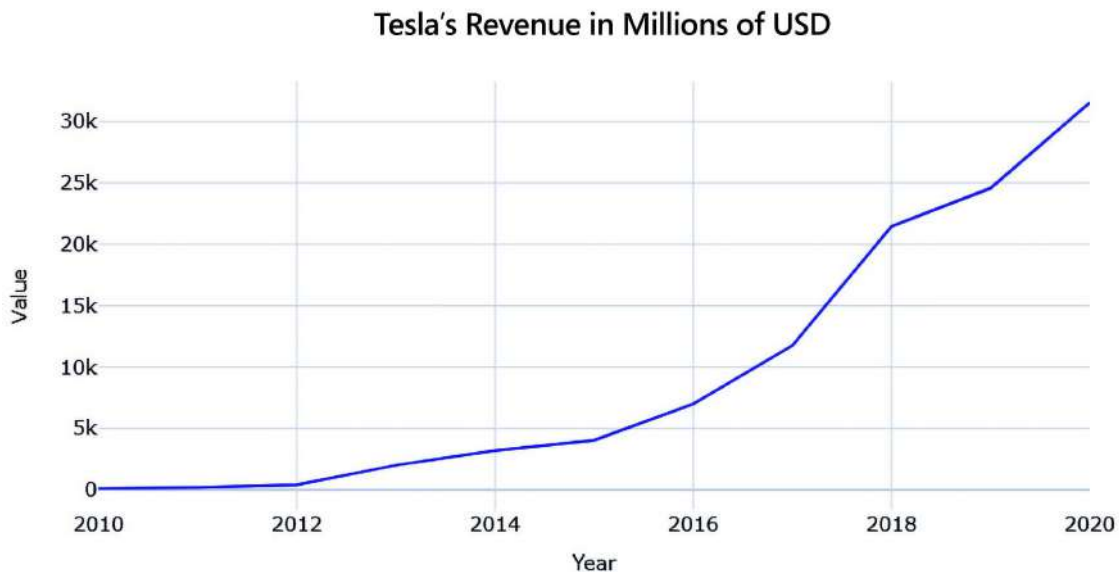


Figure 3.1: Tesla's revenue in millions of USD

Looking at the preceding figure, we can say that Tesla's revenue is having an increasing trend. The trend doesn't need to be linear; it can also be non-linear.

The seasonal component

When a time series exhibits regular, repetitive, up-and-down fluctuations, we call that **seasonality**. For instance, retail sales typically shoot up during the holidays, especially during Christmas in Western countries. Similarly, electricity consumption peaks during the summer months in the tropics and the winter months in colder countries. In all these examples, you can see a specific up-and-down pattern repeating every year. Another example is sunspots, as shown in the following figure:

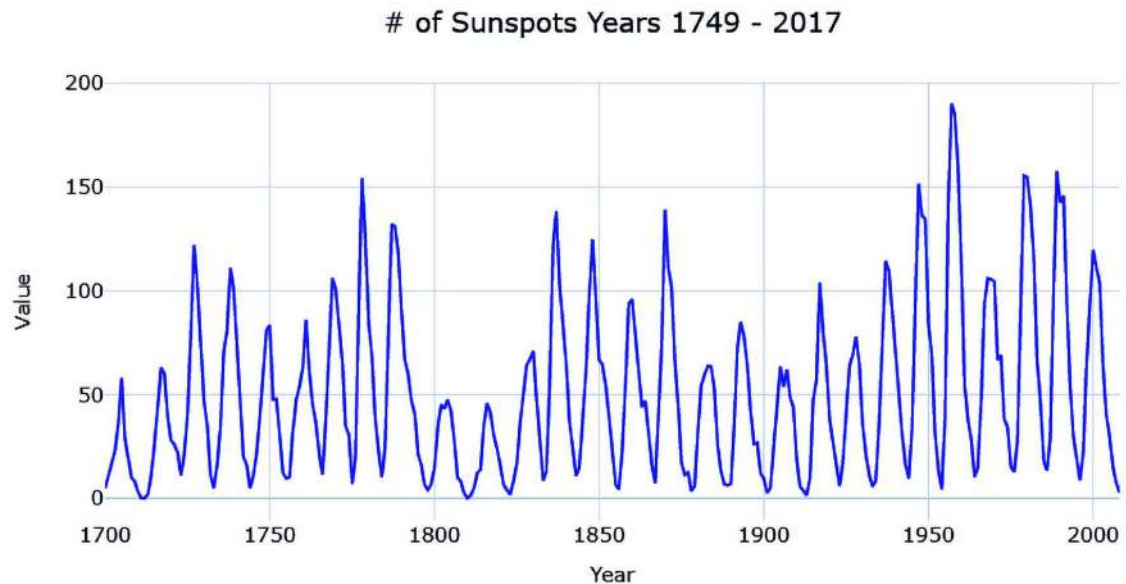


Figure 3.2: Number of sunspots from 1749 to 2017

As you can see, sunspots peak every 11 years.

The cyclical component

The **cyclical component** is often confused with seasonality, but it stands apart due to a very subtle difference. Like seasonality, the cyclical component also exhibits a similar up-and-down pattern around the trend line, but the time over which this cycle moves isn't fixed and is subject to a bit of variation around a general time frame. A good example of this is economic recession, which happens over a 10-year cycle. However, this doesn't happen like clockwork; sometimes, it can be fewer or more than every 10 years.

The irregular component

This component is left after removing the trends, seasonality, and cyclicity from a time series. Traditionally, this component is considered *unpredictable* and is also called the *residual*, *error term*, or *noise term*. In common classical statistics-based models, the point of any “model” is to capture all the other components to the point that the only part that is not captured is the irregular component.

In modern machine learning, we do not consider this component entirely unpredictable. We try to capture parts of this component by using exogenous variables. For instance, the irregular component of retail sales may be explained by the different promotional activities they run. When we have this additional information, the “unpredictable” component starts to become predictable again. But no matter how many additional variables you add to the model, there will always be some component, which is the irreducible error, that is left behind. This is the part of the time series which can never be explained no matter how strong the model is or how much additional information you add to the model.

Now that we know what the different components of a time series are, let’s see how we can visualize them.

Visualizing time series data

In *Chapter 2, Acquiring and Processing Time Series Data*, we learned how to prepare a data model as a first step toward analyzing a new dataset. If preparing a data model is like approaching someone you like and making that first contact, then EDA is like dating that person. At this point, you have the dataset, and you are trying to get to know them, trying to figure out what makes them tick, what the person likes and dislikes, and so on.

EDA often employs visualization techniques to uncover patterns, spot anomalies, form and test hypotheses, and so on. Spending some time understanding your dataset will help you a lot when you are trying to squeeze out every last bit of performance from the models. You may understand what sort of features you must create, what kind of modeling techniques should be applied, and so on.

In this chapter, we will cover a few visualization techniques that are well suited for time series datasets.



Notebook alert:

To follow along with the complete code for visualizing time series, use the `01-Visualizing_Time_Series.ipynb` notebook in the `Chapter03` folder.

Line charts

This is the most basic and common visualization that is used for understanding a time series. We just plot the time on the *x*-axis and the time series value on the *y*-axis. Let’s see what it looks like if we plot one of the households from our dataset:

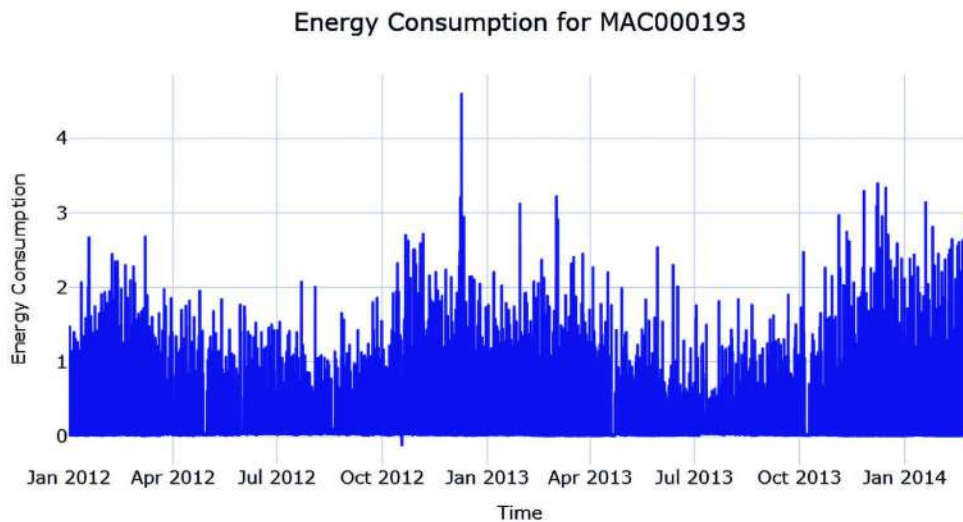


Figure 3.3: Line plot of household MAC000193

When you have a long time series with high variation, as we have, the line plot can get a bit chaotic. One of the options to get a macro view of the time series in terms of trends and movement is to plot a smoothed version of the time series. Let's see what a rolling monthly average of the time series looks like:

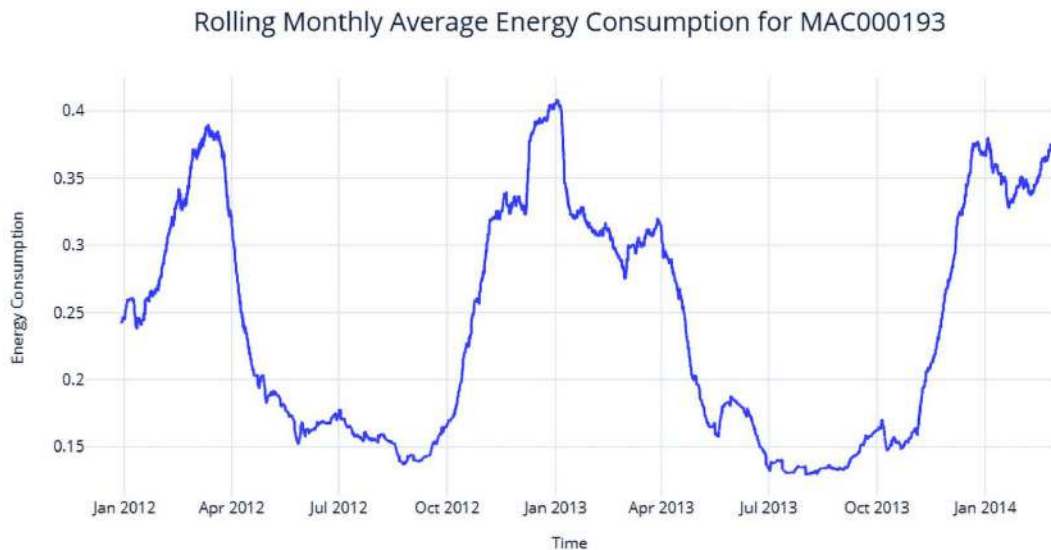


Figure 3.4: Rolling monthly average energy consumption of household MAC000193

We can see the macro patterns much more clearly now. The seasonality is clear—the series peaks in winter and troughs during summer. If you think about it critically, it makes sense. This is London we are talking about, and the energy consumption would be higher during the winter because of lower temperatures and subsequent heating system usage.

For a household in the tropics, for example, the pattern may be reversed, with the peaks coming in summer when air conditioners come into play.

Another use for the line chart is to visualize two or more time series together and investigate any correlations between them. In our case, let's try plotting the temperature along with the energy consumption and see whether the hypothesis we have about temperature influencing energy consumption holds good:

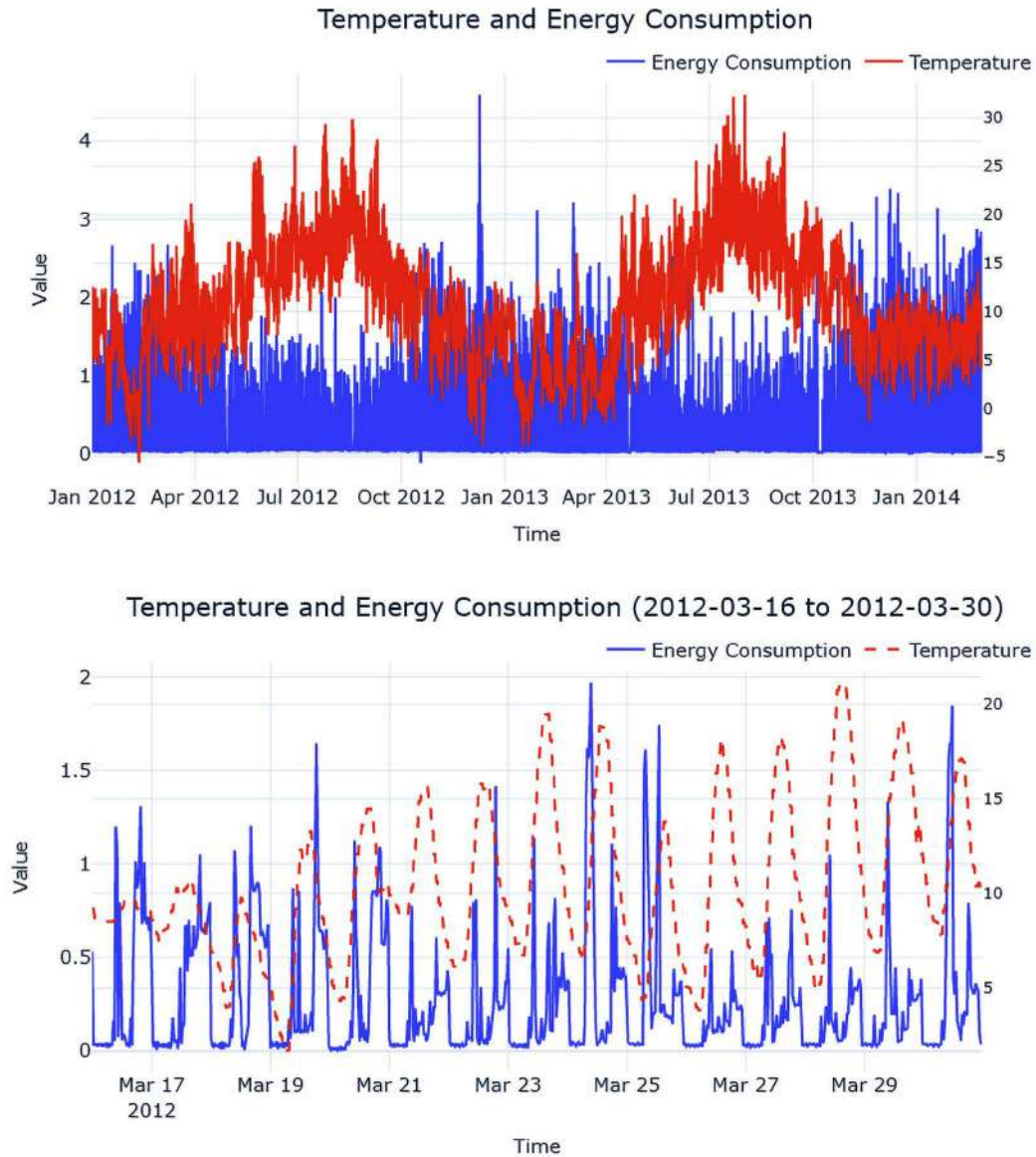


Figure 3.5: Temperature and energy consumption (zoomed-in plot at the bottom)

Here, we can see a clear negative correlation in yearly resolution between energy consumption and temperature. Winters show higher energy consumption on a macro scale. We can also see the daily patterns that are loosely correlated with temperature, but maybe because of other factors such as people coming back home after work and so on.

There are a few other visualizations that are more suited to bringing out seasonality in a time series. Let's take a look.

Seasonal plots

A **seasonal plot** is very similar to a line plot, but the key difference here is that the x -axis denotes the “seasons,” the y -axis denotes the time series value, and the different seasonal cycles are represented in different colors or line types. For instance, the yearly seasonality at a monthly resolution can be depicted with months on the x -axis and different years in different colors.

Let's see what this looks like for our household in question. Here, we have plotted the average monthly energy consumption across multiple years:

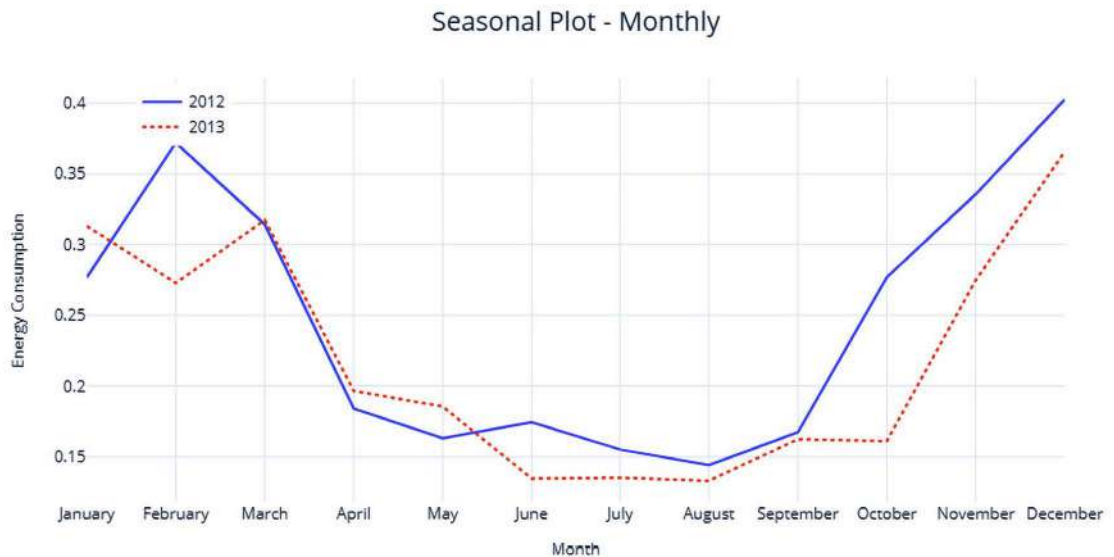


Figure 3.6: Seasonal plot at a monthly resolution

We can instantly see the appeal in this visualization because it lets us visualize the seasonality pattern easily. We can see that the consumption goes down in the summer months and we can also see that it happens consistently across multiple years. In the two years that we have data for, we can see that in October, the behavior in 2013 slightly deviated from 2012. Maybe there is something else that can help us explain this difference—what about temperature?

We can also plot the seasonal plots with another variable of interest, such as the temperature:

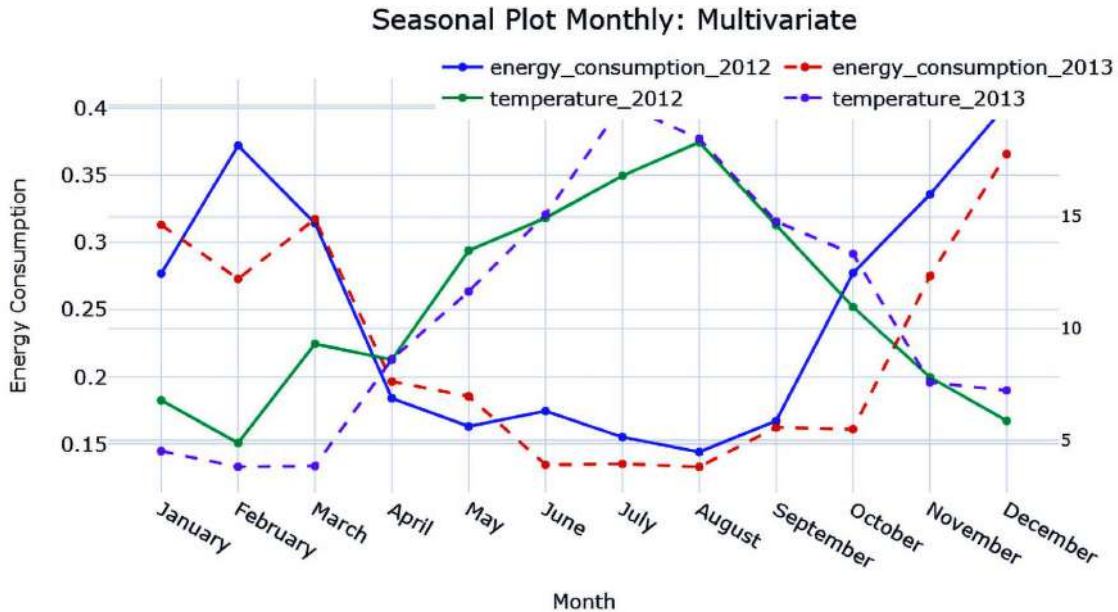


Figure 3.7: Seasonal plot at a monthly resolution (energy consumption versus temperature)

Notice October? In October 2013, the temperature stayed warmer for one month more, hence why the energy consumption pattern was slightly different from last year.

We can plot these kinds of plots at other resolutions as well, such as hourly seasonality. All we need to do is calculate the average consumption for each hour and day of the month and plot them with hours on the x -axis and different days of the month in different colors (Figure 3.8 (top)). But when there are too many seasonal cycles to be plotted, it increases visual clutter. An alternative to a seasonal plot is a seasonal box plot.

Seasonal box plots

Instead of plotting the different seasonal cycles in different colors or line types, we can represent them as a box plot (Figure 3.8 (bottom)). This instantly clears up the clutter in the plot. The additional benefit you get from this representation is that it lets us understand the variability across seasonal cycles:

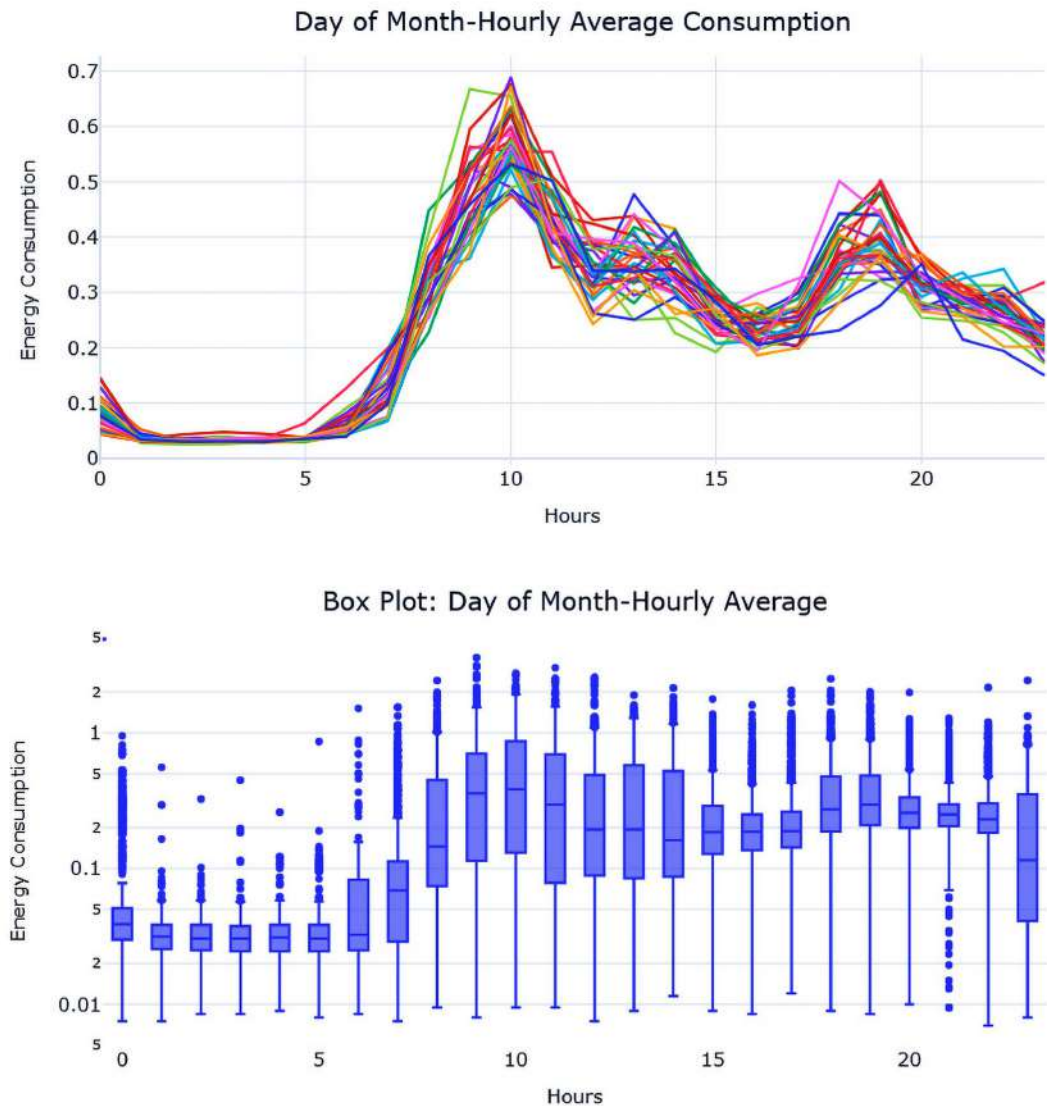


Figure 3.8: Seasonal plot (top) and seasonal box plot (bottom) at an hourly resolution

Here, we can see that the seasonal plot at this resolution is too cluttered to make out the pattern and the variation across seasonal cycles. However, the seasonal box plot is much more informative. The horizontal line in the box tells us about the median, the box is the **interquartile range (IQR)**, and the points that are marked are the outliers. By looking at the medians, we can see that the peak consumption occurs from 9 A.M. onward. But the variability is also higher from 9 A.M. If you plot separate box plots for each week, for example, you will see that the patterns are slightly different on Sundays (additional visualizations are in the associated notebook).

However, there is another visualization that lets you inspect these patterns along two dimensions.

Calendar heatmaps

Instead of having separate box plots or separate line charts for each week of the day, it would be useful if we could condense that information into a single plot. This is where **calendar heatmaps** come in. A heatmap visualization uses color gradients to represent data values, with different colors indicating varying intensities or frequencies. A calendar heatmap uses colored cells in a rectangular block to represent the information. Along the two sides of the rectangle, we can find two separate granularities of time, such as month and year. In each intersection, the cell is colored relative to the value of the time series at that intersection.

Let's look at the hourly average energy consumption across the different weekdays in a calendar heatmap (refer to the color images file:<https://packt.link/gbp/9781835883181>):

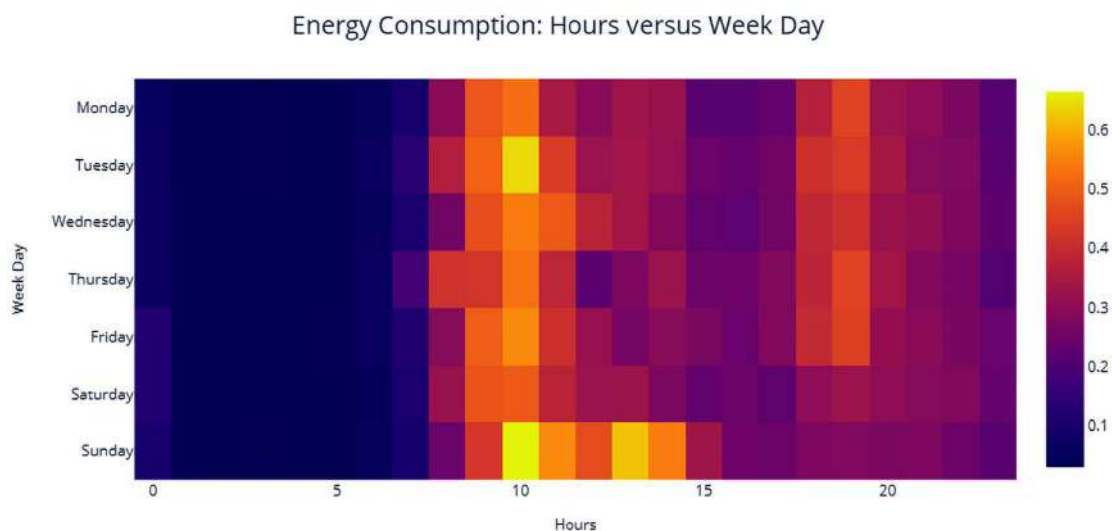


Figure 3.9: A calendar heatmap for energy consumption

From the color scale on the right, we know that lighter colors mean higher values. We can see how Monday to Saturday have similar peaks—that is, once in the morning and once in the evening. However, Sunday has a slightly different pattern, with higher consumption throughout the day.

So far, we've reviewed a lot of visualizations that can bring out seasonality. Now, let's look at a visualization for inspecting autocorrelation.

Autocorrelation plot

If correlation indicates the strength and direction of the linear relationship between two variables, autocorrelation is the correlation between the values of a time series in successive periods. Most time series have a heavy dependence on the value in the previous period, and this is a critical component in a lot of the forecasting models we will be seeing as well.

Something such as ARIMA (which we will briefly look at in *Chapter 4, Setting a Strong Baseline Forecast*) is built on autocorrelation. So, it's always helpful to just visualize and understand how strong the dependence on previous time steps is.

This is where **autocorrelation plots** come in handy. In such plots, we have the different lags ($t-1$, $t-2$, $t-3$, and so on) on the x -axis and the correlations between t and the different lags on the y -axis. In addition to autocorrelation, we can also look at **partial autocorrelation**, which is very similar to autocorrelation but with one key difference: partial autocorrelation removes any indirect correlation that may be present before presenting the correlations. Let's look at an example to understand this. If t is the current time step, let's assume $t-1$ is highly correlated to t . So, by extending this logic, $t-2$ will be highly correlated with $t-1$ and because of this correlation, the autocorrelation between t and $t-2$ would be high. However, partial autocorrelation corrects this and extracts the correlation, which can be purely attributed to $t-2$ and t .

One thing we need to keep in mind is that the autocorrelation and partial autocorrelation analysis works best if the time series is stationary (we will talk about stationarity in detail in *Chapter 6, Feature Engineering for Time Series Forecasting*).

Best practice:



There are many ways of making a series stationary, but a quick and dirty way is to use seasonal decomposition and just pick the residuals. It should be devoid of trends and seasonality, which are the major drivers of non-stationarity in a time series. But as we will see later in this book, this is not a foolproof method of making a series stationary in the truest sense.

Now, let's see what these plots look like for our household from the dataset (after making it stationary):

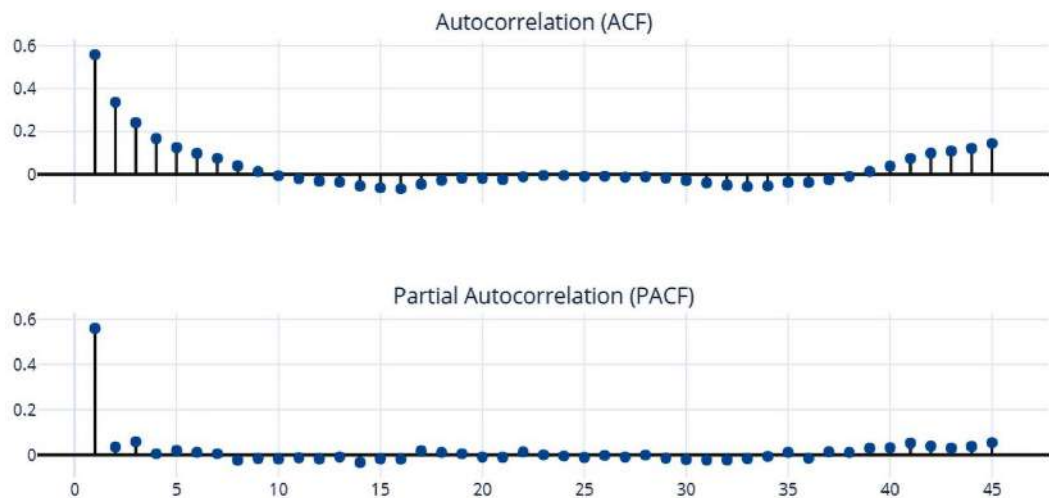


Figure 3.10: Autocorrelation and partial autocorrelation plots

Here, we can see that the first lag ($t-1$) has the most influence and that its influence quickly drops down to close to zero in the partial autocorrelation plot. This means that the energy consumption of a day is highly correlated with the energy consumption the day before.

If you've seen such charts before, you would have seen an envelope over this showing the confidence intervals as a guide to selecting significant autocorrelations. While that is a good thumb of rule, it's not included here because I don't want you to use it as a rule. The relevance of the confidence intervals depends on some assumptions (normality and so on), which may not be satisfied all the time, especially in real-world use cases.

With that, we've looked at the different components of a time series and learned how to visualize a few of them. Now, let's see how we can decompose a time series into its components.

Decomposing a time series

Seasonal decomposition is the process by which we deconstruct a time series into its components—typically, trend, seasonality, and residuals. The general approach for decomposing a time series is as follows:

1. **Detrending:** Here, we estimate the **trend component** (which is the smooth change in the time series) and remove it from the time series, giving us a **detrended time series**.
2. **Deseasonalizing:** Here, we estimate the seasonality component from the detrended time series. After removing the seasonal component, what is left is the residual.

Let's discuss them in detail.

Detrending

Detrending can be done in a few different ways. Two popular ways of doing it are by using **moving averages** and **locally estimated scatterplot smoothing (LOESS) regression**.

Moving averages

One of the easiest ways of estimating trends is by using a moving average along the time series. It can be seen as a window that is moved along the time series in steps, and at each step, the average of all the values in the window is recorded. This moving average is a smoothed-out time series and helps us estimate the slow change in a time series, which is the trend. The downside is that the technique is quite noisy. Even after smoothing out a time series using this technique, the extracted trend will not be smooth; it will be noisy. The noise should ideally reside with the residuals and not the trend (see the trend line shown in *Figure 3.13*).

LOESS

The LOESS algorithm, which is also called *locally weighted polynomial regression*, was developed by Bill Cleveland from the 70s to the 90s. It is a non-parametric method that is used to fit a smooth curve onto a noisy signal. We use an ordinal variable that moves between the time series as the independent variable and the time series signal as the dependent variable.

For each value in the ordinal variable, the algorithm uses a fraction of the closest points and estimates a smoothed trend using only those points in a weighted regression. The weights in the weighted regression are the closest points to the point in question. This is given the highest weight and it decays as we move farther away from it. This gives us a very effective tool for modeling the smooth changes in the time series (trend) (see the trend line shown in *Figure 3.14*).

Deseasonalizing

The seasonality component can also be estimated in a few different ways. The two most popular ways of doing this are by using period-adjusted averages or a Fourier series.

Period adjusted averages

This is a pretty simple technique wherein we calculate a seasonality index for each period in the expected cycle by taking the average values of all such periods over all the cycles. To make that clear, let's look at a monthly time series where we expect an annual seasonality in this time series. So, the up-and-down pattern would complete a full cycle in 12 months, or the seasonality period is 12. In other words, every 12 points in the time series have similar seasonal components. So, we take the average of all January values as the period-adjusted average for January. In the same way, we calculate the period average for all 12 months. At the end of the exercise, we have 12 period averages, and we can also calculate an *average* period average. Now, we can make these period averages into an index by either subtracting the average of all period averages from each of the period averages (for additive) or dividing the average of all period averages from each of the period averages (multiplicative).

Fourier series

In the late 1700s, Joseph Fourier, a mathematician and physicist, while studying heat flow, realized something profound—*any* periodic function can be broken down into a simple series of sine and cosine waves. Let's dwell on that for a minute. Any periodic function, no matter the shape, curve, or absence of it, or how wildly it oscillates around the axis, can be broken down into a series of sine and cosine waves.



Additional information:

For the more mathematically inclined, the original theory proposes to decompose any periodic function into an integral of exponentials. Using Euler's identity, $e^{iy} = \cos(y) + i \cdot \sin(y)$, we can consider them as a summation of sine and cosine waves. The *Further reading* section contains a few resources if you want to delve deeper and explore related concepts, such as the Fourier transform.

It is this property that we use to extract seasonality from a time series because seasonality is a periodic function, and any periodic function can be approximated by a combination of sine and cosine waves. The sine-cosine form of a Fourier series is as follows:

$$S_{N(x)} = \frac{a_0}{2} + \sum_{n=1}^N \left(a_n \cdot \cos\left(\frac{2\pi}{P} \cdot n \cdot x\right) + b_n \cdot \sin\left(\frac{2\pi}{P} \cdot n \cdot x\right) \right)$$

Here, S_N is the N -term approximation of the signal, S . Theoretically, when N is infinite, the resulting approximation is equal to the original signal. P is the maximum length of the cycle.

We can use this Fourier series, or a few terms from the Fourier series, to model our seasonality. In our application, P is the maximum length of the cycle we are trying to model. For instance, for a yearly seasonality for monthly data, the maximum length of the cycle (P) is 12. x would be an ordinal variable that increases from 1 to P . In this example, x would be 1, 2, 3, ... 12. Now, with these terms, all that is left to do is find a_n and b_n , which we can do by regressing on the signal.

We've seen that with the right combination of Fourier terms, we can replicate any signal. But the question is, should we? What we want to learn from data is a generalized seasonality profile that does well with unseen data as well. So, we use N as a hyperparameter to extract as complex a signal as we want from the data.

This is a good time to brush up on your trigonometry and remember what sine and cosine waves look like. The first Fourier term ($n=1$) is your age-old sine and cosine waves, which complete one full cycle in the maximum cycle length (P). As we increase n , we get sine and cosine waves that have multiple cycles in the maximum cycle length (P). This can be seen in the following figure:

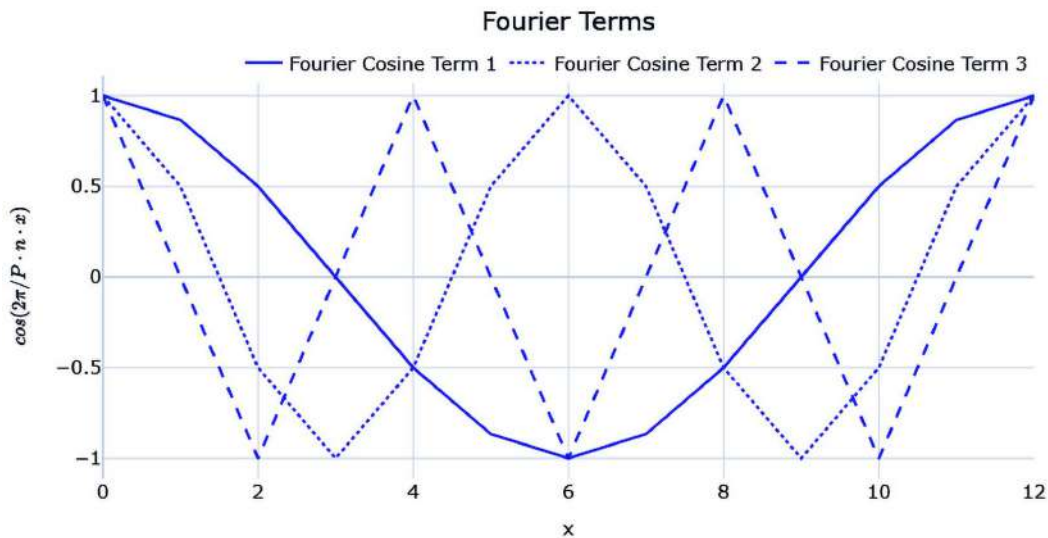


Figure 3.11: Cosine Fourier terms ($n = 1, 2, 3$)

The sine and cosine waves are complementary to each other, as shown in the following figure:

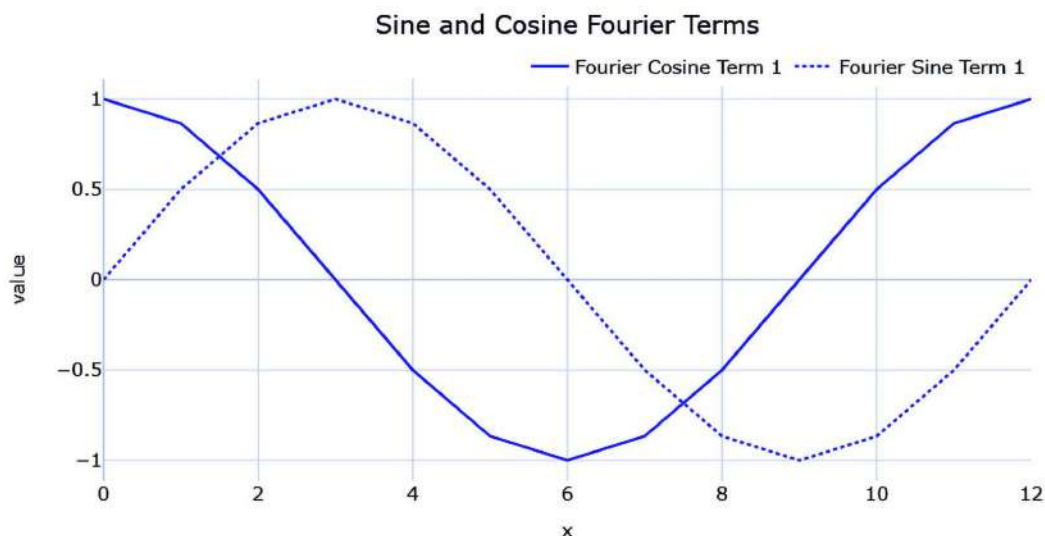


Figure 3.12: Sine and cosine Fourier terms ($n = 1$)

Now, let's see how we can use this in practice.

Implementations



Notebook alert:

To follow along with the complete code for decomposing time series, use the `02-Decomposing_Time_Series.ipynb` notebook in the `Chapter03` folder.

There are four implementations that we will cover here in the following subsections.

seasonal_decompose from statsmodel

`statsmodels.tsa.seasonal` has a function called `seasonal_decompose`. This is an implementation that uses moving averages for the trend component and period-adjusted averages for the seasonal component. It supports both additive and multiplicative modes of decomposition. However, it doesn't tolerate missing values. Let's see how we can use it:

```
#Does not support missing values, so using imputed ts instead
res = seasonal_decompose(ts, period=7*48, model="additive", extrapolate_
trend="freq")
```

A few key parameters to keep in mind are as follows:

- `period` is the seasonal period you expect the pattern to repeat.
- `model` takes `additive` or `multiplicative` as an argument to determine the type of decomposition.

- `filt` takes in an array that is used as the weights in the moving average (convolution, to be specific). It can also be used to define the window over which we need our moving average. We can increase it to smooth out the trend component to some extent.
- `extrapolate_trend` is a parameter that we can use to extend the trend component to both sides to avoid the missing values that are generated when applying the moving average filter.
- `two_sided` is a parameter that lets us define how the moving averages are calculated. If `True`, which it is by default, the moving average is calculated using the past as well as future values because the window for the moving average is centered. If `False`, it only uses past values to calculate the moving average.

Let's see how well we have been able to decompose one of the time series in our datasets. We used `period=7*48` to capture a weekday-hourly profile and `filt=np.repeat(1/(30*48), 30*48)` to make the moving average over 30 days with uniform weights:

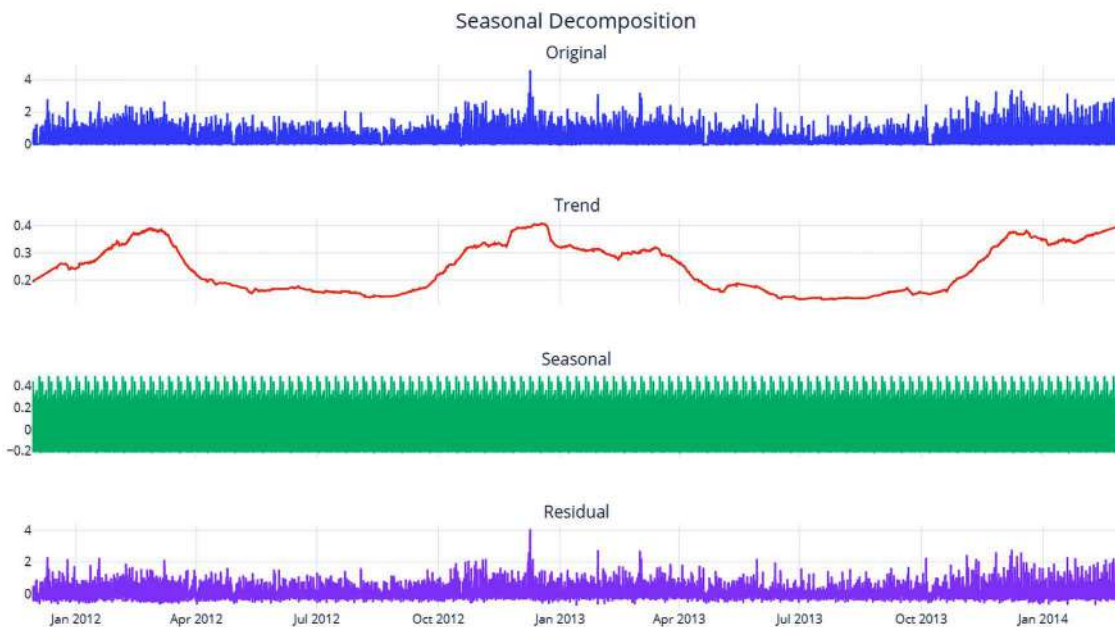


Figure 3.13: Seasonal decomposition using statsmodels

We can't see the seasonal pattern because it's too small in the grand scale of the plot. The associated notebook has zoomed-in plots to help you understand the seasonal pattern. Even with a large window (for example, 20 days) of smoothing, the trend still has some noise in it. We may be able to reduce this a bit more by increasing the window, but there is a better alternative, as we will see now.

Seasonality and trend decomposition using LOESS (STL)

As we saw earlier, LOESS is much more suited for trend estimation. STL is an implementation that uses LOESS for trend estimation and period averages for seasonality. Although statsmodels has an implementation, we have reimplemented it for better performance and flexibility.

This implementation can be found in this book's GitHub repository under `src.decomposition.seasonal.py`. It expects a pandas DataFrame or series with a datetime index as an input. Let's see how we can use this:

```
stl = STL(seasonality_period=7*48, model = "additive")
res_new = stl.fit(ts_df.energy_consumption)
```

The key parameters here are as follows:

- `seasonality_period` is the seasonal period you expect the pattern to repeat.
- `model` takes `additive` or `multiplicative` as an argument to determine the type of decomposition.
- `lo_frac` is the fraction of the data that will be used to fit the LOESS regression.
- `lo_delta` is the fractional distance within which we use linear interpolation instead of weighted regression. Using a non-zero `lo_delta` significantly decreases computation time.

Let's see what this decomposition looks like. Here, we used `seasonality_period=7*48` to capture a weekday-hourly profile:

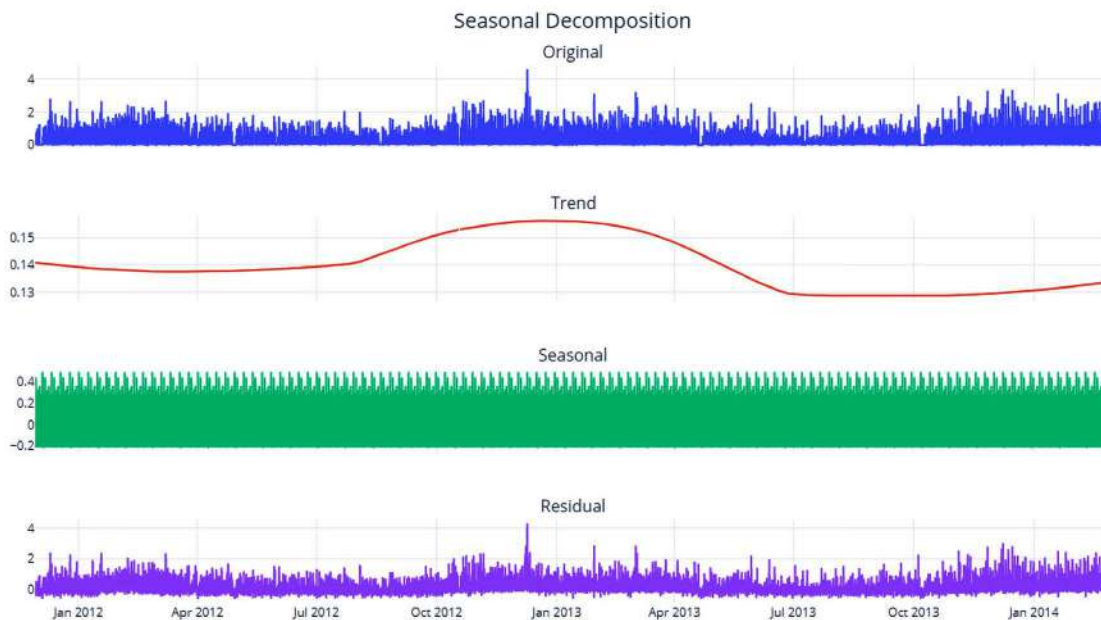


Figure 3.14: STL decomposition

Let's also look at the decomposition for just one month to see the extracted seasonality patterns clearer:

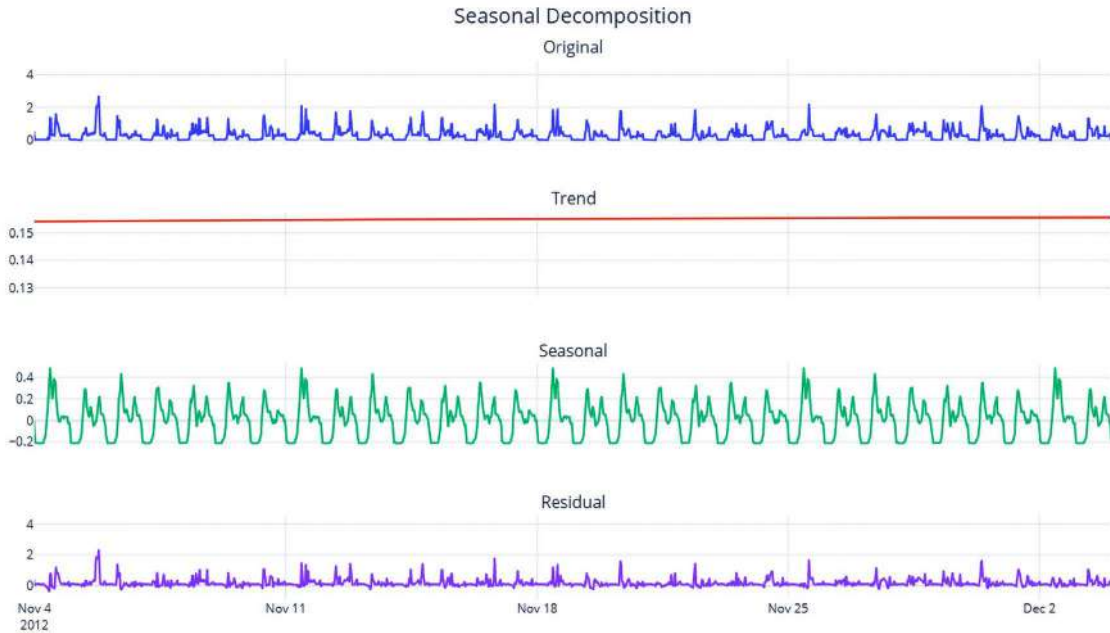


Figure 3.15: STL decomposition (zoomed-in for a month)

The trend is smooth enough now and seasonality has also been captured. Here, we can clearly see the hourly peaks and valleys and the higher peaks on weekends. However, since we are relying on averages to derive the seasonality, it is also highly influenced by outliers. A few very high or very low values in the time series will skew your seasonality profile that's been derived from period averages. Another disadvantage of this technique is that the “goodness” of the seasonality that's been extracted suffers when the difference between the resolution of the data and the expected seasonality cycle is greater. For instance, when extracting a yearly seasonality on daily or sub-daily data, this would make the extracted seasonality very noisy. This technique will also not work if you have less than two cycles of the expected seasonality—for instance, if we want to extract a yearly seasonality, but we have less than 2 years of data.

Fourier decomposition

We can find the Python implementation for decomposing a time series using Fourier terms in `src.decomposition.seasonal.py`. It uses LOESS for trend detection and Fourier terms for seasonality extraction. There are two ways we can use it. First, we can specify `seasonality_period` as one of the pandas datetime properties (such as `hour` and `week_of_day`):

```
stl = FourierDecomposition(seasonality_period="hour", model = "additive", n_
fourier_terms=5)
res_new = stl.fit(pd.Series(ts.squeeze(), index=ts_df.index))
```

Alternatively, we can create any custom seasonality array that's the same length as the time series that has an ordinal representation of the seasonality. If it is an annual seasonality of daily data, the array would have a minimum value of 1 and a maximum value of 365 as it increases by one every day of the year:

```
#Making a custom seasonality term
ts_df["dayofweek"] = ts_df.index.dayofweek
ts_df["hour"] = ts_df.index.hour
#Creating a sorted unique combination df
map_df = ts_df[["dayofweek", "hour"]].drop_duplicates().sort_
values(["dayofweek", "hour"])
# Assigning an ordinal variable to capture the order
map_df["map"] = np.arange(1, len(map_df)+1)
# mapping the ordinal mapping back to the original df and getting the
seasonality array
seasonality = ts_df.merge(map_df, on=["dayofweek", "hour"], how='left',
validate="many_to_one")['map']
stl = FourierDecomposition(model = "additive", n_fourier_terms=50)
res_new = stl.fit(pd.Series(ts, index=ts_df.index), seasonality=seasonality)
```

The key parameters that are involved in this process are as follows:

- `seasonality_period` is the seasonality to be extracted from the *datetime index*. pandas date-time properties, such as `week_of_day` and `month`, can be used to specify the most prominent seasonality. If left set to `None`, you need to provide the seasonality array while calling `fit`.
- `model` takes `additive` or `multiplicative` as an argument to determine the type of decomposition.
- `n_fourier_terms` determines the number of Fourier terms to be used to extract the seasonality. The more we increase this parameter, the more complex the seasonality that is extracted from the data.
- `lo_frac` is the fraction of the data that will be used to fit the LOESS regression.
- `lo_delta` is the fractional distance within which we use linear interpolation instead of weighted regression. Using a non-zero `lo_delta` significantly decreases computation time.

Let's see the zoomed-in plot for the decomposition using `FourierDecomposition`:

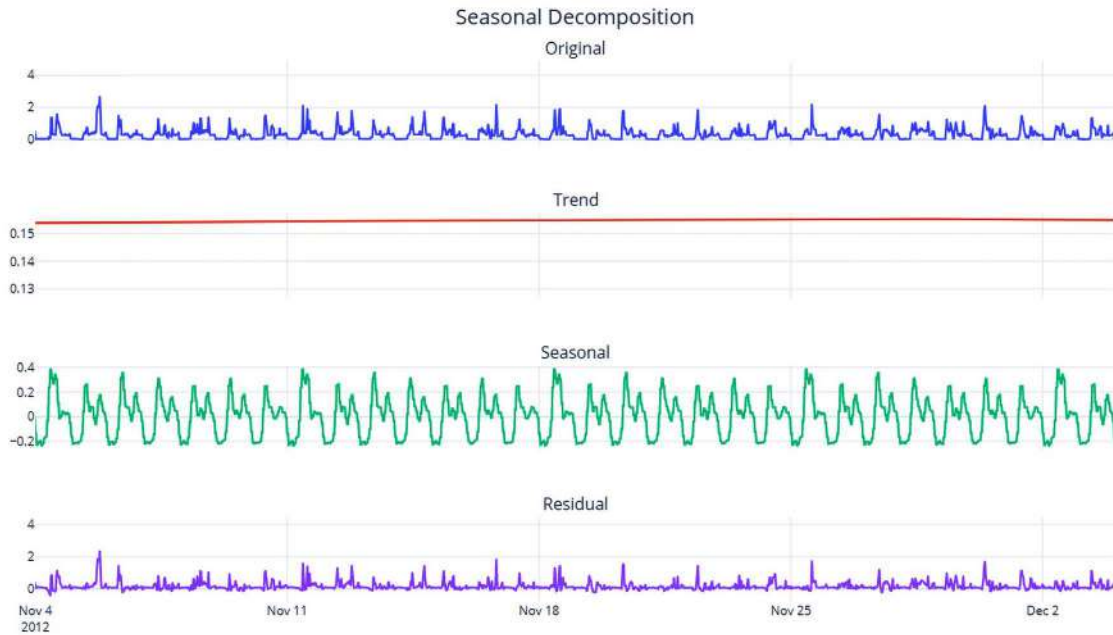


Figure 3.16: Decomposition using Fourier terms (zoomed-in for a month)

The trend is going to be the same as the STL one because we are using LOESS here as well. The seasonality profile may be slightly different and robust to outliers because we are doing regularized regression using the Fourier terms on the signal. Another advantage is that we have decoupled the resolution of the data and the expected seasonality. Now, extracting a yearly seasonality on sub-daily data is not as challenging as with period averages.

So far, we have only seen techniques that extract one seasonality per series; mostly, we extract the major seasonality. So, what do we do when we have multiple seasonal patterns?

Multiple seasonality decomposition using LOESS (MSTL)

Time series with high-frequency data (such as daily or hourly data) are prone to exhibit multiple seasonal patterns. For instance, there may be an hourly seasonality pattern, a weekly seasonality pattern, and a yearly seasonality pattern. But if we extract only the dominant pattern and leave the rest to residuals, we are not doing justice to the decomposition. Kasun Bandara et al. proposed an extension of STL decomposition for multiple seasonality, known as **MSTL**, and a corresponding implementation is present in the R ecosystem. A very similar implementation in Python can be found in `src.decomposition.seasonal.py`. In addition to MSTL, the implementation extracts multiple seasonality using Fourier terms.



Reference check:

The research paper by Kasun Bandara et al. is cited in the *References* section as reference 1.

Let's look at an example of how we can use this:

```
stl = MultiSeasonalDecomposition(seasonal_model="fourier", seasonality_
periods=["day_of_year", "day_of_week", "hour"], model = "additive", n_fourier_
terms=10)
res_new = stl.fit(pd.Series(ts, index=ts_df.index))
```

The key parameters here are as follows:

- `seasonality_periods` is the list of expected seasonalities. For `stl`, it is a list of seasonal periods, while for `FourierDecomposition`, it is a list of strings that denotes pandas datetime properties.
- `seasonality_model` takes `fourier` or `averages` as an argument to determine the type of seasonality decomposition.
- `model` takes `additive` or `multiplicative` as an argument to determine the type of decomposition.
- `n_fourier_terms` determines the number of Fourier terms to be used to extract the seasonality. As we increase this parameter, the more complex the seasonality that is extracted from the data.
- `lo_frac` is the fraction of the data that will be used to fit the LOESS regression.
- `lo_delta` is the fractional distance within which we use linear interpolation instead of weighted regression. Using a non-zero `lo_delta` significantly decreases computation time.

Let's see what the decomposition looks like when using Fourier decomposition:

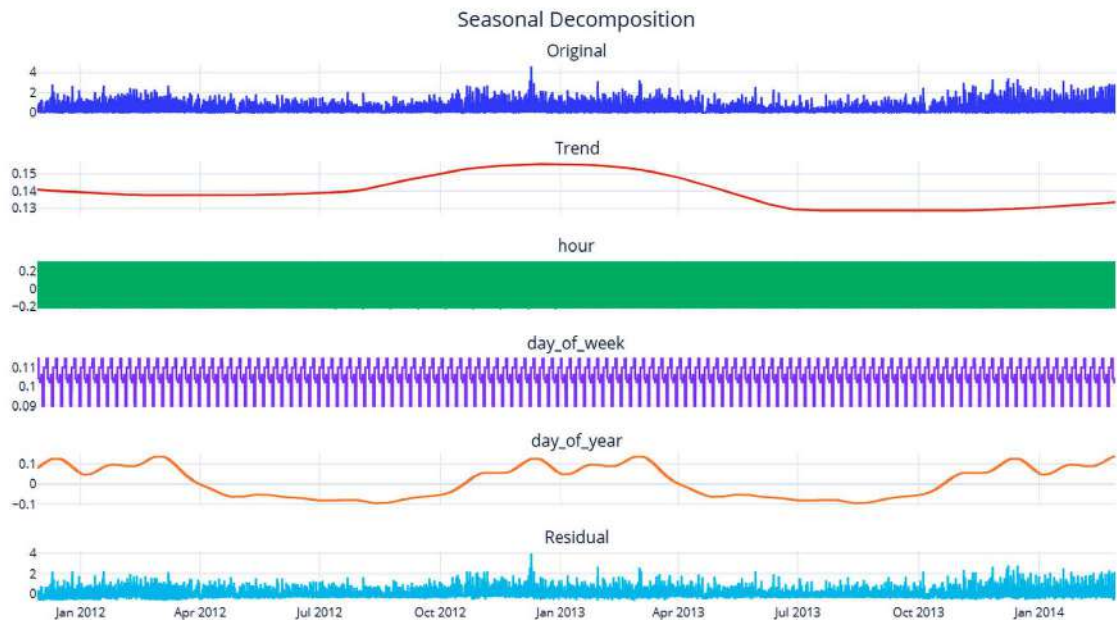


Figure 3.17: Multiple seasonality decomposition using Fourier terms

Here, we can see that the `day_of_week` seasonality has been extracted. To see the `day_of_week` and hour seasonal components, we need to zoom in a bit:

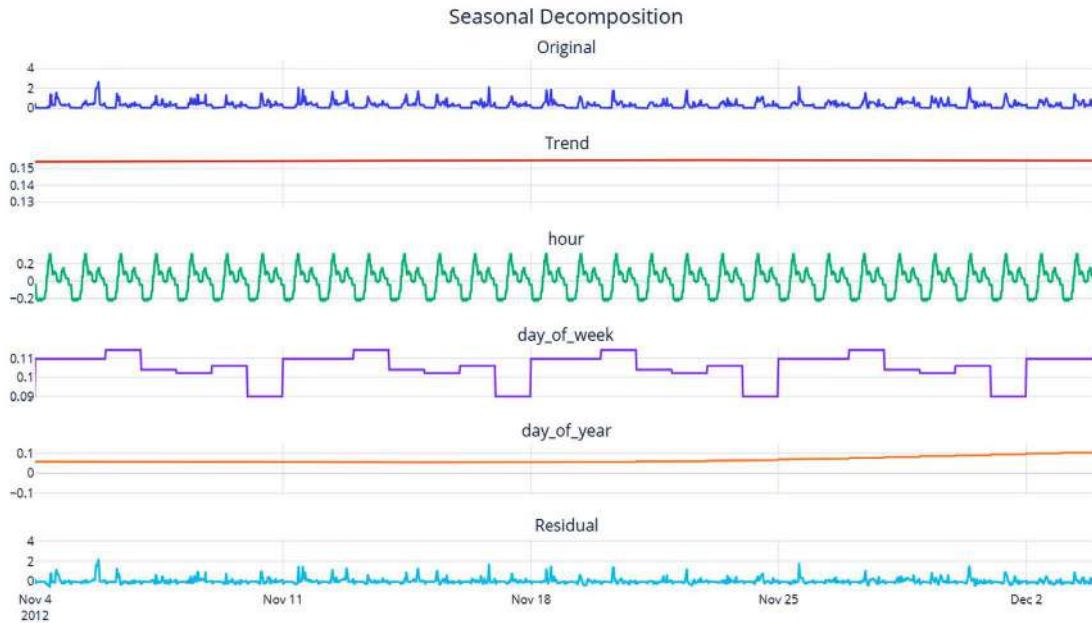


Figure 3.18: Multiple seasonality decomposition using Fourier terms (zoomed-in for a month)

Here, we can observe that the hour seasonality has been extracted well and that it has also isolated the `day_of_week` seasonal component, which peaks on weekends. The **discrete step** nature of the `day_of_week` seasonal component is because the frequency of the data is half-hourly, and for 48 data points, `day_of_week` will be the same.

We have summarized the four techniques we've covered in the following table:

Implementation	Trend	Seasonal	Supports Multiple Seasonality?	Supports Missing?
Seasonality decomposition	Moving Averages	Period-Adjusted Averages	No	No
STL	LOESS	Period-Adjusted Averages	No	Yes
Fourier decomposition	LOESS	Fourier Terms	No	No
Multiple seasonality decomposition	LOESS	Period-Adjusted Averages / Fourier Terms	Yes	No

Table 3.1: Different seasonal decomposition techniques



MSTL has also been implemented in statsmodels and the accompanying notebook has the code to use that. The key difference between statsmodels and the implementation bundled within the code for the book is that the one in the book also has the option for using Fourier-series-based decomposition.

Now, let's understand and analyze a time series dataset.

Detecting and treating outliers

An **outlier**, as its name suggests, is an observation that lies at an abnormal distance from the rest of the observations. If we are looking at a **data-generating process (DGP)** as a stochastic process that generates the time series, the outliers are the points that have the least probability of being generated from the DGP. This can be for many reasons, including faulty measurement equipment, incorrect data entry, and black-swan events, to name a few. Being able to detect such outliers and *treat* them may help your forecasting model understand the data better.

Outlier/anomaly detection is a specialized field itself in time series, but in this book, we are going to restrict ourselves to simpler techniques of identifying and treating outliers. This is because our main aim is not to detect outliers, but to clean the data for our forecasting models to perform better. If you want to learn more about anomaly detection, head over to the *Further reading* section for a few resources to get started.

Now, let's look at a few techniques for identifying outliers.



Notebook alert:

To follow along with the complete code for detecting outliers, use the `03-Outlier_Detection.ipynb` notebook in the `Chapter03` folder.

Standard deviation

This is a rule of thumb that almost everyone who has worked with data for some time would have heard of—if μ is the mean of the time series and σ is the standard deviation, then anything that falls beyond $\mu \pm 3\sigma$ is an **outlier**. The underlying theory is deeply rooted in statistics. If we assume that the values of the time series follow a normal distribution (which is a symmetrical distribution with very desirable properties), using probability theory, we can derive that 68% of the area under the normal distribution lies within one standard deviation on either side of the mean, about 95% of the area within two standard deviations, and about 99% of the area within three standard deviations. So, when we make the bounds as three standard deviations (by using the rule of thumb), what we are saying is that if any observation whose probability of belonging to the probability distribution is less than 1%, then they are an outlier.

Moving slightly to more practical issues, this cutoff of three standard deviations is in no way sacrosanct. We need to try out different values of this multiple and determine the right multiple by subjectively evaluating the results we get. The higher the multiple is, the fewer outliers there will be.

For highly seasonal data, the naïve way of applying the rule to the raw time series will not work well. In such cases, we must deseasonalize the data using any of the techniques we discussed earlier and then apply the outliers to the residuals. If we don't do that, we may flag a seasonal peak as an outlier, which is not what we want.

Another key assumption here is the normal distribution. However, in reality, a lot of the time series we come across may not be normal and hence the rule will lose its theoretical guarantees fast.

IQR

Another very similar technique is using the IQR instead of the standard deviation to define the bounds beyond which we mark the observations as outliers. A quantile arranges all your data in order and then splits it into equal parts, so each part has the same number of items. A quartile does the same but specifically divides your data into four equal parts. IQR is the difference between the 3rd quartile (or the 75th percentile or 0.75 quantile) and the 1st quartile (or the 25th percentile or 0.25 quantile). The upper and lower bounds are defined as follows:

- $Upper\ bound = Q3 + n \times IQR$
- $Lower\ bound = Q1 - n \times IQR$

Here, $IQR = Q3 - Q2$, and n is the multiple of IQRs that determines the width of the acceptable area.

For datasets where we observe high occurrences of outliers and wild variations, this is slightly more robust than the standard deviation. This is because the standard deviation and the mean are highly influenced by individual points in the dataset. If 2σ was the rule of thumb in the earlier method, here, it is 1.5 times the IQR. This also ties back to the same normal distribution assumption, and 1.5 times the IQR is equivalent to $\sim 3\sigma$ (2.7σ to be exact). The point about deseasonalizing before applying the rule applies here as well. It applies to all the techniques we will see here.

Isolation Forest

Isolation Forest is an unsupervised anomaly detection algorithm based on decision trees. A typical anomaly detection algorithm models the *normal* points and profiles outliers as any points that do not fit the *normal*. However, Isolation Forest takes a different path and models the outliers directly. It does this by creating a forest of decision trees by randomly splitting the feature space. This technique works on the assumption that the outlier points fall in the outer periphery and are easier to fall into a leaf node of a tree. Therefore, you can find the outliers in short branches, whereas normal points, which are closer together, will require longer branches. The “anomaly score” of any point is determined by the depth of the tree to be traversed before reaching that particular point. scikit-learn has an implementation of the algorithm under `sklearn.ensemble.IsolationForest`. Apart from the standard parameters for decision trees, the key parameter here is `contamination`. It is set to `auto` by default but can be set to any value between 0 and 0.5. This parameter specifies what percentage of the dataset you expect to be anomalous.

But one thing we have to keep in mind is that `IsolationForest` does not consider time at all and just highlights values that fall *outside the norm*.

Extreme studentized deviate (ESD) and seasonal ESD (S-ESD)

This statistics-based technique is more sophisticated than the basic $\pm\sigma$ technique but still uses the same assumption of normality. It is based on another statistical test, called Grubbs’s test, which is used to find a *single outlier* in a normally distributed dataset. ESD iteratively uses Grubbs’s test by identifying and removing an outlier at each step. It also adjusts the critical value based on the number of points left. For a more detailed understanding of the test, go to the *Further reading* section, where we have provided a couple of resources about ESD and S-ESD. In 2017, Hochenbaum et al. from Twitter Research proposed to use the generalized ESD with deseasonalization as a method of detecting outliers for time series.

We have adapted an existing implementation of the algorithm for our use case, and it is available in this book’s GitHub repository. While all the other methods leave it to the user to determine the right level of outliers by tweaking a few parameters, S-ESD only takes in an upper bound on the number of expected outliers and then identifies the outliers independently. For instance, we set the upper bound to 800 and the algorithm identified ~400 outliers in the data we are working with.



Reference check:

The research paper by Hochenbaum et al. is cited in the *References* section as reference 2.

Let’s see how the outliers were detected using all the techniques we have reviewed:

	# of Outliers	% of Outliers
3SD	802	2.12%
2SD on Residuals	728	1.92%
4IQR	747	1.97%
4SD on Residuals	468	1.24%
Isolation Forest	364	0.96%
Isolation Forest on Residuals	359	0.95%
ESD	420	1.11%
S-ESD	424	1.12%

Figure 3.19: Outliers detected using different techniques

Now that we've learned how to detect outliers, let's talk about how we can treat them and clean the dataset.

Treating outliers

The first question that we must answer is whether or not we should correct the outliers we have identified. The statistical tests that identify outliers automatically should go through another level of human verification. If we blindly “treat” outliers, we might be chopping off a valuable pattern that will help us forecast the time series. If you are only forecasting a handful of time series, then it still makes sense to look at the outliers and anchor them to reality by looking at the causes for such outliers.

But when you have thousands of time series, a human can't inspect all the outliers, so we will have to resort to automated techniques. A common practice is to replace an outlier with a heuristic such as the maximum, minimum, and 75th percentile. A better method is to consider the outliers as missing data and use any of the techniques we discussed earlier to impute the outliers.

One thing we must keep in mind is that outlier correction is not a necessary step in forecasting, especially when using modern methods such as machine learning or deep learning. Whether we do outlier correction or not is something we have to experiment with and figure out.

Well done! This was a pretty busy chapter, with a lot of concepts and code, so congratulations on finishing it. Feel free to head back and revise a few topics as needed.

Summary

In this chapter, we learned about the key components of a time series and familiarized ourselves with terms such as trend and seasonality. We also reviewed a few time series-specific visualization techniques that will come in handy during EDA. Then, we learned about techniques that let you decompose a time series into its components and saw techniques for detecting outliers in the data. Finally, we learned how to treat the identified outliers. Now, you are all set to start forecasting the time series, which we will start in the next chapter.

References

The following are the references for this chapter:

1. Kasun Bandara and Rob J Hyndman and Christoph Bergmeir. (2021). *MSTL: A Seasonal-Trend Decomposition Algorithm for Time Series with Multiple Seasonal Patterns*. arXiv:2107.13462 [stat. AP]. <https://arxiv.org/abs/2107.13462>.
2. Hochenbaum, J., Vallis, O., & Kejariwal, A. (2017). *Automatic Anomaly Detection in the Cloud Via Statistical Learning*. ArXiv, abs/1704.07706. <https://arxiv.org/abs/1704.07706>.

Further reading

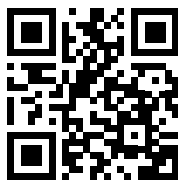
To learn more about the topics that were covered in this chapter, take a look at the following resources:

- Fourier Series: <https://www.setzeus.com/public-blog-post/the-fourier-series>
- Fourier Series from Khan Academy: <https://www.youtube.com/watch?v=UKHBWzoOKsY>
- Fourier Transform: <https://betterexplained.com/articles/an-interactive-guide-to-the-fourier-transform/>
- Ane Blázquez-García, Angel Conde, Usue Mori, and Jose A. Lozano. (2021). *A Review on Outlier/Anomaly Detection in Time Series Data*. arXiv:2002.04236. <https://arxiv.org/abs/2002.04236>
- Braei, M., & Wagner, S. (2020). *Anomaly Detection in Univariate Time-series: A Survey on the State-of-the-Art*. ArXiv, abs/2004.00433. <https://arxiv.org/abs/2004.00433>
- Generalized ESD Test for Outliers: <https://www.itl.nist.gov/div898/handbook/eda/section3/eda35h3.htm>

Join our community on Discord

Join our community's Discord space for discussions with authors and other readers:

<https://packt.link/mts>



4

Setting a Strong Baseline Forecast

In the previous chapter, we saw some techniques we can use to understand **time series data**, do some **Exploratory Data Analysis (EDA)**, and so on. But now, let's get to the crux of the matter—**time series forecasting**. The point of understanding the dataset and looking at patterns, seasonality, and so on was to make the job of forecasting that series easier. And with any machine learning exercise, one of the first things we need to establish before going further is a **baseline**.

A baseline is a simple model that provides reasonable results without requiring a lot of time to come up with them. Many people think of a baseline as something that is derived from common sense, such as an average or some rule of thumb. But as a best practice, a baseline can be as sophisticated as we want it to be, so long as it is quickly and easily implemented. Any further progress we want to make will be in terms of the performance of this baseline.

In this chapter, we will look at a few classical techniques that can be used as baselines, and strong baselines at that. Some may feel that the forecasting techniques we will be discussing in this chapter shouldn't be baselines, but we are keeping them in here because these techniques have stood the test of time—and for good reason. They are also very mature and can be applied with very little effort, thanks to the awesome open source libraries that implement them. There can be many types of problems/datasets where it is difficult to beat the baseline techniques we will discuss in this chapter, and in those cases, there is no shame in just sticking to one of these baseline techniques.

In this chapter, we will cover the following topics:

- Setting up a test harness
- Generating strong baseline forecasts
- Assessing the forecastability of a time series

Technical requirements

You will need to set up the Anaconda environment following the instructions in the *Preface* of the book to get a working environment with all the libraries and datasets required for the code in this book. Any additional library will be installed while running the notebooks.

You will need to run the following notebook before using the code in this chapter:

- The `02-Preprocessing_London_Smart_Meter_Dataset.ipynb` preprocessing notebook from Chapter02

The code for this chapter can be found at <https://github.com/PacktPublishing/Modern-Time-Series-Forecasting-with-Python-2E/tree/main/notebooks/Chapter04>.

Setting up a test harness

Before we start forecasting and setting up baselines, we need to set up a **test harness**. In software testing, a test harness is a collection of code and inputs that have been configured to test a program under various situations. In terms of machine learning, a test harness is a set of code and data that can be used to evaluate algorithms. It is important to set up a test harness so that we can evaluate all future algorithms in a standard and quick way.

The first thing we need is **holdout (test)** and **validation** datasets.

Creating holdout (test) and validation datasets

As a standard practice, in machine learning, we set aside two parts of the dataset, name them *validation data* and *test data*, and don't use them at all to train the model. The validation data is used in the modeling process to assess the quality of the model. To select between different model classes, tune the hyperparameters, perform feature selection, and so on, we need a dataset. Test data is like the final test of your chosen model. It tells you how well your model is doing in unseen data. If validation data is like the mid-term exams, the test data is your final exam.

In regular regression or classification, we usually sample a few records at random and set them aside. But while dealing with time series, we need to respect the temporal aspect of the dataset. Therefore, a best practice is to set aside the latest part of the dataset as the test data. Another rule of thumb is to set equal-sized validation and test datasets so that the key modeling decisions we make based on the validation data are as close as possible to the test data. The dataset that we introduced in *Chapter 2, Acquiring and Processing Time Series Data*, the London Smart Energy dataset, contains the energy consumption readings of households in London from November 2011 to February 2014. So, we are going to put aside January 2014 as the validation data and February 2014 as the test data.

Let's open `01-Setting_up_Experiment_Harness.ipynb` from the Chapter04 folder and run it. In the notebook, we must create the train-test split both before and after filling the missing values with `SeasonalInterpolation` and save them accordingly. Once the notebook finishes running, you will have created the following files in the pre-processed folder with the 2014 data saved separately:

- `selected_blocks_train.parquet`
- `selected_blocks_val.parquet`
- `selected_blocks_test.parquet`
- `selected_blocks_train_missing_imputed.parquet`
- `selected_blocks_val_missing_imputed.parquet`
- `selected_blocks_test_missing_imputed.parquet`

Now that we have a fixed dataset that can be used to fairly evaluate multiple algorithms, we need a way to evaluate the different forecasts.

Choosing an evaluation metric

In machine learning, we have a handful of metrics that can be used to measure continuous outputs, mainly **Mean Absolute Error** and **Mean Squared Error**. But in the time series forecasting realm, there are scores of metrics with no real consensus on which ones to use. One of the reasons for this overwhelming number of metrics is that no one metric measures every characteristic of a forecast. Therefore, we have a whole chapter devoted to this topic (*Chapter 19, Evaluating Forecast Errors—A Survey of Forecast Metrics*). For now, we will just review a few metrics, all of which we are going to use to measure the forecasts. We are just going to consider them at face value:

- **Mean Absolute Error (MAE)**: MAE is a very simple metric. It is the average of the unsigned (ignoring the sign) error between the forecast at timestep $t(f_t)$ and the observed value at time $t(y_t)$. The formula is as follows:

$$MAE = \frac{1}{N \times L} \times \sum_i^N \sum_j^L |f_{i,j} - y_{i,j}|$$

Here, N is the number of time series, L is the length of time series (in this case, the length of the test period), and f and y are the forecast and observed values, respectively.

- **Mean Squared Error (MSE)**: MSE is the average of the squared error between the forecast (f_t) and observed (y_t) values:

$$MSE = \frac{1}{N \times L} \times \sum_i^N \sum_j^L (f_{i,j} - y_{i,j})^2$$

- **Mean Absolute Scaled Error (MASE)**: MASE is slightly more complicated than MSE and MAE but gives us a slightly better measure to overcome the scale-dependent nature of the previous two measures. If we have multiple time series with different average values, MAE and MSE will show higher errors for the high-value time series as opposed to the low-valued time series. MASE overcomes this by scaling the errors based on the in-sample MAE from the **naïve forecasting method** (which is one of the most basic forecasts possible; we will review it later in this chapter). Intuitively, MASE gives us the measure of how much better our forecast is as compared to the naïve forecast:

$$MASE = \frac{\frac{1}{L} \times \sum_i^L |f_i - y_i|}{\frac{1}{L-1} \times \sum_{j=2}^L |y_j - y_{j-1}|}$$

- **Forecast Bias (FB)**: This is a metric with slightly different aspects from the other metrics we've seen. While the other metrics help assess the *correctness* of the forecast, irrespective of the direction of the error, forecast bias lets us understand the overall *bias* in the model. Forecast bias is a metric that helps us understand whether the forecast is continuously over- or under-forecasting.

We calculate forecast bias as the difference between the sum of the forecast and the sum of the observed values, expressed as a percentage over the sum of all actuals:

$$FB = \frac{\sum_i^N \sum_j^L f_{i,j} - \sum_i^N \sum_j^L y_{i,j}}{\sum_i^N \sum_j^L y_{i,j}}$$

Now, our test harness is ready. We also know how to evaluate and compare forecasts that have been generated from different models on a single, fixed holdout dataset with a set of predetermined metrics. Now, it's time to start forecasting.

Generating strong baseline forecasts

Time series forecasting has been around since the early 1920s, and through the years, many brilliant people have come up with different models, some statistical and some heuristic-based. I refer to them collectively as **classical statistical models** or **econometrics models**, although they are not strictly statistical/econometric.

In this section, we are going to review a few such models that can form really strong baselines when we want to try modern techniques in forecasting. As an exercise, we are going to use an excellent open source library for time series forecasting—NIXTLA (<https://github.com/Nixtla>). The 02-Baseline_Forecasts_using_NIXTLA.ipynb notebook contains the code for this section so that you can follow along.

Before we start looking at forecasting techniques, let's quickly understand how to use the NIXTLA library to generate forecasts. We are going to pick one consumer from the dataset and try out all the baseline techniques on the validation dataset one by one.

The first thing we need to do is select the consumer we want using the unique ID for each customer, the LCLid column (from the expanded form of data), and set the timestamp as the index of the DataFrame:

```
ts_train = train_df.loc[train_
df.LCLid=="MAC000193", ['LCLid', 'timestamp', 'energy_consumption']]
ts_val = val_df.loc[val_df.LCLid=="MAC000193", ['LCLid', 'timestamp', 'energy_
consumption']]
ts_test = test_df.loc[test_df.LCLid=="MAC000193", ['LCLid', 'timestamp', 'energy_
consumption']]
```

NIXTLA has the flexibility to work directly with either pandas or Polars DataFrames. By default, NIXTLA looks for three columns:

- **id_col:** By default, it expects a column `unique_id`. This column uniquely identifies the time series. If you only have one time series, add a dummy column with the same unique identifier.
- **time_col:** By default, it expects a column `ds`. This is the column of your timestamp.
- **target_col:** By default, it expects a column `y`. This column is what you want NIXTLA to forecast.

This is very convenient as there is no need for further manipulation to go from data to modeling. NIXTLA follows the scikit-learn style with `.fit()` and `.predict()` and also adopts a `.forecast()` method, which is a memory-efficient method that doesn't store the partial model outputs, whereas the scikit-learn interface stores the fitted models:

```
sf = StatsForecast(
    models=[model],
    freq=freq,
    n_jobs=-1,
    fallback_model=Naive()
)
sf.fit(df = _ts_train,          id_col = 'LCLid',
      time_col = 'timestamp',
      target_col = 'energy_consumption',
)
baseline_test_pred_df = sf.predict(len(ts_test) )
```

 **Quick tip:** Enhance your coding experience with the **AI Code Explainer** and **Quick Copy** features. Open this book in the next-gen Packt Reader. Click the **Copy** button (1) to quickly copy code into your coding environment, or click the **Explain** button (2) to get the AI assistant to explain a block of code to you.

```
function calculate(a, b) {
  return {sum: a + b};
};
```

Copy

Explain

1

2



 **The next-gen Packt Reader** is included for free with the purchase of this book. Unlock it by scanning the QR code below or visiting <https://www.packtpub.com/unlock/9781835883181>.



NIXTLA also has a `.forecast()` method, which is a memory-efficient method that doesn't store the partial model outputs, whereas the scikit-learn interface stores the fitted models:

```
# Efficiently fit and predict without storing memory
y_pred = sf.forecast(
```



```

h=len(ts_test),
df=ts_train,
id_col = 'LCLid',
time_col = 'timestamp',
target_col = 'energy_consumption',
)

```

When we call `.predict` or `.forecast`, we have to tell the model how long into the future we have to predict. This is called the horizon of the forecast. In our case, we need to predict our test period, which we can easily do by just taking the length of the `ts_test` array.

We can also calculate the metrics we discussed earlier in the test harness easily using NIXTLA's classes. For added flexibility, we can loop through a list of metrics to get multiple measurements for each forecast:

```

# Calculate metrics
metrics = [mase, mae, mse, rmse, smape, forecast_bias]
for metric in metrics:
    metric_name = metric.__name__
    if metric_name == 'mase':
        evaluation[metric_name] =
metric(results[target_col].values, results[model_name].values,
ts_train[target_col].values, seasonality=48)
    else:
        evaluation[metric_name] =
metric(results[target_col].values,
results[model_name].values)

```

Notice that, for MASE, the training set is also included.

For ease of experimentation, we have encapsulated all of this into a handy function, `evaluate_performance`, in the notebook. This returns the predictions and the calculated metrics in a `DataFrame`.

Now, let's start looking at a few very simple methods of forecasting.

Naïve forecast

A naïve forecast is as simple as you can get. The forecast is just the last/most recent observation in a time series. If the latest observation in a time series is 10, then the forecast for all future timesteps is 10. This can be implemented as follows using the `Naive` class in NIXTLA:

```

from statsforecast.models import Naive
models = Naive()

```

Once we have initialized the model, we can call our helpful `evaluate_performance` function in the notebook to run and record the forecast and metrics.

Let's visualize the forecast we just generated:

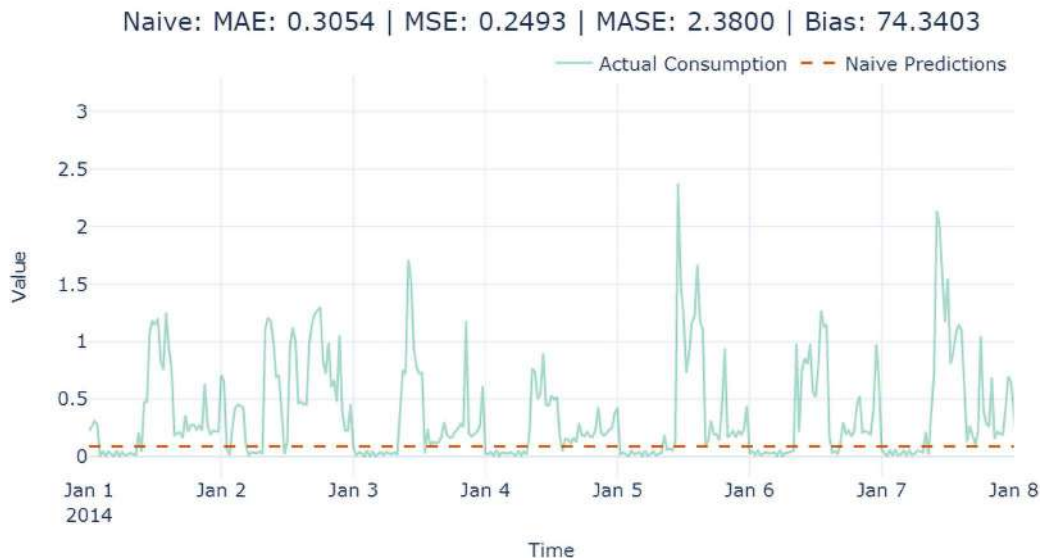


Figure 4.1: Naïve forecast

Here, we can see that the forecast is a straight line and completely ignores any pattern in the series. This is by far the simplest way to forecast, hence why it is naïve. Now, let's look at another simple method.

Moving average forecast

While a naïve forecast memorizes the most recent past, it also memorizes the noise at any timestep. A **moving average forecast** is another simple method that tries to overcome the pure memorization of the naïve method. Instead of taking the latest observation, it takes the mean of the latest n steps as the forecast. Moving average is not one of the models present in NIXTLA, but we have implemented a NIXTLA-compatible model in this book's GitHub repository in the `Chapter04` folder:

```
from src.forecasting.baselines import NaiveMovingAverage
#Taking a moving average over 48 timesteps, i.e., one day
naive_model = NaiveMovingAverage(window=48)
```

Let's look at the forecast we generated:

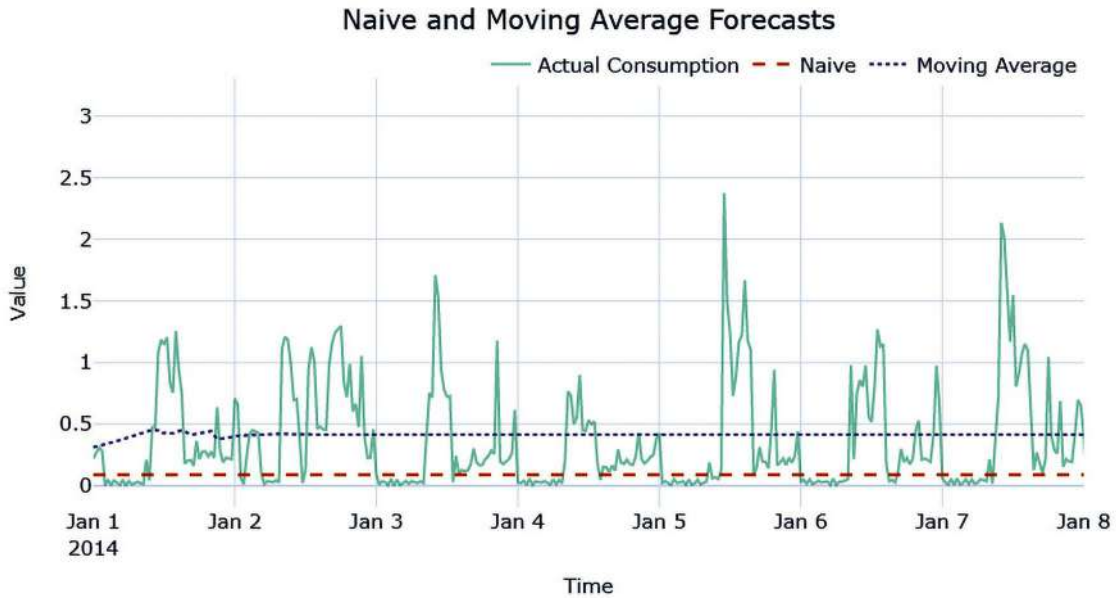


Figure 4.2: Moving average forecast

This forecast is also almost a straight line. Now, let's look at another simple method, but one that considers seasonality as well.

Seasonal naive forecast

A **seasonal naive forecast** is a twist on the simple naive method. In the naive method, we took the last observation (Y_{t-1}), whereas in seasonal naive, we take the Y_{t-k} observation. So, we look back k steps for each forecast. This enables the algorithm to mimic the last seasonality cycle. For instance, if we set $k=48*7$, we will be able to mimic the latest seasonal weekly cycle.

This method is implemented in NIXTLA and we can use it like so:

```
from statsforecast.models import SeasonalNaive
seasonal_naive = SeasonalNaive(season_length=48*7)
```

Let's see what this forecast looks like:

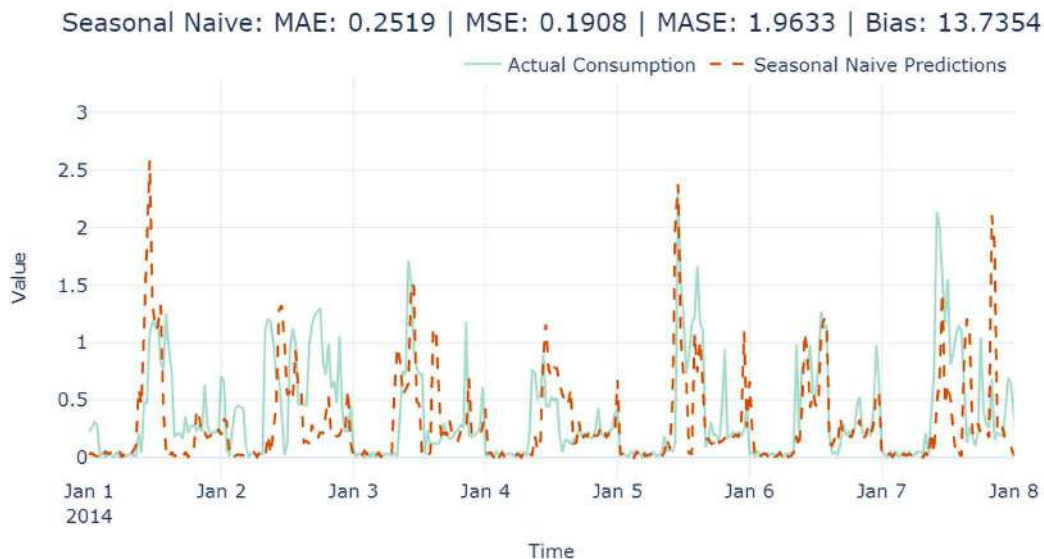


Figure 4.3: Seasonal naïve forecast

Here, we can see that the forecast is trying to mimic the seasonality pattern. However, it's not very accurate because it is blindly following the last seasonal cycle.

Now that we've looked at a few simple methods, let's look at a few statistical models.

Exponential smoothing

Exponential smoothing is one of the most popular methods for generating forecasts. It has been around since the late 1950s and has proved its mettle and stood the test of time. There are a few different variants of ETS—**single exponential smoothing**, **double exponential smoothing**, **Holt-Winters' seasonal smoothing**, and so on. But all of them have one key idea that has been used in different ways. In the naïve method, we were just using the latest observation, which is like saying only the most recent data point in history matters and no data point before that matters. On the other hand, the moving average method considers the last n observations to be equally important and takes the mean of them.

ETS combines both these intuitions and says that all the history is important, but the recent history is more important. Therefore, the forecast is generated using a weighted average where the weights decrease exponentially as we move farther into the history:

$$f_t = \alpha \cdot y_{t-1} + \alpha \cdot (1 - \alpha) \cdot y_{t-2} + \alpha \cdot (1 - \alpha)^2 \cdot y_{t-3} + \dots$$

Here, $0 \leq \alpha \leq 1$ is the smoothing parameter that lets us decide how fast or slow the weights should decay, y_t is the actuals at timestep t , and f_t is the forecast at timestep t .

Simple exponential smoothing (SES) is when you simply apply this smoothing procedure to the history. This is more suited for time series that have no trends or seasonality, and the forecast is going to be a flat line. The forecast is generated using the following formula:

<i>Forecast Equation</i>	$f_t = \alpha y_{t-1} + (1 - \alpha) f_{t-1}$
--------------------------	---

Double exponential smoothing (DES) extends the smoothing idea to model trends as well. It has two smoothing equations—one for the level and the other for the trend. Once you have the estimate of the level and trend, you can combine them. This forecast is not necessarily flat because the estimated trend is used to extrapolate it into the future. The forecast is generated according to the following formula:

<i>Forecast Equation</i>	$f_{t+h} = l_t + h \cdot b_t$
<i>Level Equation</i>	$l_t = \alpha \cdot y_t + (1 - \alpha) \cdot (l_{t-1} + b_{t-1})$
<i>Trend Equation</i>	$b_t = \beta \cdot (l_t - l_{t-1}) + (1 - \beta) \cdot b_{t-1}$

First, we estimate the level (l_t) using the *Level Equation* with the available observations. Then, we estimate the trend using the *Trend Equation*. Finally, to get the forecast, we combine l_t and b_t using the *Forecast Equation*.

Researchers have found empirical evidence that this kind of constant extrapolation can result in over-forecasts over the long-term forecast. This is because, in the real world, time series data doesn't increase at a constant rate forever. Motivated by this, an addition to this has also been introduced that dampens the trend by a factor of $0 < \phi < 1$, such that when $\phi = 1$, there is no damping, and it is identical to DES.

Triple exponential smoothing or Holt-Winters' (HW) takes this one step forward by including another smoothing term to model the seasonality. This has three parameters (α, β, γ) for the smoothing and uses a seasonality period (m) as input parameters. You can also choose between additive or multiplicative seasonality. The forecast equations for the additive model are as follows:

<i>Forecast Equation</i>	$f_{t+h} = l_t + h \cdot b_t + s_{t+h-m(k+1)}$
<i>Level Equation</i>	$l_t = \alpha \cdot y_t - s_{\{t-m\}} + (1 - \alpha) \cdot (l_{t-1} + b_{t-1})$
<i>Trend Equation</i>	$b_t = \beta \cdot (l_t - l_{t-1}) + (1 - \beta) \cdot b_{t-1}$
<i>Seasonality Equation</i>	$s_t = \gamma(y_t - l_{t-1} - b_{t-1}) + (1 - \gamma)s_{t-m}$

These formulae are also used like in the double exponential case. Instead of estimating level and trend, we estimate level, trend, and seasonality separately.

The family of ETS methods is not limited to the three that we just discussed. A way to think about the different models is in terms of the trend and seasonal components of these models. The trend can either be no trend, additive, or additive damped. The seasonality can be no seasonality, additive, or multiplicative. Every combination of these parameters is a different technique in the family, as shown in the following table:

Trend component	Seasonal component		
	N (None)	A (Additive)	M (Multiplicative)
N (None)	Simple Exponential Smoothing	-	-
A (Additive)	Double Exponential Smoothing	Additive Holt-Winters	Multiplicative Holt-Winters
Ad (Additive damped)	Damped Double Exponential Smoothing	-	Damped Holt-Winters

Table 4.1: Exponential smoothing family

NIXTLA has an entire family of ETS methods.

Let's see how we can initialize the ETS model in NIXTLA:

```
from statsforecast.models import (SimpleExponentialSmoothing, Holt,
                                  HoltWinters, AutoETS)

exp_smooth = HoltWinters(error_type = 'A', season_length = 48)]
```

Here, `error_type = 'A'` refers to additive error. The user has the option for either additive error or multiplicative error, which could be called using `error_type = 'M'`. NIXTLA models have an option to use `AutoETS()`. This model will automatically choose which exponential smoothing model is the best option: simple exponential smoothing, double exponential smoothing (Holt's method), or triple exponential smoothing (Holt-Winters method). It will also choose which parameters and error types are best for each individual time series. Refer to the GitHub notebooks for examples of how to use `AutoETS()`.

Let's see what the forecast using ETS looks like in *Figure 4.4*:

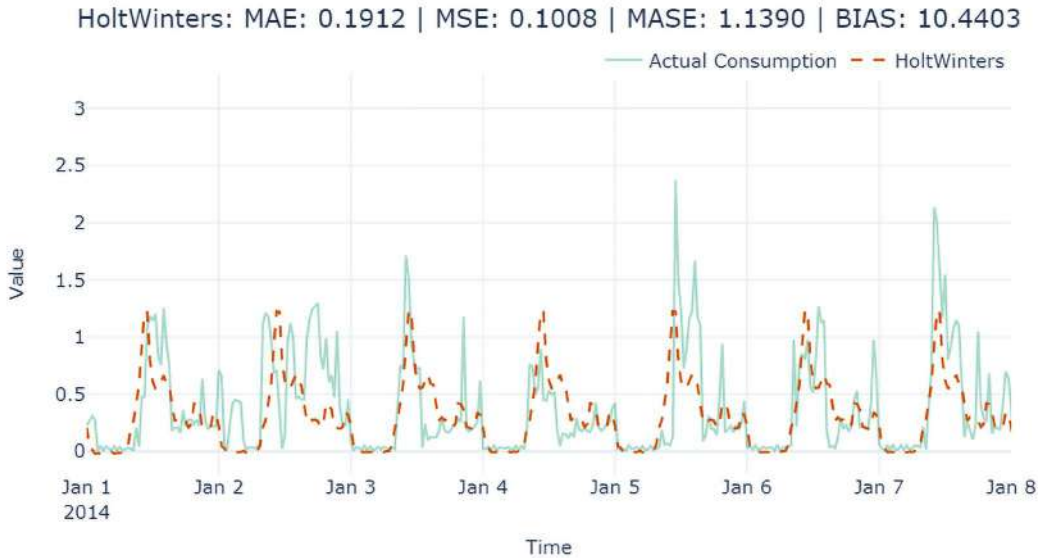


Figure 4.4: Exponential smoothing forecast

The forecast has captured the seasonality but has failed to capture the peaks. But we can see the improvement in MAE already.

Now, let's look at one of the most popular forecasting methods out there.

AutoRegressive Integrated Moving Average (ARIMA)

ARIMA models are the other class of methods that, like ETS, have stood the test of time and are one of the most popular classical methods of forecasting. The ETS family of methods is modeled around trend and seasonality, while ARIMA relies on **autocorrelation** (the correlation of y_t with y_{t-1} , y_{t-2} , and so on).

The simplest in the family are the AR (p) models, which use **linear regression** with p previous timesteps or, in other words, p lags. Mathematically, it can be written as follows:

$$y_t = c + \phi_1 y_{t-1} + \phi_2 y_{t-2} + \dots + \phi_p y_{t-p} + \epsilon_t$$

Here, c is the intercept, and ϵ_t is the noise or error at timestep t .

The next in the family are MA (q) models, in which, instead of past observed values, we use the past q errors in the forecast (which is assumed to be pure white noise) to come up with a forecast:

$$y_t = c + \theta_1 \epsilon_{t-1} + \theta_2 \epsilon_{t-2} + \dots + \theta_q \epsilon_{t-q}$$

Here, ϵ is white noise and c is the intercept.

This is not typically used on its own but in conjunction with AR (p) models, which makes the next one on our list ARMA (p, q) models. ARMA (AutoRegressive Moving Average) models are defined as $y_t = \text{AR}(p) + \text{MA}(q)$.

In all the ARIMA models, there is one underlying assumption—the *time series is stationary* (we talked about stationarity in *Chapter 1, Introducing Time Series*, and will elaborate on this in *Chapter 6, Feature Engineering for Time Series Forecasting*). There are many ways to make the series stationary but taking the difference of successive values is one such technique. This is known as **differencing**. Sometimes, we need to do differencing once, while other times, we have to perform successive differencing before the time series becomes stationary. The number of times we do the differencing operation is called the *order of differencing*. The I in ARIMA, and the final piece of the puzzle, stands for *Integrated*. It defines the order of differencing we need to do before the series becomes stationary and is denoted by d .

So, the complete ARIMA (p, d, q) model says that we do the d^{th} order of differencing and then consider the last p terms in an autoregressive manner, and then include the last q moving average terms to come up with the forecast.

The ARIMA models we have discussed so far only handle non-seasonal time series. However, using the same concepts we discussed, but on a seasonal cycle, we get **seasonal ARIMA**. p , d , and q are slightly tweaked so that they work on the seasonal period, m . To differentiate them from the normal p , d , and q , we call the seasonal values P , D , and Q . For instance, if p means taking the last p lags, P means taking the last P seasonal lags. If p_1 is y_{t-1} , P_1 would be y_{t-m} . Similarly, D means the order of seasonal differencing.

Picking the right p , d , and q and P , D , and Q values is not very intuitive, and we will have to resort to statistical tests to find them. However, this becomes a bit impractical when you are forecasting many time series. An automatic way of iterating through the different parameters and finding the best p , d , and q , and P , D , and Q values for the data is called **AutoARIMA**. In Python, NIXTLA has implemented this method, `AutoARIMA()`. NIXTLA also has a normal ARIMA implementation as well, which is much faster but requires p , d , and q to be entered manually.

Practical considerations:



Although ARIMA and AutoARIMA can give you good-performing models in many cases, they can be quite slow when you have long seasonal periods and a long time series. In our case, where we have almost 27K observations in the history, ARIMA becomes very slow and a memory hog. Even when subsetting the data, a single AutoARIMA fit takes around 60 minutes. Letting go of the seasonal parameters brings down the runtime drastically, but for a seasonal time series such as energy consumption, it doesn't make sense. AutoARIMA includes many such fits to identify the best parameters and, therefore, becomes impractical for long time series datasets. Almost all the implementations in the Python ecosystem suffer from this drawback. NIXTLA claims to have the fastest and most accurate version of AutoARIMA, faster than the original R method as well.

Let's see how we can apply ARIMA and AutoARIMA using NIXTLA:

```
from statsforecast.models import (ARIMA, AutoARIMA)

#ARIMA model by specifying parameters
arima_model = ARIMA(order = (2,1,1), seasonal_order = (1,1,1), season_length =
48)

#AutoARIMA model by specifying max limits for parameters and letting the
algorithm find the best ones
auto_arima_model = AutoARIMA( max_p = 2, max_d=1, max_q = 2, max_P=2, max_D =
1, max_Q = 2, stepwise = True, season_length=48)
```

For the entire list of parameters for AutoARIMA, head over to the NIXTLA documentation at <https://nixtlaverse.nixtla.io/statsforecast/docs/models/autoarima.html>.

Let's see what the ETS and ARIMA forecasts look like for the households we were experimenting with:

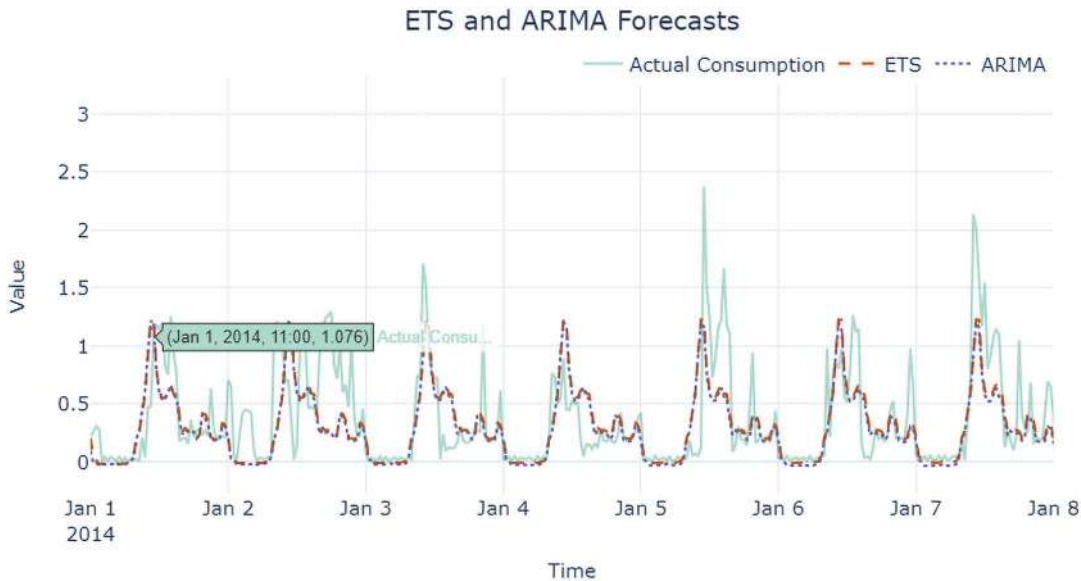


Figure 4.5: ETS and ARIMA forecasts

With NIXTLA, both ETS and ARIMA have done a good job of capturing both the seasonality and the peaks. The resulting MAE scores are also very similar, with 0.191 and 0.203, respectively. Now, let's look at another method—the Theta forecast.

Theta forecast

The **Theta forecast** was the top-performing submission in the M3 forecasting competition that was held in 2002. The method relies on a parameter, θ , that amplifies or smooths the local curvature of a time series, depending on the value chosen. Using θ , we smooth or amplify the original time series. These smoothed lines are called **Theta lines**. V. Assimakopoulos and K. Nikolopoulos proposed this method as a decomposition approach to forecasting. Although, in theory, any number of Theta lines can be used, the originally proposed method used two Theta lines, $\theta = 0$ and $\theta = 2$, and took an average of the forecast of the two Theta lines as the final forecast.



The M competitions are forecasting competitions organized by Spyros Makridakis, a leading forecasting researcher. They typically curate a dataset of time series, lay down the metrics with which the forecasts will be evaluated, and open these competitions to researchers all around the world to get the best forecast possible. These competitions are considered to be some of the biggest and most popular time series forecasting competitions in the world. At the time of writing, six such competitions have already been completed. To learn more about the latest competition, visit this website: <https://mofc.unic.ac.cy/the-m6-competition/>.

In 2002, Rob Hyndman et al. simplified the Theta method and showed that we can use ETS with a drift term to get equivalent results to the original Theta method, which is what is adapted into most of the implementations of the method that exist today. The main steps that are involved in the Theta forecast (which is implemented in NIXTLA) are as follows:

1. **Deseasonalization:** Apply a classical multiplicative decomposition to remove the seasonal component from the time series (if it exists). This focuses the analysis on the underlying trend and cyclical components. Deseasonalization is done using `statsmodels.tsa.seasonal.seasonal_decompose`. This step creates a new deseasonalized time series.
2. **Theta Coefficients Application:** Decompose the deseasonalized series into two “Theta” lines using coefficients θ_1 and θ_2 . These coefficients modify the second difference of the time series to either dampen ($\theta < 1$) or accentuate ($\theta > 1$) local fluctuations.
3. **Extrapolation of Theta Lines:** Treat each Theta line as a separate series and forecast them into the future. This is done using linear regression for the Theta line where $\theta_1 = 0$, producing a straight line, and simple exponential smoothing for the Theta line where $\theta_2 = 2$.
4. **Recomposition:** Combine the forecasts from the two Theta lines. The original method uses equal weighting for both lines, which integrates the long-term trend and short-term movements effectively.
5. **Reseasonalize:** If the data was deseasonalized in the beginning.

NIXTLA has very different variations of the Theta method. More information on the specifics of the NIXTLA implementation can be found here: <https://nixtlaverse.nixtla.io/statsforecast/docs/models/autotheta.html>.

Let's see how we can use it practically:

```
theta_model = Theta(season_length =48, decomposition_type = 'additive' )
```

The key parameters here are as follows: `season_length` and `decomposition_type`. These parameters are used for the initial seasonal decomposition. If left empty, the implementation automatically tests for seasonality and deseasonalizes the time series automatically using multiplicative decomposition. It is recommended to set these parameters with our domain knowledge if we know them. The decomposition type can be multiplicative (default) or additive.

Let's visualize the forecast we just generated using the Theta forecast:

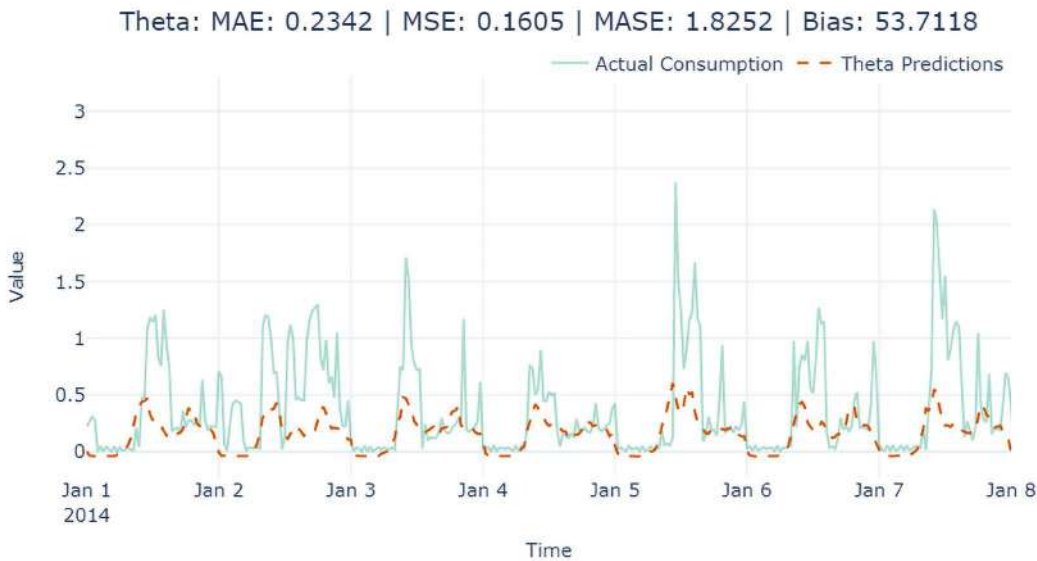


Figure 4.6: The Theta forecast



Reference check:

The research paper in which V. Assimakopoulos and K. Nikolopoulos proposed the Theta method is cited as reference 1 in the *References* section, while subsequent simplification by Rob Hyndman is cited as reference 2.

The seasonality pattern is captured, but it's not hitting the peaks. Let's look at another very strong method, TBATS.

TBATS

Sometimes, a time series has more than one seasonality pattern or a non-integer seasonal period, commonly referred to as complex seasonality. An example would be an hourly forecast that could have a daily seasonality for the time of day, a weekly seasonality for the day of the week, and a yearly seasonality for the day of the year. Additionally, most time series models are designed for smaller integer seasonal periods, such as monthly (12) or quarterly (4) data, but yearly seasonality can pose a problem since a year is 364.25 days. TBATS was meant to combat these many challenges that pose problems for many forecasting models. However, with any automated approach, at times it is susceptible to poor forecasts.

TBATS stands for:

- Trigonometric seasonality
- Box-Cox transformation
- ARMA errors
- Trend
- Seasonal components

This model was first introduced by Rob J. Hyndman, Alysha M. De Livera, and Ralph D. Snyder in 2011. There is also another variant of TBATS, referred to as BATS, which is without the trigonometric seasonality component. TBATS is from the state space model family. In state space forecasting models, the observed time series is assumed to be a combination of the underlying state variables and a measurement equation that relates the state variables to the observed data. The state variables capture the underlying patterns, trends, and relationships in the data.

BATS has parameters $(\omega, \phi, p, q, m_1, m_2, \dots, m_t)$ indicating the Box-Cox parameter, damping parameter, ARMA parameters (p, q) and the seasonal periods $(m_1, m_2, \dots, m_t)$. Due to its flexibility, the BATS model can be considered a family of models encompassing many other models we have seen earlier. For instance:

- $BATS(1, 1, 0, 0, m_1) = \text{Holt-Winters Additive Seasonality}$
- $BATS(1, 1, 0, 0, m_2) = \text{Holt-Winters Additive Double Seasonality}$

BATS has the flexibility for multiple seasonality; however, it is limited to only integer-based seasonal periods, and with multiple seasonalities, it can have a large number of states resulting in increasing model complexity. This is what TBATS was meant to address.

For reference, the TBATS parameter space is:

$$TBAT(\omega, \phi, p, q, \{m_1, k_1\}, \dots, \{m_T, k_T\})$$

The main advantages of TBATS are as follows:

- Works with single, complex, and non-integer seasonality (trigonometric seasonality)
- Handles nonlinear patterns common in real-world time series (Box-Cox transformation)
- Handles autocorrelation in the residuals (Autoregressive moving average errors)

To better understand the inner workings of TBATS, let's break down each step.

The order in which operations are done (unlike the order in the acronym) using TBATS is:

1. Box-Cox transformation
2. Exponentially smoothed trend
3. Seasonal decomposition using Fourier series (trigonometric seasonality)
4. AutoRegressive Moving Average (ARMA)
5. Parameter estimation through a likelihood-based approach

Box-Cox transformation

Box-Cox is a transformation in the family of power transformations.

In time series, making data stationary is an important step before forecasting (as discussed in *Chapter 1*). Stationarity ensures that our data does not statistically change over time, and thus more accurately resembles a probability distribution. There are several possible transformations that could be applied. More details on various target transformations, including Box-Cox, can be found in *Chapter 7*.

As a preview, here is a sample output from a Box-Cox transformation. After the transformation, our data more closely resembles that of a normal distribution. Box-Cox transformations can only be used with positive data, but in practice, this is often the case.

Figure 4.7 shows an example of how a time series might look before and after a Box-Cox transformation.

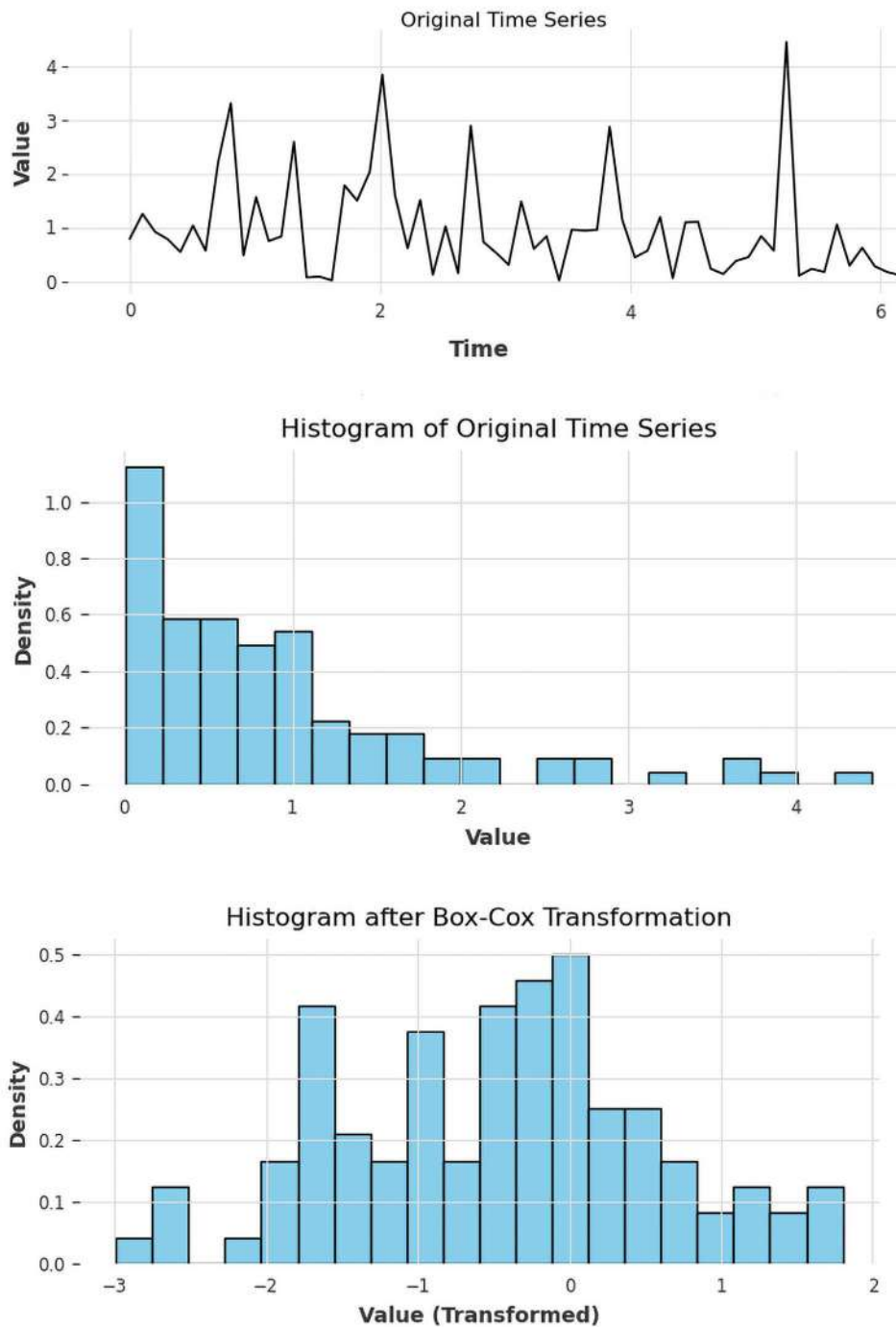


Figure 4.7: Box-Cox transformation

Exponentially smoothed trend

Using Locally Estimated Scatterplot Smoothing (LOESS), a smoothed trend is extracted from the time series:

$$l_t = l_{t-1} + \phi b_{t-1} + \alpha d_t \text{ (Local)}$$

$$b_t = (1 - \phi)b + \phi b_{t-1} + \beta d_t \text{ (Global)}$$

LOESS works by applying a locally weighted, low-degree polynomial regression over the data points to create a smooth, flowing line through them. This technique is highly effective in capturing local trend variations without assuming a global form for the data, which makes it particularly useful for data with varying trends or seasonal variations. This is the same LOESS that we used to decompose a time series into a trend back in *Chapter 3*.

Seasonal decomposition using Fourier series (trigonometric seasonality)

The remaining residuals are then modeled using Fourier terms (discussed in *Chapter 3*) to decompose the seasonality component.

$$y_t^{(\omega)} = l_{t-1} + \phi b_{t-1} + \sum_{i=1}^M S_{t-m_i}^{(i)} + d_t$$

$$S_t^{(i)} = \sum_{j=1}^{k_t} S_{j,t}^{(i)}$$

$$S_{j,t}^{(i)} = S_{j,t-1}^{(i)} \cos \lambda_j^{(i)} + S_{j,t-1}^{(i)} \sin \lambda_j^{(i)} + \gamma_1^{(i)} d_t$$

$$S_{j,t}^{(i)} = -S_{j,t-1}^{(i)} \sin \lambda_j^{(i)} + S_{j,t-1}^{(i)} \cos \lambda_j^{(i)} + \gamma_2^{(i)} d_t$$

The main advantage of using Fourier to model seasonality is its ability to model multiple seasonalities, as well as non-integer seasonality, such as yearly seasonality with daily data since there are 364.25 days in a year. Most other decomposition methods cannot handle the non-integer period and have to resort to rounding to 365, which can fail to identify the true seasonality. An example of what a decomposed time series would look like using Fourier is below. The observed time series in this example is hourly data. Therefore, our seasonal periods are:

$$\text{daily} = 24$$

$$\text{weekly} = 24 * 7 = 168$$

Here, you can clearly see the defined seasonal patterns, the trend, and the remaining residuals. *Figure 4.8* shows the decomposition of the trend and seasonality, after which the residuals are modeled using an ARMA process.

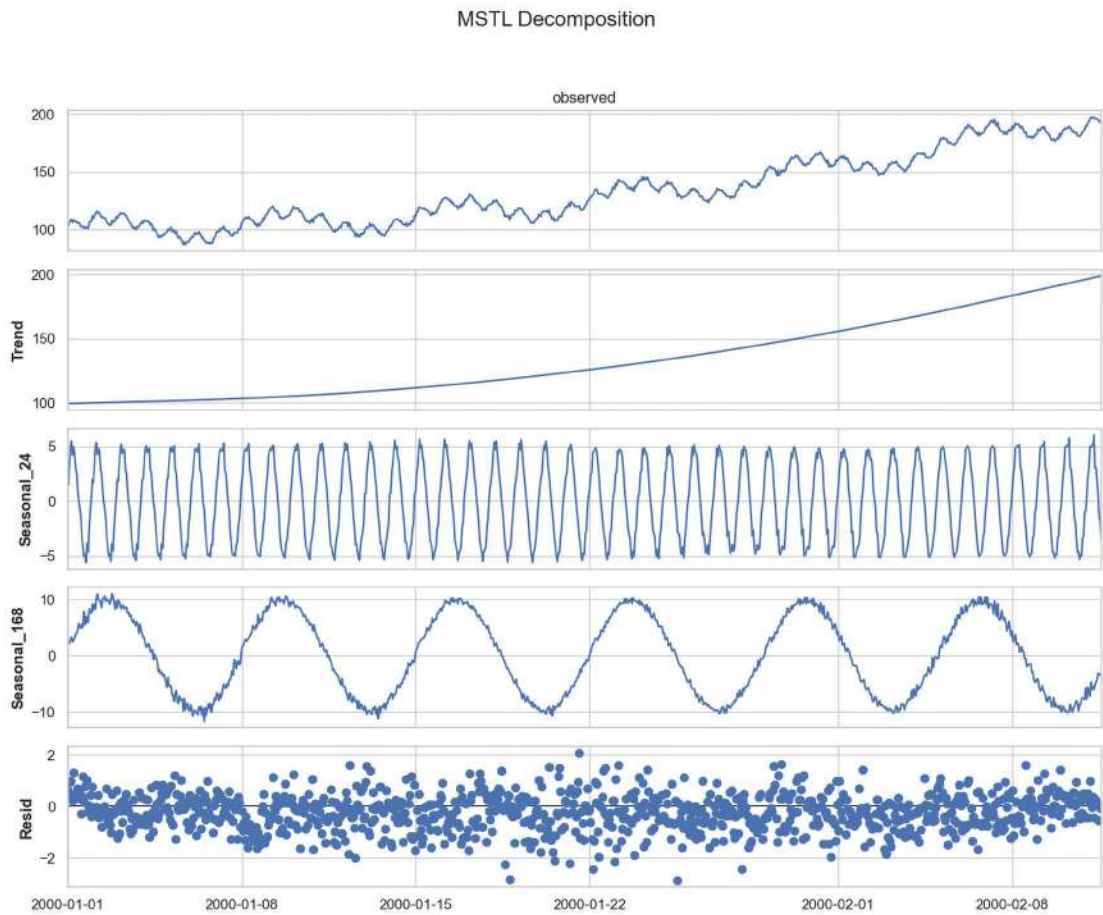


Figure 4.8: Decomposed time series

ARMA

ARMA was discussed earlier as a subset of the ARIMA family:

$$d_t = \sum_{i=1}^p \phi_i d_{t-i} + \sum_{j=1}^q \theta_j \varepsilon_{t-j} + \varepsilon_t$$

The ARMA model in TBATS is used to model the remaining residuals to capture any autocorrelations of the lagged variables. The **autoregressive (AR)** component captures the correlation between an observation and several lagged observations. This deals with the momentum or continuation of the series. The **moving average (MA)** component models the error terms as a linear combination of errors at previous time periods, capturing information not explained by the AR part alone.

Parameter optimization

To select the optimal parameter space, TBATS will fit several models and automatically select the best parameters. A few of the models TBATS fits internally are:

- With and without Box-Cox transformation
- With and without trend
- With and without trend damping
- Season and non-seasonal model
- ARMA (p, q) parameters

The final model is chosen by which combination of parameters minimizes the **Akaike Information Criterion (AIC)**, and AutoARIMA is used to determine the ARMA parameters.

As with all forecasting methods, there are benefits and trade-offs to different models. While TBATS offers some enhancements on many other models' shortcomings, the trade-off is the need to build many models, which results in longer computation times. This can pose a problem if you have to model multiple time series. Additionally, TBATS does not allow for the inclusion of exogenous variables.



Practitioner's note:

TBATS cannot handle exogenous regression since it is related to ETS models, as per Hyndman himself, who suggests it is unlikely to include covariates (Hyndman, 2014; Reference 7). If external regressors are to be used, other methods such as ARIMAX or SARIMAX should be used. If the time series has complex seasonality, you can add Fourier features as covariates to your ARIMAX or SARIMAX model to help capture the seasonal patterns.

This is implemented in NIXLA, and we can use the implementation shown here:

```
TBATS_model = TBATS(seasonal_periods = 48, use_trend=True, use_damped_trend=True)
```

In NIXTLA, you can also use AutoTBATS to let the system optimize how to handle the various parameters.

Let's see what the TBATS forecast looks like:

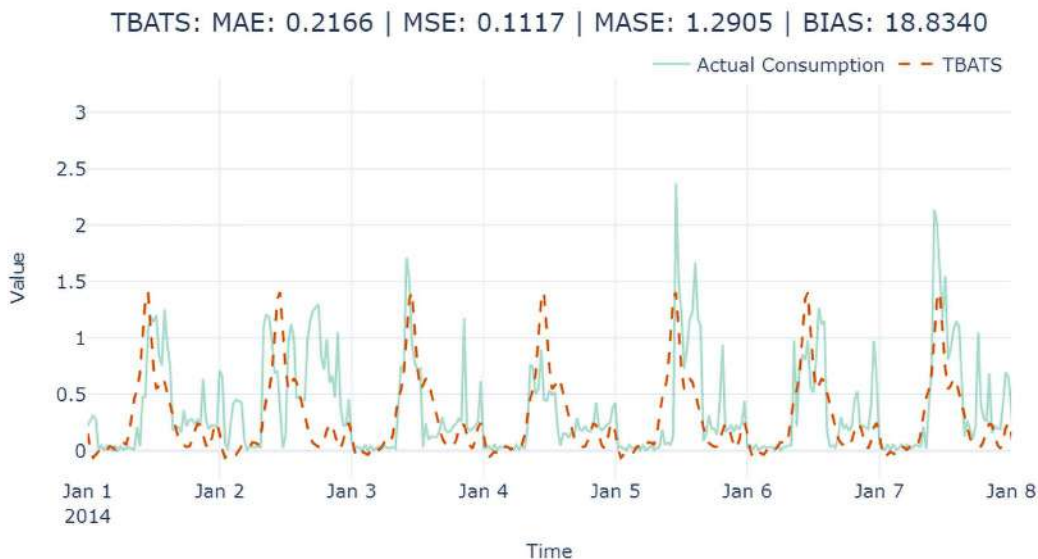


Figure 4.9: TBATS forecast

Again, the seasonality pattern has been replicated and is capturing most of the peaks in the forecast. Now let's take a look at another method that is well suited for highly seasonal time series (even if it has multiple seasonalities like our case).

Multiple Seasonal-Trend decomposition using LOESS (MSTL)

Remember the time series decomposition we did back in *Chapter 3*? What if we can use the same techniques to forecast? That's exactly what MSTL does. Let's look at the components of a time series again:

- Trend
- Cyclical
- Seasonality
- Irregular

Trend and cyclical components can be extracted using LOESS regression. If we fit a simple model on the trend values, we can use it to extrapolate to the future. And the seasonality component can easily be extrapolated because it is supposed to be a repeating pattern. Combining these, we get a forecasting model that works pretty well.

The MSTL method in NIXTLA applies the LOESS technique to decompose a time series into its various seasonal components. Following this decomposition, it employs a specialized non-seasonal model to forecast the trend, and a Seasonal Naive model to predict each of the seasonal components. This approach allows for the detailed analysis and forecasting of time series with complex seasonal patterns:

```
MSTL_model = MSTL(season_length = 48)
```

Let's see what the MSTL forecast looks like:

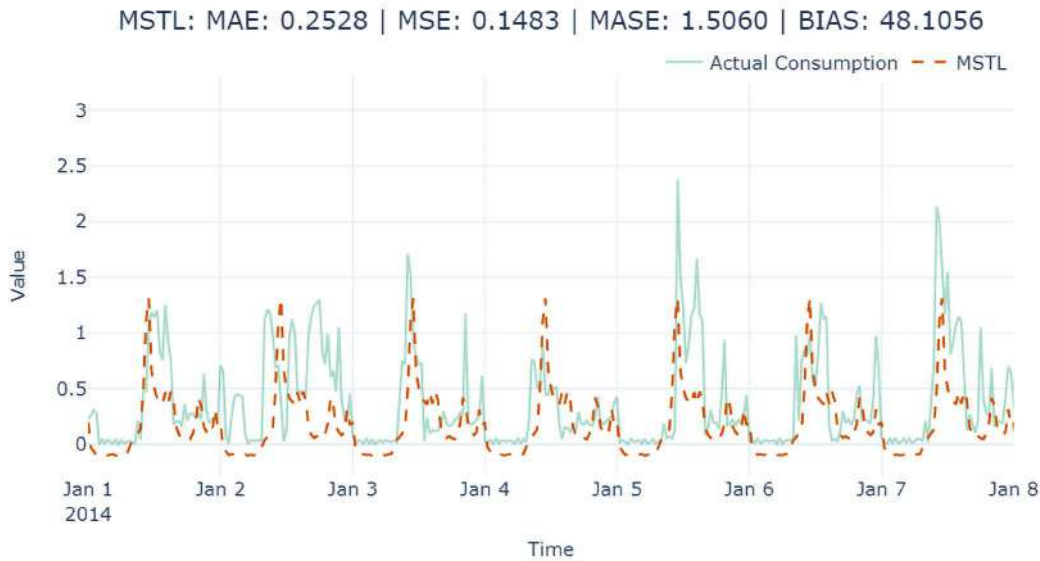


Figure 4.10: MSTL forecast

Let's also take a look at how the different metrics that we chose did for each of these forecasts for the household we were experimenting with (from the notebook):

	mase	mae	mse	rmse	smape	forecast_bias	LCLid	Time Elapsed	Model
0	1.820	0.305	0.249	0.499	113.588	74.34%	MAC000193	0.123053	Naive
1	1.501	0.252	0.191	0.437	78.739	13.74%	MAC000193	0.106017	SeasonalNaive
2	1.857	0.312	0.183	0.428	100.698	11.51%	MAC000193	0.293633	WindowAverage
3	1.139	0.191	0.101	0.318	88.890	10.44%	MAC000193	43.050361	HoltWinters
4	1.139	0.191	0.101	0.318	88.890	10.44%	MAC000193	22.924915	AutoETS
5	1.213	0.204	0.106	0.326	98.717	25.34%	MAC000193	29.812297	ARIMA
6	1.428	0.240	0.168	0.410	104.389	56.92%	MAC000193	109.861857	Theta
7	1.290	0.217	0.112	0.334	94.635	18.83%	MAC000193	11.040304	TBATS
8	1.506	0.253	0.148	0.385	117.314	48.11%	MAC000193	11.896484	MSTL

Figure 4.11: Summary of all the baseline algorithms

Out of all the baseline algorithms we tried, AutoETS is performing the best on MAE as well as MSE. ARIMA was the second-best model followed by TBATS. However, if you look at the **Time Elapsed** column, TBATS stands out taking just 7.4 seconds vs. 19 seconds for ARIMA. Since they had similar performance, we will choose TBATS over ARIMA, along with AutoETS as our baseline, and run them on all 399 households in the dataset (both validation and test) we’ve chosen (the code for this is available in the 02-Baseline_Forecasts_using_NIXTLA.ipynb notebook).

Evaluating the baseline forecasts

Since we have the baseline forecasts generated from ETS as well as TBATS, we should also evaluate these forecasts. The aggregate metrics for all the selected households for both these methods are as follows:

Validation	Algorithm	MAE	MSE	meanMASE	Forecast Bias
	0 AutoETS	0.120	0.058	1.028	4.27%
	1 TBATS	0.156	0.089	1.385	6.98%
Test	Algorithm	MAE	MSE	meanMASE	Forecast Bias
	0 AutoETS	0.119	0.060	0.997	-9.88%
	1 TBATS	0.143	0.096	1.175	-11.64%

Figure 4.12: The aggregate metrics of all the selected households (both validation and test)

It looks like AutoETS is performing much better in all three metrics. We also have these metrics calculated at a household level. Let’s look at the distribution of these metrics in the validation dataset for all the selected households:

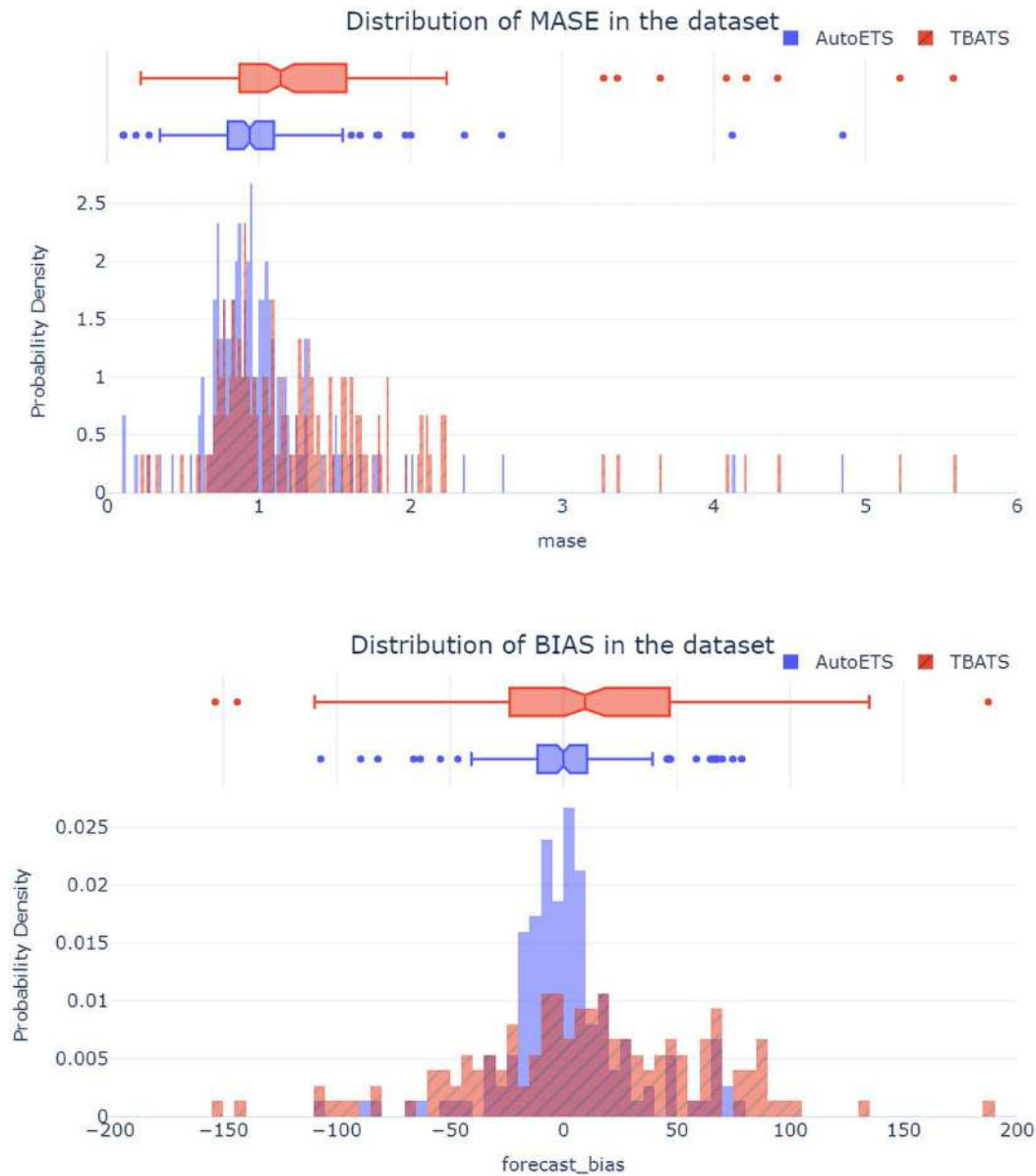


Figure 4.13: The distribution of MASE and forecast bias of the baseline forecast in the validation dataset

The MASE histogram of ETS seems to have a smaller spread than TBATS. ETS also has a lower median MASE than TBATS. We can see a similar pattern for forecast bias as well, with the forecast bias of ETS centered around zero and much less spread.

Back in *Chapter 1, Introducing Time Series*, we saw why every time series is not equally predictable and saw three factors to help us think about the issue—understanding the **Data Generating Process (DGP)**, the amount of data, and adequately repeating the pattern. In most cases, the first two are pretty easy to evaluate, but the third one requires some analysis. Although the performance of baseline methods gives us some idea about how predictable any time series is, they still are model-dependent. So, instead of measuring how well a time series is forecastable, we might be better measuring how well the chosen model can approximate the time series. This is where a few more fundamental techniques (relying on the statistical properties of a time series) come in.

Assessing the forecastability of a time series

Although there are many statistical measures that we can use to assess the predictability of a time series, we will just look at a few that are easier to understand and practical when dealing with large time series datasets. The associated notebook (`02-Forecastability.ipynb`) contains the code to follow along.

Coefficient of variation

The **Coefficient of Variation (CoV)** relies on the fact that the more variability that you find in a time series, the harder it is to predict it. And how do we measure variability in a random variable? **Standard deviation**.

In many real-world time series, the variation we see in the time series is dependent on the scale of the time series. Let's imagine that there are two retail products, *A* and *B*. *A* has a mean monthly sale of 15, while *B* has 50. If we look at a few real-world examples like this, we will see that if *A* and *B* have the same standard deviation, *B*, which has a higher mean, is much more forecastable than *A*. To accommodate this phenomenon and to make sure we bring all the time series in a dataset to a common scale, we can use the CoV:

$$CoV_n = \frac{\sigma_n}{\mu_n}$$

Here, σ_n is the standard deviation, and μ_n is the mean of the time series, *n*.

The CoV is the relative dispersion of data points around the mean, which is much better than looking at the pure standard deviation.

The larger the value for the CoV, the worse the predictability of the time series. There is no hard cutoff, but a value of 0.49 is considered a rule of thumb to separate time series that are relatively easier to forecast from the hard ones. Depending on the general *hardness* of the dataset, we can tweak this cutoff. Something I have found useful is to plot a histogram of CoV values in a dataset and derive cutoffs based on that.

Even though the CoV is widely used in the industry, it suffers from a few key issues:

- It doesn't consider seasonality. A sine or cosine wave will have a higher CoV than a horizontal line, but we know both are equally predictable.
- It doesn't consider the trend. A linear trend will make a series have a higher CoV, but we know it is equally predictable, like a horizontal line.

- It doesn't handle negative values in the time series. If you have negative values, it makes the mean smaller, thereby inflating the CoV.

To overcome these shortcomings, we propose another derived measure.

Residual variability

The thought behind **residual variability** (RV) is to try and measure the same kind of variability that we were trying to capture with the CoV but without the shortcomings. I was brainstorming on ways to avoid the problems of using the CoV, typically the seasonality issue, and was applying the CoV to the residuals after seasonal decomposition. It was then I realized that the residuals would have a few negative values and that the CoV wouldn't work well. Stefan de Kok, who is a thought leader in demand forecasting and probabilistic forecasting, suggested using the mean of the original actuals, which worked.

To calculate RV, you must perform the following steps:

1. Perform seasonal decomposition.
2. Calculate the standard deviation of the residuals or the irregular component.
3. Divide the standard deviation by the mean of the original observed values (before decomposition).

Mathematically, it can be represented as:

$$RV_n = \frac{\sigma_n^{res}}{\mu_n^y}$$

where, σ_n^{res} is the standard deviation of the residuals after decomposition and μ_n^y is the mean of the original observed values.

The key assumption here is that seasonality and trend are components that can be predicted. Therefore, our assessment of the predictability of a time series should only look at the variability of the residuals. However, we cannot use CoV on the residuals because the residuals can have negative and positive values, so the mean of the residuals loses the interpretation of the level of the series and tends to zero. When residuals tend to zero, the CoV measure tends to infinity because of the division by mean. Therefore, we use the mean of the original series as the scaling factor.

Let's see how we can calculate RV for all the time series in our dataset (which are in a compact form):

```
block_df["rv"] = block_df.progress_apply(lambda x: calc_norm_
sd(x['residuals'],x['energy_consumption']), axis=1)
```

In this section, we looked at two measures that are based on the standard deviation of the time series. Now, let's look at assessing the forecastability of a time series.

Entropy-based measures

Entropy is a ubiquitous term in science. We see it popping up in physics, quantum mechanics, social sciences, and information theory. And everywhere, it is used to talk about a measure of chaos or lack of predictability in a system. The entropy we are most interested in now is the one from information theory. Information theory involves quantifying, storing, and communicating digital information.

Claude E. Shannon presented the qualitative and quantitative model of communication as a statistical process in his seminal paper *A Mathematical Theory of Communication*. While the paper introduced a lot of ideas, some of the concepts that are relevant to us are information entropy and the concept of a *bit*—a fundamental unit of measurement of information.



Reference check:

A Mathematical Theory of Communication by Claude E. Shannon is cited as reference 3 in the *References* section.

The theory in itself is quite a lot to cover, but to summarize the key bits of information, take a look at the following short glossary:

- Information is nothing but a sequence of *symbols*, which can be transmitted from the *receiver* to the *sender* through a medium, which is called a *channel*. For instance, when we are texting somebody, the sequence of symbols is the letters/words of the language in which we are texting; the channel is the electronic medium.
- *Entropy* can be thought of as the amount of *uncertainty* or *surprise* in a sequence of symbols given some distribution of the symbols.
- A *bit*, as we mentioned earlier, is a unit of information and is a binary digit. It can either be 0 or 1.

Now, if we were to transfer one bit of information, it would reduce the uncertainty of the receiver by two. To understand this better, let's consider a coin toss. We toss the coin in the air, and as it is spinning through the air, we don't know whether it is going to be heads or tails. But we do know it is going to be one of these two. When the coin hits the ground and finally comes to rest, we find that it is heads. We can represent whether the coin toss is heads or tails with one bit of information (0 for heads and 1 for tails). So, the information that was passed to us when the coin fell reduced the possible outcomes from two to one (heads). This transfer was possible with one bit of information.

In information theory, the entropy of a discrete random variable is the average level of *information*, *surprise*, or *uncertainty* inherent in the variable's possible outcomes. In more technical parlance, it is the expected number of bits required for the best possible encoding scheme of the information present in the random variable.



Additional reading:

If you want to intuitively understand entropy, cross-entropy, Kullback-Leibler divergence, and so on, head over to the *Further reading* section. There are a couple of links to blogs (one of which is my own) where we try to lay down the intuition behind these metrics.

Entropy is formally defined as follows:

$$H(X) = -\sum_{i=1}^n P(x_i) \cdot \log P(x_i)$$

Here, X is the discrete random variable with possible outcomes, $x_1, x_2, \dots, x_n$. Each of those outcomes has a probability of occurring, which is denoted by $P(x_1), P(x_2), \dots, P(x_n)$.

To develop some intuition around this, we can think that the more spread out a probability distribution is, the more chaos is in the distribution, and thus more entropy. Let's quickly check this with some code:

```
# Creating an array with a well balanced probability distribution
flat = np.array([0.1,0.2, 0.3,0.2, 0.2])
# Calculating Entropy
print((-np.log2(flat)* flat).sum())
```

```
>> 2.2464393446710154
```

```
# Creating an array with a peak in probability
sharp = np.array([0.1,0.6, 0.1,0.1, 0.1])
# Calculating Entropy
print((-np.log2(sharp)* sharp).sum())
```

```
>> 1.7709505944546688
```

Here, we can see that the probability distribution that spreads its mass has higher entropy.

In the context of a time series, n is the total number of time series observations, and $P(x_i)$ is the probability for each symbol of the time series alphabet. A sharp distribution means that the time series values are concentrated on a small area and should be easier to predict. On the other hand, a wide or flat distribution means that the time series value can be equally likely across a wider range of values and hence is difficult to predict.

If we have two time series—one containing the result of a coin toss and the other containing the result of a dice throw—the dice throw would have any output between one and six, whereas the coin toss would be either zero or one. The coin toss time series would have lower entropy and be easier to predict than the dice throw time series.

However, since time series is typically continuous, and entropy requires a discrete random variable, we can resort to a few strategies to convert the continuous time series into a discrete one. Many strategies, such as quantization or binning, can be applied, which leads to a myriad of complexity measures. Let's review one such measure that is useful and practical.

Spectral entropy

To calculate the entropy of a time series, we need to discretize the time series. One way to do that is by using **Fast Fourier Transform (FFT)** and **power spectral density (PSD)**. This discretization of the continuous time series is used to calculate spectral entropy.

We learned what Fourier Transform is earlier in this chapter and used it to generate a baseline forecast. But using FFT, we can also estimate a quantity called power spectral density. This answers the question, *How much of the signal is at a particular frequency?* There are many ways of estimating power spectral density from a time series, but one of the easiest ways is by using the **Welch method**, which is a non-parametric method based on Discrete Fourier Transform. This is also implemented as a handy function with the `periodogram(x)` signature in `scipy`.

The returned *PSD* will have a length equal to the number of frequencies estimated, but these are densities and not well-defined probabilities. So, we need to normalize *PSD* to be between zero and one:

$$nPSD_i = \frac{PSD_i}{\sum_{j=1}^F PSD_j}$$

Here, F is the number of frequencies that are part of the returned power spectrum density.

Now that we have the probabilities, we can just plug this into the entropy formula and arrive at the spectral entropy:

$$H_{s(X)} = -\sum_{i=1}^n nPSD_i \cdot \log(nPSD_i)$$

When we introduced **entropy-based measures**, we saw that the more spread out the probability mass of a distribution is, the higher the entropy is. In this context, the more frequencies across which the spectral density is spread, the higher the spectral entropy. So, a higher spectral entropy means the time series is more complex and, therefore, more difficult to forecast.

Since FFT has an assumption of stationarity, it is recommended that we make the series stationary before using spectral entropy as a metric. We can even apply this metric to a detrended and deseasonalized time series, which we can refer to as **residual spectral entropy**. This book's GitHub repository contains an implementation of spectral entropy under `src.forecastability.entropy.spectral_entropy`. This implementation also has a parameter, `transform_stationary`, which, if set to `True`, will detrend the series before we apply spectral entropy. Let's see how we can calculate spectral entropy for our dataset:

```
from src.forecastability.entropy import spectral_entropy
block_df["spectral_entropy"] = block_df.energy_consumption.progress_
    apply(lambda x: spectral_entropy(x, transform_stationary=True))
block_df["residual_spectral_entropy"] = block_df.residuals.progress_
    apply(spectral_entropy)
```

There are other entropy-based measures such as approximate entropy and sample entropy, but we will not cover them in this book. They are more computationally intensive and don't tend to work for time series that contain fewer than 200 values. If you are interested in learning more about these measures, head over to the *Further reading* section.

Another metric that takes a slightly different path is the Kaboudan metric.

Kaboudan metric

In 1999, Kaboudan defined a metric for time series predictability, calling it the η -metric. The idea behind it is very simple. If we block-shuffle a time series, we are essentially destroying the information in the time series. **Block shuffling** is the process of dividing the time series into blocks and then shuffling those blocks. So, if we calculate the **sum of squared errors (SSE)** of a forecast that's been trained on a time series and then contrast it with the SSE of a forecast trained on a shuffled time series, we can infer the predictability of the time series. The formula to calculate this is as follows:

$$\eta = 1 - \frac{SSE_Y}{SSE_S}$$

Here, SSE_Y is the SSE of the forecast that was generated from the original time series, while SSE_S is the SSE of the forecast that was generated from the block-shuffled series.

If the time series contains some predictable signals, SSE_Y would be lower than SSE_S and η would approach one. This is because there was some information or patterns that were broken due to the block shuffling. On the other hand, if a series is just white noise (which is unpredictable by definition), there would be hardly any difference between SSE_Y and SSE_S , and η would approach zero.

In 2002, Duan investigated this metric and suggested some modifications in his thesis. One of the problems he identified, especially in long time series, is that the η values are found in a narrow band around 1 and suggested a slight modification to the formula. We call this the **modified Kaboudan metric**. The measure on the lower side is also clipped to zero. Sometimes, the metric can go below zero because SSE_S is lower than SSE_Y , which is because the series is unpredictable and, by pure chance, block shuffling made the SSE lower:

$$\eta_{modified} = 1 - \sqrt{\frac{SSE_Y}{SSE_S}}$$



Reference check:

The research paper that proposed the Kaboudan metric is cited as reference 4 in the *References* section. The subsequent modification that Duan suggested is cited as reference 5.

This modified version, as well as the original, has been implemented in this book's GitHub repository.

There is no restriction on the forecasting model you use to generate the forecast, which makes it a bit more flexible. Ideally, we can choose one of the classical statistical methods that is fast enough to be applied to the whole dataset. But this also makes the Kaboudan metric dependent on the model, and the limitations of the model are inherent in the metric. The metric measures a combination of how difficult a series is to forecast and how difficult it is for the model to forecast the series.

Again, both metrics are implemented in this book's GitHub repository. Let's see how we can use them:

```
from src.forecastability.kaboudan import kaboudan_metric, modified_kaboudan_metric
```

```

model = Theta(theta=3, seasonality_period=48*7, season_mode=SeasonalityMode.
ADDITIVE)
block_df["kaboudan_metric"] = [kaboudan_metric(r[0], model=model, block_size=5,
backtesting_start=0.5, n_folds=1) for r in tqdm(zip(*block_df[["energy_
consumption"]].to_dict("list").values()), total=len(block_df))]
block_df["modified_kaboudan_metric"] = [modified_kaboudan_metric(r[0],
model=model, block_size=5, backtesting_start=0.5, n_folds=1) for r in
tqdm(zip(*block_df[["energy_consumption"]].to_dict("list").values()),
total=len(block_df))]

```

Although there are many more metrics we can use for this purpose, the metrics we just reviewed for assessing forecastability cover a lot of the popular use cases and should be more than enough to gauge any time series dataset in regards to the difficulty of forecasting it. We can use these metrics to compare one time series with another time series or to profile a whole set of related time series in a dataset with another dataset for benchmarking purposes.

Additional reading:



If you want to delve a little deeper and analyze the behavior of these metrics, how similar they are to each other, and how effective they are in measuring forecastability, go to the end of the 03-Forecastability.ipynb notebook. We compute rank correlations among these metrics to understand how similar these metrics are. We can also find rank correlations with the computed metrics from the best-performing baseline method to understand how well these metrics did in estimating the forecastability of a time series. I strongly encourage you to play around with the notebook and understand the differences between the different metrics. Pick a few time series and check how the different metrics give you slightly different interpretations.

Congratulations on generating your baseline forecasts—the first set of forecasts we have generated using this book! Feel free to head over to the notebooks, play around with the parameters of the methods, and see how forecasts change. It'll help you develop an intuition around what the baseline methods are doing. If you are interested in learning more about how to make these baseline methods better, head over to the *Further reading* section, where we have provided a link to the paper *The Wisdom of the Data: Getting the Most Out of Univariate Time Series Forecasting*, by F. Petropoulos and E. Spiliotis.

Summary

And with this, we have come to the end of *Part 1, Getting Familiar with Time Series*. We have come a long way from just understanding what a time series is to generating competitive baseline forecasts. Along the way, we learned how to handle missing values and outliers and how to manipulate time series data using pandas. We used all those skills on a real-world dataset regarding energy consumption. We also looked at ways to visualize and decompose time series. In this chapter, we set up a test harness, learned how to use the NIXTLA library to generate a baseline forecast, and looked at a few metrics that can be used to understand the forecastability of a time series.

For some of you, this may be a refresher, and we hope this chapter added some value in terms of some subtleties and practical considerations. For the rest of you, we hope you are in a good place foundationally to start venturing into modern techniques using machine learning in the next part of the book.

In the next chapter, we will discuss the basics of machine learning and delve into time series forecasting.

References

The following references were provided in this chapter:

1. Assimakopoulos, Vassilis and Nikolopoulos, K. (2000). *The theta model: A decomposition approach to forecasting*. International Journal of Forecasting. 16. 521-530. https://www.researchgate.net/publication/223049702_The_theta_model_A_decomposition_approach_to_forecasting.
2. Rob J. Hyndman, Baki Billah. (2003). *Unmasking the Theta method*. International Journal of Forecasting. 19. 287-290. <https://robjhyndman.com/papers/Theta.pdf>.
3. Shannon, C.E. (1948), *A Mathematical Theory of Communication*. Bell System Technical Journal, 27: 379-423. <https://people.math.harvard.edu/~ctm/home/text/others/shannon/entropy/entropy.pdf>.
4. Kaboudan, M. (1999). *A measure of time series' predictability using genetic programming applied to stock returns*. Journal of Forecasting, 18, 345-357: http://www.aiecon.org/conference/efmaci2004/pdf/GP_Basics_paper.pdf.
5. Duan, M. (2002). *TIME SERIES PREDICTABILITY*: <https://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.68.1898&rep=rep1&type=pdf>.
6. De Livera, A. M., & Hyndman, R. J. (2009). Forecasting time series with complex seasonal patterns using exponential smoothing (Department of Econometrics and Business Statistics Working Paper Series 15/09)
7. Hyndman, Rob. "Rob J Hyndman - TBATS with Regressors." Rob J Hyndman, 6 Oct. 2014, <http://robjhyndman.com/hyndsight/tbats-with-regressors>

Further reading

To learn more about the topics that were covered in this chapter, take a look at the following resources:

- *Information Theory and Entropy*, by Manu Joseph: <https://deep-and-shallow.com/2020/01/09/deep-learning-and-information-theory/>.
- *Visual Information*, by Chris Olah: <https://colah.github.io/posts/2015-09-Visual-Information>.
- Fourier Transform: <https://betterexplained.com/articles/an-interactive-guide-to-the-fourier-transform/>.
- *Fourier Transform* by 3blue1brown—a visual introduction: <https://www.youtube.com/watch?v=spUNpyF58BY&v1=en>.
- *Understanding Fourier Transform by Example*, by Richie Vink: <https://www.ritchievink.com/blog/2017/04/23/understanding-the-fourier-transform-by-example/>.

- Delgado-Bonal A, Marshak A. *Approximate Entropy and Sample Entropy: A Comprehensive Tutorial*. Entropy. 2019; 21(6):541. <https://www.mdpi.com/1099-4300/21/6/541>.
- Yentes, J.M., Hunt, N., Schmid, K.K. et al. *The Appropriate Use of Approximate Entropy and Sample Entropy with Short Data Sets*. Ann Biomed Eng 41, 349–365 (2013): <https://doi.org/10.1007/s10439-012-0668-3>
- Ponce-Flores M, Frausto-Solís J, Santamaría-Bonfil G, Pérez-Ortega J, González-Barbosa JJ. *Time Series Complexities and Their Relationship to Forecasting Performance*. Entropy. 2020; 22(1):89. <https://www.mdpi.com/1099-4300/22/1/89>
- Petropoulos F, Spiliotis E. *The Wisdom of the Data: Getting the Most Out of Univariate Time Series Forecasting*. Forecasting. 2021; 3(3):478-497. <https://doi.org/10.3390/forecast3030029>

**Unlock this book's exclusive
benefits now**

This book comes with additional benefits
designed to elevate your learning experience.



Note: Have your purchase invoice ready before you begin. <https://www.packtpub.com/unlock/9781835883181>

Part 2

Machine Learning for Time Series

In this part, we will be looking at ways of applying modern machine learning techniques for time series forecasting. This part also covers powerful forecast combination methods and the exciting new paradigm of global models. By the end of this part, you will be able to set up a modeling pipeline using modern machine learning techniques for time series forecasting.

This part comprises the following chapters:

- *Chapter 5, Time Series Forecasting as Regression*
- *Chapter 6, Feature Engineering for Time Series Forecasting*
- *Chapter 7, Target Transformations for Time Series Forecasting*
- *Chapter 8, Forecasting Time Series with Machine Learning Models*
- *Chapter 9, Ensembling and Stacking*
- *Chapter 10, Global Forecasting Models*

5

Time Series Forecasting as Regression

In the previous part of the book, we developed a fundamental understanding of time series and equipped ourselves with tools and techniques to analyze and visualize time series and even generate our first baseline forecasts. We have mainly covered classical and statistical techniques in this book so far. Let's now dip our toes into modern **machine learning** and learn how we can leverage this comparatively new field for **time series forecasting**. Machine learning is a field that has grown in leaps and bounds in recent times, and being able to leverage these new techniques for time series forecasting is a skill that will be invaluable in today's world.

In this chapter, we will be covering these main topics:

- Understanding the basics of machine learning
- Time series forecasting as regression
- Local versus global models

Understanding the basics of machine learning

We want to use machine learning for time series forecasting. But before we get started with it, let's spend some time establishing what machine learning is and setting up a framework to demonstrate what it does (if you are already very comfortable with machine learning, feel free to skip ahead to the next section, *Time series forecasting as regression*, or just stay with us and refresh the concepts). In 1959, Arthur Samuel defined machine learning as a “*field of study that gives computers the ability to learn without being explicitly programmed.*” Traditionally, programming has been a paradigm under which we know a set of rules/logic to perform an action, and that action is performed on the given data to get the output that we want. But machine learning flipped this on its head.

In machine learning, we start with data and the output, and we ask the computer to tell us about the rules with which the desired output can be achieved from the data:

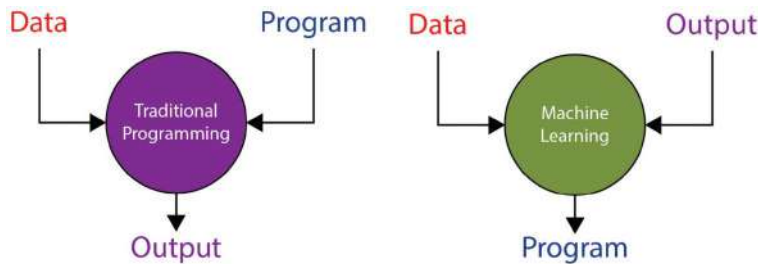


Figure 5.1: Traditional programming versus machine learning

There are many kinds of problem settings in machine learning, such as supervised learning, unsupervised learning, self-supervised learning, and so on, but we will stick to supervised learning, which is the most common one and the most applicable to the content of this book. Supervised learning refers to what we already touched upon in the paradigm shift example with the program, data, and output. We use a dataset with paired examples of input and expected output and ask the model to learn the relationship.

Let's start our discussion small and slowly build up to the whole schematic, which encapsulates most of the key components of a supervised machine learning problem:

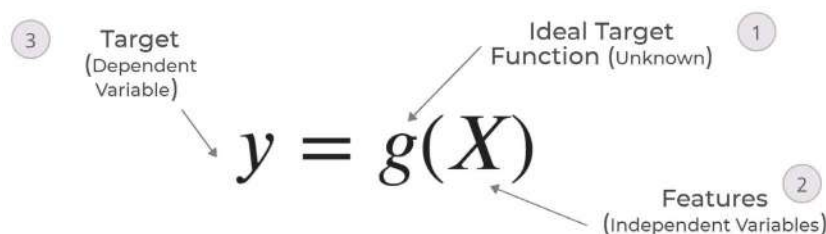


Figure 5.2: Supervised machine learning schematic, part 1—the ideal function

As we have already discussed, what we want from machine learning is to *learn* from the data and come up with a set of rules/logic. The closest analogy in mathematics for logic/rules is a function, which takes in an input (here, data) and provides an output. Mathematically, it can be written as follows:

$$y = g(X)$$

where X is the set of features and g is the **ideal target function** (denoted by 1 in Figure 5.2) that maps the X input (denoted by 2 in the schematic) to the target (ideal) output, y (denoted by 3 in the schematic). The ideal target function is largely an unknown function, similar to the **data-generating process (DGP)** we saw in Chapter 1, *Introducing Time Series*, which is not in our control.

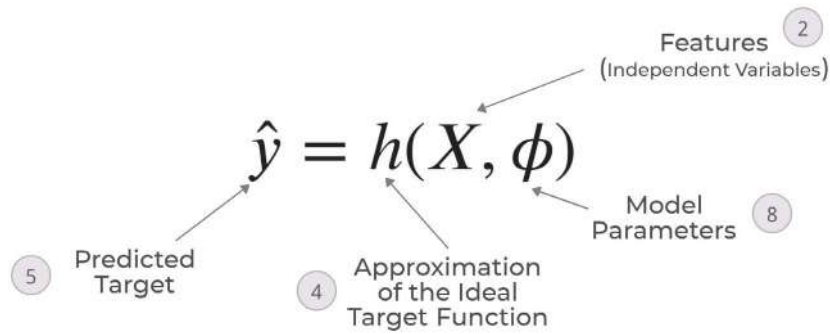


Figure 5.3: Supervised machine learning schematic, part 2—the learned approximation

But we want the computer to *learn* this ideal target function. This approximation of the ideal target function is denoted by another function, h (4 in the schematic), which takes in the same set of features, X , and outputs a predicted target, $\hat{y}$ (5 in the schematic). Φ is the parameters of the h function (or model parameters):

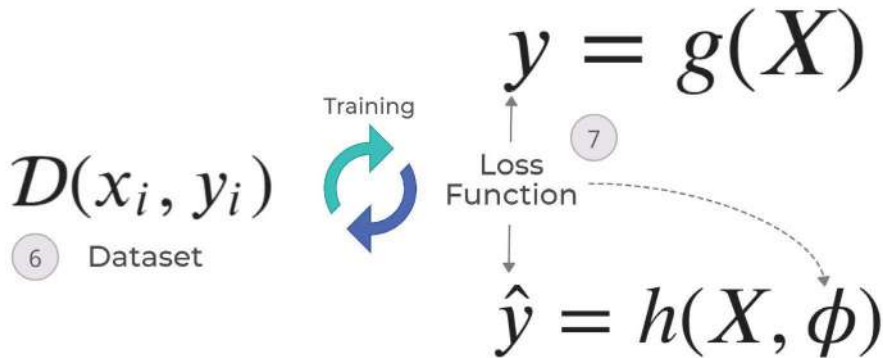


Figure 5.4: Supervised machine learning schematic, part 3—putting it all together

Now, how do we find this approximation h function and its parameters, Φ ? With the dataset of examples (6 in the schematic). The supervised machine learning problem works on the premise that we are able to collect a set of examples that shows the features, X , and the corresponding target, y , which is also referred to as *labels* in the literature. It is from this set of examples (the dataset) that the computer *learns* the approximation function, h , and the optimal model parameters, Φ . In the preceding diagram, the only real unknown entity is the ideal target function, g . So, we can use the training dataset, D , to get predicted targets for every sample in the dataset. We already know the ideal target for all the examples. We need a way to compare the ideal targets and predicted targets, and this is where the loss function (7 in the schematic) comes in. This loss function tells us how far away from the real truth we are with the approximated function, h .

Although h can be any function, it is typically chosen from a set of a well-known class of functions, H . H is the finite set of functions that can be fit to the data. This class of functions is what we colloquially call **models**. For instance, h can be chosen from all the linear functions or all the tree-based functions, and so on. Choosing an h from H is done by a combination of hyperparameters (which the modeler specifies) and the model parameter, which is learned from data.

Now, all that is left is to run through the different functions so that we find the best approximation function, h , which gives us the lowest loss. This is an optimization process that we call **training**.

Let's also take a look at a few key concepts, which will be important in all our discussions ahead.

Supervised machine learning tasks

Machine learning can be used to solve a wide variety of tasks, such as **regression**, **classification**, and **recommendation**. But since classification and regression are the most common classes of problems, we will spend just a little bit of time reviewing what they are.

The difference between classification and regression tasks is very simple. In the machine learning schematic (*Figure 5.2*), we talked about y , the target. This target can be either a real-valued number or a class of items. For instance, we could be predicting the stock price for next week or we could just predict whether the stock was going to go up or down. In the first case, we are predicting a real-valued number, which is called **regression**. In the other case, we are predicting one out of two classes (*up* or *down*), and this is called **classification**.

Overfitting and underfitting

The biggest challenge in machine learning systems is that the model we trained must perform well on a new and unseen dataset. The ability of a machine learning model to do that is called the **generalization capability** of the model. The training process in a machine learning setup is akin to mathematical optimization, with one subtle difference. The aim of mathematical optimization is to arrive at the global maxima in the provided dataset. But in machine learning, the aim is to achieve a low test error by using the training error as a proxy. How well a machine learning model is doing on the training error and testing error is closely related to the concepts of overfitting and underfitting. Let's use an example to understand these terms.

The learning process of a machine learning model has many parallels to how humans learn. Suppose three students, *A*, *B*, and *C*, are studying for an examination. *A* is a slacker and went clubbing the night before. *B* decided to double down and memorize the textbook from end to end. *C* paid attention in class and understood the topics for the examination.

As expected, *A* flunked the examination, *C* got the highest score, and *B* did OK.

A flunked the examination because they didn't learn enough. This happens to machine learning models as well when they don't learn enough patterns, and this is called **underfitting**. This is characterized by high training errors and high test errors.

B didn't score as highly as expected; after all, they did memorize the whole text, word for word. But many questions in the examination weren't directly from the textbook and *B* wasn't able to answer them correctly. In other words, the questions in the examination were *new and unseen*. And because *B* memorized everything but didn't make an effort to understand the underlying concepts, *B* wasn't able to *generalize* the knowledge they had to new questions. This situation, in machine learning, is called **overfitting**. This is typically characterized by a big delta in training and test errors. Typically, we will see very low training errors and high test errors.

The third student, *C*, learned the right way and understood the underlying concepts, and because of that was able to *generalize* to *new and unseen* questions. This is the ideal state for a machine learning model, as well. This is characterized by reasonably low test errors and a small delta between training and test errors.

We just saw the two greatest challenges in machine learning. Now, let's also look at a few ways we have that can be used to tackle these challenges.

There is a close relationship between the **capacity** of a model and underfitting or overfitting. A model's capacity is its ability to be flexible enough to fit a wide variety of functions. Models with low capacity may struggle to fit the training data, leading to underfitting. Models with high capacity may overfit by memorizing the training data too much. Just to develop an understanding of this concept of capacity, let's look at an example. When we move from linear regression to polynomial regression, we are adding more capacity to the model. Instead of fitting just straight lines, we are letting the model fit curved lines as well.

Machine learning models generally do well when their capacity is appropriate for the learning problem at hand.

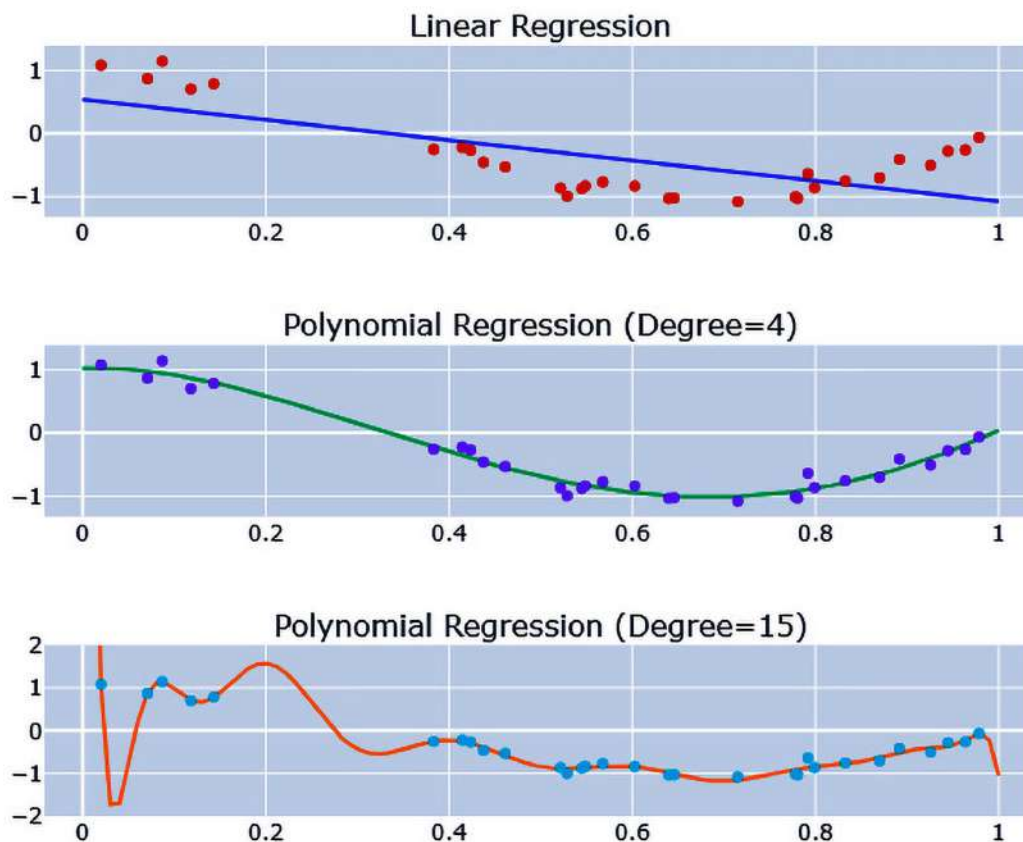


Figure 5.5: Underfitting versus overfitting

Figure 5.5 shows a very popular case to illustrate overfitting and underfitting. We create a few random points using a known function and try to learn that by using those data samples. We can see that the linear regression, which is one of the simplest models, has underfitted the data by drawing a straight line through those points. Polynomial regression is linear regression, but with some higher-order features. For now, you can consider the move from linear regression to polynomial regression with higher degrees as increasing the capacity of the model. So, when we use a degree of 4, we see that the learned function fits the data well and matches our ideal function. But if we keep increasing the capacity of the model and reach degree = 15, we see that the learned function is still passing through the training samples, but has learned a very different function, overfitting to the training data. Finding the optimal capacity to learn a generalizable function is one of the core challenges of machine learning.

While capacity is one aspect of the model, another aspect is **regularization**. Even with the same capacity, there are multiple functions a model can choose from the hypothesis space of all functions. With regularization, we try to give preference to a set of functions in the hypothesis space over the others.

While all these functions are valid functions that can be chosen, we nudge the optimization process in such a way that we end up with a kind of function toward which we have a preference. Although regularization is a general term used to refer to any kind of constraint we place on the learning process to reduce the complexity of the learned function, more commonly, it is used in the form of a weight decay. Let's take an example of linear regression, which is when we fit a straight line to the input features by learning a weight associated with each feature.

A linear regression model can be written mathematically as follows:

$$\hat{y} = c + \sum_{i=1}^N w_i \times x_i$$

Here, N is the number of features, c is the intercept, x_i is the i^{th} feature, and w_i is the weight associated with the i^{th} feature. We estimate the right weight (L) by considering this as an optimization problem that minimizes the error between $\hat{y}$ and y (real output).

Now, with regularization, we add an additional term to L , which forces the weights to become smaller. Commonly, this is done using an $L1$ or $L2$ regularizer. An $L1$ regularizer is when you add the sum of squared weights to L :

$$L + \lambda \sum_{i=1}^N w_i^2$$

where λ is the regularization coefficient that determines how strongly we penalize the weights. An $L2$ regularizer is when you add the sum of absolute weights to L :

$$L + \lambda \sum_{i=1}^N |w_i|$$

In both cases, we are enforcing a preference for smaller weights over larger weights because it keeps the function from relying too much on any one feature from the ones used in the machine learning model. Regularization is an entire topic unto itself; if you want to learn more, head over to the *Further reading* section for a few resources on regularization.

Another really effective way to reduce overfitting is to simply train the model with more data. With a larger dataset, the chances of the model overfitting become less because of the sheer variety that can be captured in a large dataset.

Now, how do we tune the knobs to strike a balance between underfitting and overfitting? Let's look at it in the following section.

Hyperparameters and validation sets

Almost all machine learning models have a few hyperparameters associated with them. **Hyperparameters** are parameters of the model that are not learned from data but rather are set before the start of training. For instance, the weight of the regularization is a hyperparameter. Most hyperparameters either help us control the capacity of the model or apply regularization to the model. By controlling either capacity or regularization or both, we can travel the frontier between underfitting and overfitting models and arrive at a model that is just right.

But since these hyperparameters have to be set outside of the algorithm, how do we estimate the best hyperparameters? Although it is not part of the core *learning process*, we also learn the hyperparameters from the data. But if we just use the training data to learn the hyperparameters, it will just choose the maximum possible model capacity, which results in overfitting. This is where we need a **validation set**, a part of the data that the training process does not have access to. Following the same analogy we saw earlier, the validation set is the mock exam students take to check if they have learned well enough. But when the dataset is small (not hundreds of thousands of samples), the performance on a single validation set doesn't guarantee a fair evaluation. In such cases, we rely on **cross-validation**. The general trick is to repeat the training and evaluation procedure on different subsets of the original dataset. A common way of doing this is called **k-fold cross-validation**, where the original dataset is divided into k equal, non-overlapping, and random subsets, and each subset is evaluated after training on all the other subsets. We have provided a link in the *Further reading* section if you want to read up on cross-validation techniques. Later in the book, we will also be covering this topic, but from the time series perspective, which has a few differences from the standard way of doing cross-validation.

Suggested reading:



Although we have scratched the surface of machine learning in this book, there is a lot more, and to truly appreciate the rest of the book better, we suggest gaining more understanding of machine learning. We suggest starting with *Machine Learning by Stanford* (Andrew Ng)—<https://www.coursera.org/learn/machine-learning>. If you are in a hurry, *Machine Learning Crash Course* by Google is also a good starting point—<https://developers.google.com/machine-learning/crash-course/ml-intro>.

Machine learning has progressed a lot in recent times, and along with it, powerful models that are able to learn complex patterns from data have come out of it. When we compare these models with the classical models of time series forecasting, we can see that there is a lot of potential in these newer classes of models. But there are some fundamental differences between machine learning and time series forecasting. In the next section, let's understand how we can get over those differences and use machine learning for time series forecasting.

Time series forecasting as regression

A time series, as we saw in *Chapter 1, Introducing Time Series*, is a set of observations taken sequentially in time. And typically, time series forecasting is about trying to predict what these observations will be in the future. Given a sequence of observations of arbitrary length of history, we predict the future to an arbitrary horizon.

We saw that regression, or machine learning to predict a continuous variable, works on a dataset of examples, and each example is a set of input features and targets. We can see that regression, which is tasked with predicting a single output provided with a set of inputs, is fundamentally incompatible with forecasting, where we are given a set of historical values and asked to predict the future values. This fundamental incompatibility between the time series and machine learning regression paradigms is why we cannot use regression for time series forecasting directly.

Moreover, time series forecasting, by definition, is an extrapolation problem, whereas regression, most of the time, is an interpolation one. Extrapolation is typically harder to solve using data-driven methods. Another key assumption in regression problems is that the samples used for training are **independent and identically distributed (iid)**. But time series break that assumption as well because subsequent observations in a time series display considerable dependence.

However, to use the wide variety of techniques from machine learning, we need to cast time series forecasting as a regression. Thankfully, there are ways to convert a time series into a regression and get over the IID assumption by introducing some memory to the machine learning model through some features. Let’s see how it can be done.

Time delay embedding

We talked about the ARIMA model in *Chapter 4, Setting a Strong Baseline Forecast*, and saw how it is an autoregressive model. We can use the same concept to convert a time series problem into a regression one. Let’s use the following diagram to make the concept clear:

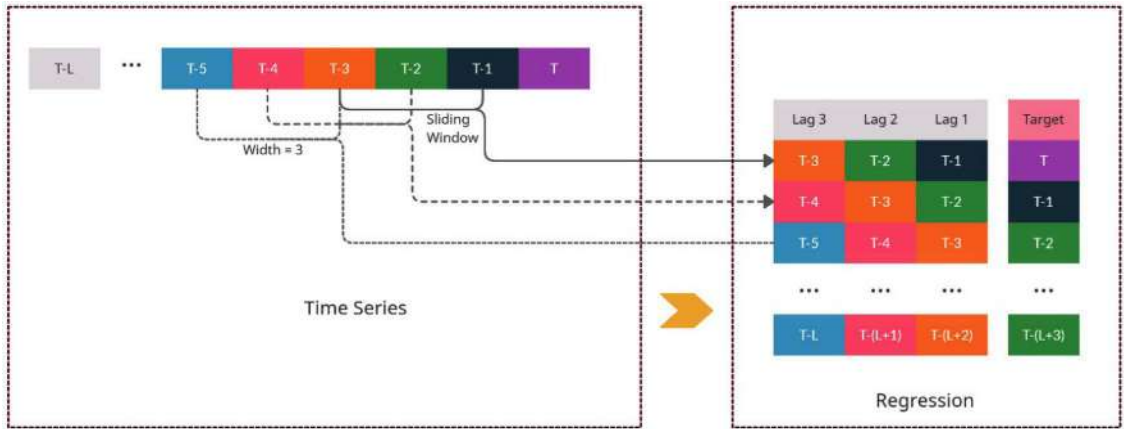


Figure 5.6: Time series to regression conversion using a sliding window

Let’s assume we have a time series with L time steps, just like in the diagram. We have T as the latest observation, $T - 1$, $T - 2$, and so on as we move backward in time, all the way to $T - L$. In an ideal world, each observation should be conditioned on all the previous observations when we forecast. But this is not practical because L can be arbitrarily long. We often restrict the forecasting function to use only the most recent M observations of the series, where $M < L$. These are called finite memory models, or **Markov models**, and M is called the order of autoregression, memory size, or the receptive field.

Therefore, in time delay embedding, we assume a window of arbitrary length $M < L$ and extract fixed-length subsequences from the time series by sliding the window over the length of the time series.

In the diagram, we have taken a sliding window with a memory size of 3. So, the first subsequence we can extract (if we are starting from the most recent and working backward) is $T - 3$, $T - 2$, $T - 1$. And T is the observation that comes right after the subsequence. This becomes our first example in the dataset (row 1 in the table in the diagram).

Now, we slide the window one time step to the left (backward in time) and extract the new subsequence, $T - 4$, $T - 3$, $T - 2$. The corresponding target would become $T - 1$. We repeat this process as we move back to the beginning of the time series, and at each step of the sliding window, we add one more example to the dataset.

At the end of it, we have an aligned dataset with a fixed vector size of features (which will be equal to the window size) and a single target, which is what a typical machine learning dataset looks like.

Now that we have a table with three features, let's also assign semantic meaning to the three features. If we look at the right-most column in the table in the diagram, we can see that the time step present in the column is always one time step behind the target. We call it **Lag 1**. The second column from the right is always two time steps behind the target, and this is called **Lag 2**. Generalizing this, the feature that has observations that are n time steps behind the target, we call **Lag n** .

This transformation from time series to regression using **time-delay embedding** encodes the autoregressive structure of a time series in a way that can be utilized by standard regression frameworks. Another way we can think about using regression for time series forecasting is to perform **regression on time**.

Temporal embedding

If we rely on previous observations in autoregressive models, we rely on the concept of time for temporal embedding models. The core idea is that we forget the autoregressive nature of the time series and assume that any value in the time series is only dependent on time. We derive features that capture time, the passage of time, the periodicity of time, and so on, from the timestamps associated with the time series, and then we use these features to predict the target using a regression model. There are many ways to do this, from simply aligning a monotonically and uniformly increasing numerical column that captures the passage of time to sophisticated **Fourier** terms to capture the periodic components in time. We will talk about those techniques in detail in *Chapter 6, Feature Engineering for Time Series Forecasting*.

Before we wind up the chapter, let's also talk about a key concept that is gaining ground steadily in the time series forecasting space. A large part of this book embraces this new paradigm of forecasting.

Global forecasting models—a paradigm shift

Traditionally, each time series was treated in isolation. Because of that, traditional forecasting has always looked at the history of a single time series alone in fitting a forecasting function. But recently, because of the ease of collecting data in today's digital-first world, many companies have started collecting large amounts of time series from similar sources, or related time series.

For example, retailers such as Walmart collect data on the sales of millions of products across thousands of stores. Companies such as Uber and Lyft collect the demand for rides from all the zones in a city. In the energy sector, energy consumption data is collected across all consumers. All these sets of time series have shared behavior and are hence called **related time series**.

We can consider that all the time series in a related time series come from separate DGPs, and thereby model them all separately. We call these the **local** models of forecasting. An alternative to this approach is to assume that all the time series are coming from a single DGP. Instead of fitting a separate forecast function for each time series individually, we fit a single forecast function to all the related time series. This approach has been called **global** or **cross-learning** in the literature. Most modern deep learning models as well as machine learning approaches adopt the global model paradigm. We will see those in detail in the coming chapters.



Reference check:

The term *global* was introduced by David Salinas *et al.* in the *DeepAR* paper (reference 1) and *cross-learning* by Slawek Smyl (reference 2).

We saw earlier that having more data will lead to lower chances of overfitting and, therefore, lowers the generalization error (the difference between training and testing errors). This is one of the shortcomings of the local approach. Traditionally, time series are not very long, and in many cases, it is difficult and time-consuming to collect more data as well. Fitting a machine learning model (with all its expressiveness) on small data is prone to overfitting. This is why time series models that enforce strong priors were used to forecast such time series, traditionally. But these strong priors, which restrict the fitting of traditional time series models, can also lead to a form of underfitting and limit accuracy.

Strong and expressive data-driven models, as in machine learning, require a larger amount of data to have a model that generalizes to new and unseen data. A time series, by definition, is tied to time, and sometimes, collecting more data means waiting for months or years and that is not desirable. So, if we cannot increase the *length* of the time series dataset, we can increase the *width* of the time series dataset. If we add multiple time series to the dataset, we increase the width of the dataset, and thereby increase the amount of data the model is getting trained with.

Figure 5.7 shows the concept of increasing the width of a time series dataset visually:

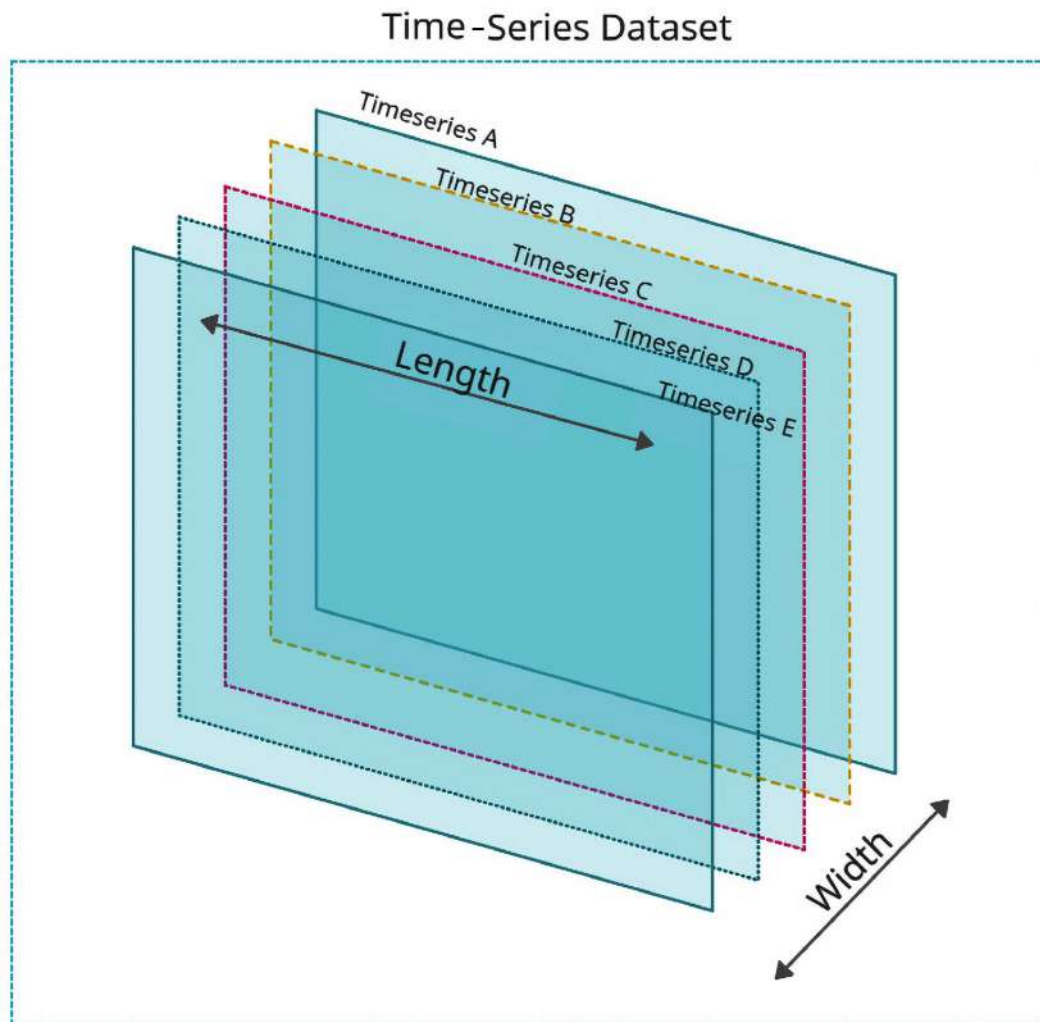


Figure 5.7: The length and width of a time series dataset

This works in favor of machine learning models because with higher flexibility in fitting a forecast function and the addition of more data to work with, the machine learning model can learn a more complex forecast function than traditional time series models, which are typically shared between the related time series, in a completely data-driven way.

Another shortcoming of the local approach revolves around scalability. In the case of Walmart we mentioned earlier, there are millions of time series that need to be forecasted and it is not possible to have human oversight on all these models. If we think about this from an engineering perspective, training and maintaining millions of models in a production system would give any engineer a nightmare. But under the global approach, we only train a single model for all these time series, which drastically reduces the number of models we need to maintain and yet can generate all the required forecasts.

This new paradigm of forecasting has gained traction and has consistently been shown to improve the local approaches in multiple time series competitions, mostly in datasets of related time series. In Kaggle competitions, such as *Rossmann Store Sales* (2015), *Wikipedia WebTraffic Time Series Forecasting* (2017), *Corporación Favorita Grocery Sales Forecasting* (2018), and *M5 Competition* (2020), the winning entries were all global models—either machine learning or deep learning or a combination of both. The *Intermarché Forecasting Competition* (2021) also had global models as the winning submissions. Links to these competitions are provided in the *Further reading* section.

Although we have many empirical findings where the global models have outperformed local models for related time series, global models are still a relatively new area of research. *Montero-Manson and Hyndman* (2020) showed a few very interesting results and showed that any local method can be approximated by a global model with required complexity, and the most interesting finding they put forward is that the global model will perform better, even with unrelated time series. We will talk more about global models and strategies for global models in *Chapter 10, Global Forecasting Models*.



Reference check:

The *Montero-Manson and Hyndman* (2020) research paper is cited in *References* under reference 3.

Summary

We have started our journey beyond baseline forecasting methods and dipped our toes into the world of machine learning. After a brief refresher on machine learning, where we looked at key concepts such as overfitting, underfitting, regularization, and so on, we saw how we can convert a time series forecasting problem into a regression problem from the machine learning world. We also developed a conceptual understanding of different embeddings, such as time delay embedding and temporal embedding, which can be used to convert a time series problem into a regression problem. To wrap things up, we also learned about a new paradigm in time series forecasting—global models—and contrasted them with local models on a conceptual level. In the next few chapters, we will start putting these concepts into practice, and see techniques for feature engineering and strategies for global models.

References

The following are the references that we used in this chapter:

1. David Salinas, Valentin Flunkert, Jan Gasthaus, Tim Januschowski (2020). *DeepAR: Probabilistic forecasting with autoregressive recurrent networks*. International Journal of Forecasting. 36-3: 1181–1191: <https://doi.org/10.1016/j.ijforecast.2019.07.001>
2. Slawek Smyl (2020). *A hybrid method of exponential smoothing and recurrent neural networks for time series forecasting*. International Journal of Forecasting. 36-1: 75–85 <https://doi.org/10.1016/j.ijforecast.2019.03.017>
3. Pablo Montero-Manso, Rob J Hyndman (2020), *Principles and Algorithms for Forecasting Groups of Time Series: Locality and Globality*. arXiv:2008.00444[cs.LG]: <https://arxiv.org/abs/2008.00444>

Further reading

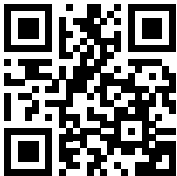
You can check out the following resources for further reading:

- *Regularization for Sparsity from Google Machine Learning Crash Course*: <https://developers.google.com/machine-learning/crash-course/regularization-for-sparsity/l1-regularization>
- *L1 & L2 Regularization, Inside Bloomberg*: <https://www.youtube.com/watch?v=d6XD0S4btck>
- *Cross-validation: evaluating estimator performance from scikit-learn*: https://scikit-learn.org/stable/modules/cross_validation.html
- *Rossmann Store Sales*: <https://www.kaggle.com/c/rossmann-store-sales>
- *Web Traffic Time Series Forecasting*: <https://www.kaggle.com/c/web-traffic-time-series-forecasting>
- *Corporación Favorita Grocery Sales Forecasting*: <https://www.kaggle.com/c/favorita-grocery-sales-forecasting>
- *M5 Forecasting—Accuracy*: <https://www.kaggle.com/c/m5-forecasting-accuracy>

Join our community on Discord

Join our community's Discord space for discussions with authors and other readers:

<https://packt.link/mts>



6

Feature Engineering for Time Series Forecasting

In the previous chapter, we started looking at **machine learning (ML)** as a tool to solve the problem of **time series forecasting**. We also discussed a few techniques, such as **time delay embedding** and **temporal embedding**, which cast time series forecasting problems as classical regression problems from the ML paradigm. In this chapter, we'll look at those techniques in detail and go through them in a practical sense, using the dataset we have worked with throughout this book.

In this chapter, we will cover the following topics:

- Understanding feature engineering
- Avoiding data leakage
- Setting a forecast horizon
- Time delay embedding
- Temporal embedding

Technical requirements

You will need to set up the **Anaconda** environment, following the instructions in the *Preface* of the book, to get a working environment with all the libraries and datasets required for the code in this book. Any additional libraries needed will be installed while running the notebooks.

You will need to run the following notebooks before using the code in this chapter:

- 02-Preprocessing_London_Smart_Meter_Dataset.ipynb from Chapter02
- 01-Setting_up_Experiment_Harness.ipynb from Chapter04

The code for this chapter can be found at <https://github.com/PacktPublishing/Modern-Time-Series-Forecasting-with-Python-2E/tree/main/notebooks/Chapter06>.

Understanding feature engineering

Feature engineering, as the name suggests, is the process of engineering features from data, mostly using domain knowledge, to make the learning process smoother and more efficient. In a typical ML setting, engineering good features is essential to get good performance from any ML model. Feature engineering is a highly subjective part of ML, where each problem at hand has a different path of solution—one that is handcrafted for that problem. Suppose you have a dataset of house prices and you have a feature, *Year Built*, which tells you the year the house was built. Now, to make the information better, we can create another feature, *House Age*, from the *Year Built* feature. This may give the model better information, and this is called feature engineering.

When we are casting a time series problem as a regression problem, there are a few standard techniques that we can apply. This is a key step in the process, as how well an ML model acquires an understanding of *time* is dependent on how well we engineer features to capture *time*. The baseline methods we covered in *Chapter 4, Setting a Strong Baseline Forecast*, are the methods that are created for the specific use case of time series forecasting, and because of that, the temporal aspect of the problem is built into those models. For instance, ARIMA doesn't need any feature engineering to understand time because it is built into the model. However, a standard regression model has no explicit understanding of time, so we need to create good features to embed the temporal aspect of the problem.

In the previous chapter (*Chapter 5, Time Series Forecasting as Regression*), we talked about two main ways to encode time in the regression framework: **time delay embedding** and **temporal embedding**. Although we touched on these concepts at a high level, it is time to dig deeper and see them in action.



Notebook alert:

To follow along with the complete code, use the 01-Feature_Engineering.ipynb notebook in the Chapter06 folder.

We have already split the dataset that we were working on into train, validation, and test datasets. However, since we are generating features that are based on previous observations, operationally, it is better when we have the train, validation, and test datasets combined. It will be clearer why shortly, but for now, let's take it on faith and move ahead. Now, let's combine the two datasets:

```
# Reading the missing value imputed and train test split data
train_df = pd.read_parquet(preprocessed / "selected_blocks_train_missing_
imputed.parquet")
val_df = pd.read_parquet(preprocessed / "selected_blocks_val_missing_imputed.
parquet")
test_df = pd.read_parquet(preprocessed / "selected_blocks_test_missing_imputed.
parquet")
#Adding train, validation and test tags to distinguish them before combining
train_df['type'] = "train"
val_df['type'] = "val"
```

```
test_df['type'] = "test"
full_df = pd.concat([train_df, val_df, test_df]).sort_values(["LCLid",
"timestamp"])
del train_df, test_df, val_df
```

Now, we have a `full_df`, which combines the train, validation, and test datasets. Some of you may already have alarm bells ringing in your head at combining the train and test sets. What about **data leakage**? Let's check it out.

Avoiding data leakage

Data leakage occurs when a model is trained with some information that would not be available at the time of prediction. Typically, this leads to high performance in the training set but very poor performance in unseen data. There are two types of data leakage:

- **Target leakage** is when the information about the target (that we are trying to predict) leaks into some of the features in the model, leading to an overreliance of the model on those features, ultimately leading to poor generalization. This includes features that use the target in any way.
- **Train-test contamination** is when there is some information leakage between the train and test datasets. This can happen because of the careless handling and splitting of data. But it can also happen in more subtle ways, such as scaling a dataset before splitting the train and test sets.

When we work with time series forecasting problems, the biggest and most common mistake that we can make is target leakage. We will have to think hard about each of the features to ensure that we don't use any data that will not be available during prediction. The following diagram will help us remember and internalize this concept:

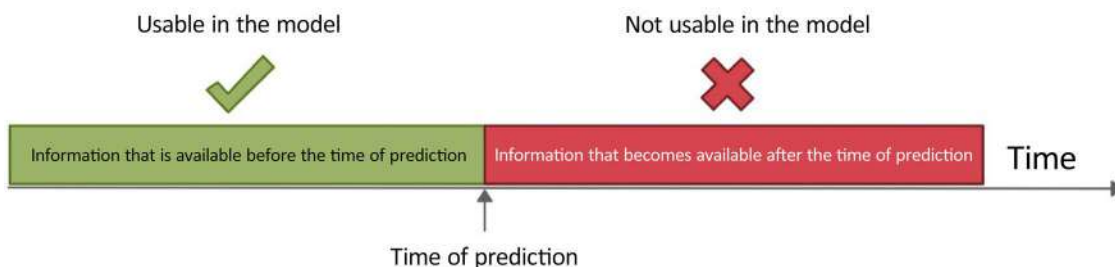


Figure 6.1: Usable and unusable information to avoid data leakage

To make this concept clearer and more relevant to the time series forecasting context, let's look at an example. Let's say we are forecasting sales for shampoo, and we are using sales for conditioner as a feature. We developed the model, trained it on the training data, and tested it on the validation data. The model does very well. The moment we start predicting for the future, we can see a problem. We don't know what the sales for conditioner are in the future either. While this example is pretty straightforward, there will be times when this becomes not so obvious. And that is why we need to exercise a fair amount of caution while creating features and always evaluate the features through the lens of *will this feature be available at the time of prediction?*

Best practices:

There are many ways of identifying target leakage, apart from thinking hard about the features:



- If the model you've built is too good to be true, you most likely have a leakage problem
- If any single feature has too much weightage in the feature importance of the model, that feature may have a problem with leakage
- Double-check the features that are highly correlated with the target

Although we generated forecasts earlier in this book, we never explicitly discussed **forecast horizons**. It is an important concept and essential for what we will discuss. Let's take a bit of time to understand forecast horizons.

Setting a forecast horizon

A forecast horizon is the number of time steps into the future we want to forecast at any point in time. For instance, if we want to forecast the next 24 hours for the electricity consumption dataset that we have worked with, the forecast horizon becomes 48 (because the data is half-hourly). In *Chapter 5, Time Series Forecasting as Regression*, where we generated baselines, we just predicted the entire test data at once. In such cases, the forecast horizon becomes equal to the length of the test data.

We never had to worry about this until now because, in the classical statistical methods of forecasting, this decision is decoupled from modeling. If we train a model, we can use it to predict any future point without retraining. But with *time series forecasting as regression*, we have a constraint on the forecast horizon, and it has its roots in data leakage. This might be unclear to you now, so we'll revisit this point after we have learned about the feature engineering techniques. For now, let's only look at single-step-ahead forecasting. In the context of the dataset we are working with, this means that we will be answering the question, *What is the energy consumption in the next half an hour?* We will talk about multi-step forecasting and other mechanics of forecasting in *Part 4, The Mechanics of Forecasting*.

Now that we have set some ground rules, let's start looking at the different feature engineering techniques. To follow along with the Jupyter notebook, head over to the `Chapter06` folder and use the `01-Feature_Engineering.ipynb` file.

Time delay embedding

The basic idea behind time delay embedding is to embed time in terms of recent observations. In *Chapter 5, Time Series Forecasting as Regression*, we discussed including previous observations of a time series as **lags** (Figure 5.6 under the subsection *Time delay embedding*).

However, there are a few more ways to capture recent and seasonal information using this concept.

- Lags
- Rolling window aggregations

- Seasonal rolling window aggregations
- Exponentially weighted moving averages

Let's take a look.

Lags or backshift

Let's assume we have a time series with time steps, Y_L . Consider that we are at time T and that we have a time series where the length of history is L . So our time series will have y_T as the latest observation in the time series, and then y_{T-1} , y_{T-2} , and so on as we move back in time. So lags, as explained in *Chapter 5, Time Series Forecasting as Regression*, are features that include the previous observations in the time series, as shown in the following diagram:

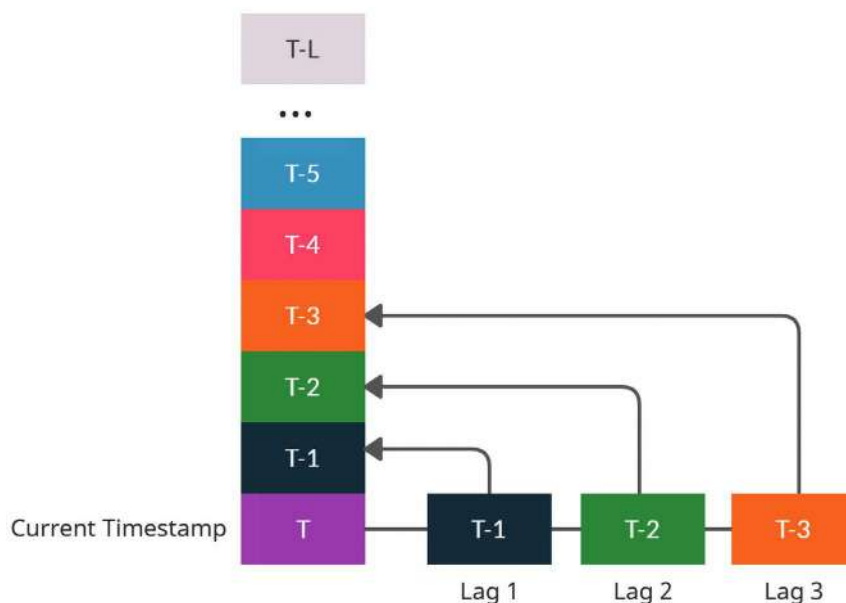


Figure 6.2: Lag features

We can create multiple lags by including observations that are a timesteps before (y_{T-a}); we will call this *Lag a* . In the preceding diagram, we have shown *Lag 1*, *Lag 2*, and *Lag 3*. However, we can add any number of lags we like. Let's learn how to do that now in code:

```
df["lag_1"] = df["column"].shift(1)
```

Remember when we combined the train and test datasets and I asked you to take it in good faith? It's time to repay that faith. If we consider the lag operation (or any autoregressive feature), it relies on a continuous representation along the time axis. If we consider the test dataset, for the first few rows (or earliest dates), the lags would be missing because they are part of the training dataset. So by combining the two, we create a continuous representation along the time axis where standard functions in pandas, such as `shift`, can be utilized to create these features easily and efficiently.

It is as simple as that, but we need to perform the lag operation for each LCLid separately. We have included a helpful method in `src.feature_engineering.autoregressive_features` called `add_lags` that adds all the lags you want for each LCLid quickly and efficiently. Let's see how we can use that.

We are going to import the method and use a few of its parameters to configure the lag operation the way we want:

```
from src.feature_engineering.autoregressive_features import add_lags
# Creating first 5 lags and then same 5 lags but from previous day and previous
# week to capture seasonality
lags = (
    (np.arange(5) + 1).tolist()
    + (np.arange(5) + 46).tolist()
    + (np.arange(5) + (48 * 7) - 2).tolist()
)
full_df, added_features = add_lags(
    full_df, lags=lags, column="energy_consumption", ts_id="LCLid", use_32_
    bit=True
)
```

Now, let's look at the parameters that we used in the previous code snippet:

- `lags`: This parameter takes in a list of integers, denoting all the lags we need to create as features.
- `column`: The name of the column to be lagged. In our case, this is `energy_consumption`.
- `ts_id`: The name of the column that contains the unique ID of a time series. If `None`, it assumes that the DataFrame only contains a single time series. In our case, `LCLid` is the name of that column.
- `use_32_bit`: This parameter doesn't do anything functionally but makes the DataFrames much smaller in memory, sacrificing the precision of the floating-point numbers.

This method returns the DataFrame with the lags added, as well as a list with the column names of the newly added features.

Rolling window aggregations

With lags, we connect the present points to single points in the past, but with rolling window features, we connect the present with an aggregate statistic of a window from the past. Instead of looking at the observation from previous time steps, we would look at an average of the observations from the last three timesteps. Take a look at the following diagram to understand this better:



Figure 6.3: Rolling window aggregation features

We can calculate rolling statistics with different windows, and each of them will capture slightly different aspects of the history. In the preceding diagram, we can see an example of a window of three and a window of four. When we are at timestep T , a rolling window of three would have $y_{T-3}, y_{T-2}, y_{T-1}$ as the vector of past observations. Once we have these, we can apply any aggregation functions, such as the mean, standard deviation, min, max, and so on. Once we have a scalar value after the aggregation function, we can include that as a feature for timestep t .



We do not include y_T in the vector of past observations because that leads to data leakage.

Let's see how we can do this with pandas:

```
# We shift by one to make sure there is no data leakage
df["rolling_3_mean"] = df["column"].shift(1).rolling(3).mean()
```

Similar to the lags, we need to do this operation for each LCLid column separately. We have included a helpful method in `src.feature_engineering.autoregressive_features` called `add_rolling_features` that adds all the rolling features you want for each LCLid quickly and efficiently. Let's see how we can use that.

We are going to import this method and use a few of its parameters to configure the rolling operation the way we want:

```
from src.feature_engineering.autoregressive_features import add_rolling_
features
full_df, added_features = add_rolling_features(
    full_df,
    rolls=[3, 6, 12, 48],
    column="energy_consumption",
    agg_funcs=["mean", "std"],
    ts_id="LCLid",
    use_32_bit=True,
)
```

Now, let's look at the parameters that we used in the previous code snippet:

- **rolls:** This parameter takes in a list of integers denoting all the windows over which we need to calculate the aggregate statistics.
- **column:** The name of the column to be lagged. In our case, this is `energy_consumption`.
- **agg_funcs:** This is a list of aggregations that we want to do for each window we declared in `rolls`. Allowable aggregation functions include `{mean, std, max, min}`.
- **n_shift:** This is the number of timesteps we need to shift before doing the rolling operation. This parameter avoids data leakage. Although we are shifting by one here, there are cases where we need to shift by more than one as well. This is typically used in multi-step forecasting, which we will cover in *Part 4, The Mechanics of Forecasting*.
- **ts_id:** The name of the column name that contains the unique ID of a time series. If `None`, it assumes that the DataFrame only has a single time series. In our case, `LCLid` is the name of that column.
- **use_32_bit:** This parameter doesn't do anything functionally but makes the DataFrames much smaller in memory, sacrificing the precision of the floating-point numbers.

This method returns the DataFrame with the rolling features added, as well as a list with the column names of the newly added features.

Seasonal rolling window aggregations

Seasonal rolling window aggregations are very similar to rolling window aggregations, but instead of taking past n consecutive observations in the window, they take a seasonal window, skipping a constant number of timesteps between each item in a window. The following diagram will make this clearer:

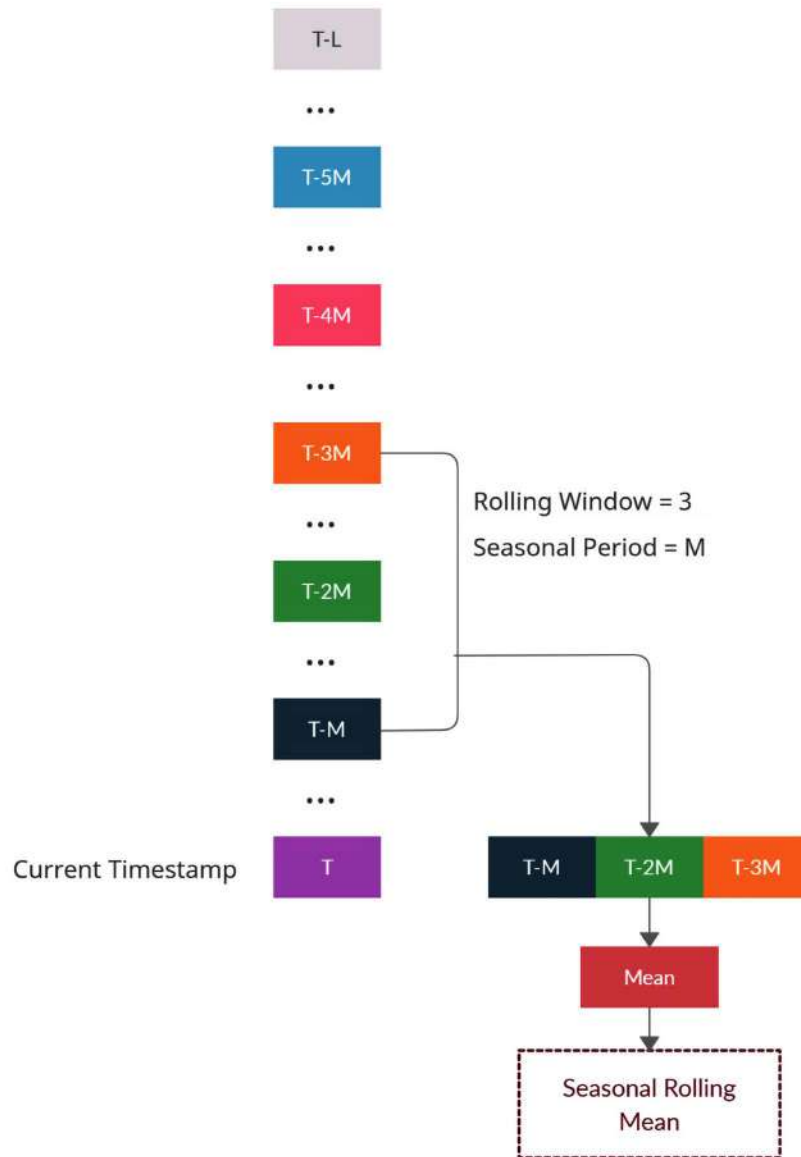


Figure 6.4: Seasonal rolling window aggregations

The key parameter here is the seasonality period, which is commonly referred to as M . This is the number of timesteps after which we expect the seasonality pattern to repeat. When we are at timestep T , a rolling window of three would have y_{T-3} , y_{T-2} , y_{T-1} , as the vector of past observations. But the seasonal rolling window would skip m timesteps between each item in the window. This means that the observations that are there in the seasonal rolling window would be y_{T-M} , y_{T-2M} , y_{T-3M} . Also, as usual, once we have the window vector, we just need to apply the aggregation function to get a scalar value and include that as a feature.



We do *not* include y_T as an element in the seasonal rolling window vector to avoid data leakage.

This is not an operation that you can do easily and efficiently with pandas. Some fancy NumPy indexing and Python loops should do the trick. We will use an implementation from github.com/jmoralez/window_ops/ that uses NumPy and Numba to make the operation fast and efficient.

Just like the features we saw earlier, we need to do this operation for each LCLid separately. We have included a helpful method in `src.feature_engineering.autoregressive_features` called `add_seasonal_rolling_features` that adds all the seasonal rolling features you want for each LCLid quickly and efficiently. Let's see how we can use that.

We are going to import the method and use a few parameters of the method to configure the seasonal rolling operation the way we want:

```
from src.feature_engineering.autoregressive_features import add_seasonal_rolling_features
full_df, added_features = add_seasonal_rolling_features(
    full_df,
    rolls=[3],
    seasonal_periods=[48, 48 * 7],
    column="energy_consumption",
    agg_funcs=["mean", "std"],
    ts_id="LCLid",
    use_32_bit=True,
)
```

Now, let's look at the parameters that we used in the previous code snippet:

- **seasonal_periods:** This is a list of seasonal periods that should be used in the seasonal rolling windows. In the case of multiple seasonalities, we can include the seasonal rolling features of all the seasonalities.
- **rolls:** This parameter takes in a list of integers denoting all the windows over which we need to calculate aggregate statistics.
- **column:** The name of the column to be lagged. In our case, this is `energy_consumption`.
- **agg_funcs:** This is a list of aggregations that we want to do for each window we declared in `rolls`. The allowable aggregation functions are {mean, std, max, min}.
- **n_shift:** This is the number of seasonal timesteps we need to shift before doing the rolling operation. This parameter prevents data leakage.
- **ts_id:** The name of the column name that contains the unique ID of a time series. If None, it assumes that the DataFrame only contains a single time series. In our case, `LCLid` is the name of that column.

- `Use_32_bit`: This parameter doesn't do anything functionally but makes the DataFrames much smaller in memory, sacrificing the precision of the floating-point numbers.

As always, the method returns the DataFrame with seasonal rolling features and a list containing the column names of the newly added features.

Exponentially weighted moving average (EWMA)

With the rolling window mean operation, we calculated the average of the window, and it works synonymously with the **moving average**. EWMA is the slightly smarter cousin of the moving average. While the moving average considers a rolling window and considers each item in the window equally on the computed average, EWMA tries to do a weighted average on the window, and the weights decay at an exponential rate. There is a parameter, α , that determines how fast the weights decay. Because of this, we can consider all the history available as a window and let the α parameter decide how much recency is included in EWMA. This can be written simply and recursively, as follows:

$$EWMA_T = \alpha \times y_T + (1 - \alpha) \times EWMA_{T-1}$$

Here, we can see that the larger the value of α , the more the average is skewed toward recent values (see *Figure 6.6* to get a visual impression of how the weights would be). If we expand the recursion, the weights of each term work out to be:

$$W_{T-k} = \alpha \times (1 - \alpha)^k$$

where k is the number of timesteps behind T . If we plot the weights, we can see them in an exponential decay; α determines how fast the decay happens. Another way to think about α is in terms of **span**. Span is the number of periods at which the decayed weights approach zero (not in a strictly mathematical way but intuitively). α and span are related through this equation:

$$\alpha = \frac{2}{1 + \text{span}}$$

This will become clearer in the following diagram, where we have plotted how the weights decay for different values of α :

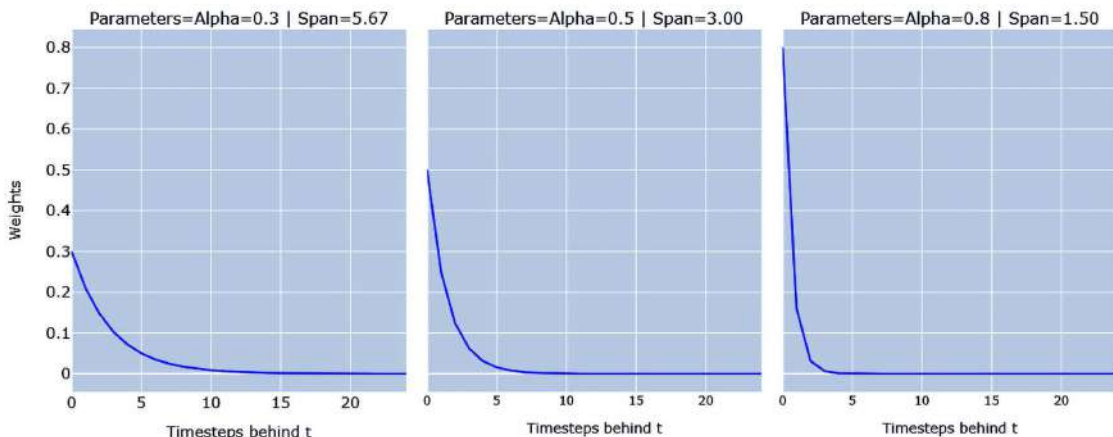


Figure 6.5: Exponential weight decay for different values of α

Here, we can see that the weight becomes small by the time we reach the span.

Intuitively, we can think of EWMA as an average of the entire history of the time series, but with parameters such as α and **span**, we can make different periods of history more representative of the average. If we define a 60-period span, we can think that the last 60 time periods are what majorly drive the average. So making EWMA with different spans or α s gives us representative features that capture different periods of history.

The overall process is depicted in the following diagram:

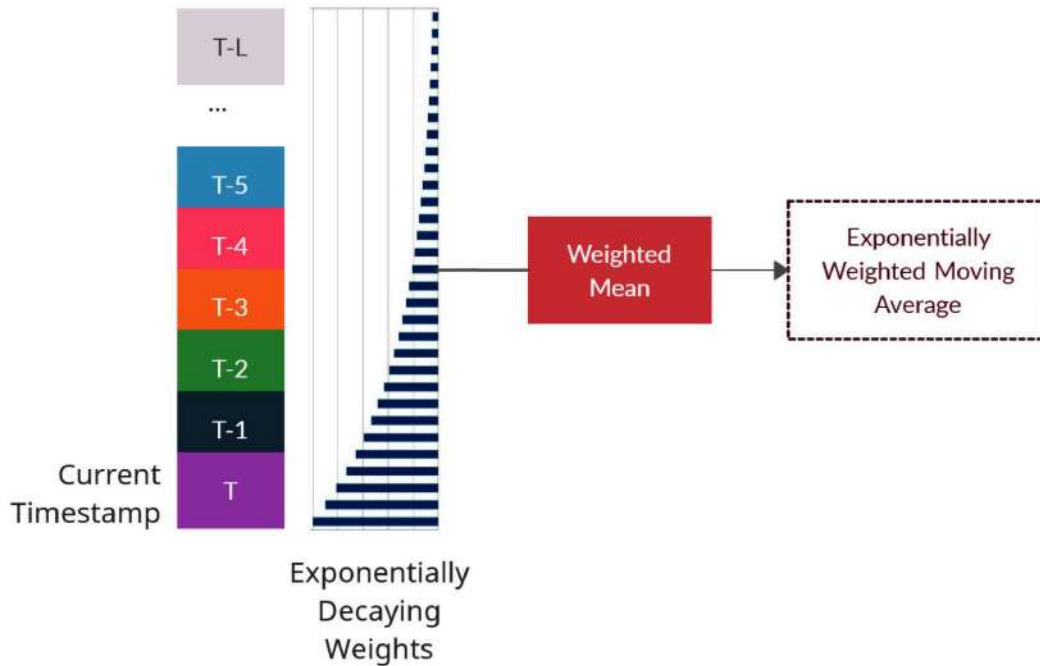


Figure 6.6: EWMA features

Now, let's see how we can do this in pandas:

```
df["ewma"] = df['column'].shift(1).ewm(alpha=0.5).mean()
```

Like the other features we discussed earlier, EWMA also needs to be done for each LCLid separately. We have included a helpful method in `src.feature_engineering.autoregressive_features` called `add_ewma` that adds all the EWMA features you want for each LCLid quickly and efficiently. Let's see how we can use that.

We are going to import the method and use a few parameters of the method to configure EWMA the way we want to:

```
from src.feature_engineering.autoregressive_features import add_ewma
full_df, added_features = add_ewma(
    full_df,
```

```
spans=[48 * 60, 48 * 7, 48],  
column="energy_consumption",  
ts_id="LCLid",  
use_32_bit=True,  
)
```

Now, let's look at the parameters that we used in the previous code snippet:

- **alphas**: This is a list of all α s we need to calculate the EWMA features for.
- **spans**: Alternatively, we can use this to list all the spans we need to calculate the EWMA features for. If you use this feature, **alphas** will be ignored.
- **column**: The name of the column to be lagged. In our case, this is `energy_consumption`.
- **n_shift**: This is the number of seasonal timesteps we need to shift before doing the rolling operation. This parameter avoids data leakage.
- **ts_id**: The name of the column name that has a unique ID for a time series. If `None`, it assumes the `DataFrame` only contains a single time series. In our case, `LCLid` is the name of that column.
- **use_32_bit**: This parameter doesn't do anything functionally but makes the `DataFrames` much smaller in memory, sacrificing the precision of the floating-point numbers.

As always, the method returns the `DataFrame` containing the EWMA features, as well as a list with the column names of the newly added features.

These are a few standard ways of including time delay embedding in your ML model, but you are not restricted to just these. As always, feature engineering is a space that is not bound by rules, and we can get as creative as we want and inject domain knowledge into the model. Apart from the features we have seen, we can include the difference in lag as custom lags that inject domain knowledge, and so on. In most practical cases, we end up using more than one way of time delay embedding into our models. The lag feature is the most basic and essential in most cases, but we do end up encoding more information with seasonal lags, rolling features, and so on. As with everything in ML, there is no silver bullet. Each dataset has its own intricacies, which makes feature engineering very important and different for each case.

Now, let's look at the other class of features we can add via **temporal embedding**.

Temporal embedding

In *Chapter 5, Time Series Forecasting as Regression*, we briefly discussed temporal embedding as a process where we try to embed *time* into features that an ML model can leverage. If we think about *time* for a second, we can see that two aspects of time are important to us in the context of time series forecasting—the *passage of time* and the *periodicity of time*.

There are a few features that we can add to help us capture these aspects in an ML model:

- Calendar features
- Time elapsed
- Fourier terms

Let's look at each of them.

Calendar features

The first set of features that we can extract are features based on calendars. Although the strict definition of time series is a set of observations taken sequentially in time, more often than not, we will have the timestamps of these collected observations alongside the time series. We can utilize these timestamps and extract calendar features, such as the month, quarter, day of the year, hour, minutes, and so on. These features capture the periodicity of time and help an ML model capture seasonality well. Only the calendar features that are temporally higher than the frequency of the time series make sense. For instance, an hour feature in a time series with a weekly frequency doesn't make sense, but a month feature and week feature make sense. We can utilize in-built datetime functionalities in pandas to create these features and treat them as categorical features in the model.

Time elapsed

This is another feature that captures the passage of time in an ML model. This feature increases monotonically as time increases, giving the ML model a sense of the passage of time. There are many ways to create this feature, but one of the easiest and most efficient ways is to use the integer representation of dates in NumPy:

```
df['time_elapsed'] = df['timestamp'].values.astype(np.int64)/(10**9)
```

We have included a helpful method in `src.feature_engineering.temporal_features` called `add_temporal_features` that adds all relevant temporal features automatically. Let's see how we can use it.

We are going to import the method and use a few parameters of this method to configure and create the temporal features:

```
full_df, added_features = add_temporal_features(
    full_df,
    field_name="timestamp",
    frequency="30min",
    add_elapsed=True,
    drop=False,
    use_32_bit=True,
)
```

Now, let's look at the parameters that we used in the previous code snippet:

- `field_name`: This is the column name that contains the datetime that should be used to create features.
- `frequency`: We should provide the frequency of the time series as input so that the method automatically extracts the relevant features. These are standard pandas frequency strings.
- `add_elapsed`: This flag turns the creation of the time elapsed feature on or off.
- `use_32_bit`: This parameter doesn't do anything functionally but makes the DataFrames much smaller in memory, sacrificing the precision of the floating-point numbers.

Just like the previous methods we discussed, this also returns the new DataFrame with the temporal features added and a list containing the column names of the newly added features.

Fourier terms

Previously, we extracted a few calendar features such as the month, year, and so on, and we discussed using them as categorical variables in the ML model. Another way we can represent the same information, but on a continuous scale, is by using **Fourier terms**. We discussed the Fourier series in *Chapter 3, Analyzing and Visualizing Time Series Data*. Just to reiterate, the sine-cosine form of the Fourier series is as follows:

$$S_{N(x)} = \frac{a_0}{2} + \sum_{n=1}^N \left(a_n \cdot \cos\left(\frac{2\pi}{P} \cdot n \cdot x\right) + b_n \cdot \sin\left(\frac{2\pi}{P} \cdot n \cdot x\right) \right)$$

Here, S_N is the N -term approximation of the signal, S . Theoretically, when N is infinite, the resulting approximation is equal to the original signal. P is the maximum length of the cycle, a_n and b_n are the coefficients of the cosine and sine term, respectively, of the n^{th} term in the expansion, and a_0 is the intercept.

We can create these cosine and sine functions as features to represent the seasonal cycle. If we encode the month, we know that it goes from 1 to 12 and then repeats itself. So P , in this case, will be 12, and x will be 1, 2, ...12. Therefore, for each x , we can calculate the cosine and sine terms and add them as features to the ML model. Intuitively, we can think that the model will infer the coefficients based on the data and, thus, help the model predict the time series easier.

The following diagram shows the difference in representations between the month on an ordinal scale and as a Fourier series:

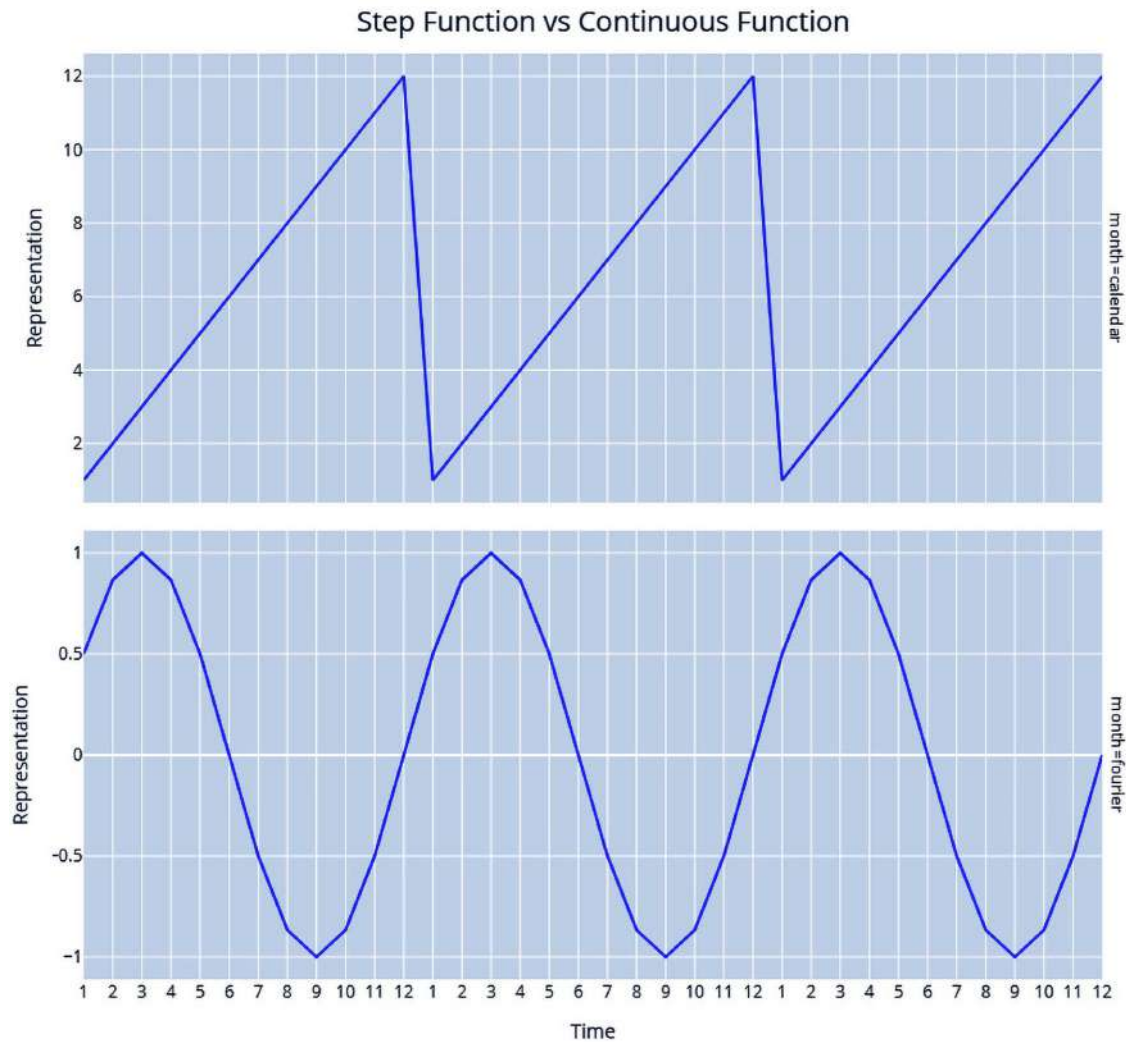


Figure 6.7: Month as an ordinal step function (top) versus Fourier terms (bottom)

The preceding diagram shows just a single Fourier term; we can add multiple Fourier terms to help capture complex seasonality.

We cannot say that continuous representation of seasonality is better than categorical because it depends on the type of model you use and the dataset. This is something we will have to find out empirically.

To make the process of adding Fourier features easy, we have made some easy-to-use methods available in `src.feature_engineering.temporal_features`, in a file called `bulk_add_fourier_features` that adds Fourier features for all the calendar features we want automatically. Let's see how we can use that.

We are going to import the method and use a few of its parameters to configure and create the Fourier series-based features:

```
full_df, added_features = bulk_add_fourier_features(  
    full_df,  
    ["timestamp_Month", "timestamp_Hour", "timestamp_Minute"],  
    max_values=[12, 24, 60],  
    n_fourier_terms=5,  
    use_32_bit=True,  
)
```

Now, let's look at the parameters that we used in the previous code snippet:

- `columns_to_encode`: This is the list of calendar features we need to encode using Fourier terms.
- `max_values`: This is a list of max values for the seasonal cycle for the calendar features, in the same order as they are given in `columns_to_encode`. For instance, for month to encode as a column, we give 12 as the corresponding `max_value`. If not given, `max_value` will be inferred. This is only recommended if the data you have contains at least a single complete seasonal cycle.
- `n_fourier_terms`: This is the number of Fourier terms to be added. This is synonymous with n in the equation for the Fourier series mentioned previously.
- `use_32_bit`: This parameter doesn't do anything functionally but makes the DataFrames much smaller in memory, sacrificing the precision of the floating-point numbers.

Just like the previous methods we've discussed, this also returns a new DataFrame with the Fourier features added, as well as a list with column names of the newly added features.

After executing the `01-Feature_Engineering.ipynb` notebook in Chapter06, we will have the following feature-engineered files written to disk:

- `selected_blocks_train_missing_imputed_feature_engg.parquet`
- `selected_blocks_val_missing_imputed_feature_engg.parquet`
- `selected_blocks_test_missing_imputed_feature_engg.parquet`

In this section, we looked at a few popular and effective ways to generate features for time series. But there are many more, and depending on your problem and the domain, many of them will be relevant.

Additional information:



The world of feature engineering is vast, and there are a few open-source libraries that make exploring that space easier. A few of them are <https://github.com/Nixtla/tsfeatures>, <https://tsfresh.readthedocs.io/en/latest/>, and <https://github.com/DynamicsAndNeuralSystems/catch22>. A preprint by Ben D. Fulcher titled *Feature-based time-series analysis* at <https://arxiv.org/abs/1709.08055> also gives a nice summary of the space.

A newer library called functime (<https://github.com/funtime-org/funtime>) also provides fast feature engineering routines, written in Polars, and is worth checking out. A lot of the feature engineering discussed in the book can be made even faster using functime and Polars.

Summary

After a brief overview of the ML for time series forecasting paradigm in the previous chapter, in this chapter, we looked at this practically and saw how we can prepare the dataset with the required features to start using these models. We reviewed a few time series-specific feature engineering techniques, such as lags, rolling, and seasonal features. All the techniques we learned in this chapter are tools with which we can quickly iterate through experiments to find out what works for our dataset. However, we only talked about feature engineering, which affects one side of the standard regression equation ($y = mX + c$). The other side, which is the target (y) we predict, is also equally important. In the next chapter, we'll look at a few concepts such as stationarity and some transformations that affect the target.

Unlock this book's exclusive benefits now

This book comes with additional benefits designed to elevate your learning experience.



Note: Have your purchase invoice ready before you begin. <https://www.packtpub.com/unlock/9781835883181>

7

Target Transformations for Time Series Forecasting

In the previous chapter, we delved into how we can do temporal embedding and time delay embedding by making use of feature engineering techniques. But that was just one side of the regression equation—the features. Often, we see that the other side of the equation—the target—does not behave the way we want. In other words, the target doesn't have some desirable properties that make forecasting easier. One of the major culprits in this area is **stationarity**—or more specifically, the lack of it. And it creates problems with the assumptions we make while developing a **machine learning (ML)**/statistical model. In this chapter, we will look at some techniques for handling such problems with the target.

In this chapter, we will cover the following topics:

- Handling non-stationarity in time series
- Detecting and correcting for unit roots
- Detecting and correcting for trends
- Detecting and correcting for seasonality
- Detecting and correcting for heteroscedasticity
- AutoML approach to target transformation

Technical requirements

You will need to set up the **Anaconda** environment following the instructions in the *Preface* of the book to get a working environment with all the libraries and datasets required for the code in this book. Any additional library will be installed while running the notebooks.

You will need to run the following notebooks before using the code in this chapter:

- 02-Preprocessing_London_Smart_Meter_Dataset.ipynb in the Chapter02 folder
- 01-Setting_up_Experiment_Harness.ipynb in the Chapter04 folder
- 01-Feature_Engineering.ipynb in the Chapter06 folder

The associated code for this chapter can be found at <https://github.com/PacktPublishing/Modern-Time-Series-Forecasting-with-Python-/tree/main/notebooks/Chapter07>.

Detecting non-stationarity in time series

Stationarity is a prevalent assumption in most econometrics models and is a rigorous and mathematical concept. But without getting into a lot of math, we can intuitively think about stationarity as the state where the statistical properties of the distribution from which the time series is sampled remain constant over time. This is relevant in time series as regression as well because we are estimating a single forecasting function across time. And if the *behavior* of the time series changes with time, the single function that we estimate may not be relevant all the time. For instance, if we think about the number of visitors to the nearby park in a day as a time series, we know that those patterns are going to be very different for pre- and post-pandemic periods. In the ML world, this phenomenon is called **concept drift**.

Intuitively, we can understand that it is easier to forecast a stationary series than a non-stationary series. But here comes the punchline: in the real world, almost all time series do not satisfy the stationarity assumption—more specifically, the **strict stationarity** assumption. Strict stationarity is when all the statistical properties such as the mean, variance, skewness, and so on do not change with time. Many times, this strict stationarity assumption is relaxed in favor of **weak stationarity**, where we only stipulate that the mean and the variance of the time series do not change with time.

There are four main questions we can ask ourselves to check whether our time series is stationary or not:

- Does the mean change over time? Or in other words, is there a trend in the time series?
- Does the variance change over time? Or in other words, is the time series heteroscedastic?
- Does the time series exhibit periodic changes in the mean? Or in other words, is there seasonality in the time series?
- Does the time series have a unit root?

Out of these questions, the first three can be ascertained using a simple visual inspection. **Unit roots** are more difficult to understand. We will take a deeper look at unit roots shortly. Let's take a look at a few time series and check whether we can tell whether they are stationary or not via visual inspection (you can note your answers):

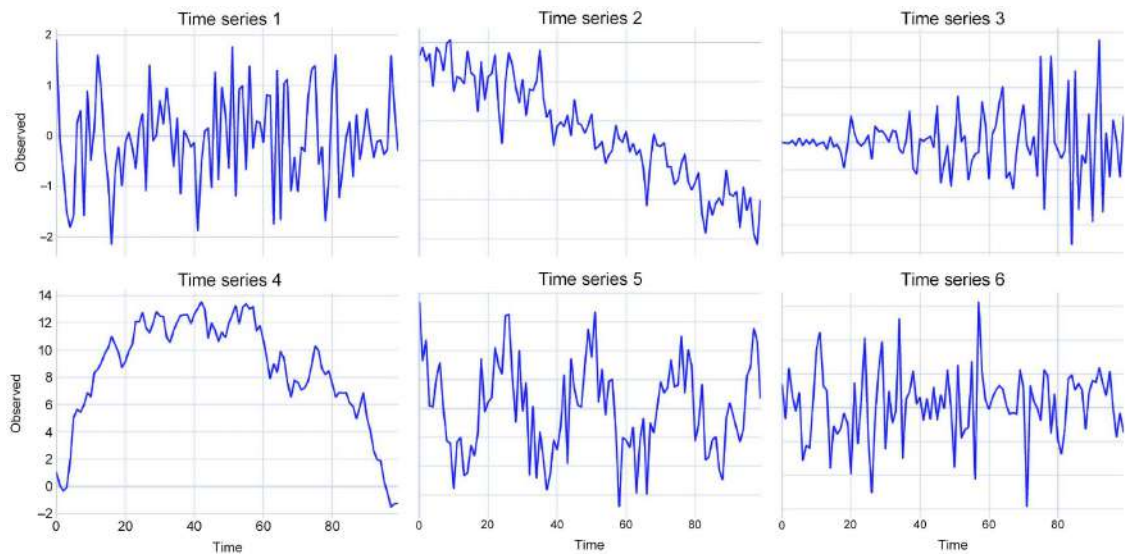


Figure 7.1: Testing your understanding of stationarity

 **Quick tip:** Need to see a high-resolution version of this image? Open this book in the next-gen Packt Reader or view it in the PDF/ePub copy.

 **The next-gen Packt Reader and a free PDF/ePub copy** of this book are included with your purchase. Unlock them by scanning the QR code below or visiting <https://www.packtpub.com/unlock/9781835883181>.



Now, check your responses and see how many of them you guessed correctly. If you got at least four out of six, you are doing great with your intuition of stationarity:

- **Time series 1** is *stationary* as it is a white noise process that, by definition, has zero mean and a constant variance. It checks our checklist for the first three questions.
- **Time series 2** is *non-stationary* as it has an obvious downward linear trend. This means that the mean of the series at the beginning of the series is not the same toward the end. So, it fails our first question in the checklist.
- **Time series 3** may look stationary at first because it is essentially oscillating around 0, but the oscillations are wider as we progress through time. This means that it has an increasing variance—or in other words, it is heteroscedastic. So, although this answers our first question, it doesn't pass our second check of having constant variance. Hence, it is *non-stationary*.
- Now, we are coming to the problem child—**Time series 4**. At first glance, we may think it is stationary because even though it had a trend in the beginning, it also reversed it, making the mean almost constant. And it's not obvious that the variance is also widely varying. But this is a time series with a unit root (we will talk about this in detail later, in the *Unit roots* section), and typically, unit root time series are difficult to judge visually.
- **Time series 5** answers the first two questions—the constant mean and constant variance—but it has a very obvious seasonal pattern and hence is *non-stationary*.
- **Time series 6** is another white noise process, included just to trick you. This is also *stationary*.

When we have hundreds, or even millions, of time series, we can't practically do a visual inspection to ascertain whether they are stationary or not. So, now, let's look at a few ways of detecting these key properties using statistical tests and also how to try and correct them.



Although we are talking about correcting or making a time series stationary, it is not always essential to do that in the ML paradigm because some of these can be handled by using the right kind of features in the model. Whether to make a series stationary or not is a decision we will have to make after experimenting with the techniques. This is because, as you will see, while there are advantages to making a series stationary, there are also disadvantages to using some of these techniques, as we will see when we discuss each transformation in detail.



Notebook alert:

To follow along with the complete code, use the `02-Dealing_with_Non-Stationarity.ipynb` notebook in the `Chapter06` folder.

Detecting and correcting for unit roots

Let's talk about unit roots first since this is what is most commonly tested for stationarity. Time series analysis has its roots in econometrics and statistics and unit root is a concept derived directly from those fields.

Unit roots

Unit roots are quite complicated to understand fully but to develop some intuition, we can look at a simplification. Let's consider an autoregressive model of order 1 (AR(1) model):

$y_t = \phi y_{t-1} + \epsilon_t$, where ϵ_t is the white noise and ϕ is the AR coefficient.

If we think about the different values of ϕ in the equation, we can come up with three scenarios (Figure 7.2):

1. $|\phi| > 1$: When $|\phi|$ is greater than 1, every successive value in the time series is multiplied by a number greater than 1, which means it will have a strong and rapidly increasing/decreasing trend and thereby be non-stationary.
2. $|\phi| < 1$: When $|\phi|$ is less than 1, every successive value in the time series is multiplied by a number less than 1, which means over the long term, the mean of the series trends to zero and will oscillate around it. Therefore, it is stationary.
3. $|\phi| = 1$: When $|\phi|$ is equal to 1, things become trickier. When $|\phi| = 1$ for an AR(1) model, this is known as it having a unit root and the equation becomes $y_t = y_{t-1} + \epsilon_t$. This is called random walk in econometrics and is a very popular kind of time series in financial and economic domains. Mathematically, we can prove that such a series will have a constant mean but a non-constant variance:

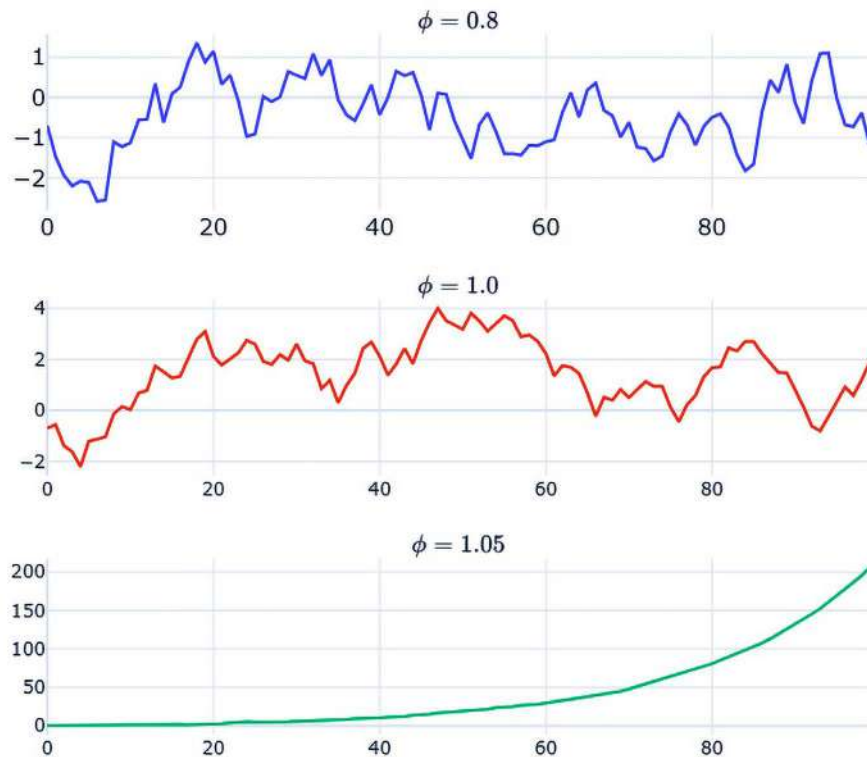


Figure 7.2: Autoregressive time series with different ϕ parameters. Top: Scenario 1, $\phi < 1$; Middle: Scenario 2, $\phi = 1$; Bottom: Scenario 3, $\phi > 1$

While we discussed unit roots in an AR(1) process, we can extend the same intuition to multiple lags or an AR(p) model. Calculating and testing unit roots is more complicated there, but still possible.

So, now that we know what a unit root is, how can we statistically test this? This is where the Dickey-Fuller test comes in.

The Augmented Dickey-Fuller (ADF) test

The null hypothesis in this test is that the ϕ in an AR(1) model of the time series is equal to 1, and by extension non-stationary. The alternate hypothesis is that the ϕ in the AR(1) model is less than 1. The ADF test takes the Dickey-Fuller test and extends it to an AR(p) model because most time series are not defined by just one lag of the time series. This is the standard and most popular statistical test to check for unit roots. The core of the test involves running a regression on the lags of the time series and calculating the statistic on the variance of the residuals.

Let's see how we can do this in Python using statsmodels:

```
from statsmodels.tsa.stattools import adfuller
result = adfuller(y)
```

result from adfuller is a tuple that contains the *test statistic*, *p-value*, and *critical values* at different confidence levels. Here, we are most interested in the p-value, which is an easy and practical way to check whether the null hypothesis is rejected or not. If $p < 0.05$, there is a 95% probability that the series does not have a unit root; the series is stationary from a unit root perspective.

To make this process even easier, we have included a method called `check_unit_root` in `src.transforms.stationary_utils` that does the inference for you (comparing the returned probability with the confidence and rejecting or accepting the null hypothesis) and returns a `namedtuple` with a Boolean attribute called `stationary`, and the entire results from statsmodels in `results`:

```
from src.transforms.stationary_utils import check_unit_root
# We pass the time series along with the confidence with which we need the
results
check_unit_root(y, confidence=0.05)
```

Now that we've learned how to check whether a series has a unit root or not, how do we make it stationary? Let's look at a few transforms that help us do that.

Differencing transform

The differencing transform is a very popular transform to make a time series stationary, or at least get rid of unit roots. The concept is simple: we transform the time series from the domain of observation to the domain of change in observations. The differencing transform subtracts subsequent observations from one another:

$$z_t = y_t - y_{t-1}$$

Differencing helps us stabilize the mean of the time series and, with that, reduce or eliminate trend and seasonality. Let's see how differencing can make a series stationary.

Let the time series in question be $y_t = \beta_0 + \beta_1 t + \epsilon_t$, where β_0 and β_1 are the coefficients and ϵ is white noise. From this equation, we can see that time, t , is part of the equation, making y_t a time series with a trend. So, the differenced time series z would be as follows:

$$\begin{aligned} z_t &= y_t - y_{t-1} \\ &= (\beta_0 + \beta_1 t + \epsilon_t) - (\beta_0 + \beta_1(t-1) + \epsilon_{t-1}) \\ &= \beta_1 + (\epsilon_t - \epsilon_{t-1}) \end{aligned}$$

What we need to look for in this new equation is that there is no mention of t . This means that the dependence on t , which created the trend, has been removed, and now the time series has constant mean and variance at any point in time.

Differencing does not remove all kinds of non-stationarity but works for the majority of time series. But there are a few drawbacks to this approach as well. One of them is that we lose the scale of the time series while modeling. Many times, the scale of the time series holds some information that is useful for forecasting. For instance, in a supply chain, SKUs with higher sales exhibit a different kind of pattern from SKUs with lower sales and when we do differencing, this information about the distinction is lost.

Another drawback is more from an operational point of view. When we use differencing for forecasting, we also need to inverse the transform after we get the differenced output from the model. This is an additional layer of complexity that we have to manage. One way is to keep the most recent observation in memory and keep adding the differences to it to inverse the transform. Another way is to have y_{t-1} ready for every t that we need to inverse transform and keep adding the difference to y_{t-1} .

We have implemented the latter using the datetime index as a key to align and fetch the y_{t-1} observation in `src.transforms.target_transformations.py` in this book's GitHub repository. Let's see how we can use it:

```
from src.transforms.target_transformations import
AdditiveDifferencingTransformer
diff_transformer = AdditiveDifferencingTransformer()
# [1:] because differencing reduces the length of the time series by one
y_diff = diff_transformer.fit_transform(y, freq="1D")[1:]
```

`y_diff` will have the transformed series. To get back to the original time series, we can call `inverse_transform` using `diff_transformer`. The associated notebook has examples and plots to see how the differencing changes the time series.

Here, we saw differencing as the process of subtracting subsequent values in the time series. But we can also do differencing with other operators such as division (y_t / y_{t-1}), which is implemented in the `src.transforms.target_transformations.py` file as `MultiplicativeDifferencingTransformer`. We can also experiment with these transforms to check whether these work best for your dataset.

Although differencing solves the majority of stationarity issues, it's not guaranteed to take care of all kinds of trends (non-linear or piecewise trends), seasonality, and so on. Sometimes, we may not want to difference the series but still handle trends and seasonality. So, let's see how we can detect and remove trends in a time series.

Detecting and correcting for trends

In *Chapter 5, Time Series Forecasting as Regression*, we talked about forecasting being a difficult problem because it is intrinsically an extrapolation problem. Trends are one of the major contributors to forecasting being an extrapolation problem. If we have a time series that is trending upward, any model that attempts to forecast it needs to extrapolate beyond the range of values it has seen during training. ARIMA handles this using autoregression, whereas exponential smoothing handles it by modeling the trend explicitly. But standard regression may not be naturally suited to extrapolation. However, with suitable features, such as lags, it can start to do that. But if we can confidently estimate and extract a trend in the time series, we can simplify the problem we have to apply regression to by detrending the time series.

But before we move ahead, it is worth learning about two major types of trends.

Deterministic and stochastic trends

Let's take the simple $AR(1)$ model we saw earlier to develop intuitions about this one too. Earlier, we saw that having $\phi > 1$ in an $AR(1)$ model leads to a trend in the time series. But another way we can think about a trending time series is if we include time as an ordinal variable in the equation defining the time series. For instance, let's consider two time series:

Time series 1:

$$y_t = \phi y_{t-1} + \epsilon_t; \epsilon_t \sim \mathcal{N}(0, \sigma^2)$$

Time series 2:

$$y_t = \beta_0 + \beta_1 t + \epsilon_t; \epsilon_t \sim \mathcal{N}(0, \sigma^2)$$

These can be seen in the following graphs:

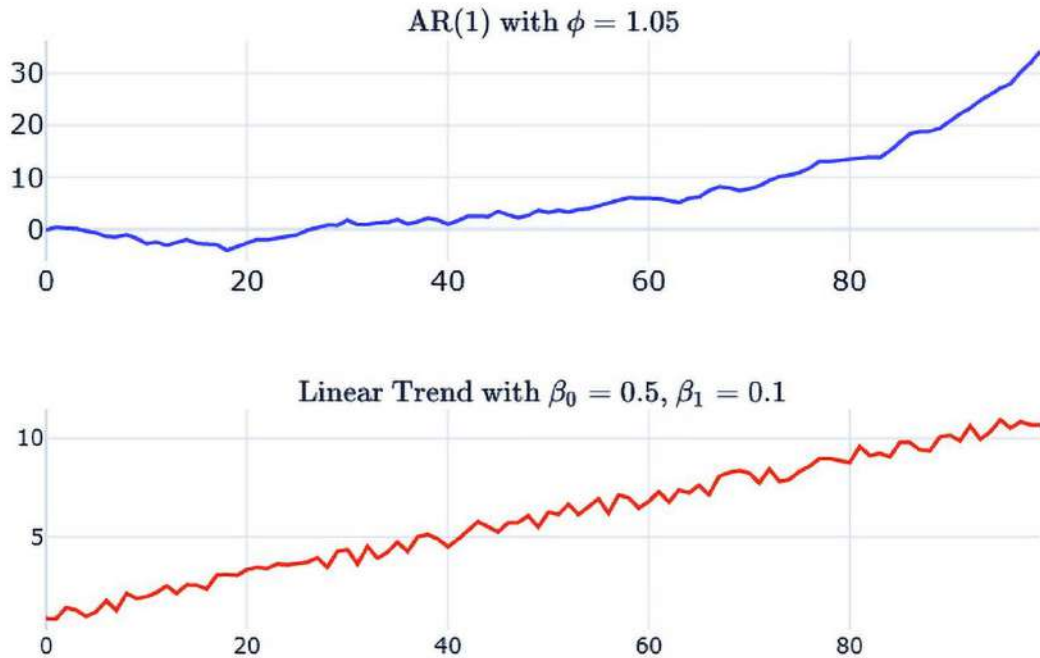


Figure 7.3: Top: Stochastic trend; Bottom: Deterministic trend

We saw both of these equations earlier; *Time series 1* is the $AR(1)$ model, while *Time series 2* is the time series equation we chose to illustrate differencing. We already know that for $\phi > 1$, both *Time series 1* and *Time series 2* have trends. But there is a difference between the two trends. In *Time series 2*, the trend is constant and can be perfectly modeled. In this case, just a linear fit would explain the trend perfectly. But in *Time series 1*, the trend is not something that can be explained by a simple linear fit. It is inherently dependent on the previous value of the time series that has ε_{t-1} and hence is stochastic. Therefore, *Time series 2* has a deterministic trend and *Time series 1* has a stochastic trend.

We can use the same ADF test we saw earlier in this chapter to check whether a time series has deterministic or stochastic trends. Without going into the math of the statistical test, we know that it tests for a unit root by fitting an $AR(p)$ model to the time series. There are a few variants of this test that we can specify using the regression parameter in the `statsmodels` implementation. This parameter takes in the following values:

- `c`: This means we are including a constant intercept in the $AR(p)$ model. Practically, this means that we will be considering a time series as stationary even if the series is not around zero. This is the default setting in `statsmodels`.
- `n`: This means we do not even include a constant intercept in the $AR(p)$ model.
- `ct`: If we supply this option, the $AR(p)$ model will also have a constant intercept and a linear, deterministic trend component. What this means is that even if there is a deterministic trend in the time series, it will be ignored and the series will be tested as stationary.
- `ctt`: This is when we include a constant intercept—that is, a linear and quadratic trend.

So, if we run an ADF test with `regression="c"`, it will be non-stationary. Now, if we run the ADF test with `regression="ct"`, it will come out as stationary. This means that when we removed a deterministic trend from the time series, it became stationary. This test is what we can use to determine whether a trend that we observe in a time series is deterministic or stochastic. In the *Further reading* section, we have provided a link to a blog post by *Fabian Kostadinov*, where he experiments with a few time series to make the distinction between the different variants of ADF tests clear.

We have implemented this test in `src.transforms.stationary_utils` as `check_deterministic_trend`, which does the inference for you and returns a `namedtuple` with a Boolean attribute of `deterministic_trend`. The `namedtuple` also includes the raw results from the two `adfuller` tests we did under `adf_res` and `adf_ct_res` if you want to investigate further. Let's see how we can use this test:

```
check_deterministic_trend(y, confidence=0.05)
```

This will tell us whether the trend is stationary or deterministic. Now, let's look at a couple of ways to identify and statistically test trends (irrespective of whether it is deterministic or not) in a time series.

Kendall's Tau

Kendall's Tau is a measure of correlation but is carried out on the ranks of the data. Kendall's Tau is a non-parametric test and therefore does not make assumptions about the data. The correlation coefficient, Tau, returns a value between -1 and 1, where 0 shows no relationship and 1 or -1 is a perfect relationship. We will not dive into the details of how Kendall's Tau is calculated or how the significance test is done as this is outside the scope of this book. The *Further reading* section contains a link that explains this well.

In this section, we will see how we can use Kendall's Tau to measure the trend in our time series. As mentioned earlier, Kendall's Tau calculates a rank correlation between two variables. If we chose one of those variables as the time series and set the other as the ordinal representation of time, the resulting Kendall's Tau would represent the trend in the time series. An additional benefit is that the higher the value of Kendall's Tau is, the stronger we expect the trend to be.

scipy has an implementation of Kendall's Tau that we can use as follows:

```
import scipy.stats as stats
tau, p_value = stats.kendalltau(y, np.arange(len(y)))
```

We can compare the returned p-value to our required confidence (typically, this is 0.05) and say that if `p_value < confidence`, we conclude that the trend is statistically significant. The sign of tau tells us whether this is an increasing trend or a decreasing one.

We have made an implementation of Kendall's Tau in `src.transforms.stationary_utils` as `check_trend`, which checks for the presence of a trend for you. The only parameters we need to provide are as follows:

- `y`: The time series to check
- `confidence`: The confidence level against which the resulting p-value will be checked

A few more parameters are there, but those are for the **Mann-Kendall (M-K)** test, which will be explained next.

Let's see how we can use this test:

```
check_trend(y, confidence=0.05)
```

This method also checks whether the trend that has been identified is deterministic or stochastic and calculates the direction of the trend. The result is returned as a `namedtuple` with the following parameters:

- `trend`: This is a Boolean flag signifying the presence of a trend.
- `direction`: This will be either `increasing` or `decreasing`.
- `slope`: This is the slope of the estimated trend line. For Kendall's Tau, it will be the Tau.
- `p`: This is the p-value of the statistical test.
- `deterministic`: This is a Boolean flag signifying the deterministic trend.

Now, let's look at the Mann-Kendall test.

Mann-Kendall test (M-K test)

The Mann-Kendall test is used to check for the presence of a monotonic upward or downward trend. And since the M-K test is a non-parametric test, like Kendall's Tau, there is no assumption of normality or linearity. The test is done by analyzing the signs between consecutive points in the time series. The crux of the test is the idea that in the presence of a trend, the sign values, if summed up, increase or decrease constantly.

Although non-parametric, there were a few assumptions in the original test:

- There is no auto-correlation in the time series
- There is no seasonality in the time series

Numerous alterations have been made to the original tests to tackle these problems over the years and a lot of such alterations, along with the original test, have been implemented at <https://github.com/mmhs013/pyMannKendall>. They are available in pypi as `pymannkendall`.

Pre-whitening is a common technique used to remove the autocorrelation in a time series. In a nutshell, the idea is as follows:

1. Identify ϕ with an AR(1) model
2. $y_t^{\text{prewhiten}} = y_t - \phi * y_{t-1}$

M. Bayazit and B. Önöz (2007) suggested not using pre-whitening before doing the M-K test if the sample size is larger than 50 and if the trend is strong enough ($\text{slope} > 0.01$). For seasonal data, a seasonal variant of the M-K test has also been implemented in `pymannkendall`.



Reference check:

The research paper by M. Bayazit and B. Önöz is cited in the *References* section under reference 1.

The same method we discussed earlier, `check_trend`, also implements M-K tests that can be enabled by setting `mann_kendall=True`. However, one thing we need to keep in mind is that the M-K test is considerably slower than Kendall's Tau, especially for long time series. There are a couple more parameters specific to the M-K test:

- `seasonal_period`: The default value is `None`. But if there is seasonality, we can provide `seasonal_period` here and the seasonal variant of the M-K test will be retrieved.
- `prewhiten`: This is a Boolean flag that's used to pre-whiten the time series before applying the M-K test. The default value is `None`. In that case, using the condition we discussed earlier ($N > 50$), we decide whether to pre-whiten or not. If we explicitly pass `True` or `False` here, it will be respected.

Let's see how we can use this test:

```
check_trend(y, confidence=0.05, mann_kendall=True)
```

The result is returned as a `namedtuple` with the following parameters:

- `trend`: This is a Boolean flag signifying the presence of a trend.
- `direction`: This will be either `increasing` or `decreasing`.
- `slope`: This is the slope of the estimated trend line. For the M-K test, it will be the slope estimated using the Theil-Sen estimator.
- `p`: This is the p-value of the statistical test.
- `deterministic`: This is a Boolean flag signifying the deterministic trend.

Let's see an example where we applied both these tests on a time series (see 02-Dealing_with_Non-Stationarity.ipynb for the full code):

```
# y_unit_root is the a synthetic unit root timeseries
y_unit_root.plot()
plt.show()
```

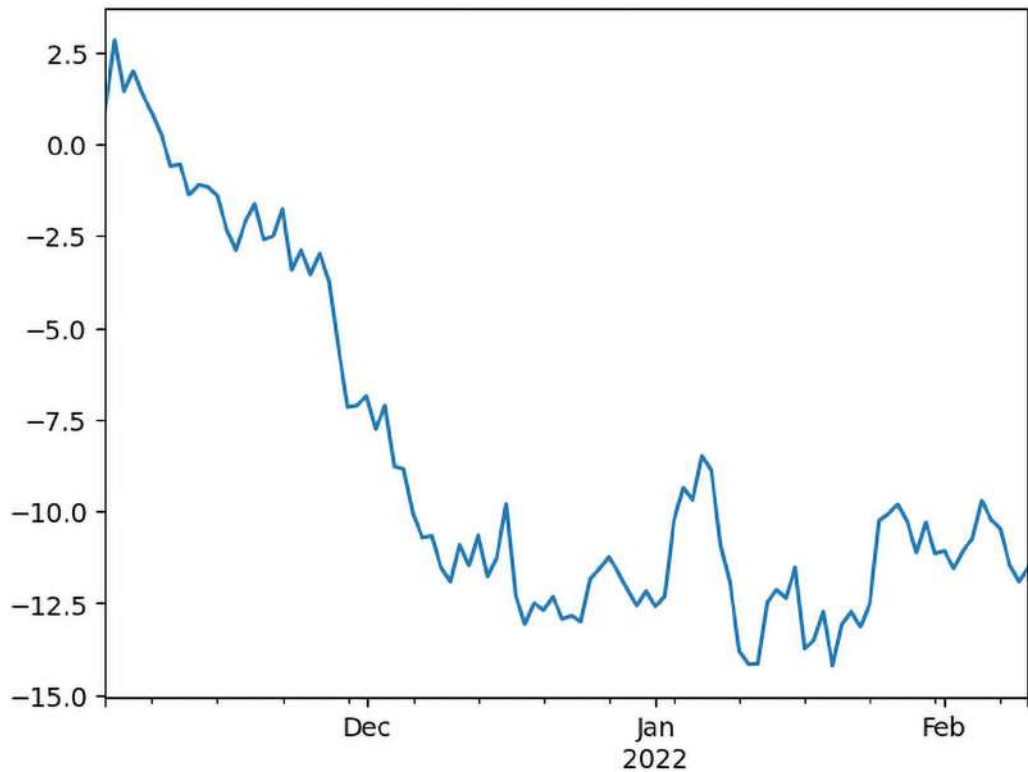


Figure 7.4: M-K test

```
kendall_tau_res = check_trend(y_unit_root, confidence=0.05)
mann_kendall_res = check_trend(y_unit_root, confidence=0.05, mann_kendall=True)
print(f"Kendalls Tau: Trend: {kendall_tau_res.trend} | Direction: {kendall_tau_res.direction} | Deterministic: {kendall_tau_res.deterministic}")
print(f"Mann-Kendalls: Trend: {mann_kendall_res.trend} | Direction: {mann_kendall_res.direction} | Deterministic: {mann_kendall_res.deterministic}")
## Output
```

```
>> Kendalls Tau: Trend: True | Direction: decreasing | Deterministic: False
>> Mann-Kendalls: Trend: True | Direction: decreasing | Deterministic: False
```

It would benefit you if you can generate some time series, or even pick a few time series you have come across and use these functions to see how it works and how the results help you. The associated notebook has some examples to get you started. You can observe how the direction and slope are different for different types of trends.

Now that we know how to detect a trend, let's look at detrending.

Detrending transform

If the trend is deterministic, removing the trend would add some value to the modeling procedure. In *Chapter 3, Analyzing and Visualizing Time Series Data*, we discussed detrending as it was an integral part of the decomposition we were doing. But techniques such as moving average or LOESS regression have one drawback—they can't extrapolate. But if we are considering a deterministic linear (or even polynomial) trend, it can be easily estimated by using linear regression. The added advantage here is that the trend that is identified can easily be extrapolated.

The procedure is simple: we regress the time series on the ordinal representation of time and extract the parameters. Once we have these parameters, using the dates, we can extrapolate the trend to any point in the future. The core logic in Python is shown below:

```
# y is the time series we are detrending
x = np.arange(len(y))
# degree is the degree of trend we are estimating. Linear, or polynomial
# Fitting a regression on y using a linearly increasing x
linear_params = np.polyfit(x=x, y=y, deg=degree)
# Extract trend using fitted parameters
trend = get_trend(y)
# Now this extracted trend can be removed from y
detrended = y - trend
```

We have made and implemented this detrender as a transformer in `src.transforms.target_transformations.py` as `DetrendingTransformer`. You can see how we have implemented it in the GitHub repository. Now, let's see how we can use it:

```
from src.transforms.target_transformations import DetrendingTransformer
detrending_transformer = DetrendingTransformer(degree=1)
y_detrended = detrending_transformer.fit_transform(y, freq="1D")
```

`y_detrended` will contain the detrended series. To get the original time series back, we can call `inverse_transform` using `detrending_transformer`. The associated notebook has examples and plots to see how the detrending changes the time series.



Best practice:

We have to be careful with the trend assumptions, especially if we are forecasting for the long term. Even a linear trend assumption can lead to an unrealistic forecast because trends don't continue the same way forever in the real world. It is always advisable to dampen the trend by some factor, ϕ , to be conservative in our extrapolation of the trend. This dampening can be as simple as $f_{t+h}^{\text{damped}} = f_{t+h} \times \phi^h$, where $\phi < 1$.

Another key aspect that makes a time series non-stationary is seasonality. Let's look at how to identify seasonality and remove it.

Detecting and correcting for seasonality

A vast majority of real-world time series have seasonality such as retail sales, energy consumption, and so on. And generally, the presence or absence of seasonality comes as part of the domain knowledge. But when we are working with a time series dataset, the domain knowledge becomes slightly diluted. The majority of time series may exhibit seasonality, but that doesn't mean every time series in the dataset is seasonal. For instance, within a retail dataset, there might be items that are seasonal and some items that are not. Therefore, when working with a time series dataset, being able to determine whether a particular time series is seasonal or not has some value.

Detecting seasonality

There are two popular ways to check for seasonality, apart from just eyeballing it: autocorrelation and fast Fourier transform. Either is equally capable of identifying the seasonality period automatically. For our discussion, we'll cover the autocorrelation method and examine how we can use that to determine seasonality.

Autocorrelation, as explained in *Chapter 3, Analyzing and Visualizing Time Series Data*, is the correlation of a time series to its lagged values. Typically, we expect the correlation to be higher in the immediate lags (lag 1, lag 2, and so on) and gradually die down as we move farther into the past. But for time series with seasonality, we will also see a spike in the seasonal periods.

Let's understand this by looking at an example. Consider a synthetic time series that is just white noise combined with a sinusoidal signal with a seasonality cycle of 25 (identical to the seasonal time series we saw earlier, in *Figure 7.1*):

```
#WhiteNoise + Seasonal
y_random = pd.Series(np.random.randn(length), index=index)
t = np.arange(len(y_random))
y_seasonal = (y_random+1.9*np.cos((2*np.pi*t)/(length/4)))
```

If we plot the **autocorrelation function (ACF)** for this time series, it will look as follows (the code to calculate and plot this can be found in the `02-Dealing_with_Non-Stationarity.ipynb` notebook):

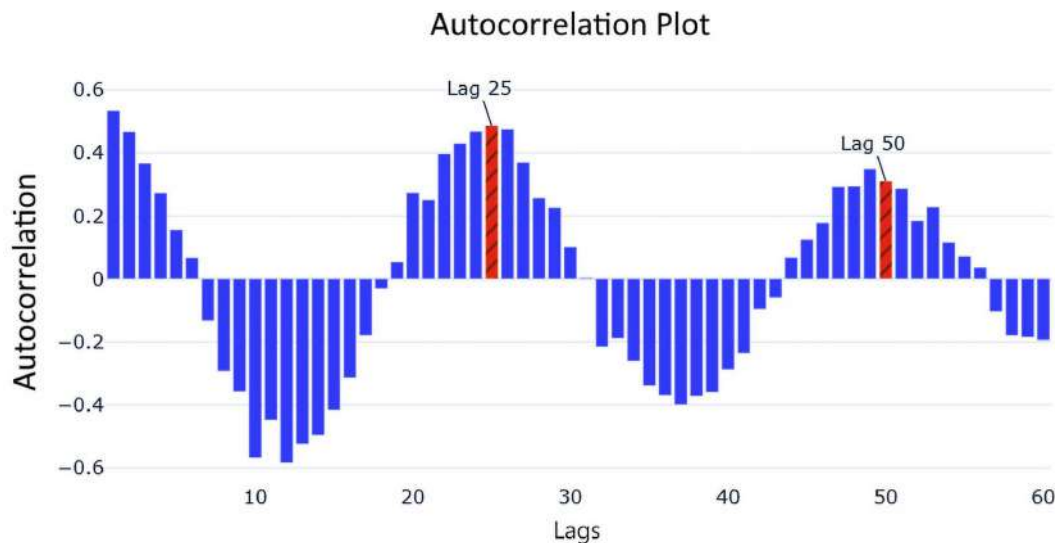


Figure 7.5: Autocorrelation plot of the synthetic time series with a seasonality cycle of 25

We can see that apart from the first few lags, the autocorrelation increases as we approach the seasonal cycle and peaks at the exact seasonality. We can use this property of the ACF to detect seasonality. `darts`, a time series forecasting library, has an implementation of this technique that identifies seasonality. But since it was designed to work for the time series data structure of `darts`, we have adapted the same logic to work on regular pandas series in `src.transforms.stationary_utils.py` under the name `check_seasonality`. The implementation can do two kinds of seasonality checks. It can take a `seasonality_period` as input and verify whether a seasonality corresponding to that `seasonality_period` exists in the data or not. If we do not give a `seasonality_period` ahead of time, it will return to you the shortest `seasonality_period` that is statistically significant.

The procedure, at a high level, does the following:

1. It calculates the ACF.
2. It finds all the relative maxima in the ACF. A relative maximum is a point where the function changes direction from increasing to decreasing.
3. It checks whether the provided `seasonal_period` is a relative maximum. If not, we conclude there is no seasonality associated with `seasonality_period`.
4. Now, we take the assumption that the ACF is normally distributed and compute the upper limit at the specified confidence level. The upper bound is given by:

$$UB = z_{1-\frac{\alpha}{2}} \times SE(r_h)$$

where r_h is the estimated autocorrelation at lag h , SE is the standard error, and $z_{1-\frac{\alpha}{2}}$ is the quantile of the normal distribution based on the required confidence, α . The SE is approximated using Bartlett's formula (for the math behind this, head over to the *Further reading* section).

- Each of our candidates for `seasonality_period` is checked against this upper limit and the ones that are above this limit are deemed statistically significant.

There are only three parameters for this function, apart from the time series itself:

- `max_lag`: This specifies the maximum lag that should be included in the ACF and subsequent search for seasonality. This should be at least one more than the expected seasonality period.
- `seasonal_period`: This is where we give our intuition of the seasonality period from domain knowledge and the function verifies that assumption for us.
- `confidence`: This is the standard statistical confidence level. The default value is 0.05.

Let's see how we can use this function on the same data we saw in *Figure 7.4* (with a seasonal period of 25). This will give you a namedtuple with `seasonal`, a Boolean flag to indicate seasonality, and `seasonal_periods`, the seasonal periods with significant seasonality, as parameters:

```
# Running the function without specifying seasonal period to identify the
seasonality
seasonality_res = check_seasonality(y_seasonal, max_lag=60, confidence=0.05)
print(f"Seasonality identified for: {seasonality_res.seasonal_periods}")
## Output
```

```
>> Seasonality identified for: 25
```

```
This function can also be used to verify if your assumption about the
seasonality is right.
# Running the function specifying seasonal period to verify
seasonality_res = check_seasonality(y_seasonal, max_lag=30, seasonal_period=25,
confidence=0.05)print(f"Seasonality Test for 25th lag: {seasonality_res.
seasonal}")
## Output
```

```
>> Seasonality Test for 25th lag: True
```

Now that we know how to identify and test for seasonality, let's talk about deseasonalizing.

Deseasonalizing transform

In *Chapter 3, Analyzing and Visualizing Time Series Data*, we reviewed techniques for seasonal decomposition. We can use the same techniques here as well, but with just one tweak. Earlier, we were not concerned with projecting the seasonality into the future. But when we are using deseasonalizing in forecasting, it is essential to be able to project it into the future as well. We are in luck since projecting the seasonal cycle forward is trivial. This is because we are looking at a fixed seasonality profile that will always keep repeating in the seasonal cycle. For instance, if we identified a seasonality profile for the 12 months of a year (yearly seasonality at monthly frequency data), the seasonality that's extracted for these 12 months will just repeat itself in chunks of 12 months.

Using this property, we have implemented a transformer in `src.transforms.target_transformations.py` as `DeseasonalizingTransformer`. There are a few parameters and properties that we need to be aware of:

- `seasonality_extraction`: This transformer supports two ways of extracting seasonality—"period_averages", where the seasonality profile is estimated using seasonal averaging, and "fourier_terms", where we regress on Fourier terms to extract the seasonality.
- `seasonality_period`: Depending on the technique we use for seasonality extraction, this can either be an integer or a string. If "period_averages", this parameter denotes the number of periods after which the seasonal cycle repeats. If "fourier_terms", this denotes the seasonality to be extracted from the datetime index. `pandas` datetime properties such as `week_of_day`, `month`, and so on can be used to specify the most prominent seasonality. Similar to `FourierDecomposition`, which we saw earlier, we can also omit this parameter and provide custom seasonality in the `fit/transform` methods in the implementation.
- `n_fourier_terms`: This parameter specifies the number of Fourier terms to be included in the regression. Increasing this parameter makes the fitted seasonality more complex.
- There is no detrending in this implementation because we already saw a `DetrendingTransformer`. This implementation expects any trend to be removed before using the `fit` function.

Let's see how we can use it:

```
from src.transforms.target_transformations import DeseasonalizingTransformer
deseasonalizing_transformer = DeseasonalizingTransformer(seasonality_
extraction="period_averages", seasonal_period=25)
y_deseasonalized = deseasonalizing_transformer.fit_transform(y, freq="1D")
```

`y_deseasonalized` will have the deseasonalized time series. To get back to the original time series, we can use the `inverse_transform` function. Typically, this can be used to add the seasonality back after making predictions.

Best practice:



Modeling seasonality can be done either separately, as discussed here, or by using the seasonal features that we discussed earlier in this chapter. Although the final evaluation on which one works better has to be found out empirically for each dataset, we can have a few rules of thumb/guidelines to decide on priority.

When we have enough data, letting the model learn seasonality as part of the main forecasting problem seems to work better. But in cases where data is not that rich, extracting seasonality separately before feeding it to an ML model works well.

When the dataset has varied seasonality (different seasonal cycles for different time series), then it should be treated accordingly. Either deseasonalize each time series separately or split the global ML model into different local models, each with its own seasonality pattern.

The last aspect that we talked about earlier is heteroscedasticity. Let's quickly take a look at that as well.

Detecting and correcting for heteroscedasticity

Despite having a scary name, heteroscedasticity is a simple enough concept. It is derived from ancient Greek, where *hetero* means *different* and *skedasis* means *dispersion*. True to its name, heteroscedasticity is defined when the variability of a variable is different across another variable. In the context of a time series, we say a time series is heteroscedastic when the variability or dispersion of the time series varies with time. For instance, let's think about the spending of a household over a number of years. In these years, this particular household went from being poor to middle class and finally upper middle class. When the household was poor, the spending was less and only on essentials, and because of that, the variability in spending was less. But as they approached the upper middle class, the household could afford luxuries, which created spikes in the time series and therefore higher variability. If we refer back to *Figure 7.1*, we can see what a heteroscedastic time series looks like.

But in addition to visual inspection, it would be neat if we could carry out an automated statistical test to ascertain heteroscedasticity.

Detecting heteroscedasticity

There are many ways to detect heteroscedasticity, but we will be using one of the most popular techniques, known as the **White test**, proposed by Halbert White in 1980. The White test uses an auxiliary regression task to check for constant variance. We run an initial regression using some covariates and calculate the residuals of this regression. Then, we fit another regression model with these residuals as the target and the covariates used in the first regression, and their squares and cross-products. The final statistic is estimated by using the R^2 value of this auxiliary regression. For a detailed account of the test, head over to the *Further reading* section; for the rigorous mathematical procedure, the research paper is cited in the *References* section.



Reference check:

To learn more about the rigorous mathematical procedure of the White test, take a look at the research paper cited in the *References* section under reference 2.

In the context of a time series, we adapt this formulation by using a deterministic trend model. The initial regression is done by using time as an ordinal variable and the residuals are used to carry out the White test. The White test has an implementation in `statsmodels` of `het_white`, which we will be using to carry out this test. The `het_white` test returns two statistics and p-values—Lagrangian Multiplier and F-Statistic. Lagrangian Multiplier tests if there is any relationship between the variance of the residuals and the independent variables in the regression model. F-Statistic compares the fit of your original model to a model allowing for varying error variance. A p-value less than confidence in either of these tests indicates heteroscedasticity. But to be conservative, we can also use both tests and mark something as heteroscedastic only when both of the p-values are less than confidence.

We have wrapped all of this in a helpful function in `src.transforms.stationary_utils` as `check_heteroscedasticity`, which has only one additional parameter—confidence. Let's see the core implementation of the method in Python:

```
import statsmodels.api as sm
# Fitting a linear trend regression
x = np.arange(len(y))
x = sm.add_constant(x)
model = sm.OLS(y,x)
results = model.fit()
# Using the het_white test on residuals
lm_stat, lm_p_value, f_stat, f_p_value = het_white(results.resid, x)
# Checking if both p values are less than confidence
if lm_p_value < confidence and f_p_value < confidence:
    hetero = True
else:
    hetero = False
```

Now, let's see how we can use this function:

```
from src.transforms.stationary_utils import check_heteroscedasticity
check_heteroscedasticity(y, confidence=0.05)
```

This returns a namedtuple with the following parameters:

- **Heteroscedastic:** A Boolean flag indicating the presence of heteroscedasticity
- **lm_statistic:** The **Lagrangian Multiplier (LM)** statistic
- **lm_p_value:** The p-value associated with the LM statistic



Best practice:

The heteroscedasticity test we are doing only considers a trend in the regression and therefore, in the presence of seasonality, may not work very well. It is advised to deseasonalize the data before applying the function.

Detecting heteroscedasticity was the easier part. There are a few transforms that attempt to remove heteroscedasticity but with advantages and disadvantages. Let's take a look at a few such transforms.

Log transform

Log transform, as the name suggests, is about applying a logarithm to the time series. There are two main properties of a log transform—variance stabilization and reducing skewness—thereby making the data distribution more *normal*. And out of these, we are more interested in the first property because that is what combats heteroscedasticity.

Log transforms are typically known to reduce the variance of the data and thereby remove heteroscedasticity in the data. Intuitively, we can think of a log transform as something that *pulls in* the extreme values on the right of the histogram, at the same time stretching back the very low values on the left of the histogram.

But it has been shown that the log transform does not always stabilize the variance. In addition to that, the log transform poses another challenge in ML. The optimization of loss now happens on the log scale. Since the log transformation compresses the lower end of the value range more than the higher one, the learned model can be less sensitive to errors in the lower range as compared to the higher one. Another key disadvantage is that the log transform can only be applied to strictly positive data. And if any of your data is zero or less than zero, then you will need to offset the whole distribution by adding some constant, M , and then applying the transform. This will also create some disturbance in the data, which can have adverse effects.

The bottom line is that we should be careful when applying a log transform. We have implemented a transformer in `src.transforms.target_transformations.py` as `LogTransformer` with just one parameter, `add_one`, which adds one before the transform and subtracts one after the inverse. The key logic in Python is as simple as applying an `np.log1p` or `np.log` function in the transform and reversing it with `np.expm1` or `np.exp`, respectively:

```
# Transform
np.log1p(y) if self.add_one else np.log(y)
# Inverse Transform
np.expm1(y) if self.add_one else np.exp(y)
y_log = log_transformer.fit_transform(y)
```

All we have done is wrap this into a nice and easy-to-use transformer. Let's see how we can use it:

```
from src.transforms.target_transformations import LogTransformer
log_transformer = LogTransformer(add_one=True)
y_log = log_transformer.fit_transform(y)
```

`y_log` is the log-transformed time series. We can call `inverse_transform` to get the original time series back.

Box-Cox transformation

The log transform, although effective and common, is very *strong*. But the log is not the only monotonic transform that we can use. There are many other transforms, such as y^2 , $\frac{1}{y}$, $\frac{1}{\sqrt{y}}$ and so on, which are collectively part of the family of power transforms. One set of transforms that is very famous and widely used in this family is the Box-Cox transformations:

$$y(\lambda) = \frac{y^\lambda - 1}{\lambda}, \quad \text{if } \lambda \neq 0$$

$$\text{and } \log(y), \quad \text{if } \lambda = 0$$

**Reference check:**

The original research paper by Box and Cox is cited in the *References* section under reference 3.

Intuitively, we can see that the Box-Cox transformation is a generalized logarithm transform. The log transform is just a special case of the Box-Cox transformation (when $\lambda = 0$). At different values of λ , it approximates other transforms such as y^2 when $\lambda = 2$, $\frac{1}{\sqrt{y}}$ when $\lambda = -0.5$, $\sqrt{y}$ when $\lambda = 0.5$, and so on. When $\lambda = 1$, there is no major transformation.

A lot of the disadvantages that we mentioned for log transforms apply here as well, but the degree to which those effects are there varies, and we have a parameter, λ , to help us decide on the right level of those effects. Like log transforms, Box-Cox transformations also only use strictly positive data. The same addition of a constant to offset the data distribution has to be done here as well. The flip side of the parameter is that there is one more hyperparameter to tune.

There are a few automated methods to find the optimum λ for any data distribution. One of them is by minimizing the log-likelihood of the data distribution, assuming normality. So, essentially, what we will be doing is finding the optimal λ that makes the data distribution most *normal*. This optimization is already implemented in popular implementations such as the `boxcox` function in the `scipy.special` module in `scipy`.

Another way to find the optimal λ is to use Guerrero's method, which is typically suited for a time series. In this method, instead of trying to conform the data distribution to a normal distribution, we try to minimize the variability of the time series across different sub-series in the time series that are homogenous. The definition of this sub-series is slightly subjective but, usually, we can safely assume the sub-series as the seasonal length. Therefore, what we will be trying to minimize is the variability of the time series across different seasonality cycles.

**Reference check:**

The research paper proposing Guerrero's method is cited in the *References* section under reference 4.

There are stark differences in the way both these optimization methods work and we need to be careful when using them. If our main concern is to remove the heteroscedastic behavior of the time series, Guerrero's method is what we can use.

We have made a transformer available in `src.transforms.target_transformations.py` called `BoxCoxTransformer`. There are a few parameters and properties that we need to be aware of:

- `box_cox_lambda`: This is the λ parameter to be used for the Box-Cox transform. If left set to `None`, the implementation will find an optimal λ .
- `optimization`: This can either be `guerrero`, which is the default setting, or `loglikelihood`. This determines how the λ parameter is estimated.

- `seasonal_period`: This is an input for finding the optimal λ parameter using Guerrero's method. Technically, this is the length of the sub-series, usually taken as the seasonality period.
- `bounds`: This is another parameter that controls the optimization using Guerrero's method. This is a tuple with lower and upper bounds in the search for the optimal λ parameter.
- `add_one`: This is a flag that adds one to the series before applying a log transform to avoid $\log 0$.

The core logic implemented in the Transformer is as follows:

```
## Fit Process
# Add one if needed
y = self._add_one(y)
# Find optimum box cox lamda if optimization is Guerrero
self.boxcox_lambda = self._optimize_lambda(y)
## Transform Process
boxcox(y.values, lmbda=self.boxcox_lambda)
## Inverse Transform
self._subtract_one(inv_boxcox(y.values, self.boxcox_lambda))
```

Now, let's see how we can use it:

```
from src.transforms.target_transformations import BoxCoxTransformer
boxcox_transformer = BoxCoxTransformer()
y_boxcox = boxcox_transformer.fit_transform(y)
```

`y_boxcox` will contain the Box-Cox transformed time series. To get back to the original time series, we can use the `inverse_transform` function.

Both Box-Cox and Log Transform can be used for correcting heteroscedasticity. But, as mentioned before, log transform is a strong transformation and Box-Cox gives us another lever to tweak and tune the transformation to suit our data. We can look at Box-Cox as a flexible log transform, which can be tuned to make the right transformation for our data. Do check out the notebook, where you can see and play around with these different transformations and get a feel of what it will do to your data.

When we approach the forecasting problem at scale, we will have hundreds, thousands, or millions of time series that we will need to analyze before forecasting. In such scenarios, an AutoML approach is needed to be practical.

AutoML approach to target transformation

So far, we have discussed many ways to make a series *more* stationary (we are using the word stationary here in the non-mathematical sense), such as detrending, deseasonalizing, differencing, and monotonic transformations. We've also looked at statistical tests to check whether trends, seasonality, and so on are present in a time series. So, the natural next step is to put it all together to carry out these transforms in an automated way while choosing good defaults wherever possible. This is exactly what we did and implemented an `AutoStationaryTransformer` in `src.transforms.target_transformations`.

The following flow chart explains the logic of this in an automated way:

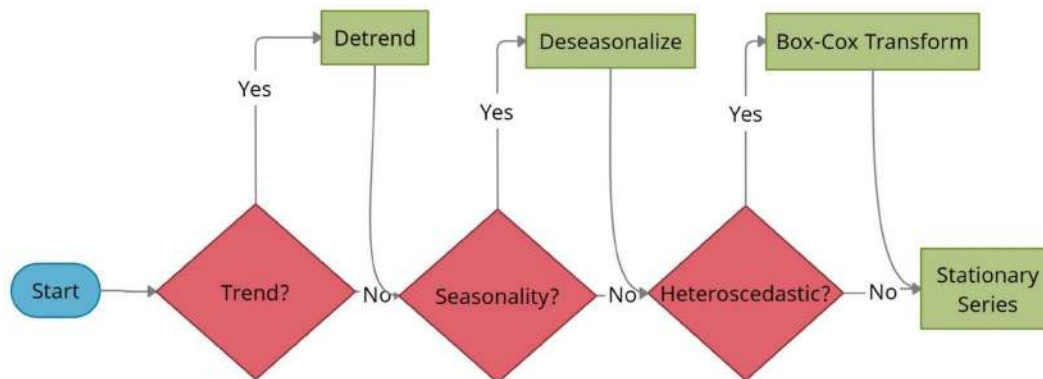


Figure 7.6: Flow chart for AutoStationaryTransformer

We have excluded differencing from this implementation for two reasons:

- Differencing, in the context of predictions, comes with considerable baggage of technical debt. If you do differencing, you are inherently making it difficult to carry out multi-step forecasting. It is possible, but just more difficult and less flexible.
- Differencing can be looked at as a different way of doing what we have done here. This is because differencing removes linear trends and seasonal differencing removes seasonality as well. So, for autoregressive time series, differencing can do a lot and deserves to be a standalone transformation.

Now, let's see what parameters we can use to tweak AutoStationaryTransformer:

- `confidence`: This is the confidence level for the statistical tests. It defaults to `0.05`.
- `seasonal_period`: This is the number of periods after which the seasonality cycle repeats itself. If it is set to `None`, `seasonal_period` will be inferred from the data. It defaults to `None`.
- `seasonality_max_lags`: This is only used if `seasonality_period` is not given. This sets the maximum lags within which we search for seasonality. It defaults to `None`.
- `trend_check_params`: These are the parameters that are used in the statistical tests for trends. `check_trend` defaults to `{"mann_kendall": False}`.
- `detrender_params`: These are the parameters passed to `DetrendingTransformer`. This defaults to `{"degree": 1}`.
- `deseasonalizer_params`: The parameters passed to `DeseasonalizingTransformer`. `seasonality_extraction` are fixed as `period_averages`.
- `box_cox_params`: These are the parameters that are passed to `BoxCoxTransformer`. They default to `{"optimization": "guerrero"}`.

Let's apply this `AutoStationaryTransformer` to a synthetic time series and see how well it works (full code in the associated notebook):

```
from src.transforms.target_transformations import AutoStationaryTransformer
auto_stationary = AutoStationaryTransformer(seasonal_period=25)
y_stat = auto_stationary.fit_transform(y_final)
```

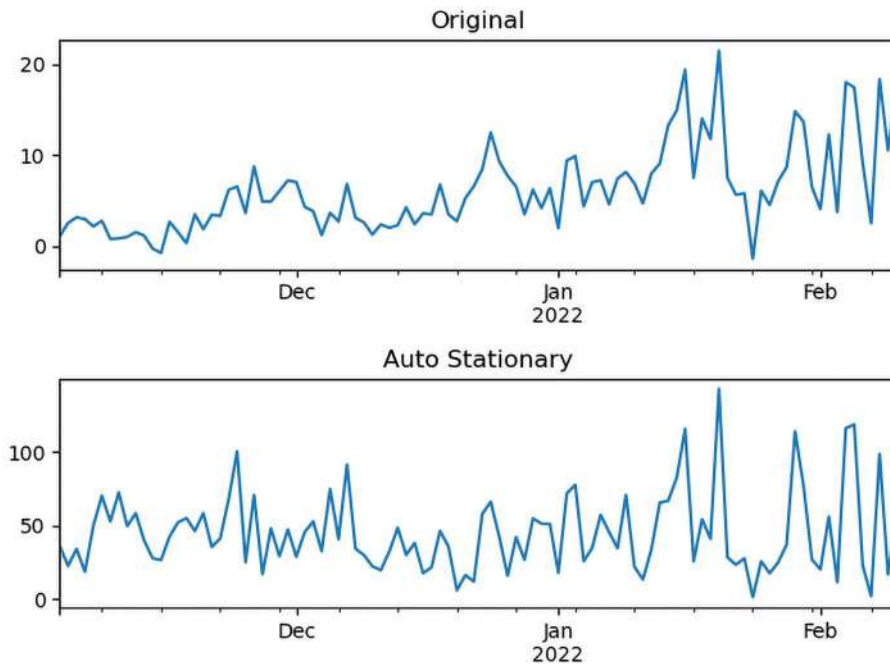


Fig 7.7: `AutoStationaryTransformer`—Before and after

We can see that the `AutoStationaryTransformer` has deseasonalized and de-trended the time series. In this particular example, Detrending, Deseasonalizing, and Box-Cox Transformation were applied by the `AutoStationaryTransformer`.

Now, let's apply this automatic transformation to the dataset we have been working with:

```
train_df = pd.read_parquet(preprocessed/"selected_blocks_train_missing_imputed_
feature_engg.parquet")
transformer_pipelines = {}
for _id in tqdm(train_df["LCLid"].unique()):
    #Initialize the AutoStationaryTransformer with a seasonality period of 48*7
    auto_stationary = AutoStationaryTransformer(seasonal_period=48*7)
    #Creating the timeseries with datetime index
    y = train_df.loc[train_df["LCLid"]==_id, ["energy_
consumption", "timestamp"]].set_index("timestamp")
    #Fitting and transforming the train
```

```

y_stat = auto_stationary.fit_transform(y, freq="30min")
# Setting the transformed series back to the dataframe
train_df.loc[train_df["LCLid"]==_id, "energy_consumption"] = y_stat.values
#Saving the pipeline
transformer_pipelines[_id] = auto_stationary

```

The code to execute this is split into two notebooks called `02-Dealing_with_Non-Stationarity.ipynb` and `02a-Dealing_with_Non-Stationarity-Train+Val.ipynb` in the `Chapter06` folder. The former does the auto-stationary transformation on the train data, while the latter does it on train and validation data combined. This is to simulate how we would predict for validation data (by just using train data for training) and for test data (where we use the train and validation data for training).

This process is slightly time-consuming. I suggest that you run the notebook, grab lunch or a snack, and come back. Once it's done, the `02-Dealing_with_Non-Stationarity.ipynb` notebook will save a couple of files:

- `selected_blocks_train_auto_stat_target.parquet`: A DataFrame that has LCLid and timestamp as indices and the transformed target
- `auto_transformer_pipelines_train.pkl`: A Python dictionary of `AutoStationaryTransformer` for each LCLid so that we can reverse the transformations in the future

The `02a-Dealing_with_Non-Stationarity-Train+Val.ipynb` notebook also saves the corresponding files for the train and validation datasets.

The dataset we are working on has almost negligible trends and is pretty stationary throughout. The impact of these transformations will be more evident in time series with strong trends and heteroscedasticity.

Best practice:



This kind of explicit detrending and deseasonalizing before modeling can also be seen as a form of **boosting**. This should be considered as just another alternative to modeling all of this together. There can be situations where letting the model learn from end to end in a data-driven manner performs better than injecting these strong inductive biases using explicit detrending and deseasonalization and vice versa. Cross-validated test scores should always have the last word.

Congratulations on making it through a heavy chapter full of new concepts, some statistics, and mathematics. From the point of view of applying ML models for time series, the concepts in this chapter will be really helpful in taking your models to the next level.

Summary

After getting down to a practical level in the previous chapter, we stayed there and plowed on to review concepts such as stationarity and how to deal with such non-stationary time series. We learned about techniques we can use to explicitly handle non-stationary time series, such as differencing, detrending, deseasonalizing, and so on. To put this all together, we saw an automatic way of transforming the target, learned how to use the implementation provided, and applied it to our dataset. Now that we have the necessary skills to effectively transform a time series into an ML dataset, in the next chapter, we will start applying a few ML models to the dataset using the features we've created.

References

The following are the references for this chapter:

1. Bayazit, M. and Önöz, B. (2007), *To prewhiten or not to prewhiten in trend analysis?*, Hydrological Sciences Journal, 52:4, 611–624. <https://doi.org/10.1623/hysj.52.4.611>.
2. White, H. (1980), *A Heteroskedasticity-Consistent Covariance Matrix Estimator and a Direct Test for Heteroskedasticity*. Econometrica Vol. 48, No. 4 (May 1980), pp. 817–838 (22 pages). <https://doi.org/10.2307/1912934>.
3. Box, G. E. P. and Cox, D. R. (1964), *An analysis of transformations*. Journal of the Royal Statistical Society, Series B, 26, 211–252. <http://www.ime.usp.br/~abe/lista/pdfQWacMbK68.pdf>.
4. Guerrero, Victor M. (1993), *Time-series analysis supported by power transformations*. Journal of Forecasting, Volume 12, Issue 1, 37–48. <https://onlinelibrary.wiley.com/doi/10.1002/for.3980120104>.

Further reading

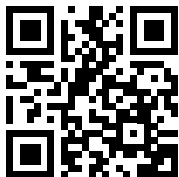
To learn more about the topics that were covered in this chapter, take a look at the following resources:

- *Stationarity in time series analysis*, by Shay Palachy: <https://towardsdatascience.com/stationarity-in-time-series-analysis-90c94f27322>
- *Comparing ADF Test Functions in R*, by Fabian Kostadinov (the same concepts can be implemented in Python as well): <https://fabian-kostadinov.github.io/2015/01/27/comparing-adf-test-functions-in-r/>
- *Kendall's Tau*: <https://www.statisticshowto.com/kendalls-tau/>
- *Mann-Kendall trend test*: <https://www.statisticshowto.com/wp-content/uploads/2016/08/Mann-Kendall-Analysis-1.pdf>
- *Theil-Sen estimator*: https://en.wikipedia.org/wiki/Theil%E2%80%93Sen_estimator
- *Statistical inference with correlograms*—Wikipedia: https://en.wikipedia.org/wiki/Correlogram#Statistical_inference_with_correlograms
- *White test for Heteroscedasticity Detection*: <https://itfeature.com/hetero/white-test-of-heteroscedasticity/>

Join our community on Discord

Join our community's Discord space for discussions with authors and other readers:

<https://packt.link/mts>



8

Forecasting Time Series with Machine Learning Models

In the previous chapter, we started looking at machine learning as a tool to solve the problem of time series forecasting. We talked about a few techniques such as time delay embedding and temporal embedding, both of which cast a time series forecasting problem as a classical regression problem from the machine learning paradigm. In this chapter, we'll look at these techniques in detail and go through them in a practical sense using the London Smart Meters dataset we have been working with throughout this book.

In this chapter, we will cover the following topics:

- Training and predicting with machine learning models
- Generating single-step forecast baselines
- Standardized code to train and evaluate machine learning models
- Training and predicting for multiple households

Technical requirements

You will need to set up the **Anaconda** environment following the instructions in the *Preface* of the book to get a working environment with all the libraries and datasets required for the code in this book. Any additional library will be installed while running the notebooks.

You will need to run the following notebooks before using the code in this chapter:

- 02-Preprocessing_London_Smart_Meter_Dataset.ipynb in Chapter02
- 01-Setting_up_Experiment_Harness.ipynb in Chapter04
- 01-Feature_Engineering.ipynb in Chapter06
- 02-Dealing_with_Non-Stationarity.ipynb in Chapter07
- 02a-Dealing_with_Non-Stationarity-Train+Val.ipynb in Chapter07

The code for this chapter can be found at <https://github.com/PacktPublishing/Modern-Time-Series-Forecasting-with-Python-/tree/main/notebooks/Chapter08>.

Training and predicting with machine learning models

In *Chapter 5, Time Series Forecasting as Regression*, we talked about a schematic for supervised machine learning (*Figure 5.2*). In the schematic, we mentioned that the purpose of a supervised learning problem is to come up with a function, $\hat{y} = h(X, \phi)$, where $\hat{y}$ is the predicted value, X is the set of features as the input, ϕ is the model parameters, and h is the approximation of the ideal function. In this section, we are going to talk about h in more detail and see how we can use different machine learning models to estimate it.

h is any function that approximates the ideal function, but it can be thought of as an element of all possible functions from a family of functions. More formally, we can say the following:

$$\hat{y} = h(X, \phi), \quad \text{where } h \in H$$

Here, H is a family of functions that we also call a model. For instance, linear regression is a type of model or a family of functions. For each value of the coefficients, the linear regression model gives you a different function and H becomes the set of all possible functions a linear regression model can produce.

There are many families of functions, or models, available. For a more complete understanding of the space, we will need to refer to other machine learning resources. The *Further reading* section contains a few resources that may help you start the journey. As for the scope of this book, we narrowly define it as the application of machine learning models for forecasting, rather than machine learning in general. And although we can use any regression model, we will only review a few popular and useful ones for time series forecasting and see them in action. We leave it to you to strike out on your own and explore the other algorithms to become familiar with them as well. But before we look at the different models, we need to generate a few baselines again.

Generating single-step forecast baselines

We reviewed and generated a few baseline models back in *Chapter 4, Setting a Strong Baseline Forecast*. But there is a small issue – the prediction horizon. In *Chapter 6, Feature Engineering for Time Series Forecasting*, we talked about how the machine learning model can only predict one target at a time and that we are sticking with a single-step forecast. The baselines we generated earlier were not single-step, but multi-step. Generating a single-step forecast for baseline algorithms such as ARIMA or ETS requires us to fit on history, predict one step ahead, and then fit again using one more day. Predicting in such an iterative fashion for our test or validation period requires us to do this iteration ~1,440 times (48 data points a day for 30 days) and repeat this for all the households in our selected dataset (150, in our case). This would take quite a long time to compute.

We have chosen the naïve method and seasonal naïve (*Chapter 4, Setting a Strong Baseline Forecast*), which can be implemented as native pandas methods, as two baseline methods to generate single-step forecasts. Naïve forecasts perform unreasonably well for single-step-ahead forecasts and can be considered a strong baseline. In the `Chapter08` folder, there is a notebook named `00-Single_Step_Backtesting_Baselines.ipynb` that generates these baselines and saves them to disk. Let's run the notebook now. The notebook generates the baselines for both the validation and test datasets and saves the predictions, metrics, and aggregate metrics to disk. The aggregate metrics for the test period are as follows:

	MAE	MSE	meanMASE	Forecast Bias
Naive	0.086	0.045	1.050	0.02%
Seasonal Naive	0.122	0.072	1.487	4.07%

Figure 8.1: Aggregate metrics for a single-step baseline

To make training and evaluating these models easier, we have used a standard structure throughout. Let's quickly review that structure as well so that you can follow along with the notebooks closely.

Standardized code to train and evaluate machine learning models

There are two main ingredients while training a machine learning model – *data* and the *model* itself. Therefore, to standardize the pipeline, we defined three configuration classes (`FeatureConfig`, `MissingValueConfig`, and `ModelConfig`) and another wrapper class (`MLForecast`) over scikit-learn-style estimators (`.fit` - `.predict`) to make the process smooth. Let's look at each of them.



Notebook alert:

To follow along with the code, use the `01-Forecasting_with_ML.ipynb` notebook in the `Chapter08` folder and the code in the `src` folder.

FeatureConfig

`FeatureConfig` is a Python dataclass that defines a few key attributes and functions that are necessary while processing the data. For instance, continuous, categorical, and Boolean columns need separate kinds of preprocessing before being fed into the machine learning model. Let's see what `FeatureConfig` holds:

- **date:** A mandatory column that sets the name of the column with `date` in the `DataFrame`.
- **target:** A mandatory column that sets the name of the column with `target` in the `DataFrame`.

- `original_target`: If `target` contains a transformed target (log, differenced, and so on), `original_target` specifies the name of the column with the target without transformation. This is essential for calculating metrics such as MASE, which relies on training history. If not given, it is assumed that `target` and `original_target` are the same.
- `continuous_features`: A list of continuous features.
- `categorical_features`: A list of categorical features.
- `boolean_features`: A list of Boolean features. Boolean features are categorical but only have two unique values.
- `index_cols`: A list of columns that are set as a DataFrame index while preprocessing. Typically, we would give the datetime and, in some cases, the unique ID of a time series as indices.
- `exogenous_features`: A list of exogenous features. The features in the DataFrame may be from the feature engineering process, such as the lags or rolling features, but also external sources such as the temperature data in our dataset. This is an optional field that lets us bifurcate the exogenous features from the rest of the features. The items in this list should be a subset of `continuous_features`, `categorical_features`, or `boolean_features`.

In addition to a bit of validation on the inputs, there is also a helpful method called `get_X_y` in the class, with the following parameters:

- `df`: A DataFrame that contains all the necessary columns, including the target, if available
- `categorical`: A Boolean flag for including categorical features or not
- `exogenous`: A Boolean flag for including exogenous features or not

The function returns a tuple of (`features`, `target`, `original_target`).

All we need to do is initialize the class, like any other class, with the feature names separated into the parameters of the class. The entire code that contains all the features is available in the accompanying notebook.

After setting the `FeatureConfig` data class, we can pass any DataFrame with the features defined to the `get_X_y` function to get the features, target, and original target:

```
train_features, train_target, train_original_target = feat_config.get_X_y(
    sample_train_df, categorical=False, exogenous=False
)
```

 **Quick tip:** Enhance your coding experience with the **AI Code Explainer** and **Quick Copy** features. Open this book in the next-gen Packt Reader. Click the **Copy** button (1) to quickly copy code into your coding environment, or click the **Explain** button (2) to get the AI assistant to explain a block of code to you.



 The next-gen Packt Reader is included for free with the purchase of this book. Unlock it by scanning the QR code below or visiting <https://www.packtpub.com/unlock/9781835883181>.



As you can see, we are not using categorical features or exogenous features here, as I want to focus on the core algorithms and show how they can be drop-in replacements for other classical time series models we saw earlier. We will talk about how to handle categorical features in *Chapter 15, Strategies for Global Deep Learning Forecasting Models*.

MissingValueConfig

Another key setting is how to deal with missing values. We saw a few ways to fill in missing values from a time series context in *Chapter 3, Analyzing and Visualizing Time Series Data*, and we have already filled in missing values and prepared our datasets. But a few missing values will be created in the feature engineering required to convert a time series into a regression problem. For instance, when creating lag features, the earliest date in the dataset will not have enough data to create a lag and will be left empty.

**Best practice:**

Although filling with zero or mean is the default or go-to method for the majority of the data scientist community, we should always make an effort to fill in missing values as intelligently as possible. In terms of lag features, filling with zero can distort the feature. Instead of filling with zero, a backward fill (using the earliest value in the column to fill backward) might be a much better fit.

Some machine learning models handle empty or NaN features naturally, while for other machine learning models, we will need to deal with such missing values before training. It's helpful if we can define a config in which we set for a few columns where we expect NaN information on how to fill those. `MissingValueConfig` is a Python dataclass that does just that. Let's see what it holds:

- `bfill_columns`: A list of column names that need to use a backward fill strategy to fill missing values.
- `ffill_columns`: A list of column names that need to use a forward fill strategy to fill missing values. If a column name is repeated across both `bfill_columns` and `ffill_columns`, that column is filled using backward fill first and the rest of the missing values are filled with the forward fill strategy.
- `zero_fill_columns`: A list of column names that need to be filled with zeros.

The order in which the missing values are filled is `bfill_columns`, then `ffill_columns`, and then `zero_fill_columns`. As the default strategy, the data class uses the column mean to fill in missing values so that even if you have not defined any strategy for a column, the missing value will be filled in using a column mean. There is a method called `impute_missing_values` that takes in the `DataFrame` and fills the empty cells with a value according to the specified strategy.

ModelConfig

`ModelConfig` is a Python dataclass that holds a few details regarding the modeling process, such as whether to normalize the data, whether to fill in missing values, and so on. Let's take a detailed look at what it holds:

- `model`: This is a mandatory parameter that can be any scikit-learn-style estimator.
- `name`: A string name or identifier for the model. If it's not used, it will revert to the name of the class that was passed in as `model`.
- `normalize`: A Boolean flag to set whether to apply `StandardScaler` to the input or not.
- `fill_missing`: A Boolean flag to set whether to fill empty values before training or not. Some models can handle NaN naturally, while others can't.
- `encode_categorical`: A Boolean flag to set whether to encode categorical columns as part of the fitting procedure. If `False`, categorical encoding is expected to be done separately and included as part of continuous features.
- `categorical_encoder`: If `encode_categorical` is `True`, `categorical_encoder` is the scikit-learn-style encoder we can use.

Let's see how we can define the `ModelConfig` data class:

```
model_config = ModelConfig(  
    model=LinearRegression(),  
    name="Linear Regression",  
    normalize=True,  
    fill_missing=True,  
)
```

This has just one method, `clone`, that clones the estimator, along with the config, into a new instance.

MLForecast

Last but not least, we have the wrapper class around a scikit-learn-style model. It uses the different configurations we have discussed to encapsulate the training and prediction functions. Let's see what parameters are available when initializing the model:

- `model_config`: The instance of the `ModelConfig` class we discussed in the *ModelConfig* section.
- `feature_config`: The instance of the `FeatureConfig` class we discussed earlier.
- `missing_config`: The instance of the `MissingValueConfig` class we discussed earlier.
- `target_transformer`: The instance of target transformers from `src.transforms`. It should support `fit`, `transform`, and `inverse_transform`. It should also return `pd.Series` with a datetime index to work without errors. If we have done the target transform separately, then this is also used to perform `inverse_transform` during prediction.

`MLForecast` has a few functions that can help us manage the life cycle of a model, once initialized. Let's take a look.

The fit function

The `fit` function is similar in purpose to the scikit-learn `fit` function but does a little extra by handling the standardization, categorical encoding, and target transformations using the information in the three configs. The parameters of the function are as follows:

- `x`: This is the pandas `DataFrame` with features to be used in the model as columns.
- `y`: This is the target and can be a pandas `DataFrame`, pandas `Series`, or a numpy array.
- `is_transformed`: This is a Boolean parameter that lets us know whether the target is already transformed or not. If `True`, the `fit` method won't be transforming the target, even if we have initialized the object with `target_transformer`.
- `fit_kwargs`: This is a Python dictionary of keyword arguments that need to be passed to the `fit` function of the estimator.

The predict function

The `predict` function handles inferencing. It wraps around the `predict` function of the scikit-learn estimator, but like `fit`, it does a few other things, such as standardization, categorical encoding, and reversing the target transformation. There is only one parameter for this function:

- **x**: The pandas DataFrame with features to be used in the model as columns. The index of the DataFrame is passed on to the prediction.

The feature_importance function

The `feature_importance` function retrieves the feature importance from the model, if available. For linear models, it extracts the coefficients, while for tree-based models, it extracts the built-in importance and returns it in a sorted DataFrame.

Helper functions for evaluating models

While the other functions we saw earlier deal with core training and predicting, we also want to evaluate the model, plot the results, and so on. We have also defined these functions in the notebooks or in the code base. The below function is to evaluate the models in the notebook:

```
def evaluate_model(
    model_config,
    feature_config,
    missing_config,
    train_features,
    train_target,
    test_features,
    test_target,
):
    ml_model = MLForecast(
        model_config=model_config,
        feature_config=feat_config,
        missing_config=missing_value_config,
    )
    ml_model.fit(train_features, train_target)
    y_pred = ml_model.predict(test_features)
    feat_df = ml_model.feature_importance()
    metrics = calculate_metrics(test_target, y_pred, model_config.name, train_target)
    return y_pred, metrics, feat_df
```

This provides us with a standard way of evaluating all the different models, as well as automating the process at scale. We also have a function for calculating the metrics, `calculate_metrics`, defined in `src/forecasting/ml_forecasting.py`.



The standard implementation that we have provided with this book is in no way a one-size-fits-all approach, but rather something that works best with the flow and dataset of this book. Please do not consider it as a robust library, but rather a good starting point and guide to help you develop your own code.

Now that we have the baselines and a standard way to apply different models, let's get back to what the different models are. For the discussion ahead, let's keep *time* out of our minds because we have converted a time series forecasting problem into a regression problem and factored in *time* as a feature of the problem (the lags and rolling features).

Linear regression

Linear regression is a family of functions that takes the following form:

$$\hat{y} = \beta_0 + \sum_{i=1}^k X_i \beta_i$$

Here, k is the number of features in the model and β are the parameters of the model. There is a β for each feature, as well as a β_0 , which we call the intercept, which is estimated from data. Essentially, the output is a linear combination of the feature vectors, X_i . As the name suggests, this is a linear function.

The model parameters can be estimated from data, $D(X_i, y_i)$, using an optimization method and loss, but the most popular method of estimation is using **ordinary least squares (OLS)**. Here, we find the model parameters, β , which minimizes the residual sum of squares (**mean squared error (MSE)**):

$$RSS = \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

The loss function here is very intuitive. We are essentially minimizing the distance between the training samples and our predicted points. The square term acts as a technique that does not cancel out positive and negative errors. Apart from the intuitiveness of the loss, another reason why this is widely chosen is that an analytical solution exists for least squares and because of that, we don't need to resort to more compute-intensive optimization techniques such as gradient descent.

Linear regression has one foot firmly planted in statistics and with the right assumptions, it can be a powerful tool. Commonly, five assumptions are associated with linear regression, as follows:

- The relationship between the independent and dependent variables is linear.
- The errors are normally distributed.
- The variance of the errors is constant across all the values of the independent variable.
- There is no autocorrelation in the errors.
- There is little to no correlation between independent variables (multi-collinearity).

But unless you are concerned about using linear regression to come up with prediction intervals (a band in which the prediction would lie with some probability), we can disregard all but the first assumption to some extent.

The linearity assumption (the first assumption) is relevant because if the variables are not linearly related, it will result in an underfit and thus poor performance. We can get around this problem to some extent by projecting the inputs into a higher dimensional space. Theoretically, we can project a non-linear problem into a higher-dimensional space, where the problem is linear. For instance, let's consider a non-linear function, $y = 3x_1^2 + 2x_2^2 + 6x_1x_2$. If we run linear regression in the input space of x_1 and x_2 , we know the resulting model will be highly underfitting. But if we project the input space from x_1 and x_2 to x_1^2 , x_2^2 , and xy by using a polynomial transform, the function for y becomes a perfect linear fit.

The multi-collinearity assumption (the final assumption) is partly relevant to the fit of the linear function because when we have highly correlated independent variables, the estimated coefficients are highly unstable and difficult to interpret. The fitted function would still be working well, but because we have multi-collinearity, even small changes in the inputs would make the coefficients change magnitude and sign. It is a best practice to check for multi-collinearity if you are using a pure linear regression. This is typically a problem in time series because the features we have extracted, such as the lag and rolling features, may be correlated with each other. Therefore, we will have to be careful while using and interpreting linear regression on time series data.

Now, let's see how we can use linear regression and evaluate the fit of a sample household from our validation dataset:

```
from sklearn.linear_model import LinearRegression
model_config = ModelConfig(
    model=LinearRegression(),
    name="Linear Regression",
    # LinearRegression is sensitive to normalized data
    normalize=True,
    # LinearRegression cannot handle missing values
    fill_missing=True,
)
y_pred, metrics, feat_df = evaluate_model(
    model_config,
    feat_config,
    missing_value_config,
    train_features,
    train_target,
    test_features,
    test_target,
)
```

The single-step forecast looks good and is already better than the naïve forecast (MAE = 0.173):



Figure 8.2: Linear regression forecast

The coefficients of the model, β (which can be accessed using the `coef_` attribute of a trained scikit-learn model), show how much influence each feature has on the output. So, extracting and plotting them gives us our first level of visibility into the model. Let’s take a look at the coefficients of the model:



Figure 8.3: Feature importance of linear regression (top 15)

If we look at the Y-axis in the feature importance chart, we can see it is in billions as the coefficient for a couple of features is in orders of magnitude in billions. We can also see that those features are Fourier series-based features, which are correlated with each other. Even though we have a lot of coefficients that are in billions, we can find them on both sides of zero, so they will essentially cancel out each other in the function. This is the problem with multi-collinearity that we talked about earlier. We can go about removing multi-collinear features and then perform some sort of feature selection (forward selection or backward elimination) to make the linear model even better.

But instead of doing that, let's look at a few modifications we can make to the linear model that are a bit more robust to multi-collinearity and feature selection.

Regularized linear regression

We briefly talked about regularization in *Chapter 5, Time Series Forecasting as Regression*, and mentioned that regularization, in the general sense, is any kind of constraint we place on the learning process to reduce the complexity of the learned function. One of the ways linear models can become more complex is by having a high magnitude of coefficients. For instance, in the linear fit, we have a coefficient of 20 billion. Any small change in that feature is going to cause a huge fluctuation in the resulting prediction. Intuitively, if we have a large coefficient, the function becomes more flexible and complex. One way we can fix this is to apply regularization in the form of weight decay. Weight decay is when we add a term that penalizes the magnitude of the coefficients to the loss function. The loss function, the residual sum of squares, now becomes as follows:

$$RSS = \sum_{i=1}^N (y_i - \hat{y}_i)^2 + \lambda W$$

Here, W is the weight decay and $\lambda < 0$ is the strength of regularization.

W is typically the norm of the weight matrix. In linear algebra, the norm of a matrix is a measure of how large its elements are. There are many norms for a matrix, but the two most common norms that are used for regularization are the **L1** and **L2** norms. When we use the L1 norm to regularize linear regression, we call it **lasso regression**, while when we use the L2 norm, we call it **ridge regression**. When we apply weight decay regularization, we are forcing the coefficients to be lower, which means that it also acts as an internal feature selection because the features that don't add a lot of value will get very low or zero (depending on the type of regularization) coefficients, which means they contribute little to nothing in the resulting function.

The L1 norm is defined as the sum of the absolute values of the matrix. For weight decay regularization, the L1 norm would be as follows:

$$W = \sum_{i=1}^k |\beta_i|$$

The L2 norm is defined as the sum of squared values of a matrix. For weight decay regularization, the L2 norm would be as follows:

$$W = \sum_{i=1}^k |\beta_i^2|$$

By adding this term to the loss function of linear regression, we are forcing the coefficients to be small because while the optimizer is reducing the RSS, it is also incentivized to reduce W .

Another way we can think about regularization is in terms of linear algebra and geometry.



The following section discusses the geometric intuition of regularization. Although it would make your understanding of regularization more solid, it is not essential to be able to follow the rest of this book. So, feel free to skip the next section and just read the *Key point* callout if you are pressed for time or if you want to come back to it later when you have time.

Regularization—a geometric perspective

If we look at the L1 and L2 norms from a slightly different perspective, we will see that they are measures of distance.

Let B be the vector of all the coefficients, β , in linear regression. A vector is an array of numbers, but geometrically, it is also an arrow from the origin to a point in the n -dimensional coordinate space. Now, the L2 norm is nothing but the Euclidean distance from the origin on that point in space defined by the vector, B . The L1 norm is the Manhattan distance or taxicab distance from the origin on that point in space defined by the vector, B . Let's see this in a diagram:

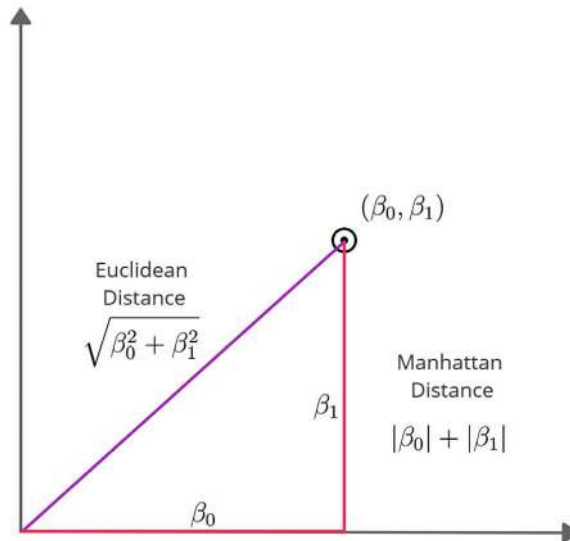


Figure 8.4: Euclidean versus Manhattan distance

Euclidean distance is the length of the direct path from the origin to the point. But if we can only move parallel to the two axes, we will have to travel the distance of β_0 along the one axis first, and then a distance of β_1 along the other. This is the Manhattan distance.

Let's say we are in a city (for example, Manhattan) where the buildings are laid out in square blocks and the straight streets intersect at right angles, and we want to travel from point A to point B. Euclidean distance is the direct distance from point A to point B, which in the real sense is only possible if we parkour along the tops of the buildings. On the other hand, the Manhattan distance is the actual distance a taxicab would take while traveling along the right-angled roads from point A to point B.

To develop further geometrical intuition about the L1 and L2 norms, let's do one thought experiment. If we move the point, (β_0, β_1) , in the 2D space while keeping the Euclidean distance or the L2 norm the same, we will end up with a circle with its center at the origin. This becomes a sphere in 3D and a hypersphere in n -D. If we trace out the same but keep the L1 norm the same, we will end up with a diamond with its center at the origin. This would become a cube in 3D and a hypercube in n -D.

Now, when we are optimizing for the weights, in addition to the main objective of reducing the loss function, we are also encouraging the coefficients to stay within a defined distance (norm) from the origin. Geometrically, this means that we are asking the optimization to find a vector, β , that minimizes the loss function and stays within the geometric shape (circle or square) defined by the norm. We can see this in the following diagram:

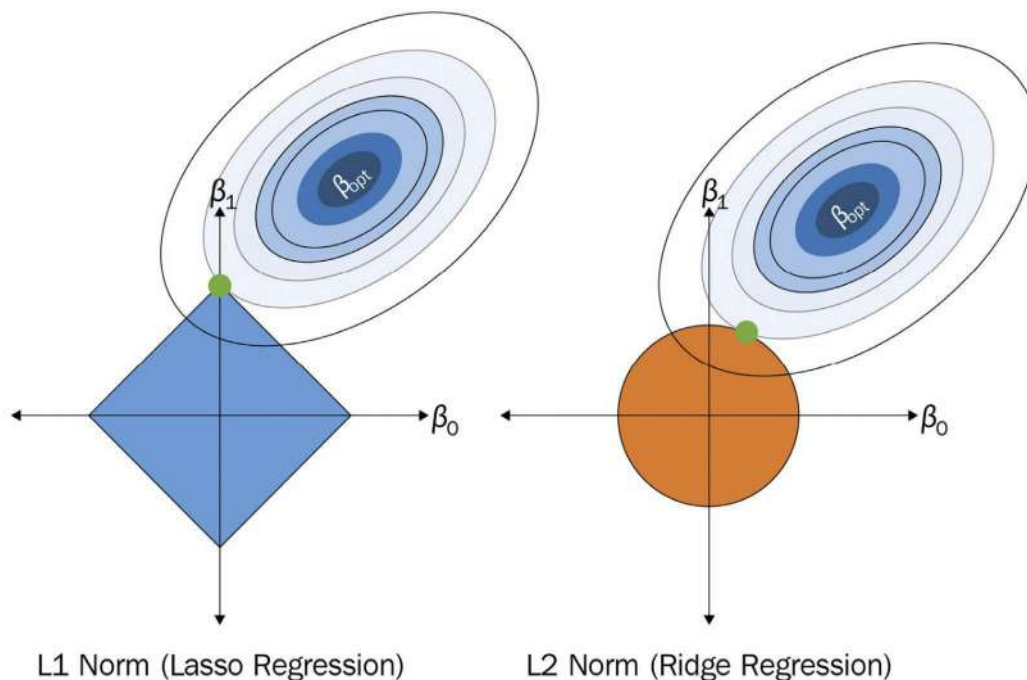


Figure 8.5: Regularization with the L1 norm (lasso regression) versus the L2 norm (ridge regression)

The concentric circles in the diagram are the contours of the loss function, with the innermost being the lowest. As we move outward, the loss increases. So, instead of selecting a β_{opt} , regularized regression will select a β that intersects with the norm geometry.

This geometric interpretation also makes understanding another key difference between ridge and lasso regression easier. Lasso regression, because of the L1 norm, produces a sparse solution. Earlier, we mentioned that weight decay regularization does implicit feature selection. But depending on whether you are applying the L1 or L2 norm, the kind of implicit feature selection differs.



Key point:

For an L2 norm, the coefficients of less relevant features are pushed close to zero, but not exactly zero. The feature will still play a role in the final function, but its influence will be minuscule. The L1 norm, on the other hand, pushes the coefficients of such features completely to zero, resulting in a sparse solution. Therefore, L1 regularization promotes sparsity and feature selection, whereas L2 regularization reduces model complexity by shrinking the coefficients toward zero without necessarily eliminating any.

This can be understood better using the geometrical interpretation of regularization. In optimization, the interesting points are usually found in the extrema or *corners* of a shape. There are no corners in a circle, so an L2 norm is created; the minima can lie anywhere on the edge of the circle. But for the diamond, we have four corners, and the minima would lie in those corners. So, with the L2 norm, the solution can move very close to zero, but not necessarily zero. However, with the L1 norm, the solution would be on the corners, where the coefficient can be pushed to an absolute zero.

Now, let's see how we can use ridge regression and evaluate the fit on a sample household from our validation dataset:

```
from sklearn.linear_model import RidgeCV
model_config = ModelConfig(
    model=RidgeCV(),
    name="Ridge Regression",
    # RidgeCV is sensitive to normalized data
    normalize=True,
    # RidgeCV does not handle missing values
    fill_missing=True
)
y_pred, metrics, feat_df = evaluate_model(
    model_config,
    feat_config,
    missing_value_config,
    train_features,
    train_target,
    test_features,
    test_target,
)
```

Let's look at the single-step-ahead forecast from RidgeCV. It looks very similar to linear regression. Even the MAE is the same for this household:

Ridge Regression: MAE: 0.1595 | MSE: 0.0748 | MASE: 1.2430 | Bias: 6.1637

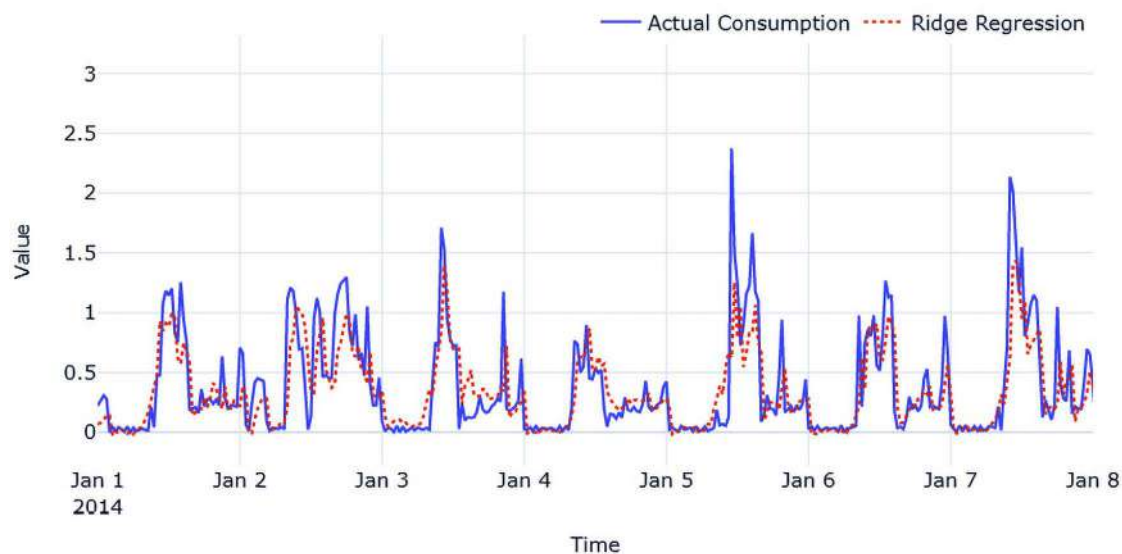


Figure 8.6: Ridge regression forecast

But it is interesting to look at the coefficients with the L2 regularized model. Let's take a look at the coefficients of the model:

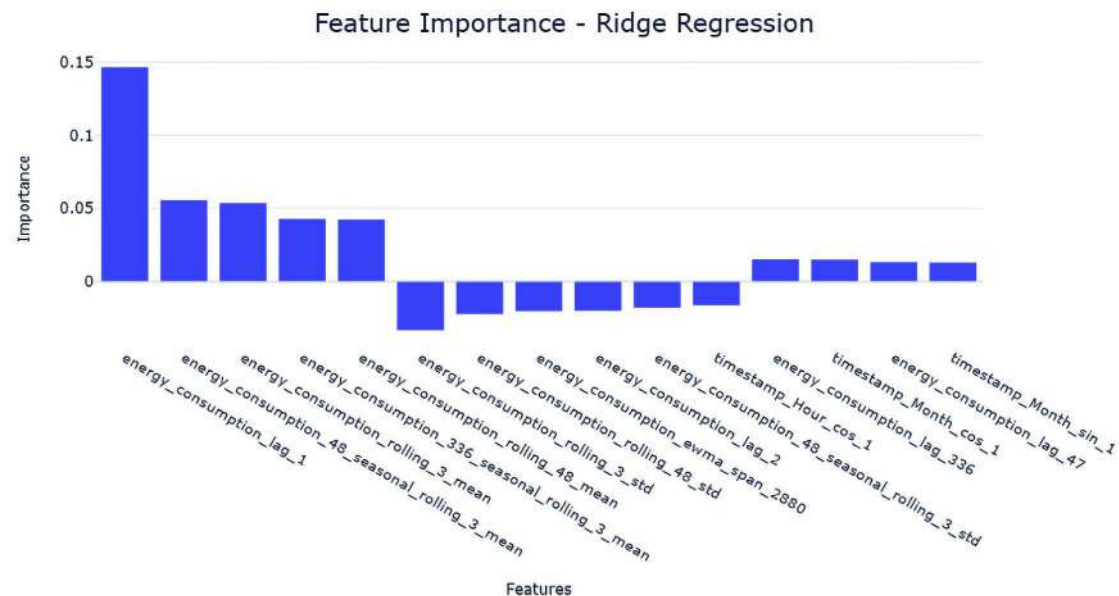


Figure 8.7: Feature importance of ridge regression (top 15)

Now, the Y-axis looks reasonable and small. The coefficients for the multi-collinear features have shrunk to a more reasonable level. Features such as the lag features, which should ideally be highly influential, have gained the top spots. As you may recall, in the linear regression (*Figure 8.3*), these features were dwarfed by the huge coefficients on the Fourier features. We have just plotted the top 15 features here, but if you look at the entire list, you will see that there will be a lot of features for which the coefficients are close to zero.

Now, let's try lasso regression on the sample household:

```
from sklearn.linear_model import LassoCV
model_config = ModelConfig(
    model=LassoCV(),
    name="Lasso Regression",
    # LassoCV is sensitive to normalized data
    normalize=True,
    # LassoCV does not handle missing values
    fill_missing=True
)
y_pred, metrics, feat_df = evaluate_model(
    model_config,
    feat_config,
    missing_value_config,
    train_features,
    train_target,
    test_features,
    test_target,
)
```

Let's look at the single-step-ahead forecast from LassoCV. Like ridge regression, there is hardly any visual difference from linear regression:

Lasso Regression: MAE: 0.1599 | MSE: 0.0743 | MASE: 1.2463 | Bias: 3.6669

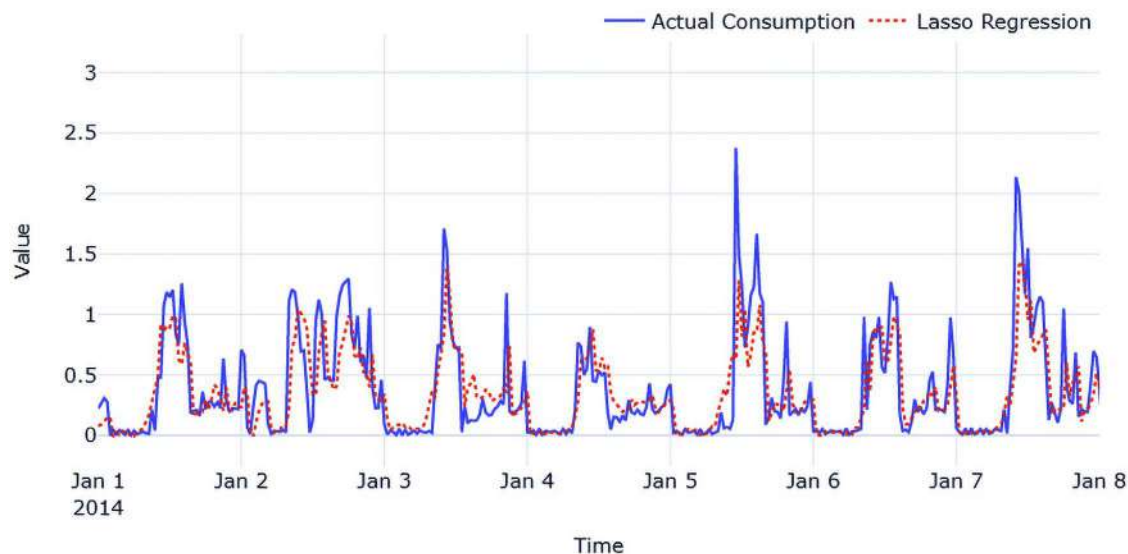


Figure 8.8: Lasso regression forecast

Let's look at the coefficients of the model:

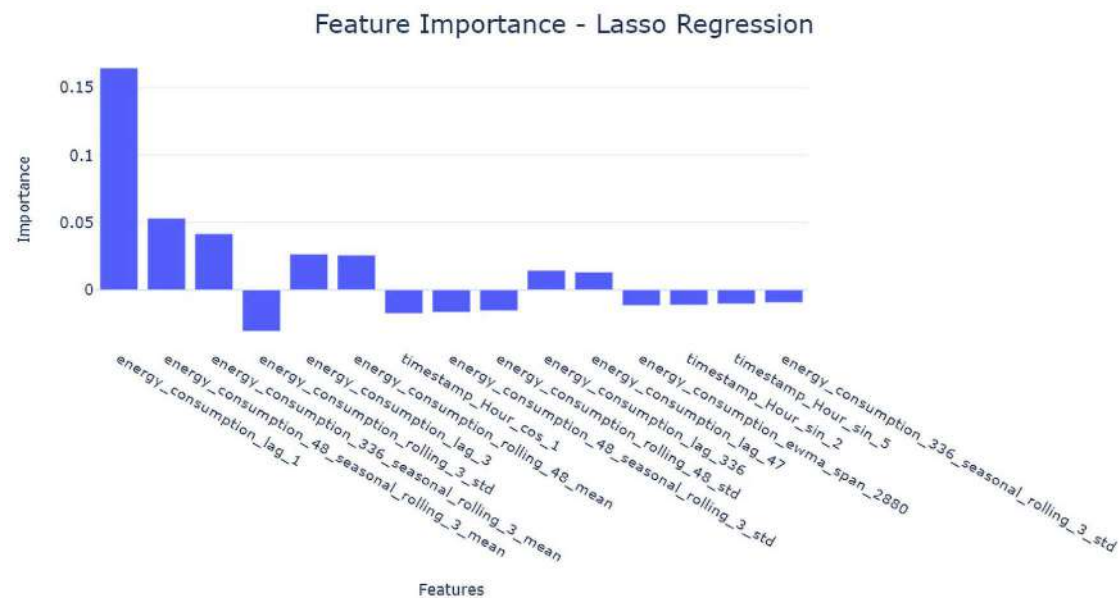


Figure 8.9: Feature importance of lasso regression (top 15)

The coefficients are very similar to ridge regression, but if you look at the full list of coefficients (in the notebook), you will see that there are a lot of features where the coefficients will be zero.

Even with the same MAE, MSE, and so on, ridge or lasso regression is preferred to linear regression because of the additional stability and robustness that comes with regularized regression, especially for forecasting, where multi-collinearity is almost always present. But we need to keep in mind that all the linear regression models are still only capturing linear relationships. If the dataset has a non-linear relationship, the resulting fit from linear regression won't be as good and, sometimes, will be terrible.

Now, let's switch tracks and look at another class of models – **decision trees**.

Decision trees

Decision trees are another family of functions that is much more expressive than a linear function. Decision trees split the feature space into different sub-spaces and fit a very simple model (such as an average) to each. Let's understand how this partitioning works with an example. Let's consider a regression problem for predicting Y with just one feature, X , as shown in the following diagram:

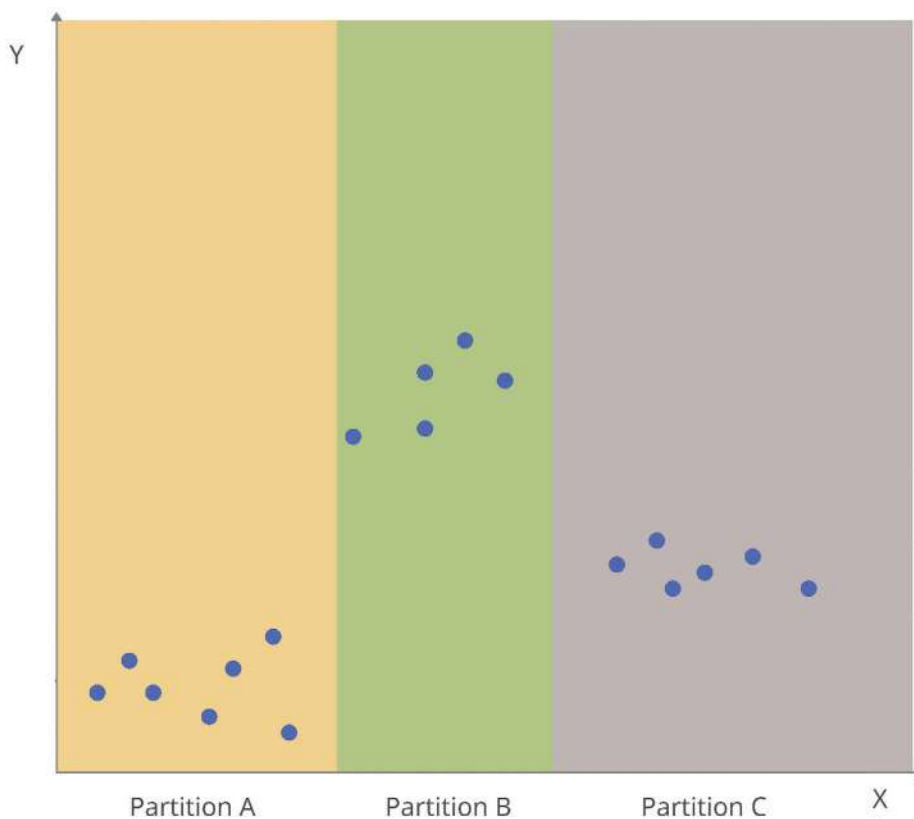


Figure 8.10: The feature space partitioned by a decision tree

Right away, we can see that fitting a linear function would result in an underfit. But what decision trees do is split the feature space (here, it is just X) into different regions where the target, Y , is similar and then fit a simple function such as an average (because it is a regression problem). In this case, the decision tree has split the feature space into partitions – A, B, and C. Now, for any X that falls into partition A, the prediction function will return the average of all the points in partition A.

These partitions are formed by creating a decision tree using data. Intuitively, a decision tree creates a set of if-else conditions and tries to arrive at the best way to partition the feature space to maximize the homogeneity of the target variable within the partition. One helpful way to understand what a decision tree does is to think of data points as beads flowing down a tree, taking a path based on its features, and ending up in a final resting place. Before we talk about how to create a decision tree from data, let's take a look at its components and understand the terminology surrounding it:

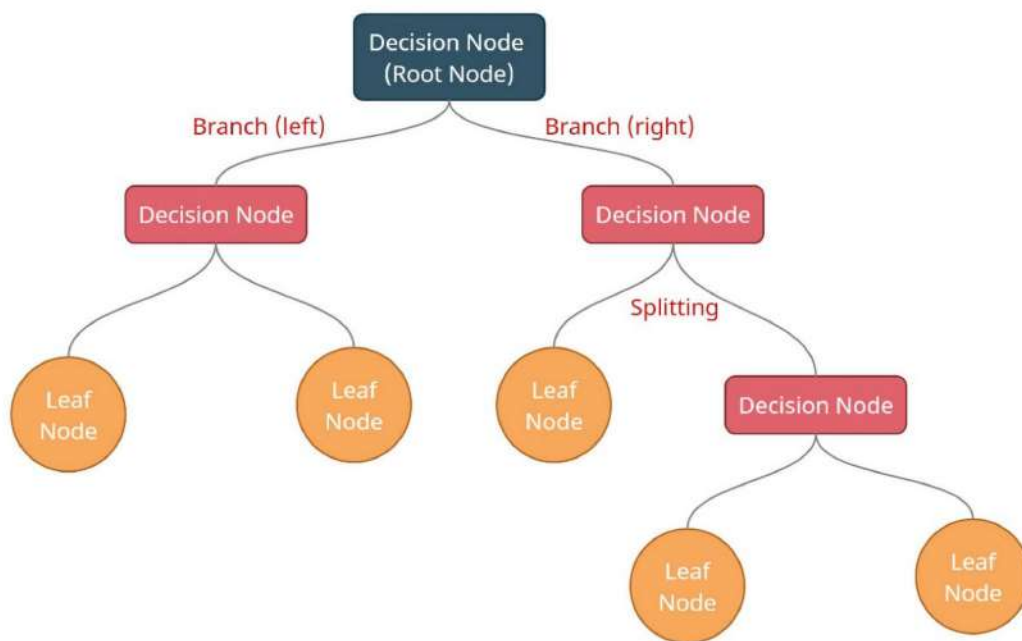


Figure 8.11: Anatomy of a decision tree

There are two types of nodes in a decision tree—a **decision node** and a **leaf node**. A decision node is the *if-else* statement we mentioned previously. This node will have a condition based on whether the data points that flow down the tree take the left or right **branch**. The decision node that sits right at the top has a special name—the **root node**. Finally, the process of dividing the data points based on a condition and directing it to the right or left branch is called **splitting**. Leaf nodes are nodes that don't have any other branches below them. These are the final resting points in the *beads flowing down a tree* analogy. These are the partitions we discussed earlier in this section.

Formally, we can define the function that's been generated by a decision tree that has M partitions, $P_1, P_2, \dots, P_M$, as follows:

$$\hat{y} = \sum_{m=1}^M c_m I(x \in P_m)$$

Here, x is the input, c_m is the constant response for the region, P_m , and I is a function that is 1 if $(x \in P_m)$; otherwise, it's 0.

For regression trees, we usually adopt the squared loss as the loss function. In that case, c_m is usually set as the average of all y , where the corresponding x falls in the P_m partition.

Now that we know how a decision tree functions, the only thing left to understand is how to decide which feature to split on and where to split the feature.



Many algorithms have been proposed over the years on how to create a decision tree from data such as ID3, C4.5, CART, and so on. Using **Classification and Regression Trees (CART)** is one of the most popular methods out of the lot and supports regression as well. Therefore, we will just stick to CART in this book. Classification Trees are used when the target variable is categorical (e.g., predicting a class label). Regression Trees are used when the target variable is continuous (e.g., predicting a numerical value).

The most optimal set of binary partitions that minimizes the sum of squares globally is generally intractable. So, we adopt a greedy algorithm to create the decision tree. Greedy optimization is a heuristic that builds up a solution stage by stage, selecting a local optimum at each stage. Therefore, instead of finding the best feature splits globally, we will create the decision tree, decision node by decision node, where we choose the most optimal feature split at each stage. For a regression tree, we choose a split feature, f , and split point, s , so that it creates two partitions, P_1 and P_2 , that minimize as follows:

$$\sum_{x_i \in P_1} (y_i - c_1)^2 + \sum_{x_i \in P_2} (y_i - c_2)^2$$

Here, c_1 and c_2 are the averages of all y , where the corresponding x falls in between P_1 and P_2 .

Therefore, by using this criterion, we can keep splitting the regions further and further. With each level we split, we increase the **depth** of the tree by one. At some point, we will start overfitting the dataset. But if we don't do enough splits, we might be underfitting the data as well. One strategy is to stop creating further splits when we reach a predetermined depth. In the scikit-learn implementation of `DecisionTreeRegressor`, this corresponds to the `max_depth` parameter. This is a hyperparameter that needs to be estimated using a validation dataset. There are other strategies to stop the splits, such as setting a minimum number of samples required to split (`min_samples_split`), or a minimum decrease in cost to carry out a split (`min_impurity_decrease`). For a complete list of parameters in `DecisionTreeRegressor`, please refer to the documentation at <https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeRegressor.html>.

Now, let's see how we can use a decision tree and evaluate the fit on a sample household from our validation dataset:

```
from sklearn.tree import DecisionTreeRegressor
model_config = ModelConfig(
    model=DecisionTreeRegressor(max_depth=4, random_state=42),
    name="Decision Tree",
    # Decision Tree is not affected by normalization
    normalize=False,
    # Decision Tree in scikit-learn does not handle missing values
    fill_missing=True,
)
y_pred, metrics, feat_df = evaluate_model(
    model_config,
    feat_config,
    missing_value_config,
    train_features,
    train_target,
    test_features,
    test_target,
)
```

Let's take a look at the single-step forecast from `DecisionTreeRegressor`. It's not doing as well as the linear or regularized linear regression models we have run so far:

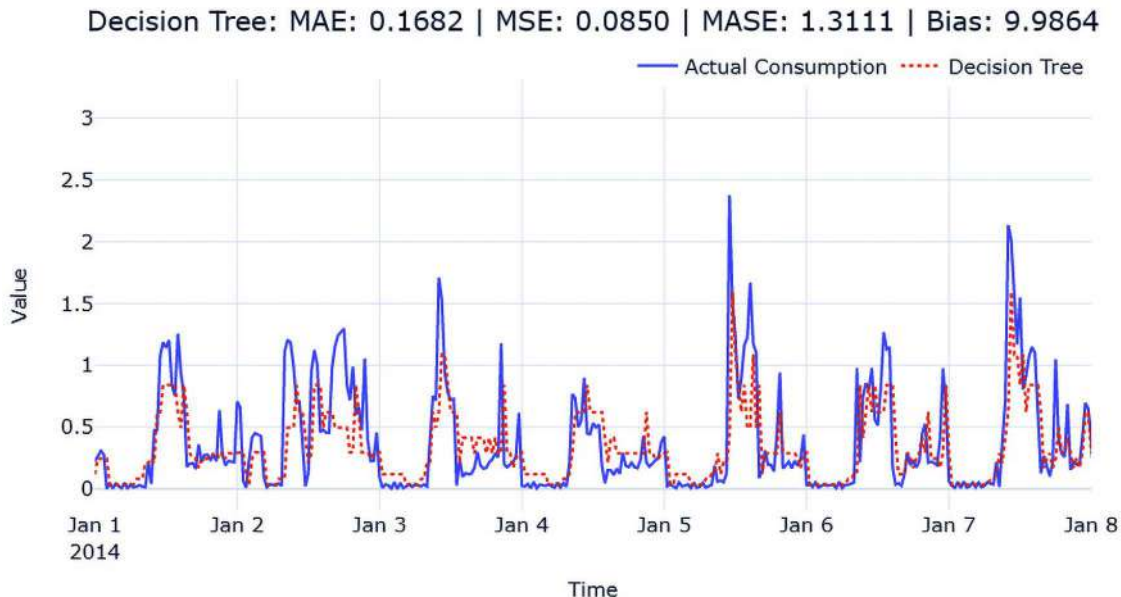


Figure 8.12: Decision tree forecast

For the linear models, some coefficients helped us understand how much each feature was important to the prediction function. In decision trees, we don't have any coefficients, but the feature importance is still estimated using the mean decrease in the loss function, which is attributed to each feature in the tree construction process. This can be accessed in scikit-learn models by using the `feature_importance_` attribute of the trained model. Let's take a look at this feature importance:



Figure 8.13: Feature importance of a decision tree (top 15)

Best practice:

Although the default feature importance is a quick and easy way to check how the different features are used, due diligence should be applied before using them for any other purposes, such as feature selection or making business decisions. This way of assessing feature importance gives misleadingly high values for some continuous features and high cardinality categorical features. It is recommended to use permutation importance (`sklearn.inspection.permutation_importance`) for an easy but better assessment of feature importance. The *Further reading* section contains some resources regarding the interpretability of models, which can be a good start to understanding what influences the models.

Here, we can see that the important features such as the lag and seasonal rolling features are coming up at the top.

We talked about overfitting and underfitting in *Chapter 5, Time Series Forecasting as Regression*. These are also referred to as high bias (underfitting) and high variance (overfitting) in machine learning parlance (the *Further reading* section contains links if you wish to read up more about bias and variance and the trade-off between them).

A decision tree is an algorithm that is highly prone to overfitting or high variance because, unlike the linear function, if given enough expressiveness, it can memorize the training dataset by partitioning the feature space. Another key disadvantage is a decision tree's inability to extrapolate. Let's consider a feature, f , that linearly increases our target variable, y . The training data we have has $f_{\max}$ as the maximum value for f and $y_{\max}$ as the maximum value for y . Since the decision tree partitions the feature space and assigns a constant value for that partition, even if we provide $f > f_{\max}$, we will still only get a prediction of $\hat{y} \leq y_{\max}$.

Now, let's look at a model that uses decision trees, but in an ensemble, and doesn't overfit as much.

Random forest

Random Forest is an ensemble learning method that builds multiple decision trees during training and merges their results for improved accuracy and robustness. It excels in both classification and regression tasks by reducing overfitting and enhancing predictive performance through bagging and feature randomness.

Ensemble learning is a process in which we use multiple models, or experts, and combine them in a way to solve the problem at hand. It taps into the *wisdom of the crowd* approach, which suggests that the decision-making of a group of people is typically better than any individual in that group. In the machine learning context, these individual models are called **base learners**. A single model may not perform well because it's overfitting the dataset, but when we combine multiple such models, they can form a strong learner.

Bagging is a form of ensemble learning where we use bootstrap sampling (sampling repeatedly with replacement from a population) to draw different subsets of the dataset, train weak learners on each of these subsets, and combine them by averaging or voting (for regression and classification, respectively). Bagging works best for high-variance, low-bias weak learners and the decision tree is a prime successful candidate with bagging. Theoretically, bagging maintains the same level of bias on the weak learners but reduces the variance, resulting in a better model. But if the weak learners are correlated with each other, the benefits of bagging will be limited.

In 2001, Leo Breiman proposed **Random Forest**, which substantially modifies standard bagging by building a large collection of decorrelated trees. He proposed to alter the tree-building procedure slightly to make sure all the trees that are grown on bootstrapped datasets are not correlated with each other.



Reference check:

The original research paper for Random Forest is cited in the *References* section as reference 1.

In the Random Forest algorithm, we decide how many trees to build. Let's call that M trees. Now, for each tree, the following steps are repeated:

1. Draw a bootstrap sample from the training dataset.
2. Select f features at random from all the features.

3. Pick the best split just using f features and split the node into two child nodes.
4. Repeat *steps* 2 and 3 until we hit any of the defined stopping criteria.

This set of M trees is the Random Forest. The key difference here from regular trees is the random sampling of features at each split, which increases randomness and reduces the correlation in the outputs of different trees. While predicting, we use each of these M trees to get a prediction. For regression problems, we average them, while for classification problems, we take the majority vote. The final prediction function that we learn from the Random Forest for regression is as follows:

$$\hat{y} = \frac{1}{M} \sum_{t=1}^M T_t(x)$$

Here, $T_t(x)$ is the output of the t^{th} tree in the Random Forest.

All the hyperparameters that we have to control the complexity of the decision tree are applicable here as well (RandomForestRegressor from scikit-learn). In addition to those, we have two other important parameters – the number of trees to build in the ensemble (`n_estimators`) and the number of features randomly chosen for each split (`max_features`).

Now, let's see how we can use Random Forest and evaluate the fit on a sample household from our validation dataset:

```
from sklearn.ensemble import RandomForestRegressor
model_config = ModelConfig(
    model=RandomForestRegressor(random_state=42, max_depth=4),
    name="Random Forest",
    # RandomForest is not affected by normalization
    normalize=False,
    # RandomForest in scikit-learn does not handle missing values
    fill_missing=True,
)
y_pred, metrics, feat_df = evaluate_model(
    model_config,
    feat_config,
    missing_value_config,
    train_features,
    train_target,
    test_features,
    test_target,
)
```

Let's take a look at this single-step forecast from RandomForestRegressor. It's better than the decision tree, but it's not as good as the linear models. However, we should keep in mind that we have not tuned the model and may be able to get better results by setting the right hyperparameters.

Now, let's take a look at the forecast that was generated using Random Forest:

Random Forest: MAE: 0.1657 | MSE: 0.0820 | MASE: 1.2913 | Bias: 7.7789

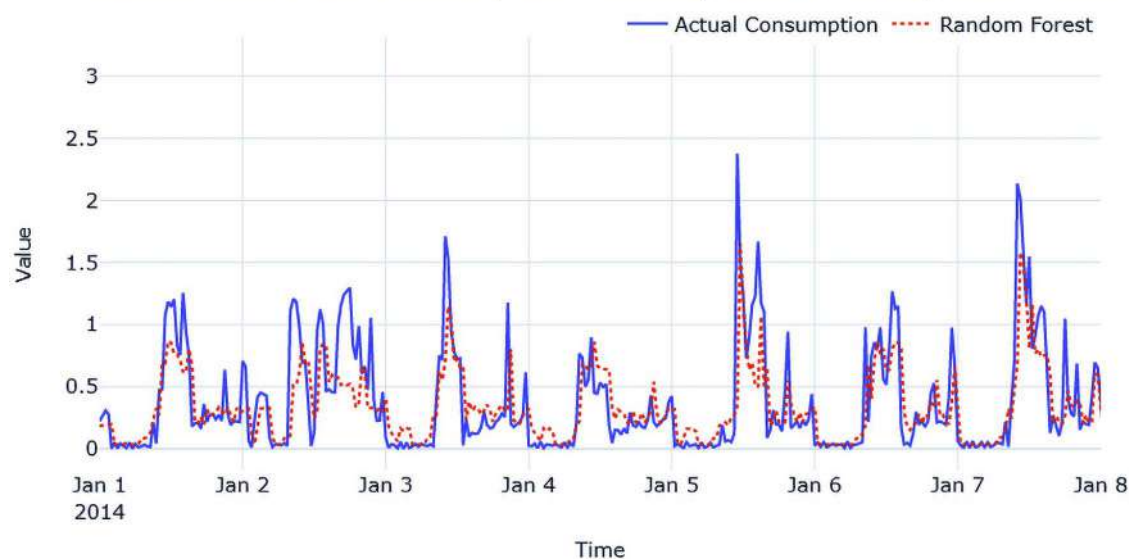


Figure 8.14: Random Forest forecast

Just like the feature importance in decision trees, Random Forests also have a very similar mechanism for estimating the feature importance. Since we have a lot of trees in the Random Forest, we accumulate the decrease in split criterion across all the trees in the forest and arrive at a single feature of importance for the Random Forest. This can be accessed in scikit-learn models by using the `feature_importance_` attribute of the trained model. Let's take a look at the feature importance:

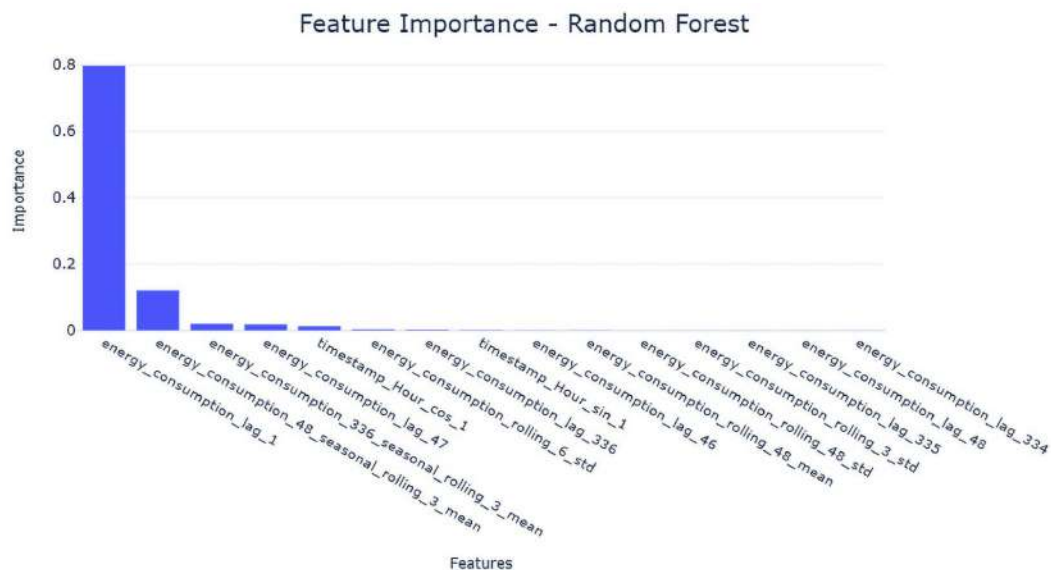


Figure 8.15: Feature importance of a decision tree (top 15)

Here, we can see that the feature importance is very similar to decision trees. The same caveat about this kind of feature importance applies here as well. This is just a quick and dirty way of looking at what the model is using internally.

Typically, Random Forest achieves good performance on many datasets with very little tuning, so Random Forests are a very popular option in machine learning. The fact that it is difficult to overfit with a Random Forest also increases their appeal. But since Random Forest uses decision trees as the weak learners, the inability of decision trees to extrapolate is passed down to Random Forest as well.



The scikit-learn implementation of Random Forest can get a bit slow for a large number of trees and data sizes. The `XGBRFRegressor` from the `XGBoost` library offers an alternative implementation of a Random Forest that can be faster, especially on larger datasets, due to `XGBoost`'s optimized algorithms and parallelization capabilities. Moreover, `XGBRFRegressor` uses similar hyperparameters to those in the scikit-learn Random Forest, making it relatively straightforward to switch between implementations while tuning the model. In most cases, this is a drop-in replacement and gives almost the same results. The minor difference is due to small implementation details. We have used this variant in the notebooks as well. This variant is preferred going forward because of obvious runtime considerations. It also handles missing values natively and saves us from an additional preprocessing step. More details about the implementation and how to use it can be found at <https://xgboost.readthedocs.io/en/latest/tutorials/rf.html>.

Now, let's look at one last family of functions that is one of the most powerful learning methods and has been proven exceedingly well in a wide variety of datasets—gradient boosting.

Gradient boosting decision trees

Boosting, like bagging, is another ensemble method that uses a few weak learners to produce a powerful committee of models. The key difference between bagging and boosting is in the way the weak learners are combined. Instead of building different models in parallel on bootstrapped datasets, as bagging does, boosting uses the weak learners in a sequential manner, with each weak learner applied to repeatedly modified versions of the data.

To understand the additive function formulation, let's consider this function:

$$F(x) = 25 + x^2 + \cos(x)$$

We can break this function into $f_1(x) = 25$, $f_2(x) = x^2$, $f_3(x) = \cos(x)$ and rewrite $F(x)$ as follows:

$$F(x) = f_1(x) + f_2(x) + f_3(x)$$

This is the kind of additive ensemble function we are learning in boosting. Although, in theory, we can use any weak learner, decision trees are the most popular choice. So, let's use decision trees to explore how gradient boosting works.

Earlier, when we were discussing decision trees, we saw that a decision tree that has M partitions, $P_1, P_2, \dots, P_M$, is as follows:

$$T(x) = \sum_{m=1}^M c_m I(x \in P_m)$$

Here, x is the input, c_m is the constant response for the region, P_m , and I is a function that is 1 if $x \in P_m$; otherwise, it is 0. A boosted decision tree model is a sum of such trees:

$$\hat{y} = \sum_{k=1}^M T_k(x)$$

Since finding the optimal partitions, P , and the constant value, c , for all the trees in the ensemble is a very difficult optimization problem, we usually adopt a suboptimal, stagewise solution where we optimize each step as we build the ensemble. In gradient boosting, we use the gradient of the loss to direct our optimization, hence the name.

Let the loss function we are using in the training be $L(\hat{y}, y)$. Since we are looking at a stagewise additive functional form, we can replace $\hat{y}_k$ with $\hat{y}_{k-1} + T_k(x)$, where $\hat{y}_{k-1}$ is the prediction of the sum of all trees until $k-1$ and $T_k(x)$ is the prediction of the tree at stage k . Let's look at what the gradient boosting learning procedure for training data D with N samples is:

1. Initialize the model with a constant value by minimizing the loss function:

$$F_{0(x)} = \underset{b_0}{\operatorname{argmin}} \sum_{i=1}^N L(y_i, b_0)$$

- b_0 is the prediction of the model that minimizes the loss function at the 0th iteration. At this iteration, we do not have any weak learners yet and this optimization is independent of any feature.
- For squared error loss, this works out to be the average of all training samples, while for the absolute error loss, it's the median.

2. Now that we have the initial solution, we can start the tree-building process. For $k=1$ to M , we must do the following:

- i. Compute $r_k = - \left[\frac{\delta L(y, F_{k-1}(x))}{\delta F_{k-1}(x)} \right]$ for all the training samples:
 - r_k is the derivative of the loss function with respect to $F(x)$ from the last iteration. It's also called pseudo-residuals.
 - For squared error loss, this is just the residual, $(\hat{y} - y)$.
- ii. Build a regular regression tree to the r_k values with M_k partitions or leaf nodes, P_{mk} .
- iii. Compute $\rho_t = \operatorname{arg min}_{\rho} \sum_{i=1}^N L(y_i, F_{k-1}(x_i) + \rho T_k(x_i))$
 - ρ_k is the scaling factor of the leaf or partition values for the current stage.

- $T_k(x_i)$ is the function that was learned by the decision tree from the current stage.
- iv. Update $F_k(x) = F_{k-1}(x) + \eta + \rho_k \times T_k(x_i)$
- η is the shrinkage parameter or learning rate.

This process of “boosting” the errors of the previous weak model gives the algorithm its name—gradient boosting, where the gradient here means the residual on the previous weak model.

Boosting, typically, is a high-variance algorithm. This means that the chance of overfitting the training dataset is quite high and that enough measures need to be taken to make sure it doesn’t happen. There are many ways regularization and capacity constraining have been implemented in gradient-boosted trees. As always, all the key parameters that decision trees have to reduce capacity to fit the data are valid here because the weak learner is a decision tree. In addition to that, there are two other key parameters – the number of trees, M (`n_estimators` in scikit-learn), and the learning rate, η (`learning_rate` in scikit-learn).

When we apply a learning rate in the additive formulation, we are essentially shrinking each weak learner, thus reducing the effect of any one weak learner on the overall function. This was originally referred to as shrinkage, but now, in all the popular implementations of gradient-boosted trees, it is referred to as the learning rate. The number of trees and the learning rate are highly interdependent. For the same problem, we will need a greater number of trees if we reduce the learning rate. It has been empirically shown that a lower learning rate improves the generalization error. Therefore, a very effective and convenient way is to set the learning rate to a very low value (<0.1), set a very high value for the number of trees ($>5,000$), and train the gradient-boosted tree with early stopping. Early stopping is when we use a validation dataset to monitor the out-of-sample performance while training the model. We stop adding more trees to the ensemble when the out-of-sample error stops reducing.

Another key technique a lot of the implementations adopt is subsampling. Subsampling can be done on rows and columns. Row subsampling is similar to bootstrapping, where each candidate in the ensemble is trained on a subsample of the dataset. Column subsampling is similar to random feature selection in Random Forest. Both of these techniques introduce a regularization effect to the ensemble and help reduce generalization errors. Some implementations of gradient-boosted trees, such as XGBoost and LightGBM, implement L1 and L2 regularization directly in the objective function as well.

There are many implementations of regression gradient-boosted trees. A few popular implementations are as follows:

- GradientBoostingRegressor and HistGradientBoostingRegressor in scikit-learn
- XGBoost by T Chen
- LightGBM from Microsoft
- CatBoost from Yandex

Each of these implementations offers changes that range from subtle to very fundamental regarding the standard gradient boosting algorithm. We have included a few resources in the *Further reading* section so that you can read up on these differences and get acquainted with the different parameters they support.

For our exercise, we are going to use LightGBM from Microsoft Research because it is one of the fastest and best-performing implementations. LightGBM and CatBoost also support categorical features out of the box and handle missing values natively.

**Reference check:**

The original research papers for XGBoost, LightGBM, and CatBoost are cited in the *References* section as 2, 3, and 4, respectively.

Now, let's see how we can use LightGBM and evaluate the fit on a sample household from our validation dataset:

```
from lightgbm import LGBMRegressor
model_config = ModelConfig(
    model=LGBMRegressor(random_state=42),
    name="LightGBM",
    # LightGBM is not affected by normalization
    normalize=False,
    # LightGBM handles missing values
    fill_missing=False,
)
y_pred, metrics, feat_df = evaluate_model(
    model_config,
    feat_config,
    missing_value_config,
    train_features,
    train_target,
    test_features,
    test_target,
)
```

Let's take a look at the single-step forecast from LGBMRegressor. It's already significantly better than all the other models we have tried so far:

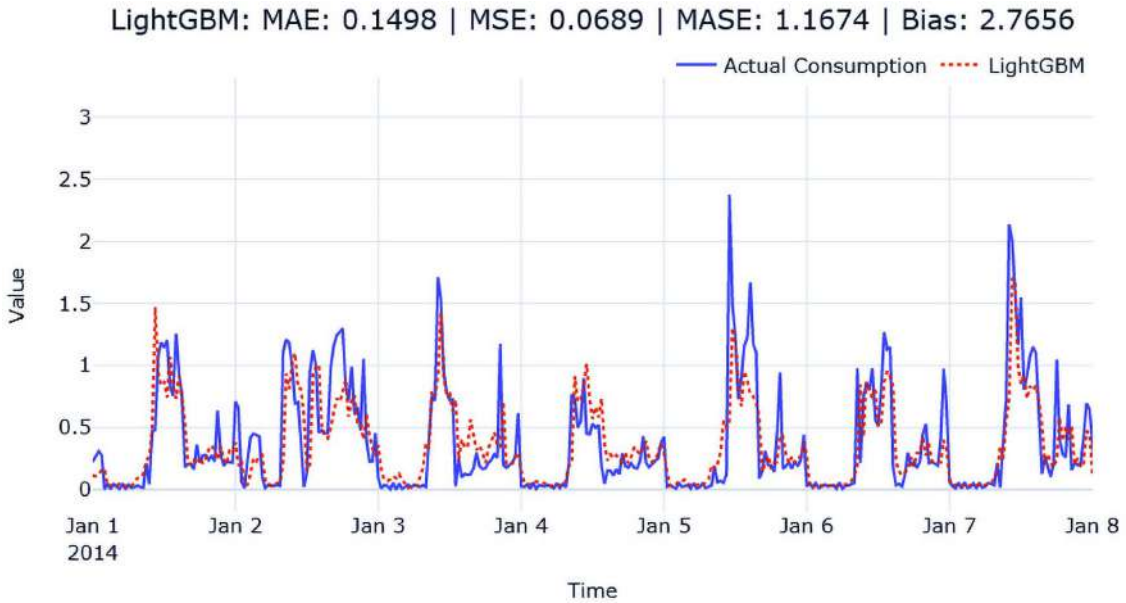


Figure 8.16: LightGBM forecast

Just like the feature importance in decision trees, gradient-boosting implementations also have a very similar mechanism for estimating the feature importance. The feature importance for the ensemble is given by the average of split criteria reduction attributed to each feature in all the trees. This can be accessed in the scikit-learn API as the `feature_importance_` attribute of the trained model. Let’s take a look at the feature importance:

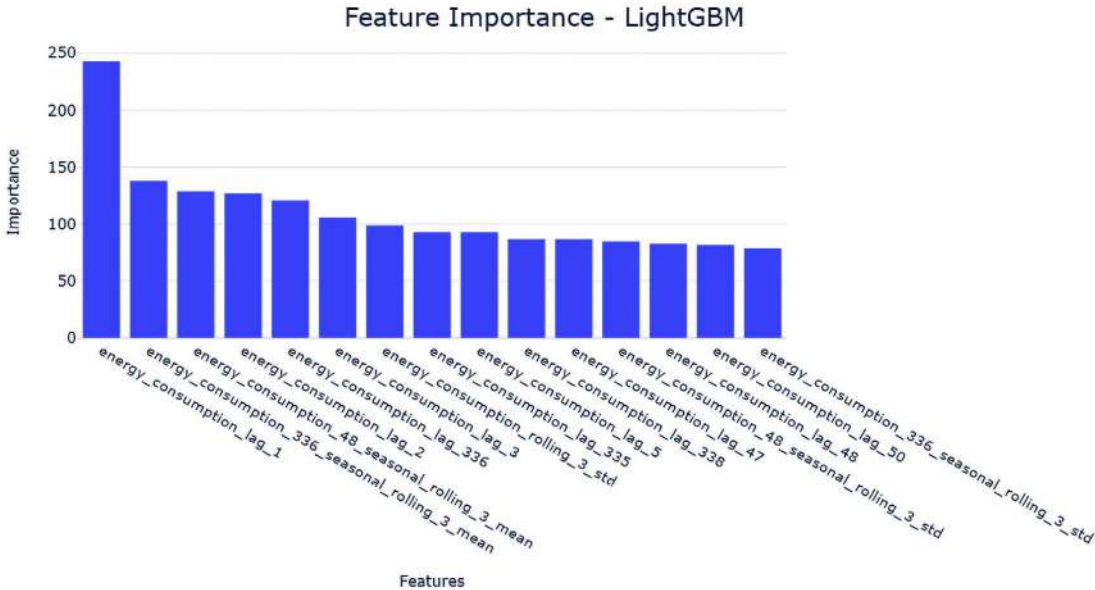


Figure 8.17: Feature importance of LightGBM (top 15)

There are multiple ways of getting feature importance from the model, and each implementation has slightly different ways of calculating it. This is controlled by parameters. The most common ways of extracting it (sticking to LightGBM terminology) are `split` and `gain`. If we choose `split`, the feature importance is the number of times a feature is used to split nodes in the trees. On the other hand, `gain` is the total reduction in the split criterion. This can be attributed to any feature. *Figure 8.17* shows `split`, which is the default value in LightGBM. We can see that the order of the feature importance is very much similar to decision trees, or Random Forests, with almost the same features taking the top three spots.

Gradient boosted decision trees (GBDTs) typically give us very good performance on tabular data and time series as regression is no exception. This very strong model has usually been part of almost all winning entries in Kaggle competitions on time series forecasting in the recent past. While it is one of the best machine learning model families, it still has a few disadvantages:

- GBDTs are high-variance algorithms and hence prone to overfitting. This is why all kinds of regularizations are applied in different ways in most of the successful implementations of GBDTs.
- GBDTs usually take longer to train (although many modern implementations have made this faster) and are not easily parallelizable as a Random Forest. In Random Forest, we can train all the trees in parallel because they are independent of each other. But in GBDTs, the sequential nature of the algorithm restricts parallelization. All the successful implementations have clever ways of enabling parallelization when creating a decision tree. LightGBM has many parallelization strategies, such as feature parallel, data parallel, and voting parallel. Details regarding these can be found at <https://lightgbm.readthedocs.io/en/latest/Features.html#optimization-in-distributed-learning> and are worth understanding. The documentation of the library also contains a helpful guide for choosing between these parallelization strategies in a table:

	Data is Small	Data is Large
# of Features is Small	Feature Parallel	Data Parallel
# of Features is Large	Feature Parallel	Voting Parallel

Figure 8.18: Parallelization strategies in LightGBM

- Extrapolation is a problem for GBDTs, just like it is a problem for all tree-based models. There is some very weak potential for extrapolation in GBDTs, but nothing that solves the problem. Therefore, if your time series has some strong trends, tree-based methods will, most likely, fail to capture the trend. Either training the model on detrended data or switching to another model class would be the way forward. An easy way to do detrending would be to use `AutoStationaryTransformer`, which we discussed in *Chapter 6, Feature Engineering for Time Series Forecasting*.

To summarize, let's look at the metrics and runtime that were taken by these machine learning models. If you have run the notebook along with this chapter, then you will find the following summary table there as well:

Algorithm	MAE	MSE	MASE	Forecast Bias	Time Elapsed
Naive	0.1753	0.1050	1.3664	0.03%	nan
Seasonal Naive	0.2377	0.1709	1.8521	4.80%	nan
Linear Regression	0.1595	0.0748	1.2431	6.18%	0.490227
Ridge Regression	0.1595	0.0748	1.2430	6.16%	0.425553
Lasso Regression	0.1599	0.0743	1.2463	3.67%	0.949244
Decision Tree	0.1682	0.0850	1.3111	9.99%	0.474296
Random Forest	0.1657	0.0820	1.2913	7.78%	26.353781
XGB Random Forest	0.1644	0.0818	1.2808	9.35%	1.786139
LightGBM	0.1498	0.0689	1.1674	2.77%	0.435123

Figure 8.18: Summary of the metrics and runtimes for a sample household

Right off the bat, we can see that all of the machine learning models we tried have performed better than the baselines in all metrics except the forecast bias. The three linear regression models perform well with almost equal performance on MAE, MASE, and MSE, with a slight increase in runtimes for regularized models. The decision tree has underperformed, but this is usually expected. Decision trees need to be tuned a little better to reduce overfitting. Random Forest (both the scikit-learn and XGBoost implementations) has improved the decision tree's performance, which is what we would expect. One key thing to note here is that the XGBoost implementation of Random Forest is almost six times faster than the scikit-learn one. Finally, LightGWM has the best performance across all metrics and a faster runtime.

Now, this was just one household out of all the selected ones. To see how well these models are doing, we need to evaluate them on all selected households.

Training and predicting for multiple households

We have picked a few models (LassoCV, XGBRFRegressor, and LGBMRegressor) that are doing better in terms of metrics, as well as runtime, to run on all the selected households in our validation dataset. The process is straightforward: loop over all the unique combinations, inner loop over the different models to run, and then train, predict, and evaluate. The code is available in the 01-Forecasting_with_ML.ipynb notebook in Chapter08, under the *Running an ML Forecast For All Consumers* heading. You can run the code and take a break because this is going to take a little less than an hour. The notebook also calculates the metrics and contains a summary table that will be ready for you when you're back.

Let's look at the summary now:

Algorithm	MAE	MSE	meanMASE	Forecast Bias
Naive	0.0882	0.0450	1.1014	-0.00%
Seasonal Naive	0.1292	0.0777	1.6004	-1.00%
Lasso Regression	0.0802	0.0271	1.0052	-0.29%
XGB Random Forest	0.0808	0.0306	1.0177	-2.43%
LightGBM	0.0772	0.0275	0.9781	0.05%

Figure 8.19: Aggregate metrics on all the households in the validation dataset

Here, we can see that even at the aggregated level, the different models we used perform as expected. The notebook also saves the predictions for the validation set on disk.



Notebook alert:

We also need to run another notebook, called `01a-Forecasting_with_ML_for_Test_Dataset.ipynb`, in Chapter08. This notebook follows the same process, generates the forecast, and calculates the metrics on the test dataset.

The aggregate metrics for the test dataset are as follows (from the notebook):

Algorithm	MAE	MSE	meanMASE	Forecast Bias
Naive	0.086	0.045	1.050	0.02%
Seasonal Naive	0.122	0.072	1.487	4.07%
Lasso Regression	0.077	0.026	0.946	0.99%
XGB Random Forest	0.078	0.030	0.966	-0.18%
LightGBM	0.075	0.027	0.914	2.57%

Figure 8.20: Aggregate metrics on all the households in the test dataset

In *Chapter 6, Feature Engineering for Time Series Forecasting*, we used `AutoStationaryTransformer` (not the `Transformer` model, which we will learn about in *Chapter 14*) on all the households and saved the transformed dataset.

Using `AutoStationaryTransformer`

The process is really similar to what we did earlier in this chapter, but with small changes. We read in the transformed targets and joined them to our regular dataset in such a way that the original target is named `energy_consumption` and the transformed target is named `energy_consumption_auto_stat`:

```
#Reading the missing value imputed and train test split data
train_df = pd.read_parquet(preprocessed/"block_0-7_train_missing_imputed_
feature_engg.parquet")
auto_stat_target = pd.read_parquet(preprocessed/"block_0-7_train_auto_stat_
target.parquet")
transformer_pipelines = joblib.load(preprocessed/"auto_transformer_pipelines_
train.pkl")
#Reading in validation as test
test_df = pd.read_parquet(preprocessed/"block_0-7_val_missing_imputed_feature_
engg.parquet")
# Joining the transformed target
train_df = train_df.set_index(['LCLId', 'timestamp']).join(auto_stat_target).
reset_index()
```

And while defining `FeatureConfig`, we used `energy_consumption_auto_stat` as target and `energy_consumption` as `original_target`.



Notebook alert:

The `02-Forecasting_with_ML_and_Target_Transformation.ipynb` and `02a-Forecasting_with_ML_and_Target_Transformation_for_Test_Dataset.ipynb` notebooks use these transformed targets to generate the forecasts for the validation and test datasets, respectively.

Let's look at the summary metrics that were generated by these notebooks on the transformed data:

Algorithm	MAE	MSE	meanMASE	Forecast Bias
Naive	0.088	0.045	1.101	-0.00%
Seasonal Naive	0.129	0.078	1.600	-1.00%
Lasso Regression	0.080	0.027	1.005	-0.29%
XGB Random Forest	0.081	0.031	1.018	-2.43%
LightGBM	0.077	0.028	0.978	0.05%
Lasso Regression_auto_stat	0.083	0.030	1.055	-3.50%
XGB Random Forest_auto_stat	0.086	0.033	1.098	-8.33%
LightGBM_auto_stat	0.079	0.029	1.002	-4.41%

Figure 8.21: Aggregate metrics on all the households with transformed targets in the validation dataset

The target transformed models are not performing as well as the original ones. This might be because the dataset doesn't have any strong trends.

Congratulations on making it through a very heavy and packed chapter full of theory as well as practice. We hope this has enhanced your understanding of machine learning and ability to apply these modern techniques to time series data.

Summary

This was a very practical and hands-on chapter in which we developed some standard code to train and evaluate multiple machine learning models. Then, we reviewed a few key machine learning models like ridge regression, lasso regression, decision trees, Random Forest, and gradient-boosted trees and how they work behind the hood. To complete and reinforce what we learned, we applied the machine learning models we learned about to the London Smart Meters dataset and saw how well they did. This chapter sets you up to tackle the coming chapters, where we will use the standardized code and these models to go deeper into forecasting with machine learning.

In the next chapter, we will start combining different forecasts into a single forecast and explore concepts such as combinatorial optimization and stacking to achieve state-of-the-art results.

References

The following references were provided in this chapter:

1. Breiman, L. Random Forests, Machine Learning 45, 5–32 (2001): <https://doi.org/10.1023/A:1010933404324>.

2. Chen, Tianqi and Guestrin, Carlos. (2016). *XGBoost: A Scalable Tree Boosting System*. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '16). Association for Computing Machinery, New York, NY, USA, 785–794: <https://doi.org/10.1145/2939672.2939785>.
3. Ke, Guolin et.al. (2017), *LightGBM: A Highly Efficient Gradient Boosting Decision Tree*. Advances in Neural Information Processing Systems, pages 3149-3157: <https://dl.acm.org/doi/pdf/10.5555/3294996.3295074>.
4. Prokhorenkova, Liudmila et al. (2018), *CatBoost: unbiased boosting with categorical features*. Proceedings of the 32nd International Conference on Neural Information Processing Systems (NIPS'18): <https://dl.acm.org/doi/abs/10.5555/3327757.3327770>.

Further reading

To learn more about the topics that were covered in this chapter, take a look at the following resources:

- *The difference between L1 and L2 regularization*, by Terrence Parr: <https://explained.ai/regularization/L1vsL2.html>
- *L1 Norms versus L2 Norms*, by Aleksey Bilogur: <https://www.kaggle.com/residentmario/l1-norms-versus-l2-norms>
- *Interpretability – Cracking Open the Black Box*, by Manu Joseph: <https://deep-and-shallow.com/2019/11/13/interpretability-cracking-open-the-black-box-part-ii/>
- *The Gradient Boosters – Part III: XGBoost*, by Manu Joseph: <https://deep-and-shallow.com/2020/02/12/the-gradient-boosters-iii-xgboost/>
- *The Gradient Boosters – Part IV: LightGBM*, by Manu Joseph: <https://deep-and-shallow.com/2020/02/21/the-gradient-boosters-iii-lightgbm/>
- *The Gradient Boosters – Part V: CatBoost*, by Manu Joseph: <https://deep-and-shallow.com/2020/02/29/the-gradient-boosters-v-catboost/>
- *The Gradient Boosters – Part II: Regularized Greedy Forest*, by Manu Joseph: <https://deep-and-shallow.com/2020/02/09/the-gradient-boosters-ii-regularized-greedy-forest/>
- *LightGBM Distributed Learning Guide*: <https://lightgbm.readthedocs.io/en/latest/Parallel-Learning-Guide.html>

Unlock this book's exclusive benefits now

This book comes with additional benefits designed to elevate your learning experience.



Note: Have your purchase invoice ready before you begin. <https://www.packtpub.com/unlock/9781835883181>

9

Ensembling and Stacking

In the previous chapter, we looked at a few machine learning algorithms and used them to generate forecasts on the London Smart Meters dataset. Now that we have multiple forecasts for all the households in the dataset, how do we come up with a single forecast by choosing or combining these different forecasts? At the end of the day, we can only have one forecast that will be used for planning whatever task for which you are forecasting. That is what we will be doing in this chapter—we will learn how to leverage combinatorial and mathematical optimization to come up with a single forecast.

In this chapter, we will cover the following topics:

- Strategies for combining forecasts
- Stacking or blending

Technical requirements

You will need to set up the Anaconda environment following the instructions in the *Preface* of the book to get a working environment with all the libraries and datasets required for the code in this book. Any additional library will be installed while running the notebooks.

You will need to run the following notebooks before using the code in this chapter:

- 02-Preprocessing_London_Smart_Meter_Dataset.ipynb in Chapter02
- 01-Setting_up_Experiment_Harness.ipynb in Chapter04
- 02-Baseline_Forecasts_using_darts.ipynb in Chapter04
- 01-Feature_Engineering.ipynb in Chapter06
- 02-Dealing_with_Non-Stationarity.ipynb in Chapter07
- 02a-Dealing_with_Non-Stationarity-Train+Val.ipynb in Chapter07
- 00-Single_Step_Backtesting_Baselines.ipynb in Chapter08
- 01-Forecasting_with_ML.ipynb in Chapter08
- 01a-Forecasting_with_ML_for_Test_Dataset.ipynb in Chapter08
- 02-Forecasting_with_Target_Transformation.ipynb in Chapter08
- 02a-Forecasting_with_Target_Transformation(Test).ipynb in Chapter08

The code for this chapter can be found at <https://github.com/PacktPublishing/Modern-Time-Series-Forecasting-with-Python-/tree/main/notebooks/Chapter09>.

Combining forecasts

We have generated forecasts using many techniques—some univariate, some machine learning, and so on. But at the end of the day, we would need a single forecast, and that means choosing a forecast or combining a variety. The most straightforward option is to choose the algorithm that does the best in the validation dataset, which in our case is `LightGBM`. We can think of this *selection* as another function that takes the forecasts that we generated as inputs and combines them into a final forecast. Mathematically, this can be represented as follows:

$$Y = F(Y_1, Y_2, \dots, Y_N)$$

Here, F is the function that combines N forecasts. We can use the F function to choose the best-performing model in the validation dataset. However, this function can be as complex as it wants to be, and choosing the right F function while balancing bias and variance is a must.



Notebook alert:

To follow along with the code, use the `01-Forecast_Combinations.ipynb` notebook in the `Chapter09` folder.

We will start by loading all the forecasts (both the validation and test forecasts) and the corresponding metrics for all the forecasts we have generated so far and combining them into `pred_val_df` and `pred_test_df`. Now, we must reshape the DataFrame using `pd.pivot` to get it into the shape we want. Up to this point, we have been tracking multiple metrics. But to meet this objective, we will need to choose one. For this exercise, we are going to choose the MAE as the metric. The validation metrics can be combined and reshaped into `metrics_combined_df`:

		FFT	Lasso Regression	Lasso Regression_auto_stat	LightGBM	LightGBM_auto_stat	Theta	XGB Random Forest	XGB Random Forest_auto_stat	energy_consumption
LCLid	timestamp									
MAC000002	2014-01-01 00:00:00	0.255057	0.455158	0.418127	0.419273	0.414226	0.398836	0.39043	0.347244	0.496
	2014-01-01 00:30:00	0.234834	0.471182	0.422963	0.425602	0.403881	0.380464	0.39043	0.328894	0.427
	2014-01-01 01:00:00	0.207254	0.443879	0.418469	0.373510	0.378720	0.369484	0.39043	0.317926	0.469
	2014-01-01 01:30:00	0.175136	0.438553	0.382969	0.345316	0.333053	0.328537	0.39043	0.276990	0.362
	2014-01-01 02:00:00	0.144155	0.309471	0.295905	0.304884	0.277279	0.306195	0.36219	0.242762	0.452

Figure 9.1: Reshaped predictions DataFrame

Now, let's look at some different strategies for combining the forecasts.

Best fit

This strategy for choosing the best forecast is by far the most popular and is as simple as choosing the best forecast for each time series based on the validation metrics. This strategy has been made popular by many automated forecasting software tools, which call this the “best fit” forecast. The algorithm is very simple:

1. Find the best-performing forecast for each time series using a validation dataset.
2. For each time series, select the forecast from the same model for the test dataset.

We can do this easily:

```
# Finding the lowest metric for each LCLid
best_alg = metrics_combined_df.idxmin(axis=1)
#Initialize two columns in the dataframe
pred_wide_test["best_fit"] = np.nan
pred_wide_test["best_fit_alg"] = ""
#For each LCL id
for lcl_id in tqdm(pred_wide_test.index.get_level_values(0).unique()):
    # pick the best algorithm
    alg = best_alg[lcl_id]
    # and store the forecast in the best_fit column
    pred_wide_test.loc[lcl_id, "best_fit"] = pred_wide_test.loc[lcl_id, alg].
values
    # also store which model was chosen for traceability
    pred_wide_test.loc[lcl_id, "best_fit_alg"] = alg
```

This will create a new column called `best_fit` with the forecasts that have been chosen according to the strategy we discussed. Now, we can evaluate this new forecast and get the metrics for the test dataset. The following table shows the best individual model (LightGBM) and the new strategy—`best_fit`:

Algorithm	MAE	MSE	meanMASE	Forecast Bias
LightGBM	0.0751	0.0271	0.9142	2.57%
best_fit	0.0740	0.0269	0.8971	0.14%

Figure 9.2: Aggregate metrics for the best-fit strategy

Here, we can see that the best-fit strategy is performing better than the best individual model overall. However, one drawback of this strategy is the fundamental assumption of this strategy—whatever model does best in the validation period also performs best in the test period. There is no hedging of bets with other forecasting models and so on. Given the dynamic nature of time series, this is not always the best strategy. Another drawback of this approach is the instability of the final forecast.

When we are using such a rule in a live environment, where we retrain and rerun the best fit every week, the forecast for any time series can jump back and forth between different forecast models, which can generate wildly different forecasts. Therefore, the final forecast shows a lot of week-over-week instability, which hampers the downstream actions we use these forecasts for. We can look at a few other techniques that don't have this instability.

Measures of central tendency

Another prominent strategy is to use either an average or median to combine the forecasts. This is a function, F , that is independent of validation metrics. This is both the appeal and angst of this method. It is impossible to overfit the validation metrics because we are not using them at all. But on the other hand, without any information from the validation metric, we may be including some very bad models, which pulls down the ensemble. However, empirically, this simple averaging of taking the median has proven to be a very strong combination method for forecast and is hard to outperform. Let's see how this can be done:

```
# ensemble_forecasts is a list of column names(forecast) we want to combine
pred_wide_test["average_ensemble"] = pred_wide_test[ensemble_forecasts].
mean(axis=1)
pred_wide_test["median_ensemble"] = pred_wide_test[ensemble_forecasts].
median(axis=1)
```

The preceding code will create two new columns called `average_ensemble` and `median_ensemble` with the combined forecasts. Now, we can evaluate this new forecast and get the metrics for the test dataset. The following table shows the best individual model (LightGBM) and the new strategies:

Algorithm	MAE	MSE	meanMASE	Forecast Bias
LightGBM	0.0751	0.0271	0.9142	2.57%
best_fit	0.0740	0.0269	0.8971	0.14%
median_ensemble	0.0767	0.0279	0.9304	-0.86%
average_ensemble	0.0828	0.0285	1.0159	1.58%

Figure 9.3: Aggregate metrics for the mean and median strategies

Here, we can see that neither the mean nor median strategy is working better than the best individual model overall. This can be because we are including methods such as Theta and FFT, which are performing considerably worse than the other machine learning methods. But since we are not taking any information from the validation dataset, we do not know this information. We can make an exception and say that we are going to use the validation metrics to choose which models we include in the average or median. But we have to be careful because now, we are moving closer to the assumption that what works in the validation period is going to work in the test period.

There are a few manual techniques we can use here, such as **trimming** (discarding the worst-performing models in the ensemble) and **skimming** (selecting only the best few models in the ensemble). While effective, these are a bit subjective, and often, they become hard to use, especially when we have scores of models to choose from.

If we think about this problem, it is essentially a combinatorial optimization problem where we have to select the best combination of models that optimizes our metric. If we consider the average for combining the different forecasts, mathematically, it can be thought of as follows:

$$\hat{Y} = \operatorname{argmin}_L \left(\frac{1}{\sum_{i=1}^N w_i} \sum_{i=1}^N w_i \times \hat{Y}_i, Y \right)$$

Here, L is the loss or metric that we are trying to minimize. In our case, we chose that to be the MAE. $w_N \in [0,1]$ is the binary weight of each of the base forecasts. Finally, $\hat{Y}_N$ is the set of N base forecasts and Y is the real observed values of the time series.

But unlike pure optimization, where there is no concept of bias and variance, we need an optimal solution that can be generalized. Therefore, selecting the global minima in the training data is not advisable because in that case, we might be further overfitting the training dataset, increasing the variance of the resulting model. For this minimization, we typically use out-of-sample predictions, which can be the forecast during the validation period in this case.

The most straightforward solution is to find w , which minimizes this function on validation data. But there are two problems with this approach:

- The possible candidates (different combinations of the base forecasts) increase exponentially as we increase the number of base forecasts, N . This becomes computationally intractable very soon.
- Selecting the global minima in the validation period may not be the best strategy because of overfitting the validation period.

Now, let's take a look at a few heuristics-based solutions to this combinatorial optimization problem.



Heuristic problem-solving is a strategy that uses rules of thumb or shortcuts to find solutions quickly, even if they may not be optimal. Heuristics can be useful when exact solutions are computationally expensive or time-consuming to find. However, they may lead to suboptimal or even incorrect solutions in some cases.

Heuristics are often used in conjunction with other problem-solving methods, such as metaheuristics, to improve the efficiency and effectiveness of the search process. Metaheuristics are high-level, problem-independent strategies used to solve optimization problems. They offer a framework for developing heuristic algorithms that can efficiently explore complex search spaces and find near-optimal solutions. Unlike traditional optimization methods, metaheuristics often draw inspiration from natural phenomena or biological processes.



Common examples of metaheuristic methods include genetic algorithms (inspired by natural selection), simulated annealing (inspired by metallurgy), particle swarm optimization (inspired by bird flocking), and ant colony optimization (inspired by ants foraging for food). These methods employ probabilistic or stochastic approaches to balance exploration and exploitation, allowing them to avoid getting stuck in local optima and discover potentially better solutions.

Simple hill climbing

We briefly talked about greedy algorithms while discussing decision trees, as well as gradient-boosted trees. Greedy optimization is a heuristic that builds up a solution stage by stage, selecting a local optimum at each stage. In both of these machine learning models, we adopt a greedy, stagewise approach to finding the solution to a computationally infeasible optimization problem. To select the best subset that gives us the best combination of forecasts, we can employ a simple greedy algorithm called hill climbing. If we consider the objective function surface as a hill, to find the maxima, we would need to climb the hill. As its name suggests, hill climbing ascends the hill, one step at a time, and in each of those steps, it takes the best possible path, which increases the objective function. The illustration below can make it clearer.

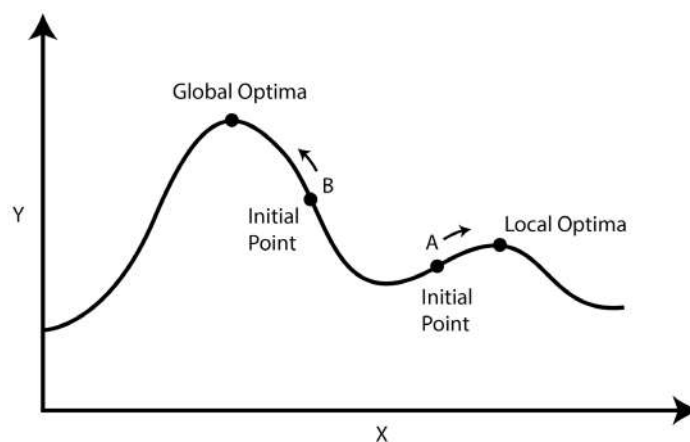


Figure 9.4: Hill climbing algorithm illustration for a one-dimensional objective

We can see in Figure 9.4 that the objective function (the function we need to optimize) has multiple peaks (hills) and in the hill climbing algorithm, we “climb” the hill to reach the peak in a step-by-step manner. What we also need to keep in mind is that depending on where we start the climb, we might reach different points in the objective. In Figure 9.4, if we start at point A, we reach the local optima and miss the global optima. Now, let’s see how the algorithm works in a more rigorous manner.

Here, C is a set of candidates (base forecasts) and O is the objective we want to minimize. The algorithm for the simple hill-climb is as follows:

1. Initialize a starting solution, C_{best} , as the candidate that gives the minimum value in O , O_{best} , and remove C_{best} from C .
2. While the length of $C > 0$, do the following:
 - i. Evaluate all members of C by averaging the base forecasts in C_{best} with each element in C and select the best member ($C_{stage\ best}$) that was added to C_{best} to minimize the objective function, O ($O_{stage\ best}$).
 - ii. If $O_{stage\ best} > O_{best}$, then do the following:
 - a. $C_{best} = C_{best} \cup C_{stage\ best}$.
 - b. $O_{best} = O_{stage\ best}$.
 - c. Remove $C_{stage\ best}$ from C .
 - iii. Otherwise, exit.

At the end of the run, we have C_{best} , which is the best combination of forecasts we got through greedy optimization. We have made an implementation of this available in `src.forecasting.ensembling.py` under the `greedy_optimization` function. The parameters for this function are as follows:

- **objective:** This is a callable that takes in a list of strings as the candidates and returns a float objective value.
- **candidates:** This is a list of candidates to be included in the optimization.
- **verbose:** A flag that specifies whether progress is printed or not.

The function returns a tuple of the best solution as a list of strings and the best score that was obtained through optimization.

Let's see how we can use this in our example:

1. Import all the required libraries/functions:

```
# Used to partially construct a function call

from functools import partial

# calculate_performance is a custom method we defined to calculate the
MAE provided a list of candidates and prediction dataframe

from src.forecasting.ensembling import calculate_performance, greedy_
optimization
```

2. Define the objective function and run greedy optimization:

```
# We partially construct the function call by passing the necessary
parameters
objective = partial(
    calculate_performance, pred_wide=pred_wide_val, target="energy_
consumption"
)
# ensemble forecasts is the list of candidates
solution, best_score = greedy_optimization(objective, ensemble_forecasts)
```

3. Once we have the best solution, we can create the combination forecast in the test DataFrame:

```
pred_wide_test["greedy_ensemble"] = pred_wide_test[solution].mean(axis=1)
```

Once we run this code, we will have the combination forecast under the name `greedy_ensemble` in our prediction DataFrame. The candidates that are part of the optimal solution are LightGBM, Lasso Regression, and LightGBM_auto_stat. Now, let's evaluate the results and look at the aggregated metrics:

Algorithm	MAE	MSE	meanMASE	Forecast Bias
LightGBM	0.0751	0.0271	0.9142	2.57%
best_fit	0.0740	0.0269	0.8971	0.14%
median_ensemble	0.0767	0.0279	0.9304	-0.86%
average_ensemble	0.0828	0.0285	1.0159	1.58%
greedy_ensemble	0.0733	0.0251	0.8951	0.81%

Figure 9.5: Aggregate metrics for a simple hill climbing-based ensemble

As we can see, the simple hill-climb is performing better than any individual models or ensemble techniques we have seen so far. This greedy approach seems to be working well in this case. Now, let's understand a few limitations of hill climbing, as follows:

- **Runtime considerations:** Since a simple hill-climb requires us to evaluate all the candidates at any step, this can cause a bottleneck in terms of runtime. If the number of candidates is large, this approach can take longer to finish.
- **Short-sightedness:** Hill climbing optimization is short-sighted. During optimization, it always picks the best in each step. Sometimes, by choosing a slightly worse solution in a step, we may get to a better overall solution.
- **Forward-only:** Hill climbing is a forward-only algorithm. Once a candidate has been admitted into the solution, we can't go back and remove it.

The greedy approach may not always get us the best solution, especially when there are scores of models to combine. So, let's look at a small variation of hill climbing that tries to get over some of the limitations of the greedy approach.

Stochastic hill climbing

The key difference between simple hill climbing and stochastic hill climbing is in the evaluation of candidates. In a simple hill climb, we *evaluate all possible options* and pick the best among them. However, in a stochastic hill climb, we *randomly pick a candidate* and add it to the solution if it is better than the current solution. In other words, in hill climbing we always move up the hill in steps, but in Stochastic hill climbing, we magically teleport to different points in the objective function and check we are higher than we have been before. This addition of stochasticity helps the optimization not get the local maxima/minima, but also introduces quite a bit of uncertainty in reaching any kind of optima. Let's take a look at the algorithm.

Here, C is a set of candidates (base forecasts), O is the objective we want to minimize, and N is the maximum number of iterations we want to run the optimization for. The algorithm for stochastic hill climbing is as follows:

1. Initialize a starting solution, C_{best} , as the candidate. This can be done by picking a candidate at random or choosing the best-performing model.
2. Set the value of the objective function for C_{best} , O , as O_{best} , and remove C_{best} from C .
3. Repeat this for N iterations:
 - i. Draw a random sample from C , add it to C_{best} , and store it as C_{stage} .
 - ii. Evaluate C_{stage} on the objective function, O , and store it as O_{stage} .
 - iii. If $O_{stage} > O_{best}$, then do the following:
 - a. $C_{best} = C_{best} \cup C_{stage}$
 - b. $O_{best} = O_{stage}$.
 - c. Remove C_{best} from C .

At the end of the run, we have C_{best} , which is the best combination of forecasts we got through stochastic hill climbing. We have made an implementation of this available in `src.forecasting.ensembling.py` under the `stochastic_hillclimbing` function. The parameters for this function are as follows:

- **objective:** This is a callable that takes in a list of strings as the candidates and returns a float objective value.
- **candidates:** This is a list of candidates to be included in the optimization.
- **n_iterations:** The number of iterations to run the hill-climb for. If this is not given, a heuristic (twice the number of candidates) is used to set this.
- **init:** This determines the strategy to be used for the initial solution. This can be random or best.
- **verbose:** A flag that specifies whether progress is printed or not.
- **random_state:** A seed that gets repeatable results.

The function returns a tuple of the best solution as a list of strings and the best score obtained through optimization.

This can be used in a very similar fashion to `greedy_optimization`. We will only show the different parts here. The full code is available in the notebook:

```
from src.forecasting.ensembling import stochastic_hillclimbing
# ensemble_forecasts is the list of candidates
solution, best_score = stochastic_hillclimbing(
    objective, ensemble_forecasts, n_iterations=10, init="best", random_state=9
)
```

Once we run this code, we will have the combination forecast called `stochastic_hillclimb_ensemble` in our prediction DataFrame. The candidates that are part of the optimal solution are LightGBM, Lasso Regression, `auto_stat`, `LightGBM_auto_stat`, and Lasso Regression. Now, let's evaluate the results and look at the aggregated metrics:

Algorithm	MAE	MSE	meanMASE	Forecast Bias
LightGBM	0.0751	0.0271	0.9142	2.57%
best_fit	0.0740	0.0269	0.8971	0.14%
median_ensemble	0.0767	0.0279	0.9304	-0.86%
average_ensemble	0.0828	0.0285	1.0159	1.58%
greedy_ensemble	0.0733	0.0251	0.8951	0.81%
stochastic_hillclimb_ensemble	0.0751	0.0257	0.9206	1.18%

Figure 9.6: Aggregate metrics for a stochastic hill climbing-based ensemble

The stochastic hill climb is not doing better than the greedy approach but is better than the mean, median, and best-fit ensembles. We discussed three disadvantages of simple hill climbing earlier—runtime considerations, short-sightedness, and forward-only. Stochastic hill climbing solves the runtime consideration because we are not evaluating all the combinations and selecting the best. Instead, we are randomly evaluating the combinations and adding them to the ensemble as soon as we see a solution that performs better. It partly solves the short-sightedness purely because the randomness in the algorithm may end up choosing a sub-optimal solution for each stage. But it still only chooses solutions that are better than the current solution.

Now, let's look at another modification of hill climbing that handles this issue as well.

Simulated annealing

Simulated annealing is a modification of hill climbing that is inspired by a physical phenomenon—annealing solids. Annealing is the process of heating a solid to a predetermined temperature (usually above its recrystallization point, but below its melting point), holding it for a while, and then cooling it (either slowly or quickly by quenching it in water).

This is done to ensure that the atoms assume a new global minimum energy state, which induces desirable properties in some metals, such as iron.

In 1952, Metropolis proposed simulated annealing as an optimization technique. The annealing analogy applies to the optimization context as well. When we say we heat the system, we mean that we encourage random perturbations. So, when we start an optimization with a high temperature, the algorithm explores the space and comes up with an initial structure of the problem. And as we reduce the temperature, the structure is refined to arrive at a final solution. This technique helps us avoid getting stuck in any local optima. Local optima are extrema in the objective function surface that are better than other values nearby but may not be the absolute best solution possible. The *Further reading* section contains a resource that explains what local and global optima are in concise language.

Now, let's look at the algorithm.

Here, C is a set of candidates (base forecasts), O is the objective we want to minimize, N is the maximum number of iterations we want to run the optimization for, T_{max} is the maximum temperature, and α is the temperature decay. The algorithm for simulated annealing is as follows:

1. Initialize a starting solution, C_{best} , as the candidate. This can be done by picking a candidate at random or choosing the best-performing model.
2. Set the value of the objective function for C_{best} , O , as O_{best} , and remove C_{best} from C .
3. Set the current temperature, t , to T_{max} .
4. Repeat this for N iterations:
 - i. Draw a random sample from C , add it to C_{best} , and store it as C_{stage} .
 - ii. Evaluate C_{stage} on the objective function, O , and store it as O_{stage} .
 - iii. If $O_{stage} > O_{best}$, then do the following:
 - a. $C_{best} = C_{best} \cup C_{stage}$
 - b. $O_{best} = O_{stage}$
 - c. Remove C_{best} from C .
 - iv. Otherwise, do the following:
 - a. Calculate the acceptance probability, $s = e^{-\frac{O_{best} - O_{stage}}{t}}$.
 - b. Draw a random sample between 0 and 1, as p .
 - c. If $p < s$, then do the following:
 - i. $C_{best} = C_{best} \cup C_{stage}$
 - ii. $O_{best} = O_{stage}$
 - iii. Remove C_{best} from C .
- v. $t = t - \alpha$ (for linear decay) and $t = t/\alpha$ (for geometric decay).
- vi. Exit when C is empty.

At the end of the run, we have C_{best} , which is the best combination of forecasts we got through simulated annealing. We have provided an implementation of this in `src.forecasting.ensembling.py` under the `simulated_annealing` function. Setting the temperature to the right value is key for the algorithm to work well and is typically the hardest hyperparameter to set. More intuitively, we can think of temperature in terms of the probability of accepting a worse solution in the beginning. In the implementation, we have also made it possible to input the starting and ending probability of accepting a worse solution.

In 1989, D.S. Johnson et al. proposed a procedure for estimating the temperature range from the given probability range. This has been implemented in `initialize_temperature_range`.

To summarize, the algorithm starts with random solutions and evaluates how good each is. Then, it keeps trying new solutions, sometimes accepting worse ones to avoid getting stuck in a local optima, but over time, it becomes less likely to accept bad solutions as it “cools down” (just like how metals cool and harden). It repeats this process until it either runs out of options or the temperature gets too low, leaving the best solution found so far.



Reference check:

The research paper by D.S. Johnson, titled *Optimization by Simulated Annealing: An Experimental Evaluation; Part I, Graph Partitioning*, is cited in the *References* section as reference 1.

The parameters for the `simulated_annealing` function are as follows:

- **objective:** This is a callable that takes in a list of strings as the candidates and returns a float objective value.
- **candidates:** This is a list of candidates to be included in the optimization.
- **n_iterations:** The number of iterations to run simulated annealing for. This is a mandatory parameter.
- **p_range:** The starting and ending probabilities as a tuple. This is the probability with which a worse solution is accepted in simulated annealing. The temperature range (`t_range`) is inferred from `p_range` during optimization.
- **t_range:** We can use this if we want to directly set the temperature range as a tuple (start, end). If this is set, `p_range` is ignored.
- **init:** This determines the strategy that’s used for the initial solution. This can be `random` or `best`.
- **temperature_decay:** This specifies how to decay the temperature. It can be `linear` or `geometric`.
- **verbose:** A flag that specifies whether progress is printed or not.
- **random_state:** The seed for getting repeatable results.

The function returns a tuple of the best solution as a list of strings and the best score that was obtained through optimization.

This can be used in a very similar fashion to the other ways of combining forecasts. We will show just the part that is different here. The full code is available in the notebook:

```

from src.forecasting.ensembling import simulated_annealing
# ensemble_forecasts is the list of candidates
solution, best_score = simulated_annealing(
    objective,
    ensemble_forecasts,
    p_range=(0.5, 0.0001),
    n_iterations=50,
    init="best",
    temperature_decay="geometric",
    random_state=42,
)

```

Once we run this code, we will have a combination forecast called `simulated_annealing_ensemble` in our prediction DataFrame. The candidates that are part of the optimal solution are LightGBM, Lasso Regression, `auto_stat`, LightGBM `auto_stat`, and XGB Random Forest. Let's evaluate the results and look at the aggregated metrics:

Algorithm	MAE	MSE	meanMASE	Forecast Bias
LightGBM	0.0751	0.0271	0.9142	2.57%
best_fit	0.0740	0.0269	0.8971	0.14%
median_ensemble	0.0767	0.0279	0.9304	-0.86%
average_ensemble	0.0828	0.0285	1.0159	1.58%
greedy_ensemble	0.0733	0.0251	0.8951	0.81%
stochastic_hillclimb_ensemble	0.0751	0.0257	0.9206	1.18%
simulated_annealing_ensemble	0.0735	0.0248	0.9041	0.26%

Figure 9.7: Aggregate metrics for a simulated annealing-based ensemble

Simulated annealing seems to be doing better than stochastic hill climbing. We discussed three disadvantages of simple hill climbing earlier—runtime considerations, short-sightedness, and forward-only. Simulated annealing solves the runtime consideration because we are not evaluating all the combinations and selecting the best. Instead, we are randomly evaluating the combinations and adding them to the ensemble as soon as we see a solution that performs better. It also solves the short-sightedness problem because, by using temperature, we are also accepting solutions that are slightly worse toward the beginning of the optimization. However, it is still a forward-only procedure.

So far, we have looked at combinatorial optimization because we said. But if we can relax this constraint and make $W_N \in [0, 1]$. $W_N \in \mathbb{R}$ (real numbers), the combinatorial optimization problem can be relaxed to a general mathematical optimization problem. Let's see how we can do that.

Optimal weighted ensemble

Previously, we defined the optimization problem we are trying to solve as follows:

$$\hat{Y} = \underset{w}{\operatorname{argmin}} L\left(\frac{1}{\sum_{i=1}^N w_i} \sum_{i=1}^N w_i \times \hat{Y}_i, Y\right)$$

Here, L is the loss or metric that we are trying to minimize. In our case, we chose that to be the MAE. $\hat{Y}_N$ is the set of N base forecasts while Y is the real observed values of the time series. Instead of defining $W_N \in [0, 1]$, let's make $W_N \in \mathbb{R}$, the continuous weights of each of the base forecasts. With this new relaxation, the combination becomes a weighted average between the different base forecasts. Now, we are looking at a soft mixing of the different forecasts as opposed to the hard-choice-based combinatorial optimization (which was what we had been using up until this point).

This is an optimization problem that can be solved using off-the-shelf algorithms from `scipy`. Let's see how we can use `scipy.optimize` to solve this problem.

First, we need to define a loss function that takes in a set of weights as a list and returns the metric we need to optimize:

```
def loss_function(weights):
    # Calculating the weighted average
    fc = np.sum(pred_wide[candidates].values * np.array(weights), axis=1)
    # Using any metric function to calculate the metric
    return metric_fn(pred_wide[target].values, fc)
```

Now, all we need to do is call `scipy.optimize` with the necessary parameters. Let's learn how to do this:

```
from scipy import optimize
opt_weights = optimize.minimize(
    loss_function,
    # set x0 as initial values, which is a uniform distribution over all
    # the candidates
    x0=[1 / len(candidates)] * len(candidates),
    # Set the constraint so that the weights sum to one
    constraints=({"type": "eq", "fun": lambda w: 1 - sum(w)}),
    # Choose the optimization technique. Should be gradient-free and
    # bounded.
    method="SLSQP",
    # Set the lower and upper bound as a tuple for each element in the
    # candidate list.
    # We set the maximum values between 1 and 0
    bounds=[(0.0, 1.0)] * len(candidates),
    # Set the tolerance for termination
    options={"ftol": 1e-10},
)["x"]
```

The optimization is usually fast and we will get the weights as a list of floating-point numbers. We have wrapped this in a function in `src.forecasting.ensembling.py` called under the `find_optimal_combination` function. The parameters for this function are as follows:

- `candidates`: This is a list of candidates to be included in the optimization. They are returned in the same order in which the returned weights would.
- `pred_wide`: This is the prediction DataFrame on which we need to learn the weights.
- `target`: This is the column name of the target.
- `metric_fn`: This is any callable with a `metric(actuals, pred)` signature.

The function returns the optimal weights as a list of floating-point numbers. Let's see what the optimal weights are when we learned them through our validation forecast:

Forecast	Weights
LightGBM	0.4221
LightGBM_auto_stat	0.2991
Lasso Regression_auto_stat	0.1266
Lasso Regression	0.1012
XGB Random Forest	0.0510
FFT	0.0000
Theta	0.0000
XGB Random Forest_auto_stat	0.0000

Figure 9.8: The optimal weights that were learned through optimization

Here, we can see that the optimization automatically learned to ignore FFT, Theta, XGB Random Forest, and XGB Random Forest_auto_stat because they didn't add much value to the ensemble. It has also learned some non-zero weights for each of the forecasts. The weights already resemble the selection we made using the techniques we discussed previously. Now, we can use these weights to come up with a weighted average and call it `optimal_combination_ensemble`.

The aggregated results should be as follows:

Algorithm	MAE	MSE	meanMASE	Forecast Bias
LightGBM	0.0751	0.0271	0.9142	2.57%
best_fit	0.0740	0.0269	0.8971	0.14%
median_ensemble	0.0767	0.0279	0.9304	-0.86%
average_ensemble	0.0828	0.0285	1.0159	1.58%
greedy_ensemble	0.0733	0.0251	0.8951	0.81%
stochastic_hillclimb_ensemble	0.0751	0.0257	0.9206	1.18%
simulated_annealing_ensemble	0.0735	0.0248	0.9041	0.26%
optimal_combination_ensemble	0.0732	0.0248	0.8956	0.81%

Figure 9.9: Aggregate metrics for the optimal combination-based ensemble

Here, we can see that this soft mixing of the forecasts is doing much better than all of the hard-choice-based ensembles on all three metrics.



In all the techniques we discussed, we were using the MAE as the objective function. But we can use any metric, a combination of metrics, or even metrics with regularization as the objective function. When we discussed Random Forest, we talked about how decorrelated trees were essential to getting better performance. A very similar principle applies while choosing ensembles as well. Having decorrelated base forecasts adds value to the ensemble. So, we can use any measure of variety to regularize our metric as well. For instance, we can use correlation as a measure and create a regularized metric to be used in these techniques. The `01-Forecast_Combinations.ipynb` notebook in the `Chapter09` folder contains a bonus section that shows how to do that.

We started by discussing combining forecasts with a mathematical formulation:

$$Y = F(Y_1, Y_2, \dots, Y_N)$$

Here, F is the function that combines the N forecasts.

We did all this while looking at ways to come up with this function as an optimization problem, using something such as a mean or median to combine the metrics. But we have also seen another way to learn this function, F , from data, haven't we? Let's see how that can be done.

Stacking and blending

We started this chapter by talking about machine learning algorithms, which learn a function from a set of inputs and outputs. While using those machine learning algorithms, we learned about the functions that forecast our time series, which we'll call base forecasts now.

Why not use the same machine learning paradigm to learn this new function, F , that we are trying to learn as well?

This is exactly what we do in stacking (often called stacked generalization), where we train another learning algorithm on the predictions of some base learners to combine these predictions. This second-level model is often called a **stacked model** or a **meta model**. And typically, this meta model performs equal to or better than the base learners. This is very similar to blending where the only difference being the way we split the data.

Although the idea originated with Wolpert in 1992, Leo Breiman formalized this idea in the way it is used now in his 1996 paper titled *Stacked Regressions*. And in 2007, M. J. Van der Laan et al. established the theoretical underpinnings of the technique and provided proof that this meta model will perform at least as well as or even better than the base learners.

**Reference check:**

The research papers by Leo Breiman (1996) and Mark J. Van der Laan et al. (2007) are cited in the *References* section as 2 and 3, respectively.

This is a very popular technique in machine learning competitions such as Kaggle and is considered a secret art among machine learning practitioners. We also discussed some other techniques, such as bagging and boosting, which combine base learners into something more. But those techniques require the base learner to be a weak learner. This is where stacking differs because stacking tries to combine a *diverse* set of *strong* learners.

The intuition behind stacking is that different models or families of functions learn the output function slightly differently, capturing different properties of the problem. For instance, one model may have captured the seasonality very well, whereas the other may have captured any particular interaction with an exogenous variable better. The stacking model will be able to combine these base models into a model that learns to look toward one model for seasonality and the other for interaction. This is done by making the meta model learn the predictions of the base models. But to prevent data leakage and thereby avoid overfitting, the meta model should be trained on out-of-sample predictions. There are two small variations of this technique that are used today—stacking and blending.

Stacking is when the meta model is trained on the entire training dataset, but with out-of-sample predictions. The following steps are involved in stacking:

1. Split the training dataset into k parts.
2. Iteratively train the base models on $k-1$ parts, predict on the k^{th} part, and save the predictions. Once this step is done, we have the out-of-sample predictions for the training dataset from all base models.
3. Train a meta model on these predictions.

Blending is similar to this but slightly different in the way we generate out-of-sample predictions. The following steps are involved in blending:

1. Split the training dataset into two parts—train and holdout.
2. Train the base models on the training dataset and predict on the holdout dataset.
3. Train a meta model on the validation dataset with the predictions of the base model as the features.

Intuitively, we can see that stacking can work better because it uses a much larger dataset (usually all the training data) as the out-of-sample prediction, so the meta model may be more generalized. But there is a caveat: we assume that the entire training data is **independent and identically distributed (iid)**. This is typically an assumption that is hard to meet in time series since the data-generating process can change at any time (either gradually or drastically). If we know that the data distribution has changed significantly over time, blending the holdout period (which is usually the most recent part of the dataset) is better because the meta model is only learning on the latest data, thus paying respect to the temporal changes in the distribution of data.

There is no limit to the number of models we can include as base models, but usually, there is a plateau that we reach where additional models do not add much to the stacked ensemble. We can also add multiple levels of stacking. For instance, let's assume there are four base learners: B_1 , B_2 , B_3 and B_4 . We have also trained two meta models M_1 and M_2 , on the base models. Now, we can train a second-level meta model, M , on the outputs of M_1 and M_2 and use that as the final prediction. We can use the `pystacknet` Python library (<https://github.com/h2oai/pystacknet>), which is the Python implementation of an older library, called `stacknet`, to make the process of creating multi-level (or single-level) stacked ensembles easy.

Another key point to keep in mind is the type of models we usually use as meta models. It is assumed that the bulk of the learning has been taken care of by the base models, which are the multi-dimensional data for patterns for prediction. Therefore, the meta models are usually simple models such as linear regression, a decision tree, or even a random forest with much lower depth than the base models. Another way to think about this is in terms of bias and variance. Stacking can overfit the training or holdout set and by including model families with larger flexibility or expressive power, we are enabling this overfitting. The *Further reading* section contains a few links that explain different techniques of stacking from a general machine learning perspective.

Now, let's quickly see how we can use this in our dataset:

```
from sklearn.linear_model import LinearRegression
stacking_model = LinearRegression()
# ensemble_forecasts is the list of candidates
stacking_model.fit(
    pred_wide_val[ensemble_forecasts], pred_wide_val["energy_consumption"]
)
pred_wide_test["linear_reg_blending"] = stacking_model.predict(
    pred_wide_test[ensemble_forecasts]
)
```

This would save the blended prediction for linear regression as `linear_reg_blending`. We can use the same code but swap the models to try out other models as well.



Best practice:

When there are many base models and we want to do implicit base model selection as well, we can opt for one of the regularized linear models, such as Ridge or Lasso regression. Breiman, in his original paper, *stacked regressions*, proposed to use linear regression with positive coefficients and no intercept as the meta model. He argued that this gives a theoretical guarantee that the stacked model will be at least as good as any best individual model. But in practice, we can relax those assumptions while experimenting. Non-negative regression without intercepts is very close to the optimal weighted ensemble we discussed earlier. Finally, if we are evaluating multiple stacked models to select which one works well, we should resort to either having a separate validation dataset (instead of a *train-validation-test* split, we can use a *train-validation-validation_meta-test* split) or using cross-validated estimates. If we just pick the stacked model that performs best on the test dataset, we are overfitting the test dataset.

Now, let's see how the blended models are doing on our test data:

Algorithm	MAE	MSE	meanMASE	Forecast Bias
LightGBM	0.0751	0.0271	0.9142	2.57%
best_fit	0.0740	0.0269	0.8971	0.14%
median_ensemble	0.0767	0.0279	0.9304	-0.86%
average_ensemble	0.0828	0.0285	1.0159	1.58%
greedy_ensemble	0.0733	0.0251	0.8951	0.81%
stochastic_hillclimb_ensemble	0.0751	0.0257	0.9206	1.18%
simulated_annealing_ensemble	0.0735	0.0248	0.9041	0.26%
optimal_combination_ensemble	0.0732	0.0248	0.8956	0.81%
linear_reg_blending	0.0755	0.0245	0.9260	4.35%
ridge_reg_blending	0.0737	0.0243	0.9082	1.84%
lasso_reg_blending	0.0736	0.0243	0.9068	1.94%
huber_reg_blending	0.0704	0.0246	0.8989	-6.42%

Figure 9.10: Aggregate metrics for blending models

Here, we can see that a simple linear regression has learned a meta model that performs much better than any of our average ensemble methods. And the Huber regression (which is a way to optimize the MAE directly) performs much better on the MAE benchmark. However, keep in mind that this is not universal and has to be evaluated for each problem you come across. Choosing the metric to optimize for and the model to use to combine makes a lot of difference. And often, the simple average ensemble is a very formidable benchmark for combining models.



Huber regression is another version of linear regression (like Ridge and Lasso) where the loss function is a combination of squared loss (used in regular linear regression) and absolute loss (used in L1 methods). It behaves like squared loss for small residuals and like absolute loss for large residuals. This makes it less sensitive to outliers. Scikit-Learn has `HuberRegressor` (https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.HuberRegressor.html), which implements this.



Additional reading:

There are other more innovative ways to combine base forecasts. This is an active area of research. The *Further reading* section contains links to two such ideas that are very similar. **Feature-Based Forecast Model Averaging (FFORMA)** extracts a set of statistical features from the time series and uses it to train a machine learning model that predicts the weights in which the base forecast should be combined. Another technique (**self-supervised learning for fast and scalable time-series hyper-parameter tuning**), from Facebook (Meta) Research, trains a classifier to predict which of the base learners does best, given a set of statistical features extracted from the time series.

Summary

Continuing with the streak of practical lessons in the previous chapter, we completed yet another hands-on lesson. In this chapter, we generated forecasts from different machine learning models from the previous chapter. We learned how to combine these different forecasts into a single forecast that performs better than any single model. Then, we explored concepts such as combinatorial optimization and stacking/blending to achieve state-of-the-art results.

In the next chapter, we will start talking about global models of forecasting and explore strategies, feature engineering, and so on to enable such modeling.

References

The following references were provided in this chapter:

1. David S. Johnson, Cecilia R. Aragon, Lyle A. McGeoch, and Catherine Schevon (1989), *Optimization by Simulated Annealing: An Experimental Evaluation; Part I, Graph Partitioning*. Operations Research, 1989, vol. 37, issue 6, 865-892: <http://dx.doi.org/10.1287/opre.37.6.865>

2. L. Breiman (1996), *Stacked regressions*. Mach Learn 24, 49–64: <https://doi.org/10.1007/BF00117832>
3. Mark J. van der Laan; Eric C. Polley; and Alan E. Hubbard (2007), *Super Learner*. U.C. Berkeley Division of Biostatistics Working Paper Series. Working Paper 222: <https://biostats.bepress.com/ucbbiostat/paper222>

Further reading

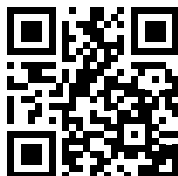
To learn more about the topics that were covered in this chapter, take a look at the following resources:

- *A Kaggle's Guide to Model Stacking in Practice*, by Ha Nguyen: <https://datasciblog.github.io/2016/12/27/a-kagglers-guide-to-model-stacking-in-practice/>
- Kai Ming Ting and Ian H. Witten (1997), *Stacked Generalization: when does it work?*: <https://www.ijcai.org/Proceedings/97-2/Papers/011.pdf>
- Pablo Montero-Manso, George Athanasopoulos, Rob J. Hyndman, Thiyanga S. Talagala (2020), *FFORMA: Feature-based forecast model averaging*. International Journal of Forecasting, Volume 36, Issue 1: <https://robjhyndman.com/papers/fforma.pdf>
- Peiyi Zhang, et al. (2021), *Self-supervised learning for fast and scalable time-series hyper-parameter tuning*: <https://www.ijcai.org/Proceedings/97-2/Papers/011.pdf>
- Local versus Global Optima: <https://www.mathworks.com/help/optim/ug/local-vs-global-optima.html>

Join our community on Discord

Join our community's Discord space for discussions with authors and other readers:

<https://packt.link/mts>



10

Global Forecasting Models

In previous chapters, we saw how we can use modern machine learning models on time series forecasting problems, essentially replacing traditional models such as ARIMA or exponential smoothing. However, before now, we were looking at the different time series in any dataset (such as households in the London Smart Meters dataset) in isolation, just as the traditional models did.

However, we will now explore a different paradigm of modeling where we use a single machine learning model to forecast a bunch of time series together. As we will learn in the chapter, this paradigm brings many benefits with it, from the perspective of both computation and accuracy.

In this chapter, we will be covering these main topics:

- Why Global Forecasting Models?
- Creating GFMs
- Strategies to improve GFMs
- Interpretability

Technical requirements

You will need to set up the **Anaconda** environment following the instructions in the *Preface* of the book to get a working environment with all the libraries and datasets required for the code in this book. Any additional library will be installed while running the notebooks.

You will need to run the following notebooks before using the code in this chapter:

- `02-Preprocessing_London_Smart_Meter_Dataset.ipynb` in Chapter02
- `01-Setting_up_Experiment_Harness.ipynb` in Chapter04
- From the Chapter06 and Chapter07 folders:
 - `01-Feature_Engineering.ipynb`
 - `02-Dealing_with_Non-Stationarity.ipynb`
 - `02a-Dealing_with_Non-Stationarity-Train+Val.ipynb`

- From the Chapter08 folder:
 - 00-Single_Step_Backtesting_Baselines.ipynb
 - 01-Forecasting_with_ML.ipynb
 - 01a-Forecasting_with_ML_for_Test_Dataset.ipynb
 - 02-Forecasting_with_Target_Transformation.ipynb
 - 02a-Forecasting_with_Target_Transformation(Test).ipynb

The associated code for the chapter can be found at <https://github.com/PacktPublishing/Modern-Time-Series-Forecasting-with-Python-/tree/main/notebooks/Chapter10>.

Why Global Forecasting Models?

We talked about global models briefly in *Chapter 5, Time Series Forecasting as Regression*, where we mentioned related datasets. We can think of many scenarios where we would encounter related time series. We may need to forecast the sales for all the products of a retailer, the number of rides requested for a cab service across different areas of a city, or the energy consumption of all the households in a particular area (which is what the London Smart Meters dataset does). We call these related time series because all the different time series in the dataset can have a lot of factors in common with each other. For instance, the yearly seasonality that might occur in retail products might be present for a large section of products, or the way an external factor such as temperature affects energy consumption may be similar for a large number of households. Therefore, one way or the other, the different time series in a related time series dataset share attributes between them.

Traditionally, we used to consider each time series an independent time series; in other words, each time series was assumed to be generated using a different data-generating process. Classical models such as ARIMA and exponential smoothing are trained for each time series. However, we can also consider all the time series in the dataset as being generated from a single data-generating process, and the subsequent modeling approach would be to train a single model to forecast all the time series in the dataset. The latter is what we refer to as **Global Forecasting Models (GFMs)**. GFMs are models that are designed to handle multiple related time series, allowing for shared learning across those time series. In contrast, the traditional approach is referred to as **Local Forecasting Models (LFMs)**.

Although we briefly talked about the drawbacks of LFMs in *Chapter 5, Time Series Forecasting as Regression*, let's summarize them in a more concrete fashion and see why GFMs help us smooth over a lot of those drawbacks.

Sample size

In most real-world applications (especially in business forecasting), the time series we have to forecast is not very long. Adopting a completely data-driven approach to modeling such a small time series is problematic. Training a highly flexible model with a handful of data points will lead to the model memorizing the training data, resulting in an overfit.

Traditionally, this has been overcome by placing strong priors or inductive bias into the models we use for forecasting. Inductive bias loosely refers to a set of assumptions or restrictions that are built into a model that should help the model predict feature combinations it has not encountered while training. For instance, double exponential smoothing has strong assumptions about seasonality and trend. The model does not allow any other more complicated patterns to be learned from the data. Therefore, using these strong assumptions, we are restricting the model search to a small section of the hypothesis space. While this helps in low-data regimes, the flip side is that these assumptions may limit accuracy.

Recent developments in the field of machine learning have shown us without a doubt that using a data-driven approach (with much fewer assumptions or priors) on large training sets will lead to us training better models. However, conventional statistical wisdom tells us that the number of data points needs to be at least 10 to 100 times the number of parameters that we are trying to learn from those data points.

So, if we stick to LFMs, scenarios in which we can adopt a completely data-driven approach will be very rare. This is where GFM shine. A GFM is able to use the history of *all* the time series in a dataset to train the model and learn a single set of parameters that work for all the time series in the dataset. Borrowing the terminology introduced in *Chapter 5, Time Series Forecasting as Regression*, we increase the *width* of the dataset, keeping the *length* the same (refer back to *Figure 5.2*). This explosion of historical information available to a single model lets us use completely data-driven techniques on time series datasets.

Cross-learning

GFMs, by design, promote cross-learning across different time series in a dataset. Imagine we have a time series that is quite new and does not have a history rich enough for teaching the model—for instance, the sales of a newly introduced retail product or the electricity consumption of a new household in a region. If we consider these time series in isolation, it will be a while before we start to get reasonable forecasts from the models we train on them, but GFMs make that process easier by enabling cross-learning. GFMs have an implicit sense of similarity between different time series and they will be able to use patterns they have seen in similar time series with a rich history to come up with a forecast on the new time series.

Another way cross-learning helps is by acting like a regularizer while estimating common parameters such as seasonality. For instance, the seasonality exhibited by similar products in a retail scenario is best estimated at an aggregate level, because each individual time series will have some sort of noise that can creep into the seasonality extraction. By enforcing common seasonality across multiple products, we are essentially regularizing the seasonality estimation and, in the process, making the seasonality estimate more robust. The good thing about GFMs is that they take a data-driven approach to define the seasonality of which products should be estimated together and which ones have different patterns. If you have different seasonality patterns in different products, a GFM may struggle to model them together. However, when provided with enough information on how to distinguish between different products, the GFM will be able to learn that difference too.

Multi-task learning

GFM can be considered multi-task learning paradigms where a single model is trained to learn multiple tasks (as forecasting each time series is a separate task). Multi-task learning is an active area of research, and there are many benefits to using multi-task models:

- When the model is learning from noisy, high-dimensional data, it becomes harder for the model to distinguish between useful and non-useful features. When we train the model on a multi-task paradigm, the model can understand useful features by looking at features that are useful for other tasks as well, thus providing the model with an additional perspective for discerning useful features.
- Sometimes, features such as seasonality might be hard to learn from a particularly noisy time series. However, under a multi-task framework, the model can learn the difficult features using other time series in the dataset.
- Finally, multi-task learning introduces a kind of regularization that forces the model to find a model that works well on all tasks, thus reducing the risk of overfitting.

Engineering complexity

LFMs pose a challenge from the engineering side as well for large datasets. If we have thousands or millions of time series to forecast, it becomes increasingly difficult to both train and manage the life cycle of these LFMs. In *Chapter 8, Forecasting Time Series with Machine Learning Models*, we trained LFMs for just a subset of households in the dataset. It took almost 20 to 30 minutes to train a machine learning model for all 150 households and we ran them with the default hyperparameters. In a normal machine learning workflow, we train multiple machine learning models and do hyperparameter tuning to find the best configuration of the model. However, carrying out all these steps for thousands of time series in a dataset becomes increasingly complex and time-consuming.

Equally, then there is the issue of managing the life cycle of these models. All these individual models need to be deployed to production, their performance needs to be monitored to check for model and data drift, and they need to be retrained at a set frequency. This becomes increasingly complex as we have more and more time series to forecast.

However, by shifting to a GFM paradigm, we drastically reduce the time and effort required to train and manage a machine learning model throughout its life cycle. As we will see in this chapter, training a GFM on these 150 households takes only a fraction of the time it takes to train LFMs.

Despite all the advantages of GFMs, they are not without some drawbacks. The main drawback is that we are assuming that all the time series in a dataset are generated by a single **Data Generating Process (DGP)**. This might not be a valid assumption and this can lead to the GFM underfitting some specific types of time series patterns that are underrepresented in the dataset.

Another open issue is whether a GFM is good for use with unrelated tasks or time series. The jury is out on this one, but Montero-Manso et al. proved that there are also gains in modeling unrelated time series with a GFM. The same finding has been put forward, although from another perspective, by Oreshkin et al., who trained a global model on the M4 dataset (a set of unrelated datasets) and obtained state-of-the-art performance. They attributed it to the meta-learning capabilities of the model.

That being said, relatedness does help the GFM, as the learning task becomes easier this way. We will see a practical application of this in upcoming sections of this chapter as well.

In the larger scheme of things, the benefits we derive from a GFM paradigm far outweigh the drawbacks. On most tasks, the GFMs either perform on par with or better than local models. It has been proven theoretically as well, by Montero-Manso et al., that a GFM, in a worst-case scenario, learns the same function as a local model. We will see this clearly in the models we are going to train in the upcoming sections. Finally, the training time and engineering complexity drop drastically as you move to a GFM paradigm.

Now that we have explained why a GFM is a worthwhile paradigm to adopt, let's see how we can train one.

Creating GFMs

Training a GFM is very straightforward. While we were training LFMs in *Chapter 8, Forecasting Time Series with Machine Learning Models*, we were looping over different households in the London Smart Meters dataset and training a model for each household. However, if we just take all the households into a single dataframe (our dataset is already that way) and train a single model on it, we get a GFM. One thing we want to keep in mind is to make sure that all the time series in the dataset have the same frequency. In other words, if we mix daily time series with weekly ones while training these models, the performance drop will be noticeable—especially if we are using time-varying features and other time-based information. For a purely autoregressive model, mixing time series in this way is much less of a problem.



Notebook alert:

To follow along with the complete code, use the notebook named `01-Global_Forecasting_Models-ML.ipynb` in the `Chapter10` folder.

The standard framework we developed in *Chapter 8, Forecasting Time Series with Machine Learning Models*, is general enough to work for GFMs as well. So, as we did in that chapter, we define `FeatureConfig` and `MissingValueConfig` in the `01-Global_Forecasting_Models-ML.ipynb` notebook. We also slightly tweaked the Python function to train and evaluate the machine learning to make it work for all households. The details and exact functions can be found in the notebook.

Now, instead of looping over different households, we input the entire training dataset into the `get_X_y` function:

```
# Define the ModelConfig
from lightgbm import LGBMRegressor
model_config = ModelConfig(
    model=LGBMRegressor(random_state=42),
    name="Global LightGBM Baseline",
    # LGBM is not sensitive to normalized data
```

```

        normalize=False,
        # LGBM can handle missing values
        fill_missing=False,
    )
    # Get train and test data
    train_features, train_target, train_original_target = feat_config.get_X_y(
        train_df, categorical=True, exogenous=False
    )
    test_features, test_target, test_original_target = feat_config.get_X_y(
        test_df, categorical=True, exogenous=False
    )

```

Now that we have the data, we need to train the model. Training the model is also exactly the same as we saw in *Chapter 8, Forecasting Time Series with Machine Learning Models*. We will just choose LightGBM, which was the best-performing LFM model, and use functions we defined earlier to train the model and evaluate the results:

```

y_pred, feat_df = train_model(
    model_config,
    _feat_config,
    missing_value_config,
    train_features,
    train_target,
    test_features,
)
agg_metrics, eval_metrics_df = evaluate_forecast(
    y_pred, test_target, train_target, model_config
)

```

Now, in `y_pred`, we will have the forecast for all the households and `feat_df` will have the feature importance. `agg_metrics` will have the aggregated metric for all the selected households.

Let's look at how well our GFM model did:

Algorithm	MAE	MSE	meanMASE	Forecast Bias	Time Elapsed
LightGBM	0.077183	0.027510	0.978056	0.050231	NaN
GFM Baseline	0.079581	0.027326	1.013393	0.218127	28.718087

Figure 10.1: Aggregate metrics with the baseline GFM

We are not doing better than the best LFM (in the first row) in terms of the metrics. However, one thing we should note is the time taken to train the model—~30 seconds. The LFM for all the selected households was taking ~30 minutes. This huge reduction in time taken gives us a lot of flexibility to iterate faster with different features and techniques.

With that said, let's now look at a few techniques with which we can improve the accuracy of the GFM.

Strategies to improve GFMs

GFMs have been in use in many forecasting competitions in Kaggle and outside of it. They have been battle-tested empirically, although very little work has gone into examining why they work so well from a theoretical point of view. Montero-Manso and Hyndman (2020) have a working paper titled *Principles and Algorithms for Forecasting Groups of Time Series: Locality and Globality*, which is an in-depth investigation, both theoretical and empirical, of GFMs and the many techniques that have been developed by the data science community collectively. In this section, we will try to include strategies to improve GFMs and, wherever possible, try to give theoretical justifications for why they would work.



Reference check:

The Montero-Manso and Hyndman (2020) research paper is cited in *References* as reference 1.

In the paper, Montero-Manso and Hyndman use a basic result in machine learning about generalization error to carry out the theoretical analysis, and it is worth spending a bit of time understanding that, at least at a high level. Generalization error, as we know, is the difference between out-of-sample error and in-sample error. Yaser S Abu-Mostafa has a free, online **Massive Open Online Course (MOOC)** and an associated book (both of which are linked in the *Further reading* section). It is a short course on machine learning and is a course that I would recommend to anyone in the machine learning field for developing a stronger theoretical and conceptual basis for what we do. One of the important concepts the course and book put forward is the use of Hoeffding's inequality from probability theory to derive bounds on a learning problem. Let's quickly look at the result to develop our understanding:

$$E_{out} < E_{in} + \sqrt{\frac{\log(|H|) + \log\left(\frac{2}{\delta}\right)}{2N}}$$

It has a probability of at least $1-\delta$.

E_{in} is the in-sample average error and E_{out} is the expected out-of-sample error. N is the total number of samples in the dataset from which we are learning and H is the hypothesis class of models. It is a finite set of functions that can potentially fit the data. The size of H , denoted by $|H|$, is the complexity of H . Although the formula of the bound looks intimidating, let's simplify the way we look at it to develop the necessary understanding.

We want E_{out} to be as close to E_{in} as possible, and for that, we need the terms in the square root to be as small as possible. There are two terms under the square root that are in our *control*, so to speak— N and $|H|$. Therefore, to make the generalization error ($E_{in} - E_{out}$) as small as possible, we either need to increase N (have more data) or decrease $|H|$ (have a less complex model). This is a result that is applicable to all machine learning but Montero-Manso and Hyndman, with a few assumptions, made this applicable to time series models as well. It is this result that they used to give theoretical backing to the arguments put forward in their working paper.

Montero-Manso and Hyndman have taken Hoeffding's inequality and applied it to LFMs and GFMs to compare them. We can see the result here (for a full mathematical and statistical understanding, refer to the original paper under *References*):

$$E_{out}^{Local} < E_{in}^{Local} + \sqrt{\frac{\log(\prod_{i=1}^K |H_i|) + \log\left(\frac{2}{\delta}\right)}{2NK}}$$

$$E_{out}^{Global} < E_{in}^{Global} + \sqrt{\frac{\log(|J|) + \log\left(\frac{2}{\delta}\right)}{2NK}}$$

E_{out}^{Local} and E_{out}^{Global} are the average in-sample errors across all the time series using the local and global approaches, respectively. E_{out}^{Local} and E_{in}^{Global} are the out-of-sample expectations under the local and global approaches, respectively. H_i is the hypothesis class for the i -th time series and J is the hypothesis class for the global approach (the global approach only fits a single function and hence, has just a single hypothesis class).

One of the most interesting results that comes out of this is that the complexity term for LFMs ($\log(\prod_{i=1}^K |H_i|)$) grows the size of the dataset. The greater the number of time series we have in the dataset, the more complexity and the worse the generalization error, whereas with GFMs, the complexity term ($\log(|J|)$) stays constant. Therefore, for a dataset of moderate size, the overall complexity of LFMs (such as exponential smoothing) can be much higher than a single GFM, no matter how complex the GFM is. As a corollary, we can also think that with the available dataset (NK), we can afford to train a model with much higher complexity than a model for LFMs. There are many ways to increase the complexity of the model, which we will see in the following section.

Now, let's return to the GFMs we were training. We saw that the performance of the GFM we trained was not up to the mark when we compared it with the best LFM (LightGBM), but it is better than the baseline and other models we tried, so right off the bat, we know the GFM we trained is not terrible. Now, let's look at a few ways to improve the performance of the model.

Increasing memory

As we discussed in *Chapter 5, Time Series Forecasting as Regression*, the machine learning models that we discuss in this book are finite memory models or Markov models. A model such as exponential smoothing takes into account the entire history of a time series while forecasting, but models such as any of the machine learning models we discussed only take in finite memory to make their predictions. In a finite memory model, the amount of memory we allow the model to access is called the size of the memory (M) or order of autoregression (from econometrics).

Providing a greater amount of memory to the model increases the complexity of the model. Therefore, one of the ways to increase the performance of the GFM is to increase the amount of memory the model has access to. There are many ways to increase the amount of memory.

Adding more lag features

If you have prior exposure to ARIMA models, you will know that the number of **Autoregressive (AR)** terms are sparingly used. We usually see AR models with single-digit lags. There is nothing stopping us from running ARIMA models with larger lags, but since we do run ARIMA in the LFM paradigm, the model has to learn the parameters of all the lags using limited data and therefore, in practice, practitioners commonly choose smaller lags. However, when we are moving to GFMs, we can afford to have much larger lags. Montero-Manso and Hyndman empirically showed the benefits of adding more lags to GFMs. For highly seasonal time series, a peculiar phenomenon was observed. The accuracy improves with an increase in lags, but it then saturates and suddenly worsens when the lag becomes equal to the seasonal cycle. On further increasing the lags beyond the seasonal cycle, the accuracy shows huge gains. This may be because of the overfitting that happens because of seasonality. It becomes very easy for the model to favor the seasonal lag because it works very well in a sample, so it's better to add a few more lags on the plus side of the seasonal cycle.

Adding rolling features

Another way to increase the memory of the model is to include rolling averages as features. Rolling averages take information from extended windows on memory and encode that information by way of descriptive statistics (such as the mean or max). This is an efficient way of including the memory because we can take very large windows for memory and include the information as a single feature in the model.

Adding EWMA features

An **Exponentially Weighted Moving Average (EWMA)** is a way to include infinite memory in a finite memory model. The EWMA essentially takes the average of the entire history but is weighted according to the α that we set. Therefore, with different values of α , we get different kinds of memory, again encoded as a single feature. Including different EWMA features has also empirically proved beneficial.

We have already included these kinds of features in our feature engineering (*Chapter 6, Feature Engineering for Time Series Forecasting*), and they are part of the baseline GFM we trained, so let's move on to the next strategy for improving the accuracy of GFMs.

Using time series meta-features

The baseline GFM we trained earlier in the *Creating Global Forecasting Models (GFMs)* section had lag features, rolling features, and EWMA features, but we have given no feature that helps the model distinguish between different time series in the dataset. The baseline GFM model learned a generalized function that generates a forecast provided the features. This might work well enough for homogeneous datasets where all the time series are very similar in nature, but for heterogeneous datasets, the information with which the model can distinguish each time series comes in handy.

So, information about the time series itself is what we call meta-features. In a retail context, it can be the product ID, the category of products, the store number, and so on. In our dataset, we have features such as `stdorToU`, `Acorn`, `Acorn_grouped`, and `LCLid`, which give some information about the time series itself. Including these meta-features in the GFM will improve the performance of the model.

However, there is just one problem—more often than not, these meta-features are categorical in nature. A feature is categorical when the values in the feature can only take discrete values. For instance, `Acorn_grouped` can only have one of three values—`Affluent`, `Comfortable`, or `Adversity`. Most machine learning models do not work well with categorical features. All the models in `scikit-learn`, the most popular machine learning library in the Python ecosystem, do not allow categorical features at all. To include categorical features in machine learning models, we need to encode them into numerical form, and there are many ways to encode categorical columns. Let's review a few popular options.

Ordinal encoding and one-hot encoding

The most popular ways of encoding categorical features are ordinal encoding and one-hot encoding, but they are not always the best choices. Let's quickly review what these techniques are and when they are suitable.

Ordinal encoding is the simplest of them all. We simply assign a numerical code to the unique values of a category and then replace the categorical value with the numerical code. To encode the `Acorn_grouped` feature from our dataset, all we need to do is assign codes, say 1 for `Affluent`, 2 for `Comfortable`, and 3 for `Adversity`, and replace all instances of the categorical values with the code we assigned. While this is really easy, this kind of encoding introduces meanings to the categorical values that we may or may not intend. When we assign numerical codes, we are implicitly saying that the categorical value that gets assigned 2 as a code is better than the categorical value with 1 as the code. This kind of encoding only works for ordinal features (features whose categorical values have an intrinsic sense of rank in their meaning) and should be sparingly used. Another way we can think about the problem is in terms of distance. When we do ordinal encoding, the distance between `Comfortable` and `Affluent` can be higher than the distance between `Comfortable` and `Adversity`, depending on the way we encode.

One-hot encoding is a better way of representing categorical features with no ordinal meaning. It essentially encodes the categorical features in a higher dimension, placing the categorical values equally distant in that space. The size of the dimension it requires to encode the categorical values is equal to the cardinality of the categorical variable. Cardinality is the number of unique values in the categorical feature. Let's see how sample data would be encoded in a one-hot encoding scheme:

Acorn_grouped		Comfortable	Adversity	Affluent
Comfortable	 One-Hot Encoding	1	0	0
Comfortable		1	0	0
Adversity		0	1	0
Affluent		0	0	1
Affluent		0	0	1

Figure 10.2: One-hot encoding of categorical features

We can see that the resulting encoding will have a column for each unique value in the categorical feature and the value is indicated by 1 in the column. For instance, the first row is *Comfortable*, and therefore, every other column except the *Comfortable* column will have 0 and the *Comfortable* column will have 1. If we calculate the Euclidean distance between any two categorical values, we can see that they are the same.

However, there are three main issues with this encoding, all of which become a problem with high cardinality categorical variables:

- The embedding is inherently sparse and many machine learning models (for instance, tree-based models and neural networks) do not really work well with sparse data (sparse data is when a majority of values in the data are zeros). When the cardinality is just 5 or 10, the sparsity introduced may not be that much of a problem, but when we consider a cardinality of 100 or 500, the encoding becomes really sparse.
- Another issue is the explosion of dimensions of the problem. When we increase the total number of features of a problem due to the large number of new features that are created through one-hot encoding, we make the problem harder to solve. This can be explained by the curse of dimensionality. The *Further reading* section has a link with more information on the curse of dimensionality.
- The last problem is related to practical concerns. For a large dataset, if we one-hot encode a categorical value with hundreds or thousands of unique values, the resulting dataframe is not going to be easy to work with because it will not fit in the computer memory.

There is a slightly different way of one-hot encoding where we drop one of these dimensions, called **dummy variable encoding**. This has the added benefit of making the encoding linearly independent, which, in turn, has some advantages, especially for vanilla linear regression. The *Further reading* section has a link if you want to know more.

Since the categorical columns that we must encode have high cardinality (at least a few of them), we will not be doing this encoding. Instead, let's look at a few encoding techniques that can handle high cardinality categorical variables better.

Frequency encoding

Frequency encoding is an encoding schema that does not increase the dimensions of the problem. It takes a single categorical array and returns a single numeric array. The logic is very simple—it replaces the categorical values with the number of times the value occurs in the training dataset. Although it's not perfect, this works pretty well, as it lets the model distinguish between different categories based on how frequently they occur.

There is a popular library, `category_encoders`, that implements a lot of different encoding schemes in a standard scikit-learn style estimator, and we will be using that in our experiments as well. The standard framework we developed in *Chapter 8, Forecasting Time Series with Machine Learning Models*, also had a couple of functionalities that we didn't use—`encode_categorical` and `categorical_encoder`.

So, let's use them and train our model now:

```
from category_encoders import CountEncoder
from lightgbm import LGBMRegressor

#Define which columns names are categorical features
cat_encoder = CountEncoder(cols=cat_features)
model_config = ModelConfig(
    model=LGBMRegressor(random_state=42),
    name="Global LightGBM with Meta Features (CountEncoder)",
    # LGBM is not sensitive to normalized data
    normalize=False,
    # LGBM can handle missing values
    fill_missing=False,
    # Turn on categorical encoding
    encode_categorical=True,
    # Pass the categorical encoder to be used
    categorical_encoder=cat_encoder
)
```

The rest of the process is the same as what we saw in the *Creating Global Forecasting Models (GFM)* section and we get the forecast using the encoded meta-features:

Algorithm	MAE	MSE	meanMASE	Forecast Bias	Time Elapsed
LightGBM	0.077183	0.027510	0.978056	0.050231	NaN
GFM Baseline	0.079581	0.027326	1.013393	0.218127	28.718087
GFM+Meta (CountEncoder)	0.079411	0.027233	1.011801	0.037475	68.020298

Figure 10.3: Aggregate metrics with the GFM with meta-features (frequency encoding)

Right away, we can see that there is a reduction in error, although it is minimal. We can also see that the training time has almost doubled. This may be because now we have an additional step of encoding the categorical features in addition to training the machine learning model.

The main issue with frequency encoding is that it doesn't work with features that are uniformly distributed in the dataset. For instance, the `LCLid` feature, which is just a unique code for each household, is uniformly distributed in the dataset and when we use frequency encoding, all the `LCLid` features will come to almost the same frequency, and hence the machine learning model considers them almost the same.

Let's now look at a slightly different approach.

Target mean encoding

Target mean encoding, in its most vanilla form, is a very simple concept. It is a *supervised* approach that uses the target in the training dataset to encode the categorical columns. Let's look at an example:



Figure 10.4: Target mean encoding

The vanilla target mean encoding has a few limitations. It increases the chance of overfitting the training data because we are using the mean targets directly and thereby leaking the target into the model in a way. Another problem with the approach is that when the categorical values are unevenly distributed, there may be a few categorical values with very small sample sizes, and therefore, the mean estimate becomes noisy. Extending this problem to the extreme, we get another case where an unseen categorical value comes up in test data. This is also not supported in the vanilla version. Therefore, in practice, this simple version is almost never used, but slightly more sophisticated versions of this concept are widely used and are an effective strategy for encoding categorical features.

In `category_encoders`, there are many variations of this concept, but let's look at two popular and effective ones here.

In 2001, Daniele Micci-Barreca proposed a variant of mean encoding. If we consider the target as a binary variable, say 1 and 0, the mean (which is the number of 1s or number of samples) is also the probability of having 1. Using this interpretation of the means, Daniele proposed blending two probabilities—prior and posterior probabilities—as the final encoding for the categorical features.



Reference check:

The research paper by Daniele Micci-Barreca is cited in *References* as reference 2.

The prior probability is defined as follows:

$$P_{prior} = \frac{n_y}{n_{TR}}$$

Here, n_y is the number of cases such that $target = 1$, and n_{TR} is the number of samples in the training data. The posterior probability is defined for category i as follows:

$$P_{posterior}^i = \frac{n_{iY}}{n_i}$$

Here, n_{iY} is the number of samples in the dataset where $category = i$ and $Y = 1$, and n_i is the number of samples in the dataset where $category = i$.

Now, the final encoding for category i is as follows:

$$S_i = \lambda(n_i) \times P_{posterior}^i + (1 - \lambda(n_i)) \times P_{prior}$$

Here, λ is the weighting factor, which is a monotonically increasing function on n_i that is bounded between 0 and 1. So, this function gives a larger weight to the posterior probability as the number of samples increases.

Adapting this to the regression setting, the probabilities change to expected values so that the formula becomes the following:

$$S_i = \lambda(n_i) \times \frac{\sum_{k \in TR_i} Y_k}{n_i} + (1 - \lambda(n_i)) \left\{ \frac{\sum_{k \in TR} Y_k}{n_{TR}} \right\}$$

Here, TR_i is all the rows where $category = 1$ and $\sum_{k \in TR} Y_k$ is the sum of Y for TR_i . $\sum_{k \in TR} Y_k$ is the sum of Y for all the rows in the training dataset. As with the binary variable, we are mixing the expected value of Y , given $category = i$ ($E[Y|category = i]$) and the expected value of Y ($E[Y]$) for the final categorical encoding.

There are many functions that we can use for λ . Daniele mentions a very common functional form (sigmoid):

$$\lambda(n_i) = \frac{1}{1 + e^{-\frac{n_i - k}{f}}}$$

Here, n_i is the number of samples in the dataset where, $category = i$ and k and f are tunable hyperparameters. k determines half of the minimal sample size for which we completely trust the estimate. If $k = 1$, what we are saying is that we trust the posterior estimate from a category that has only two samples. f determines how fast the sigmoid transitions between the two extremes. As f tends to infinity, the transition becomes a hard threshold between prior and posterior probabilities. TargetEncoder from category_encoders has implemented this λ . The k parameter is called `min_samples_leaf` with a default value of 1, and the f parameter is called `smoothing` with a default value of 1. Let's see how this encoding works on our problem. Using a different encoder in the framework we are working on is as simple as passing a different `cat_encoder` (the initialized categorical encoder) to `ModelConfig`:

```
from category_encoders import TargetEncoder
cat_encoder = TargetEncoder(cols=cat_features)
```

The rest of the code is exactly the same. We can find the full code in the corresponding notebook. Let's see how well the new encoding has done:

Algorithm	MAE	MSE	meanMASE	Forecast Bias	Time Elapsed
LightGBM	0.077183	0.027510	0.978056	0.050231	NaN
GFM Baseline	0.079581	0.027326	1.013393	0.218127	28.718087
GFM+Meta (CountEncoder)	0.079411	0.027233	1.011801	0.037475	68.020298
GFM+Meta (TargetEncoder)	0.079537	0.027218	1.012400	0.335610	43.607325

Figure 10.5: Aggregate metrics with the GFM with meta-features (target encoding)

It's not doing that well, is it? As with machine learning models, the **No Free Lunch Theorem (NFLT)** applies to categorical encoding as well. There is no one encoding scheme that works well all the time. Although not directly related to the topic, if you want to know more about the NFLT, head to the *Further reading* section.



With all these *supervised* categorical encoding techniques, such as target mean encoding, we have to be really careful not to induce data leakage. The encoder should be fit using training data and not using the validation or test data. Another very popular technique is to generate categorical encoding using cross-validation and use the out-of-sample encodings to absolutely avoid data leakage or overfitting.

There are many more encoding schemes, such as `MEstimateEncoder` (which uses additive smoothing as the λ), `HashingEncoder`, and so on, in `category_encoders`. Another very effective way of encoding categorical features is using embedding from deep learning. The *Further reading* section has a link to a tutorial for doing this.

Before now, all this categorical encoding was a separate step before the modeling. Now, let's look at a technique that considers categorical features natively for model training.

LightGBM's native handling of categorical features

A few machine learning model implementations handle categorical features natively, especially gradient-boosting models. CatBoost and LightGBM, two of the most popular GBM implementations, handle categorical features out of the box. CatBoost has a unique way of encoding categorical features into numerical ones internally using something similar to additive smoothing. The *Further reading* section has links to further information on how this encoding is done. `category_encoders` has implemented this logic as `CatBoostEncoder` so that we can use this type of encoding for any machine learning model as well.

While CatBoost handles this internal conversion into numerical features, LightGBM takes a more native approach to dealing with categorical features. LightGBM considers the categorical features as is while growing and splitting the trees. For a categorical feature with k unique values (cardinality of k), there are $2^{k-1}-1$ possible partitions. This soon becomes intractable, but for regression trees, Walter D. Fisher proposed a technique back in 1958 that makes the complexity of finding an optimal split much less. The essence of the method is to use average target statistics for each categorical value to order them and then find the optimal split in the ordered categorical values.



Reference check:

The research paper by Fisher is cited in *References* as reference 3.

LightGBM's `scikit-learn` API supports this feature by taking an argument, `categorical_feature`, which has a list of categorical feature names, during `fit`. We can use the `fit_kwargs` argument in the `fit` of our `MLModel` that we defined in *Chapter 8, Forecasting Time Series with Machine Learning Models*, to pass in this parameter. Let's see how we can do this:

```
from lightgbm import LGBMRegressor
model_config = ModelConfig(
    model=LGBMRegressor(random_state=42),
    name="Global LightGBM with Meta Features (NativeLGBM)",
    # LGBM is not sensitive to normalized data
    normalize=False,
    # LGBM can handle missing values
    fill_missing=False,
    # We are using inbuilt categorical feature handling
    encode_categorical=False,
)
# Training the model and passing in fit_kwargs
y_pred, feat_df = train_model(
    model_config,
    _feat_config,
    missing_value_config,
    train_features,
    train_target,
    test_features,
    fit_kwargs=dict(categorical_feature=cat_features),
)
```

`y_pred` has the forecasts, which we evaluate as usual. Let's also see the results:

Algorithm	MAE	MSE	meanMASE	Forecast Bias	Time Elapsed
LightGBM	0.077183	0.027510	0.978056	0.050231	NaN
GFM Baseline	0.079581	0.027326	1.013393	0.218127	28.718087
GFM+Meta (CountEncoder)	0.079411	0.027233	1.011801	0.037475	68.020298
GFM+Meta (TargetEncoder)	0.079537	0.027218	1.012400	0.335610	43.607325
GFM+Meta (NativeLGBM)	0.079209	0.027329	1.002630	-0.083755	30.316029

Figure 10.6: Aggregate metrics with the GFM with meta-features (native LightGBM)

We can observe a good reduction in MAE as well as meanMASE with the native handling of categorical features. We can also see a reduction in the total training time because we don't have a separate step for encoding the categorical feature. Empirically, the native handling of categorical features works better most of the time.

Now that we have encoded the categorical features, let's look at another way to improve accuracy.

Tuning hyperparameters

A hyperparameter is a setting that controls how a machine learning model is trained but is not learned from the data. In contrast, model parameters are learned from the data during training. For example, in **Gradient Boosting Decision Trees (GBDT)**, model parameters are the *decision thresholds in each tree*, learned from the data. Hyperparameters, like the *number of trees*, *learning rate*, and *tree depth*, are set before training and control the model's structure and how it learns. While parameters adjust based on the data, hyperparameters must be tuned externally.

Although hyperparameter tuning is common practice in machine learning, we haven't been able to do so because of the sheer number of models we had under the LFM paradigm. Now that we have a GFM that finishes training in 30 seconds, hyperparameter tuning becomes feasible. From a theoretical perspective, we also saw that GFMs can afford a larger complexity and can therefore evaluate a greater number of functions to pick the best without overfitting.

Mathematical optimization is defined as the selection of a best element, with regard to some criterion, from some set of available alternatives. In most cases, this involves finding the maximum or minimum value of some function (an **objective function**) from a set of alternatives (the **search space**) subject to some conditions (**constraints**). The search space can be discrete variables, continuous variables, or a mixture of both, and the objective function can be differentiable or non-differentiable. There is a large body of research that tackles these variations.

You may be wondering why we are talking about mathematical optimization now, right? Hyperparameter tuning is a mathematical optimization problem. The objective function here is non-differentiable and returns the metric for which we are optimizing—for instance, the **Mean Absolute Error (MAE)**.

The search space comprises the different hyperparameters we are tuning—say, the number of trees or depth of the trees. It could be a mixture of continuous and discrete variables and the constraints would be any restriction on the search space we impose—for instance, a particular hyperparameter cannot be negative, or a particular combination of hyperparameters cannot occur. Therefore, being aware of the terms used in mathematical optimization will help us in our discussion.

Even though hyperparameter tuning is a standard machine learning concept, we will quickly review three main techniques (besides manual trial and error) for doing hyperparameter tuning.

Grid search

Grid search can be thought of as a brute-force method where we define a discrete grid over the search space, check the objective function at each point in the grid, and pick the best point in that grid. The grid is defined as a set of discrete points for each of the hyperparameters we choose to tune. Once the grid is defined, all the intersections of the grid are evaluated to search for the best objective value. If we are tuning 5 hyperparameters and the grid has 20 discrete values for each parameter, the total number of trials for a grid search would be 3,200,000 (20^5). This means training a model 3.2 million times and evaluating it. This becomes quite limiting because most modern machine learning models have many hyperparameters. For instance, LightGBM has more than 100, and out of those, at least 20 are highly impactful parameters when tuned. So, using a brute force approach such as grid search forces us to make the search space quite small so that it becomes feasible to carry out the tuning in a reasonable amount of time.

For our case, we have defined a very small grid of just 27 trials by limiting ourselves to a really small search space. Let's see how we do that:

```
from sklearn.model_selection import ParameterGrid
grid_params = {
    "num_leaves": [16, 31, 63],
    "objective": ["regression", "regression_l1", "huber"],
    "random_state": [42],
    "colsample_bytree": [0.5, 0.8, 1.0],
}
parameter_space = list(ParameterGrid(grid_params))
```

We just tune three hyperparameters (num_leaves, objective, and colsample_bytree), and with just three options for each parameter. Performing the grid search after this is just about looping over the parameter space and evaluating the model at each combination of hyperparameters:

```
scores = []
for p in tqdm(parameter_space, desc="Performing Grid Search"):
    _model_config = ModelConfig(
        model=LGBMRegressor(**p, verbose=-1),
        name="Global Meta LightGBM Tuning",
        # LGBM is not sensitive to normalized data
        normalize=False,
```

```

        # LGBM can handle missing values
        fill_missing=False,
    )
    y_pred, feat_df = train_model(
        _model_config,
        _feat_config,
        missing_value_config,
        train_features,
        train_target,
        test_features,
        fit_kwargs=dict(categorical_feature=cat_features),
    )
    scores.append(ts_utils.mae(
        test_target['energy_consumption'], y_pred
    ))

```

This takes about 15 minutes to complete and gives us the best MAE of 0.73454, which is already a great improvement from our untuned GFM.

However, this makes us wonder whether there is an even better solution that we haven't covered in the grid we defined. One option is to expand the grid and run the grid search again. This increases the number of trials exponentially and soon becomes infeasible.

Let's look at a different method where we can explore a larger search space with the same number of trials.

Random search

Random search takes a slightly different route. In random search, we also define the search space, but instead of discretely defining specific points in the space, we define probability distributions over the range we want to explore. These probability distributions can be anything from a uniform distribution (which says any point in the range is equally likely) to a Gaussian distribution (which has the familiar peak in the middle), or any other esoteric distributions, such as gamma or beta distributions. As long as we can sample from the distribution, we can use it for random search. Once we define the search space, we can sample points from the distribution and evaluate each of the points to find the best hyperparameter.

While the number of trials is a function of the defined search space for grid search, it is a user input for random search, so we get to decide how much time or computational budget we need to use for hyperparameter tuning and, because of that, we can also search over a larger search space.

With this new flexibility, let's define a larger search space for our problem and use random search:

```

import scipy
from sklearn.model_selection import ParameterSampler
random_search_params = {

```

```

# A uniform distribution between 10 and 100, but only integers
"num_leaves": scipy.stats.randint(10,100),
# A list of categorical string values
"objective": ["regression", "regression_l1", "huber"],
"random_state": [42],
# List of floating point numbers between 0.3 and 1.0 with a resolution of
0.05
"colsample_bytree": np.arange(0.3,1.0,0.05),
# List of floating point numbers between 0 and 10 with a resolution of 0.1
"lambda_l1": np.arange(0,10,0.1),
# List of floating point numbers between 0 and 10 with a resolution of 0.1
"lambda_l2": np.arange(0,10,0.1)
}
# Sampling from the search space number of iterations times
parameter_space = list(ParameterSampler(random_search_params, n_iter=27,
random_state=42))

```

This also runs for about 15 minutes, but we have explored a larger search space. However, the best MAE reported was just 0.73752, which is lower than with grid search. Maybe if we run the search for a greater number of iterations, we will get a better score, but that is just a shot in the dark. Ironically, that is pretty much what random search also does. It closes its eyes and throws a dart at random places on the dartboard and hopes it hits the bull's eye.

There are two terms in mathematical optimization called exploration and exploitation. Exploration ensures the optimization algorithm reaches different regions of the search space, whereas exploitation makes sure we search more in regions that are giving us better results. Random search is purely explorative and is unaware of what is happening as it evaluates different trials.

Let's look at one last technique that tries to balance between exploration and exploitation.

Bayesian optimization

Bayesian optimization has a lot of similarities with random search. Both define their search space as probability distributions, and in both techniques, the user decides how many trials it needs to evaluate, but where they differ is the key advantage of Bayesian optimization. While random search is randomly sampling from the search space, Bayesian optimization is doing it intelligently. Bayesian optimization is aware of its past trials and the objective values that came out of those trials so that it can adapt future trials to exploit the regions where better objective values were seen. At a high level, it does this by building a probability model of the objective function and using it to focus trials on promising areas. The details of the algorithm are worth knowing and we have linked to a couple of resources in *Further reading* to help you along the way.

Now, let's use a popular library, *optuna*, to implement Bayesian optimization for hyperparameter tuning on the GFM we have been training.

The process is quite simple. We need to define a function that takes in a parameter called `trial`. Inside the function, we sample the different parameters we want to tune from the `trial` object, train the model, evaluate the forecast, and return the metric we want to optimize (the MAE). Let's quickly do that:

```
def objective(trial):
    params = {
        # Sample an integer between 10 and 100
        "num_leaves": trial.suggest_int("num_leaves", 10, 100),
        # Sample a categorical value from the list provided
        "objective": trial.suggest_categorical(
            "objective", ["regression", "regression_l1", "huber"]
        ),
        "random_state": [42],
        # Sample from a uniform distribution between 0.3 and 1.0
        "colsample_bytree": trial.suggest_uniform("colsample_bytree", 0.3,
1.0),
        # Sample from a uniform distribution between 0 and 10
        "lambda_l1": trial.suggest_uniform("lambda_l1", 0, 10),
        # Sample from a uniform distribution between 0 and 10
        "lambda_l2": trial.suggest_uniform("lambda_l2", 0, 10),
    }
    _model_config = ModelConfig(
        # Use the sampled params to initialize the model
        model=LGBMRegressor(**params, verbose=-1),
        name="Global Meta LightGBM Tuning",
        # LGBM is not sensitive to normalized data
        normalize=False,
        # LGBM can handle missing values
        fill_missing=False,
    )
    y_pred, feat_df = train_model(
        _model_config,
        _feat_config,
        missing_value_config,
        train_features,
        train_target,
        test_features,
        fit_kwargs=dict(categorical_feature=cat_features),
    )
    # Return the MAE metric as the value
    return ts_utils.mae(test_target["energy_consumption"], y_pred)
```

Once we have defined the objective function, we need to initialize a sampler. `optuna` has many samplers, such as `GridSampler`, `RandomSampler`, and `TPESampler`. For all standard use cases, `TPESampler` is the one to use. `GridSampler` does grid search and `RandomSampler` does random search. When defining a **Tree Parzen Estimator (TPE)** sampler, there are two parameters that we should pay attention to:

- `seed`: This sets the seed for the random sampling. This makes the process reproducible.
- `n_startup_trials`: This is the number of trials that are purely exploratory. This is done to understand the search space before the exploitation kicks in. The default value is 10. We can reduce or increase this depending on how large our sample space is and how many trials we are planning to do.

The rest of the parameters are best left untouched for the most common use cases.

Now, we create a study, which is the object that runs the trials and stores all the details about the trials:

```
# Create a study
study = optuna.create_study(direction="minimize", sampler=sampler)
# Start the optimization run
study.optimize(objective, n_trials=27, show_progress_bar=True)
```

Here, we define the direction of optimization, and we pass in the sampler we initialized earlier. Once the study is defined, we need to call the `optimize` method and pass the objective function we defined, the number of trials we need to run, and some other parameters. A full list of parameters for the `optimize` method is available here—<https://optuna.readthedocs.io/en/stable/reference/generated/optuna.study.Study.html#optuna.study.Study.optimize>.

This runs slightly longer, maybe because of the additional computation required to generate new trials, but still only takes about 20 minutes for the 27 trials. As expected, this has come up with another combination of hyperparameters for which the objective value is 0.72838 (the lowest before now).

To fully illustrate the difference between the three, let's compare how the three techniques spent their computational budget:

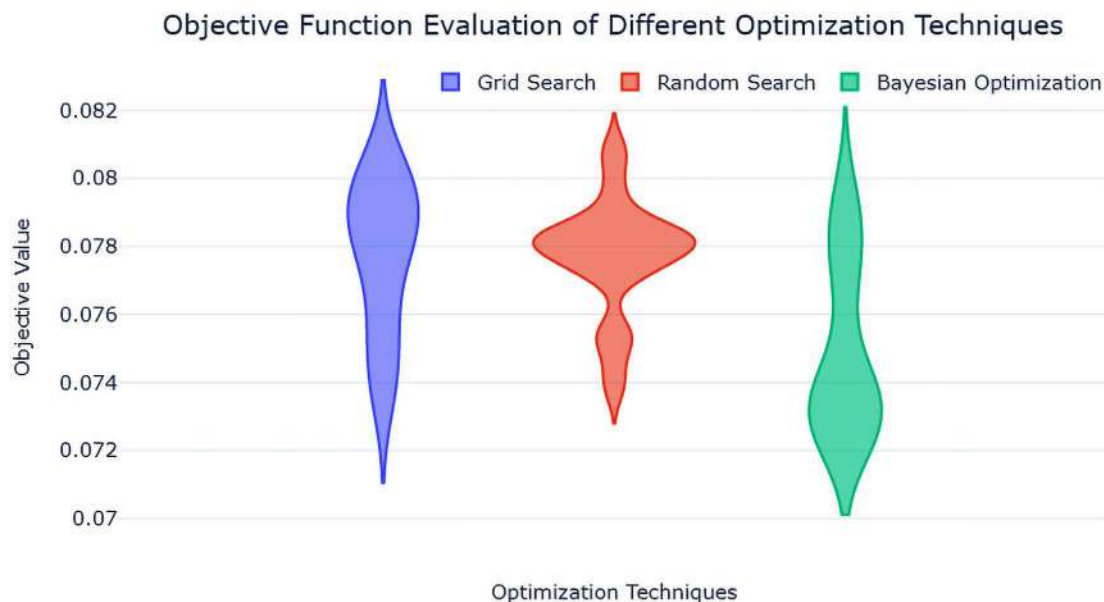


Figure 10.7: Distribution of computational effort (grid versus random versus Bayesian optimization)

We can see that the Bayesian optimization has a fat tail on the lower side, indicating that it spent most of its computational budget evaluating and exploiting the optimal regions in the search space.

Let's look at how the different trials with these techniques fared as the optimization procedure progressed.

The notebook has a more detailed comparison and commentary on the three techniques.

The bottom line is that if we have unlimited computation, grid search with a well-defined and fine-grained grid is the best option, but if we value the efficiency of our computational effort, we should go for Bayesian optimization.

Let's see how the new parameters worked out for us:

Algorithm	MAE	MSE	meanMASE	Forecast Bias	Time Elapsed
LightGBM	0.077183	0.027510	0.978056	0.050231	NaN
GFM Baseline	0.079581	0.027326	1.013393	0.218127	28.718087
GFM+Meta (CountEncoder)	0.079411	0.027233	1.011801	0.037475	68.020298
GFM+Meta (TargetEncoder)	0.079537	0.027218	1.012400	0.335610	43.607325
GFM+Meta (NativeLGBM)	0.079209	0.027329	1.002630	-0.083755	30.316029
Tuned GFM+Meta	0.072918	0.030641	0.900749	-12.412786	57.936451

Figure 10.8: Aggregate metrics with the tuned GFM with meta-features

We have had huge improvements in MAE and meanMASE, mostly because we were optimizing for the MAE when hyperparameter tuning. The MAE and MSE have slightly different priorities and we will spend more time on that in *Part 4, Mechanics of Forecasting*. The runtime also increased because the new parameters build more leaves for a tree than before and are more complex than the default parameters.

Now, let's look at another strategy for improving the performance of a GFM.

Partitioning

Out of all the strategies we have discussed so far, this is the most counter-intuitive, especially if you are coming from a standard machine learning or statistics background. Normally, we would expect the model to do well with more data, but partitioning or splitting the dataset into multiple, almost equal parts has been shown (empirically) to improve the accuracy of the model. While this has been seen empirically, why this happens is something that is still not quite clear. One explanation is that the GFMs have a slightly simpler job of learning when trained on a subset of similar entities and hence, can learn specific functions to subsets of similar entities. Another explanation for the phenomenon has been put forward by Montero-Manso and Hyndman (Reference 1). They put forward that partitioning the data is another form of increasing the complexity because instead of having $\log(|J|)$ as the complexity term, we have

$$\log\left(\prod_{i=1}^P |H_i|\right)$$

where P is the number of partitions. With this rationale, the LFM are special cases where P is equal to the number of time series in the dataset.

There are many ways we can partition the data, each with varying degrees of complexity.

Random partition

The simplest method is to randomly split the dataset into P -equal partitions and train separate models for each partition. This method faithfully follows the explanation that Montero-Manso and Hyndman provide because we are splitting the dataset randomly, with no concern for the similarity of the different households. Let's see how we can do that:

```
# Define a function which splits a list into n partitions
def partition (list_in, n):
    random.shuffle(list_in)
    return [list_in[i::n] for i in range(n)]
# split the unique LCLids into partitions
partitions = partition(train_df.LCLid.cat.categories.tolist(), 3)
```

Then, we just loop over these partitions and train separate models for each partition. The exact code can be found in the notebook. Let's see how well the random partition does:

Algorithm	MAE	MSE	meanMASE	Forecast Bias	Time Elapsed
LightGBM	0.077183	0.027510	0.978056	0.050231	NaN
GFM Baseline	0.079581	0.027326	1.013393	0.218127	28.718087
GFM+Meta (CountEncoder)	0.079411	0.027233	1.011801	0.037475	68.020298
GFM+Meta (TargetEncoder)	0.079537	0.027218	1.012400	0.335610	43.607325
GFM+Meta (NativeLGBM)	0.079209	0.027329	1.002630	-0.083755	30.316029
Tuned GFM+Meta	0.072918	0.030641	0.900749	-12.412786	57.936451
Tuned GFM+Meta+Random Part	0.072598	0.030681	0.898618	-12.361642	49.178089

Figure 10.9: Aggregate metrics with the tuned GFM with meta-features and random partitioning

We can see a decrease in MAE and meanMASE even with a random partition. There is even a decrease in runtime because the individual models are working on less data and hence, train faster.

Now, let's see another way of partitioning, keeping the similarity of different time series in mind.

Judgmental partitioning

Judgmental partitioning is when we use some attribute of the time series to split the dataset, and this is called judgmental because, usually, this depends on the judgment of the person who is working on the model. There are many ways of doing this. We can use some meta-feature, or we can use some characteristics of the time series (such as volume, variability, intermittency, or a combination of them) to partition the dataset.

Let's use a meta-feature called `Acorn_grouped` to partition the dataset. Again, we will just loop over the unique values in `Acorn_grouped` and train a model for each value. We will also not use `Acorn_grouped` as a feature. The exact code is in the notebook. Let's see how well this partitioning does:

Algorithm	MAE	MSE	meanMASE	Forecast Bias	Time Elapsed
LightGBM	0.077183	0.027510	0.978056	0.050231	NaN
GFM Baseline	0.079581	0.027326	1.013393	0.218127	28.718087
GFM+Meta (CountEncoder)	0.079411	0.027233	1.011801	0.037475	68.020298
GFM+Meta (TargetEncoder)	0.079537	0.027218	1.012400	0.335610	43.607325
GFM+Meta (NativeLGBM)	0.079209	0.027329	1.002630	-0.083755	30.316029
Tuned GFM+Meta	0.072918	0.030641	0.900749	-12.412786	57.936451
Tuned GFM+Meta+Random Part	0.072598	0.030681	0.898618	-12.361642	49.178089
Tuned GFM+Meta+ACORN Part	0.072567	0.030786	0.898071	-12.316822	52.118687

Figure 10.10: Aggregate metrics with the tuned GFM with meta-features and `Acorn_grouped` partitioning

This does even better than random partitioning. We can assume each of the partitions (Affluent, Comfortable, and Adversity) has some kind of similarity, which makes the learning easier, and hence, we get better accuracy.

Now, let's look at another way to partition the dataset, again, using similarity.

Algorithmic partitioning

In judgmental partitioning, we pick some meta-features or time series characteristics for partitioning the dataset. We pick a handful of dimensions to partition the dataset because we are doing it in our minds and our mental faculties cannot handle more than two or three dimensions well, but we can see this partitioning as an unsupervised clustering approach and this approach is called algorithmic partitioning.

There are two ways we can cluster time series:

- Extracting features for each time series and using those features to form clusters
- Using time series clustering techniques using the **Dynamic Time Warping (DTW)** distance

`tslearn` is an open source Python library that has implemented a few time series clustering approaches based on the distances between time series. There is a link in *Further reading* for more information on the library and how it can be used for time series clustering.

In our example, we are going to use the first method, where we derive a few time series characteristics and use them for clustering. There are many features from statistical and temporal literature, such as autocorrelation, mean, variance, entropy, and peak-to-peak distance, that we can extract from the time series.

We can use another open source Python library called the **Time Series Feature Extraction Library** (tsfel) to make the process easier.

The library has many classes of features—statistical, temporal, and spectral domains—that we can choose from, and the rest is handled by the library. Let's see how we can generate these features and create a dataframe to perform clustering:

```
import tsfel
cfg = tsfel.get_features_by_domain("statistical")
cfg = {**cfg, **tsfel.get_features_by_domain("temporal")}
uniq_ids = train_df.LCLid.cat.categories
stat_df = []
for id_ in tqdm(uniq_ids, desc="Calculating features for all households"):
    ts = train_df.loc[train_df.LCLid==id_, "energy_consumption"]
    res = tsfel.time_series_features_extractor(cfg, ts, verbose=False)
    res['LCLid'] = id_
    stat_df.append(res)
stat_df = pd.concat(stat_df).set_index("LCLid")
```

The dataframe looks something like this:

	0_ECDF_0	0_ECDF_1	0_ECDF_2	0_ECDF_3	0_ECDF_4	0_ECDF_5	0_ECDF_6	0_ECDF_7	0_ECDF_8	0_ECDF_9	...	0_Median diff
LCLid												
MAC000061	0.000028	0.000057	0.000085	0.000114	0.000142	0.000171	0.000199	0.000228	0.000256	0.000285	...	0.000
MAC000062	0.000028	0.000057	0.000085	0.000114	0.000142	0.000171	0.000199	0.000228	0.000256	0.000285	...	-0.003
MAC000066	0.000028	0.000057	0.000085	0.000114	0.000142	0.000171	0.000199	0.000228	0.000256	0.000285	...	0.000
MAC000086	0.000028	0.000057	0.000085	0.000114	0.000142	0.000171	0.000199	0.000228	0.000256	0.000285	...	-0.002
MAC000126	0.000028	0.000057	0.000085	0.000114	0.000142	0.000171	0.000199	0.000228	0.000256	0.000285	...	-0.002

Figure 10.11: Features extracted from different time series

Now that we have the dataframe with each row representing a time series with different features, we can ideally apply any clustering method, such as k-means, k-medoids, or HDBSCAN, and find clusters. However, in high dimensions, a lot of the distance metrics (including Euclidean) do not work as well as they are supposed to. There is a seminal paper on the topic by Charu C. Agarwal et al. from 2001 that explores the topic. When we increase the dimensionality of the space, our common sense (which conceptualizes three dimensions) does not work as well and, as a consequence, common distance metrics such as Euclidean distance do not work very well with high dimensions. We have linked to a blog summarizing the paper (in *Further reading*) and the paper itself (Reference 5), which make the concept clearer. So, a common way of handling high-dimensional clustering is by performing dimensionality reduction first and then using normal clustering.

Principal Component Analysis (PCA) was the go-to tool in the field, but since PCA only captures and details linear relationships while reducing the dimensions, nowadays, another class of techniques is starting to become more popular—manifold learning.

t-distributed Stochastic Neighbor Embeddings (t-SNE) is a popular technique from this category, which is really popular for high-dimensional visualization. It is a really clever technique where we project the points from a high-dimensional space to a lower dimension, keeping the distribution of distance in the original space as close as possible to the one in lower dimensions. There is a lot to learn here that is beyond the scope of this book. There are links in the *Further reading* section that can help you get started.

To cut a long story short, we will be using t-SNE to reduce the dimensions of the dataset we have and then cluster the dataset with the reduced dimensions. If you really want to cluster time series and use those clusters in some other way, I would not suggest using t-SNE because it doesn't preserve the distance between points and the density of points. The [distil.pub](#) article in *Further reading* throws more light on the issue. But in our case, we are using the clusters just as a grouping for training another model, so this approximation can do well. Let's see how we do that:

```
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from src.utils.data_utils import replace_array_in_dataframe
from sklearn.manifold import TSNE #T-Distributed Stochastic Neighbor Embedding
# Standardizing to make distance calculation fair
X_std = replace_array_in_dataframe(stat_df, StandardScaler().fit_
transform(stat_df))
#Non-Linear Dimensionality Reduction
tsne = TSNE(n_components=2, perplexity=50, learning_rate="auto", init="pca",
random_state=42, metric="cosine", square_distances=True)
X_tsne = tsne.fit_transform(X_std.values)
# Clustering reduced dimensions into 3 clusters
kmeans = KMeans(n_clusters=3, random_state=42).fit(X_tsne)
cluster_df = pd.Series(kmeans.labels_, index=X_std.index)
```

Since we reduced the dimensions to two, we can also visualize the clusters formed:

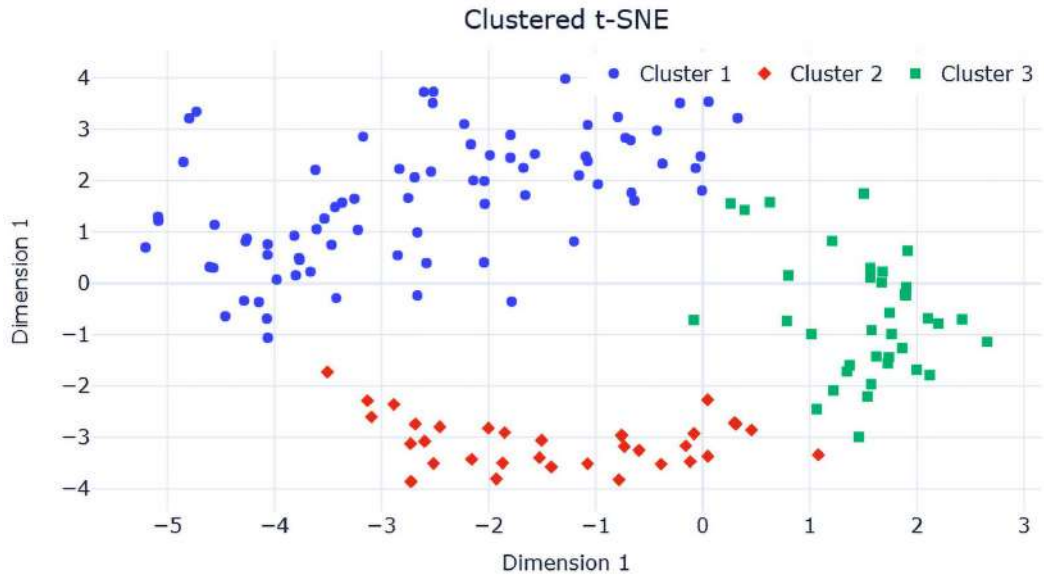


Figure 10.12: Clustered time series after t-SNE dimensionality reduction

We have three well-defined clusters formed and now we are just going to use these clusters to train a model for each cluster. As usual, we loop over the three clusters and train the models. Let’s see how we did so:

	Algorithm	MAE	MSE	meanMASE	Forecast Bias	Time Elapsed
0	LightGBM	0.077183	0.027510	0.978056	0.050231	NaN
1	GFM Baseline	0.079581	0.027326	1.013393	0.218127	28.718087
2	GFM+Meta (CountEncoder)	0.079411	0.027233	1.011801	0.037475	68.020298
3	GFM+Meta (TargetEncoder)	0.079537	0.027218	1.012400	0.335610	43.607325
4	GFM+Meta (NativeLGBM)	0.079209	0.027329	1.002630	-0.083755	30.316029
5	Tuned GFM+Meta	0.072918	0.030641	0.900749	-12.412786	57.936451
6	Tuned GFM+Meta+Random Part	0.072598	0.030681	0.898618	-12.361642	49.178089
7	Tuned GFM+Meta+ACORN Part	0.072567	0.030786	0.898071	-12.316822	52.118687
8	Tuned GFM+Meta+Clustered Part	0.072347	0.029976	0.905182	-12.521149	66.373510

Figure 10.13: Aggregate metrics with the tuned GFM with meta-features and clustered partitioning

It looks as though this is the best MAE we have seen in all our experiments, but the three partition techniques have very similar MAEs. We can't see whether any one is better than the other just by looking at a single hold-out set. For good measure, we can run these forecasts with a test dataset using the 01a-Global_Forecasting_Models-ML-test.ipynb notebook in the Chapter08 folder. Let's see how the aggregate metrics are on the test dataset:

Algorithm	MAE	MSE	meanMASE	Forecast Bias	Time Elapsed
LightGBM	0.0751	0.0271	0.9142	2.57%	nan
GFM Baseline	0.0773	0.0280	0.9586	0.71%	38.147337
GFM+Meta (CountEncoder)	0.0772	0.0276	0.9600	0.69%	71.797225
GFM+Meta (TargetEncoder)	0.0773	0.0276	0.9612	0.99%	48.736455
GFM+Meta (NativeLGBM)	0.0770	0.0279	0.9483	0.84%	32.467879
Tuned GFM+Meta	0.0700	0.0310	0.8384	-12.38%	62.091281
Tuned GFM+Meta+Random Part	0.0706	0.0339	0.8405	-12.96%	55.361825
Tuned GFM+Meta+ACORN Part	0.0696	0.0305	0.8342	-12.43%	52.066995
Tuned GFM+Meta+Clustered Part	0.0685	0.0285	0.8282	-11.94%	57.231065

Figure 10.14: Aggregate metrics on test data

As expected, the clustered partition is still the methodology that performs the best in this case.

In *Chapter 8, Forecasting Time Series with Machine Learning Models*, it took us 8 minutes and 20 seconds to train an LFM for all the households in our dataset. Now, with the GFM paradigm, we finished training a model in 57 seconds (in the worst-case scenario). That's 777% less training time and this comes with an 8.78% decrease in the MAE.

We chose to do these experiments with LightGBM. This does not mean that LightGBM or any other gradient-boosting model is the only choice for GFMs, but they are a pretty good default. A well-tuned gradient-boosted trees model is a very difficult baseline to beat, but as always in machine learning, we should check what works best using well-defined experiments.

Although there are no hard and fast rules or cutoffs for when a GFM makes more sense than an LFM, as the number of time series in a dataset increases, the GFM becomes more favorable, both from the perspective of accuracy and computation.

Although we have achieved good results using GFMs, typically the complex models that do well in this paradigm are black boxes. Let's look at some ways to open the black box and understand and explain the model better.

Interpretability

Interpretability can be defined as the degree to which a human can understand the cause of a decision. In machine learning and artificial intelligence, that translates to the degree to which someone can understand the how and why of an algorithm and its predictions. There are two ways to look at interpretability—transparency and post hoc interpretation.

Transparency is when the model is inherently simple and can be simulated or thought about using human cognition. A human should be able to fully understand the inputs and the process a model takes to convert these inputs to outputs. This is a very stringent condition that almost none of the model machine learning or deep learning models satisfy.

This is where *post hoc interpretation* techniques shine. There is a wide variety of techniques that use the inputs and outputs of a model to understand why a model has made the predictions it has.

There are many popular techniques such as *permutation feature importance*, *Shapley values*, and *LIME*. All of these are general-purpose interpretation techniques that can be used on any machine learning model and that includes the GFMs we were discussing. Let's talk about a few of them at a high level.

Mean decrease in impurity:

This is the regular “feature importance” that we get out of the box from tree-based models. This technique measures how much a feature reduces impurity (such as Gini impurity in classification or variance in regression) when used to split nodes in a decision tree. The higher the reduction in impurity, the more important the feature is considered. However, it's biased towards continuous features or those with high cardinality. It is fast and readily available in libraries like scikit-learn but may give misleading results if features have varying scales or many categories.

Drop column importance (Leave One Covariate Out (LOCO)):

This method assesses feature importance by iteratively removing one feature at a time and retraining the model. The drop in performance from the baseline model indicates the importance of that feature. It is model-agnostic and captures interactions between features, but is computationally expensive since it requires retraining the model for each feature removed. It can also give misleading results if collinear features exist, as the model may compensate for the removed feature.

Permutation importance:

Permutation importance measures the drop in model performance when the values of a single feature are randomly shuffled, disrupting its relationship with the target. This technique is intuitive and model-agnostic, and it doesn't require retraining the model, making it computationally efficient. However, it can inflate the importance of correlated features, as models can rely on related features to compensate for the permuted one.

Partial Dependence Plots (PDPs) and Individual Conditional Expectation (ICE) plots:

PDPs visualize the average effect of a feature on the model's predictions, showing how the target variable changes as the feature's values change, while ICE plots show the effect of a feature for individual instances. These plots help understand the feature-target relationship but assume independence between features, which can lead to misleading interpretations in the presence of correlated variables.

Local Interpretable Model-agnostic Explanations (LIME):

LIME is a model-agnostic technique that explains individual predictions by approximating a complex model locally using simpler, interpretable models, like linear regression. It works by generating perturbations of the data point in question and fitting a local model to these samples. This method is intuitive and widely applicable to both structured and unstructured data (text and images), but defining the right locality for perturbations can be challenging, especially for tabular data.

SHapley Additive exPlanations (SHAP):

SHAP unifies several interpretation methods, including Shapley values and LIME, into a single framework that attributes feature importance in a model-agnostic way. SHAP provides both local and global interpretations and benefits from fast implementations for tree-based models (TreeSHAP). It combines the theoretical strength of Shapley values with practical efficiency, although it can still be computationally intensive for large datasets.

Each technique has its strengths and trade-offs, but SHAP stands out due to its strong theoretical foundation and ability to connect local and global interpretations effectively. For more extensive coverage of such techniques, I have included a few links in *Further reading*. The blog series by yours truly and the free book by Christopher Molnar are excellent resources for you to get up to speed (more on interpretability in *Chapter 17*).

Congratulations on finishing the second part of the book! It has been quite an intensive part where we went over quite a bit of theory and practical lessons, and we hope you are now comfortable with using machine learning for time series forecasting.

Summary

To round up the second part of the book nicely, we explored GFM in detail and saw why they are important and why they are an exciting new direction in time series forecasting. We saw how we can use a GFM using machine learning models and also reviewed many techniques to make GFM perform better, most of which are quite frequently used in competitions and industry use cases alike. We also took a high-level look at the interpretability techniques. Now that we have wrapped up the machine learning section of the book, we will move on to a specific type of machine learning that has become well-known over the past few years—**deep learning**—in the next chapter.

References

The following are sources that we have referenced throughout the chapter:

1. Montero-Manso, P., Hyndman, R.J. (2020), *Principles and algorithms for forecasting groups of time series: Locality and globality*. arXiv:2008.00444[cs.LG]: <https://arxiv.org/abs/2008.00444>.

2. Micci-Barreca, D. (2001), *A preprocessing scheme for high-cardinality categorical attributes in classification and prediction problems*. *SIGKDD Explor. Newsl.* 3, 1 (July 2001), 27–32: <https://doi.org/10.1145/507533.507538>.
3. Fisher, W. D. (1958). *On Grouping for Maximum Homogeneity*. *Journal of the American Statistical Association*, 53(284), 789–798: <https://doi.org/10.2307/2281952>.
4. Fisher, W.D. (1958), *A preprocessing scheme for high-cardinality categorical attributes in classification and prediction problems*. *SIGKDD Explor. Newsl.* 3, 1 (July 2001), 27–32.
5. Aggarwal, C. C., Hinneburg, A., and Keim, D. A. (2001). *On the Surprising Behavior of Distance Metrics in High Dimensional Spaces*. In *Proceedings of the 8th International Conference on Database Theory (ICDT '01)*. Springer-Verlag, Berlin, Heidelberg, 420–434: <https://dl.acm.org/doi/10.5555/645504.656414>.
6. Oreshkin, B. N., Carpow D., Chapados N., and Bengio Y. (2020). *N-BEATS: Neural basis expansion analysis for interpretable time series forecasting*. *8th International Conference on Learning Representations, ICLR 2020*: <https://openreview.net/forum?id=r1ecqn4YwB>.

Further reading

The following are a few resources that you can explore for a detailed study:

- *Learning From Data* by Yaser Abu-Mostafa: <https://work.caltech.edu/lectures.html>
- *Curse of Dimensionality*—Georgia Tech: <https://www.youtube.com/watch?v=OyPcbeiwps8>
- *Dummy Variable Trap*: <https://www.learndatasci.com/glossary/dummy-variable-trap/>
- Using deep learning to learn categorical embeddings: <https://pytorch-tabular.readthedocs.io/en/latest/tutorials/03-Neural%20Embedding%20in%20Scikit-Learn%20Workflows/>
- Handling categorical features—CatBoost: https://catboost.ai/en/docs/concepts/algorithm-main-stages_cat-to-numeric
- *Exploring Bayesian Optimization*—from Distil.pub: <https://distill.pub/2020/bayesian-optimization/>
- Frazier, P.I. (2018). *A Tutorial on Bayesian Optimization*. arXiv:1807.02811 [stat.ML]: <https://arxiv.org/abs/1807.02811>
- Time series clustering using tslearn: https://tslearn.readthedocs.io/en/stable/user_guide/clustering.html
- *The Surprising Behaviour of Distance Metrics in High Dimensions*: <https://towardsdatascience.com/the-surprising-behaviour-of-distance-metrics-in-high-dimensions-c2cb72779ea6>
- *An illustrated introduction to the t-SNE algorithm*: <https://www.oreilly.com/content/an-illustrated-introduction-to-the-t-sne-algorithm/>
- *How to Use t-SNE Effectively*—from Distil.pub: <https://distill.pub/2016/misread-tsne/>
- The NFLT: https://en.wikipedia.org/wiki/No_free_lunch_in_search_and_optimization
- *Interpretability: Cracking open the black box* – parts I, II, and III by Manu Joseph: <https://deep-and-shallow.com/2019/11/13/interpretability-cracking-open-the-black-box-part-i/>

- *Interpretable Machine Learning: A Guide for Making Black Box Models Explainable* by Christoph Molnar: <https://christophm.github.io/interpretable-ml-book/>
- *Global models for time series forecasting*: <https://www.sciencedirect.com/science/article/abs/pii/S0031320321006178>

**Unlock this book's exclusive
benefits now**

This book comes with additional benefits
designed to elevate your learning experience.



Note: Have your purchase invoice ready before you begin. <https://www.packtpub.com/unlock/9781835883181>

Part 3

Deep Learning for Time Series

In this part, we focus on the exciting field of deep learning to tackle time series problems. This part starts with a good introduction of the necessary concepts and slowly builds up to different specialized architectures that are suited to handle time series data. It also talks about global models in deep learning and some strategies to make them work better. And to top it off, we dive deep into generating probabilistic forecasts which is highly relevant in today's forecasting landscape.

This part comprises the following chapters:

- *Chapter 11, Introduction to Deep Learning*
- *Chapter 12, Building Blocks of Deep Learning for Time Series*
- *Chapter 13, Common Modeling Patterns for Time Series*
- *Chapter 14, Attention and Transformers for Time Series*
- *Chapter 15, Strategies for Global Deep Learning Forecasting Models*
- *Chapter 16, Specialized Deep Learning Architectures for Forecasting*
- *Chapter 17, Probabilistic Forecasting and More*

11

Introduction to Deep Learning

In the previous chapter, we learned how to use modern machine learning models to tackle time series forecasting. Now, let's focus our attention on a subfield of machine learning that has shown a lot of promise in the last few years—**deep learning**. We will be trying to demystify deep learning and go into why it is popular nowadays. We will also break down deep learning into major components and learn about the workhorse behind deep learning—gradient descent.

In this chapter, we will be covering these main topics:

- What is deep learning and why now?
- Components of a deep learning system
- Representation learning
- Linear layers and activation functions
- Gradient descent

Technical requirements

You will need to set up the **Anaconda** environment following the instructions in the *Preface* of the book to get a working environment with all the libraries and datasets required for the code in this book. Any additional libraries will be installed while running the notebooks.

The associated code for the chapter can be found at <https://github.com/PacktPublishing/Modern-Time-Series-Forecasting-with-Python-2E/tree/main/notebooks/Chapter11>.

What is deep learning and why now?

In *Chapter 5, Time Series Forecasting as Regression*, we talked about machine learning and borrowed a definition from Arthur Samuel: “*Machine Learning is a field of study that gives computers the ability to learn without being explicitly programmed.*” And we further saw how we can learn useful functions from data using machine learning. Deep learning is a subfield of this same field of study. The objective of deep learning is also to learn useful functions from data but with a few specifications on how it does that.

Before we talk about what is special about deep learning, let's answer another question first. Why are we talking about this subfield of machine learning as a separate topic? The answer to that lies in the unreasonable effectiveness of deep learning methods in countless applications. Deep learning has taken the world of machine learning by storm, overthrowing state-of-the-art systems across types of data such as images, videos, text, and so on. If you remember the speech recognition systems on phones a decade ago, they were more meme-worthy than really useful. But today, you can say *Hey Google*, *play Pink Floyd*, and *Comfortably Numb* will start playing on your phone or speakers. Multiple deep learning systems made this process possible in a smooth way. The voice assistant on your phone, self-driving cars, web search, language translation—the list of applications of deep learning in our day-to-day lives just keeps on going.

By now, you might be wondering what this new technology called deep learning is all about, right? Deep learning is not a new technology. The origins of deep learning can be traced way back to the late 1940s and early 1950s. It only appears to be new because of the recent surge in popularity of the field.

Let's quickly see why deep learning is suddenly popular.

Why now?

There are two main reasons why deep learning has gained a lot of ground in the last two decades:

- Increase in compute availability
- Increase in data availability

Let's discuss the preceding points in detail in the following sections.

Increase in compute availability

Back in 1960, Frank Rosenblatt wrote a paper (Reference 5) about a three-layer neural network and stated that it went a long way in demonstrating the ability of neural networks as a pattern-recognizing device. But in the same paper, he noted that the burden on a digital computer (of the 1960s) was too great as we increased the number of connections. However, in the decades that followed, computer hardware showed close to 50,000 times more improvement, which provided a good boost to neural networks and deep learning. However, it was still not enough as neural networks were still not considered to be good enough for *large-scale applications*.

This is when a particular type of hardware, which was initially developed for gaming, came to the rescue—GPUs. It's not entirely clear who started using GPUs for deep learning. Kyoung-Su Oh and Keechul Jung published a paper titled *GPU implementation of neural networks* back in 2004, which seems to be the first to show massive speed-ups in using GPUs for deep learning. One of the earliest and more popular research papers on the topic came from Rajat Raina, Anand Madhavan, and Andrew Ng, who published a paper titled *Large-scale deep unsupervised learning using graphics processors* back in 2009. It showed the effectiveness of GPUs for deep learning.

Although many groups led by LeCun, Schmidhuber, Bengio, and so on were playing around with using GPUs, the turning point came when Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton used a GPU-based deep learning system that outperformed all the other competing technologies in an image recognition contest called the *ImageNet Large Scale Visual Recognition Challenge 2012*.

The introduction of GPUs provided a much-needed boost to the widespread use of deep learning and accelerated the progress in the field.

**Reference check:**

The research papers *GPU implementation of neural networks*, *Large-scale deep unsupervised learning using graphics processors*, and *ImageNet Classification with Deep Convolutional Neural Networks* are cited in the *References* section under 1, 2, and 3, respectively.

Increase in data availability

In addition to the skyrocketing compute capability, the other main factor that helped deep learning was the sheer increase in data. As the world became more and more digitized, the amount of data that we generated increased drastically. Tables that had hundreds and thousands of rows now exploded into millions and billions of rows, and the ever-decreasing cost of storage helped this explosion of data collection.

And why would an increase in data availability help deep learning? This lies in the way deep learning works. Deep learning is quite data-hungry and needs large amounts of data to learn good models. Therefore, if we keep increasing the data that we provide to a deep learning model, the model will be able to learn better and better functions. However, the same can't be said for traditional machine learning models. Let's cement this learning with a chart that Andrew Ng, a world-renowned machine learning educator and an adjunct professor at Stanford, popularized in his famous machine learning course—*Machine Learning by Stanford University* on Coursera (<https://www.coursera.org/specializations/machine-learning-introduction>) (Figure 11.1).

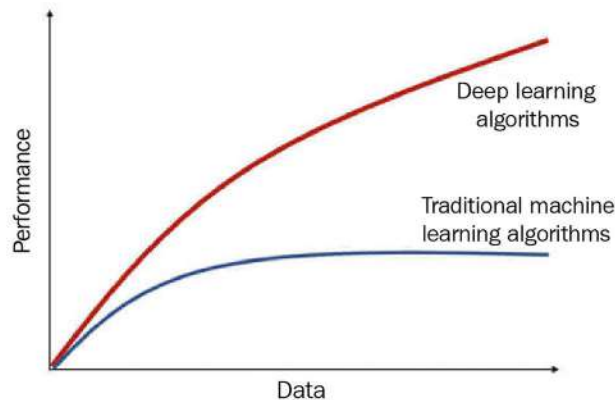


Figure 11.1: Deep learning versus traditional machine learning as we increase the data size

In Figure 11.1, which was popularized by Andrew Ng, we can see that as we increase the data size, traditional machine learning hits a plateau and won't improve anymore.

It has been proven empirically that there are significant benefits to the overparameterization of a deep learning model. **Overparameterization** means that there are more parameters in the model than the number of data points available to train. In classical statistics, this is a big no-no because, under this scenario, the model invariably overfits. But deep learning seems to flaunt this rule with ease. One of the examples of overparameterization is the current state-of-the-art image recognition system, **Noisy-Student**. It has 480 million parameters, but it was trained on *ImageNet* with 1.2 million data points.

It has been argued that the way deep learning models are trained (stochastic gradient descent, which we will be explaining soon) is the key because it has a regularizing effect. In a research paper titled *The Computational Limits of Deep Learning*, Niel C. Thompson and others tried to illustrate this using a simple experiment. They set up a dataset with 1,000 features, but only 10 of them had any signal in them. Then, they tried to learn four models based on the dataset using varying dataset sizes:

- **Oracle model:** A model that uses the exact 10 parameters that have any signal in them.
- **Expert model:** A model that uses 9 out of 10 significant parameters.
- **Flexible model:** A model that uses all 1,000 parameters.
- **Regularized model:** A model that uses all 1,000 parameters, but is now a regularized (lasso) model. (We covered regularization back in *Chapter 8, Forecasting Time Series with Machine Learning Models*.)

Let's see *Figure 11.2* from the research paper with the study:

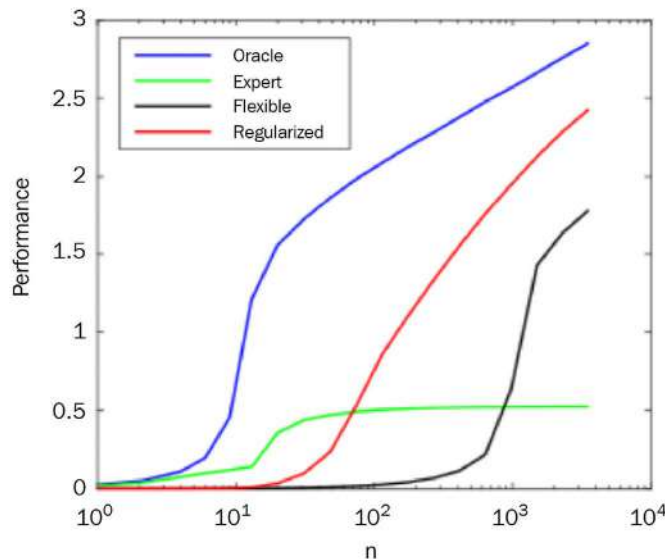


Figure 11.2: The chart shows how different models perform under different sizes of data

The chart has a number of data points used on the x -axis and the performance ($-\log(\text{Mean Squared Error})$) on the y -axis. The different colored lines show the different types of models. The Oracle model sets the upper limit for learning because it has access to perfect information.

The expert model plateaus because there is a definite lack of information as it doesn't have access to one of the ten features that are important. The flexible model (which uses all 1,000 features) takes a large number of data points to start recognizing the important ones but still keeps approaching the Oracle performance as the data size gets bigger. The regularized model (which is a proxy for deep learning models) keeps improving as we give the model more and more data. This model uses regularization to figure out which of these 1,000 features is relevant to the problem and starts to leverage them with much fewer data points, and performance keeps increasing with more data points. This strengthens the concept Andrew Ng popularized once more—with more data, deep learning starts to outperform traditional machine learning.

A lot more factors, apart from compute and data availability, have contributed to the success of deep learning. Sara Hooker, in her essay *The Hardware Lottery* (Reference 9), talks about how an idea wins not necessarily because it is superior to other ideas but because it is suited to the software and hardware available at the time. And once a research direction wins the lottery, it snowballs because more funding and big research organizations get behind that idea and it eventually becomes the most prominent idea in the space of ideas.

We have talked about deep learning for some time but have still not understood what it is. Let's do that now.

What is deep learning?

There is no single definition of deep learning because it means slightly different things to different people. However, a large majority of people agree on one thing: a model is called deep learning when it involves automatic feature learning from raw data. As Yoshua Bengio (a Turing Award winner and one of the *godfathers* of AI) explains in his 2021 paper titled *Deep Learning of Representations for Unsupervised and Transfer Learning*:



“Deep learning algorithms seek to exploit the unknown structure in the input distribution in order to discover good representations, often at multiple levels, with higher-level learned features defined in terms of lower-level features.”

In a 2016 presentation, *Deep Learning and Understandability versus Software Engineering and Verification*, Peter Norvig, the Director of Research at Google, had a similar but simpler definition:



“A kind of learning where the representation you form have (sic) several levels of abstraction, rather than a direct input to output.”

Another key feature of deep learning a lot of people agree upon is compositionality. Yann LeCun, a Turing Award winner and another one of the *godfathers* of AI, has a slightly more complex but more exact definition of deep learning (tweet from @y1ecun on January 9th, 2020):



“DL is methodology: building a model by assembling parameterized modules into (possibly dynamic) graphs and optimizing it with gradient-based methods.”

The key points we would like to highlight here are as follows:

- **Assembling parametrized modules:** This refers to the compositionality of deep learning. Deep learning systems, as we will shortly see, are composed of a few submodules with a few parameters (some without) assembled into a graph-like structure.
- **Optimizing it with gradient-based methods:** Although having gradient-based learning as a sufficient criterion for deep learning is not widely accepted, we can still see empirically that, today, most successful deep learning systems are trained using gradient-based methods. (If you are not aware of what a gradient-based optimization method is, don't worry. We will be covering it soon in this chapter.)

If you have read anything about deep learning before, you may have seen neural networks and deep learning used together or interchangeably. But we haven't talked about neural networks till now. Before we do that, let's look at a fundamental unit of any neural network.

Perceptron, the first neural network

A lot of what we call deep learning and neural networks are deeply influenced by the human brain and its inner workings. Although recent studies have shown very little similarity between human brains and artificial neural networks, the seed behind the idea was inspired by human biology. The human desire to create intelligent beings like themselves was manifested as early as back in Greek mythology (Galatea and Pandora). And owing to this desire, humans have studied and looked for inspiration from human anatomy for years. One of the organs of the human body that has been studied intensely is the brain because it is the center of intelligence, creativity, and everything else that makes a human.

Even though we still don't know a lot about the brain, we do know a bit about it, and we use that little information to design artificial systems. The fundamental unit of the human brain is something we call a **neuron**, as shown here:

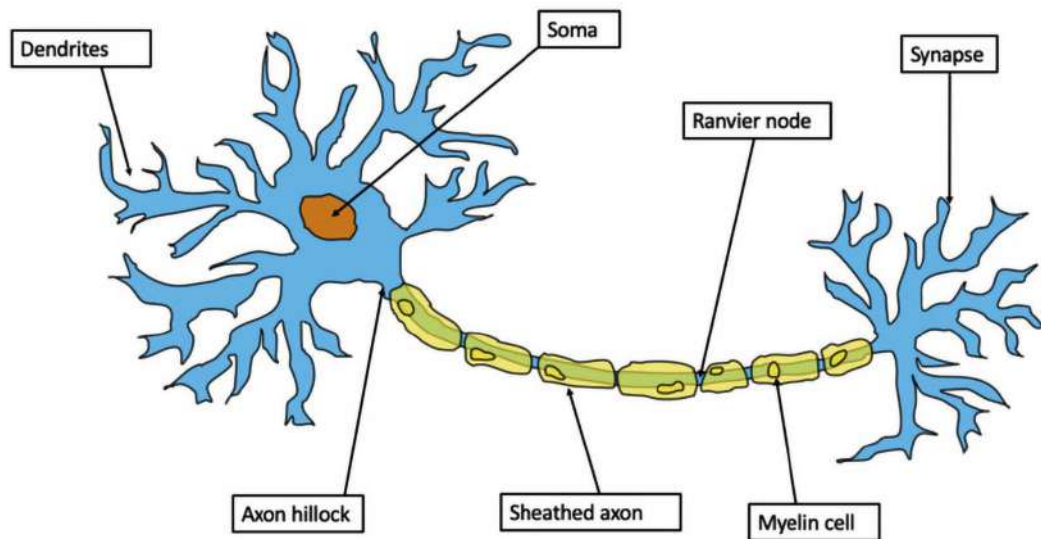


Figure 11.3: A biological neuron

Many of you might have come across this in biology or in the context of machine learning as well. But let's refresh this anyway. The biological neuron has the following parts:

- **Dendrites** are branched extensions of the nerve cell that collect inputs from surrounding cells or other neurons.
- **Soma**, or the cell body, collects these inputs, joins them, and is passed on.
- **The axon hillock** connects the soma to the axon, and it controls the firing of the neuron. If the strength of a signal exceeds a threshold, the axon hillock fires an electrical signal through the axon.
- **Axon** is the fiber that connects the soma to the nerve endings. It is the axon's duty to pass on the electrical signal to the endpoints.
- **Synapses** are the end points of the nerve cell and transmit the signal to other nerve cells.

McCulloch and Pitts (1943) were the first to design a mathematical model for the biological neuron. However, the McCulloch-Pitts model had a few limitations:

- It only accepted binary variables.
- It considered all input variables equally important.
- There was only one parameter, a threshold, which was not learnable.

In 1957, Frank Rosenblatt generalized the McCulloch-Pitts model and made it a full model whose parameters could be learned. The similarity between modern deep learning networks and the human brain ends here. The fundamental unit of learning that started this line of research was inspired by human biology and was a rather cheap imitation of it as well.



Reference check:

The original research paper for Frank Rosenblatt's *perceptron* is cited in *References* under reference 5.

Let's understand the perceptron in detail because it is the fundamental building block of all neural networks:

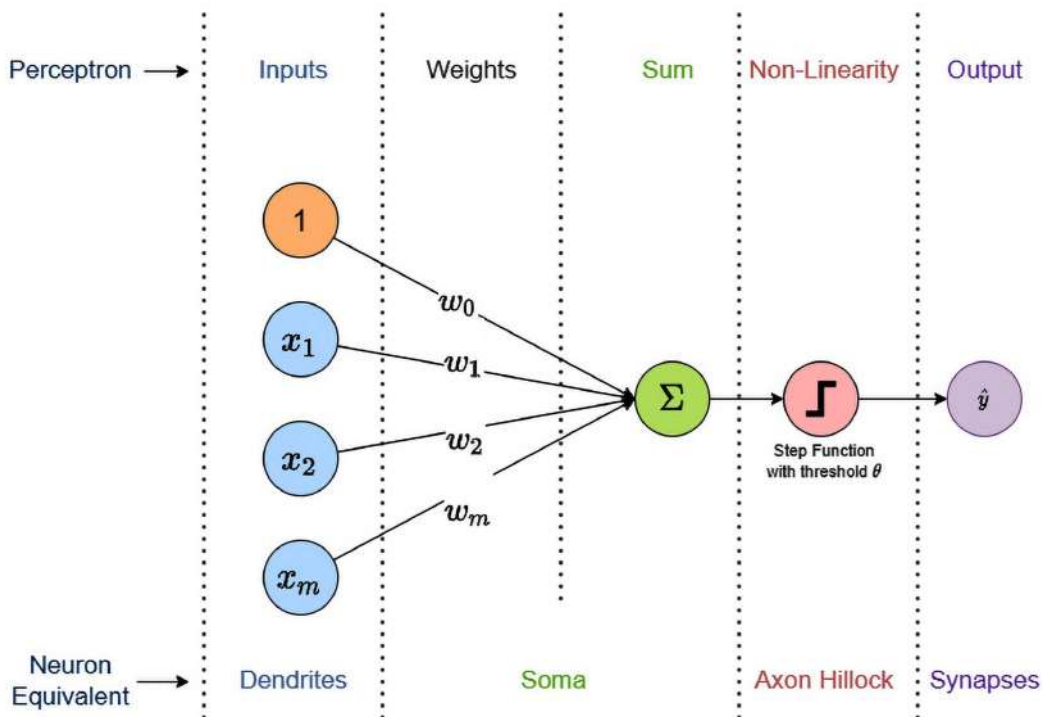


Figure 11.4: Perceptron

As we see from Figure 11.4, the perceptron has the following components:

- **Inputs:** These are the real-valued inputs that are fed to a perceptron. This is like the dendrites in neurons that collect the input.
- **Weighted sum:** Each input is multiplied by a corresponding weight and summed up. The weights determine the importance of each input in determining the outcome.

- **Non-linearity:** The weighted sum goes through a non-linear function. For the original perceptron, it was a step function with a threshold activation. The output would be positive or negative based on the weighted sum and the threshold of the unit. Modern-day perceptrons and neural networks use different kinds of activation functions, but we will see that later on.

We can write the perceptron in the mathematical form as follows:

The diagram shows the equation $\hat{y} = g\left(w_0 + \sum_{i=1}^m x_i w_i\right)$ with arrows pointing to different parts:

- Output** (purple) points to $\hat{y}$.
- Bias** (orange) points to w_0 .
- Linear Combination of Inputs** (blue) points to the summation term $\sum_{i=1}^m x_i w_i$.
- Non-Linearity** (red) points to the activation function g .

Below this, the equation is written in vector form: $\hat{y} = g(\mathbf{X}_T \mathbf{W})$.

Vector Form $\longrightarrow$ where

$$\mathbf{X} = \begin{bmatrix} 1 \\ x_1 \\ \dots \\ x_m \end{bmatrix} \quad \text{and} \quad \mathbf{W} = \begin{bmatrix} w_0 \\ w_1 \\ \dots \\ w_m \end{bmatrix}$$

Figure 11.5: Perceptron, a math perspective

As shown in Figure 11.5, the perceptron output is defined by the weighted sum of inputs, which is passed in through a non-linear function. Now, we can think of this using linear algebra as well. This is an important perspective for two reasons:

- The linear algebra perspective will help you understand neural networks faster.
- It will also make the whole thing feasible because matrix multiplications are something that our modern-day computers and GPUs are really good at. Without linear algebra, multiplying these inputs with corresponding weights would require us to loop through the inputs, and it quickly becomes infeasible.

Linear algebra intuition recap

Let's take a look at a couple of concepts as a refresher. Feel free to jump ahead if you are already aware of vectors, vector spaces, and matrix multiplication.

Vectors and vector spaces

At the superficial level, a **vector** is an array of numbers. However, in linear algebra, a vector is an entity that has both magnitude and direction. Let's take an example to elucidate:

$$A = \begin{bmatrix} 5 \\ 0 \end{bmatrix}$$

We can see that this is an array of numbers. But if we plot this point in the two-dimensional coordinate space, we get a point. And if we draw a line from the origin to this point, we will get an entity with direction and magnitude. This is a vector.

The two-dimensional coordinate space is called a **vector space**. A two-dimensional vector space, informally, is all the possible vectors with two entries. Extending it to n -dimensions, an n -dimensional vector space is all the possible vectors with n entries.

The final intuition I want to leave with you is this: *a vector is a point in the n -dimensional vector space.*

Matrices and transformations

Again, at the superficial level, a **matrix** is a rectangular arrangement of numbers that looks like this:

$$M = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}$$

Matrices have many uses but the one intuition that is most relevant for us is that a matrix specifies a linear transformation of the vector space it resides in. When we multiply a vector with a matrix, we are essentially transforming the vector, and the values and dimensions of the matrix define the kind of transformation that happens. Depending on the content of the matrix, it does *rotation*, *reflection*, *scaling*, *shearing*, and so on.

We have included a notebook in the `Chapter11` folder titled `01-Linear_Algebra_Intuition.ipynb`, which explores matrix multiplication as a transformation. We also apply these transformation matrices to vector spaces to develop intuition on how matrix multiplication can rotate and warp the vector spaces.

I highly suggest heading over to the *Further reading* section, where we have given a few resources to get started and solidify necessary intuition.



If we consider the inputs as vectors in the feature space (vector space with m -dimensions), the term $\sum_{i=1}^m x_i w_i$ is nothing but a linear combination of input vectors. We can rewrite the equation in vector form as below:

$$\hat{y} = g(\mathbb{X}_T \mathbb{W})$$

$$\text{where, } \mathbb{X} = \begin{bmatrix} 1 \\ x_1 \\ \dots \\ x_m \end{bmatrix}, \mathbb{W} = \begin{bmatrix} w_0 \\ w_1 \\ \dots \\ w_m \end{bmatrix}$$

The bias is also included here as a dummy input with a fixed value of 1 and adding w_0 to the $\mathbb{W}$ vector.

Now that we have had an introduction to deep learning, let us recall one of the aspects of deep learning we discussed earlier—compositionality—and explore it a bit more deeply in the next section.

Components of a deep learning system

Let us recall Yann LeCun’s definition of deep learning:



“Deep learning is a methodology: building a model by assembling parameterized modules into (possibly dynamic) graphs and optimizing it with gradient-based methods.”

The core idea here is that deep learning is an extremely modular system. Deep learning is not just one model but, rather, a language to express any model in terms of a few parametrized modules with these specific properties:

1. It should be able to produce an output from a given input through a series of computations.
2. If the desired output is given, it should be able to pass on information to its inputs on how to change, to arrive at the desired output. For instance, if the output is lower than what is desired, the module should be able to tell its inputs to change in some direction so that the output becomes closer to the desired one.

The more mathematically inclined may have figured out the connection to the second point of differentiation. And you would be correct. To optimize these kinds of systems, we predominantly use gradient-based optimization methods. Therefore, condensing the two properties into one, we can say that these parameterized modules should be *differentiable functions*.

Let's take the help of a visual to aid further discussion.

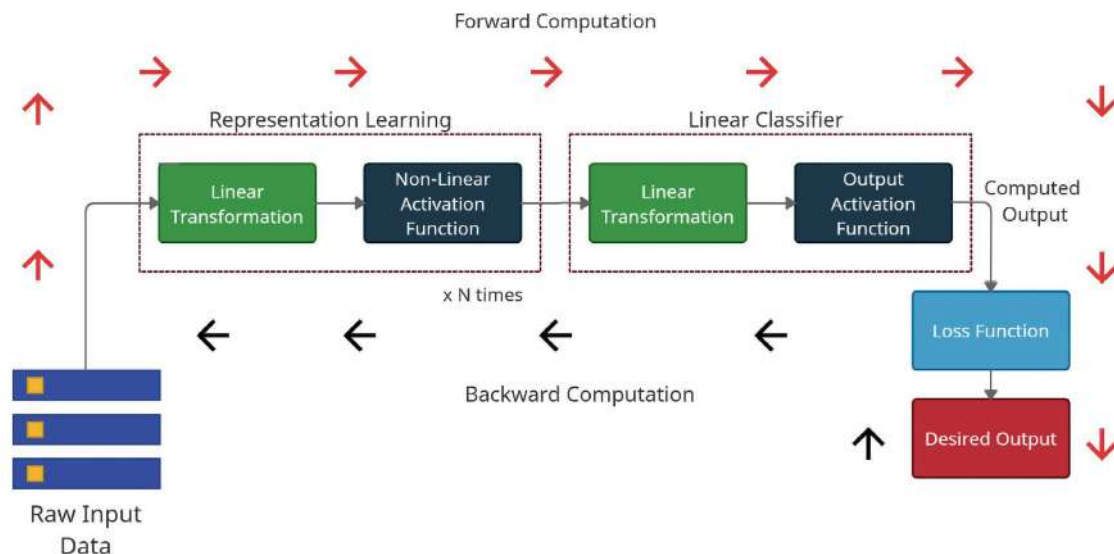


Figure 11.6: A deep learning system

As shown in *Figure 11.6*, deep learning can be thought of as a system that takes in raw input data through a series of linear and non-linear transforms to provide us with an output. It also can adjust its internal parameters to make the output as close as possible to the desired output through learning. To make the diagram simpler, we have chosen a paradigm that fits most of the popular deep learning systems. It all starts with raw input data. The raw input data goes through N blocks of linear and non-linear functions that do representation learning. Let's explore these blocks in some detail.

Representation learning

Representation learning, informally, learns the best features by which we can make the problem linearly separable. Linearly separable means when we can separate the different classes (in a classification problem) with a straight line (*Figure 11.7*):

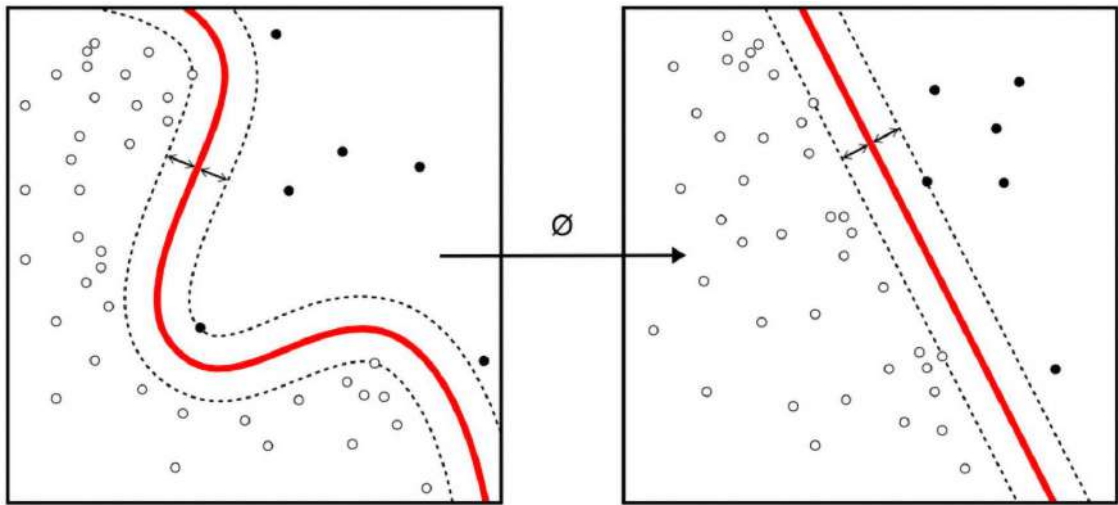


Figure 11.7: Transforming non-linearly separable data into linearly separable using a function, ϕ

The *representation learning* block in Figure 11.6 may have multiple linear and non-linear functions stacked on top of each other, and the overall function of the block is to learn a function, ϕ , which transforms the raw input into good features that make the problem linearly separable.

Another way to look at this is through the lens of linear algebra. As we explored earlier in the chapter, matrix multiplication can be thought of as a linear transformation of vectors. And if we extend that intuition to the vector spaces, we can see that matrix multiplication warps the vector space in some way or another. When we stack multiple linear and non-linear transformations on top of each other, we are essentially warping, twisting, and squeezing the input vector space (with the features) into another space. When we ask a parameterized system to warp the input space (pixels of images) in such a way as to perform a particular task (such as the classification of dogs versus cats), the representation learning block learn the right transformations, which makes the task (separating cats from dogs) easier.

I have created a video illustrating this because nothing establishes intuition better than a video of what is happening. I've taken a sample dataset that is not linearly separable, trained a neural network on the problem to classify, and then visualized how the input space was transformed by the model into a linearly separable representation. You can find the video here: <https://www.youtube.com/watch?v=5xYEa9PPDTE>.

Now, let's look inside the representation learning block. We can see there is a linear transformation and a non-linear activation.

Linear transformation

Linear transformations are just transformations that are applied to the vector space. When we say linear transformation in a neural network context, we actually mean affine transformations.

A linear transformation fixes the origin while applying the transformation, but an affine transformation doesn't. Rotation, reflection, scaling, and so on are purely linear transformations because the origin won't change while we do this. But something like a translation, which moves the vector space, is an affine transformation. Therefore, $A \cdot X^T$ is a linear transformation, but $A \cdot X^T + b$ is an affine transformation.

So, linear transformations are simply matrix multiplications that transform the input vector space, and this is at the heart of any neural network or deep learning system today.

What happens if we stack linear transformations on top of each other? For instance, we first multiply the input, X , with a transformation matrix, A , and then multiply the results with another transformation matrix, B :

$$T = B \cdot (A \cdot X)$$

By the associative property (which is applicable to linear algebra as well), we can rewrite this equation as follows:

$$T = (B \cdot A) \cdot X$$

Generalizing this to a stack of N transformation matrices, we can see that it all works out to be a single linear transformation. This kind of defeats the purpose of stacking N layers, doesn't it?

This is where the non-linearity becomes essential and we introduce non-linearities by using a non-linear function, which we call activation functions.

Activation functions

Activation functions are non-linear differentiable functions. In a biological neuron, the axon hillock decides whether to fire a signal based on the inputs. The activation functions serve a similar function and are key to the neural network's ability to model non-linear data. In other words, activation functions are key in neural networks' ability to transform input vector space (which is linearly inseparable) to a linearly separable vector space, informally. To *unwarp* a space such that linearly inseparable points become linearly separable, we need to have non-linear transformations.

We repeated the same experiment we did in the last section, where we visualized the trained transformation of a neural network on the input vector space, but this time, without any non-linearities. The resulting video can be found here: <https://www.youtube.com/watch?v=z-nV8oBpH2w>. The best transformation that the model learned is just not sufficient and the points are still linearly inseparable.

Theoretically, an activation function can be any non-linear differentiable (differentiable almost everywhere, to be exact) function. However, over the course of time, there are a few non-linear functions that are popularly used as activation functions. Let's look at a few of them.

Sigmoid

Sigmoid is one of the most common activation functions around, and probably one of the oldest. It is also known as the logistic function. When we discussed perceptron, we mentioned a step (also called *Heaviside* in literature) function as the activation function. The step function is not a continuous function and hence *is not* differentiable everywhere. A very close substitute is the sigmoid function.

It is defined as follows:

$$g(x) = \frac{1}{1+e^{-x}}$$

Where g is the sigmoid function and x is the input value.

Sigmoid is a continuous function and therefore *is* differentiable everywhere. The derivative is also computationally simpler to calculate. Because of these properties of the sigmoid, it was adopted widely in the early days of deep learning as a standard activation function.

Let's see what a sigmoid function looks like and how it transforms a vector space:

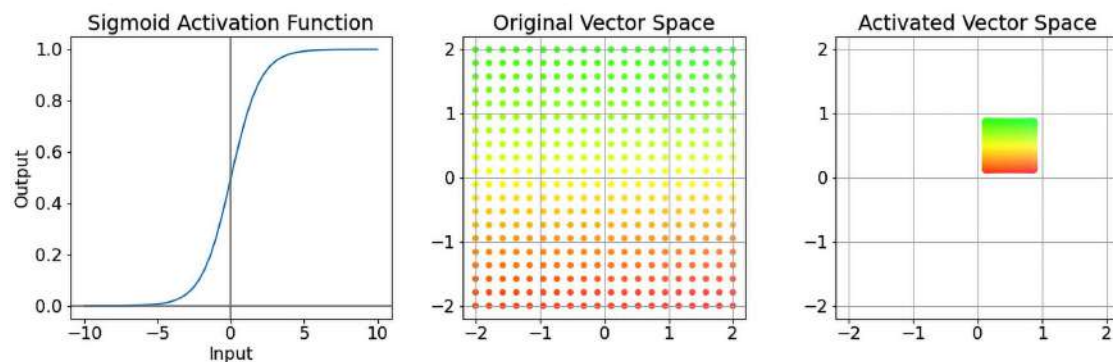


Figure 11.8: Sigmoid activation function (left) and original and activated vector space (middle and right)

 **Quick tip:** Need to see a high-resolution version of this image? Open this book in the next-gen Packt Reader or view it in the PDF/ePub copy.

 **The next-gen Packt Reader and a free PDF/ePub copy** of this book are included with your purchase. Unlock them by scanning the QR code below or visiting <https://www.packtpub.com/unlock/9781835883181>.



The sigmoid function squashes the input between 0 and 1, as seen in *Figure 11.8 (left)*. We can observe the same phenomenon in the vector space. One of the drawbacks of the sigmoid function is that the gradients tend to zero on the flat portions of the sigmoid. When a neuron approaches this area in the function, the gradients that it receives and propagates become negligible and the unit stops learning. We call this *saturating of the activation*. Because of this, nowadays, *sigmoid* is not typically used in deep learning, except in the output layer (we will be talking about this usage soon).

Hyperbolic tangent (tanh)

Hyperbolic tangents are another popular activation. They can be easily defined as follows:

$$\tanh(x) = \frac{\sinh(x)}{\cosh(x)}$$

It is very similar to sigmoid. In fact, we can express *tanh* as a function of sigmoid. Let's see what the activation function looks like:

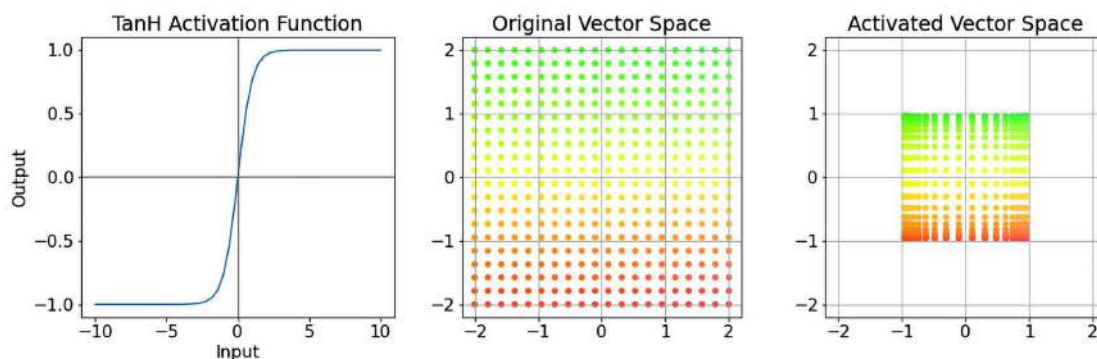


Figure 11.9: Tanh activation function (left) and original and activated vector space (middle and right)

We can see that the shape is similar to sigmoid, although a bit sharper. But the key difference is that the *tanh* function outputs a value between -1 and 1. And because of the sharpness, we can also see the vector space getting pushed out to the edges as well. The fact that the function outputs a value that is symmetrical around the origin (0) works well with the optimization of the network and hence *tanh* was preferred over *sigmoid*. But since the *tanh* function is also a saturating function, the same problem of very small gradients hampering the flow of gradients and, in turn, learning plagues *tanh* activations as well.

Rectified linear units and variants

As neuroscience gained more information about the human brain, researchers found out that only one to four percent of neurons in the brain are activated at any time. But with all the activation functions such as *sigmoid* or *tanh*, almost half of the neurons in a network are activated. In 2010, Vinod Nair and Geoffrey Hinton proposed **rectified linear units (ReLU)** in the seminal paper *Rectified Linear Units Improve Restricted Boltzmann Machines*. Since then, ReLUs have taken over as the de facto activation functions for deep neural networks.

ReLU

A ReLU is defined as follows:

$$g(x) = \max(x, 0)$$

It is just a linear function but with a kink at zero. Any value greater than zero is retained as is, but all values below zero are squashed to zero. The range of the output goes from 0 to ∞ . Let's see how it looks visually:

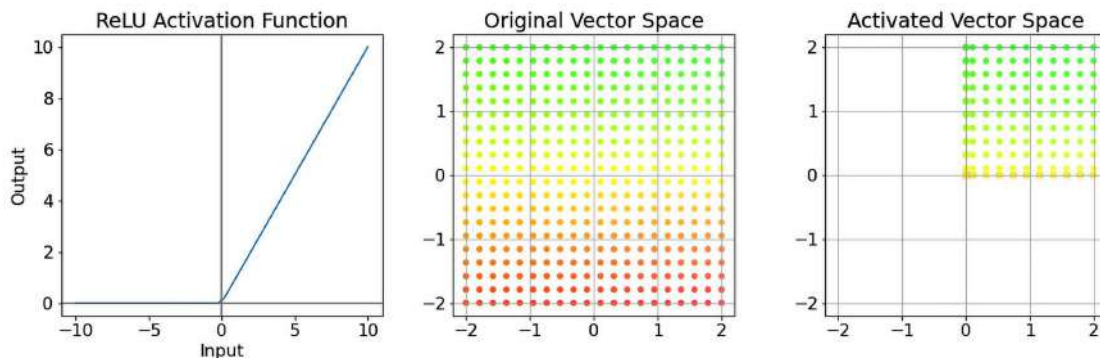


Figure 11.10: ReLU activation function (left) and original and activated vector space (middle and right)

We can see that the points in the left and bottom quadrants are all pushed into the axes' lines. This squashing is what gives the non-linearity to the activation function. Because of the way the activation sharply becomes zero and does not tend to zero like the sigmoid or tanh, ReLUs are non-saturating.



Reference check:

The research paper that proposed ReLU is cited in *References* under reference 7.

There are a few advantages to using ReLUs:

- The computations of the activation function as well as its gradients are really cheap.
- Training converges much faster than those with saturating activation functions.
- ReLU helps bring sparsity in the network (by having the activation as zero, a large majority of neurons in the network can be turned off) and resembles how biological neurons work.

But ReLUs are not without problems:

- When $x < 0$, the gradients become zero. This means a neuron that has an output < 0 will have zero gradients and, therefore, the unit will not learn anymore. These are called dead ReLUs.
- Another disadvantage is that the average output of a ReLU unit is positive, and when we stack multiple layers, this might lead to a positive bias in the output.

Let's see a few variants that try to resolve the problems we discussed for ReLU.

Leaky ReLU and parametrized ReLU

Leaky ReLU is a variant of standard ReLU that resolves the *dead ReLU* problem. It was proposed by Maas and others in 2013. A leaky ReLU can be defined as follows:

$$g(x) = \begin{cases} x, & \text{if } x \geq 0 \\ \alpha x, & \text{if } x < 0 \end{cases}$$

Here, α is the slope parameter (typically set to a very small value such as 0.001) and is considered a hyperparameter. This makes sure the gradients are not zero when $x < 0$ and thereby ensures there are no *dead* ReLUs. But the sparsity that ReLU provides is lost here because there is no zero output that turns off a unit completely. Let's visualize this activation function:

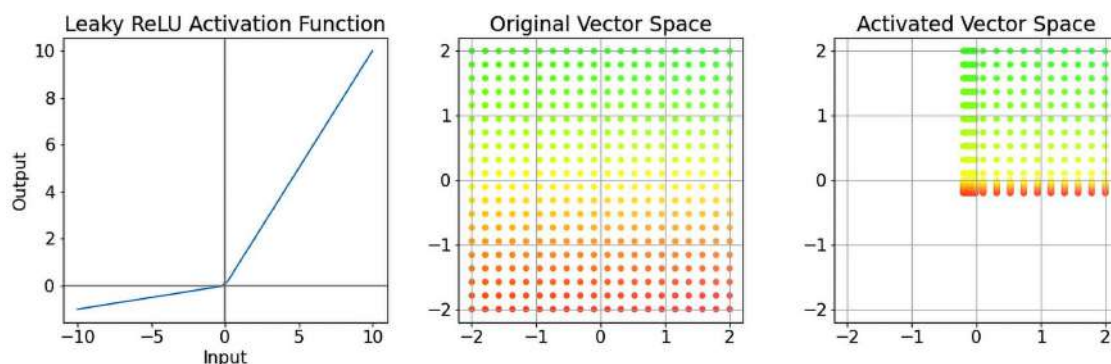


Figure 11.11: Leaky ReLU activation function (left) and original and activated vector space (middle and right)

In 2015, K. He and others proposed another minor modification to leaky ReLU called **parametrized ReLU**. In parametrized ReLU, instead of considering α as a hyperparameter, they considered it as a learnable parameter.



Rerefence check:

The research paper that proposed leaky ReLU is cited in *References* under reference 8, and parametrized ReLU is cited under reference 9.

There are many other activation functions that are less popularly used but still have enough use cases to be included in *PyTorch*. You can find a list of them here: <https://pytorch.org/docs/stable/nn.html#non-linear-activations-weighted-sum-nonlinearity>. We encourage you to use the notebook titled 02-Activation_Functions.ipynb in the Chapter 11 folder to try out different activation functions and see how they warp the vector space.

And with that, we now have an idea of the components of the first block in *Figure 11.6*, representation learning. The next block in there is the linear classifier, which has a linear transformation and an output activation. We already know what a linear transformation is, but what is an output activation?

Output activation functions

Output activation functions are functions that enforce a few desirable properties to the output of the network.



Additional reading:

These functions have a deeper connection with **maximum likelihood estimation (MLE)** and the chosen loss function, but we will not be getting into that because it is out of the scope of this book. We have linked to the book *Deep Learning* by Ian Goodfellow, Yoshua Bengio, and Aaron Courville in the *Further reading* section. If you are interested in a deeper understanding of deep learning, we suggest you use the book to that effect.

If we want the neural network to predict a continuous number in the case of regression, we just use a linear activation function (which is like saying there is no activation function). The raw output from the network is considered the prediction and fed into the loss function.

But in the case of classification, the desired output is a class out of all possible classes. If there are only two classes, we can use our old friend, the *sigmoid* function, which has an output between 0 and 1. We can also use *tanh* because its output is going to be between -1 and 1. The *sigmoid* function is preferred because of the intuitive probabilistic interpretation that comes along with it. The closer the value is to one, the more confident the network is about that prediction.

Now, *sigmoid* works for binary classification. What about multiclass classification where the possible classes are more than two?

Softmax

Softmax is a function that converts a vector of K real values into another K -positive real value, which sums up to 1. *Softmax* is defined as follows:

$$\text{Softmax}(x_i) = \frac{e^{x_i}}{\sum_{j=1}^K e^{x_j}}$$

This function converts the raw output from a network into something that resembles a probability across K classes. This has a strong relation with *sigmoid*—*sigmoid* is a special case of *softmax* when $K = 2$.

In the following figure, let's see how a random vector of size 3 is converted into probabilities that add up to 1:

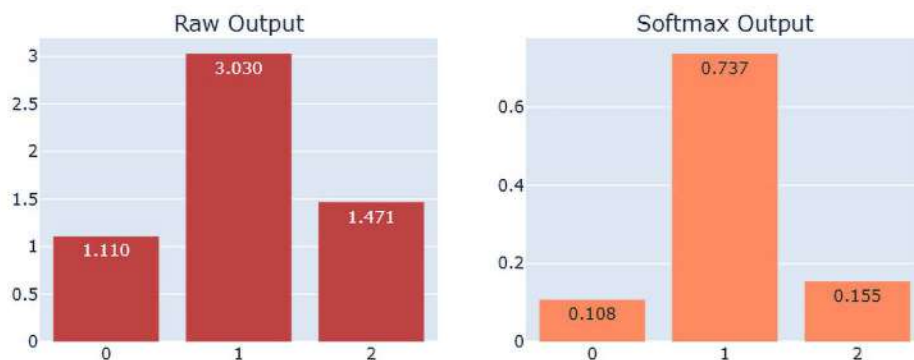


Figure 11.12: Raw output versus softmax output

If we look closely, we can see that in addition to converting the real values into something that resembles probability, it also increases the relative gap between the maximum and the rest of the values. This activation is a standard output activation for multiclass classification problems.

Now, there is only one major component left in the diagram (Figure 11.6)—the loss function.

Loss function

The loss function we touched upon in *Chapter 5, Time Series Forecasting as Regression*, translates nicely to deep learning. In deep learning also, the loss function is a way to tell how good the predictions of the model are. If the predictions are way off the target, the loss function will be higher, and as we get closer to the truth, it becomes smaller. In the deep learning paradigm, we just have one more additional requirement of the loss function—it should be differentiable.

Common loss functions from classical machine learning, such as **mean squared error** or **mean absolute error**, are valid in deep learning as well. In fact, in regression tasks, they are the default choices that practitioners adopt. For classification tasks, we adopt a concept borrowed from information theory called **cross-entropy loss**. However, since deep learning is a very flexible framework, we can use any loss function as long as it is differentiable. There are a lot of loss functions people have already tried and found work in many situations. A lot of them are part of PyTorch's API as well. You can find them here: <https://pytorch.org/docs/stable/nn.html#loss-functions>.

Now that we have covered all the components of a deep learning system, let's also briefly look at how we train the whole system.

Forward and backward propagation

In Figure 11.6, we can see two sets of arrows, one going toward the desired output from the input, marked as *Forward Computation*, and another going backward to the input from the desired output, marked *Backward Computation*. These two steps are at the core of learning a deep learning system.

In *Forward Computation*, popularly known as **forward propagation**, we use the series of computations that are defined in the layers and propagate the input all the way through the network to get the output. Now that we have the output, we would use the loss function to assess how close or far we are from the desired output. This information is now used in *Backward Computation*, popularly known as **backward propagation**, to calculate the gradient with respect to all the parameters.

Now, what is a gradient and why do we need it? In high school math, we might have come across gradients or derivatives in another form called **slope**. It is the rate of change of a quantity when we change a variable by unit measure. Derivatives inform us of the local slope of a scalar function. While derivatives are always with respect to a single variable, gradients are a generalization of derivatives to multivariate functions. Intuitively, both gradients and derivatives inform us of the local slope of the function. And with the gradient of the loss function, we can use one of the techniques from mathematical optimization, called **gradient descent**, to optimize our loss function.

Let's see this with an example.

Gradient descent

Any machine learning or deep learning model can be thought of as a function that converts an input, x , to an output, $\hat{y}$, using a few parameters, θ . Here, θ can be the collection of all the matrix transformations that we do to the input throughout the network. But to simplify the example, let's assume there are only two parameters, a and b . If we think about the whole process of learning a bit, we will see that by keeping the input and expected output the same, the way to change your loss would be by changing the parameters of the model. Therefore, we can postulate the loss function to be parameterized by the parameters—in this case, a and b .



Notebook alert:

To follow along with the complete code, use the notebook named `03-Gradient_Descent.ipynb` in the `Chapter11` folder and the code in the `src` folder.

Let's assume the loss function takes the following form:

$$\mathcal{L}(a, b) = (a - 8)^2 + (b - 2)^2$$

Let's see what the function looks like. We can use a three-dimensional plot to visualize a function with two parameters, as seen in *Figure 11.13*. Two dimensions will be used to denote the two parameters and, at each point in that two-dimensional mesh, we can plot the loss value in the third dimension.

This kind of plot of the loss function is also called a loss curve (in univariate settings), or loss surface (in multivariate settings).

$$\text{Loss Surface for } \mathcal{L}(a, b) = (a - 8)^2 + (b - 2)^2$$

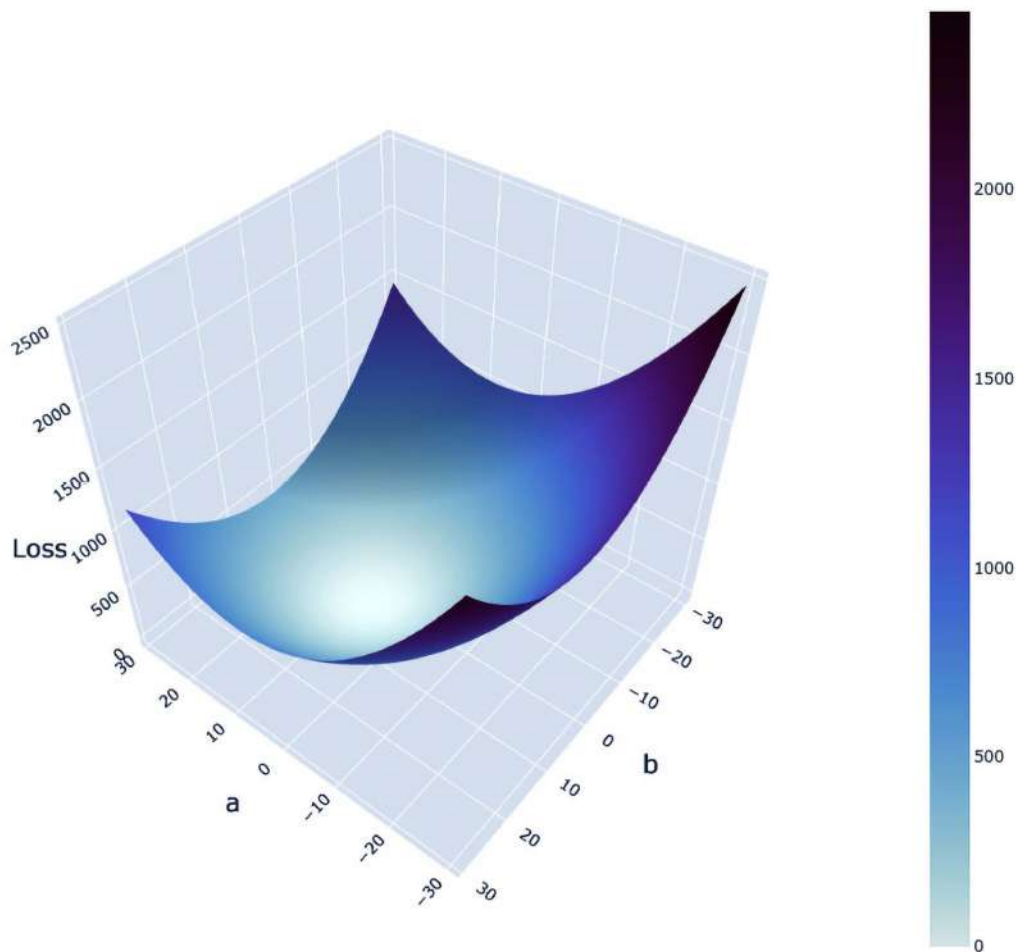


Figure 11.13: Loss surface plot

The lighter portion of the 3D shape is where the loss function is less, and as we move away from there, it increases.

In machine learning, our aim is to minimize the loss function, or in other words, find the parameters that make our predicted output as close as possible to the ground truth. This falls under the realm of mathematical optimization, and a particular technique lends itself suitable for this approach—**gradient descent**.

Gradient descent is a mathematical optimization algorithm used to minimize a cost function by iteratively moving in the direction of the steepest descent. In a univariate function, the derivative (or the slope) gives us the direction (and magnitude) of the steepest ascent. For instance, if we know that the slope of a function is 1, we know if we move to the right, we are climbing up the slope, and moving to the left, we will be climbing down. Similarly, in the multivariate setting, the gradient of a function at any point will give us the direction (and magnitude) of the steepest ascent. And since we are concerned with minimizing a loss function, we will be using the negative gradient, which will point us in the direction of the steepest descent.

So, let's define the gradient for our loss function. We are using high school calculus, but even if you are not comfortable, you don't need to worry:

$$\nabla f(a, b) = \begin{bmatrix} \frac{df}{da} \\ \frac{df}{db} \end{bmatrix} = \begin{bmatrix} 2(a - 8) \\ 2(b - 2) \end{bmatrix}$$

Now, how does the algorithm work? Very simply, as follows:

1. Initialize the parameters to random values.
2. Compute the gradient at that point.
3. Make a step in the direction opposite to the gradient.
4. Repeat steps 2 and 3 until it converges or we reach maximum iterations.

There is just one more aspect that needs more clarity: how much of a step do we take in each iteration?

Ideally, the magnitude of the gradient tells you how fast the function is changing in that direction, and we should just take the step equal to the gradient. But there is a property of the gradient that makes that a bad idea. The gradient only defines the direction and magnitude of the steepest ascent in the infinitesimally small locality of the current point and is blind to what happens beyond it. Therefore, we use a hyperparameter, commonly called the **learning rate**, to temper the steps we take in each iteration. Therefore, instead of taking a step equal to the gradient, we take a step equal to the learning rate multiplied by the gradient.

Mathematically, if θ is the vector of parameters, at each iteration, we update the parameters using the following formula:

$$\theta = \theta - \eta \times \Delta f(a, b)$$

Here, η is the learning rate, and Δf is the gradient at the point.

Let's see a very simple implementation of gradient descent (refer to the notebook in the GitHub repository for the end-to-end definition and working code). First, let's define a function that returns the gradient at any point:

```
def gradient(a, b):
    return 2*(a-8), 2*(b-2)
```

Now we define a few initial parameters such as the maximum iterations, learning rate, and initial value of a and b :

```
# maximum number of iterations that can be done
maximum_iterations = 500
# current iteration
current_iteration = 0
# Learning Rate
learning_rate = 0.01
#Initial value of a, b
current_a_value = 28
current_b_value = 27
Now, all that is left is the actual process of gradient descent:

while current_iteration < maximum_iterations:
    previous_a_value = current_a_value
    previous_b_value = current_b_value
    # Calculating the gradients at current values
    gradient_a, gradient_b = gradient(previous_a_value, previous_b_value)
    # Adjusting the parameters using the gradients
    current_a_value = current_a_value - learning_rate * gradient_a *
(previous_a_value)
    current_b_value = current_b_value - learning_rate * gradient_b *
(previous_b_value)
    current_iteration = current_iteration + 1
```

 **Quick tip:** Enhance your coding experience with the **AI Code Explainer** and **Quick Copy** features. Open this book in the next-gen Packt Reader. Click the **Copy** button (1) to quickly copy code into your coding environment, or click the **Explain** button (2) to get the AI assistant to explain a block of code to you.



 The next-gen Packt Reader is included for free with the purchase of this book. Unlock it by scanning the QR code below or visiting <https://www.packtpub.com/unlock/9781835883181>.



We know the minimum for this function will be at $a = 8$ and $b = 2$ because that would make the loss function zero. And gradient descent finds a solution that is pretty accurate— $a = 8.000000000000005$ and $b = 2.000000002230101$. We can also visualize the path it took to reach the minimum, as seen in *Figure 11.14*:

Gradient Descent

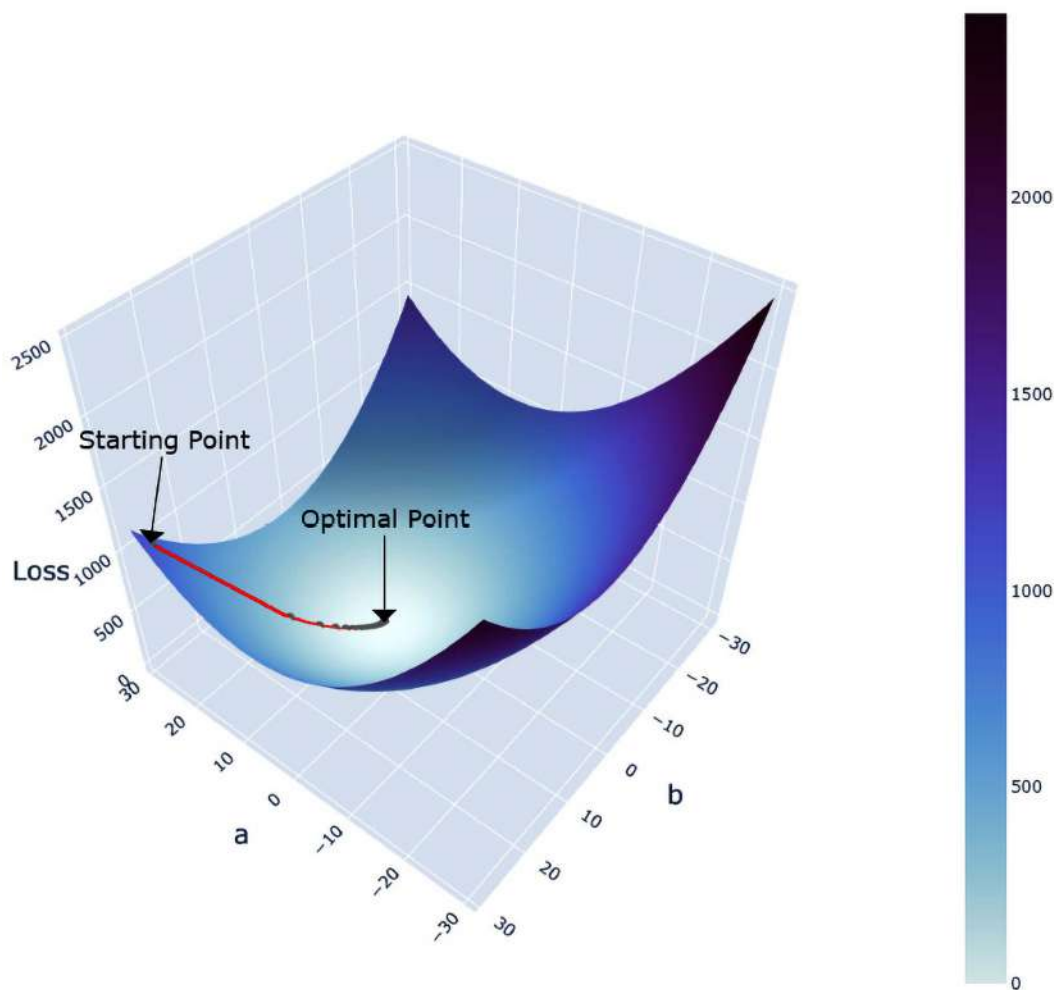


Figure 11.14: Gradient descent optimization on the loss surface

We can see that even though we initialized the parameters far from the actual origin, the optimization algorithm makes a direct path to the optimal point. At each point, the algorithm looks at the gradient of the point, moves in the opposite direction, and eventually converges on the optimum.

When gradient descent is adopted in a learning task, there are a few kinks to be noted. Let's say we have a dataset of N samples. There are three popular variants of gradient descent that are used in learning and each of them has its pros and cons.

Batch gradient descent

We run *all* N samples through the network and average the losses across *all* N instances. Now, we use this loss to calculate the gradient, make a step in the right direction, and repeat.

The pros are as follows:

- The optimization path is direct.
- The optimization path has guaranteed convergence.

The cons are as follows:

- The entire dataset needs to be evaluated for a single step, and that is computationally expensive. The computation per optimization step becomes prohibitively high for huge datasets.
- The time taken per optimization step is high, and hence, the convergence will also be slow.

Stochastic gradient descent (SGD)

In SGD, we randomly sample *one* instance from N samples, calculate the loss and gradients, and then update the parameters.

The pros are as follows:

- Since we only use a single instance to do the optimization step, computation per optimization step is very low.
- The time taken per optimization step is also faster.
- Stochastic sampling also acts as regularization and helps to avoid overfitting.

The cons are as follows:

- The gradient estimates are noisy because we make the step based on just one instance. Therefore, the path toward optimum will be choppy and noisy.
- Just because the time taken per optimization is low, it need not mean convergence is faster. We may not be taking the right step many times because of noisy gradient estimates.

Mini-batch gradient descent

Mini-batch gradient descent is a technique that falls somewhere between the spectrum of batch gradient descent and SGD. In this variant, we have another quality called mini-batch size (or simply batch size), b . In each optimization step, we randomly pick b instances from N samples and calculate gradients on the average loss of all b instances. With $b = N$, we have *batch gradient descent*, and with $b = 1$, we have *stochastic gradient descent*. This is the most popular way neural networks are trained today. By varying the batch size, we can travel between the two variants and manage the pros and cons of each option.

Nothing develops intuition better than a visual playground where we can see the effects of the different components we discussed. *Tensorflow Playground* is an excellent resource (see the link in the *Further reading* section) to do just that. I strongly urge you to head over there and play with the tool, train a few neural networks right in the browser, and see in real time how the learning happens.

Summary

We kicked off a new part of the book with an introduction to deep learning. We started with a bit of history to understand why deep learning is so popular today and we also explored its humble beginnings in perceptron. We understood the composability of deep learning and understood and dissected the different components of deep learning, such as the representation learning block, linear layers, activation functions, and so on. Finally, we rounded off the discussion by looking at how a deep learning system uses gradient descent to learn from data. With that understanding, we are now ready to move on to the next chapter, where we will drive the narrative toward time series models.

References

Following is the list of references used throughout this chapter:

1. Kyoung-Su Oh and Keechul Jung. (2004), *GPU implementation of neural networks*. Pattern Recognition, Volume 37, Issue 6, 2004: <https://doi.org/10.1016/j.patcog.2004.01.013>.
2. Rajat Raina, Anand Madhavan, and Andrew Y. Ng. (2009), *Large-scale deep unsupervised learning using graphics processors*. In Proceedings of the 26th Annual International Conference on Machine Learning (ICML '09): <https://doi.org/10.1145/1553374.1553486>.
3. Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. (2012), *ImageNet Classification with Deep Convolutional Neural Networks*. Commun. ACM 60, 6 (June 2017), 84–90: <https://doi.org/10.1145/3065386>.
4. Neil C. Thompson, Kristjan Greenewald, Keeheon Lee, and Gabriel F. Manso. (2020). *The Computational Limits of Deep Learning*. arXiv:2007.05558v1 [cs.LG]: <https://arxiv.org/abs/2007.05558v1>.
5. Frank Rosenblatt. (1957), *The perceptron – A perceiving and recognizing automaton*, Technical Report 85-460-1, Cornell Aeronautical Laboratory.
6. Charu C. Aggarwal, Alexander Hinneburg, and Daniel A. Keim. (2001). *On the Surprising Behavior of Distance Metrics in High Dimensional Spaces*. In Proceedings of the 8th International Conference on Database Theory (ICDT '01). Springer-Verlag, Berlin, Heidelberg, 420–434: <https://dl.acm.org/doi/10.5555/645504.656414>.
7. Nair, V. and Hinton, G.E. (2010). *Rectified Linear Units Improve Restricted Boltzmann Machines*. ICML: <https://icml.cc/Conferences/2010/papers/432.pdf>.
8. Andrew L. Maas, Awni Y. Hannun, and Andrew Y. Ng. (2013). *Rectifier nonlinearities improve neural network acoustic models*. ICML Workshop on Deep Learning for Audio, Speech, and Language Processing: https://ai.stanford.edu/~amaas/papers/relu_hybrid_icml2013_final.pdf.
9. Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun (2015). *Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification*. 2015 IEEE International Conference on Computer Vision (ICCV), 1026-1034: <https://ieeexplore.ieee.org/document/7410480>.
10. Sara Hooker. (2021). *The hardware lottery*. Commun. ACM, Volume 64: <https://doi.org/10.1145/3467017>.

Further reading

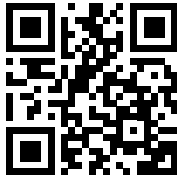
You can check out the following sources if you want to read more about a few topics covered in this chapter:

- *Linear Algebra* course from Gilbert Strang: <https://ocw.mit.edu/resources/res-18-010-a-2020-vision-of-linear-algebra-spring-2020/videos/>
- *Essence of Linear Algebra* from 3Blue1Brown: https://www.youtube.com/playlist?list=PLZHQObOWTQDPD3MizzM2xVFitgF8hE_ab
- *Neural Networks – A Linear Algebra Perspective* by Manu Joseph: <https://deep-and-shallow.com/2022/01/15/neural-networks-a-linear-algebra-perspective/>
- *Deep Learning* – Ian Goodfellow, Yoshua Bengio, Aaron Courville: <https://deep-and-shallow.com/2022/01/15/neural-networks-a-linear-algebra-perspective/>
- *Tensorflow Playground*: <https://playground.tensorflow.org/>

Join our community on Discord

Join our community's Discord space for discussions with authors and other readers:

<https://packt.link/mts>



12

Building Blocks of Deep Learning for Time Series

While we laid the foundations of deep learning in the previous chapter, it was very general. Deep learning is a vast field with applications in all possible domains, but in this book, we will focus on the applications in time series forecasting.

So in this chapter, let's strengthen the foundation by looking at a few building blocks of deep learning that are commonly used in time series forecasting. Even though the global machine learning models perform well in time series problems, some deep learning approaches have also shown good promise. They are a good addition to your toolset due to the flexibility they allow when modeling.

In this chapter, we will cover the following topics:

- Understanding the encoder-decoder paradigm
- Feed-forward networks
- Recurrent neural networks
- Long short-term memory networks
- Gated recurrent unit
- Convolution networks

Technical requirements

You will need to set up the **Anaconda** environment, following the instructions in the *Preface* of the book, to get a working environment with all the libraries and datasets required for the code in this book. Any additional libraries will be installed while running the notebooks.

The associated code for this chapter can be found at <https://github.com/PacktPublishing/Modern-Time-Series-Forecasting-with-Python-2E/tree/main/notebooks/Chapter12>.

Understanding the encoder-decoder paradigm

In *Chapter 5, Time Series Forecasting as Regression*, we saw that machine learning is all about learning a function that maps our inputs to the desired output:

$$y = h(x)$$

where x is the input and y is our desired output.

Adapting this to time series forecasting (using univariate time series forecasting to keep things simple), we can rewrite it as follows:

$$y_t = h(y_{t-1}, y_{t-2}, \dots, y_{t-N})$$

Here, t is the current timestep and N is the total amount of history available at time t .

Deep learning, like any other machine learning approach, is tasked with learning this function, which maps history to the future. In *Chapter 11, Introduction to Deep Learning*, we saw how deep learning learns good features using representation learning, and then it uses the learned features to carry out the task at hand. This understanding can be further refined to the time series perspective by using the encoder-decoder paradigm.

Like everything in research, it is not entirely clear when and who proposed this idea of the encoder-decoder architecture. In 1997, Ramon Neco and Mikel Forcada proposed an architecture for machine translation that had ideas reminiscent of the encoder-decoder paradigm. In 2013, Nal Kalchbrenner and Phil Blunsom proposed an encoder-decoder model for machine translation, although they did not call it that. But it was when Ilya Sutskever et al. (2014) and Cho et al. (2014) proposed two new models for machine translation that worked independently that this idea took off. Cho et al. called it the encoder-decoder architecture, while Sutskever et al. called it the Seq2Seq architecture. The key innovation it drove was the ability to model variable-length inputs and outputs in an end-to-end fashion.



Reference check:

The research papers by Ramon Neco et al., Nal Kalchbrenner et al., Cho et al., and Ilya Sutskever et al. are cited in the *References* section as 1, 2, 3, and 4, respectively.

The idea is very straightforward, but before we get into that, we need to have a high-level understanding of latent spaces and feature/input spaces.

The **feature space**, or the **input space**, is the vector space where your data resides. If the data has 10 dimensions, then the input space is the 10-dimensional vector space. Latent space is an abstract vector space that encodes a meaningful internal representation of the feature space. To understand this, we can think about how we, as humans, recognize a tiger. We do not remember every minute detail of a tiger; we just have a general idea of what a tiger looks like and its prominent features, such as its stripes. It is a compressed understanding of this concept that helps our brains process and recognize a tiger faster.

In the machine learning domain, there are techniques like **Principal Component Analysis (PCA)** that do similar transformations to a latent space, preserving the essential features of the input data. With this intuition, rereading the definition may bring a bit more clarity to the concept.

Now that we have an idea about latent spaces, let's see what an encoder-decoder architecture does.

An encoder-decoder architecture has two main parts—an encoder and a decoder:

- **Encoder:** The encoder takes in the input vector, x , and encodes it into a latent space. This encoded representation is called the latent vector, z .
- **Decoder:** The decoder takes in the latent vector, z , and decodes it into the kind of output we need ($\hat{y}$).

The following diagram shows the encoder-decoder setup visually:

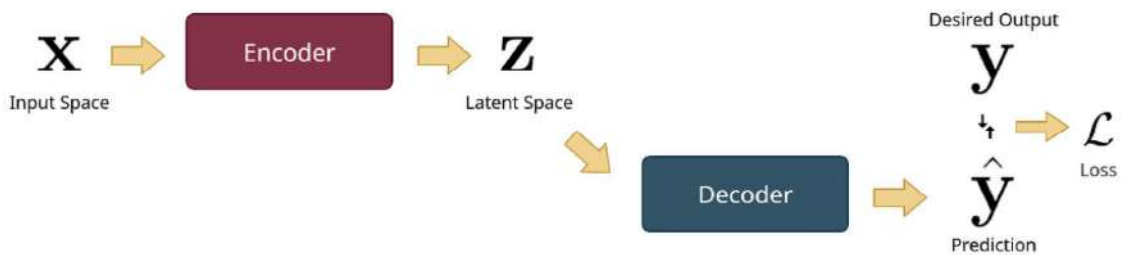


Figure 12.1: The encoder-decoder architecture

In the context of time series forecasting, the encoder consumes the history and retains the information that is required for the decoder to generate the forecast. As we learned previously, time series forecasting can be written as follows:

$$y_t = h(y_{t-1}, y_{t-2}, \dots, y_{t-N})$$

Now, using the encoder-decoder paradigm, we can rewrite it as follows:

$$z_t = h(y_{t-1}, y_{t-2}, \dots, y_{t-N})$$

$$y_t = g(z_t)$$

Here, h is the encoder and g is the decoder.

Each encoder and decoder can be some special architecture suited for time series forecasting. Let's look at a few common components that are used in the encoder-decoder paradigm.

Feed-forward networks

Feed-forward networks (FFNs) or **fully connected networks** are the most basic architecture a neural network can take. We discussed perceptrons in *Chapter 11, Introduction to Deep Learning*. If we stack multiple perceptrons (both linear units and non-linear activations) and create a network of such units, we get what we call an FFN. The following diagram will help us understand this:

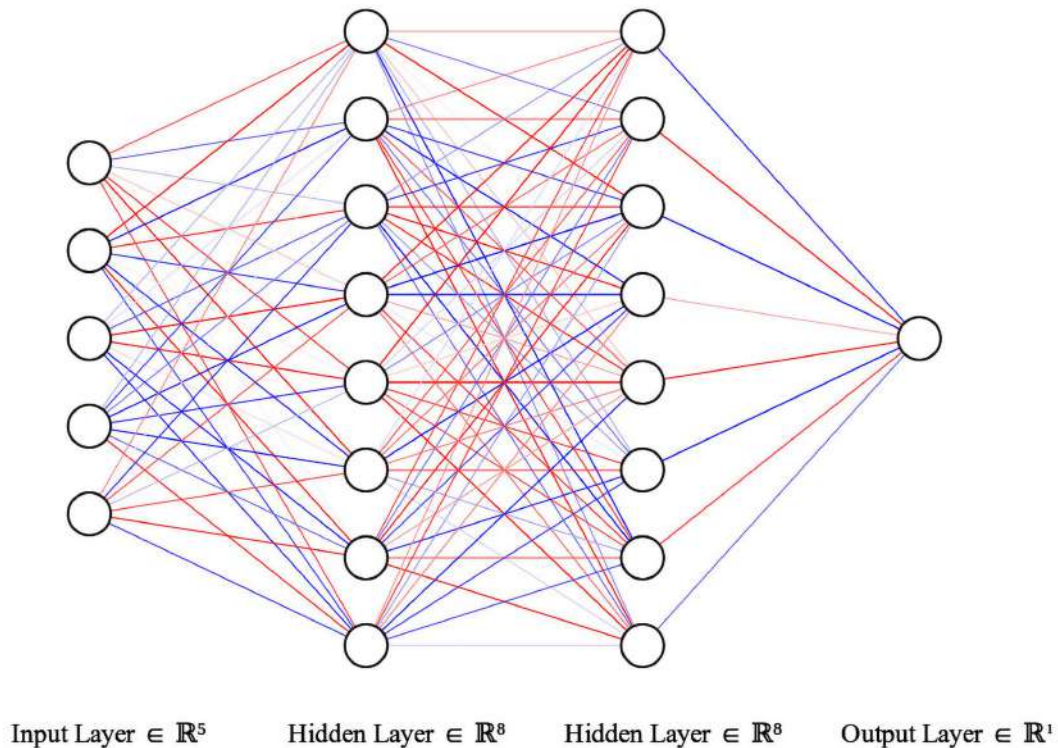


Figure 12.2: A Feed Forward Network (FFN)

An FFN takes a fixed-size input vector and passes it through a series of computational layers leading up to the desired output. This architecture is called feed-forward because the information is fed forward through the network. This is also called a **fully connected network** because every unit in a layer is connected to every unit in the previous layer and every unit in the next layer.

The first layer is called the input layer, and this is equal to the dimension of the input. The last layer is called the output layer, which is defined as per our desired output. If we need a single output, we will need 1 unit, while if we need 10 outputs, we will need 10 units. All the layers in between are called **hidden layers**. Two hyperparameters define the structure of the network—the number of hidden layers and the number of units in each layer. For instance, in Figure 12.2, we have a network with two hidden layers and eight units per layer.

In the time series forecasting context, an FFN can be used as an encoder as well as a decoder. As an encoder, we can use an FFN just like we used machine learning models in Chapter 5, *Time Series Forecasting as Regression*. We embed time and convert a time series problem into a regression problem before feeding it into the FFN. As a decoder, we use it on the latent vector (the output from the encoder) to get to the output (this is the most common usage of an FFN in time series forecasting).



Additional reading:

We are going to be using PyTorch throughout this book to work with deep learning. If you are not comfortable with PyTorch, don't worry—I'll try and explain the concepts when necessary. To get a head start, you can go through the `01-PyTorch_Basics.ipynb` notebook in Chapter12, where we have explored the basic functionalities of tensors and trained a very small neural network from scratch using PyTorch. I also suggest heading over to the *Further reading* section at the end of this chapter, where you'll find a few resources to learn PyTorch.

Now, let's put on our practical hats and see some of these in action. PyTorch is an open source deep learning framework developed primarily by the **Facebook AI Research (FAIR) Lab**. Although it is a library that can manipulate **tensors** (which are n -dimensional matrices) and accelerate such manipulations with a GPU, a large part of the use case for such a library is in building and training deep learning systems. Because of that, PyTorch provides a lot of ready-to-use components that we can use to build a deep learning system. Let's see how we can use PyTorch for an FFN.



Notebook alert:

To follow along with the complete code, use the `02-Building_Blocks.ipynb` notebook in the Chapter12 folder and the code in the `src` folder.

As we learned earlier in the section, an FFN is a network of linear and non-linear units arranged in a network. A linear operation consists of multiplying the input vector, X , with a weight matrix, W , and adding a bias term, b . This operation, $WX + b$, is encapsulated in a `Linear` class in the `nn` module of the **PyTorch** library. We can import this from the library using `torch.nn import Linear`. But usually, we must import the whole `nn` module because we would be using a lot of components from that module. For non-linearity, let's use **ReLU** (as introduced in *Chapter 11, Introduction to Deep Learning*), which is also a class in the `nn` module.

Before moving on, let's create a random walk time series whose length is 20:

```
N = 20
df = pd.DataFrame({
    "date": pd.date_range(periods=N, start="2021-04-12", freq="D"),
    "ts": np.random.randn(N)
})
```

We can use this tensor directly in the FFN, but usually, we use a sliding window technique to split the tensor and train the networks. We do this for multiple reasons:

- We can see this as a data augmentation technique that creates a greater number of samples as opposed to using the entire sequence just once.
- It helps us reduce and restrict computation by limiting the calculation to a fixed window.

Let's do that now:

```
ts = torch.from_numpy(df.ts.values).float()
window = 15
# Creating windows of 15 over the dataset
ts_dataset = ts.unfold(0, size=window, step=1)
```

Now, we have a tensor, `ts_dataset`, whose size is 6×15 (this can create 6 samples of 15 input features each when we move the sliding window across the length of the series). For a standard FFN, the input shape is specified as *batch size $\times$ input features*. So 6 becomes our batch size and 15 becomes the input feature size.

Now, let's define the layers in the FFN. For this exercise, let's assume the network's structure is as follows:

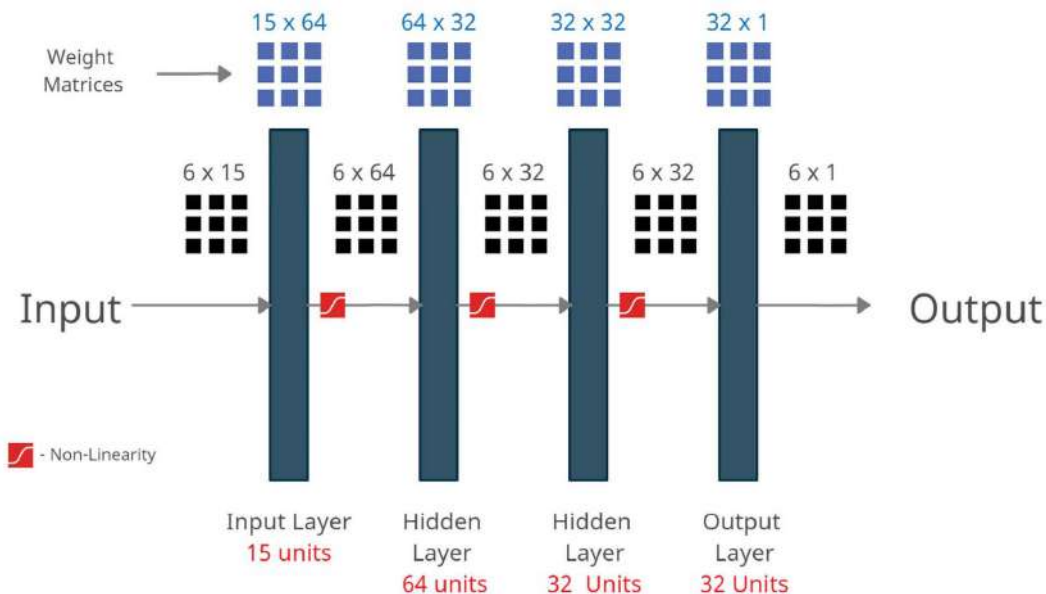


Figure 12.3: FFNs—a matrix multiplication perspective

The input data (6×15) will be passed through these layers one by one. Here, we can see how the tensor dimensions change as they flow through the network. Each of the linear layers is essentially a matrix multiplication that converts the input into the output of a specified dimension. After each linear transformation, we stack a non-linear activation function in there. These alternative linear and non-linear modules are what give the neural network the expressive power it has. The linear layers are an affine transformation of the vector space (rotation, translation, and so on), and the non-linearity *squashes* the vector space. Together, they can morph the input space so that it's useful for the task at hand. Now, let's see how we can code this in PyTorch.

We are going to use a handy module from PyTorch called **Sequential**, which allows us to stack different sub-components together and use them with ease:

```
# The FFN we define would have this architecture
# window(windowed input) >> 64 (hidden Layer 1) >> 32 (hidden Layer 2) >> 32
# (hidden Layer 2) >> 1 (output)
ffn = nn.Sequential(
    nn.Linear(in_features=window,out_features=64), # (batch-size x window) -->
    (batch-size x 64)
    nn.ReLU(),
    nn.Linear(in_features=64,out_features=32), # (batch-size x 64) --> (batch-
    size x 32)
    nn.ReLU(),
    nn.Linear(in_features=32,out_features=32), # (batch-size x 32) --> (batch-
    size x 32)
    nn.ReLU(),
    nn.Linear(in_features=32,out_features=1), # (batch-size x 32) --> (batch-
    size x 1)
)
```

Now that we have defined the FFN, let's see how we can use it:

```
ffn(ts_dataset)
# or more explicitly
ffn.forward(ts_dataset)
```

This will return a tensor whose shape is based on *batch size x output units*. We can have any number of output units, not just one. Therefore, when using an encoder, we can have an arbitrary dimension for the latent vector. Then, when we use it as a decoder, we can have the output units equal the number of timesteps we forecast.



Sneak peak:

We have not seen multi-step forecasting until now because it will be covered in more detail in *Chapter 18, Multi-Step Forecasting*. But for now, just understand that there are cases where we will need to forecast multiple timesteps into the future. The classical statistical models do this out of the box. But for machine learning and deep learning, we need to design systems that can do that. Fortunately, there are a few different techniques to do so, which will be covered later in this chapter.

FFNs are designed for non-temporal data. We can use FFNs by embedding our data temporally and then passing that to the network. Also, the computational cost in an FFN is directly proportional to the memory we use in the embedding (the number of previous timesteps we include as features). We will also not be able to handle variable-length sequences in this setting.

Now, let's look at another common architecture that is specifically designed for temporal data.

Recurrent neural networks

Recurrent neural networks (RNNs) are a family of neural networks specifically designed to handle sequential data. They were first proposed by *Rumelhart et al.* (1986) in their seminal work, *Learning Representations by Back-Propagating Errors*. The work borrows ideas such as parameter sharing and recurrence from previous work in statistics and machine learning, resulting in a neural network architecture that helps overcome many of the disadvantages that FFNs have when processing sequential data.

RNN architecture

Parameter sharing is when we use the same set of parameters for different parts of a model. Apart from a regularization effect (restricting the model to using the same set of weights for multiple tasks, which regularizes the model by constraining the search space while optimizing the model), parameter sharing enables us to extend and apply the model to examples of different forms. RNNs can scale to much longer sequences because of this. In an FFN, each timestep (each feature) has a fixed weight, and even if the motif we are looking for shifts by just one timestep, the network may not capture it correctly. In an RNN enabled by parameter sharing, they are captured in a much better way.

In a sentence (which is also a sequence), we may want a model to recognize that “*Tomorrow I will go to the bank*” and “*I will go to the bank tomorrow*” are the same thing. An FFN can't do this, but an RNN will be able to because it uses the same parameters at all positions and will be able to identify the motif “*I will go to the bank*” wherever it occurs. Intuitively, we can think of RNNs as applying the same FFN at each time window but enhanced with some kind of memory to store relevant information for the task at hand.

Let's visualize how an RNN processes inputs:

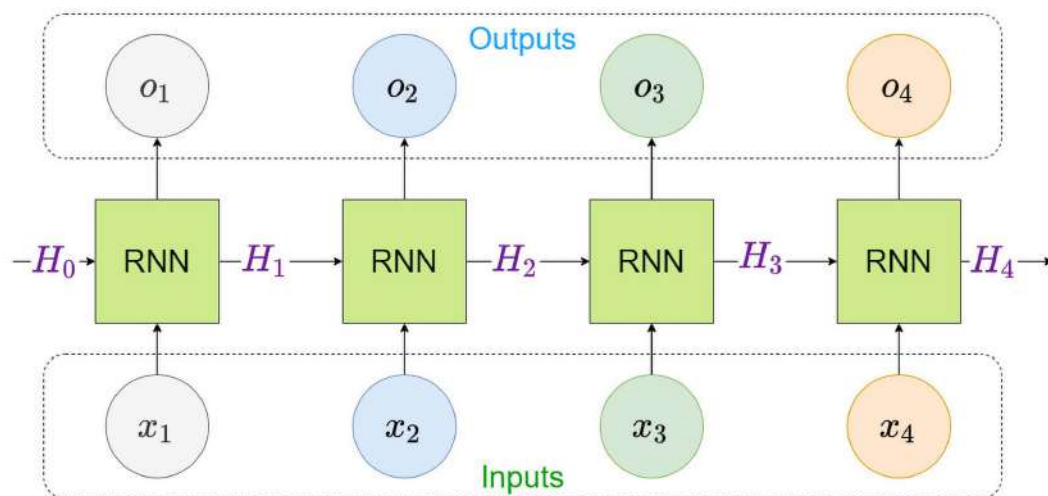


Figure 12.4: How an RNN processes input sequences

Let's assume we are talking about a sequence with four elements in it, x_1 to x_4 . Any RNN block (let's consider it as a black box for now) consumes input and a hidden state (memory), producing an output. In the beginning, there is no memory, so we start with an initial memory (H_0), which is typically an array filled with zeroes. Now, the RNN block takes in the first input (x_1) along with the initial hidden state (H_0), producing an output (o_1) and a hidden state (H_1).

To process the second element in the sequence, *the same RNN* block takes in the hidden state from the previous timestep (H_1) and the input at the current timestep (x_2), producing the output at the second timestep (o_2) and a new hidden state (H_2). This process continues until we reach the end of the sequence. After processing the entire sequence, we will have all the outputs at each timestep (o_1 through o_4) and the final hidden state (H_4).

These outputs and the hidden state will have encoded the information contained in the sequence and can be used for further processing, such as to predict the next step using a decoder. The RNN block can also be used as a decoder that takes in the encoded representation and produces the outputs. Because of this flexibility, the RNN blocks can be arranged to suit a wide variety of input and output combinations, such as the following:

- Many-to-one, where we have many inputs and a single output—for instance, single-step forecasting or time series classification
- Many-to-many, where we have many inputs and many outputs—for instance, multi-step forecasting

Now, let's look at what happens inside an RNN.

Let the input to the RNN at time t be x_t and the hidden state from the previous timestep be H_{t-1} . The updated equations are as follows:

$$A_t = W \cdot H_{t-1} + U \cdot x_t + b_1$$

$$H_t = \tanh(A_t)$$

$$o_t = V \cdot H_t + b_2$$

Here, U , V , and W are learnable weight matrices, and b_1 and b_2 are two learnable bias vectors. U , V , and W can be easily remembered as *input-to-hidden*, *hidden-to-output*, and *hidden-to-hidden* matrices based on the kind of transformation they perform, respectively. Intuitively, we can think of the operation that the RNN does as a kind of learning and forgetting information as it sees fit. The *tanh* activation, as we saw in *Chapter 11, Introduction to Deep Learning*, produces a value between -1 and 1, which acts analogous to forgetting and remembering. So the RNN transforms the input into a latent dimension, uses the *tanh* activation to decide what information from the current timestep and previous memory to keep and forget, and uses this new memory to generate an output.

In standard backpropagation, we backpropagate gradients from one unit to another. But in recurrent nets, we have a special situation where we have to backpropagate the gradients within a single unit, but through time or the different timesteps. A special case of backpropagation, called **Back Propagation Through Time (BPTT)**, has been developed for RNNs.

Thankfully, all the major deep learning frameworks are capable of doing this without any problems. For a more detailed understanding and the mathematical foundations of BPTT, please refer to the *Further reading* section.

PyTorch has made RNNs available as ready-to-use modules—all you need to do is import one of the modules from the library and start using it. But before we do that, we need to understand a few more concepts.

The first concept we will look at is the possibility of *stacking multiple layers* of RNNs on top of each other so that the outputs at each timestep become the input to the RNN in the next layer. Each layer will have a hidden state or memory. This enables hierarchical feature learning, which is one of the bedrocks of successful deep learning today.

Another concept is *bidirectional* RNNs, introduced by Schuster and Paliwal in 1997. Bidirectional RNNs are very similar to RNNs. In a vanilla RNN, we process the inputs sequentially from start to end (forward). However, a bidirectional RNN uses one set of input-to-hidden and hidden-to-hidden weights to process the inputs from start to end, and then it uses another set to process the inputs in reverse (end to start) and concatenate the hidden states from both directions. It is on this concatenated hidden state that we apply the output equation.



Reference check:

The research papers by Rumelhart, et al and Schuster and Paliwal are cited in the *References* section as 5 and 6, respectively.

RNN in PyTorch

Now, let's understand the PyTorch implementation of RNN. As with the **Linear** module, the **RNN** module is also available from `torch.nn`. Let's look at the different parameters that the implementation provides while initializing:

- **input_size**: The number of expected features in the input. If we use just the history of the time series, this is 1. However, when we use history along with some other features, this is >1.
- **hidden_size**: The dimension of the hidden state. This defines the size of the input-to-hidden and hidden-to-hidden matrices.
- **num_layers**: This is the number of RNNs that will be stacked on top of each other. The default is 1.
- **nonlinearity**: The non-linearity to use. Although tanh is the originally proposed non-linearity, PyTorch also allows us to use ReLU (`relu`). The default is `tanh`.
- **bias**: This parameter decides whether or not to add bias to the update equations we discussed earlier. If the parameter is **False**, there will be no bias. The default is **True**.
- **batch_first**: There are two input data configurations that the RNN cell can use—we can have the input as (*batch size, sequence length, number of features*) or (*sequence length, batch size, number of features*). `batch_first = True` selects the former as the expected input dimensions. The default is **False**.

- **dropout**: This parameter, if non-zero, uses a dropout layer on the outputs of each RNN layer except the last. Dropout is a popular regularization technique where randomly selected neurons are ignored during training (the *Further reading* section contains a link to the paper that proposed this). The dropout probability will be equal to **dropout**. The default is 0.
- **bidirectional**: This parameter enables a bidirectional RNN. If **True**, a bidirectional RNN is used. The default is **False**.

To continue applying the model to the same synthetic data we generated earlier in this chapter, let's initialize the RNN model, as follows:

```
rnn = nn.RNN(
    input_size=1,
    hidden_size=32,
    num_layers=1,
    batch_first=True,
    dropout=0,
    bidirectional=False,
)
```

Now, let's look at the inputs and outputs that are expected from an RNN cell.

As opposed to the **Linear** layer we saw earlier, the RNN cell takes in *two inputs*—the input sequence and the hidden state vector. The input sequence can be either (*batch size, sequence length, number of features*) or (*sequence length, batch size, number of features*), depending on whether we have set `batch_first=True`. The hidden state is a tensor whose size is (*D *number of layers, batch size, hidden size*), where $D = 1$ for `bidirectional=False` and $D = 2$ for `bidirectional=True`. The hidden state is an optional input and will default to zero tensors if left blank.

There are two outputs of the RNN cell: an output and a hidden state. The output can be either (*batch size, sequence length, D *hidden size*) or (*sequence length, batch size, D *hidden size*), depending on `batch_first`. The hidden state has the dimension of (*D *number of layers, batch size, hidden size*). Here, $D = 1$ or 2 is based on the **bidirectional** parameter.

So let's run our sequence through an RNN and look at the inputs and outputs (for more detailed steps, refer to the accompanying notebook):

```
#input dim: torch.Size([6, 15, 1])
# batch size = 6, sequence length = 15 and number of features = 1, batch_first
= True
output, hidden_states = rnn(rnn_input)
# output.shape -> torch.Size([6, 15, 32])
# hidden_states.shape -> torch.Size([1, 6, 32]))
```



Although we saw that the RNN cell contains the output as well as the hidden state, we also know that the output is just an affine transformation of the hidden state. Therefore, to provide flexibility to the users, PyTorch only implements the update equations regarding the hidden states in the module. There are cases where we have no use for the outputs at each timestep (such as in a many-to-one scenario) and we can save computation if we do not do the output update at each step. Therefore, output from the PyTorch RNN is just the hidden states at each timestep, and `hidden_states` is the latest hidden state.

We can verify this by checking whether the hidden-state tensor is equal to the last-output tensor:

```
torch.equal(hidden_states[0], output[:, -1]) # -> True
```

To make this clearer, let's look at it visually:

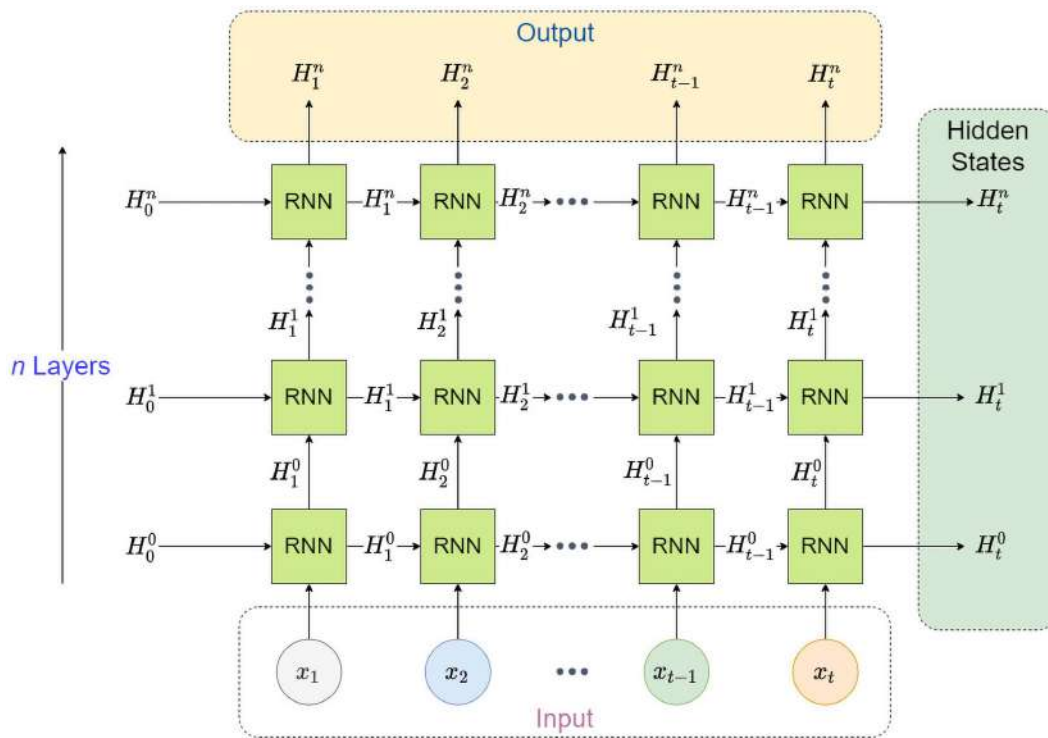


Figure 12.5: PyTorch implementation of stacked RNNs

The hidden states at each timestep are used as input for the subsequent layer of RNNs and the hidden states of the last layer of RNNs are collected as the output. But each layer has a hidden state (that's not shared with the others), and the PyTorch RNN collects the last hidden state from each layer and gives us that as well.

Now, it is up to us to decide how to use these outputs. For instance, in a one-step-ahead forecast, we can use the output hidden states and stack a few linear layers on top of it to get the next timestep prediction. Alternatively, we can use the hidden states to transfer memory into another RNN as a decoder and generate predictions for multiple timesteps. There are many more ways we can use this output and PyTorch gives us that flexibility.

RNNs, while very effective in modeling sequences, have one big flaw. Because of BPTT, the number of units through which you need to backpropagate increases drastically with the length of the sequence to be used for training. When we have to backpropagate through such a long computational graph, we will encounter **vanishing** or **exploding gradients**. This is when the gradient, as it is backpropagated through the network, either shrinks to zero or explodes to a very high number. The former makes the network stop learning, while the latter makes the learning unstable.

We can think of what's happening as akin to what happens when we multiply a scalar number repeatedly by itself. If the number is less than one, with every subsequent multiplication, the number becomes smaller and smaller until it is practically zero. If the number is greater than one, then the number becomes larger and larger at an exponential scale. This was discovered, independently, by Hochreiter in his diploma thesis (1991) and Yoshua Bengio et al. in two papers published in 1993 and 1994, respectively. Over the years, many tweaks to the model and training process have been proposed to tackle this disadvantage. Nowadays, vanilla RNNs are hardly used in practice and have been replaced almost completely by their newer cousins.



Reference check:

The references for Hochreiter (1991) and Bengio et al. (1993, 1994) are cited in the *References* section as 7, 8, and 9, respectively.

Now, let's look at two key improvements that have been made to the RNN architecture that have shown good performance, gaining popularity in the machine learning community.

Long short-term memory (LSTM) networks

Hochreiter and Schmidhuber proposed a modification of the classical RNNs in 1997—LSTM networks. It aimed to resolve the vanishing and exploding gradients in vanilla RNNs. The design of the LSTM was inspired by the logic gates of a computer. It introduces a new component, called a **memory cell**, which serves as long-term memory and is used in addition to the hidden-state memory of classical RNNs. In an LSTM, multiple gates are tasked with reading, adding, and forgetting information from these memory cells. This memory cell acts as a *gradient highway*, allowing the gateways to pass relatively unhindered through a network. This is the key innovation that avoided vanishing gradients in RNNs.

LSTM architecture

Let's imagine that the input to the LSTM at time t is x_t , and the hidden state from the previous timestep is H_{t-1} . Now, there are three gates that process information. Each gate is nothing but two learnable weight matrices (one for the input and one for the hidden state from the last step) and a bias term that is multiplied/added to the input and hidden state, and finally, it is passed through a sigmoid activation.

The output of these gates will be a real number between 0 and 1. Let's look at each of these gates in detail:

- **Input gate:** The function of this gate is to decide how much information to read from the current input and previous hidden state. The update equation for this is:

$$I_t = \sigma(W_{xi}) \cdot x_i + W_{hi} \cdot H_{t-1} + b_i$$

- **Forget gate:** The forget gate decides how much information to forget from long-term memory. The updated equation for this is:

$$F_t = \sigma(W_{xf}) \cdot x_i + W_{hf} \cdot H_{t-1} + b_f$$

- **Output gate:** The output gate decides how much of the current cell state should be used to create the current hidden state, which is the output of the cell. The update equation for this is:

$$O_t = \sigma(W_{xo}) \cdot x_i + W_{ho} \cdot H_{t-1} + b_o$$

Here, W_{xi} , W_{xf} , W_{xo} , W_{hi} , W_{hf} , and W_{ho} are learnable weight parameters, and b_i , b_f , and b_o are learnable bias parameters.

Now, we can introduce a new long-term memory (cell state), C_t . The three gates mentioned previously serve to update and forget from this memory. If the cell state from the previous timestep is C_{t-1} , then the LSTM cell calculates a candidate cell state, $\tilde{C}_t$, using another gate, but this time with $\tanh$ activation:

$$\tilde{C}_t = \tanh(W_{xc} \cdot x_t + W_{hc} \cdot H_{t-1} + b_c)$$

Here, W_{xc} , and W_{hc} are learnable weight parameters and b_c is the learnable bias parameter.

Now, let's look at the key update equation, which updates the cell state or long-term memory of the cell:

$$C_t = F_t \odot C_{t-1} + I_t \odot \tilde{C}_t$$

Here, $\odot$ is elementwise multiplication. We use the forget gate to decide how much information from the previous timestep to carry forward, and we use the input gate to decide how much of the current candidate cell state will be written into long-term memory.

Last but not least, we use the newly created current cell state and the output gate to decide how much information to pass on to the predictor through the current hidden state:

$$H_t = O_t \odot \tanh(C_t)$$

A visual representation of this process can be seen in *Figure 12.6*.

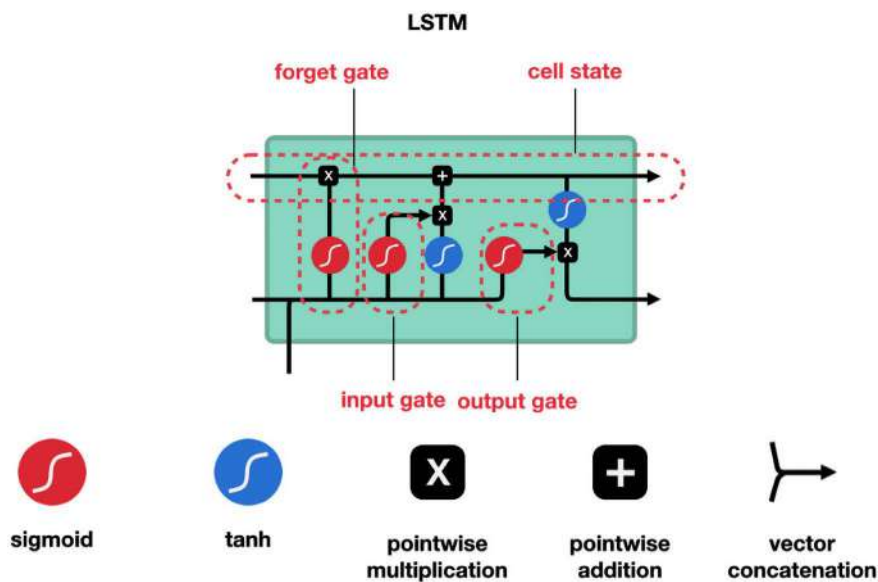


Figure 12.6: A gating diagram of LSTM

LSTM in PyTorch

Now, let's understand the PyTorch implementation of LSTM. It is very similar to the RNN implementation we saw earlier, but it has one key difference: the parameters to initialize the class are pretty much the same. The API for this can be found at <https://pytorch.org/docs/stable/generated/torch.nn.LSTM.html#torch.nn.LSTM>. The key difference here is how the hidden states are used. While the RNN has a single tensor as a hidden state, the LSTM expects a **tuple** of tensors of the same dimensions: (**hidden state**, **cell state**).

LSTMs, just like RNNs, have stacked and bidirectional variants, and PyTorch handles them in the same way.

Now, let's initialize some LSTM modules and use the synthetic data we have been using to see them in action:

```
lstm = nn.LSTM(
    input_size=1,
    hidden_size=32,
    num_layers=5,
    batch_first=True,
    dropout=0,
    # bidirectional=True,
)
output, (hidden_states, cell_states) = lstm(rnn_input)
output.shape # -> [6, 15, 32]
hidden_states.shape # -> [5, 6, 32]
cell_states.shape # -> [5, 6, 32]
```

Now, let's look at another modification that's been made to vanilla RNNs that has resolved the vanishing and exploding gradient problems.

Gated recurrent unit (GRU)

In 2014, Cho et al. proposed another variant of the RNN that has a much simpler structure than an LSTM, called a GRU. The intuition behind this is similar to when we use a bunch of gates to regulate the information that flows through the cell, but a GRU eliminates the long-term memory component and uses just the hidden state to propagate information. So instead of the memory cell becoming the *gradient highway*, the hidden state itself becomes the “gradient highway.” In keeping with the same notation convention we used in the previous section, let's look at the updated equations for a GRU.

GRU architecture

While we had three gates in an LSTM, we only have two in a GRU:

- **Reset gate:** This gate decides how much of the previous hidden state will be considered as the candidate's hidden state of the current timestep. The equation for this is:

$$R_t = \sigma(W_{xr} \cdot x_t + W_{hr} \cdot H_{t-1} + b_r)$$

- **Update gate:** The update gate decides how much of the previous hidden state should be carried forward and how much of the current candidate's hidden state will be written into the hidden state. The equation for this is:

$$U_t = \sigma(W_{xu} \cdot x_t + W_{hu} \cdot H_{t-1} + b_u)$$

Here W_{xr} , W_{xu} , W_{hr} , and W_{hu} are learnable weight parameters, and b_r and b_u are learnable bias parameters.

Now, we can calculate the candidate's hidden state ($\tilde{H}_t$), as follows:

$$\tilde{H}_t = \tanh(W_{xh} \cdot x_t + W_{hh} \cdot R_t \odot H_{t-1} + b_h)$$

Here, W_{xh} and W_{hh} are learnable weight parameters, and b_h is the learnable bias parameter. Here, we use the reset gate to throttle the information flow from the previous hidden state to the current candidate's hidden state.

Finally, the current hidden state (the output that goes to a predictor) is computed using the following equation:

$$H_t = U_t \odot H_{t-1} + (1 - U_t) \odot \tilde{H}_t$$

We use the update gate to decide how much from the previous hidden state and how much from the current candidate will be passed to the next timestep or predictor.



Reference check:

The research papers for LSTM and GRUs are cited in the *References* section as 10 and 11, respectively.

A visual representation of this process can be found in *Figure 12.7*:

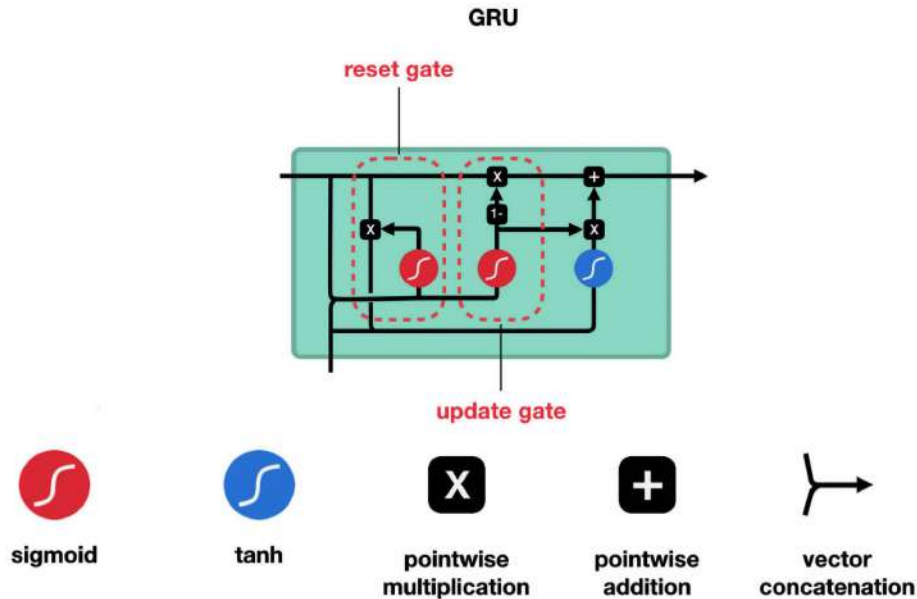


Figure 12.7: A gating diagram of GRU

GRU in PyTorch

Now, let's understand the PyTorch implementation of the GRU. The APIs, inputs, and outputs are the same as with an RNN. The API for this can be referenced here: <https://pytorch.org/docs/stable/generated/torch.nn.GRU.html#torch.nn.GRU>. The key difference is the internal workings of the modules, where the GRU update equations are used instead of the standard RNN ones.

Now, let's initialize a GRU module and use the synthetic data we have been using to see it in action:

```
Gru = nn.GRU(
    input_size=1,
    hidden_size=32,
    num_layers=5,
    batch_first=True,
    dropout=0,
    # bidirectional=True,
)
output, hidden_states = gru(rnn_input)
output.shape # -> [6, 15, 32]
hidden_states.shape # -> [5, 6, 32]
```

Now, let's look at another major component that can be used for sequential data.

Convolution networks

Convolution networks, also called **convolutional neural networks** (CNNs), are like neural networks for processing data in the form of a grid. This grid can be 2D (such as an image), 1D (such as a time series), 3D (such as data from LIDAR sensors), and so on. Although this book is about time series and, typically, 1D convolutions are used in time series forecasting, it's easier to understand convolutions in the 2D context (an image and so on) and then move back to a single-dimensional grid for time series.

The basic idea behind CNNs is inspired by how human vision works. In 1979, Fukushima proposed Neocognitron (Reference 12). It was a one-of-a-kind architecture that was directly inspired by how human vision works. But CNNs came into existence as we know them today in 1989 when Yann Le Cun used backpropagation to learn such a network, proving it by getting state-of-the-art results in handwritten digit recognition (Reference 13). In 2012, when AlexNet (a CNN architecture for image recognition) won the annual challenge of image recognition called ImageNet, that too by a large margin between it and competing non-deep learning approaches, the interest and research in CNNs peaked. People soon figured out that, apart from images, CNNs are effective with sequences, such as language and time series data.

Convolution

At the heart of CNNs is a mathematical operation called **convolution**. The mathematical interpretation of a convolution operation is beyond the scope of this book, but there are a couple of links in the *Further reading* section if you want to learn more. For our purposes, we'll develop an intuitive understanding of the convolution operation.

Since CNNs rose to popularity using image data, let's start by discussing the image domain and then move on to the sequence domain.

Any image (for simplicity, let's assume it's grayscale) can be considered as a grid of pixel values, each value denoting how bright a point is, with 1 being pure white and 0 being pure black. Before we start discussing convolution, let's understand what a **kernel** is. For now, let's think of a kernel as a 2D matrix with some values in it. Typically, the kernel's size is smaller than the size of the image we are using. Since the kernel is smaller than the image, we can "fit" the kernel inside the image. Let's start with the kernel aligned with the top-left edge. With the kernel at the current position, there is a set of values in the image that this kernel is superpositioned over. We can perform elementwise multiplication between this subset of the image and the kernel, and then we add up all the elements into a single scalar. Now, we can repeat this process by "sliding" the kernel into all positions in the image. For instance, the following shows a sample image input whose size is 4x4 and how a convolution operation is carried out on that, using a kernel whose size is 2x2:

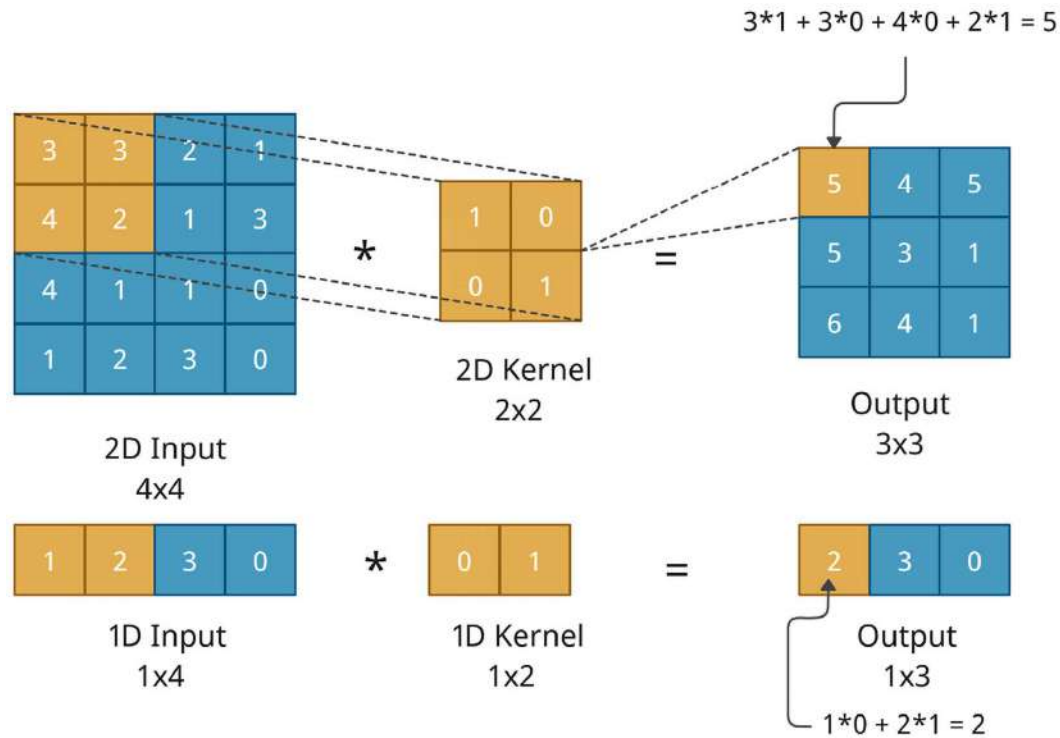


Figure 12.8: A convolution operation on 2D and 1D inputs

So if we place the 2x2 kernel at the top-left position and perform the element-wise multiplication and the summation, we get the top-left item in the 3x3 output. If we slide the kernel by one position to the right, we get the next element in the top row of the output, and so on. Similarly, if we slide the kernel by one position down, we get the second element in the first column in the output.

While this is interesting, we want to understand convolutions from a time series perspective. To do so, let's shift our paradigm to 1D convolutions—convolutions performed on 1D data such as a sequence. In the preceding diagram, we can also see an example of a 1D convolution where we take the 1D kernel and slide it across the sequence to get an output of 1x3.

Although we have set the kernel weights so that they're convenient to understand and compute, in practice, these weights are learned by the network from data. If we set the kernel size as n and all the kernel weights as $1/n$, what would such a convolution give us? This is something we covered in *Chapter 6, Feature Engineering for Time Series Forecasting*. Yes, they result in the rolling means with a window of n . Remember, we learned this as a feature engineering technique for machine learning models. So 1D convolutions can be thought of as a more powerful feature generator, where the features are learned from data. With different weights on the kernels, we will extract different features. It is this knowledge that we should hold onto while learning about CNNs for time series data.

Padding, stride, and dilations

Now that we understand what a convolution operation is, we need to understand a few more terms, such as **padding**, **stride**, and **dilations**.

Before we start discussing these terms, let's look at an equation that gives the output dimensions (O) of a convolutional layer, given the input dimensions (L), kernel size (k), padding size (p_l for left padding and p_r for right padding), stride (s), and dilation (d):

$$O = \frac{L + p_l + p_r - d \times (k - 1) - 1}{s} + 1$$

The default values (padding, strides, and dilations are special cases of a convolution process) of these terms are $p_r, p_l = 0, s = 1, d = 1$. Don't worry if you don't understand the formula or the terms in it—just keep the default values in mind so that when we understand each term, we can negate the others.

In *Figure 12.8*, we saw that the convolution operation always reduces the size of the input. So in the default case, the formula becomes $O = L - (k - 1)$. This is because the earliest position we can place the kernel in the sequence is from $t = 0$ to $t = k$. Then, by convolving through the sequence, we get $L - (k - 1)$ terms in the output. Padding is when we add some values to the beginning or the end of the sequence. The value we use for padding is dependent on the problem. Typically, we choose zero as a padding value. So padding a sequence essentially increases the size of the input. Therefore, in the preceding formula, we can think of $L + p_l + p_r$ as the effective length of the sequence after padding.

The next two terms (stride and dilation) are closely related to the **receptive field** of the convolutional layer. The receptive field of a convolutional layer is the region in the input space that influences the feature that's generated by the convolutional layer. In other words, it is the size of the window of input over which we have performed the convolution operation. For a single convolutional layer (with default settings), this is pretty much the kernel size. For multi-layered CNNs, this calculation becomes a bit more complicated because of the hierarchical structure (the *Further reading* section contains a link to a paper by Arujo et al. who derived a formula to calculate the receptive field of a CNN). But generally, increasing the receptive field of a CNN is associated with an increase in the accuracy of the CNN. For computer vision, Araujo et al. noted the following:



"We observe a logarithmic relationship between classification accuracy and receptive field size, which suggests that large receptive fields are necessary for high-level recognition tasks, but with diminishing rewards."

In time series, this is important because if the receptive field of a CNN is smaller than the long-term dependency, such as the seasonality, that we want to capture, then the network will fail to do so. Making the CNN deeper by stacking more convolutional layers on top of the others is one way to increase the receptive field of a network. However, there are a few ways to increase the receptive field of a single convolutional layer. Strides and dilations are two such ways:

- **Stride:** Earlier, when we talked about *sliding* the kernel over the sequence, we mentioned that we move the kernel by one position at a time. This is called the stride of the convolutional layer, and there is no necessity that the stride should be 1. If we set the stride to 2, the convolution operation would be performed by skipping a position in between, as shown in *Figure 12.9*. This can make each layer in the convolutional network look at a larger slice of history, thereby increasing the receptive field.
- **Dilation:** Another way we can tweak the basic convolutional layer is by dilating the input connections. In the standard convolutional layer with a kernel size of 3, we apply the kernel to three consecutive elements in the input with a dilation of 1. If we increase the dilation to 2, then the kernel will be dilated spatially and will be applied. Instead of being applied to three consecutive elements, an element in between will be skipped. *Figure 12.8* shows how this works. As we can see, this can also increase the receptive field of the network.

Both these techniques are similar but different and are compatible with each other. The following diagram shows what happens when we apply strides and dilations together (although this doesn't happen frequently):

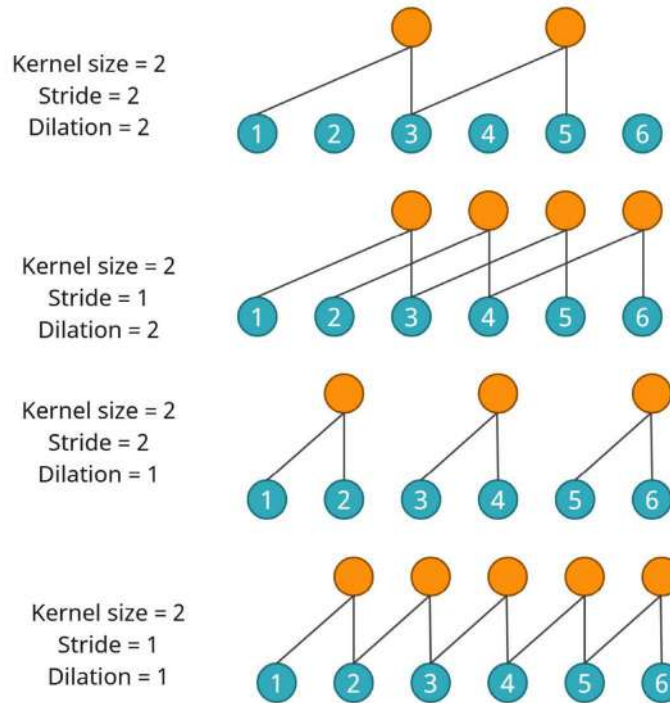


Figure 12.9: Strides and dilations in convolutions

Now, what if we want to make the output dimensions the same as the input dimensions? By using some basic algebra and rearranging the previous formula, we get the following:

$$P_l + p_r = d(k-1) + L(s-1) - (s-1)$$

And in time series, we typically pad on the left rather than the right because of the strong autocorrelation that is typically present. Padding the latest few entries with zeros or some other values will make the learning of the prediction function very hard, as the latest hidden states are directly influenced by the padded values. The *Further reading* section contains a link to an article by Kilian Batzner about autoregressive convolutions. It is a must-read if you wish to really understand the concepts we have discussed here and also understand a few limitations. The *Further reading* section also contains a link to a GitHub repository that contains animations of convolutions for 2D inputs, which will give you a good intuition of what is happening.

There is just one more term that you may hear often about convolutions, especially in time series—**causal convolutions**. All you have to keep in mind is that causal convolutions are not special types of convolutions. So long as we ensure that we won't use future timesteps to predict the current timestep while training, we are performing causal operations. This is typically done by offsetting the target and padding the inputs.

Convolution in PyTorch

Now, let's understand the PyTorch implementation of the CNN (a one-dimensional CNN, which is typically used for sequences such as time series). Let's look at the different parameters the implementation provides while initializing. We have just discussed the following terms, so they should be familiar to you by now:

- **in_channels**: The number of expected features in the input. If we are using just the history of the time series, then this would be 1. But when we use history along with some other features, then this will be >1. For subsequent layers, the **out_channels** you used in the previous layer become your **in_channels** in the current layer.
- **out_channels**: The number of kernels or filters applied to the input. Each kernel/filter produces a convolution operation with its own weights.
- **kernel_size**: This is the size of the kernel we use for convolving.
- **stride**: The stride of the convolution. The default is 1.
- **padding**: This is the padding that is added to *both* sides. If we set the value as 2, the sequence that we pass to the layer will have a padded position on both the left and right. We can also give **valid** or **same** as input. These are easy ways of mentioning the kind of padding we need to add. **padding='valid'** is the same as no padding. **padding='same'** pads the input so that the output has the shape as the input. However, this mode doesn't support any stride values other than 1. The default is 0.
- **padding_mode**: This defines how the padded positions should be filled with values. The most common and default option is **zeros**, where all the padded tokens are filled with zeros. Another useful mode that is relevant for time series is **replicate**, which behaves like forward and backward fill in pandas. The other two options—**reflect** and **circular**—are more esoteric and are only used for specific use cases. The default is **zeros**.
- **dilation**: The dilation of the convolution. The default is 1.
- **groups**: This parameter lets you control the way input channels are connected to output channels. The number specified in **groups** specifies how many groups will be formed so that the convolutions happen within a group and not across. For instance, **group=2** means that half the input channels will be convolved by one set of kernels and that the other half will be convolved by a separate set of kernels. This is equivalent to running two convolution layers side by side. Check the documentation for more information on this parameter. Again, this is for an esoteric use case. The default is 1.
- **bias**: This parameter adds a learnable bias to the convolutions. The default is **True**.

Let's apply a CNN model to the same synthetic data we generated earlier in this chapter with a kernel size of 3:

```
conv = nn.Conv1d(in_channels=1, out_channels=1, kernel_size=k)
```

Now, let's look at the inputs and outputs that are expected from a CNN.

Conv1d expects the inputs to have three dimensions—(batch size, number of channels, sequence length). For the initial input layer, the number of channels is the number of features you feed into the network; for intermediate layers, it is the number of kernels we used in the previous layer. The output from Conv1d is in the form of (batch size, number of channels (output), sequence length (output)).

So let's run our sequence through Conv1d and look at the inputs and outputs (for more detailed steps, refer to the 02-Building_Blocks.ipynb notebook):

```
#input dim: torch.Size([6, 1, 15])
# batch size = 6, number of features = 1 and sequence length = 15
output = conv(cnn_input)
# Output should be in_dim - k + 1
assert output.size(-1)==cnn_input.size(-1)-k+1
output.shape #-> torch.Size([6, 1, 13])
```

The notebook provides a slightly more detailed analysis of Conv1d, with tables showing the impact that the hyperparameters have on the shape of the output, what kind of padding is used to make the input and output dimensions the same, and how a convolution with equal weights is just like a rolling mean. I highly suggest that you check it out and play around with the different options to get a feel of what the layer does for you.



The inbuilt padding in Conv1d has its roots in image processing, so the padding technique defaults to adding to both sides. However, for sequences, it is preferable to use padding on the left, and because of that, it is also preferable to handle how the input sequences are padded separately and not use the inbuilt mechanism. `torch.nn.functional` has a handy method called `pad` that can be used to this effect.

Other building blocks are used in time series forecasting because the architecture of a deep neural network is only limited by creativity. But the point of this chapter was to introduce you to the common ones that appear in many different architectures. We also intentionally left out one of the most popular architectures used nowadays: the transformer. This is because we have devoted another chapter (*Chapter 14, Attention and Transformers for Time Series*) to understanding attention before we look at transformers. Another major block that is slowly gaining popularity is graph neural networks, which can be thought of as specialized CNNs that operate on graph-based data rather than grids. However, these are outside the scope of this book, since they are an area of active research.

Summary

After introducing deep learning in the previous chapter, in this chapter, we gained a deeper understanding of the common architectural blocks that are used for time series forecasting. The encoder-decoder paradigm was explained as a fundamental way to structure a deep neural network for forecasting. Then, we learned about FFNs, RNNs (LSTMs and GRUs), and CNNs, exploring how they are used to process time series. We also saw how we could use all these major blocks in PyTorch by using the associated notebook, and then we got our hands dirty with some PyTorch code.

In the next chapter, we'll learn about a few major patterns we can use to arrange these blocks to perform time series forecasting.

References

The following references were used in this chapter:

1. Neco, R. P. and Forcada, M. L. (1997), *Asynchronous translations with recurrent neural nets*. Neural Networks, 1997., International Conference on (Vol. 4, pp. 2535–2540). IEEE: <https://ieeexplore.ieee.org/document/614693>.
2. Kalchbrenner, N. and Blunsom, P. (2013), *Recurrent Continuous Translation Models*. EMNLP (Vol. 3, No. 39, p. 413): <https://aclanthology.org/D13-1176/>.
3. Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. (2014), *Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation*. Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP), pages 1724–1734, Doha, Qatar. Association for Computational Linguistics: <https://aclanthology.org/D14-1179/>.
4. Ilya Sutskever, Oriol Vinyals, and Quoc V. Le. (2014), *Sequence to sequence learning with neural networks*. Proceedings of the 27th International Conference on Neural Information Processing Systems – Volume 2: <https://dl.acm.org/doi/10.5555/2969033.2969173>.
5. Rumelhart, D., Hinton, G., and Williams, R (1986). *Learning representations by back-propagating errors*. Nature 323, 533–536: <https://doi.org/10.1038/323533a0>.
6. Schuster, M. and Paliwal, K. K. (1997). *Bidirectional recurrent neural networks*. IEEE Transactions on Signal Processing, 45(11), 2673–2681: <https://doi.org/10.1109/78.650093>.
7. Sepp Hochreiter (1991) *Untersuchungen zu dynamischen neuronalen Netzen*. Diploma thesis, TU Munich: <https://people.idsia.ch/~juergen/SeppHochreiter1991ThesisAdvisorSchmidhuber.pdf>.
8. Y. Bengio, P. Frasconi, and P. Simard (1993), *The problem of learning long-term dependencies in recurrent networks*. IEEE International Conference on Neural Networks, pp. 1183–1188 vol.3: 10.1109/ICNN.1993.298725.
9. Y. Bengio, P. Simard, and P. Frasconi (1994) *Learning long-term dependencies with gradient descent is difficult* in IEEE Transactions on Neural Networks, vol. 5, no. 2, pp. 157–166, March 1994: 10.1109/72.279181.
10. Hochreiter, S. and Schmidhuber, J. (1997). *Long Short-Term Memory*. Neural Computation, 9(8), 1735–1780: <https://doi.org/10.1162/neco.1997.9.8.1735>.
11. Cho, K., Merriënboer, B.V., Gülçehre, Ç., Bahdanau, D., Bougares, F., Schwenk, H., and Bengio, Y. (2014). *Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation*. EMNLP: <https://www.aclweb.org/anthology/D14-1179.pdf>.
12. Fukushima, K. *Neocognitron: A self-organizing neural network model for a mechanism of pattern recognition unaffected by shift in position*. Biol. Cybernetics 36, 193–202 (1980): <https://doi.org/10.1007/BF00344251>.

13. Y. Le Cun, B. Boser, J. S. Denker, R. E. Howard, W. Hubbard, L. D. Jackel, and D. Henderson. 1990. *Handwritten Digit Recognition with a Back-Propagation Network*. Advances in neural information processing systems 2. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 396–404: <https://proceedings.neurips.cc/paper/1989/file/53c3bce66e43be4f209556518c2fcb54-Paper.pdf>.

Further reading

Take a look at the following resources to learn more about the topics that were covered in this chapter:

- Official PyTorch tutorials: <https://pytorch.org/tutorials/beginner/basics/intro.html>
- *Essence of linear algebra*, by 3Blue1Brown: https://www.youtube.com/playlist?list=PLZHQObOWTQDPD3MizzM2xVFitgF8hE_ab
- *Neural Networks – A Linear Algebra Perspective*, by Manu Joseph: <https://deep-and-shallow.com/2022/01/15/neural-networks-a-linear-algebra-perspective/>
- *Deep Learning*, by Ian Goodfellow, Yoshua Bengio, and Aaron Courville: <https://deep-and-shallow.com/2022/01/15/neural-networks-a-linear-algebra-perspective/>
- *Understanding LSTMs*, by Christopher Olah: <http://colah.github.io/posts/2015-08-Understanding-LSTMs/>
- *Intuitive Guide to Convolution*: <https://betterexplained.com/articles/intuitive-convolution/>
- *Computing Receptive Fields of Convolutional Neural Networks*, by Andre Araujo, Wade Norris, and Jack Sim: <https://distill.pub/2019/computing-receptive-fields/>
- *Convolutions in Autoregressive Neural Networks*, by Kilian Batzner: <https://theblog.github.io/post/convolution-in-autoregressive-neural-networks/>
- *Convolution Arithmetic*, by Vincent Dumoulin and Francesco Visin: https://github.com/vdumoulin/conv_arithmetic
- *Dropout: A Simple Way to Prevent Neural Networks from Overfitting*, by Nitish Srivastava et al.: <https://jmlr.org/papers/v15/srivastava14a.html>

Unlock this book's exclusive benefits now

This book comes with additional benefits designed to elevate your learning experience.



Note: Have your purchase invoice ready before you begin. <https://www.packtpub.com/unlock/9781835883181>

13

Common Modeling Patterns for Time Series

We reviewed a few major and common building blocks of a **deep learning (DL)** system, specifically suited for time series, in the last chapter. Now that we know what those blocks are, it's time for a more practical lesson. Let's see how we can put these common blocks together in the various ways in which time series forecasting is modeled using the dataset we have been working with all through this book.

In this chapter, we will be covering these main topics:

- Tabular regression
- Single-step-ahead recurrent neural networks
- Sequence-to-sequence models

Technical requirements

You will need to set up the Anaconda environment following the instructions in the *Preface* of the book to get a working environment with all the libraries and datasets required for the code in this book. Any additional libraries will be installed while running the notebooks.

You need to run the following notebooks for this chapter:

- 02-Preprocessing_London_Smart_Meter_Dataset.ipynb in Chapter02
- 01-Setting_up_Experiment_Harness.ipynb in Chapter04
- 01-Feature_Engineering.ipynb in Chapter06
- 00-Single_Step_Backtesting_Baselines.ipynb, 01-Forecasting_with_ML.ipynb, and 02-Forecasting_with_Target_Transformation.ipynb in Chapter08
- 01-Global_Forecasting_Models-ML.ipynb in Chapter10

The associated code for the chapter can be found at <https://github.com/PacktPublishing/Modern-Time-Series-Forecasting-with-Python-/tree/main/notebooks/Chapter13>.

Tabular regression

In *Chapter 5, Time Series Forecasting as Regression*, we saw how we can convert a time series problem into a standard regression problem with temporal embedding and time delay embedding. In *Chapter 6, Feature Engineering for Time Series Forecasting*, we have already created the necessary features for the household energy consumption dataset we have been working on, and in *Chapter 8, Forecasting Time Series with Machine Learning Models*, *Chapter 9, Ensembling and Stacking*, and *Chapter 10, Global Forecasting Models*, we used traditional **machine learning (ML)** models to create a forecast.

Just as we used standard ML models for forecasting, we can also use DL models built for tabular data using the feature-engineered dataset we have created. We already talked about data-driven methods and how they are better when given larger amounts of data. DL models take that paradigm even further and enable us to learn highly data-driven models. One of the advantages of using a DL model in this setting, over the ML models, is the flexibility DL offers us. All through *Chapters 8, 9, and 10*, we only saw how we can create single-step-ahead forecasting using ML models. We have a separate section on multi-step forecasting in *Chapter 18*, where we go into detail on different strategies with which we can generate multi-step forecasts, and we address one of the limitations of standard ML models in multi-step forecasting. But right now, let's just understand that standard ML models are designed to have a single output and, because of that fact, getting multi-step forecasts is not straightforward. But with tabular DL models, we have the flexibility to train the model to predict multiple targets, and this enables us to generate multi-step forecasts easily.

PyTorch Tabular is an open-source library (https://github.com/manujosephv/pytorch_tabular) that makes it easy to work with DL models in the tabular data domain, and it also has ready-to-use implementations of many state-of-the-art DL models. We are going to use PyTorch Tabular to generate forecasts using the feature-engineered datasets we created in *Chapter 6, Feature Engineering for Time Series Forecasting*.

PyTorch Tabular has very detailed documentation and tutorials to get you started here: <https://pytorch-tabular.readthedocs.io/en/latest/>. Although we won't be going into detail on all the intricacies of the library, we will look at how we can use a bare-bones version to generate a forecast on the dataset we are working on using a FTTransformer model. FTTransformer is one of the state-of-the-art DL models for tabular data. DL for tabular data is a whole different kind of model, and I've linked a blog post in the *Further reading* section as a primer to the field of study. For our purposes, we can treat them as any standard ML model in scikit-learn.



Notebook alert:

To follow along with the complete code, use the notebook named `01-Tabular_Regression.ipynb` in the `Chapter13` folder and the code in the `src` folder.

We start off, pretty much like before, by loading the libraries and necessary datasets. Just one additional thing we are doing here is that instead of taking the same selection of blocks we worked with in *Part 2, Machine Learning for Time Series*, we take smaller data by selecting half the number of blocks as before.

This is done to make the **neural network (NN)** training smoother and faster and for it to fit into GPU memory (if any). I'd like to stress here that this is done purely for hardware reasons, and provided we have sufficiently powerful hardware, we need not have smaller datasets for DL. On the contrary—DL loves larger datasets. But since we want to keep the focus on the modeling side, the engineering constraints and techniques in working with larger datasets have been kept outside the scope of this book.

```
uniq_blocks = train_df.file.unique().tolist()
sel_blocks = sorted(uniq_blocks, key=lambda x: int(x.replace("block_", "")))
[:len(uniq_blocks)//2]
train_df = train_df.loc[train_df.file.isin(sel_blocks)]
test_df = test_df.loc[test_df.file.isin(sel_blocks)]
sel_lclids = train_df.LCLid.unique().tolist()
```

After handling the missing values, we are ready to start using PyTorch Tabular. We first import the necessary classes from the library, like so:

```
from pytorch_tabular.config import DataConfig, OptimizerConfig, TrainerConfig
from pytorch_tabular.models import FTTransformerConfig
from pytorch_tabular import TabularModel
```

PyTorch Tabular uses a set of config files to define the parameters required for running the model, and these configs include everything from how the DataFrame is configured to what kind of preprocessing needs to be applied, what kind of training we need to do, what model we need to use, what the hyperparameters of the model are, and so on. Let's see how we can define a bare-bones configuration (because PyTorch Tabular makes use of intelligent defaults wherever possible to make the usage easier for the practitioner):

```
data_config = DataConfig(
    target=[target], #target should always be a list
    continuous_cols=[
        "visibility",
        "windBearing",
        ...,
        "timestamp_Is_month_start",
    ],
    categorical_cols=[
        "holidays",
        ...,
        "LCLid"
    ],
    normalize_continuous_features=True
)
trainer_config = TrainerConfig(
```



```

    auto_lr_find=True, # Runs the LRFinder to automatically derive a Learning
    rate
    batch_size=1024,
    max_epochs=1000,
    auto_select_gpus=True,
    gpus=-1
)
optimizer_config = OptimizerConfig()

```

We use a very high `max_epochs` parameter in `TrainerConfig` because, by default, PyTorch Tabular employs a technique called **early stopping**, where we continuously keep track of the performance on a validation set and stop the training when the validation loss starts to increase.

Selecting which model to use from the implemented models in PyTorch Tabular is as simple as choosing the right configuration. Each model is associated with a configuration that defines the hyperparameters of the model. So, just by using that configuration, PyTorch Tabular understands which model the user wants to use. Let's choose the `FTTransformerConfig` model and define a few hyperparameters:

```

model_config = FTTransformerConfig(
    task="regression",
    num_attn_blocks=3,
    num_heads=4,
    transformer_head_dim=64,
    attn_dropout=0.2,
    ff_dropout=0.1,
    out_ff_layers="32",
    metrics=["mean_squared_error"]
)

```

The main and only mandatory parameter here is `task`, which tells PyTorch Tabular whether it is a *regression* or *classification* task.



Although PyTorch Tabular provides the best defaults, we only set these parameters to make the training faster and fit into the memory of the GPU we are running on. If you are not running the notebook on a machine with a GPU, choosing a smaller and faster model such as `CategoryEmbeddingConfig` would be better.

Now, all that is left to do is put all these configs together in a class called `TabularModel`, which is the workhorse of the library, and as with any scikit-learn model, call `fit` on the object. But, unlike a scikit-learn model, you don't need to split `x` and `y`; we just need to provide the `DataFrame`, as follows:

```

tabular_model.fit(train=train_df)

```

Once the training is complete, you can save the model by running the following code:

```
tabular_model.save_model("notebooks/Chapter13/ft_transformer_global")
```

If for any reason you have to close your notebook instance after training, you can always load the model back by using the following code:

```
tabular_model = TabularModel.load_from_checkpoint("notebooks/Chapter13/ft_
transformer_global")
```

This way, you don't need to spend a lot of time training the model again, but instead, use it for prediction.

Now, all that is left is to make predictions using the unseen data and evaluate the performance. Here's how we can do this:

```
forecast_df = tabular_model.predict(test_df)
agg_metrics, eval_metrics_df = evaluate_forecast(
    y_pred=forecast_df[f"{target}_prediction"],
    test_target=forecast_df["energy_consumption"],
    train_target=train_df["energy_consumption"],
    model_name=model_config._model_name,
)
```

We have used the untuned global forecasting model with metadata that we trained in *Chapter 10, Global Forecasting Models*, as the baseline against which we can do a cursory check on how well the DL model is doing, as illustrated in the following screenshot:

Algorithm	MAE	MSE	meanMASE	Forecast Bias
GFM+Meta (NativeLGBM)	0.0873	0.0340	1.0627	-0.68%
FTTransformerModel	0.0913	0.0332	1.1598	5.90%

Figure 13.1: Evaluation of the DL-based tabular regression

We can see that the FTTransformer model is competitive with the LightGBM model we trained in *Chapter 10*. Maybe, with the right amount of tuning and partitioning, the FTTransformer model can do as well as or better than the LightGBM model. Training a competitive DL model in the same way as LightGBM is useful in many ways. First, it provides flexibility and trains the model to predict multiple timesteps at once. Second, this can also be combined with the LightGBM model in an ensemble, and because of the variety the DL model brings to the mix, this can make the ensemble performance better.



Things to try:

Use PyTorch Tabular's documentation and play around with other models or change the parameters to see how the performance changes.

Select a few households and plot them to see how well the forecast matches up to the targets.

Now, let's look at how we can use **recurrent neural networks** (RNNs) for single-step-ahead forecasting.

Single-step-ahead recurrent neural networks

Although we took a little detour to check out how DL regression models can be used to train the same global models we learned about in *Chapter 10, Global Forecasting Models*, now we are back to looking at DL models and architectures specifically built for time series. As always, we will look at simple one-step-ahead and local models first before moving on to more complex modeling paradigms. In fact, we have another chapter (*Chapter 15, Strategies for Global Deep Learning Forecasting Models*) entirely devoted to techniques we can use to train global DL models.

Now, let's bring our attention back to one-step-ahead local models. We saw RNNs (vanilla RNN, **long short-term memory** (LSTM), and **gated recurrent unit** (GRU)) as a few blocks we can use for sequences such as time series. Now, let's see how we can use them in an **end-to-end** (E2E) model on the dataset we have been working on (the *London smart meters* dataset).

Although we will be looking at a few libraries (such as *darts*) that make the process of training DL models for time series forecasting easier, in this chapter, we will be looking at how to develop such models from scratch. Understanding how a DL model for time series forecasting is put together from the ground up will give you a good grasp of the concepts that are needed to use and tweak the libraries that we will be looking at later.

We will be using PyTorch, and if you are not comfortable, I suggest you head to *Chapter 12, Building Blocks of Deep Learning for Time Series*, and the associated notebooks for a quick refresher. On top of that, we are also going to use PyTorch Lightning, which is another library built on top of PyTorch to make training models using PyTorch easy, among other benefits.

We talked about *time delay embedding* in *Chapter 5, Time Series Forecasting as Regression*, where we discussed using a window in time to embed the time series into a format more suitable for regression. When training NNs for time series forecasting also, we need such windows. Suppose we are training on a single time series. We can give this super-long time series to an RNN as is, but then it only becomes one sample in the dataset. And with just one sample in the dataset, it's close to impossible to train any ML or DL models. So, it's advisable to sample multiple windows from the time series to convert the time series into a number of data samples in a process that is very similar to time delay embedding. This window also sets the memory of the DL model.

The first step we need to take is to create a PyTorch dataset that takes the raw time series and prepares these samples' windows. A dataset is like an iterator over the data that gives us samples corresponding to a provided index. Defining a custom dataset for PyTorch is as simple as defining a class that takes in a few arguments (data being one of them) and defining two mandatory methods in the class, as follows:

- `__len__(self)`: This sets the maximum number of samples in the dataset.
- `__getitem__(self, idx)`: This picks the idx^{th} sample from the dataset.

We have defined a dataset in `src/dl/dataloaders.py` with the name `TimeSeriesDataset`, which takes in the following parameters:

- **Data**: This argument can either be a pandas `DataFrame` or a NumPy array with the time series. This is the entire time series, including train, validation, and test, and the splits occur inside the class.
- **window**: This sets the length of each sample.
- **horizon**: This sets the number of future timesteps we want to get as the target.
- **n_val**: This parameter can either be a `float` or an `int` data type. If `int`, it represents the number of timesteps to be reserved as validation data. If `float`, this represents the percent of total data to be reserved as validation data.
- **n_test**: This parameter is similar to `n_val`, but does the same for test data.
- **normalize**: This parameter defines how we want to normalize the data. This takes in three options: `none` means no normalizing and `global` means we calculate the mean and standard deviation of the train data and use it to standardize the entire series using this equation:

$$\frac{\text{series} - \text{mean}}{\text{std}}$$

`local` means we use the window mean and standard deviation to standardize the series.

- **normalize_params**: This parameter takes in a tuple of mean and standard deviations. If provided, this can be used to standardize in *global* standardization. This is typically used to use the train mean and standard deviation on validation and test data as well.
- **mode**: This parameter sets which dataset we want to make. It takes in one of three values: `train`, `val`, or `test`.

Each sample from this dataset returns to you two tensors—the window (X) and the corresponding target (Y) (see Figure 13.2):

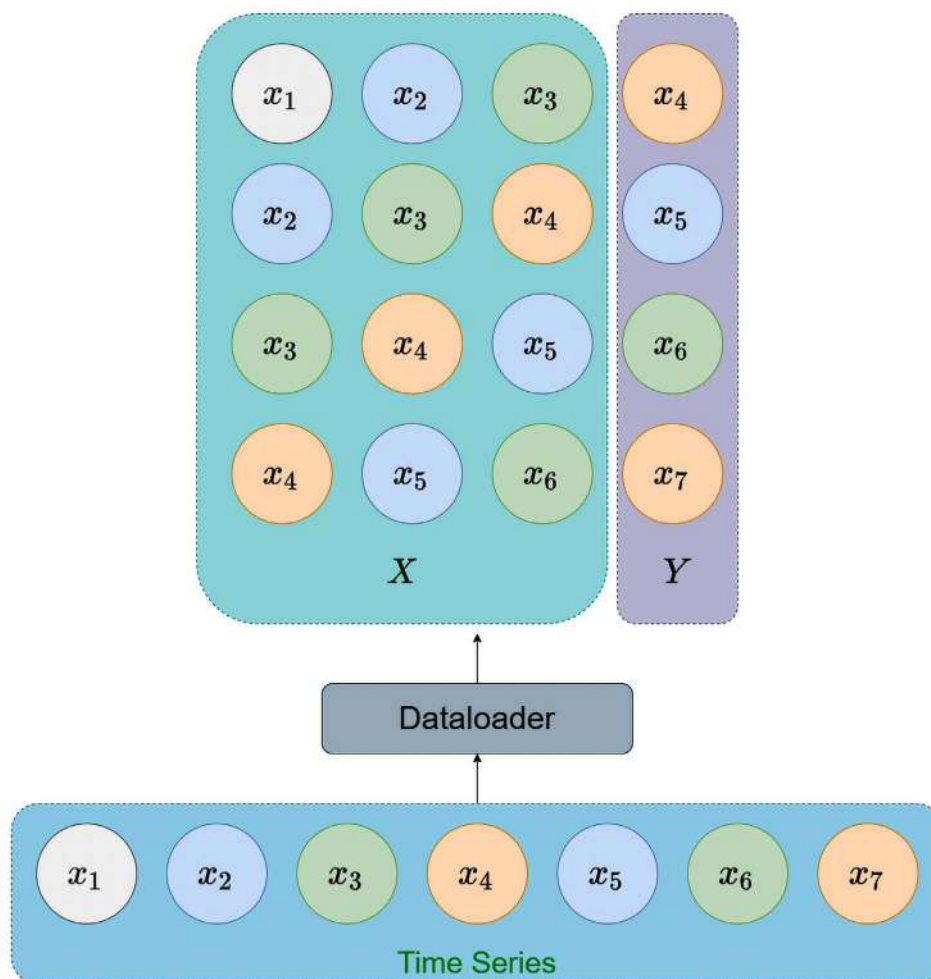


Figure 13.2: Sampling the time series using a dataset and dataloader

Now that we have the dataset defined, we need another PyTorch artifact called a dataloader. A dataloader uses the dataset to pick samples into a batch of samples, among other things. In the PyTorch Lightning ecosystem, we have another concept called a datamodule, which is a standard way of generating dataloaders. We need train dataloaders, validation dataloaders, and test dataloaders. Datamodules provide a good abstraction to encapsulate the whole data part of the pipeline. We have defined a datamodule in `src/dl/dataloaders.py` called `TimeSeriesDataModule` that takes in the data along with the batch size and prepares the datasets and dataloaders necessary for training. The parameters are exactly the same as `TimeSeriesDataset`, with `batch_size` as the only additional parameter.

**Notebook alert:**

To follow along with the complete code, use the notebook named 02-One-Step_RNN.ipynb in the Chapter13 folder and the code in the src folder.

We will not be going into each and every step in the notebook but will be just stressing the key points. The code in the notebook is well commented, and we urge you to follow the code along with the book.

We have already sampled a household from the data, and now, let's see how we can define a datamodule:

```
datamodule = TimeSeriesDataModule(data = sample_df[[target]],
    n_val = sample_val_df.shape[0],
    n_test = sample_test_df.shape[0],
    window = 48, # giving enough memory to capture daily seasonality
    horizon = 1, # single step
    normalize = "global", # normalizing the data
    batch_size = 32,
    num_workers = 0)
datamodule.setup()
```

`datamodule.setup()` is the method that calculates and sets up the dataloaders. Now, we can access the train dataloader by simply calling `datamodule.train_dataloader()`, and similarly, validation and test by `val_dataloader` and `test_dataloader` methods, respectively. We can access the samples as follows:

```
# Getting a batch from the train_dataloader
for batch in datamodule.train_dataloader():
    x, y = batch
    break
print("Shape of x: ", x.shape) #-> torch.Size([32, 48, 1])
print("Shape of y: ", y.shape) #-> torch.Size([32, 1, 1])
```

We can see that each sample has two tensors—`x` and `y`. There are three dimensions for the tensors, and they correspond to *batch size*, *sequence length*, and *features*.

Now that we have the data pipeline ready, we need to build out the modeling and training pipelines. PyTorch Lightning has a standard way of defining these so that they can be plugged into the training engine they provide (which makes our life so much easier). The PyTorch Lightning documentation (<https://pytorch-lightning.readthedocs.io/en/latest/starter/introduction.html>) has good resources to get started with and to go into depth on as well. We have also linked to a video in the *Further reading* section that makes the transition from pure PyTorch to PyTorch Lightning easy. I strongly urge you to take some time to familiarize yourself with it.

When defining a model in PyTorch, a standard method called `forward` is the only mandatory method you have to define, apart from `__init__`. This is because the training loop is something that we will have to write on our own. In the `01-PyTorch_Basics.ipynb` notebook for *Chapter 12, Building Blocks of Deep Learning for Time Series*, we saw how we can write a PyTorch model and a training loop to train a simple classifier. But now that we are delegating the training loop to PyTorch Lightning, we have to include a few additional methods as well:

- `training_step`: This method takes in a batch and uses the model to get the outputs, calculate the loss/metrics, and return the loss.
- `validation_step` and `test_step`: These methods take in the batch and use the model to get the outputs and calculate the loss/metrics.
- `predict_step`: This method is used to define the step to be taken while inferencing. If there is anything special we have to do for inferencing, we can define this method. If this is not defined, it uses `test_step` for the prediction use case as well.
- `configure_optimizers`: This method defines the optimizer to be used, for instance, Adam or RMSProp.

We have defined a `BaseModel` class in `src/dl/models.py` that implements all the common functions, such as loss and metric calculation and result logging, as a framework to implement new models. Using this `BaseModel` class, we have defined a `SingleStepRNNModel` class that takes in a standard config (`SingleStepRNNConfig`) and initializes an RNN, LSTM, or GRU model.

Before we look at how the model is defined, let's see what the different config (`SingleStepRNNConfig`) parameters are:

- `rnn_type`: This parameter takes in one of three strings as input: RNN, GRU, or LSTM. This defines what kind of model we will initialize.
- `input_size`: This parameter defines the number of features the RNN is expecting.
- `hidden_size`, `num_layers`, and `bidirectional`: These parameters are the same as the ones we saw in the RNN cell in *Chapter 12, Building Blocks of Deep Learning for Time Series*.
- `learning_rate`: This defines the learning rate of the optimization procedure.
- `optimizer_params`, `lr_scheduler`, and `lr_scheduler_params`: These are parameters that let us tweak the optimization procedure. Let's not worry about them for now because all of them have been set to intelligent defaults.

With this setup, defining a new model is as simple as this:

```
rnn_config = SingleStepRNNConfig(
    rnn_type="RNN",
    input_size=1,
    hidden_size=128,
    num_layers=3,
    bidirectional=True,
    learning_rate=1e-3,
```

```
seed=42,
)
model = SingleStepRNNModel(rnn_config)
```

Now, let's take a peek at the forward method, which is the heart of the model. We want our model to do one-step-ahead prediction, and from *Chapter 12, Building Blocks of Deep Learning for Time Series*, we know what a typical RNN output is and how PyTorch RNNs just output the hidden state at each timestep. Let's see what we want to do visually and then see how we can code it up:

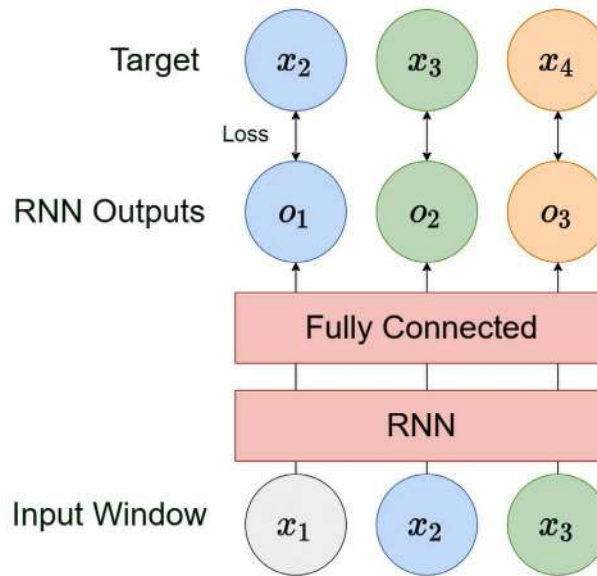


Figure 13.3: A single-step RNN

Suppose we are using the same example we saw in the dataloader—a time series with the following entries, $x_1, x_2, x_3, \dots, x_7$, and a window of three. So, one of the samples the dataloader gives will have x_1, x_2 , and x_3 as the input (x) and x_4 as the target. One way we can use this is by passing the sequence through the RNN, ignoring all the outputs except the last one, and using it to predict the target, x_4 . But that is not an efficient use of the samples we have, right? We also know that the output from the first timestep (using x_1) should output x_2 , the second timestep should output x_3 , and so on. Therefore, we can formulate the RNN in such a way that we maximize the usage of the data and, while training, use these additional points in time to also give a better signal to our model. Now, let's break down the forward method.

`forward` takes in a single argument called `batch`, which is a tuple of input and output. So, we unpack `batch` into two variables, `x` and `y`, like so:

```
x, y = batch
```

`x` will have the shape $\vec{a}$ (*batch size, window length, features*) and `y` will have the shape $\vec{a}$ (*batch size, target length, features*).

Now we need to pass the input sequence (x) through the RNN (RNN, LSTM, or GRU), like so:

```
x, _ = self.rnn(x)
```

As we saw in *Chapter 12, Building Blocks of Deep Learning for Time Series*, the PyTorch RNNs process the input and return two outputs—hidden states for each timestep and output (which is the hidden state of the last timestep). Here, we need the hidden states from all the timesteps, and therefore we capture that in the x variable. x will now have the dimension (*batch size, window length, hidden size of RNN*).

We have the hidden states, but to get the output, we need to apply a fully connected layer over the hidden states, and this fully connected layer should be shared across timesteps. An easy way to do this is to just define a fully connected layer with an input size equal to the hidden size of the RNN and then do the following:

```
x = self.fc(x)
```

x is a three-dimensional tensor, and when we use a fully connected layer on a three-dimensional tensor, PyTorch automatically applies the fully connected layer to each of the timesteps. Now, this final output is captured in x , and its dimensions would be (*batch size, window length, 1*).

Now, we have got the output of the network, but we also must do a bit of rearrangement to prepare the targets. Currently, y has just the one timestep beyond the window, but if we skip the first timestep from x and concatenate it with y , we would get the target, as we have in *Figure 13.3*:

```
y = torch.cat([x[:, 1:, :], y], dim=1)
```

By using array indexing, we select everything except the first timestep from x and concatenate it with y on the first dimension (which is the *window length*).

And with that, we have the x and y variables, which we can return, and the `BaseModel` class will calculate loss and handle the rest of the training. For the entire class, along with the forward method, you can refer to `src/dl/models.py`.

Let's test the model we have initialized by passing the batch from the dataloader:

```
y_hat, y = model(batch)
print("Shape of y_hat: ", y_hat.shape) #-> ([32, 48, 1])
print("Shape of y: ", y.shape) #-> ([32, 48, 1])
```

Now that the model is working as expected, without errors, let's start training the model. For that, we can use `Trainer` from PyTorch Lightning. There are so many options in the `Trainer` class, and a full list of all parameters to tweak the training can be found here: https://pytorch-lightning.readthedocs.io/en/stable/api/pytorch_lightning.trainer.trainer.Trainer.html#pytorch_lightning.trainer.trainer.Trainer.

But here, we are just going to use the bare minimum. Let's go over the parameters we will be using here one by one:

- **auto_select_gpus and gpus:** Together, these parameters let us select GPUs for training if present. If we set `auto_select_gpus` to `True` and `gpus` to `-1`, the `Trainer` class will choose all GPUs present in the machine, and if there are no GPUs, it falls back to CPU-based training.
- **callbacks:** PyTorch Lightning has a lot of useful callbacks that can be used during training such as `EarlyStopping`, `ModelCheckpoint`, and so on. Most useful callbacks are automatically added even if we don't explicitly set them, but `EarlyStopping` is one useful callback that needs to be set explicitly. `EarlyStopping` is a callback that lets us monitor the validation loss or metrics while training and stop the training when this starts to become worse. This is a form of regularization and helps us keep our model from overfitting to the train data. `EarlyStopping` has the following major parameters (a full list of parameters can be found here: https://pytorch-lightning.readthedocs.io/en/stable/api/pytorch_lightning.callbacks.EarlyStopping.html):
 - **monitor:** This parameter takes a string input that specifies the exact name of the metric that we want to monitor for early stopping.
 - **patience:** This specifies the number of epochs with no improvement in the monitored metric before the callback stops the training. For instance, if we set `patience` to `10`, the callback will wait for 10 epochs of the degrading metric before stopping the training. There are finer points of detail on these, which are explained in the documentation.
 - **mode:** This is a string input and takes one of `min` or `max`. This sets the direction of improvement. In `min` mode, training will stop when the quantity monitored has stopped decreasing, and in `max` mode, it will stop when the quantity monitored has stopped increasing.
- **min_epochs and max_epochs:** These parameters help us set min and max limits to the number of epochs the training should run. If we are using `EarlyStopping`, `min_epochs` decides the minimum number of epochs that will be run regardless of the validation loss/metrics, and `max_epochs` sets the upper limit on the number of epochs. So, even if the validation loss is still decreasing when we reach `max_epochs`, training will stop.

Glossary:

Here are a few terms you should know to fully digest NN training:



- **Training step:** This denotes a single gradient update to the parameter. In batched **stochastic gradient descent (SGD)**, the gradient update after each batch is considered a step.
- **Batch:** A batch is the number of data samples we run through the model and average the gradients over for the update in a training step.
- **Epoch:** An epoch is when the model has seen all the samples in a dataset, or all the batches in the dataset have been used for a gradient update.

So, let's initialize a bare-bones Trainer class:

```
trainer = pl.Trainer(
    auto_select_gpus=True,
    gpus=-1,
    min_epochs=5,
    max_epochs=100,
    callbacks=[pl.callbacks.EarlyStopping(monitor="valid_loss", patience=3)],
)
```

Now, all that is left is to trigger the training by passing in the model and datamodule to a method called `fit`:

```
trainer.fit(model, datamodule)
```

It will run for a while and, depending on when the validation loss starts to increase, it will stop the training. Once the model is trained, we can still use the Trainer class to make predictions on new data. The prediction uses the `predict_step` method that we defined in the `BaseModel` class, which in turn uses the `predict` method that we defined in the `SingleStepRNN` model. It's a very simple method that calls the `forward` method, takes in the model outputs, and just picks the last timestep from the output (which is the true output that we are projecting into the future). You can see an illustration of this here:

```
def predict(self, batch):
    y_hat, _ = self.forward(batch)
    return y_hat[:, -1, :]
```

So, let's see how we can use the Trainer class to make predictions on new data (or new dataloaders, to be exact):

```
pred = trainer.predict(model, datamodule.test_dataloader())
```

We just need to provide the trained model and the dataloader (here, we use the test dataloader that we have already set up and defined).

Now the output, `pred`, is a list of tensors, one for each batch in the dataloader. We just need to concatenate them, squeeze out any redundant dimensions, detach them from the computational graph, and convert them to a NumPy array. Here's how we can do this:

```
pred = torch.cat(pred).squeeze().detach().numpy()
```

Now, `pred` is a NumPy array of predictions for all the items in the test DataFrame (which was used to define `test_dataloader`), but remember we had applied a transformation to the raw time series to standardize it. Now, we need to reverse the transformation. The mean and standard deviation we used for the initial transformation are still stored in the train dataset. We merely retrieve them and invert the transformation we did earlier, like so:

```
pred = pred * datamodule.train.std + datamodule.train.mean
```

Now, we can do all kinds of actions on them, such as evaluate against actuals, visualize the predictions, and so on. Let's see how well the model has done. To get context, we have included the single-step ML models we did back in *Chapter 8, Forecasting Time Series with Machine Learning Models*, as well:

Algorithm	MAE	MSE	MASE	Forecast Bias
Lasso Regression	0.1598	0.0743	1.2452	3.78%
XGB Random Forest	0.1641	0.0819	1.2792	9.30%
LightGBM	0.1470	0.0666	1.1457	3.36%
RNN	0.2685	0.1721	2.0927	29.35%

Figure 13.4: Metrics of the vanilla single-step-ahead RNN on MAC000193 household

It looks like the RNN model did pretty badly. Let's also look at the predictions visually:

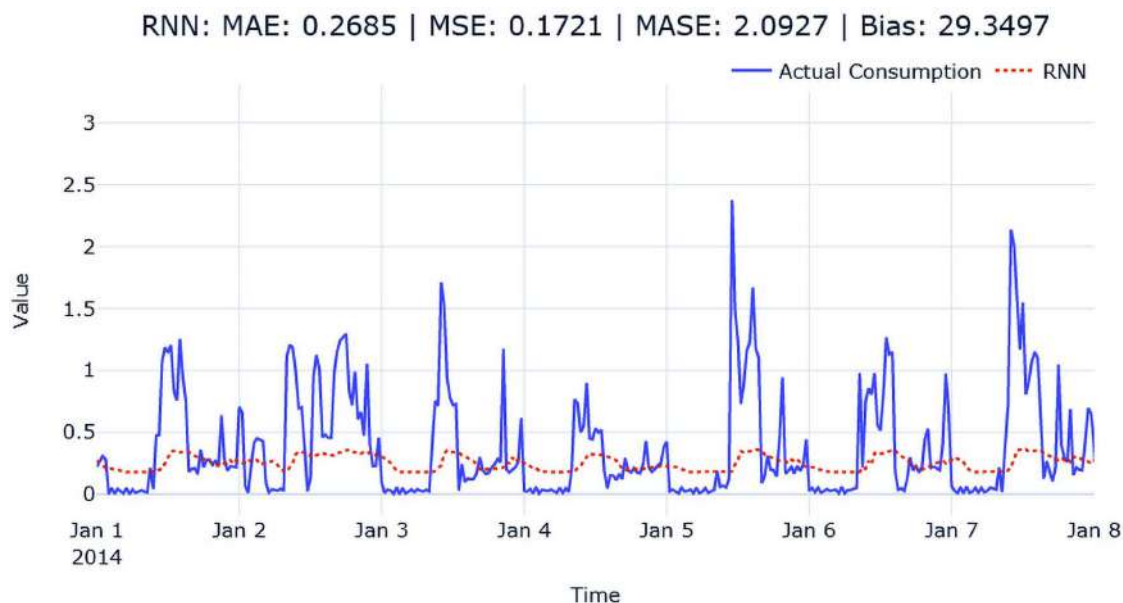


Figure 13.5: Single-step-ahead RNN predictions for MAC000193 household

We can see that the model has failed to learn the scale of the peaks and the nuances of the patterns. Maybe this is because of the problem that we discussed in terms of RNNs because the seasonality pattern here is spread over 48 timesteps; remember that the pattern requires the RNN to have long-term memory. Let's quickly swap out the model with LSTM and GRU and see how they are doing. The only thing we need to change is the `rnn_type` parameter in `SingleStepRNNConfig`.

The notebook has the code to train LSTM and GRU as well. But let's look at the metrics with LSTM and GRU:

Algorithm	MAE	MSE	MASE	Forecast Bias
Lasso Regression	0.1598	0.0743	1.2452	3.78%
XGB Random Forest	0.1641	0.0819	1.2792	9.30%
LightGBM	0.1470	0.0666	1.1457	3.36%
RNN	0.2685	0.1721	2.0927	29.35%
LSTM	0.1982	0.1125	1.5442	17.94%
GRU	0.1714	0.0899	1.3358	14.48%

Figure 13.6: Metrics for single-step-ahead LSTM and GRU on MAC000193 household

Now, it looks competitive. LightGBM is still the best model, but now the LSTM and GRU models are competitive and not entirely lacking, like the vanilla RNN model. If we look at the predictions, we can see that the LSTM and GRU models have managed to capture the pattern much better as well:

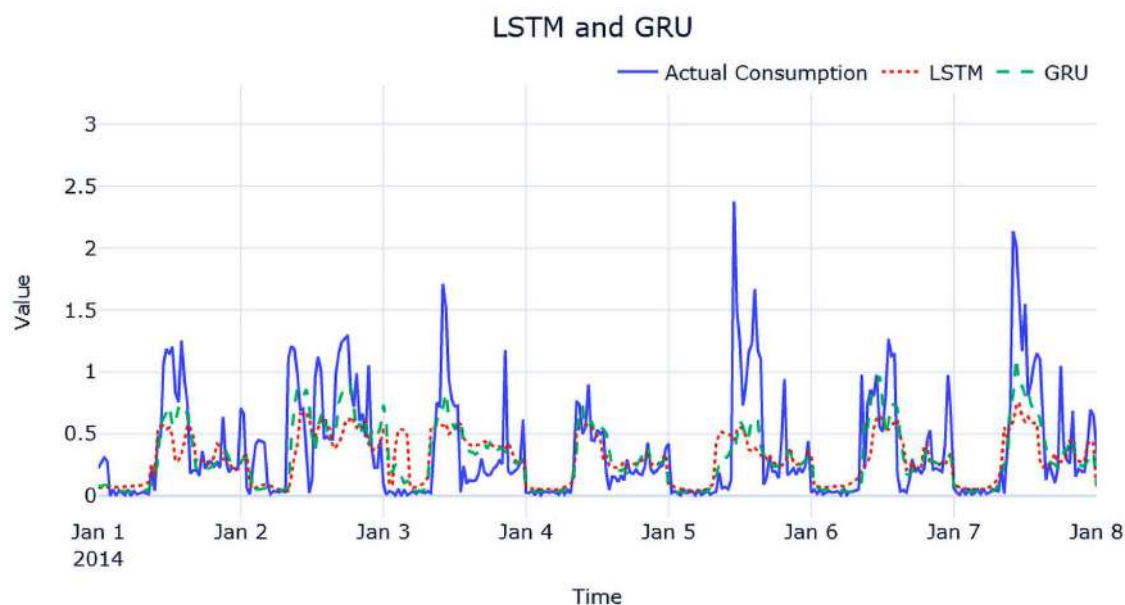


Figure 13.7: Single-step-ahead LSTM and GRU predictions for MAC000193 household



Things to try:

Try changing the parameters of the models and see how it works. How does a bidirectional LSTM perform? Can increasing the window increase performance?

Now that we have seen how a standard RNN can be used for single-step-ahead predictions, let's look at another modeling pattern that is more flexible than the one we just saw.

Sequence-to-sequence (Seq2Seq) models

We talked in detail about the Seq2Seq architecture and the encoder-decoder paradigm in *Chapter 12, Building Blocks of Deep Learning for Time Series*. Just to refresh your memory, the Seq2Seq model is kind of an encoder-decoder model by which an encoder encodes the sequence into a latent representation, and then the decoder steps in to carry out the task at hand using this latent representation. This setup is inherently more flexible because of the separation between the encoder (which does the representation learning) and the decoder, which uses the representation for predictions. One of the biggest advantages of this approach, from a time series forecasting perspective, is that the restriction of single step ahead is taken out. In this modeling pattern, we can extend the forecast to any forecast horizon we want.

In this section, let's put together a few encoder-decoder models and test out our single-step-ahead forecasts, just like we have been doing with the single-step-ahead RNNs.



Notebook alert:

To follow along with the complete code, use the notebook named `03-Seq2Seq_RNN.ipynb` in the `Chapter13` folder and the code in the `src` folder.

We can use the same mechanism we developed in the last section, such as `TimeSeriesDataModule`, the `BaseModel` class, and the corresponding code, for our Seq2Seq modeling pattern as well. Let's define a new PyTorch model called `Seq2SeqModel`, inheriting the `BaseModel` class. While we are at it, let's also define a new config file, called `Seq2SeqConfig`, to set the hyperparameters of the model. The final version of both can be found in `src/dl/models.py`.

Before we explain the different parameters in the model and the config, let's talk about the different ways we can set this Seq2Seq model.

RNN-to-fully connected network

For our convenience, let's restrict the encoder to be from the RNN family—it can be a vanilla RNN, LSTM, or GRU. Now, we saw in *Chapter 12, Building Blocks of Deep Learning for Time Series*, that in PyTorch, all the models in the RNN family have two outputs—*output* and *hidden states*, and we also saw that output is nothing but all the hidden states (final hidden states in stacked RNNs) at all timesteps. The hidden state that we get has the latest hidden states (and cell states, in the case of LSTM) of all layers in the stacked RNN setup. The encoder can be initialized just like we initialized the RNN family of models in the previous section, like so:

```
self.encoder = nn.LSTM(
    **encoder_params,
    batch_first=True,
)
```

And in the forward method, we can just do the following to encode the time series:

```
o, h = self.encoder(x)
```

Now, there are a few different ways we can decode the information. The first one we will discuss is using a fully connected layer. Either the fully connected layer can take the latest hidden state from the encoder and predict the desired output or we can flatten all the hidden states into a long vector and use that to predict the output. The latter provides more information to the decoder, but there can be more noise as well. Both are shown in *Figure 13.8*, using the same example we used in the last section as well:

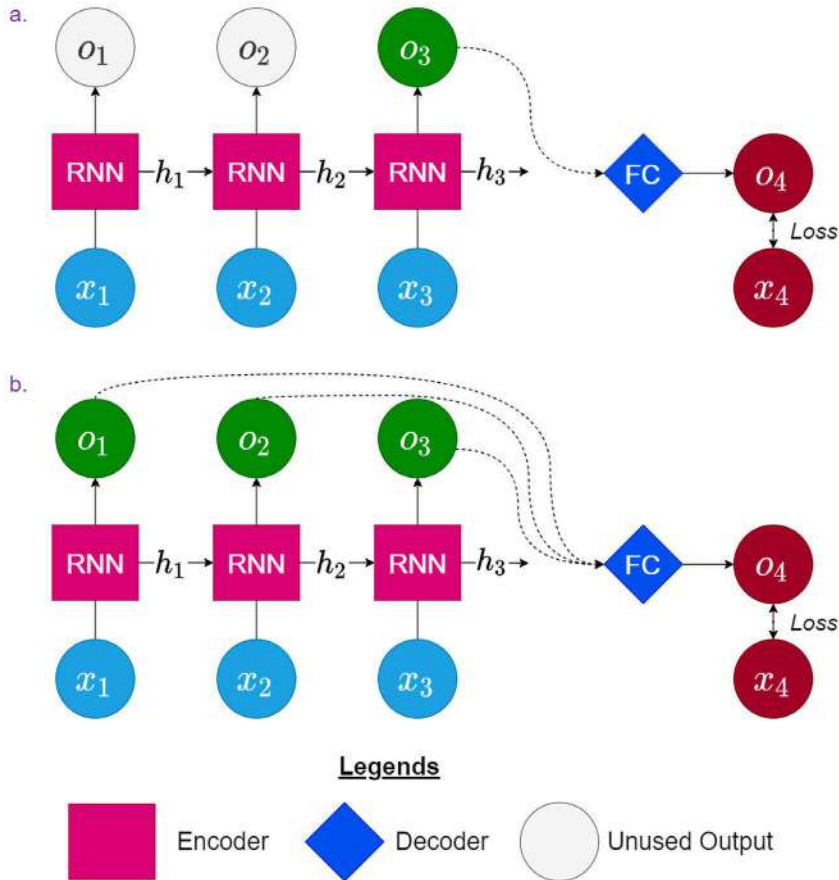


Figure 13.8: RNN as the encoder and a fully connected layer as the decoder

Let's also see how we can put this together in code. The decoder in the first case, where we are using just the last hidden state of the encoder, will look like this:

```
self.decoder = nn.Linear(
    hidden_size*bi_directional_multiplier, horizon
)
```

Here, `bi_directional_multiplier` is 2 if the encoder is bidirectional and 1 otherwise. This is done because if the encoder is bidirectional, there will be two hidden states concatenated together for each timestep. `horizon` is the number of timesteps ahead we want to forecast.

In the second case, where we are using the hidden states from all the timesteps, we need to make the decoder, as follows:

```
self.decoder = nn.Linear(
    hidden_size * bi_directional_multiplier * window_size,
    horizon
)
```

Here, the input vector will be the flattened vector of all the hidden states from all the timesteps, and hence the input dimension would be `hidden_size * window_size`.

And in the forward method, we can do the following for case 1:

```
y_hat = self.decoder(o[:, -1, :]).unsqueeze(-1)
```

Here, we are just taking the hidden state from the latest timestep and unsqueezing to maintain three dimensions as the target, `y`.

For case 2, we can do the following:

```
y_hat = self.decoder(o.reshape(o.size(0), -1)).unsqueeze(-1)
```

Here, we first reshape the entire hidden state to flatten it and then pass it through the decoder to get the predictions. We unsqueeze to insert the dimension we collapsed so that the output and target, `y`, have the same dimensions.

Even though, in theory, we can use the fully connected decoder to predict as much into the future as possible, practically, there are limitations. When we have a large number of steps to forecast, the model will have to learn that big of a matrix to generate those outputs, and that becomes harder as the matrix becomes bigger. Another point worth noting is that each of these predictions happens independently with the information encoded in the latent representation. For instance, the prediction of 5 timesteps ahead is only dependent on the latent representation from the encoder and not predictions of *timesteps* 1 to 4. Let's look at another type of Seq2Seq, which makes the decoding more flexible and aware of the temporal aspect of the problem.

RNN-to-RNN

Instead of using a fully connected layer as the decoder, we can use another RNN for decoding as well—so, one model from the RNN family takes care of the encoding and another model from the RNN family takes care of the decoding. Initializing the decoder in the model is also similar to initializing the encoder. If we want an LSTM model as the decoder, we can do the following:

```
self.decoder = nn.LSTM(
    **decoder_params,
    batch_first=True,
)
```


Let's develop our understanding of how this is done through a visual representation:

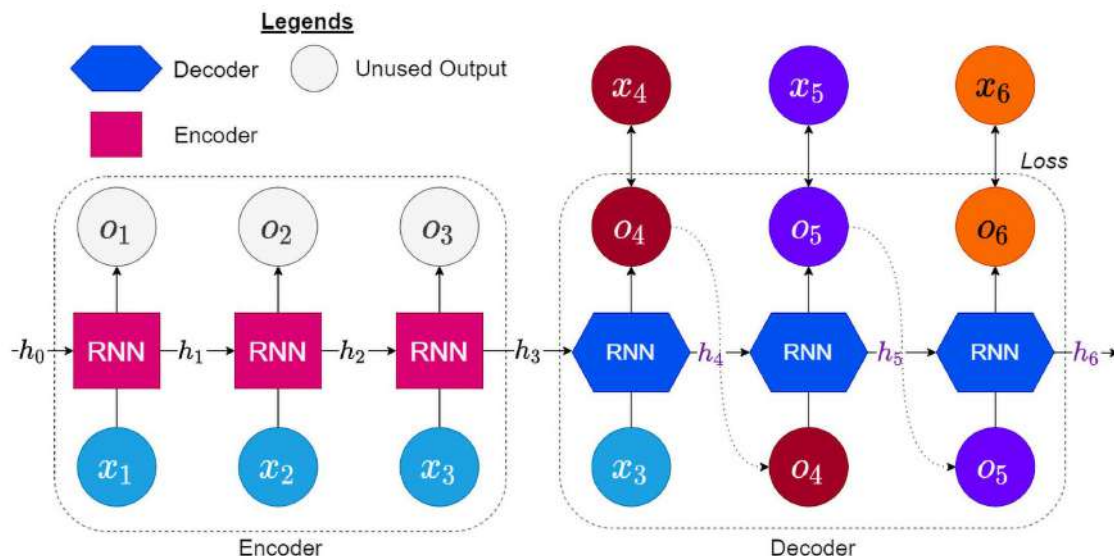


Figure 13.9: RNN as the encoder and decoder

The encoder part remains the same: it takes in the input window, x_1 to x_3 , and produces outputs, o_1 to o_3 , and the last hidden state, h_3 . Now, we have another decoder (a model from the RNN family) that takes in h_3 as the initial hidden state, and the latest input from the window to produce the next output. And now, this output is fed back into the RNN as the input and we produce the next output, and this cycle continues until we have got the required number of timesteps in our prediction.

Some of you may be wondering why we don't use the target window (x_4 to x_6) during decoding as well. In fact, this is a valid way of training the model and is called **teacher forcing** in the literature. This has strong connections to maximum likelihood and is explained well in the *Deep Learning* book by Goodfellow et al. (see the *Further reading* section). So, instead of feeding in the output of the model from the previous timestep as the input to the RNN at the current timestep, we feed in the real observation, thereby eliminating the error that might have crept in in the previous timestep.

While this sounds like the most straightforward thing to do, it does come with a few disadvantages as well. The main one is that the kinds of inputs that the decoder sees during training may not be the same as the ones it will see during actual prediction. During prediction, we will still be feeding the output of the model in the previous step to the decoder. This is because in the inference mode, we do not have access to real observations in the future. This can cause problems in some cases. One way to mitigate this problem is to randomly choose between the model's output at the previous timestep and real observation while training (Bengio et al., 2015).



Reference check:

The research paper by Bengio et al., which proposed teacher forcing, is cited in reference 1.

Now, let's see how we can code the forward method for both of these cases using a parameter called `teacher_forcing_ratio`, which is a decimal from 0 to 1. This decides how frequently teacher forcing is implemented. For instance, if `teacher_forcing_ratio=0`, then teacher forcing is never done, and if `teacher_forcing_ratio=1`, then teacher forcing is always done.

The following code block has all the code necessary for decoding, and it comes with line numbers so that we can go line by line and explain what we are doing:

```

01 y_hat = torch.zeros_like(y, device=y.device)
02 dec_input = x[:, -1:, :]
03 for i in range(y.size(1)):
04     out, h = self.decoder(dec_input, h)
05     out = self.fc(out)
06     y_hat[:, i, :] = out.squeeze(1)
07     #decide if we are going to use teacher forcing or not
08     teacher_force = random.random() < teacher_forcing_ratio
09     if teacher_force:
10         dec_input = y[:, i, :].unsqueeze(1)
11     else:
12         dec_input = out

```

The first thing we need to do is declare a placeholder to store the desired output during decoding. In *line number 1*, we do that by using `zeros_like`, which generates a tensor with all zeros with the same dimension as `y`, and in *line number 2*, we set the initial input to the decoder as the last timestep in the input window. Now, we are all set to start the decoding process, and for that, in *line number 3*, we start a loop to run `y.size(1)` times. If you remember the dimensions of `y`, the second dimension was the sequence length, so we need to run the decoding process that many times.

In *line number 4*, we pass in the last input from the input window and the hidden state from the encoder to the decoder, and it returns the current output and the hidden state. We capture the current hidden state in the same variable, overwriting the old one. If you remember, the output from the RNN is the hidden state, and we will need to pass it through a fully connected layer for the prediction. So, in *line number 5*, we do just that. In *line number 6*, we store the output from the fully connected layer to the *i*-th timestep in `y_hat`.

Now, we just have one more thing to do—decide whether to use teacher forcing or not and move on to decoding the next timestep. This we can do by generating a random number between 0 and 1 and checking whether that number is less than the `teacher_forcing_ratio` parameter or not. `random.random()` samples a number from a uniform distribution between 0 and 1. If the `teacher_forcing_ratio` parameter is 0.5, checking whether `random.random()<teacher_forcing_ratio` automatically ensures we only do teacher forcing 50% of the time. So, in *line number 8*, we do this check and get a Boolean output, `teacher_force`, which tells us whether we need to do teacher forcing in the next timestep or not. For teacher forcing, we store the current timestep from `y` as `dec_input` (*line number 10*). Otherwise, we store the current output as `dec_input` (*line number 12*), and this `dec_input` parameter is used as the input to the RNN in the next timestep.

Now, all of this (both the fully connected decoder and the RNN decoder) has been put together into a single class called `Seq2SeqModel` in `src/dl/models.py`, and a config class (`Seq2SeqConfig`) has also been defined that has all the options and hyperparameters of the models. Let's take a look at the different parameters in the config:

- `encoder_type`: A string parameter that takes in one of three values: RNN, LSTM, or GRU. This decides the sequence model we need to use as the encoder.
- `decoder_type`: A string parameter that takes in one of four values: RNN, LSTM, GRU, or FC (for *fully connected*). This decides the sequence model we need to use as the decoder.
- `encoder_params` and `decoder_params`: These parameters take a dictionary of key-value pairs as the input. These are the hyperparameters of the encoder and the decoder, respectively. For the RNN family of models, there is another config class, `RNNConfig`, which sets standard hyperparameters such as `hidden_size` and `num_layers`. And for the FC decoder, we need to give two parameters: `window_size` as the number of timesteps included in the input window, and `horizon` as the number of timesteps ahead we want to be forecasting.
- `decoder_use_all_hidden`: We discussed two ways we can use the fully connected decoder. This parameter is a flag that switches between the two. If `True`, the fully connected decoder will flatten the hidden states of all timesteps and use them for the prediction, and if `False`, it will just use the last hidden state.
- `teacher_forcing_ratio`: We discussed teacher forcing earlier, and this parameter decided the strength of teacher forcing while training. If `0`, there will be no teacher forcing, and if `1`, every timestep will be teacher-forced.
- `optimizer_params`, `lr_scheduler`, and `lr_scheduler_params`: These are parameters that let us tweak the optimization procedure. Let's not worry about these for now because all of them have been set to intelligent defaults.

Now, with this config and the model, let's run a few experiments. These work exactly the same as the set of experiments we ran in the previous section. The exact code for the experiments is available in the accompanying notebook. So, we ran the following experiments:

- `LSTM_FC_last_hidden`: Encoder = LSTM/Decoder = Fully Connected, using just the last hidden state
- `LSTM_FC_all_hidden`: Encoder = LSTM/Decoder = Fully Connected, using all the hidden states
- `LSTM_LSTM`: Encoder = LSTM/Decoder = LSTM

Let's see how they performed on the metrics we have been tracking:

Algorithm	MAE	MSE	MASE	Forecast Bias
Lasso Regression	0.1598	0.0743	1.2452	3.78%
XGB Random Forest	0.1641	0.0819	1.2792	9.30%
LightGBM	0.1470	0.0666	1.1457	3.36%
RNN	0.2685	0.1721	2.0927	29.35%
LSTM	0.1982	0.1125	1.5442	17.94%
GRU	0.1714	0.0899	1.3358	14.48%
LSTM_FC_last_hidden	0.1642	0.0815	1.2797	5.87%
LSTM_FC_all_hidden	0.1667	0.0799	1.2993	10.00%
LSTM_LSTM	0.1600	0.0795	1.2472	13.31%

Figure 13.10: Metrics for Seq2Seq models on MAC000193 household

The Seq2Seq models seem to be performing better on the metrics and the LSTM_LSTM model is even better than the Random Forest model.

There are visualizations of each of these forecasts in the notebook. I urge you to look at those visualizations, zoom in, look at different places in the horizon, and so on. The astute observers among you must have figured out something weird with the forecast. Let's look at a zoomed-in version (on one day) of the forecasts we generated to make that point clear:

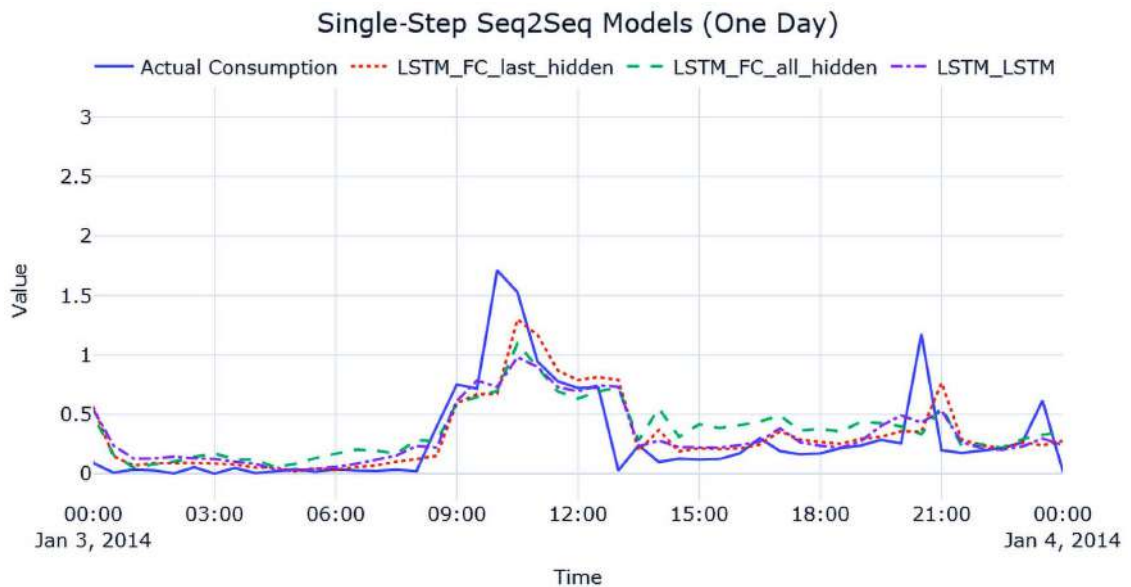


Figure 13.11: Single-step-ahead Seq2Seq predictions for MAC000193 household (one day)

What do you see now? Focus on the peaks in the time series. Are they aligned or do they seem at an offset? This phenomenon that you are seeing now is when a model learns to mimic the last seen timestep (like the naïve forecast) rather than learn the true pattern in the data. We will be getting good metrics and we might be happy with the forecast, but upon investigation, we can see that this is not the forecast we want. This is especially true in the case of single-step-ahead models where we are just optimizing to predict the next timestep. Therefore, the model has no real incentive to learn long-term patterns, such as seasonality and so on, and ends up learning a model like the naïve forecast.

Models that are trained to predict longer horizons overcome this problem because, in this scenario, the model is forced to learn the longer-term patterns in the model. Although multi-step forecasting is a topic that will be covered in detail in *Chapter 18, Multi-Step Forecasting*, let's get a little bit of a sneak peek now. In the notebook, we also train multi-step models using the Seq2Seq models.

The only changes we need to make are these:

- The horizon we define in the datamodule and the models should change.
- The way we evaluate the models should also have a small change.

Let's see how we can define a datamodule for multi-step forecasting. We have chosen to forecast a complete day, which is 48 timesteps. And as an input window, we are giving 2 X 48 timesteps:

```
HORIZON = 48
WINDOW = 48*2
datamodule = TimeSeriesDataModule(data = sample_df[[target]],
    n_val = sample_val_df.shape[0],
    n_test = sample_test_df.shape[0],
    window = WINDOW,
    horizon = HORIZON,
    normalize = "global", # normalizing the data
    batch_size = 32,
    num_workers = 0)
```

Now that we have the datamodule, we can initialize the models just like before and train them. The only change we have to make now is while predicting.

In the single-step setting, at each timestep, we were predicting the next one. But now, we are predicting the next 48 timesteps at each step. There are multiple ways to look at this and measure the metrics, which we will cover in detail in *Part 3*. For now, let's choose a heuristic and say that we are considering that we are running this model only once a day, and each such prediction has 48 timesteps. But the test dataloader is still incremented by one—in other words, the test dataloader still gives us the next 48 timesteps, for each timestep. So, executing the following code, we will get a prediction array with dimensions (*timesteps*, *horizon*):

```
pred = trainer.predict(model, datamodule.test_dataloader())  
# pred is a list of outputs, one for each batch  
pred = torch.cat(pred).squeeze().detach().numpy()
```

The predictions start at 2014, Jan 1 00:00:00. So, if we select the 48 timesteps, every 48 timesteps apart, it'll be like considering only predictions that are made at the beginning of the day. Using a bit of fancy indexing numpy provides us, it is easy to do just that:

```
pred = pred[0:48].ravel()
```

We start at index 0, which is the first prediction of 48 timesteps, and pick every 48 indices (which are timesteps) and just flatten the array. We will get an array of predictions with the desired shape, and then the standard procedure of inverse transformation and metric calculation, and so on, proceeds.

The notebook has the code to do the following experiments:

- MultiStep LSTM_FC_last_hidden: Encoder = LSTM/Decoder = Fully Connected Layer, using only the last hidden state
- MultiStep LSTM_FC_all_hidden: Encoder = LSTM/Decoder = Fully Connected Layer, using all the hidden states
- MultiStep LSTM_LSTM_teacher_forcing_0.0: Encoder = LSTM/ Decoder = LSTM, using no teacher forcing
- MultiStep LSTM_LSTM_teacher_forcing_0.5: Encoder = LSTM/ Decoder = LSTM, using stochastic teacher forcing (randomly, 50% of the time teacher forcing is enabled)
- MultiStep LSTM_LSTM_teacher_forcing_1.0: Encoder = LSTM/ Decoder = LSTM, using complete teacher forcing

Let's look at the metrics of these experiments:

Algorithm	MAE	MSE	MASE	Forecast Bias
Lasso Regression	0.1598	0.0743	1.2452	3.78%
XGB Random Forest	0.1641	0.0819	1.2792	9.30%
LightGBM	0.1470	0.0666	1.1457	3.36%
RNN	0.2685	0.1721	2.0927	29.35%
LSTM	0.1982	0.1125	1.5442	17.94%
GRU	0.1714	0.0899	1.3358	14.48%
LSTM_FC_last_hidden	0.1642	0.0815	1.2797	5.87%
LSTM_FC_all_hidden	0.1667	0.0799	1.2993	10.00%
LSTM_LSTM	0.1600	0.0795	1.2472	13.31%
MultiStep LSTM_FC_last_hidden	0.2177	0.1305	1.6967	10.59%
MultiStep LSTM_FC_all_hidden	0.2344	0.1317	1.8265	9.33%
MultiStep LSTM_LSTM_teacher_forcing_0.0	0.2058	0.1241	1.6039	15.32%
MultiStep LSTM_LSTM_teacher_forcing_0.5	0.1866	0.0997	1.4544	11.90%
MultiStep LSTM_LSTM_teacher_forcing_1	0.1754	0.0912	1.3671	12.93%

Figure 13.12: Metrics for multi-step Seq2Seq models on MAC000193 household

Although we cannot compare single-step-ahead accuracy to multi-step ones, for the time being, let's suspend that concern and use the single-step metrics as in the best-case scenario. So, we can see that our model that predicts one day ahead (48 timesteps) is not such a bad model after all, and if we visualize the predictions, the problem of imitating naïve forecasts is also not present because now the model is forced to learn long-term models and forecasts:

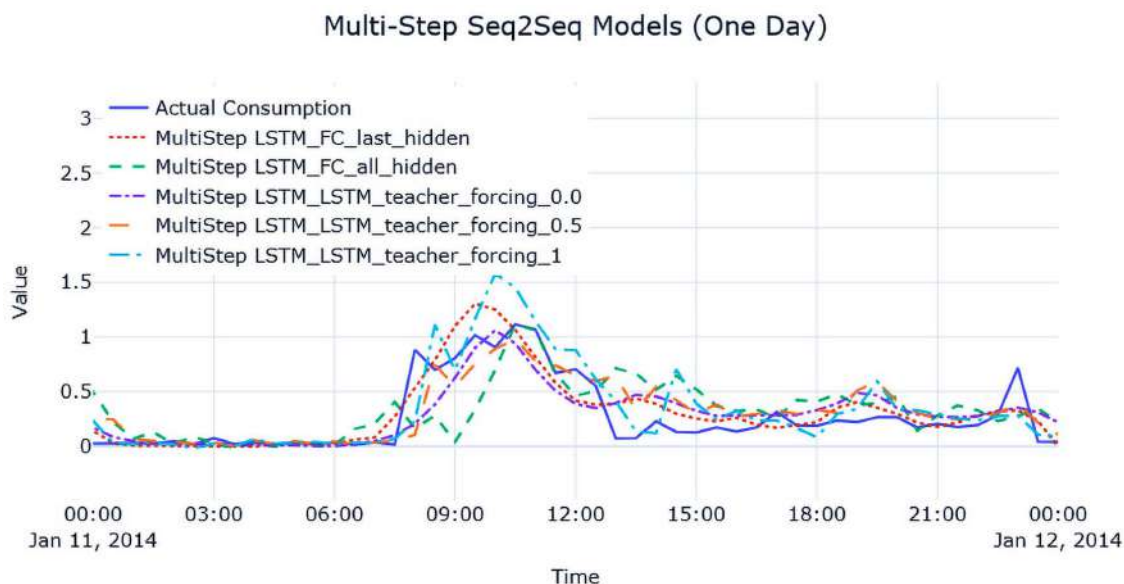


Figure 13.13: Multi-step-ahead Seq2Seq predictions for MAC000193 household (one day)

We can see that the model has tried to learn the daily patterns because it is forced to predict the next 48 timesteps. With some tuning and other training tricks, we might get a better model as well. But running a separate model for all LCLid (consumer ID) instances in the dataset may not be the best option, both from an engineering and a modeling perspective. We will tackle strategies for global modeling in *Chapter 15, Strategies for Global Deep Learning Forecasting Models*.



Things to try:

Can you train a better model? Tweak the hyperparameters and try to get better performance. Use GRUs or combine a GRU with an LSTM—the possibilities are endless.

Congratulations on getting through yet another hands-on and practical chapter. If this is the first time you are training NNs, I hope this lesson has made you confident enough to try more: trying and experimenting with these techniques is the best way to learn. There is no silver bullet for all datasets in ML, and therefore it is up to us practitioners to keep our options open and choose the right algorithm/model that suits our use case and works well in it. In this dataset, we can see that for single-step forecasting, LightGBM works really well. But the LSTM Seq2Seq model worked almost as well. When we extend to the multi-step forecasting scenario, the advantage of having a single model doing multi-step forecast with good enough performance may beat managing multiple ML models (more on this in *Chapter 18*). The techniques we learned are still considered basic in the DL world and in the subsequent chapters, we will dive deeper into the DL sea and learn about more sophisticated approaches.

Summary

Although we learned about the basic blocks of DL in the previous chapter, we put all of that into action while we used those blocks in common modeling patterns using PyTorch.

We saw how standard sequence models such as RNN, LSTM, and GRU can be used for time series prediction, and then we moved on to another paradigm of models, called Seq2Seq models. Here, we talked about how we can mix and match encoders and decoders to get the model we want. Encoders and decoders can be arbitrarily complex. Although we looked at simple encoders and decoders, it is certainly possible to have something like a combination of a convolution block and an LSTM block working together for the encoder. Last but not least, we talked about teacher forcing and how it can help models train and converge faster and also with some performance boost.

In the next chapter, we will be tackling a subject that has captured a lot of attention (pun intended) in the past few years: attention and transformers.

Reference

1. Samy Bengio, Oriol Vinyals, Navdeep Jaitly, and Noam Shazeer (2015). *Scheduled Sampling for Sequence Prediction with Recurrent Neural Networks*. *Proceedings of the 28th International Conference on Neural Information Processing Systems—Volume 1 (NIPS'15)*: <https://proceedings.neurips.cc/paper/2015/file/e995f98d56967d946471af29d7bf99f1-Paper.pdf>.

Further reading

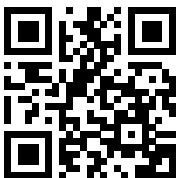
Check out the following sources for further reading:

- *From PyTorch to PyTorch Lightning*—Alfredo Canziani and William Falcon: <https://www.youtube.com/watch?v=DbESHcCOWbM>
- *Deep Learning*—Ian Goodfellow, Yoshua Bengio, and Aaron Courville (pages 376-377): <https://www.deeplearningbook.org/contents/rnn.html>
- *A Short Chronology Of Deep Learning For Tabular Data* by Sebastian Raschka: <https://sebastianraschka.com/blog/2022/deep-learning-for-tabular-data.html>

Join our community on Discord

Join our community's Discord space for discussions with authors and other readers:

<https://packt.link/mts>



14

Attention and Transformers for Time Series

In the previous chapter, we rolled up our sleeves and implemented a few **deep learning** (DL) systems for time series forecasting. We used the common building blocks we discussed in *Chapter 12, Building Blocks of Deep Learning for Time Series*, put them together in an encoder-decoder architecture, and trained them to produce the forecast we desired.

Now, let's talk about another key concept in DL that has taken the field by storm over the past few years—**attention**. Attention has a long-standing history, which has culminated in it being one of the most sought-after tools in the DL toolkit. This chapter takes you on a journey to understand attention and transformer models from the ground up from a theoretical perspective and solidify that understanding with practical examples.

In this chapter, we will be covering these main topics:

- What is attention?
- Generalized attention model
- Forecasting with sequence-to-sequence models and attention
- Transformers—Attention is all you need
- Forecasting with Transformers

Technical requirements

You will need to set up the **Anaconda** environment by following the instructions in the *Preface* of the book to get a working environment with all the libraries and datasets required for the code in this book. Any additional libraries will be installed while running the notebooks.

You need to run the following notebooks for this chapter:

- 02-Preprocessing_London_Smart_Meter_Dataset.ipynb in Chapter02
- 01-Setting_up_Experiment_Harness.ipynb in Chapter04

- 01-Feature_Engineering.ipynb in Chapter06
- 02-One-Step_RNN.ipynb and 03-Seq2Seq_RNN.ipynb in Chapter13 (for benchmarking)
- 00-Single_Step_Backtesting_Baselines.ipynb and 01-Forecasting_with_ML.ipynb in Chapter08

The associated code for the chapter can be found at <https://github.com/PacktPublishing/Modern-Time-Series-Forecasting-with-Python-/tree/main/notebooks/Chapter14>.

What is attention?

The idea of attention was inspired by human cognitive function. At any moment, the optic nerves in our eyes, the olfactory nerves in our noses, and the auditory nerves in our ears send a massive amount of sensory input to the brain. This is way too much information, definitely more than the brain can handle. But our brains have developed a mechanism that helps us to pay *attention* to only the stimuli that matter—such as a sound or a smell that doesn't belong. Years of evolution have *trained* our brains to pick out anomalous sounds or smells because that was key for us surviving in the wild, where predators roamed free. In cognitive science, attention is defined as the cognitive process that allows an individual to selectively focus on specific information while ignoring other irrelevant stimuli.

Apart from this kind of instinctive attention, we are also able to control our attention by what we call *focusing* on something. You are doing it right now by choosing to ignore all the other stimuli that you are getting and focusing your attention on the contents of this book. While you are reading, your mobile phone pings you, the screen lights up, and your brain decides to focus its attention on the mobile screen, even though the book is still open in front of you. This feature of the human cognitive function has been the inspiration behind the attention mechanism in DL. Giving learning machines the ability to acquire this kind of attention has led to big breakthroughs in all fields of AI today.

The idea was first applied to DL in Seq2Seq models, which we learned about in *Chapter 13, Common Modeling Patterns for Time Series*. In that chapter, we saw how the handshake between the encoder and decoder was done. For the **recurrent neural network (RNN)** family of models, we use the hidden states from the encoder at the end of the sequence as the initial hidden states in the decoder. Let's call this handshake the **context**. The assumption here is that all the information required for the decoding task is encoded in the context and this is done in a timestep-by-timestep manner. So, for long context windows, the information from the first timestep has to be retained through multiple writes and re-writes until it's used in the last timestep. This creates a kind of information bottleneck (*Figure 14.1*), where the model may struggle to retain important information through this limited context. There may be information in previous hidden states that can be useful for the decoding task. In 2015, Bahdanau et al. (Reference 1) proposed the first known attention model in the context of DL. They proposed to learn attention weights, α , for each hidden state corresponding to the input sequence and combine them into a single context vector while decoding.

These attention weights are re-calculated for each decoding step using the similarity between the hidden states during decoding and all the hidden states in the input sequence (Figure 14.2):

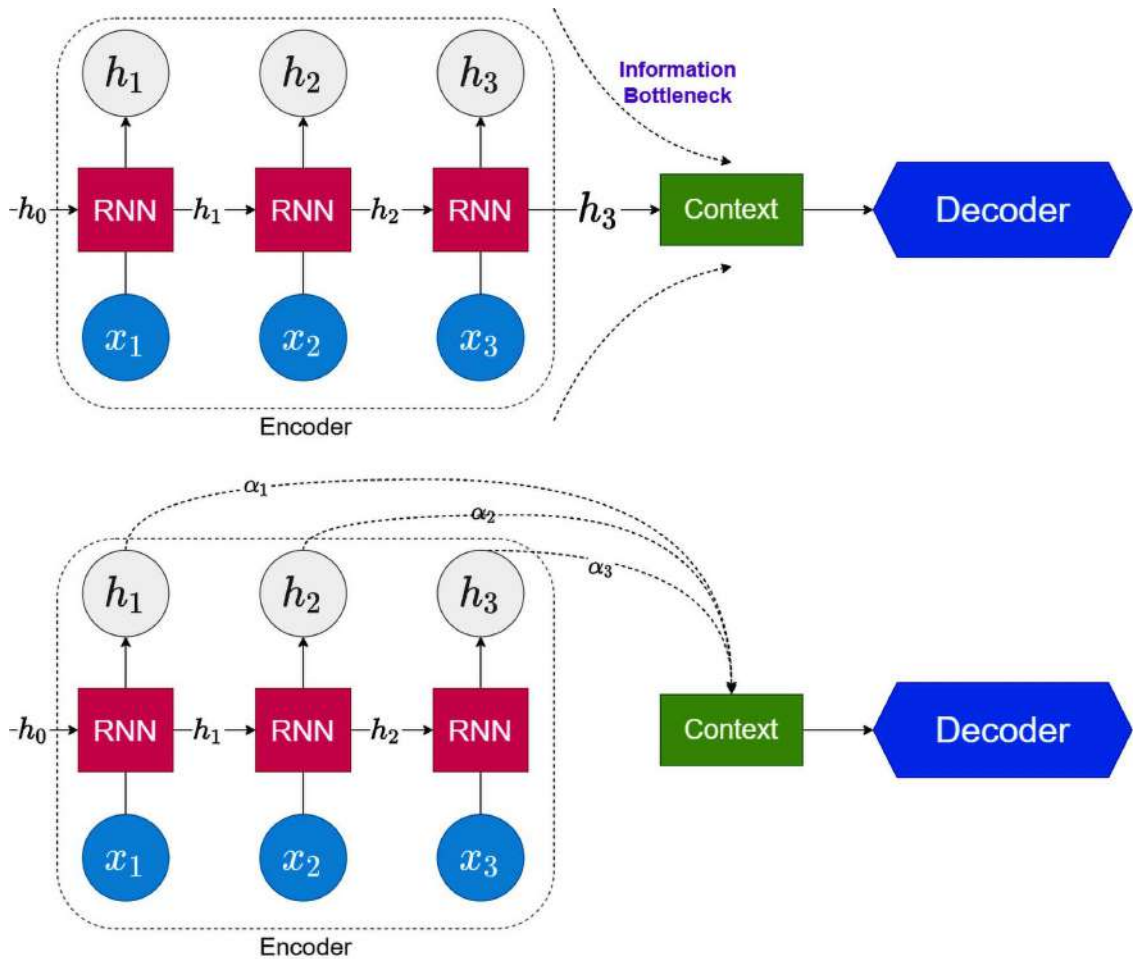


Figure 14.1: Traditional (top) versus attention model (bottom) in Seq2Seq models

To make things clearer, let's adopt a formal way of describing the mechanism. Let $H = h_i, i \in \{1, 2, \dots, T_s\}$ be the hidden states generated during the encoding process and $S = s_j, j \in \{1, 2, \dots, T_t\}$ be the hidden states generated during decoding. The context vector will be c_i :

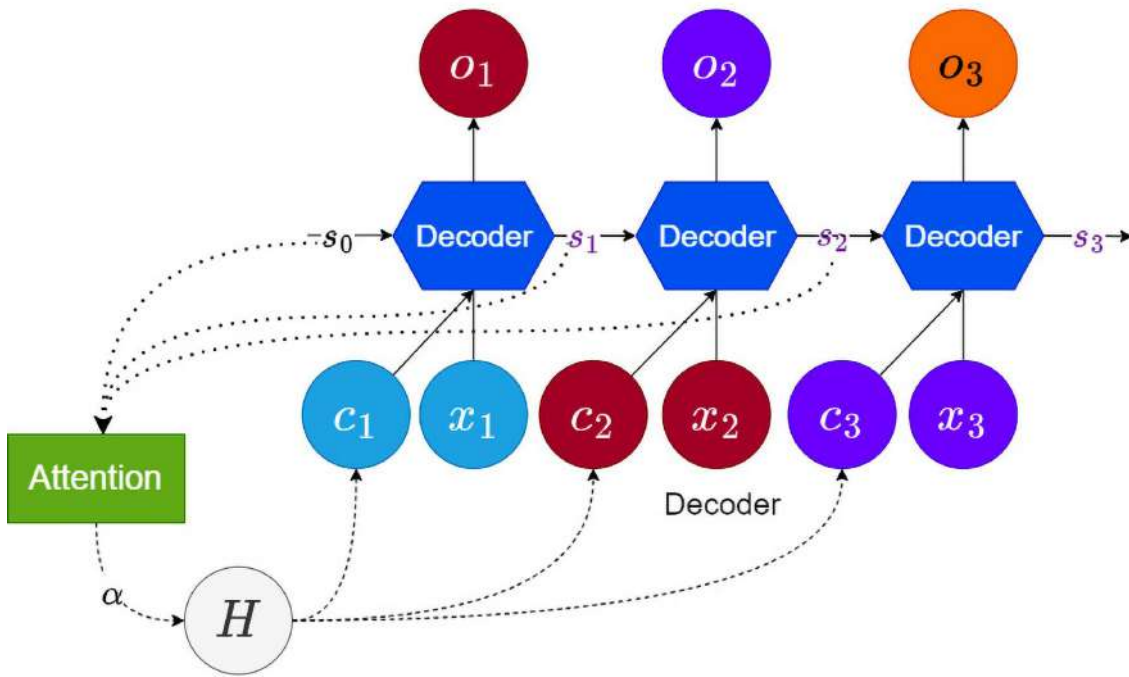


Figure 14.2: Decoding using attention

So, now we have the hidden states from the encoding stage (H), and we need to have a way to use this information in each step of decoding. The key here is that in each step of the decoding process, information from different hidden states might be relevant. This is exactly what attention weights do. So, for decoding step j , we use s_{j-1} and calculate attention weights (we'll look at how attention weights are learned in detail soon), α_i, j , using the similarity between s_{j-1} and each hidden state in H . Now, we calculate the context vector, which combines the information in H in the right way:

$$c_j = \sum_{i=1}^{T_s} \alpha_{i,j} h_i$$

There are two main ways we can use this context vector, which we will look at in more detail later in the chapter. This breaks the information bottleneck that was present in the traditional Seq2Seq model and allows the models to access a larger pool of information and decide which information is relevant at each step of the decoding process.

Now, let's see how these attention weights, α , are calculated.

The generalized attention model

Over the course of years, researchers have come up with different ways of calculating attention weights and using attention in DL models. Sneha Chaudhari et al. (Reference 8) published a survey paper on attention models that proposes a generalized attention model that tries to incorporate all the variations in a single framework. Let's structure our discussion around this generalized framework.

We can think of an attention model as learning an attention distribution (α) for a set of keys, K , using a set of queries, q . In the example we discussed in the last section, the query would be S_{j-1} —the hidden state from the last timestep during decoding—and the keys would be H —all the hidden states generated using the input sequence. In some cases, the generated attention distribution is applied to another set of inputs called values, V . In many cases, K and V are the same, but to maintain the general form of the framework, we consider these separately. Using this terminology, we can define an attention model as a function of q , K , and V :

$$A(q, K, V) = \sum_i p(a(k_i, q)) \times v_i$$

Here, a is an **alignment function** that calculates a similarity or a notion of similarity between the query (q) and keys (k_i), and v_i is the corresponding value for index i . In the example we discussed in the previous section, this alignment function calculates how relevant an encoder hidden state is to a decoder hidden state, and p is a **distribution function** that converts this score into attention weights that sum up to 1.



Reference check:

The research papers by Sneha Choudhari et al. are cited in the *References* section as reference 8.

Now that we have the generalized attention model, let's also see how we can implement this in PyTorch. The full implementation can be found in the `Attention` class in `src/dl/attention.py`, but we will cover the key parts of it here.

The only information we require beforehand to initialize such a module is the hidden dimension of the queries and keys (encoder and decoder). So, the class definition and the `__init__` function of the class look like this:

```
class Attention(nn.Module, metaclass=ABCMeta):
    def __init__(self, encoder_dim: int, decoder_dim: int):
        super().__init__()
        self.encoder_dim = encoder_dim
        self.decoder_dim = decoder_dim
```

Now, we need to define a forward function, which takes in two inputs:

- **query:** The query vector of size (*batch size, decoder dimension*), which we are going to use to find the attention weights with which to combine the keys. This is the q in $A(q, K, V)$.
- **key:** The key vector of size (*batch size, sequence length, encoder dimension*), which is the sequence of hidden states across which we will be calculating the attention weights. This is the K in $A(q, K, V)$.

We are assuming keys and values are the same because, in most cases, they are. So, from the generalized attention model, we know that there are a few steps we need to perform:

1. Calculate an alignment score— $a(k_i, q)$ —for each query and key combination.
2. Convert the scores to weights by applying a function— $p(a(k_i, q))$.
3. Use the learned weights to combine the values— $\sum_i p(a(k_i, q)) \times v_i$.

So, let's see those steps in code in the forward method:

```
def forward(
    self,
    query: torch.Tensor, # [batch_size, decoder_dim]
    values: torch.Tensor, # [batch_size, seq_length, encoder_dim]
):
    scores = self._get_scores(query, values) # [batch_size, seq_length]
    weights = torch.nn.functional.softmax(scores, dim=-1)
    return (values*weights.unsqueeze(-1)).sum(dim=1) # [batch_size,
encoder_dim]
```

The three lines of code in the forward method correspond to the three steps we discussed earlier. The first step, which is calculating the scores, is the key step that has led to many different types of attention, and therefore we have generalized that into a `_get_scores` abstract method that must be implemented by any class inheriting the `Attention` class. For the second line, we have used the `softmax` function for converting the scores to weights, and in the last line, we are doing an element-wise multiplication (*) between weights and values and summing across the sequence length to get the weighted value.

Now, let's turn our attention toward alignment functions.

Alignment functions

There are many variations of the alignment function that have come up over the years. Let's review a few popular ones that are used today.

Dot product

This is probably the simplest alignment function of all. Luong et al. proposed this form of attention in 2015. From linear algebra, we know that the dot product of two vectors tells us what amount of one vector goes in the direction of the other. It measures some kind of similarity between the two vectors, and this similarity considers both the magnitude of the vectors and the angle between them in the vector space. Therefore, when we take the dot product of our query and key vectors, we get a notion of similarity between them. One thing to note here is that the hidden dimensions of the query and the key should be the same for dot product attention to be applied. Formally, the similarity function can be defined as follows:

$$a(k_i, q) = q^T \cdot k_i$$

We need to calculate this score for each of the elements, K_i , in the key, K , and instead of running a loop over each element in K , we can use a clever matrix multiplication trick to calculate the scores for all the keys in K in one shot. Let's see how we can define the `_get_scores` function for dot product attention.

We know from the previous section that the query and values (which are the same as keys in our case) are of *(batch size, decoder dimension)* and *(batch size, sequence length, encoder dimension)* dimensions respectively, and will be called q and v in the `_get_scores` function. In this particular case, the decoder dimension and the encoder dimension are the same, so the scores can be calculated as follows:

```
scores = (q @ v.transpose(1,2))
```

Here, `@` is shorthand for `torch.matmul`, which does matrix multiplication. The entire implementation is named `DotProductAttention` and can be found in `src/dl/attention.py`.

Scaled dot product attention

In 2017, Vaswani et al. proposed this type of attention in the seminal paper, *Attention Is All You Need*. We will delve into that paper later in this chapter, but now, let's understand one key modification they suggested to the dot product attention. The modification is motivated by the concern that when the input is large, the *softmax* function we use to convert scores to weights may have very small gradients, which makes efficient learning difficult.

This is because the *softmax* function is not scale-invariant. The exponential function in the *softmax* function is the reason for this behavior. So, the higher we scale the inputs to the function, the more the largest input dominates the output, and this throttles the gradient flow in the network. If we assume q and v are d_k dimensional vectors with 0 mean and a variance of 1, then their dot product would have a mean of zero and a variance of d_k . Therefore, if we scale the output of the dot product by $\sqrt{d_k}$, then we bring the variance of the dot product back to 1. So, by controlling for the scale of the inputs to the *softmax* function, we manage a healthy gradient flow through the network. The *Further reading* section has a link to a blog post that goes into this in more depth. Therefore, the scaled dot product alignment function can be defined as follows:

$$a(k_i, q) = \frac{1}{\sqrt{d_k}} \times q^T k_i$$

Consequently, the only change we will have to make to the PyTorch implementation is one additional line:

```
scores = scores/math.sqrt(encoder_dim)
```

This has been implemented as a parameter in `DotProductAttention` in `src/dl/attention.py`. If you pass `scaled=True` while initializing the class, it will perform scaled dot product attention. We need to keep in mind that, similar to dot product attention, the scaled variant also requires the query and values to have the same dimensions.

General attention

In 2015, Luong et al. (Reference 2) proposed a slight variation of dot product attention by introducing a learnable W matrix into the calculation. They called it general attention. We can think of it as an attention mechanism that allows the query to be projected into a learned plane of the same dimension as the values/keys using the W matrix before computing the similarity score using a dot product. The alignment function can be written as follows:

$$a(k_i, q) = q^T W k_i$$

The corresponding PyTorch implementation can be found under the name `GeneralAttention` in `src/dl/attention.py`. The key line calculating the attention scores can be written as follows:

```
scores = (q @ self.W) @ v.transpose(1,2)
```

Here, `self.W` is a tensor of size (*encoder hidden dimension x decoder hidden dimension*). General attention can be used in cases where the query and key/value dimensions are different.

Additive/concat attention

In 2015, Bahdanau et al. proposed additive attention, which was one of the first attempts at introducing attention to DL systems. Instead of using a defined similarity function such as the dot product, Bahdanau et al. proposed that the similarity function can be learned, giving the network more flexibility in deciding what it deems to be similar. They suggested that we can concatenate the query and the key into a single tensor and use a learnable matrix, W , to calculate the attention scores. This alignment function can be written as follows:

$$a(k_i, q) = w_{imp}^T \tanh(W_q q^T + W_k k_i + b)$$

Here, v , W_q , and W_k are learnable matrices. In cases where the query and key have different hidden dimensions, we can use W_q and W_k to project them into a single dimension and then perform a similarity calculation on them. If the query and key have the same hidden dimension, this is also equivalent to the variant of attention used in Luong et al., which they call *concat* attention, represented as follows:

$$a(k_i, q) = w_{imp}^T \tanh(W[q^T; k_i] + b)$$

It is simple linear algebra to see that both are the same and for engineering simplicity. The *Further reading* section has a link to a Stack Overflow answer that explains the equivalence.

We have included both implementations in `src/dl/attention.py` under `ConcatAttention` and `AdditiveAttention`.

For `AdditiveAttention`, the key lines calculating the score are as follows:

```
q = q.repeat(1, v.size(1), 1) # [batch_size, seq_length, decoder_dim]
scores = self.W_q(q) + self.W_v(v) # [batch_size, seq_length, decoder_dim]
torch.tanh(scores) @ self.v # [batch_size, seq_length]
```

The first line repeats the query vector to the sequence length. This is just a linear algebra trick to calculate the score for all the encoder hidden states in a single operation rather than looping through them. *Line 2* projects both query and value into the same dimension using `self.W_q` and `self.W_v`, and *line 3* applies the `tanh` activation function and uses matrix multiplication with `self.v` to produce the final scores. `self.W_q`, `self.W_v`, and `self.v` are learnable matrices, defined as follows:

```
self.W_q = torch.nn.Linear(self.decoder_dim, self.decoder_dim)
self.W_v = torch.nn.Linear(self.encoder_dim, self.decoder_dim)
self.v = torch.nn.Parameter(torch.FloatTensor(self.decoder_dim))
```

The only difference in `ConcatAttention` is that instead of two separate weights—`self.W_q` and `self.W_v`—we just have a single weight—`self.W`—defined as follows:

```
self.W = torch.nn.Linear(self.decoder_dim + self.encoder_dim, self.decoder_dim)
```

And instead of adding the projections (*line 2*), we use the following line:

```
scores = self.W(
    torch.cat([q, v], dim=-1)
) # [batch_size, seq_length, decoder_dim]
```

Therefore, we can think of `AdditiveAttention` and `ConcatAttention` doing the same operation, but `AdditiveAttention` is adapted to handle different encoder and decoder dimensions.



Reference check:

The research papers by Luong et al., Bahdanau et al., and Vaswani et al. are cited in the *References* section as references 2, 1, and 5 respectively.

Now that we have learned about a few popular alignment functions, let's turn our attention toward the distribution function of the attention model.

The distribution function

The primary goal of the distribution function is to convert the learned scores from the alignment function into a set of weights that add up to 1. The *softmax* function is the most popular choice as a distribution function. It converts the score into a set of weights that sum up to one. This also gives us the freedom to interpret the learned weights as probabilities—the probability that the corresponding element is the most relevant.

Although *softmax* is the most popular choice, it is not without its drawbacks. The *softmax* weights are typically *dense*. What that means is that a probability mass (some weight) will be assigned to every element in the sequence over which we calculated the attention. The weights can be low, but still not 0. There are situations where sparsity in the distribution function is desirable. Maybe we want to make sure we don't give any weights some implausible options. Maybe we want to make the attention mechanism more interpretable.

There are alternate distribution functions such as `sparsemax` (Martins et al. 2016, Reference 3) and `entmax` (Peters et al. 2019, Reference 4) that are capable of assigning probability mass to a select few relevant elements and assigning zero to the rest of them. When we know that the output is only dependent on a few timesteps in the encoder, we can use such distribution functions to encode that knowledge into the model. Space activation functions like `Sparsemax` have the advantage of interpretability, as they provide a clearer distinction between the elements that are important (non-zero probabilities) and those that are not (zero probabilities).



Reference check:

The research papers by Martins et al. and Peters et al. are cited in the *References* section as references 3 and 4 respectively.

Now that we have learned about a few attention mechanisms, it's time to put them into practice.

Forecasting with sequence-to-sequence models and attention

Let's pick up the thread from *Chapter 13, Common Modeling Patterns for Time Series*, where we used `Seq2Seq` models to forecast a sample household (if you have not read the previous chapter, I strongly suggest you do it now) and modify the `Seq2SeqModel` class to also include an attention mechanism.



Notebook alert:

To follow along with the complete code, use the notebook named `01-Seq2Seq_RNN_with_Attention.ipynb` in the `Chapter14` folder and the code in the `src` folder.

We are still going to inherit the `BaseModel` class we have defined in `src/dl/models.py`, and the overall structure is going to be very similar to the `Seq2SeqModel` class. The key difference will be that in our new model, with attention, we do not accept a fully connected layer as the decoder. It is not because it is not possible, but for convenience and brevity of the implementation. In fact, implementing a `Seq2Seq` model with a fully connected decoder is something you can do on your own to really internalize the concept.

Similar to the `Seq2SeqConfig` class, we define a very similar `Seq2SeqAttnConfig` class that has the exact same set of parameters, but with some additional validation checks. One of the validation checks is disallowing a fully connected decoder. Another validation check would be making sure the decoder input size allows for the attention mechanism as well. We will see those requirements in detail shortly.

In addition to `Seq2SeqAttnConfig`, we also define a `Seq2SeqAttnModel` class to enable attention-enabled decoding. The only additional parameter here is `attention_type`, which is a string parameter that takes the following values:

- `dot`: Dot product attention
- `scaled dot`: Scaled dot product attention
- `general`: General attention

- additive: Additive attention
- concat: Concat attention

The entire code is available in `src/dl/models.py`. We will be covering only the forward function in detail in the book because that is the only place where there is a key difference. The rest of the class is about defining the right attention model based on input parameters and so on.

The encoder part is exactly the same as `SeqSeqModel`, which we saw in the last chapter. The only difference is in the decoding where we will be using attention.

Now, let's talk about how we are going to use the attention output in decoding.

As I mentioned before, there are two schools of thought on how to use attention while decoding. Using the same terminology we have been using for attention, let's see the difference between them.

Luong et al. use the decoder hidden state at step j , s_j , to calculate the similarity between itself and all the encoder hidden states, H , to calculate the context vector, c_j . This context vector, c_j , is then concatenated with the decoder hidden state, s_j , and this combined tensor is used as the input to the linear layer that generates the output.

Bahdanau et al. use attention in another way. They use the decoder hidden state from the previous timestep, s_{j-1} , and calculate the similarity with all the encoder hidden states, H , to calculate the context vector, c_j . And now, this context vector, c_j , is concatenated with the input to decoding step j , x_j . It is this concatenated input that is used in the decoding step that uses an RNN.

We can see the differences visually in Figure 14.3. The *Further reading* section also has another brilliant animation of attention in *Attn: Illustrated Attention*. This can also help you understand the mechanism well:

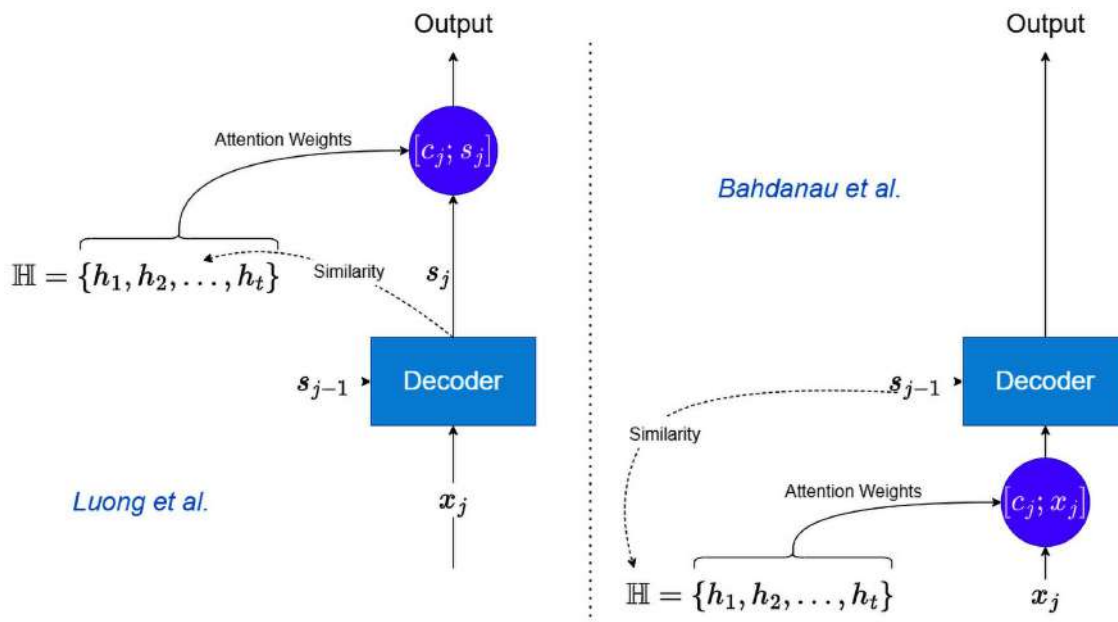


Figure 14.3: Attention-based decoding: Bahdanau versus Luong

In our implementation, we have chosen the Bahdanau way of decoding, where we use the concatenated context vector and input as the input for decoding. Because of that, there is a condition the decoder must satisfy: the `input_size` parameter of the decoder should be equal to the sum of the `input_size` parameter of the encoder and the `hidden_size` parameter of the encoder. This validation is built into `Seq2SeqAttnConfig`.

The following code block has all the code necessary for decoding with attention and has line numbers so that we can go line by line and explain what we are doing:

```

01     y_hat = torch.zeros_like(y, device=y.device)
02     dec_input = x[:, -1:, :]
03     for i in range(y.size(1)):
04         top_h = self._get_top_layer_hidden_state(h)
05         context = self.attention(
06             top_h.unsqueeze(1), o
07         )
08         dec_input = torch.cat((dec_input, context.unsqueeze(1)), dim=-1)
09         out, h = self.decoder(dec_input, h)
10         out = self.fc(out)
11         y_hat[:, i, :] = out.squeeze(1)
12         teacher_force = random.random() < self.hparams.teacher_forcing_
ratio
13         if teacher_force:
14             dec_input = y[:, i, :].unsqueeze(1)
15         else:
16             dec_input = out

```

Lines 1 and 2 are the same as in the `Seq2SeqModel` class where we set up the variable to store the prediction and extract the starting input to be passed to the decoder, and line 3 starts the loop for decoding step by step.

Now, in each step, we need to use the hidden state from the previous timestep to calculate the context vector. If you remember the output shapes of an RNN (*Chapter 12, Building Blocks of Deep Learning for Time Series*), we know that it is (number of layers, batch size, hidden size). But we need our query hidden state to be of the dimension (batch size, hidden size). Luong et al. suggested using the hidden states from the top layer of a stacked RNN model as the query, and we are doing just that here:

```
hidden_state[-1, :, :]
```

If the RNN is bi-directional, we would need to slightly alter the retrieval because now, the last two rows of the tensor would be the output from the last layer (one forward and one backward). There are many ways to combine them into a single tensor—we can concatenate them, we can sum them, or we can even mix them using a linear layer. Here, we just concatenate them:

```
torch.cat((hidden_state[-1, :, :], hidden_state[-2, :, :]), dim=-1)
```

Now that we have the hidden state, we use it as the query in the attention layer (*line 5*). In *line 8*, we concatenate the context with the input. Lines 9 to 16 do the rest of the decoding in a similar way to Seq2SeqModel.

The notebook trains a multi-step Seq2Seq model (the best-performing variant with teacher forcing) with all the different types of attention we covered in the chapter using the same setup we developed in the last chapter. The results are summarized in the following table:

Algorithm	MAE	MSE	MASE	Forecast Bias
MultiStep LSTM_LSTM_teacher_forcing_0.0	0.2058	0.1241	1.6039	15.32%
MultiStep LSTM_LSTM_teacher_forcing_1	0.1754	0.0912	1.3671	12.93%
MultiStep_Seq2Seq_dot_Attn_teacher_forcing_1	0.1536	0.0717	1.1967	8.25%
MultiStep_Seq2Seq_scaled_dot_Attn_teacher_forcing_1	0.1772	0.0878	1.3812	6.00%
MultiStep_Seq2Seq_general_Attn_teacher_forcing_1	0.1665	0.0818	1.2977	8.97%
MultiStep_Seq2Seq_concat_Attn_teacher_forcing_1	0.1632	0.0778	1.2717	6.65%
MultiStep_Seq2Seq_additive_Attn_teacher_forcing_1	0.1561	0.0734	1.2168	10.40%

Figure 14.4: Summary table for Seq2Seq models with attention

We can see that it is showing considerable improvements in MAE, MSE, and MASE by including attention, and out of all the variants of attention, the simple dot product attention performed the best, closely followed by additive attention. At this point, some of you might have a question in your mind—*Why didn't the scaled dot product work better than the dot product attention?* Scaling was supposed to make the dot product work better, wasn't it?

There is a lesson to be learned here (which applies to all **machine learning (ML)**). No matter how much better a particular technique is in theory, you can always find examples in which it performs worse. Here, we saw just one household, and it is not surprising that we saw that scaled dot product attention didn't work better than the normal dot product attention. But if we had evaluated at scale and realized that this is a pattern across multiple datasets, then it would be concerning.

So, we have seen that attention does make the models better. There was a lot of research done on using attention in various forms to enhance the performance of **neural network (NN)** models. Most of that research was carried out in **natural language processing (NLP)**, specifically in language translation and language modeling. Soon, researchers stumbled upon a surprising result that changed the course of DL progress drastically—the Transformer model.

Transformers—Attention is all you need

While the introduction of attention was a shot in the arm for RNNs and Seq2Seq models, they still had one problem. The RNNs were recurrent, and that meant they needed to process each word in a sentence in a sequential manner.

For popular Seq2Seq model applications such as language translation, it meant processing long sequences of words became really time-consuming. In short, it was difficult to scale them to a large corpus of data. In 2017, Vaswani et al. (Reference 5) authored a seminal paper titled *Attention Is All You Need*. Just as the title of the paper implies, they explored an architecture that used attention (scaled dot product attention) and threw away recurrent networks altogether. To the surprise of NLP researchers around the world, these models (which were dubbed Transformers) outperformed the then state-of-the-art Seq2Seq models in language translation.

This spurred a flurry of research activity around this new class of models, and pretty soon, in 2018, Devlin et al. (Reference 6) from Google developed a bi-directional version of Transformers and trained the now famous language model **BERT** (which stands for **Bidirectional Encoder Representations from Transformers**), and broke many state-of-the-art results in multiple tasks. This is considered to be the moment when Transformers as a model class really *arrived*. Fast-forward to 2022—Transformer models are ubiquitous. They are used in almost all tasks in NLP, and in many other sequence-based tasks such as time series forecasting and **reinforcement learning (RL)**. They have also been successfully used in **computer vision (CV)** tasks as well.

There have been numerous modifications and adaptations to the vanilla Transformer model to make it more suitable for time series forecasting. But let's discuss the vanilla Transformer architecture that Vaswani et al. proposed in 2017 first.

Attention is all you need

The model Vaswani et al. proposed (hereby referred to as the vanilla Transformer) is also an encoder-decoder model, but both the encoder and decoder are non-recurrent. They are entirely composed of attention mechanisms and feed-forward networks. Since the Transformer model was developed first for text sequences, let's use the same example to understand and then adapt to the time series context.

There are a few key components of the model that need to be understood to put the whole thing together. Let's take them one by one.

Self-attention

We saw how scaled dot product attention works earlier in this chapter (in the *Alignment functions* section), but there, we were calculating attention between the encoder and decoder hidden states. Self-attention is when we have an input sequence, and we calculate the attention scores between that input sequence itself. Intuitively, we can think of this operation as enhancing the contextual information and enabling the downstream components to use this enhanced information for further processing.

We saw the PyTorch implementation for encoder-decoder attention earlier, but that implementation was more aligned toward the step-by-step decoding of an RNN. Computing the attention scores for each query-key pair in one shot is something very simple to achieve using standard matrix multiplication and is essential to computing efficiency.

**Notebook alert:**

To follow along with the complete code, use the notebook named `02-Self-Attention_and_Multi-Headed_Attention.ipynb` in the `Chapter14` folder.

In NLP, it is standard practice to represent each word as a learnable vector called an embedding. This is because text or strings have no place in a mathematical model. For our example's sake, let's assume we use an embedding vector of size 512 for each word, and let's assume that the attention mechanism has an internal dimension of 64. Let's elucidate the attention mechanism using a sentence with 10 words.

After embedding, the sentence would be a tensor with dimensions (10, 512). We need to have three learnable weight matrices, W_q , W_k , and W_v , to project the input embedding into the attention dimension (64). See Figure 14.5:

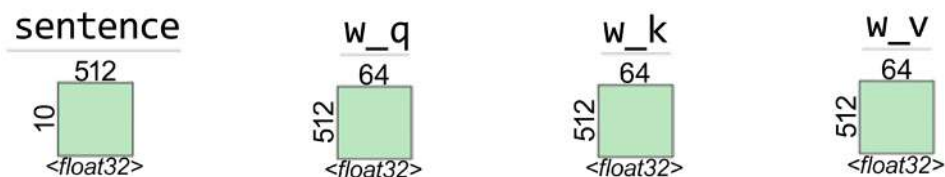


Figure 14.5: Self-attention layer: input sentence and the learnable weights

The first operation projects the sentence tensor into a query, key, and value with dimensions equal to (*sequence length*, *attention dim*). This is done by using a matrix multiplication between the sentence tensor and learnable matrices. See Figure 14.6:

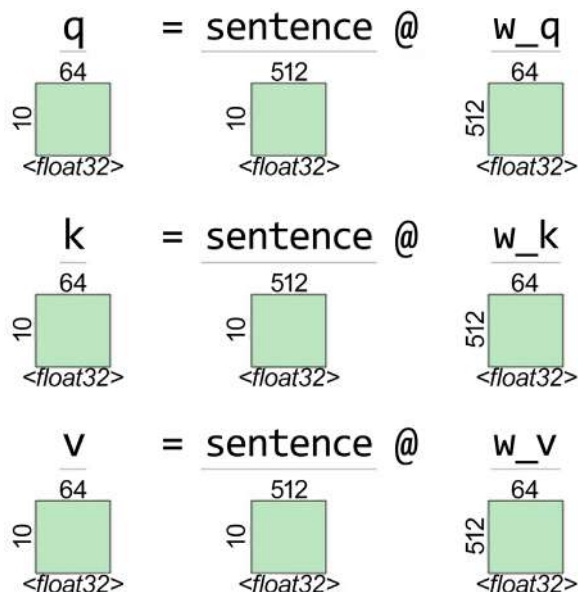


Figure 14.6: Self-attention layer: query, key, and value projection

Now that we have the query, key, and value, we can calculate the attention weights of every query-key pair using matrix multiplication between the query and the transpose of the keys. The matrix multiplication is nothing but the dot product of each query with each of the values and gives us a square matrix of $(sequence\ length, sequence\ length)$. See Figure 14.7:

$$\text{attn_scores} = \text{q} @ \text{k}^t()$$

Figure 14.7: Self-attention layer: attention scores between Q and K

Converting the attention scores to attention weights is just about scaling and applying the *softmax* function, as we discussed in the *Scaled dot product attention* section.

Now that we have the attention weights, we can use them to combine the value. The element-wise multiplication and then summing across the weights can be done efficiently using another matrix multiplication. See Figure 14.8:

$$\text{attn_output} = \text{attn_weights} @ \text{v}$$

Figure 14.8: Self-attention layer: combining V using the learned attention weights

Now, we have seen how attention is applied to all the query-key pairs in monolithic matrix operations rather than doing the same operation for each query in a sequential way. But *Attention Is All You Need* proposed something even better.

Multi-headed attention

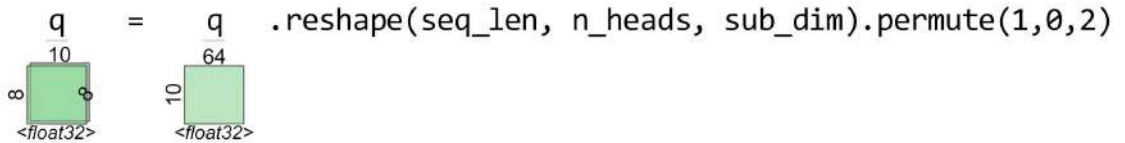
Since Transformers intended to take away the entire recurrent architecture, they needed to beef up the attention mechanism because that was the workhorse of the model. So, instead of using a single attention head, the authors of the paper proposed multiple attention heads acting together in different subspaces. We know that attention helps the model focus on a few elements of the many. **Multi-headed attention (MHA)** does the same thing but focuses on different aspects or different sets of elements, thereby increasing the capacity of the model. If we want to draw an analogy to the human mind, we consider many aspects of a situation before we make a decision.

For instance, if we decide to step out of the house, we will pay attention to the weather, we will pay attention to the time so that whatever we want to accomplish is still possible, we will pay attention to how punctual that friend you made a plan with has been in the past, and leave our house accordingly. You can think of each of these things as one head of attention. Therefore, MHA enables Transformers to *attend* to multiple aspects at the same time.

Normally, if there are eight heads, we would assume that we would have to do the computation that we saw in the last section eight times. But thankfully, that is not the case. There are clever ways of accomplishing this MHA using the same kind of matrix multiplication, but now with larger matrices. Let's see how that is done.

We will continue the same example and see a case where we have eight attention heads. There is one condition that needs to be satisfied to do this efficient calculation of MHA—the attention dimension should be divisible by the number of heads we are using.

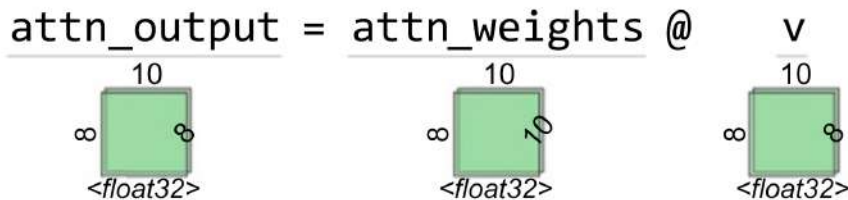
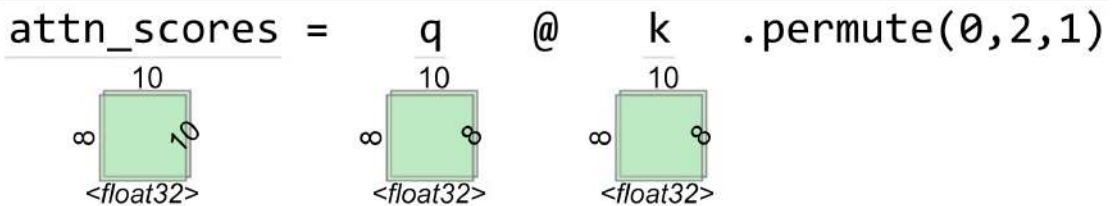
The initial steps are exactly the same. We take in the input sentence tensor and project it into the query, key, and value. Now, we split the query, key, and value into separate query, key, and value subspaces for each head by doing some basic tensor re-arrangement. See Figure 14.9:



q, k, v dimensions: n_heads, seq_len, sub_dim : 8 x 10 x 8

Figure 14.9: Multi-headed attention: reshaping Q, K, and V into subspaces for each head

Now, we calculate the attention scores for each head in a single operation and combine them with the value to get the attention output for each head. See Figure 14.10:



attn_output dimensions: n_heads, seq_len, sub_dim : 8 x 10 x 8

Figure 14.10: Multi-headed attention: calculating attention weights and combining the value

We have the attention output of each head in the `attn_output` variable. Now, all we need to do is reshape the array so that we stack the outputs from all the attention heads in a single dimension. See Figure 14.11:

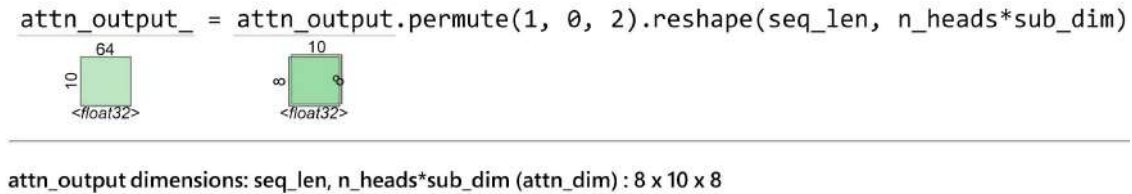


Figure 14.11: Multi-headed attention: reshaping and stacking all the attention head outputs

In this way, we can do MHA in a fast and efficient manner. Now, let's look at another key innovation that makes Transformers work.

Positional encoding

Transformers successfully prevented recurrence and unlocked a performance bottleneck of sequential operations. This also means that the Transformer model is agnostic of the order of the sequence. In mathematical terms, if RNNs were considering the sequence as a sequence, Transformers consider it as a set of values. For the Transformer, each position is independent of the other, and hence one key aspect we would seek from a model that processes sequences is missing. The original authors did propose a way to make sure we do not lose this information—**positional encoding**.

There have been many variants of positional encoding that have come up in subsequent years of research, but the most common one is still the variant that is used in the vanilla Transformer.

The solution proposed by Vaswani et al. was to add a particular vector, which encodes the position mathematically using sine and cosine functions, to each of the input tokens before processing them through the self-attention layer. If the input X , is a d_{model} -dimensional embedding for n tokens in a sequence, positional embeddings, P , is a matrix of the same size ($n \times d_{model}$). The element on the pos^{th} row and $2i^{th}$ or $(2i + 1)^{th}$ column is defined as follows:

$$p_{pos,2i} = \sin\left(\frac{pos}{10000^{\frac{2i}{d_{model}}}}\right)$$

$$p_{pos,2i+1} = \cos\left(\frac{pos}{10000^{\frac{2i}{d_{model}}}}\right)$$

Although this looks a little complicated and counterintuitive, let's break this down to understand it better.

From 20,000 feet, we know that these positional encodings capture the positional information, and we add them to the input embeddings. But why do we add them to the input embeddings? Let's make this clearer. Let's assume the embedding dimension is just 2 (this is for ease of visualization and grasping the concept better), and we have a word, A , represented using this token. For our experiment, let's assume that we have the same word, A , repeated several times in our sequence. What happens if we add the positional encoding to it?

We know that the sine or cosine functions vary between 0 and 1. So, each of these encodings we add to the word embedding just perturbs the word embedding within a unit circle. As *pos* increases, we can see the position-encoded word embedding trace a unit circle around the original embedding (Figure 14.12):

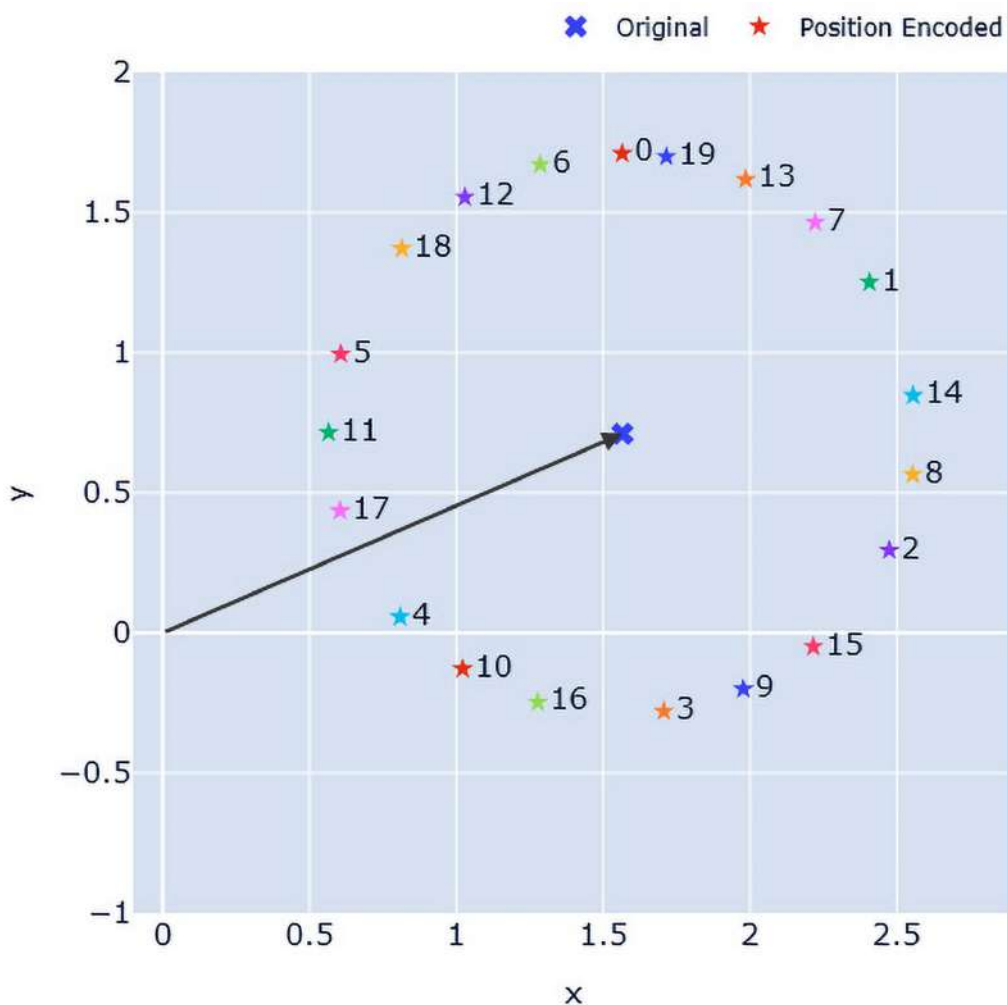


Figure 14.12: Position encoding: intuition

In Figure 14.12, we have assumed a random embedding for a word, *A* (represented by the cross marker), and added position embedding assuming *A* is in different positions. These position-encoded vectors are represented by star markers with the corresponding positions mentioned in numbers next to them. We can see how each position is a slightly perturbed point of the original vector, and it happens in a cyclical manner in a clockwise direction. We can see position 0 right at the top with 1, 2, 3, and so on in the clockwise direction. By having this representation, the model can figure out the word in different locations, and still retain the overall position in the semantic space.

Now that we know why we are adding the positional encodings to the input embeddings and have seen why it works, let's get into the details and see how the terms inside the sine and cosine are calculated. pos represents the position of the token in the sequence. If the maximum length of the sequence is 128, pos varies from 0 to 127. i represents the position along the embedding dimension, and because of the way the formula has been defined, for each value of i , we have two values—a sine and a cosine. Therefore, i will be half the number of dimensions, d_{model} , and will go from 0 to $d_{model}/2$.

With all this information, we know that the term inside the sine and cosine functions approaches 0 as we go toward the end of the embedding dimension. It also increases from 0 as we move along the sequence dimension. For each pair ($2i$ and $2i+1$) of positions in the embedding dimension, we have a complementary sine and cosine wave, as Figure 14.13 shows:

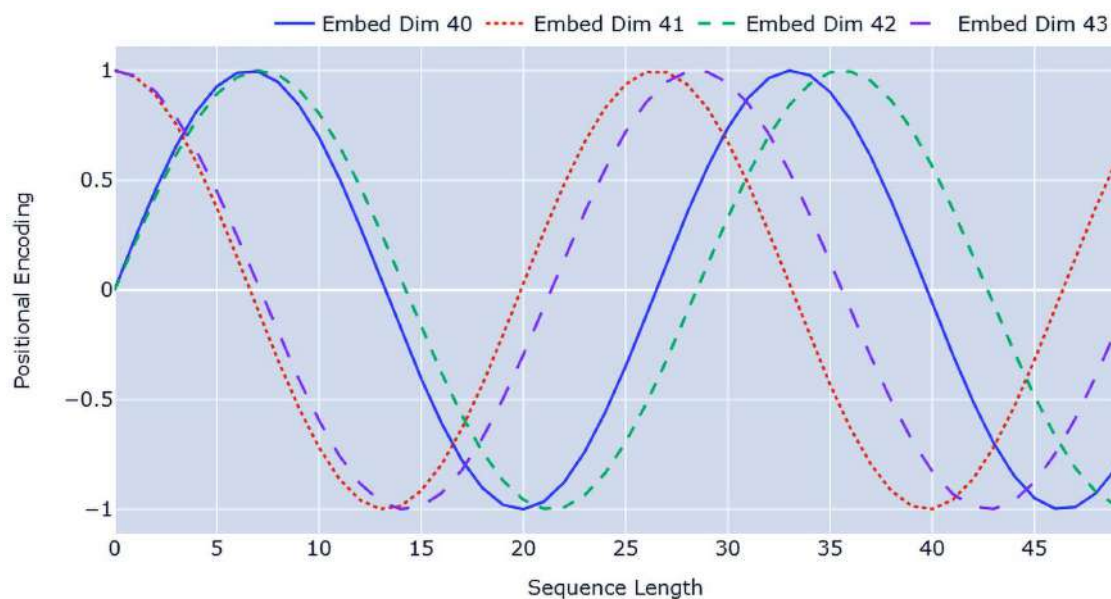


Figure 14.13: Positional encoding: sine and cosine terms

We can see that embedding dimensions 40 and 41 are sine and cosine waves of the same frequency, and embedding dimensions 40 and 42 are sine waves with a slight increase in frequency. By using the combination of sine and cosine waves of varying frequencies, the positional encoding can encode rich positional information as a vector. If we plot a heatmap (refer to the color images file: <https://packt.link/gbp/9781835883181>) of the whole positional encoding vector (Figure 14.14), we can see how the values change and encode the positional information:

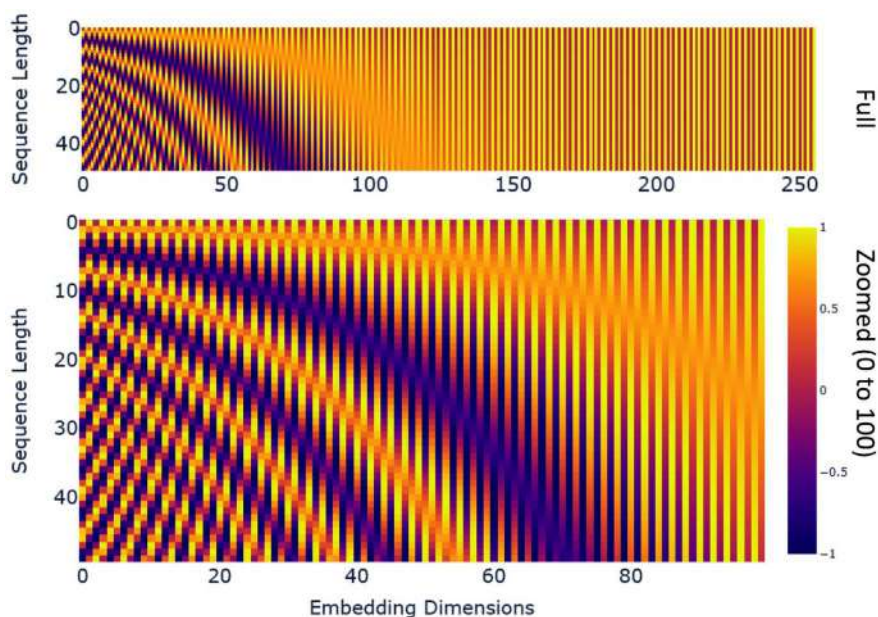


Figure 14.14: Positional encoding: heatmap of the entire vector

 **Quick tip:** Need to see a high-resolution version of this image? Open this book in the next-gen Packt Reader or view it in the PDF/ePub copy.

 **The next-gen Packt Reader and a free PDF/ePub copy** of this book are included with your purchase. Unlock them by scanning the QR code below or visiting <https://www.packtpub.com/unlock/9781835883181>.



Another interesting observation is that the positional encoding quickly shrinks to 0/1 as we move forward in the embedding dimension because the term inside the sine or cosine functions (angle in radians) quickly becomes zero on account of the large denominator. The zoomed plot shows the color differences more clearly.

Now, for the last component in the Transformer model.

Position-wise feed-forward layer

We have already covered what feed-forward networks are in *Chapter 12, Building Blocks of Deep Learning for Time Series*. The only thing to be noted here is that the position-wise feed-forward layer is when we apply the same feed-forward layer in each position, independently. If we have 12 positions (or words), we will have a single feed-forward network to process each of these positions.

Vaswani et al. defined this as a two-layer feed-forward network where the transformations were defined so that the input dimensions are expanded to four times the input dimension, with a ReLU activation function applied at that stage, and then transformed back to the input dimension again. The exact operation can be written as a mathematical formula:

$$FFN(x) = \max(0, W_1x + b_1) W_2 + b_2$$

Here, W_1 is a matrix of dimensions (input size, $4 \times \text{input size}$), W_2 is a matrix of dimensions ($4 \times \text{input size}$, input size), b_1 and b_2 are the corresponding biases, and $\max(0, x)$ is the standard ReLU operator.

There have been studies where researchers have tried replacing ReLU with other activation functions, more specifically **Gated Linear Units (GLUs)**, which have shown promise. Noam Shazeer from Google has a paper on the topic, and if you want to know more about these new activation functions, I recommend checking out his paper in the *Further reading* section.

Now that we know all the necessary components of a Transformer model, let's see how they are put together.

Encoder

The vanilla Transformer model is an encoder-decoder model. There are N blocks of encoders, and each block contains an MHA layer and a position-wise feed-forward layer with residual connections in between (Figure 14.15):

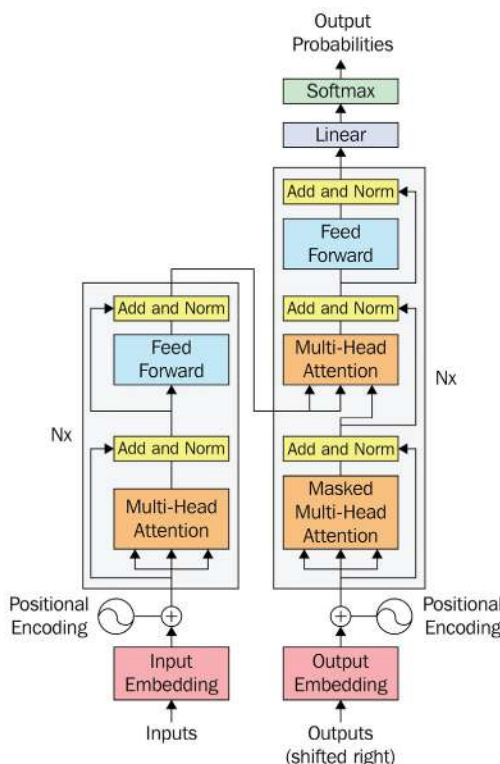


Figure 14.15: Transformer model from *Attention Is All You Need* by Vaswani et al.

For now, let's focus on the left side of *Figure 14.15*, which is the encoder. The encoder takes in the input embeddings, with the positional encoding vector added to it, as the input. The three-pronged arrow that goes into MHA denotes the query, key, and value split. The output from the MHA goes into a block named *Add and Norm*. Let's quickly see what that does.

There are two key operations that happen here—**residual connections** and **layer normalization**.

Residual connections

Residual connections (or skip connections) are a family of techniques that were introduced to DL to make learning deep networks easier. The primary benefit of the technique is that it makes the gradient flow through the network better and thereby encourages learning in all parts of the network. They incorporate a pass-through memory highway in the network. We have already seen one instance where a skip connection (although not an apparent one) resolved gradient flow issues—**long short-term memory networks (LSTMs)**. The cell state in the LSTM serves as this highway to let gradients flow through the network without getting into vanishing gradient issues.

But nowadays, when we say residual connections, we typically think of *ResNets*, which made a splash in the history of DL through a **convolutional neural network (CNN)** architecture that won major image classification challenges, including ImageNet in 2015. They introduced residual connections to train much deeper architectures than those prevalent at the time. The concept is deceptively simple. Let's visualize it:

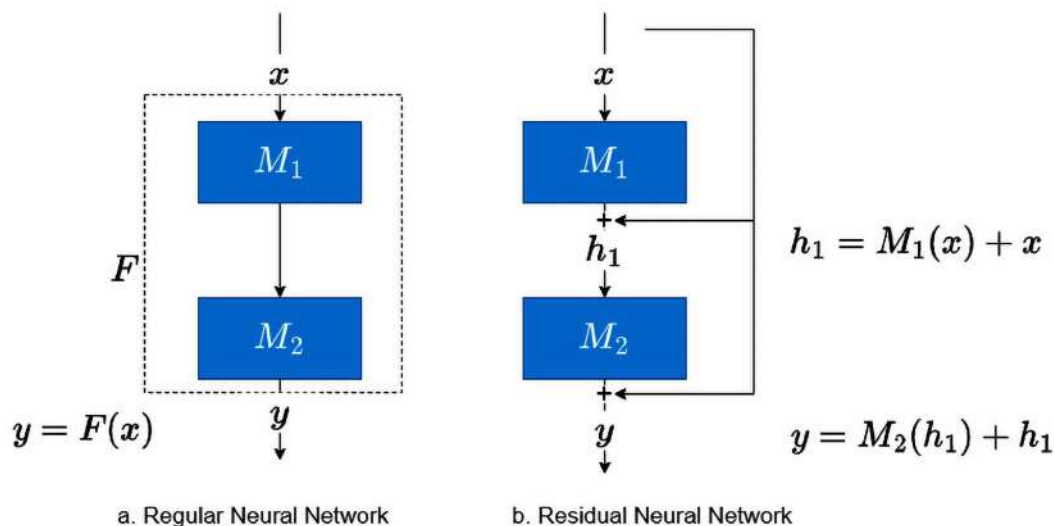


Figure 14.16: Residual networks

Let's assume a DL model with two layers with functions, M_1 and M_2 . In a regular neural network, the input, x , passes through the two layers to give us the output, y . These two individual functions can be considered as a single function that converts x to y : $y = F(x)$.

In residual networks, we change this paradigm into saying that each individual function (or layer) only learns the difference between the input to the function and the expected output. That is where the name residual connections came from. So, if h_1 is the desired output and x is the input, then $M_1(x) = h_1 - x$. Rewriting that, we get $h_1 = M_1(x) + x$. And this is what is most commonly used as residual connections.

Among many benefits such as better gradient flows, residual connections also make the loss surface smooth (Li et al. 2018, Reference 7) and more amenable to gradient-based optimization. For more details and intuition around residual networks, I urge you to check out the blog linked in the *Further reading* section.

So, the *Add* in the *Add and Norm* block in the Transformer is actually the residual connection.

Layer normalization

Normalization in **deep neural networks** (DNNs) has been an active field of research. Among many benefits, normalization leads to faster training, higher learning rates, and even a bit of regularization. Batch normalization is the most common normalization technique in use, typically in CV, which makes the input have approximately zero mean and unit variance by subtracting the input mean in the current batch and dividing it by the standard deviation.

But in NLP, researchers prefer layer normalization, where the normalization is happening in each feature. The difference can be seen in *Figure 14.17*:

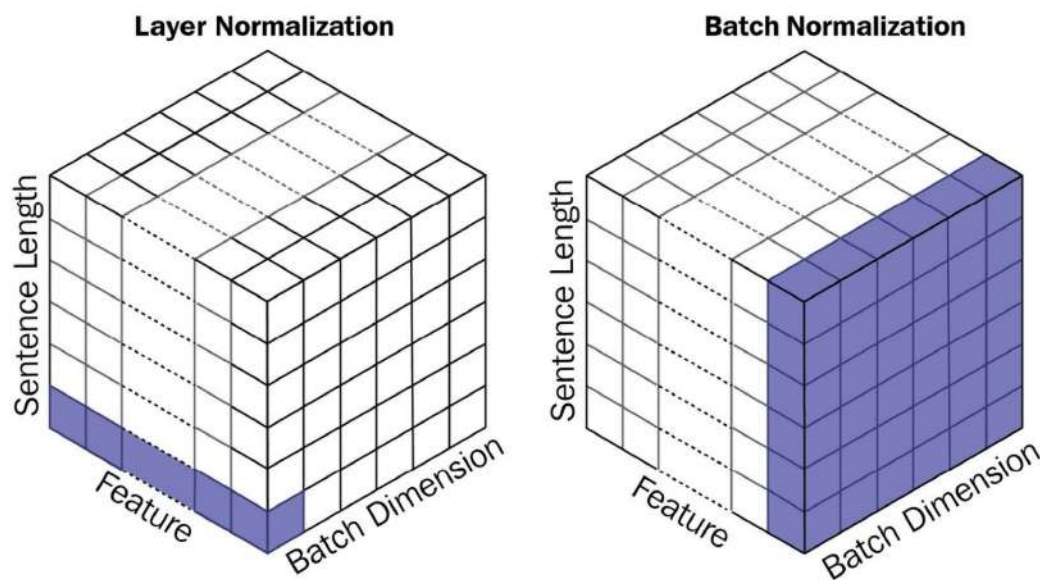


Figure 14.17: Batch normalization versus layer normalization

This preference for layer normalization emerged empirically, but there has been research about the reason for this preference. NLP data usually has a higher variance as opposed to CV data, and this variance causes some problems for batch normalization. Layer normalization, on the other hand, is immune to this because it doesn't rely on batch-level variance.

Either way, Vaswani et al. decided to use layer normalization in their *Add and Norm* block.

Now, we know the *Add and Norm* block is nothing but a residual connection that is then passed through a layer normalization. So, we can see that the position-encoded inputs are first used in the MHA layer, and the output from the MHA is added with the position-encoded inputs again and passed through a layer normalization. Now, this output is passed through the position-wise feed-forward network and another *Add and Norm* layer, and this becomes one block of the encoder. An important point to keep in mind is that the architecture of all the elements in the encoder is designed in such a way that the dimension of the input at each position is preserved throughout. In other words, if the embedding vector is of dimension 100, the output from the encoder will also have a dimension of 100. This is a convenient way to make it possible to have residual connections and stack as many layers on top of each other as possible. Now, there are multiple such encoder blocks stacked on top of each other to form the encoder of the Transformer.

Decoder

The decoder block is also very similar to the encoder block, but with one key addition. Instead of a single self-attention layer, the decoder block has a self-attention layer, which operates on the decoder input, and an encoder-decoder attention layer. The encoder-decoder attention layer takes the query from the decoder at each stage and the key and values from the top encoder block.

There is something peculiar to the self-attention that is applied in the decoder block. Let's see what that is.

Masked self-attention

We talked about how the Transformer can process sequences in parallel and be computationally efficient. But the decoding paradigm poses another challenge. Suppose we have an input sequence, $X = \{x_1, x_2, \dots, x_n\}$, and the task is to predict the next step. So, in the decoder, if we give the sequence, X , because of the parallel-processing architecture, each sequence is processed at once using self-attention. And we know self-attention is agnostic to sequence order. If left unrestricted, the model will cheat by using information from the future timesteps to predict the current timestep. This is where masked attention becomes important.

We saw earlier in the *Self-attention* section how to calculate a square matrix (if the query and key have the same length) of attention weights, and it is with these weights that we combine the information from the value vector. This self-attention has no concept of temporality, and all the tokens will attend to all other tokens irrespective of their position.

Let's see *Figure 14.18* to solidify our understanding:

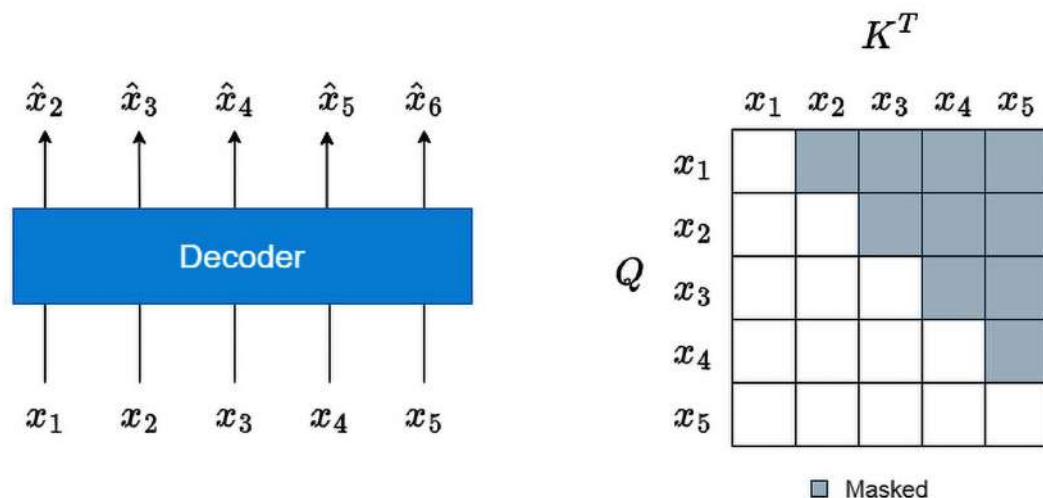


Figure 14.18: Masked self-attention

We have the sequence, $X = \{x_1, x_2, \dots, x_5\}$, and we are still trying to predict one step ahead. So, the expected output from the decoder would be $\hat{X} = \{\hat{x}_2, \hat{x}_3, \dots, \hat{x}_6\}$. When we use self-attention, the attention weights that will be learned will be a square matrix of 5 X 5 dimension. But if we look at the upper triangle of the square matrix (the part that is shaded in *Figure 14.18*), those combinations of tokens violate the temporal sanctity.

We can take care of this simply by adding a pre-generated mask that has zeros in all the white cells and $-\infty$ in all the shaded cells to the generated attention energies (the stage before applying *softmax*). This makes sure the attention weights for the shaded region will be zero, and this in turn ensures that no future information is used while calculating the weighted sum of the value vector.

Now, to wrap everything up, the output from the decoder is passed to a standard task-specific head to generate the output we desire.

We discussed the Transformer in the context of NLP, but it is a very small leap to adapt it to time series data.

Transformers in time series

Time series have a lot of similarities with NLP because of the fact that both deal with information in sequences and in both cases the order of elements matters. In time series, the elements are typically time-ordered data points, while in NLP, the elements are tokens (such as words or characters) that form sentences or documents. This can be further evidenced by the phenomenon that most of the popular techniques that are used in NLP are promptly adapted to a time series context. Transformers are no exception to that.

Instead of looking at tokens at each position, we have real numbers in each position. And instead of talking about input embeddings, we can talk in terms of input features. The vector of features at each timestep can be considered the equivalent of an embedding vector in NLP. And instead of making causal decoding an optional step (in NLP, that really depends on the task at hand), we have a strict requirement for causal decoding. There, it is trivial to adapt Transformers to time series, although in practice, there are many challenges because in time series we typically encounter sequences that are much longer than the ones in NLP, and this creates a problem because the complexity of self-attention is scaled quadratically with respect to the input sequence length. There have been many alternate proposals for self-attention that make it feasible to use it for long sequences as well, and we will be covering a few of them in *Chapter 16, Specialized Deep Learning Architectures for Forecasting*.

Now, let's try to put everything we have learned about Transformers into practice.

Forecasting with Transformers

For some continuity, we will use the same household example we were forecasting with RNNs and RNNs with attention.



Notebook alert:

To follow along with the complete code, use the notebook named `03-Transformers.ipynb` in the `Chapter14` folder and the code in the `src` folder.

Although we learned about the vanilla Transformer as a model with an encoder-decoder architecture, it was really designed for language translation tasks. In language translation, the source sequence and target sequence are quite different, and therefore the encoder-decoder architecture made sense. But soon after, researchers figured out that using the decoder part of the Transformer alone does well. It is called a decoder-only Transformer in literature. The naming is a bit confusing because if you think about it, the decoder is different from the encoder in two ways—masked self-attention and encoder-decoder attention. So, in a decoder-only Transformer, how do we make up for the encoder-decoder attention? The short answer is that we don't. The architecture of the decoder-only Transformer resembles the encoder block more, but we call it decoder-only because we use masked self-attention to make our model respect the temporal sanctity of our sequences.

We are also going to implement a decoder-only Transformer. The first thing we need to do is to define a config class, `TransformerConfig`, with the following parameters:

- `input_size`: This parameter defines the number of features the Transformer is expecting.
- `d_model`: This parameter defines the hidden dimension of the Transformer or the dimension over which all the attention calculation and subsequent operations happen.
- `n_heads`: This parameter defines how many heads we have in the MHA mechanism.
- `n_layers`: This parameter defines how many blocks of encoders we are going to stack on top of each other.
- `ff_multiplier`: This parameter defines the scale of expansion within the position-wise feed-forward layers.

- **activation:** This parameter lets us define which activation we need to use in the position-wise feed-forward layers. It can be either `relu` or `gelu`.
- **multi_step_horizon:** This parameter lets us define how many timesteps into the future we should be forecasting.
- **dropout:** This parameter lets us define the magnitude of dropout regularization to be applied in the Transformer model.
- **learning_rate:** This defines the learning rate of the optimization procedure.
- **optimizer_params, lr_scheduler, lr_scheduler_params:** These are parameters that let us tweak the optimization procedure. Let's not worry about these for now because all of them have been set to intelligent defaults.

Now, we are going to inherit the `BaseModel` class we defined in `src/dl/models.py` and define a `TransformerModel` class.

The first method we need to implement is `_build_network`. The entire model can be found in `src/dl/models.py`, but we will be covering the important aspects here as well.

The first module we need to define is a linear projection layer that takes in the `input_size` parameter and projects it into `d_model`:

```
self.input_projection = nn.Linear(
    self.hparams.input_size, self.hparams.d_model, bias=False
)
```

This is an additional step we have introduced to adapt the Transformers to the time series forecasting paradigm. In the vanilla Transformer, this is not needed because each word is represented by an embedding vector that typically has dimensions such as 200 or 500. But while doing time series forecasting, we might have to do the forecasting with just one feature (which is the history), and this seriously restricts our ability to provide capacity to the model because, without the projection layer, `d_model` can only be equal to `input_size`. Therefore, we have introduced a linear projection layer that decouples the number of features available and `d_model`.

Now, we need to have a module that adds positional encoding. We have packaged the same code we saw earlier into a PyTorch module and added it to `src/dl/models.py`. We just use that module and define our positional encoding operator, like so:

```
self.pos_encoder = PositionalEncoding(self.hparams.d_model)
```

We said earlier that we are going to use a decoder-only approach to building the model, and for that, we are using the `TransformerEncoderLayer` and `TransformerEncoder` modules defined in PyTorch. Just keep in mind that when using these layers, we will be using masked self-attention, and that makes it a decoder-only Transformer. The code is presented here:

```
self.encoder_layer = nn.TransformerEncoderLayer(
    d_model=self.hparams.d_model,
    nhead=self.hparams.n_heads,
    dropout=self.hparams.dropout,
```

```

        dim_feedforward=self.hparams.d_model * self.hparams.ff_multiplier,
        activation=self.hparams.activation,
        batch_first=True,
    )
    self.transformer_encoder = nn.TransformerEncoder(
        self.encoder_layer, num_layers=self.hparams.n_layers
    )

```

The last module we need to define is a linear layer that converts the output from the Transformer into the number of timesteps we are forecasting:

```

self.decoder = nn.Sequential(nn.Linear(self.hparams.d_model, 100),
                             nn.ReLU(),
                             nn.Linear(100, self.hparams.multi_step_horizon)
)

```

That concludes the definition of the model. Now, let's define a forward pass in the forward method.

The first step is to generate a mask we need to apply masked self-attention:

```

mask = self._generate_square_subsequent_mask(x.shape[1]).to(x.device)

```

We define the mask to have the same length as the input sequence. `_generate_square_subsequent_mask` is a method we have defined that generates a mask. Assuming the sequence length is 5, we can look at the two steps in preparing the mask:

```

mask = (torch.triu(torch.ones(5, 5)) == 1).transpose(0, 1)

```

`torch.ones(sz, sz)` creates a square matrix filled with ones, and `torch.triu(torch.ones(sz, sz))` makes a matrix with a top triangle (including the diagonal) filled with ones and the rest filled with zeros. By using an equality operator with one condition and transposing it, we get a mask that has True in all the bottom triangles, including the diagonal, and False everywhere else. The output of the previous statement will be this:

```

tensor([[ True, False, False, False, False],
        [ True,  True, False, False, False],
        [ True,  True,  True, False, False],
        [ True,  True,  True,  True, False],
        [ True,  True,  True,  True,  True]])

```

We can see that this matrix has False at all the places where we need to mask attention. Now, all we need to do is to fill all True instances with 0 and all False instances with -inf:

```

mask = (
    mask.float()
    .masked_fill(mask == 0, float("-inf"))
    .masked_fill(mask == 1, float(0.0))
)

```

These two lines of code are packaged into the `_generate_square_subsequent_mask` method, which we can use while training the model.

Now that we have created the mask for masked self-attention, let's start processing the input, `x`:

```
# Projecting input dimension to d_model
x_ = self.input_projection(x)
# Adding positional encoding
x_ = self.pos_encoder(x_)
# Encoding the input
x_ = self.transformer_encoder(x_, mask)
# Decoding the input
y_hat = self.decoder(x_)
```

In these four lines of code, we project the input to `d_model` dimensions, add positional encoding, pass it through the Transformer model, and lastly, use the linear layer to convert the output to the predictions we want.

Now we have `y_hat`, which is the prediction from the model. All we need to think of now is how to train this output to be the desired output.

We know that the Transformer model processes all the tokens in one shot, and if we have N elements in the sequence, we will have N predictions as well (each prediction corresponding to the next time-step). And if each prediction is for the next H timesteps, the shape of `y_hat` would be (B, N, H) , where B is the batch size. There are a few ways we can use this output to compare with the target. The most simple and naïve way is to just take the prediction from the last position (which will have H timesteps) and compare it with `y` (which also has H timesteps).

But this is not the most efficient way of using all the information we have, is it? We are discarding $N-1$ predictions and not giving any signal to the model on all those $N-1$ predictions. So, while training, it makes sense to use all these $N-1$ predictions also so that the model has a much richer signal feeding back while learning.

We can do that by using the original input sequence, `x`, but offsetting it by one. When $H=1$, we can think of this as a simple task where each position's prediction is compared with the target for the next position (one step ahead). We can easily accomplish this by concatenating `x[:, 1:, :]` (the input sequence offset by 1) with `y` (the original target) and treating this as the target. But when $H > 1$, this becomes slightly complicated, but we can still do it by using a helpful function from PyTorch called `unfold`:

```
y = torch.cat([x[:, 1:, :], y], dim=1).squeeze(-1).unfold(1, y.size(1), 1)
```

We first concatenate the input sequence (offset by one) with `y` and then use `unfold` to create sliding windows of `size = H`. This gives us a target of the same shape, (B, N, H) .

But during inference (when we are predicting using a trained model), we do not need the output of all the other positions, and hence we discard them, as shown here:

```
y_hat = y_hat[:, -1, :].unsqueeze(1)
```

Our `BaseModel` class that we defined also lets us define a slightly different prediction step by using the `predict` method. You can look over the complete model in `src/dl/models.py` once again to solidify your understanding now.

Now that we have defined the model, we can use the same framework we have been using to train `TransformerModel1`. The full code is available in the notebook, but we will just look at a summary table with the results:

Algorithm	MAE	MSE	MASE	Forecast Bias
MultiStep LSTM_LSTM_teacher_forcing_1	0.1754	0.0912	1.3671	12.93%
MultiStep_Seq2Seq_dot_Attn_teacher_forcing_1	0.1536	0.0717	1.1967	8.25%
MultiStep_Transformer_Multi_Step_FF_decoder	0.1949	0.1104	1.5188	16.24%

Figure 14.19: Metrics for Transformer model on MAC000193 household

We can see that the model is not doing as well as its RNN cousins. There could be many reasons for this, but the most probable one is that Transformers are really data-hungry. Transformers have far fewer inductive biases and therefore only shine where there is lots of data available to learn from. When forecasting just one household alone, our model has access to far less data and may not work very well. This is true, to an extent, for all the DL models we have seen so far. In *Chapter 10, Global Forecasting Models*, we talked about how we can train a single model for multiple households together, but that discussion was limited to classical ML models. DL is also perfectly capable of global forecasting models and that is exactly what we will be talking about in the next chapter—*Chapter 15, Strategies for Global Deep Learning Forecasting Models*.

For now, congratulations on getting through another concept-heavy and information-packed chapter. The concept of attention, which has taken the field by storm, should be clearer in your mind now than when you started the chapter. I urge you to take a second stab at the chapter, read through the *Further reading* section, and do some of your own research if it's not clear because the future chapters assume you understand this.

Summary

We have been storming through the world of DL in the last few chapters. We started off with the basic premise of DL, what it is, and why it became so popular. Then, we saw a few common building blocks that are typically used in time series forecasting and got our hands dirty learning how we can put what we have learned into practice using PyTorch. Although we talked about RNNs, LSTMs, GRUs, and so on, we purposefully left out attention and Transformers because they deserved a separate chapter.

We started the chapter by learning about the generalized attention model, helping you put a framework around all the different schemes of attention out there, and then went into detail on a few common attention schemes, such as scaled dot product, additive, and general attention. Right after incorporating attention into the Seq2Seq models we were playing with in *Chapter 12, Building Blocks of Deep Learning for Time Series*, we started with the Transformer.

We examined all the building blocks and architecture decisions involved in the original Transformer from the point of view of NLP, and after understanding the architecture, we adapted it to a time-series setting.

And finally, we capped it off by training a Transformer model for forecasting on a sample household. And now, by finishing this chapter, we have all the basic ingredients to really start using DL for time series forecasting.

In the next chapter, we are going to elevate what we have been doing and move on to the global forecasting model paradigm.

References

Following is the list of the references used in this chapter:

1. Dzmitry Bahdanau, KyungHyun Cho, and Yoshua Bengio (2015). *Neural Machine Translation by Jointly Learning to Align and Translate*. In *3rd International Conference on Learning Representations*. <https://arxiv.org/pdf/1409.0473.pdf>
2. Thang Luong, Hieu Pham, and Christopher D. Manning (2015). *Effective Approaches to Attention-based Neural Machine Translation*. In *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing*. <https://aclanthology.org/D15-1166/>
3. André F. T. Martins, Ramón Fernandez Astudillo (2016). *From Softmax to Sparsemax: A Sparse Model of Attention and Multi-Label Classification*. In *Proceedings of the 33rd International Conference on Machine Learning*. <http://proceedings.mlr.press/v48/martins16.html>
4. Ben Peters, Vlad Niculae, André F. T. Martins (2019). *Sparse Sequence-to-Sequence Models*. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*. <https://aclanthology.org/P19-1146/>
5. Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin (2017). *Attention is All you Need*. In *Advances in Neural Information Processing Systems*. <https://papers.nips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html>
6. Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova (2019). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. <https://aclanthology.org/N19-1423/>
7. Hao Li, Zheng Xu, Gavin Taylor, Christoph Studer, and Tom Goldstein (2018). *Visualizing the Loss Landscape of Neural Nets*. In *Advances in Neural Information Processing Systems*. <https://proceedings.neurips.cc/paper/2018/file/a41b3bb3e6b050b6c9067c67f663b915-Paper.pdf>
8. Sneha Chaudhari, Varun Mithal, Gungor Polatkan, and Rohan Ramanath (2021). *An Attentive Survey of Attention Models*. *ACM Trans. Intell. Syst. Technol.* 12, 5, Article 53 (October 2021). <https://doi.org/10.1145/3465055>

Further reading

Here are a few resources for further reading:

- *The Illustrated Transformer* by Jay Alammar: <https://jalammar.github.io/illustrated-transformer/>
- *Transformer Networks: A mathematical explanation why scaling the dot products leads to more stable gradients*: <https://towardsdatascience.com/transformer-networks-a-mathematical-explanation-why-scaling-the-dot-products-leads-to-more-stable-414f87391500>
- *Why is Bahdanau's attention sometimes called concat attention?*: <https://stats.stackexchange.com/a/524729>
- Noam Shazeer (2020). *GLU Variants Improve Transformer*. arXiv preprint: *Arxiv-2002.05202*. <https://arxiv.org/abs/2002.05202>
- *What is Residual Connection?* by Wanshun Wong: <https://towardsdatascience.com/what-is-residual-connection-efb07cab0d55>
- *Attn: Illustrated Attention* by Raimi Karim: <https://towardsdatascience.com/attn-illustrated-attention-5ec4ad276ee3>

Unlock this book's exclusive benefits now

This book comes with additional benefits designed to elevate your learning experience.



Note: Have your purchase invoice ready before you begin. <https://www.packtpub.com/unlock/9781835883181>

15

Strategies for Global Deep Learning Forecasting Models

All through the last few chapters, we have been building up deep learning for time series forecasting. We started with the basics of deep learning, saw the different building blocks, practically used some of those building blocks to generate forecasts on a sample household, and finally, talked about attention and Transformers. Now, let's slightly alter our trajectory and take a look at global models for deep learning. In *Chapter 10, Global Forecasting Models*, we saw why global models make sense and also saw how we can use such models in the machine learning context. We even got good results in our experiments. In this chapter, we will look at how we can apply similar concepts, but from a deep learning perspective. We will look at different strategies that we can use to make global deep learning models work better.

In this chapter, we will be covering these main topics:

- Creating global deep learning forecasting models
- Using time-varying information
- Using static/meta information
- Using the scale of the time series
- Balancing the sampling procedure

Technical requirements

You will need to set up the **Anaconda** environment by following the instructions in the *Preface* to get a working environment with all the libraries and datasets required for the code in this book. Any additional libraries will be installed while running the notebooks.

You will need to run these notebooks:

- `02-Preprocessing_London_Smart_Meter_Dataset.ipynb` in Chapter02
- `01-Setting_up_Experiment_Harness.ipynb` in Chapter04
- `01-Feature_Engineering.ipynb` in Chapter06

The associated code for the chapter can be found at <https://github.com/PacktPublishing/Modern-Time-Series-Forecasting-with-Python-/tree/main/notebooks/Chapter15>.

Creating global deep learning forecasting models

In *Chapter 10, Global Forecasting Models*, we talked in detail about why a global model makes sense. We talked about the benefits regarding increased *sample size*, *cross-learning*, *multi-task learning*, the regularization effect that comes with it, and reduced *engineering complexity*. All of these are relevant for a deep learning model as well. Engineering complexity and sample size become even more important because deep learning models are data-hungry and take quite a bit more engineering effort and training time than other machine learning models. I would go to the extent that in the deep learning context, in most practical cases where we have to forecast at scale, global models are the only deep learning paradigm that makes sense.

So, why did we spend all that time looking at individual models? Well, it's easier to grasp the concept at that level, and the skills and knowledge we gained at that level are very easily transferred to a global modeling paradigm. In *Chapter 13, Common Modeling Patterns for Time Series*, we saw how we can use a dataloader to sample windows from a single time series to train the model. To make the model a global model, all we need to do is to change the dataloader so that instead of sampling windows from a single time series, we sample from many time series. The sampling process can be thought of as a two-step process (although in practice, we do it in a single step, it is intuitive to think of it as two)—first, sample the time series we need to pick the window from, and then, sample the window from that time series. By doing that, we are training a single deep learning model to forecast all the time series together.

To make our lives easier, we are going to use the open-source libraries PyTorch Forecasting and `neuralforecast` from Nixtla in this text. We will be using PyTorch Forecasting for pedagogical purposes because it does provide more flexibility, but `neuralforecast` is more current and actively maintained and therefore more recent architectures will be added there. In *Chapter 16*, we will see how we can use `neuralforecast` to do forecasting, but for now, let's pick PyTorch Forecasting and move ahead.

PyTorch Forecasting aims to make time series forecasting with deep learning easy for both research and real-world cases alike. PyTorch Forecasting also has implementations for some state-of-the-art forecasting architectures, and we will come back to those in *Chapter 16, Specialized Deep Learning Architectures for Forecasting*. But now, let's use the high-level API in PyTorch Forecasting. This will significantly reduce our work in preparing PyTorch datasets. The `TimeSeriesDataset` class in PyTorch Forecasting takes care of a lot of boilerplate code dealing with different transformations, missing values, padding, and so on. We will be using this framework in this chapter when we look at different strategies to implement global deep learning forecasting models.



Notebook alert:

To follow along with the complete code, use the notebook named `01-Global_Deep_Learning_Models.ipynb` in the `Chapter15` folder. There are two variables in the notebook that act as a switch—`TRAIN_SUBSAMPLE = True` makes the notebook run for a subset of 10 households. `train_model = True` makes the notebook train different models (warning: training models on the full data takes upward of 3 hours each). `train_model = False` loads the trained model weights and predicts on them.

Preprocessing the data

We start by loading the necessary libraries and the dataset. We are using the preprocessed and feature-engineered dataset we created in *Chapter 6, Feature Engineering for Time Series Forecasting*. There are different kinds of features in the dataset and to make our feature assignment standardized, we use `namedtuple`. `namedtuple()` is a factory method in `collections` that lets you create subclasses of `tuple` with named fields. These named fields can be accessed using dot notation. We define `namedtuple` like this:

```
from collections import namedtuple
FeatureConfig = namedtuple(
    "FeatureConfig",
    [
        "target",
        "index_cols",
        "static_categoricals",
        "static_reals",
        "time_varying_known_categoricals",
        "time_varying_known_reals",
        "time_varying_unknown_reals",
        "group_ids"
    ],
)
```

Let's also quickly establish what these names mean:

- **target:** The column name of what we are trying to forecast.
- **index_cols:** The columns that we need to make as an index for quick access to data.
- **static_categoricals:** These are columns that are categorical in nature and do not change with time. They are specific to each time series. For instance, the *Acorn group* in our dataset is `static_categorical` because it is categorical in nature and is a value pertaining to a household.
- **static_reals:** These are columns that are numeric in nature and do not change with time. They are specific to each time series. For instance, the average energy consumption in our dataset is numeric in nature and pertains to a single household.

- `time_varying_known_categoricals`: These are columns that are categorical in nature and change with time and we know the future values. They can be seen as quantities that keep varying with time. A prime example would be holidays, which are categorical and vary with time, and we know the future holidays.
- `time_varying_known_reals`: These are columns that are numeric in nature and change with time and we know the future values. A prime example would be temperature, which is numeric and varies with time, and we know the future values (provided the source we are getting the weather from allows for forecasted weather data as well).
- `time_varying_unknown_reals`: These are columns that are numeric in nature and change with time and we don't know the future values. The target we are trying to forecast is an excellent example.
- `group_ids`: These columns uniquely identify each time series in the DataFrame.

Once defined, we can assign different values to each of these names, as follows:

```
feat_config = FeatureConfig(
    target="energy_consumption",
    index_cols=["LCLid", "timestamp"],
    static_categoricals=[
        "LCLid",
        "stdorToU",
        "Acorn",
        "Acorn_grouped",
        "file",
    ],
    static_reals=[],
    time_varying_known_categoricals=[
        "holidays",
        "timestamp_Dayofweek",
    ],
    time_varying_known_reals=["apparentTemperature"],
    time_varying_unknown_reals=["energy_consumption"],
    group_ids=["LCLid"],
)
```



The way of setting up a problem is slightly different in `neuralforecast` but the principles are the same. The different types of variables we define remain the same conceptually and just the parameter names we use to define them are different. `PyTorch Forecasting` needs the target to be included in `time_varying_unknown_reals` but `neuralforecast` doesn't. All these minor differences will be covered when we use `neuralforecast` to generate forecasts.

We can see that we are not using all the features as we did with machine learning models (*Chapter 10, Global Forecasting Models*). There are two reasons for that:

- Since we are using sequential deep learning models, a lot of the information we are trying to capture using rolling features and so on is already available to the model.
- Unlike robust gradient-boosted decision tree models, deep learning models aren't that robust to noise. So, irrelevant features would make the model worse.

There are a few preprocessing steps that are needed to make the dataset we have compatible with PyTorch Forecasting. PyTorch Forecasting needs a continuous time index as a proxy for time. Although we have a `timestamp` column, it has datetimes. So, we need to convert it to a new column, `time_idx`. The complete code is in the notebook, but the essence of the code is simple. We combine the train and test DataFrames and use a formula using the `timestamp` column to derive a new `time_idx` column. The formula is such that it increments every successive timestamp by one and is consistent between train and test. For instance, `time_idx` of the last timestep in train is 256, and `time_idx` of the first timestep in test would be 257. In addition to that, we also need to convert the categorical columns into object data types to play nicely with `TimeSeriesDataset` from PyTorch Forecasting.

For our experiments, we have chosen to have 2 days (96 timesteps) as the window and predict one single step ahead. To enable early stopping, we would need a validation set as well. **Early stopping** is a way of regularization (a technique to prevent overfitting, *Chapter 5*) where we keep monitoring the validation loss and stop training when the validation loss starts to increase. We have selected the last day of training (48 timesteps) as the validation data and 1 whole month as the final test data. But when we prepare these DataFrames, we need to take care of something: we have chosen two days as our history, and to forecast the first timestep in the validation or test set, we need the last two days of history along with it. So, we split our DataFrames as shown in the following diagram (the exact code is in the notebook):

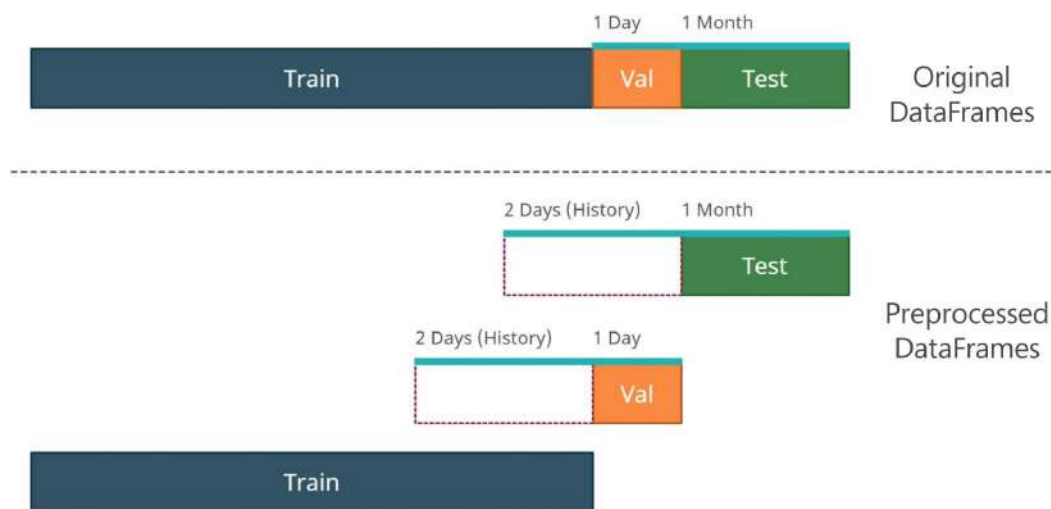


Figure 15.1: Train-validation-test split

Now, before using `TimeSeriesDataset` on our data, let's try to understand what it does and what the different parameters involved are.

Understanding `TimeSeriesDataset` from `PyTorch Forecasting`

`TimeSeriesDataset` automates the following tasks and more:

- Scaling numeric features and encoding categorical features:
 - Scaling the numeric features to have the same mean and variance helps gradient descent-based optimization to converge faster and better.
 - Categorical features need to be encoded as numbers so that we can handle them the right way inside the deep learning models.
- Normalizing the target variable:
 - In a global model context, the target variable can have different scales for different time series. For instance, a particular household typically has higher energy consumption, and some other households may be vacant and have little to no energy consumption. Scaling the target variable to a single scale helps the deep learning model to focus on learning the patterns rather than capturing the variance in scale.
- Efficiently converting the `DataFrame` into a dictionary of `PyTorch` tensors:
 - The dataset also takes in the information about different columns and converts the `DataFrame` into a dictionary of `PyTorch` tensors, separately handling the static and time-varying information.

These are the major parameters of `TimeSeriesDataset`:

- `data`: This is the pandas `DataFrame` holding all the data such that each row is uniquely identified with `time_idx` and `group_ids`.
- `time_idx`: This refers to the column name with the continuous time index we created earlier.
- `target`, `group_ids`, `static_categoricals`, `static_reals`, `time_varying_known_categoricals`, `time_varying_known_reals`, `time_varying_unknown_categoricals`, and `time_varying_unknown_reals`: We already discussed all these parameters in the *Preprocessing the data* section. These hold the same meaning.
- `max_encoder_length`: This sets the maximum window length given to the encoder.
- `min_decoder_length`: This sets the minimum window given in the decoding context.
- `target_normalizer`: This takes in a `Transformer` that normalizes the targets. There are a few normalizers built into `PyTorch Forecasting`—`TorchNormalizer`, `GroupNormalizer`, and `EncoderNormalizer`. `TorchNormalizer` does standard and robust scaling of the targets as a whole, whereas `GroupNormalizer` does the same but with each group separately (a group is defined by `group_ids`). `EncoderNormalizer` does the scaling at runtime by normalizing using the values in each window.

- **categorical_encoders:** This parameter takes in a dictionary of scikit-learn Transformers as a category encoder. By default, the category encoding is similar to `LabelEncoder`, which replaces each unique categorical value with a number, adding an additional category for unknown and NaN values.

For the full documentation, please refer to <https://pytorch-forecasting.readthedocs.io/en/stable/data.html#time-series-data-set>.

Initializing TimeSeriesDataset

Now that we know the major parameters, let's initialize a time series dataset using our data:

```
training = TimeSeriesDataSet(
    train_df,
    time_idx="time_idx",
    target=feat_config.target,
    group_ids=feat_config.group_ids,
    max_encoder_length=max_encoder_length,
    max_prediction_length=max_prediction_length,
    time_varying_unknown_reals=[
        "energy_consumption",
    ],
    target_normalizer=GroupNormalizer(
        groups=feat_config.group_ids, transformation=None
    )
)
```

Note that we have used `GroupNormalizer` so that each household is scaled separately using its own mean and standard deviation using the following well-known formula:

$$\frac{x - \text{mean}}{\text{standard deviation}}$$

`TimeSeriesDataset` also makes it easy to declare validation and test datasets as well using a factory method, `from_dataset`. It takes in another time series dataset as an argument and uses the same parameters, scalers, and so on, and creates new datasets:

```
# Defining the validation dataset with the same parameters as training
validation = TimeSeriesDataSet.from_dataset(training, pd.concat([val_
history, val_df]).reset_index(drop=True), stop_randomization=True)
# Defining the test dataset with the same parameters as training
test = TimeSeriesDataSet.from_dataset(training, pd.concat([hist_df, test_df]).
reset_index(drop=True), stop_randomization=True)
```

Notice that we concatenate the history to both `val_df` and `test_df` to make sure we can predict on the entire validation and test period.

Creating the dataloader

All that is left to do is to create the dataloader from `TimeSeriesDataset`:

```
train_dataloader = training.to_dataloader(train=True, batch_size=batch_size,
num_workers=0)
val_dataloader = validation.to_dataloader(train=False, batch_size=batch_size,
num_workers=0)
```

Before we proceed, let's solidify our understanding of the PyTorch Forecasting dataloader with the help of an example. The train dataloader we just created has split the `DataFrame` into a dictionary of PyTorch tensors. We have chosen 512 as a batch size and can inspect the dataloader using the following code:

```
# Testing the dataloader
x, y = next(iter(train_dataloader))
print("\nsizes of x =")
for key, value in x.items():
    print(f"\t{key} = {value.size()}")
print("\nsize of y =")
print(f"\ty = {y[0].size()}")
```

We will get an output as follows:

```
sizes of x =
    encoder_cat = torch.Size([512, 96, 0])
    encoder_cont = torch.Size([512, 96, 1])
    encoder_target = torch.Size([512, 96])
    encoder_lengths = torch.Size([512])
    decoder_cat = torch.Size([512, 1, 0])
    decoder_cont = torch.Size([512, 1, 1])
    decoder_target = torch.Size([512, 1])
    decoder_lengths = torch.Size([512])
    decoder_time_idx = torch.Size([512, 1])
    groups = torch.Size([512, 1])
    target_scale = torch.Size([512, 2])

size of y =
    y = torch.Size([512, 1])
```

Figure 15.2: Shapes of tensors in a batch of a train dataloader

We can see that the dataloader and `TimeSeriesDataset` have split the `DataFrame` into PyTorch tensors and packed them into a dictionary with the encoder and decoder sequences separate. We can also see that the categorical and continuous features are also separated.

The main *keys* we will be using from this dictionary are `encoder_cat`, `encoder_cont`, `decoder_cat`, and `decoder_cont`. The `encoder_cat` and `decoder_cat` keys have zero dimensions because we haven't declared any categorical features.

Visualizing how the dataloader works

Let's try to unpeel what happened here one level deeper and understand what `TimeSeriesDataset` has done visually:

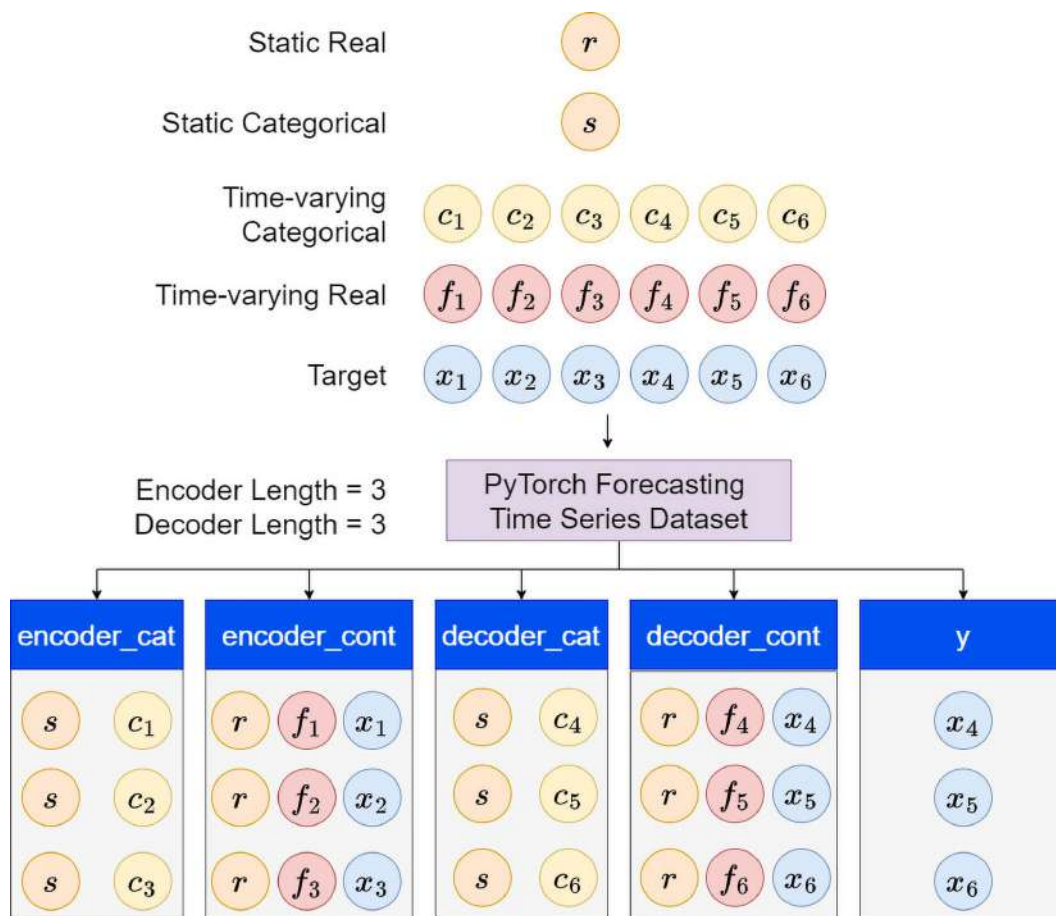


Figure 15.3: `TimeSeriesDataset`—an illustration of how it works

Let's assume we have a time series, x_1 to x_6 (this would be the target as well as `time_varying_unknown` in the `TimeSeriesDataset` terminology). We have a time-varying real, f_1 to f_6 , and a time-varying categorical, c_1 to c_6 . In addition to that, we also have a static real, r , and a static categorical, s . If we set the encoder and decoder length as 3, we will have the tensors constructed as shown in Figure 15.3. Notice how the static categorical and real are repeated for all timesteps. These different tensors are constructed so that the model encoder can be trained using the encoder tensors and the decoder tensors are used in the decoding process.

Now, let's proceed with building our first global model.

Building the first global deep learning forecasting model

PyTorch Forecasting uses PyTorch and PyTorch Lightning in the backend to define and train deep learning models. The models that can be used seamlessly with PyTorch Forecasting are essentially PyTorch Lightning models. However, the recommended approach is to inherit `BaseModel` from PyTorch Forecasting. The developer of PyTorch Forecasting has excellent documentation and tutorials to help new users use it the way they want. One tutorial worth mentioning here is titled *How to use custom data and implement custom models and metrics* (the link is in the *Further reading* section).

I have slightly modified the basic model from the tutorial to make it more flexible. The implementation can be found in `src/dl/ptf_models.py` under the name `SingleStepRNNModel`. The class takes in two parameters:

- `network_callable`: This is a callable that, when initialized, becomes a PyTorch model (inheriting `nn.Module`).
- `model_params`: This is a dictionary containing all the parameters necessary to initialize `network_callable`.

The structure is pretty simple. The `__init__` function initializes `network_callable` into a PyTorch model under the `network` attribute. The `forward` function sends the input to the network, formats the returned output the way PyTorch Forecasting wants, and returns it. It is a very short model because the bulk of the heavy lifting is done by `BaseModel`, which handles the loss calculation, logging, gradient descent, and so on. The benefit we get by defining a model this way is that we can now define standard PyTorch models and pass it to this model to make it work well with PyTorch Forecasting.

In addition to this, we also define an abstract class called `SingleStepRNN`, which takes in a set of parameters and initializes the corresponding network that is specified by the parameters. If the parameter specifies an LSTM, with two layers, then it will be initialized and saved under the `rnn` attribute. It also defines a fully connected layer under the `fc` attribute, which turns the output of the RNN into the prediction. The `forward` method is an abstract method that needs to be overwritten in any class subclassing this class.

Defining our first RNN model

Now that we have the necessary setup, let's define our first model inheriting the `SingleStepRNN` class we defined:

```
class SimpleRNNModel(SingleStepRNN):
    def __init__(
        self,
        rnn_type: str,
        input_size: int,
        hidden_size: int,
        num_layers: int,
        bidirectional: bool,
```

```

):
    super().__init__(rnn_type, input_size, hidden_size, num_layers,
bidirectional)
    def forward(self, x: Dict):
        # Using the encoder continuous which has the history window
        x = x["encoder_cont"] # x --> (batch_size, seq_len, input_size)
        # Processing through the RNN
        x, _ = self.rnn(x) # --> (batch_size, seq_len, hidden_size)
        # Using a FC Layer on last hidden state
        x = self.fc(x[:, -1, :]) # --> (batch_size, seq_len, 1)
        return x

```

This is the most straightforward implementation. We take `encoder_cont` from the dictionary and pass it through the RNN, and then use a fully connected layer on the last hidden state from the RNN to generate the prediction. If we take the example in *Figure 15.3*, we used x_1 to x_3 as the history and trained the model to predict x_4 (because we are using `min_decoder_length=1`, there will be just one timestep in the decoder and target).

Initializing the RNN model

Now, let's initialize the model using some parameters. I have defined two dictionaries for parameters:

- `model_params`: This has all the parameters necessary for the `SingleStepRNN` model to be initialized.
- `other_params`: These are all the parameters, such as `learning_rate`, `loss`, and so on, that we pass on to `SingleStepRNNModel`.

Now, we can initialize the PyTorch Forecasting model using a factory method it supports—`from_dataset`. This factory method lets us pass a dataset and infer some parameters from the dataset instead of filling everything in all the time:

```

model = SingleStepRNNModel.from_dataset(
    training,
    network_callable=SimpleRNNModel,
    model_params=model_params,
    **other_params
)

```

Training the RNN model

Training the model is just like we have been doing in previous chapters because this is a PyTorch Lightning model. We can follow these steps:

1. Initialize the trainer with early stopping and model checkpoints:

```

trainer = pl.Trainer(
    auto_select_gpus=True,

```

```

gpus=-1,
min_epochs=1,
max_epochs=20,
callbacks=[
    pl.callbacks.EarlyStopping(monitor="val_loss", patience=4*3),
    pl.callbacks.ModelCheckpoint(
        monitor="val_loss", save_last=True, mode="min", auto_insert_
metric_name=True
    ),
],
val_check_interval=2000,
log_every_n_steps=2000,
)

```

2. Fit the model:

```

trainer.fit(
    model,
    train_dataloaders=train_dataloader,
    val_dataloaders=val_dataloader,
)

```

3. Load the best model after training:

```

best_model_path = trainer.checkpoint_callback.best_model_path
best_model = SingleStepRNNModel.load_from_checkpoint(best_model_path)

```

The training can run for some time. To save you some time, I have included the trained weights for each of the models we are using, and if the `train_model` flag is `False`, it will skip training and load the saved weights.

Forecasting with the trained model

Now, after training, we can predict on the test dataset as follows:

```

pred, index = best_model.predict(test, return_index=True, show_progress_
bar=True)

```



 **Quick tip:** Enhance your coding experience with the **AI Code Explainer** and **Quick Copy** features. Open this book in the next-gen Packt Reader. Click the **Copy** button (1) to quickly copy code into your coding environment, or click the **Explain** button (2) to get the AI assistant to explain a block of code to you.

```
function calculate(a, b) {  
  return {sum: a + b};  
};
```

Copy

Explain

1

2

 **The next-gen Packt Reader** is included for free with the purchase of this book. Unlock it by scanning the QR code below or visiting <https://www.packtpub.com/unlock/9781835883181>.



We store the predictions in a DataFrame and evaluate them using our standard metrics: MAE, MSE, meanMASE, and Forecast Bias. Let's see the results:

Algorithm	MAE	MSE	meanMASE	Forecast Bias
simple	0.085441	0.030798	1.078673	1.410458

Figure 15.4: Aggregate results using the baseline global model

This is not a very good model because we know from *Chapter 10, Global Forecasting Models*, that the baseline global model using LightGBM was as follows:

- MAE = 0.079581
- MSE = 0.027326
- meanMASE = 1.013393
- Forecast Bias = 28.718087

Apart from Forecast Bias, our global model is nowhere close to the best. Let's refer to the **global machine learning model** as **GFM(ML)** and the current model as **GFM(DL)** for the rest of our discussion. Now, let's start looking at some strategies to make the global model better.

Using time-varying information

The GFM(ML) used all the available features. So, obviously, that model had access to a lot more information than the GFM(DL) we have built until now. The GFM(DL) we just built only takes in the history and nothing else. Let's change that by including time-varying information. We will just use time-varying real features this time because dealing with categorical features is a topic I want to leave for the next section.

We initialize the training dataset the same way as before, but we add `time_varying_known_reals=feat_config.time_varying_known_reals` to the initialization parameters. Now that we have all the datasets created, let's move on to setting up the model.

To set up the model, we need to understand one concept. We are now using the history of the target and time-varying known features. In *Figure 15.3*, we saw how `TimeSeriesDataset` arranges the different kinds of variables in PyTorch tensors. In the previous section, we used only `encoder_cont` because there were no other variables to worry about. But now, we have time-varying variables along with it, which brings an added complication. If we take a step back and think about it, in the single-step-ahead forecasting context, we can see that the time-varying variables and the history of the target cannot be of the same timestep. Let's use a visual example to elucidate:

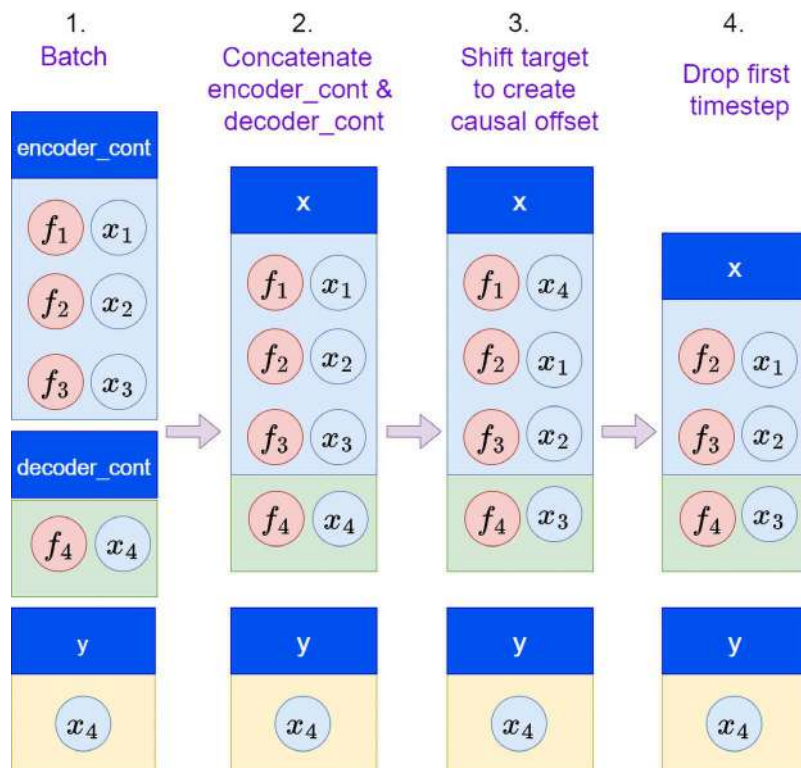


Figure 15.5: Using time-varying variables for training

Following the same spirit of the example from *Figure 15.3*, but reducing it to fit our context here, we have a time series, x_1 to x_4 , and a time-varying real variable, f_1 to f_4 . So, for `max_encoder_length=3` and `min_decoder_length=1`, we would have `TimeSeriesDataset` make the tensors, as shown in *Step 1* in *Figure 15.5*.

Now, for each timestep, we have the time-varying variable, f , and the history, x , in `encoder_cont`. The time-varying variable, f , is a variable for which we know the future values as well and therefore, there is no causal constraint on that variable. That means that for predicting the timestep, t , we can use f_t because it is known. However, the history of the target variable is not. We do not know the future because it is the very quantity we are trying to forecast. That means that there is a causal constraint on x and, because of this, we cannot use x_t to predict timestep t . But the way the tensors are formed right now, we have f and x aligned on timesteps and if we passed them through a model, we would be essentially cheating because we would be using x_t to predict timestep t . Ideally, there should be an offset between the history, x , and the time-varying feature, f , such that at timestep t , the model sees x_{t-1} , then sees f_t , and then predicts x_t .

To achieve that, we do the following:

1. Concatenate `encoder_cont` and `decoder_cont` because we need to use f_4 to predict timestep $t = 4$ (*Step 2* in *Figure 15.5*).
2. Shift the target history, x , forward by one timestep so that f_t and x_{t+1} are aligned (*Step 3* in *Figure 15.5*).
3. Drop the first timestep because we don't have the history to go with the first timestep (*Step 4* in *Figure 15.5*).

This is exactly what we need to implement in our forward method in the new model we defined, `DynamicFeatureRNNModel`, as well:

```
def forward(self, x: Dict):
    # Step 2 in Figure 15.5
    x_cont = torch.cat([x["encoder_cont"], x["decoder_cont"]], dim=1)
    # Step 3 in Figure 15.5
    x_cont[:, :, -1] = torch.roll(x_cont[:, :, -1], 1, dims=1)
    x = x_cont
    # Step 4 in Figure 15.5
    x = x[:, 1:, :] # x -> (batch_size, seq_len, input_size)
    # Processing through the RNN
    x, _ = self.rnn(x) # --> (batch_size, seq_len, hidden_size)
    # Using a FC Layer on Last hidden state
    x = self.fc(x[:, -1, :]) # --> (batch_size, seq_len, 1)
    return x
```

Now, let's train this new model and see how it performs. The exact code is in the notebook and is exactly the same as before:

Algorithm	MAE	MSE	meanMASE	Forecast Bias
simple	0.0854	0.0308	1.0787	1.41%
simple+time_varying	0.0851	0.0310	1.0764	0.73%

Figure 15.6: Aggregate results using the time-varying features

It looks like having temperature as a feature did make the model slightly better, but there's still a long way to go. Not to worry; we have other features to use.

Using static/meta information

There are some features such as the Acorn group, whether dynamic pricing is enabled, and so on, that are specific to a household, which will help the model learn patterns specific to these groups. Naturally, including that information makes intuitive sense. However, as we discussed in *Chapter 10, Global Forecasting Models*, categorical features do not play well with machine learning models because they aren't numerical. In that chapter, we discussed a few ways of encoding categorical features into numerical representations. We can use any of those in a deep learning model as well. But there is one way of handling categorical features that is unique to deep learning models—**embedding vectors**.

One-hot encoding and why it is not ideal

One of the ways of converting categorical features to numerical representation is one-hot encoding. It encodes the categorical features in a higher dimension, placing the categorical values equally distant in that space. The size of the dimension it requires to encode the categorical values is equal to the cardinality of the categorical variable. For a more detailed discussion on one-hot encoding, refer to *Chapter 10, Global Forecasting Models*.

The representation that we would get after the one-hot encoding of a categorical feature is what we call a **sparse representation**. If the cardinality of the categorical feature (number of unique values) is C , each row representing a value of the categorical feature would have $C - 1$ zeros. So, the representation is predominantly zeros and hence is called a sparse representation. This causes the overall dimension required to effectively encode a categorical feature to be equal to the cardinality of the vector. Therefore, one-hot encoding of a categorical feature with 5,000 unique values instantly adds 5,000 dimensions to the problem you are solving.

In addition to that, one-hot encoding is also completely uninformed. It places each categorical value equidistant from each other without any regard for the possible similarity between those values. For instance, if we were encoding the days in a week, one-hot encoding would place each day in a completely different dimension, making them equidistant from each other. But if we think about it, Saturday and Sunday should be closer together than the other weekdays on account of them being the weekend, right? This kind of information is not captured through one-hot encoding.

Embedding vectors and dense representations

An embedding vector is a similar representation, but instead of a sparse representation, it strives to give us a dense representation of a categorical feature. We can achieve this by using an embedding layer. The embedding layer can be thought of as a mapping between each categorical value and a numerical vector, and this vector can have a much lower dimension than the cardinality of the categorical feature. The only question that remains is “How do we know what vector to choose for each categorical value?”

The good news is that we need not because the embedding layer is trained along with the rest of the network. So, while training a model for some task, the model itself figures out what the best vector representation is for each categorical value. This approach is really popular in natural language processing, where thousands of words are embedded into dimensions as small as 200 or 300. In PyTorch, we can accomplish this by using `nn.Embedding`, which is a module that is a simple lookup table that stores the embeddings of fixed discrete values and size. There are two mandatory parameters while initializing:

- `num_embeddings`: This is the size of the dictionary of embeddings. In other words, this is the cardinality of the categorical feature.
- `embedding_dim`: This is the size of each embedding vector.

Now, let’s come back to global modeling. Let’s first introduce the static categorical features. Please note that we are also including the time-varying categorical because now we know how to deal with categorical features in a deep learning model. The code to initialize the dataset is the same, with the addition of the following two parameters to the initialization:

- `static_categoricals=feat_config.static_categoricals`
- `time_varying_known_categoricals=feat_config.time_varying_known_categoricals`

Defining a model with categorical features

Now that we have the datasets, let’s look at how we can define the `__init__` function in our new model, `StaticDynamicFeatureRNNModel`. In addition to invoking the parent model, which sets up the standard RNN and fully connected layer, we also set up the embedding layers using an input, `embedding_sizes`. `embedding_sizes` is a list of tuples (*cardinality and embedding size*) for each categorical feature:

```
def __init__(
    self,
    rnn_type: str,
    input_size: int,
    hidden_size: int,
    num_layers: int,
    bidirectional: bool,
    embedding_sizes = []
):
    super().__init__(rnn_type, input_size, hidden_size, num_layers,
                     bidirectional)
```

```
self.embeddings = torch.nn.ModuleList(
    [torch.nn.Embedding(card, size) for card, size in embedding_sizes]
)
```

We used `nn.ModuleList` to store a list of `nn.Embedding` modules, one for each categorical feature. While initializing this model, we will need to give `embedding_sizes` as input. The embedding size required for each categorical feature is technically a hyperparameter that we can tune. But there are a few rules of thumb to get you started. The idea behind these thumb rules is that the bigger the cardinality of the categorical feature, the larger the embedding size required to encode the information in them. Also, the embedding size can be much smaller than the cardinality of the categorical feature. The rule of thumb that we have adopted is as follows:

$$\min\left(50, \text{round}\left(\frac{c+1}{2}\right)\right)$$

Therefore, we create the `embedding_sizes` list of tuples using the following code:

```
# Finding the cardinality using the categorical encoders in the dataset
cardinality = [len(training.categorical_encoders[c].classes_) for c in
training.categoricals]
# using the cardinality list to create embedding sizes
embedding_sizes = [
    (x, min(50, (x + 1) // 2))
    for x in cardinality
]
```

Now, turning our attention toward the forward method, it is going to be similar to the previous model, but with an additional part to handle the categorical features. We essentially use the embedding layers to convert the categorical features into embeddings and concatenate them with the continuous features:

```
def forward(self, x: Dict):
    # Using the encoder and decoder sequence
    x_cont = torch.cat([x["encoder_cont"], x["decoder_cont"]], dim=1)
    # Roll target by 1
    x_cont[:, :, -1] = torch.roll(x_cont[:, :, -1], 1, dims=1)
    # Combine the encoder and decoder categoricals
    cat = torch.cat([x["encoder_cat"], x["decoder_cat"]], dim=1)
    # if there are categorical features
    if cat.size(-1) > 0:
        # concatenating all the embedding vectors
        x_cat = torch.cat([emb(cat[:, :, i]) for i, emb in enumerate(self.
embeddings)], dim=-1)
        # concatenating continuous and categorical
        x = torch.cat([x_cont, x_cat], dim=-1)
    else:
```

```

    x = x_cont
    # dropping first timestep
    x = x[:,1:,:] # x --> (batch_size, seq_len, input_size)
    # Processing through the RNN
    x, _ = self.rnn(x) # --> (batch_size, seq_len, hidden_size)
    # Using a FC Layer on Last hidden state
    x = self.fc(x[:, -1, :]) # --> (batch_size, seq_len, 1)
    return x

```

Now, let's train this new model with static features and see how it performs:

Algorithm	MAE	MSE	meanMASE	Forecast Bias
simple	0.0854	0.0308	1.0787	1.41%
simple+time_varying	0.0851	0.0310	1.0764	0.73%
simple+static+time_varying	0.0843	0.0297	1.0685	0.94%

Figure 15.7: Aggregate results using the static and time-varying features

Adding the static variables also improved our model. Now, let's look at another strategy that adds another key piece of information to the model.

Using the scale of the time series

We used `GroupNormalizer` in `TimeSeriesDataset` to scale each household using its own mean and standard deviation. We did this because we wanted to make the target zero mean and unit variance so that the model does not waste effort trying to change its parameters to capture the scale of individual household consumption. Although this is a good strategy, we do have some information loss here. There may be patterns that are specific to households whose consumption is on the larger side and some other patterns that are specific to households that consume much less. But now, they are both lumped in together and the model tries to learn common patterns. In such a scenario, these unique patterns seem like noise to the model because there is no variable to explain them.

The bottom line is that there is information in the scale that we removed, and adding that information back would be beneficial. So, how do we add it back? Definitely not by including the unscaled targets, which brings back the disadvantage that we were trying to get away from in the first place. A way to do it is to add the scale information as static-real features to the model. We would have kept track of the mean and standard deviation of each household when we scaled them in the first place (because we need them to do the inverse transformation and get back the original targets). All we need to do is make sure we include them as a static real variable so that the model has access to the scale information while learning the patterns in the time series dataset.

PyTorch Forecasting makes this easier for us by having a handy parameter in `TimeSeriesDataset` called `add_target_scales`. If you make it `True`, then `encoder_cont` and `decoder_cont` will also have the mean and standard deviation of individual time series.

Nothing changes in our existing model; all we need to do is add this parameter to `TimeSeriesDataset` while initializing it and train and predict using the model. Let's see how that worked out for us:

Algorithm	MAE	MSE	meanMASE	Forecast Bias
simple	0.0854	0.0308	1.0787	1.41%
simple+time_varying	0.0851	0.0310	1.0764	0.73%
simple+static+time_varying	0.0843	0.0297	1.0685	0.94%
simple+static+time_varying+scale	0.0822	0.0298	1.0395	-3.20%

Figure 15.8: Aggregate results using the static, time-varying, and scale features

The scale information has improved the model yet again. With that, let's look at one of the last strategies we will be covering in this book.

Balancing the sampling procedure

We saw a few strategies for improving a global deep learning model by adding new types of features. Now, let's look at a different aspect that is relevant in a global modeling context. In an earlier section, when we were talking about global deep learning models, we talked about how the process by which we sample a window of sequence to feed to our model can be thought of as a two-step process:

1. Sampling a time series out of a set of time series.
2. Sampling a window out of that time series.

Let's use an analogy to make the concept clearer. Imagine we have a large bowl that we have filled with N balls. Each ball in the bowl represents a time series in the dataset (a household in our dataset). Now, each ball, i , has M_i chits of paper representing all the different windows of samples we can draw from it.

In the batch sampling we use by default, we open all the balls, dump all the chits into the bowl, and discard the balls. Now, with our eyes closed, we pick B chits out of this bowl and set them aside. This is a batch that we sample from our dataset. We do not have any information that separates the chits from each other so the probability of picking any chit is equal, which can be formulated as:

$$\frac{1}{\sum_{i=0}^N M_i}$$

Now, let's add something to our analogy to the data. We know that we have different kinds of time series—different lengths, different levels of consumption, and so on. Let's pick one aspect, the length of the series, for our example (although it applies to other aspects as well). So, if we discretize the length of our time series, we end up with different bins; let's assign a color for each bin. So, now we have C different-colored balls in the bowl and the chits of paper also are colored accordingly.

In our current sampling strategy (where we dump all the chits of paper, now colored, and pick B chits at random), we would end up replicating the probability distribution of our bowl in a batch. It is not a stretch to understand that if the bowl has more of the longer time series than shorter ones, the chits we draw will also have that bias. Consequently, the batch will also be biased toward a long time series. What happens because of that?

In mini-batch stochastic gradient descent (we saw this in *Chapter 11, Introduction to Deep Learning*), we do a gradient update every mini-batch, and we use this gradient to update the model parameters so that we move closer to the minima of the loss function. Therefore, if a mini-batch is biased toward a particular type of case, then the gradient updates would be biased toward a solution that works better for them. There are good parallels to be drawn here to imbalanced learning. Longer time series and shorter time series may have different patterns, and having this sampling imbalance causes the model to learn patterns that work well for the longer time series and not so well for the shorter ones.

Visualizing the data distribution

We calculated the length of each household (LCLid) and binned them into 10 bins—bin_0 for the shortest bin and bin_9 for the longest bin:

```
n_bins= 10
# Calculating the length of each LCLid
counts = train_df.groupby("LCLid")["timestamp"].count()
# Binning the counts and renaming
out, bins = pd.cut(counts, bins=n_bins, retbins=True)
out = out.cat.rename_categories({
    c:f"bin_{i}" for i, c in enumerate(out.cat.categories)
})
```

Let's visualize the distribution of the bins in the original data:

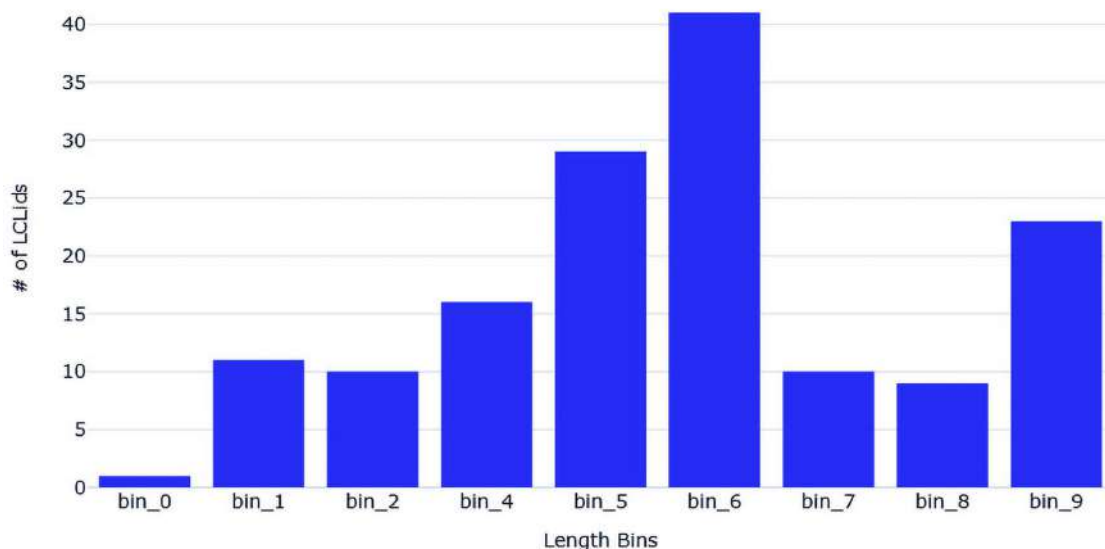


Figure 15.9: Distribution of length of time series

We can see that `bin_5` and `bin_6` are the most common lengths while `bin_0` is the least common. Now, let's get the first 50 batches from the dataloader and plot them as a stacked bar chart to check the distribution in each batch:

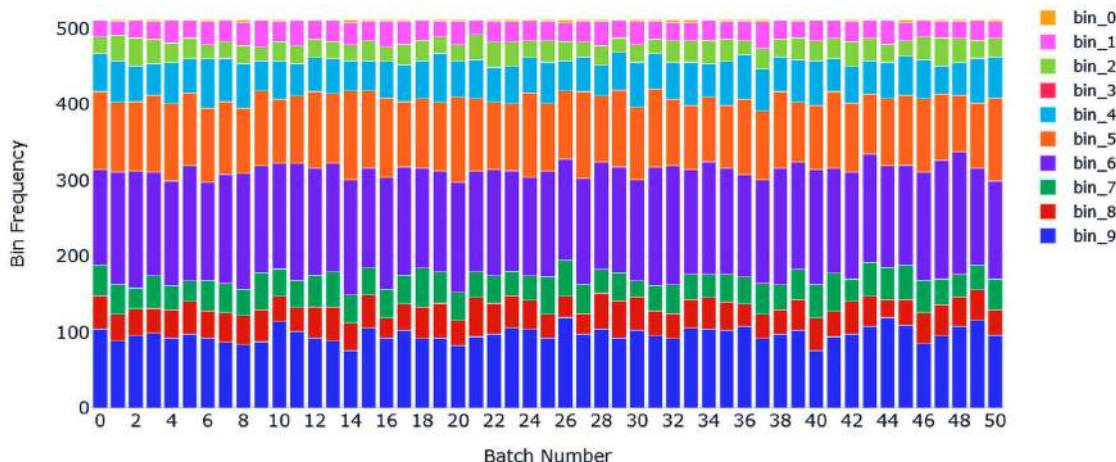


Figure 15.10: Stacked bar chart of batch distribution

We can see that the same distribution you saw in *Figure 15.9* is replicated in the batch distributions as well with `bin_5` and `bin_6` leading the pack. `bin_0` is barely making an appearance and LCLids that are in `bin_0` would not have been learned that well.

Tweaking the sampling procedure

Now what do we do? Let's step into the analogy of bowls with chits inside for a bit. We were picking a ball at random, and we saw that the resulting distribution was identical to the original distribution of colors. Therefore, to get a more balanced distribution of colors in a batch, we need to sample different colored chits at different probabilities. In other words, we should be sampling more from colors that have low representation in the original distribution and less from colors that dominate the original representation.

Let's look at the process by which we are selecting the chits from the bowl from another perspective. We know that the probability of selecting each chit in the bowl is equal. So, another way to select chits from the bowl is by using a uniform random number generator. We pick a chit from the bowl, generate a random number between 0 and 1 (p), and select the chit if the random number is less than 0.5 ($p < 0.5$). So, it is equally likely that we select or reject the chit. We continue this until we get B samples. Although a bit more inefficient than the previous procedure, this sampling process approximates the original procedure closely. The advantage here is that we have a threshold now with which we can tweak our sampling to suit our needs. Having a lower threshold makes the chit harder to accept under this sampling procedure, and having a higher threshold makes it easier to accept.

Now that we have a threshold with which we can tweak the sampling procedure, all we need to do is find out the right thresholds for each of the chits so that the resulting batch has a uniform representation of all the colors.

In other words, we need to find and assign the right weight to each LCLid such that the resulting batch will have an even distribution of all length bins.

How do we do that? There is a very simple strategy for that. We want the weights to be lower for length bins that have a lot of samples, and higher for length bins that have fewer samples. We can get this kind of weight by taking the inverse of the count of each bin. If there are C LCLids in a bin, the weight of the bin can be $1/C$. The *Further reading* section has a link where you can read more about weighted random sampling and the different algorithms used for the purpose.

TimeSeriesDataset has an internal index, which is a DataFrame with all the samples it can draw from the dataset. We can use that to construct our array of weights:

```
# TimeSeriesDataset stores a df as the index over which it samples
df = training.index.copy()
# Adding a bin column to it to represent the bins we have created
df['bins'] = [f"bin_{i}" for i in np.digitize(df["count"].values, bins)]
# Calculate Weights as inverse counts of the bins
weights = 1/df['bins'].value_counts(normalize=True)
# Assigning the weights back to the df so that we have an array of
# weights in the same shape as the index over which we are going to sample
weights = weights.reset_index().rename(columns={"index": "bins",
"bins": "weight"})
df = df.merge(weights, on='bins', how='left')
probabilities = df.weight.values
```

This way ensures that the probabilities array has the same length as the internal index over which TimeSeriesDataset samples and that is a mandatory requirement when using this technique—each possible window should have a corresponding weight attached to it.

Now that we have this weight, there is an easy way to put this into practice. We can use WeightedRandomSampler from PyTorch, which has been created specifically for this purpose:

```
from torch.utils.data import WeightedRandomSampler
sampler = WeightedRandomSampler(probabilities, len(probabilities))
```

Using and visualizing the dataloader with WeightedRandomSampler

Now, we can use this sampler in the dataloaders we create from TimeSeriesDataset:

```
train_dataloader = training.to_dataloader(train=True, batch_size=batch_size,
num_workers=0, sampler=sampler)
```

Let's visualize the first 50 batches like before and see the difference:

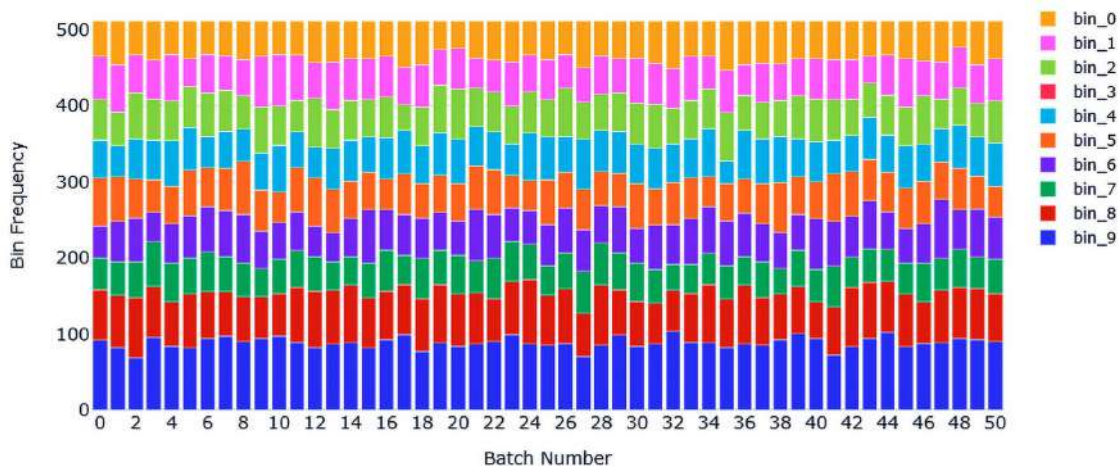


Figure 15.11: Stacked bar chart of batch distribution with weighted random sampling

Now, we can see a more uniform distribution of bins in each batch. Let's also see the results after training the model using this new dataloader:

Algorithm	MAE	MSE	meanMASE	Forecast Bias
simple	0.0854	0.0308	1.0787	1.41%
simple+time_varying	0.0851	0.0310	1.0764	0.73%
simple+static+time_varying	0.0843	0.0297	1.0685	0.94%
simple+static+time_varying+scale	0.0822	0.0298	1.0395	-3.20%
simple+static+time_varying+num_sampler	0.0815	0.0297	1.0372	-4.06%

Figure 15.12: Aggregate results using the static, time-varying, and scale features along with batch samplers

Looks like the sampler also made a good improvement in the model in all metrics, except Forecast Bias. Although we have not achieved better results than the GFM(ML) (which had an MAE of 0.079581), we are close enough. Maybe with some hyperparameter tuning, partitioning, or stronger models, we might reach closer to that number, or we may not. We used a custom sampling option to make the length of the time series balanced in a batch. We can use the same techniques to balance it on other aspects such as the level of consumption, region, or any other aspect that seems relevant. As always in machine learning, we will need to go with our experiments to say anything for sure, and all we need to do is form our hypothesis about the problem statement and construct experiments to validate that hypothesis.

With that, we have come to the end of yet another practical-heavy (and compute-heavy) chapter. Congratulations on making it through the chapter; feel free to go back and refer to any points that haven't quite landed yet.

Summary

After having built a strong foundation on deep learning models in the last few chapters, we started to look at a new paradigm of global models in the context of deep learning models. We learned how to use PyTorch Forecasting, an open-source library for forecasting using deep learning, and used the feature-filled `TimeSeriesDataset` to start developing our own models.

We started off with a very simple LSTM in the global context and saw how we can add time-varying information, static information, and the scale of individual time series to the features to make models better. We closed by looking at an alternating sampling procedure for mini-batches that helps us present a more balanced view of the problem in each batch. This chapter is by no means an exhaustive list of all such techniques to make the forecasting models better. Instead, this chapter aims to build the right kind of thinking that is necessary to work on your own models and make them work better than before.

Now that we have a strong foundation in deep learning and global models, it is time to take a look at a few specialized deep learning architectures that have been proposed over the years for time series forecasting in the next chapter.

Further reading

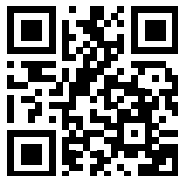
You can check out the following sources for further reading:

- *How to use custom data and implement custom models and metrics* (PyTorch Forecasting): <https://pytorch-forecasting.readthedocs.io/en/stable/tutorials/building.html>
- *Random Sampling from Databases* by Frank Olken, pages 22–23: <https://dsf.berkeley.edu/papers/UCB-PhD-olken.pdf>

Join our community on Discord

Join our community's Discord space for discussions with authors and other readers:

<https://packt.link/mts>



16

Specialized Deep Learning Architectures for Forecasting

Our journey through the world of **deep learning (DL)** is coming to an end. In the previous chapter, we were introduced to the global paradigm of forecasting and saw how we can make a simple model such as a **Recurrent Neural Network (RNN)** perform close to the high benchmark set by global machine learning models. In this chapter, we are going to review a few popular DL architectures that were designed specifically for time series forecasting. With these more sophisticated model architectures, we will be better equipped to handle problems in the wild that call for more powerful models than vanilla RNNs and LSTMs.

In this chapter, we will be covering these main topics:

- The need for specialized architectures
- Introduction to NeuralForecast
- Neural Basis Expansion Analysis for Interpretable Time Series Forecasting
- Neural Basis Expansion Analysis for Interpretable Time Series Forecasting with Exogenous Variables
- Neural Hierarchical Interpolation for Time Series Forecasting
- Autoformer
- LTSF-Linear family
- Patch Time Series Forecasting
- iTransformer
- Temporal Fusion Transformer
- TSMixer
- Time Series Dense Encoder

Technical requirements

You will need to set up an Anaconda environment by following the instructions in the *Preface* to get a working environment with all the packages and datasets required for the code in this book.

The code associated with this chapter can be found at <https://github.com/PacktPublishing/Modern-Time-Series-Forecasting-with-Python/tree/main/notebooks/Chapter16>.

You will need to run the following notebooks for this chapter:

- 02-Preprocessing_London_Smart_Meter_Dataset.ipynb in Chapter02
- 01-Setting_up_Experiment_Harness.ipynb in Chapter04
- 01-Feature_Engineering.ipynb in Chapter06

The need for specialized architectures

Inductive bias, or learning bias, refers to a set of assumptions a learning algorithm makes to generalize the function it learns on training data to unseen data. Inductive bias is not inherently a bad thing and is different from “bias” in the context of bias and variance in learning theory. We use and design inductive bias either through model architectures or through feature engineering. For instance, a **Convolutional Neural Network (CNN)** works better on images than a standard **Feed Forward Network (FFN)** on pure pixel input because the CNN has the locality and spatial bias that FFNs do not have. Although the FFN is theoretically a universal approximator, we can learn better models with the inductive bias the CNN has.

Deep learning is thought to be a completely data-driven approach where the feature engineering and final task are learned end to end, thus avoiding the inductive bias that the modelers bake in while designing the features. But that view is not entirely correct. These inductive biases, which used to be put in through the features, now make their way through the design of architecture. Every DL architecture has its own inductive bias, which is why some types of models perform better on some types of data. For instance, a CNN works well on images, but not as much on sequences because the spatial inductive bias and translational equivariance that the CNN brings to the table are most effective on images.

In an ideal world, we would have an infinite supply of good, annotated data and we would be able to learn entirely data-driven networks with no strong inductive bias. But sadly, in the real world, we will never have enough data to learn such complex functions. This is where designing the right kind of inductive bias makes or breaks the DL system. We used to heavily rely on RNNs for sequences and they had a strong auto-regressive inductive bias baked into them. But later, Transformers, which have a much weaker inductive bias for sequences, came in, and with large amounts of data, they were able to learn better functions for sequences. Therefore, this decision about how strong an inductive bias we bake into models is an important question in designing DL architectures.

Over the years, many DL architectures have been proposed specifically for time series forecasting and each of them has its own inductive bias attached to it. We’ll not be able to review every single one of those models, but we will cover the major ones that made a lasting impact on the field. We will also look at how we can use a few open-source libraries to train those models on our data.

We will exclusively focus on models that can handle the global modeling paradigm, directly or indirectly. This is because of the infeasibility of training separate models for each time series when we are forecasting at scale.

We are going to look at a few popular architectures developed for time series forecasting. One of the major factors influencing the inclusion of a model is also the availability of stable open-source frameworks that support these models. This is in no way a complete list because there are many architectures we are not covering here. I'll try and share a few links in the *Further reading* section to get you started on your journey of exploration.

Before we get into the meat of the chapter, let's understand the library we are going to use for it.

Introduction to NeuralForecast

NeuralForecast is yet another library from the wonderful folks at NIXTLA. You might recall the name from *Chapter 4, Setting a Strong Baseline Forecast*, where we used `statsforecast` for classical time series models like ARIMA, ETS, and so on. They have a whole suite of open-source libraries for time series forecasting (`mlforecast` for machine learning based forecasts, `hierarchicalforecast` for reconciling forecasts for hierarchical data, `utilsforecast` with some utilities for forecasting, `datasetsforecast` with some ready-to-use datasets, and `TimeGPT`, their foundational model for time series).

Since we have learned how to use `statsforecast`, extending that to `neuralforecast` is going to be easy because both libraries maintain similar APIs, structure, and ways of working. `neuralforecast` offers both classic and cutting-edge deep learning models in an easy-to-use API, which makes it perfect for the practical side of the chapter.

NeuralForecast is structured to offer an intuitive and flexible API that integrates seamlessly with modern data science workflows. The package includes implementations of several prominent models, each catering to different aspects of time series forecasting.

Common parameters and configurations

Similar to `statsforecast`, `neuralforecast` also expects the input data to be in a particular form:

- `ds`: This column should have the time index. It can either be a datetime column or an integer column which represents time.
- `y`: This column should have the time series we are trying to forecast.
- `unique_id`: This column lets us differentiate different time series with a unique ID that we choose. It can be the household ID in our data or any other uniquely identifying ID that we give.

Most models in the `neuralforecast` package share a set of common parameters that control aspects like:

- `stat_exog_list`: This is a list of static continuous columns.
- `hist_exog_list`: This is a list of temporal exogenous features for which the history is available.
- `futr_exog_list`: This is a list of temporal exogenous features for which the future is available.

- **learning_rate:** This dictates the speed at which a model learns. A higher rate might converge faster but can overshoot optimal weights, while a lower rate ensures more stable convergence at the cost of speed.
- **batch_size:** This influences the amount of data fed into the model at each training step, affecting both memory usage and training dynamics.
- **max_steps:** This defines the maximum number of epochs – the number of times the entire dataset is passed forward and backward through the neural network. It is max because we can also add early stopping and, in that case, the number of epochs can be lower than this as well.
- **loss:** This is the metric used to gauge the difference between predicted values and actual values, guiding the optimization process. For a list of loss functions included, refer to the NIXTLA documentation: <https://nixtlaverse.nixtla.io/neuralforecast/losses.pytorch.html>
- **scaler_type:** This is a string indicating the type of temporal normalization used. Temporal normalization does the scaling for each instance of the batch separately at the window level. Some examples include ['minmax', 'robust', 'standard']. This is only applicable for window-based models like NBEATS and TimesNet and not recurrent models like RNNs. For a full list of scalers, check the NIXTLA temporal scalers: <https://nixtlaverse.nixtla.io/neuralforecast/common.scalers.html>.
- **early_stop_patience_steps:** If defined, this sets the number of steps we will wait without any improvement in validation scores before stopping training.
- **random_seed:** This defines the random seed, which is essential for reproducibility.

Practitioner's tip:



The parameters `stat_exog_list`, `hist_exog_list`, and `futr_exog_list` are only available to models which support them. Do check the documentation of the model you are going to use to see if it supports these parameters. Some models support all three, some only support `futr_exog_list`, and so on. The entire list of models and what is available can be found here: <https://nixtlaverse.nixtla.io/neuralforecast/docs/capabilities/overview.html>.

Also, if you have some features for which you only have historical data, they go in `hist_exog_list`. If you have some features for which you have both historical and future data, they go in both `hist_exog_list` and `futr_exog_list`.

Apart from these common model parameters, `neuralforecast` also has a core class, `NeuralForecast`, which orchestrates the training (just like we have `StatsForecast` in `statsforecast`). Similar to `statsforecast`, this is where we define the list of models we need to forecast and so on. Let's look at a few parameters in this class as well:

- **models:** This defines a list of models that we need to fit or predict.
- **freq:** This sets the frequency of the time series we want to forecast. It can either be a string (a valid pandas or polars offset alias) or an integer. This is used for generating future dataframes for prediction and should be defined according to the data you have.

If `ds` is a datetime column, then `freq` can be a string indicating the frequency of repetition (like 'D' for days, 'H' for hours, etc.) or an integer indicating a multiplier of the default unit (usually days) to determine the interval between each date. And if `ds` is a numerical column, then `freq` should also be a numerical column indicating a fixed numerical increment between values.

- `local_scaler_type`: This is an alternate way to scale the time series. While `scaler_type` in Windows-based models scales the time series for each window, this scales each time series as a pre-processing step. For each `unique_id`, this step scales the time series separately and stores the scalers so that the inverse transformations can be applied while predicting.

A typical workflow involving `neuralforecast` looks as below:

```
from neuralforecast import NeuralForecast
from neuralforecast.models import LSTM
horizon = 12
models = [LSTM(h=horizon,
               max_steps=500,
               scaler_type='standard',
               encoder_hidden_size=64,
               decoder_hidden_size=64,),
          ]
nf = NeuralForecast(models=models, freq='M')
nf.fit(df=Y_df)
Y_hat_df = nf.predict()
```

“Auto” models

One of the standout features of the `neuralforecast` package is the inclusion of “auto” models. These models automate the process of hyperparameter tuning and model selection, simplifying the workflow for users. By utilizing techniques from **automated machine learning (AutoML)**, these models can adapt their architecture and settings based on the dataset, significantly reducing the manual effort involved in the model configuration. They have intelligent default ranges defined so that, even if you don't declare any ranges to tune, they will take the default ranges and tune the models. Additional information can be found here: <https://nixtlaverse.nixtla.io/neuralforecast/models.html>.

Exogenous features

`NeuralForecast` can also easily incorporate exogenous variables into the forecasting process (depending on the capability of the model). Exogenous features, which are external influences that can affect the target variable, are crucial for improving forecasting accuracy, especially when these external factors significantly impact the outcome. Many models within the `neuralforecast` package can integrate such features to refine predictions by accounting for additional information that may not be present in the time series data itself.

For instance, the inclusion of holiday effects, weather conditions, or economic indicators as exogenous variables can provide critical insights that pure historical data cannot. This feature is especially useful in models like NBEATSx, NHITS, and TSMixerx within the package, which can model complex interactions between both the historical and future exogenous inputs. By handling exogenous features effectively, NeuralForecast enhances the models' ability to forecast accurately in real-world scenarios where external factors play a pivotal role. To check which models can handle exogenous information, refer to the documentation on the website: <https://nixtlaverse.nixtla.io/neuralforecast/docs/capabilities/overview.html>

Now, without further ado, let's get started on the first model on the list.

Neural Basis Expansion Analysis for Interpretable Time Series Forecasting (N-BEATS)

The first model that used some components from DL (we can't call it DL because it is essentially a mix of DL and classical statistics) and made a splash in the field was a model that won the M4 competition (univariate) in 2018. This was a model by Slawek Smyl from Uber (at the time) and was a Frankenstein-style mix of exponential smoothing and an RNN, dubbed ES-RNN (*Further reading* has links to a newer and faster implementation of the model that uses GPU acceleration). This led to Makridakis et al. putting forward an argument that *"hybrid approaches and combinations of methods are the way forward."* The creators of the N-BEATS model aspired to challenge this conclusion by designing a pure DL architecture for time series forecasting. They succeeded in this when they created a model that beat all other methods in the M4 competition (although they didn't publish it in time to participate in the competition). It is a very unique architecture, taking a lot of inspiration from signal processing. Let's take a deeper look and understand the architecture.



Reference check:

The research paper by Makridakis et al. and the blog post by Slawek Smyl are cited in the *References* section as 1 and 2, respectively.

We need to establish a bit of context and terminology before moving ahead with the explanation. The core problem that they are solving is univariate forecasting, which means it is similar to classical methods such as exponential smoothing and ARIMA in the sense that it takes only the history of the time series to generate a forecast. There is no provision to include other covariates in the model. The model is shown a window from the history and is asked to predict the next few timesteps. The window of history is referred to as the **lookback period** and the future timesteps are the **forecast period**.

The architecture of N-BEATS

The N-BEATS architecture was different from the existing architectures (at the time) in a few aspects:

- Instead of the common encoder-decoder (or sequence-to-sequence) formulation, N-BEATS formulates the problem as a multivariate regression problem.

- Most of the other architectures at the time were relatively shallow (~5 LSTM layers). However, N-BEATS used the residual principle to stack many basic blocks (we will explain this shortly) and the paper has shown that we can stack up to 150 layers and still facilitate efficient learning.
- The model lets us extend it to human-interpretable output, still in a principled way.

Let's look at the architecture and go deeper:

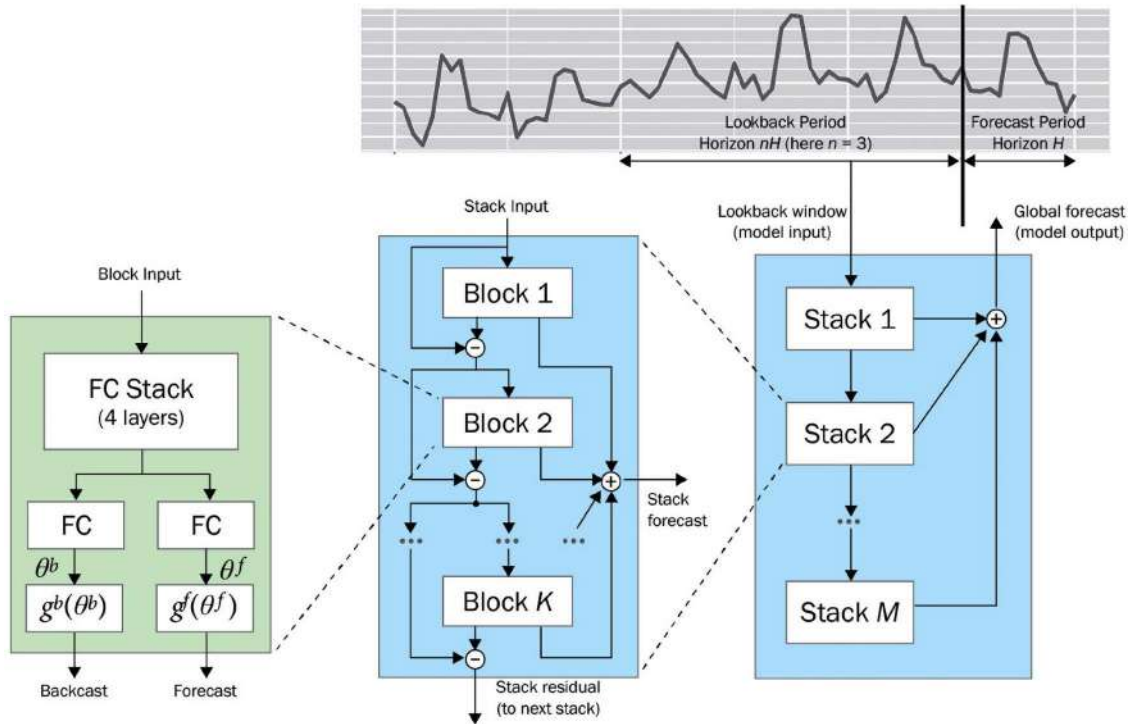


Figure 16.1: N-BEATS architecture

We can see three columns of *rectangular blocks*, each one an exploded view of another. Let's start at the leftmost (which is the most granular view) and then go up step by step, building up to the architecture. At the top, there is a representative time series, which has a lookback window and a forecast period.

Blocks

The fundamental learning unit in N-BEATS is a block. Each block, l , takes in an input, (x_l) , of the size of the lookback period and generates two outputs: a forecast, $(\hat{y}_l)$, and a backcast, $(\hat{x}_l)$. The backcast is the block's own best prediction of the lookback period. It is synonymous with fitted values in the classical sense; they tell us how the stack would have predicted the lookback window using the function it has learned. The block input is first processed by a stack of four standard, fully connected layers (complete with a bias term and non-linear activation), transforming the input into a hidden representation, h_l . Now, this hidden representation is transformed by two separate linear layers (no bias or non-linear activation) to something the paper calls expansion coefficients for the backcast and forecast, θ_l^b and θ_l^f , respectively.

The last part of the block takes these expansion coefficients and maps them to the output using a set of basis layers (g_l^b and g_l^f). We will talk about the basis layers in a bit more detail later, but for now, just understand that they take the expansion coefficients and transform them into the desired outputs ($\hat{y}_l$ and $\hat{x}_l$).

Stacks

Now, let's move one layer up the abstraction to the middle column of *Figure 16.1*. It shows how different blocks are arranged in a stack, s . All the blocks in a stack share the same kind of basis layers and therefore are grouped as a stack. As we saw earlier, each block has two outputs, $\hat{y}_l$ and $\hat{x}_l$. The blocks are arranged in a residual manner, each block processing and cleaning the time series step by step. The input to a block, l , is $x_l = x_{l-1} - \hat{x}_{l-1}$. At each step, the backcast generated by the block is subtracted from the input to that block before it's passed on to the next layer. All the forecast outputs of all the blocks in a stack are added up to make the *stack forecast*:

$$\hat{y}^s = \sum_l \hat{y}_l^s$$

The residual backcast from the last block in a stack is the *stack residual* (x^s).

The overall architecture

With that, we can move to the rightmost column of *Figure 16.1*, which shows the top-level view of the architecture. We saw that each stack has two outputs—a stack forecast ($\hat{y}^s$) and a stack residual (x^s). There can be N stacks that make up the N-BEATS model. Each stack is chained together so that for any stack (s), the stack residual out of the previous stack (x^{s-1}) is the input and the stack generates two outputs: the stack forecast ($\hat{y}^s$) and the stack residual (x^s). Finally, the N-BEATS forecast, $\hat{y}$, is the additive sum of all the stack forecasts:

$$\hat{y} = \sum_{s=1}^N \hat{y}^s$$

Now that we have understood what the model is doing, we need to come back to one point that we left for later—**basis functions**.



Disclaimer:

The explanation here is to mostly aid intuition, so we might be hand-waving over a few mathematical concepts. For a more rigorous treatment of the subject, you should refer to mathematical books/articles that cover the topic. For example, *Functions as Vector Spaces* from the *Further reading* section and *Function Spaces* (<https://cns.gatech.edu/~predrag/courses/PHYS-6124-12/StGoChap2.pdf>).

Basis functions and interpretability

To understand what basis functions are, we need to understand a concept from linear algebra. We talked about vector spaces in *Chapter 11, Introduction to Deep Learning*, and gave you a geometric interpretation of vectors and vector spaces. We talked about how a vector is a point in the n -dimensional vector space. We had that discussion regarding regular Euclidean space (R^n), which is intended to represent physical space. Euclidean spaces are defined with an origin and an orthonormal basis. An orthonormal basis is a unit vector (magnitude=1) and they are orthogonal (in simple intuition, at 90 degrees) to each other. Therefore, a vector, $A = \begin{bmatrix} 5 \\ 2 \end{bmatrix}$, can be written as $5\hat{i} + 2\hat{j}$, where $\hat{i}$ and $\hat{j}$ are the orthonormal basis. You may remember this from high school.

Now, there is a branch of mathematics that views a function as a point in a vector space (at which point, we call it a functional space). This comes from the fact that all the mathematical conditions that need to be satisfied for a vector space (things such as additivity, associativity, and so on) are valid if we consider functions instead of points. To better drive that intuition, let's consider a function, $f(x) = 2x + 4x^2$. We can consider this function as a vector in the function space with basis x and x^2 . Now, the coefficients, 2 and 4, can be changed to give us different functions; this can be any real number from $-\infty$ to $+\infty$. This space of all functions that can have a basis of x and x^2 is the functional space, and every function in the function space can be defined as a linear combination of the basis functions. We can have the basis of any arbitrary function, which gives us a lot of flexibility. From a machine learning perspective, searching for the best function in this functional space automatically means that we are restricting the function search so that we have some properties defined by the basis functions.

Coming back to N-BEATS, we talked about the expansion coefficients, θ^b and θ^f , which are mapped to the output using a set of basis layers (g_l^b and g_l^f). A basis layer can also be thought of as a basis function because we know that a layer is nothing but a function that maps its inputs to its outputs. Therefore, by learning the expansion coefficients, we are essentially searching for the best function that can represent the output but is constrained by the basis functions we choose.

There are two modes in which N-BEATS operates: *generic* and *interpretable*. The N-BEATS paper shows that under both modes, N-BEATS managed to beat the best in the M4 competition. Generic mode is where we do not have any basis function constraining the function search. We can also think of this as setting the basis function to be the identity function. So, in this mode, we are leaving the function completely learned by the model through a linear projection of the basis coefficients. This mode lacks human interpretability because we don't have any idea how the different functions are learned and what each stack signifies.

But if we have fixed basis functions that constrain the function space, we can bring in more interpretability. For instance, if we have a basis function that constrains the output to represent the trends for all the blocks in a stack, we can say that the forecast output of that stack represents the trend component. Similarly, if we have another basis function that constrains the output to represent the seasonality for all the blocks in a stack, we can say that the forecast output of the stack represents seasonality.

This is exactly what the paper has proposed as well. They have defined specific basis functions that capture trend and seasonality, and including such blocks makes the final forecast more interpretable by giving us a decomposition. The trend basis function is a polynomial of a small degree, p . So, as long as p is low, such as 1, 2, or 3, it forces the forecast output to mimic the trend component. For the seasonality basis function, the authors chose a Fourier basis (similar to the one we saw in *Chapter 6, Feature Engineering for Time Series Forecasting*). This forces the forecast output to be functions of these sinusoidal basis functions that mimic seasonality. In other words, the model learns to combine these sinusoidal waves with different coefficients to reconstruct the seasonality pattern as best as possible.

For a deeper understanding of these basis functions and how they are structured, I have linked to a *Kaggle notebook* in the *Further reading* section that provides a clear explanation of the trend and seasonality basis functions. The associated notebook also has an additional section that visualizes the first few basis functions of seasonality. Along with the original paper, these additional readings will help you solidify your understanding.

N-BEATS wasn't designed to be a global model, but it does well in the global setting. The M4 competition was a collection of unrelated time series and the N-BEATS model was trained so that the model was exposed to all those series and learned a common function to forecast each time series in the dataset. This, along with ensembling multiple N-BEATS models with different lookback windows, was the success formula for the M4 competition.



Reference check:

The research paper by Boris Oreshkin et al. (N-BEATS) is cited in the *References* section as 3.

Forecasting with N-BEATS

N-BEATS, along with many other specialized architectures we will explore in this chapter, are implemented in NIXTLA's NeuralForecast packages. First, let's look at the initialization parameters of the implementation.

The NBEATS class in NeuralForecast has lots of parameters, but here are the most important ones:

- **stack_types:** This defines the number of stacks that we need to have in the N-BEATS model. This should be a list of strings (*generic*, *trend*, or *seasonality*) denoting the number and type of stacks. Examples include ["trend", "seasonality"], ["trend", "seasonality", "generic"], and ["generic", "generic", "generic"]. However, if the entire network is generic, we can just have a single generic stack with more blocks as well.

- `n_blocks`: This is a list of integers signifying the number of blocks in each stack that we have defined. If we had defined `stack_types` as `["trend", "seasonality"]`, and we want three blocks each, we can set `n_blocks` to `[3,3]`.
- `input_size`: This is an integer which contains the autoregressive units (lags) to be tested.
- `shared_weights`: This is a list of Booleans signifying whether the weights generating the expansion coefficients are shared with other blocks in a stack. It is recommended to share the weights in the interpretable stacks and not share them in the identity stacks.

There are several other parameters, but these are not as important. A full list of parameters and their descriptions can be found at <https://nixtlaverse.nixtla.io/neuralforecast/models.nbeats.html>.

Since the strength of the model is in forecasting slightly longer durations, we can do a single-shot 48-step horizon simply by setting the forecast horizon parameter `h = 48`.



Notebook alert:

The complete code for training N-BEATS can be found in the `01-NBEATS_NeuralForecast.ipynb` notebook in the `Chapter16` folder.

Interpreting N-BEATS forecasting

N-BEATS, if we are running it in the interpretable model, also gives us more interpretability by separating the forecast into trend and seasonality. To get the interpretable output, we can call the `decompose` function. We must ensure that, in our initial parameters, we include the stack type for the trend and seasonal components: `stack_types = ['trend', 'seasonality']`.

```
model_interpretable = model_untuned.models[0]
dataset, *_ = TimeSeriesDataset.from_df(df = training_df, id_col='LCLid', time_col='timestamp', target_col='energy_consumption')
y_hat = model_interpretable.decompose(dataset=dataset)
```

This will return us an array from which trend and seasonality can be accessed, like `y_hat = [0,1]`. The order of trend or seasonality depends on how you include it in `stack_types`, though the default is `['seasonality', 'trend']`, meaning seasonality is `y_hat = [0,1]` and trend is `y_hat = [0,1]`.

Let's see how one of the household predictions decomposed:

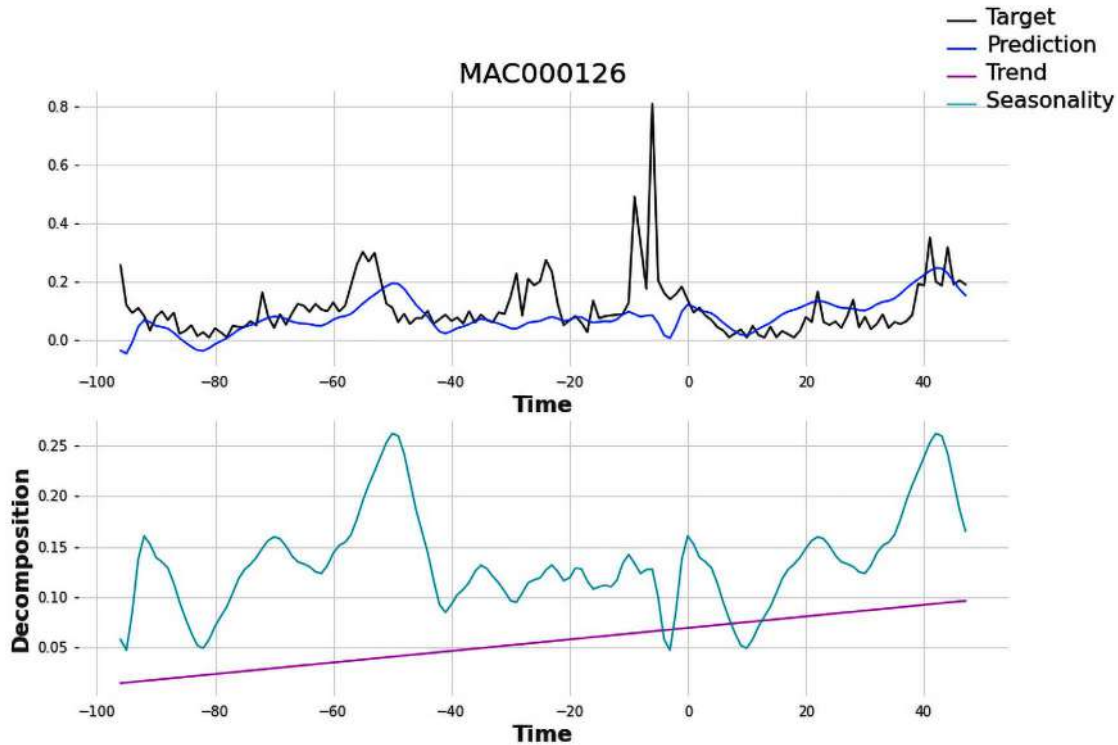


Figure 16.2: Decomposed predictions from N-BEATS (interpretable)

With all its success, N-BEATS was still a univariate model. It was not able to take in any external information, apart from its history. This was fine for the M4 competition, where all the time series in question were also univariate. However, many real-world time series problems come with additional explanatory variables (or exogenous variables). Let's look at a slight modification that was made to N-BEATS that enabled exogenous variables.

Neural Basis Expansion Analysis for Interpretable Time Series Forecasting with Exogenous Variables (N-BEATSx)

Olivares et al. proposed an extension of the N-BEATS model by making it compatible with exogenous variables. The overall structure is the same (with blocks, stacks, and residual connections) as N-BEATS (Figure 16.1), so we will only be focusing on the key differences and additions that the N-BEATSx model puts forward.



Reference check:

The research paper by Olivares et al. (N-BEATSx) is cited in the *References* section as 4.

Handling exogenous variables

In N-BEATS, the input to a block was the lookback window, y^b . But here, the input to a block is both the lookback window, y^b , and the array of exogenous variables, x . These exogenous variables can be of two types: time-varying and static. The static variables are encoded using a static feature encoder. This is nothing but a single-layer FC that encodes the static information into a dimension specified by the user. Now, the encoded static information, the time-varying exogenous variables, and the lookback window are concatenated to form the input for a block so that the hidden state representation, h_l , of block l is not $FC(y^b)$ like in N-BEATS, but $FC([y^b; x])$, where $[:]$ represents concatenation. This way, the exogenous information is part of the input to every block as it is concatenated with the residual at each step.

Exogenous blocks

In addition to this, the paper also proposes a new kind of block—an *exogenous block*. The exogenous block takes in the concatenated lookback window and exogenous variables (just like any other block) as input and produces a backcast and forecast:

$$\hat{y}_l^{exog} = \sum_{i=0}^{N_x} x_i^l \theta_l^{exog}$$

Here, N_x is the number of exogenous features.

Here, we can see that the exogenous forecast is the linear combination of the exogenous variables and that the weights for this linear combination are learned by the expansion coefficients, θ^{exog} . The paper refers to this configuration as the interpretable exogenous block because, by using the expansion weights, we can define the importance of each exogenous variable and even figure out the exact part of the forecast, which is because of a particular exogenous variable.

N-BEATSx also has a generic version (which is not interpretable) of the exogenous block. In this block, the exogenous variables are passed through an encoder that learns a context vector, C_l , and the forecast is generated using the following formula:

$$\hat{y}_l^{exog} = \sum_{i=0}^{N_x} C_l^i \theta_l^{exog}$$

They proposed two encoders: a **Temporal Convolutional Network (TCN)** and **WaveNet** (a network similar to the TCN, but with dilation to expand the receptive field). The *Further reading* section contains resources if you wish to learn more about WaveNet, an architecture that originated in the sound domain.



N-BEATSx is also implemented in NIXTLA `neuralforecast`, however, at the time of writing, it cannot yet handle categorical data. Thus, we will need to encode the categorical features into numerical representations (like we did in *Chapter 10, Global Forecasting Models*) before using `neuralforecast`.

The research paper also showed that N-BEATSx outperformed N-BEATS, ES-RNN, and other benchmarks on electricity price forecasting considerably.

Continuing with the legacy of N-BEATS, we will now talk about another modification to the architecture that makes it suitable for long-term forecasting.

Neural Hierarchical Interpolation for Time Series Forecasting (N-HiTS)

Although there has been a good amount of work from DL to tackle time series forecasting, very little focus has been on long-horizon forecasting. Despite recent progress, long-horizon forecasting remains a challenge for two reasons:

- The expressiveness required to truly capture the variation
- The computational complexity

Attention-based methods (Transformers) and N-BEATS-like methods scale quadratically in memory and the computational cost concerning the forecasting horizon.

The authors claim that N-HiTS drastically cuts long-forecasting compute costs while simultaneously showing 25% accuracy improvements compared to existing Transformer-based architectures across a large array of multi-variate forecasting datasets.



Reference check:

The research paper by Challu et al. on N-HiTS is cited in the *References* section as 5.

The Architecture of N-HiTS

N-HiTS can be considered as an alteration to N-BEATS because the two share a large part of their architectures. *Figure 16.1*, which shows the N-BEATS architecture, is still valid for N-HiTS. N-HiTS also has stacks of blocks arranged in a residual manner; it differs only in the kind of blocks it uses. For instance, there is no provision for interpretable blocks. All the blocks in N-HiTS are generic. While N-BEATS tries to decompose the signal into different patterns (trend, seasonality, and so on), N-HiTS tries to decompose the signal into multiple frequencies and forecast them separately.

To enable this, a few key improvements have been proposed:

- Multi-rate data sampling
- Hierarchical interpolation
- Synchronizing the rate of input sampling with a scale of output interpolation across the blocks

Multi-rate data sampling

N-HiTS incorporates sub-sampling layers before the fully connected blocks so that the resolution of the input to each block is different. This is similar to smoothing the signal with different resolutions so that each block is looking at a pattern that occurs at different resolutions—for instance, if one block looks at the input every day, another block looks at the output every week, and so on. This way, when arranged with different blocks looking at different resolutions, the model will be able to predict patterns that occur in those resolutions. This significantly reduces the memory footprint and the computation required as well, because instead of looking at all H steps of the lookback window, we are looking at smaller series (such as $H/2$, $H/4$, and so on).

N-HiTS accomplishes this using a Max Pooling or Average Pooling layer of kernel size k_l on the look-back window. A pooling operation is similar to a convolution operation, but the function that is used is non-learnable. In *Chapter 12, Building Blocks of Deep Learning for Time Series*, we learned about convolutions, kernels, stride, and so on. While a convolution uses weights that are learned from data while training, a pooling operation uses a non-learnable and fixed function to aggregate the data in the receptive field of a kernel. Common examples of these functions are the maximum, average, sum, and so on. N-HiTS uses `MaxPool1d` or `AvgPool1d` (in PyTorch terminology) with different kernel sizes for different blocks. Each pooling operation also has a stride equal to the kernel, resulting in non-overlapping windows over which we do the aggregation operation. To refresh our memory, let's see what max pooling with `kernel=2` and `stride=2` looks like:

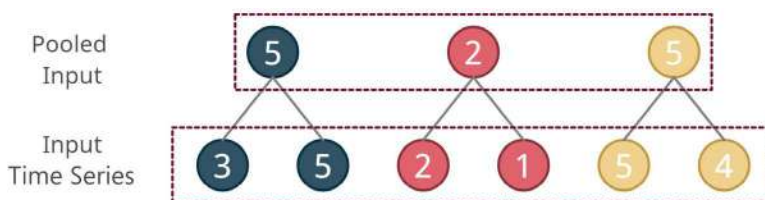


Figure 16.3: Max pooling on one dimension—kernel = 2, stride = 2

Therefore, a larger kernel size will tend to cut more high-frequency (or small-timescale) components from the input. This way, the block is forced to focus on larger-scale patterns. The paper calls this **multi-rate signal sampling**.

Hierarchical interpolation

In a standard multi-step forecasting setting, the model must forecast H timesteps. As H becomes larger, the compute requirements increase and lead to an explosion of expressive power the model needs to have.

Training a model with such a large expressive power, without overfitting, is a challenge in itself. To combat these issues, N-HiTS proposes a technique called **temporal interpolation** (not the simple interpolation between two known points in time, but something specific to the architecture).

The pooled input (which we saw in the previous section) goes into the block along with the usual mechanism to generate expansion coefficients and finally gets converted into forecast output. But here, instead of setting the dimension of the expansion coefficients as H , N-HiTS sets them as $r_l \times H$, where r_l is the **expressiveness ratio**. This parameter essentially reduces the forecast output dimension and thus controls the issues we discussed in the previous paragraph. To recover the original sampling rate and predict all the H points in the forecast horizon, we can use an interpolation function. There are many options for the interpolation functions—linear, nearest neighbor, cubic, and so on. All these options can easily be implemented in PyTorch using the `interpolate` function.

Synchronizing the input sampling and output interpolation

In addition to proposing the input sampling through pooling and output interpolation, N-HiTS also proposes arranging them in different blocks in a particular way. The authors argue that hierarchical interpolation can only happen the right way if the expressiveness ratios are distributed across blocks in a manner that is synchronized with the multi-rate sampling. Blocks closer to the input should have a smaller expressiveness ratio, r_l , and larger kernel sizes, k_l . This means that the blocks closer to the input will generate larger resolution patterns (because of aggressive interpolation) while being forced to look at aggressively subsampled input signals. The paper proposes exponentially increasing expressiveness ratios as we move from the initial block to the last block to handle a wide range of frequency bands. The official N-HiTS implementation uses the following formula to set the expressiveness ratios and pooling kernels:

```
pooling_sizes = np.exp2(
    np.round(np.linspace(0.49, np.log2(prediction_length / 2), n_stacks))
)
pooling_sizes = [int(x) for x in pooling_sizes[::-1]]
downsample_frequencies = [
    min(prediction_length, int(np.power(x, 1.5))) for x in pooling_sizes
]
```

We can also provide explicit `pooling_sizes` and `downsampling_frequencies` to reflect known cycles of the time series (weekly seasonality, monthly seasonality, and so on). The core principle of N-BEATS (one block removing the effect it captures from the signal and passing it on to the next block) is used here as well so that, at each level, the patterns or frequencies that a block captures are removed from the input signal before being passed on to the next block. In the end, the final forecast is the sum of all such individual block forecasts.

Forecasting with N-HiTS

N-HiTS is implemented in NIXTLA forecasting. We can use the same framework we were working with for NBEATS and extend it to train *N-HiTS* on our data. What's even better is that the implementation supports exogenous variables, the same way N-BEATSx handles exogenous variables (although without the exogenous block). First, let's look at the initialization parameters of the implementation.

The NHITS class in `neuralforecast` has the following parameters:

- `n_blocks`: This is a list of integers signifying the number of blocks to be used in each stack. For instance, `[1,1,1]` means there will be three stacks with one block each.
- `n_pool_kernel_size`: This is a list of integers that defines the pooling size (k_i) for each stack. This is an optional parameter, and if provided, we can have more control over how the pooling happens in the different stacks. Using an ordering of higher to lower improves results.
- `pooling_mode`: This defines the kind of pooling to be used. It should be either `'MaxPool1d'` or `'AvgPool1d'`.
- `n_freq_downsample`: This is a list of integers that defines the expressiveness ratios (r_i) for each stack. This is an optional parameter, and if provided, we can have more control over how the interpolation happens in the different stacks.



Notebook alert:

The complete code for training N-HiTS can be found in the `02-NHiTS_NeuralForecast.ipynb` notebook in the `Chapter16` folder.

Now, let's shift our focus and look at a few modifications of the Transformer model to make it better for time series forecasting.

Autoformer

Recently, Transformer models have shown superior performance in capturing long-term patterns than standard RNNs. One of the major factors of that is the fact that self-attention, which powers Transformers, can reduce the length that the relevant sequence information has to be held on to before it can be used for prediction. In other words, in an RNN, if the timestep 12 steps before holds important information, that information has to be stored in the RNN through 12 updates before it can be used for prediction. But with self-attention in Transformers, the model is free to create a shortcut between lag 12 and the current step directly because of the lack of recurrence in the structure.

But the same self-attention is also the reason why we can't scale vanilla Transformers to long sequences. In the previous section, we discussed how long-term forecasting is a challenge because of two reasons: the expressiveness required to truly capture the variation and computational complexity. Self-attention, with its quadratic computational complexity, contributes to the second reason.

The research community has recognized this challenge and has put a lot of effort into devising efficient transformers through many techniques, such as downsampling, low-rank approximations, sparse attention, and so on. For a detailed account of such techniques, refer to the link for *Efficient Transformers: A Survey* in the *Further reading* section.

Autoformer is another model that is designed for long-term forecasting. Autoformer invents a new kind of attention and couples it with aspects from time series decomposition. Let's take a look at what makes Autoformer special.

The architecture of the Autoformer model

The Autoformer model is a modification of Transformers. The following are its major contributions:

- **Uniform Input Representation:** A methodical way to include the history of the series along with other information, which will help in capturing long-term signals such as the week, month, holidays, and so on
- **Generative-style decoder:** Used to generate the long-term horizon in a single forward pass instead of via dynamic recurrence
- **AutoCorrelation mechanism:** An alternative to standard dot product attention, which takes into account sub-series similarity rather than point-to-point similarity
- **Decomposition architecture:** A specially designed architecture that separates seasonality, trend, and residual in a time series while modeling it



Reference check:

The research paper by Wu et al. on Autoformer is cited in the *References* section as 9.

Uniform Input Representation

RNNs capture time series patterns with their recurrent structure, so they only need the sequence; they don't need information about the timestamp to extract the patterns. However, the self-attention in Transformers is done via point-wise operations that are performed in sets (the order doesn't matter in a set). Typically, we include positional encodings to capture the order of the sequence. Instead of using positional encodings, we can use richer information, such as hierarchical timestamp information (such as weeks, months, years, and so on). This is what the authors proposed through **Uniform Input Representation**.

Uniform Input Representation uses three types of embeddings to capture the history of the time series, the sequence of values in the time series, and the global timestamp information. The sequence of values in the time series is captured by the standard positional embedding of the `d_model` dimension.

Uniform Input Representation uses a one-dimensional convolutional layer with `kernel=3` and `stride=1` to project the history (which is scalar or one-dimensional) into an embedding of `d_model` dimensions. This is referred to as **value embedding**.

The global timestamp information is embedded by a learnable embedding of `d_model` dimensions with limited vocabulary in a mechanism that is identical to embedding categorical variables into fixed-size vectors (*Chapter 15, Strategies for Global Deep Learning Forecasting Models*). This is referred to as **temporal embedding**.

Now that we have three embeddings of the same dimension, `d_model`, all we need to do is add them together to get the Uniform Input Representation.

Generative-style decoder

The standard way of inferencing a Transformer model is by decoding one token at a time. This autoregressive process is time-consuming and repeats a lot of calculations for each step. To alleviate this problem, the Autoformer model adopts a more generative fashion where the entire forecasting horizon is generated in a single forward pass.

In NLP, it is a popular technique to use a special token (START) to start the dynamic decoding process. Instead of choosing a special token for this purpose, the Autoformer model chooses a sample from the input sequence, such as an earlier slice before the output window. For instance, if we say the input window is t_1 to t_w , we will sample a sequence of length C from the input, t_{w-c} to t_w , and include this sequence as the starting sequence of the decoder. To make the model predict the entire horizon in a single forward pass, we can extend the decoder input tensor so that its length is $C + H$, where H is the length of the prediction horizon. The initial C tokens are filled with the sample sequence from the input, and the rest are filled as zeros—that is, $X_{de}^t = \text{Concat}(X_{token}^t, X_0^t)$. This is just the target. Although X_0^t has zeros filled in for the prediction horizon, this is just for the target. The other information, such as the global timestamps, is included in X_0^t . Sufficient masking of the attention matrix is also employed so that each position does not attend to future positions, thus maintaining the autoregressive nature of the prediction.

Now, let's look at the time series decomposition architecture.

Decomposition architecture

We saw this idea of decomposition back in *Chapter 3, Analyzing and Visualizing Time Series Data*, and even in this chapter (N-BEATS). Autoformer successfully renovated the Transformer architecture into a deep-decomposition architecture:

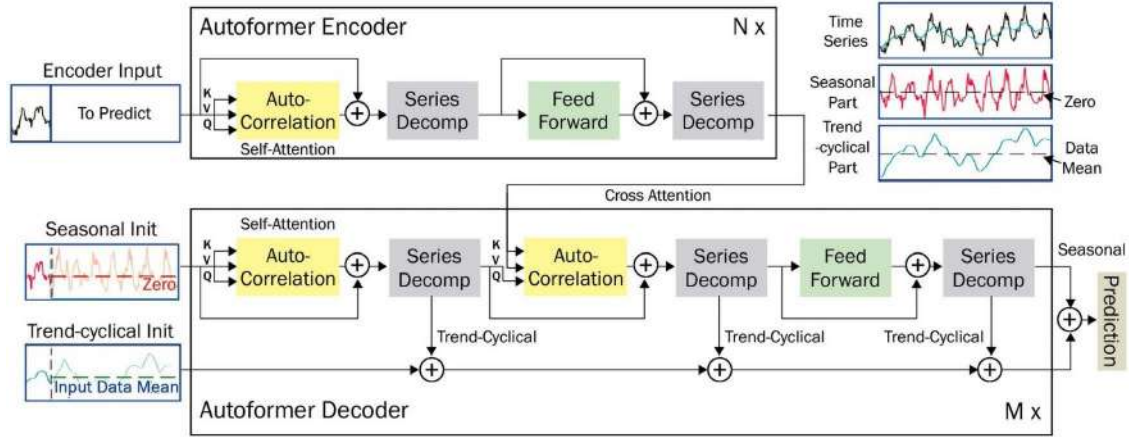


Figure 16.4: Autoformer architecture

It is easier to understand the overall architecture first and then dive deeper into the details. In *Figure 16.4*, there are boxes labeled **Auto-Correlation** and **Series Decomp**. For now, just know that auto-correlation is a type of attention and that series decomposition is a particular block that decomposes the signal into trend-cyclical and seasonal components.

Encoder

With the level of abstraction discussed in the preceding section, let's understand what is happening in the encoder:

1. The uniform representation of the time series, x_{en} , is the input to the encoder. The input is passed through an **Auto-Correlation** block (for self-attention) whose output is x_{ac} .
2. The uniform representation, x_{en} , is added back to x_{ac} as a residual connection, $x_{ac} = x_{ac} + x_{en}$.
3. Now, x_{ac} is passed through a **Series Decomp** block, which decomposes the signal into a trend-cyclical component (x_T) and a seasonal component, x_{seas} .
4. We discard x_T and pass x_{seas} to a Feed Forward network, which gives x_{FF} as an output.
5. x_{seas} is again added to x_{FF} as a residual connection, $x_{seas} = x_{FF} + x_{seas}$.
6. Finally, this x_{seas} is passed through another **Series Decomp** layer, which again decomposes the signal into the trend, $x_{\bar{T}}$, and a seasonal component, $x_{\bar{seas}}$.
7. We discard $x_{\bar{T}}$, and pass on $x_{\bar{seas}}$ as the final output from one block of the encoder.
8. There may be N blocks of encoders stacked together, one taking in the output of the previous encoder as input.

Now, let's shift our attention to the decoder block.

Decoder

The Autoformer model uses a START token-like mechanism by including a sampled window from the input sequence. But instead of just taking the sequence, Autoformer does a bit of special processing on it. Autoformer uses the bulk of its learning power to learn seasonality. The output of the transformer is also just the seasonality. Therefore, instead of including the complete window from the input sequence, Autoformer decomposes the signal and only includes the seasonal component in the START token. Let's look at this process step by step:

1. If the input (the context window) is x , we decompose it with the **Series Decomp** block into x_T^{init} and x_{seas}^{init} .
2. Now, we sample C timesteps from the end of x_{seas}^{init} and append H zeros, where H is the forecast horizon, and construct x_{ds} .
3. This x_{ds} is then used to create a uniform representation, x_{dec} .
4. Meanwhile, we sample C timesteps from the end of x_T^{init} and append H timesteps with the series mean ($mean(x)$), where H is the forecast horizon, and construct x_{dt} .

This x_{dec} is then used as the input for the decoder. This is what happens in the decoder:

1. The input, x_{dec} , is first passed through an Auto-Correlation (for self-attention) block whose output is x_{dac} .
2. The uniform representation, x_{dec} , is added back to x_{dac} as a residual connection, $x_{dac} = x_{dac} + x_{dec}$.
3. Now, x_{dac} is passed through a **Series Decomp** block that decomposes the signal into a trend-cyclical component (x_{dT1}) and a seasonal component, x_{dseas} .
4. In the decoder, we do not discard the trend component; instead, we save it. This is because we will be adding all the trend components with the trend in it (x_{dt}) to come up with the overall trend part (T).
5. The seasonal output from the **Series Decomp** block (x_{dseas}), along with the output from the encoder (x_{seas}), is then passed into another **Auto-Correlation** block where cross-attention between the decoder sequence and encoder sequence is calculated. Let the output of this block be x_{cross} .
6. Now, x_{dseas} is added back to x_{cross} as a residual connection, $x_{cross} = x_{cross} + x_{dseas}$.
7. x_{cross} is again passed through a **Series Decomp** block, which splits x_{cross} into two components— x_{dT2} and x_{dseas2} .
8. x_{dseas} is then transformed using a **Feed Forward** network into x_{diff} and x_{dseas} is added to it in a residual connection, $x_{diff} = x_{diff} + x_{dseas}$.
9. Finally, x_{diff} is passed through yet another **Series Decomp** block, which decomposes it into two components— x_{dT3} and x_{dseas3} . x_{dseas3} is the final output of the decoder, which captures seasonality.
10. Another output is the residual trend, X_{trend} , which is a projection of the summation of all the trend components extracted in the decoder's **Series Decomp** blocks. The projection layer is a **Conv1d** layer, which projects the extracted trend to the desired output dimension: $X_{trend} = Conv1d(x_{dT1} + x_{dT2} + x_{dT3})$.
11. M such decoder layers are stacked on top of each other, each one feeding its output as the input to the next one.

12. The residual trend, $X_{\widetilde{trend}}$, of each decoder layer gets added to the trend init, x_{dt} , to model the overall trend component (T).
13. The $x_{dseason}$ of the final decoder layer is considered to be the overall seasonality component and is projected to the desired output dimension (S) using a linear layer.
14. Finally, the prediction or the forecast $X_{out} = T + S$.

The whole architecture is cleverly designed so that the relatively stable and easy-to-predict part of the time series (the trend-cyclical) is removed and the difficult-to-capture seasonality can be modeled well.

Now, how does the **Series Decomp** block decompose the series? The mechanism may be familiar to you already: AvgPool1d with some padding so that it maintains the same size as the input. This acts like a moving average over the specified kernel width.

We have been talking about the **Auto-Correlation** block throughout this explanation. Now, let's understand the ingenuity of the **Auto-Correlation** block.

Auto-correlation mechanism

Autoformer uses an auto-correlation mechanism in place of standard scaled dot product attention. This discovers sub-series similarity based on periodicity and uses this similarity to aggregate similar sub-series. This clever mechanism breaks the information bottleneck by expanding the point-wise operation of the scaled dot product attention to a sub-series level operation. The initial part of the overall mechanism is similar to the standard attention procedure, where we project the query, key, and values into the same dimension using weight matrices. The key difference is the attention weight calculation and how they are used to calculate the values. This mechanism achieves this by using two salient sub-mechanisms: discovering period-based dependencies and time delay aggregation.

Period-based dependencies

Autoformer uses autocorrelation as the key measure of similarity. Auto-correlation, as we know, represents the similarity between a given time series, X_t , and its lagged series. For instance, $R_{xx}(\tau)$ is the autocorrelation between the time series X_t and $X_{t-\tau}$. Autoformer considers this autocorrelation as the unnormalized confidence of the particular lag. Therefore, from the list of all τ , we choose k most possible lags and use *softmax* to convert these unnormalized confidences into probabilities. We use these probabilities as weights to aggregate relevant sub-series (we will talk about this in the next section).

The autocorrelation calculation is not the most efficient operation and Autoformer suggests an alternative to make the calculation faster. Based on the **Wiener-Khinchin theorem** in **Stochastic Processes** (this is outside the scope of the book, but for those who are interested, I have included a link in the *Further reading* section), autocorrelation can also be calculated using **Fast Fourier Transform (FFT)**. The process can be seen as follows:

$$S_{xx}(\tau) = \mathcal{F}(X_t)\mathcal{F}^*(X_t)$$

Here, $\mathcal{F}$ denotes the FFT and $\mathcal{F}^*$ denotes the conjugate operation (the conjugate of a complex number is the number with the same real part and an imaginary part, which is equal in magnitude but with the sign reversed. The mathematics around this is outside the scope of this book).

This can easily be written in PyTorch as follows:

```
# calculating the FFT of Query and Key
q_fft = torch.fft.rfft(queries.permute(0, 2, 3, 1).contiguous(), dim=-1)
k_fft = torch.fft.rfft(keys.permute(0, 2, 3, 1).contiguous(), dim=-1)
# Multiplying the FFT of Query with Conjugate FFT of Key
res = q_fft * torch.conj(k_fft)
```

Now, $S_{xx}(\tau)$ is in the spectral domain. To bring it back to the real domain, we need to do an inverse FFT:

$$R_{xx}(\tau) = \mathcal{F}^{-1}(S_{xx}(\tau))$$

Here, $\mathcal{F}^{-1}$ denotes the inverse FFT. In PyTorch, we can do this easily:

```
corr = torch.fft.irfft(res, dim=-1)
```

When the query and key are the same, this calculates self-attention; when they are different, they calculate cross-attention.

Now, all we need to do is take the top-k values from corr and use them to aggregate the sub-series.

Time delay aggregation

We have identified the major lags that are auto-correlated using the FFT and inverse-FFT. For a more concrete example, the dataset we have been working on (*London Smart Meter Dataset*) has a half-hourly frequency and has strong daily and weekly seasonality. Therefore, the auto-correlation identification may have picked out 48 and 48*7 as the two most important lags. In the standard attention mechanism, we use the calculated probability as weights to aggregate the value. Autoformer also does something similar, but instead of applying the weights to points, it applies them to sub-series.

Autoformer does this by shifting the time series by the lag, τ , and then using the lag's weight to aggregate them:

$$\text{Auto - Correlation}(Q, K, V) = \sum_{i=1}^k \text{Roll}(V, \tau_i) \hat{R}_{Q,K}(\tau_i)$$

Here, $\hat{R}_{Q,K}(\tau_i)$ is the *softmax*-ed probabilities on the *top-k* autocorrelations.

In our example, we can think of this as shifting the series by 48 timesteps so that the previous day's timesteps are aligned with the current day and then using the weight of the 48 lag to scale it. Then, we can move on to the 48*7 lag, align the previous week's timesteps with the current week, and then use the weight of the 48*7 lag to scale it. So, in the end, we will get a weighted mixture of the seasonality patterns that we can observe daily and weekly. Since these weights are learned by the model, we can hypothesize that different blocks learn to focus on different seasonalities, and thus as a whole, the blocks learn the overall pattern in the time series.

Forecasting with Autoformer

Autoformer is implemented in NIXTLA forecasting. We can use the same framework we were working with for NBEATS and extend it to train Autoformer on our data. First, let's look at the initialization parameters of the implementation.



We have to keep in mind that the Autoformer model does not support exogenous variables. The only additional information it officially supports is global timestamp information such as the week, month, and so on, along with holiday information. We can technically extend this to any categorical feature (static or dynamic), but no real-valued information is currently supported.

Let's look at the initialization parameters of the implementation.

The Autoformer class has the following major parameters:

- `distil`: This is a Boolean flag for turning the attention distillation off and on.
- `encoder_layers`: This is an integer representing the number of encoder layers.
- `decoder_layers`: This is an integer representing the number of decoder layers.
- `n_head`: This is an integer representing the number of attention heads.
- `conv_hidden_size`: This is an integer parameter that specifies the channels of the convolutional encoder, which can be thought of similarly to controlling the number of kernels or filters in the convolutional layers. The number of channels effectively determines how many different filters are applied to the input data, each capturing different features.
- `activation`: This is a string that takes in one of two values—`relu` or `gelu`. This is the activation to be used in the encoder and decoder layers.
- `factor`: This is an int value that helps us control the top-k values that will be selected in the Auto Correlation mechanism we discussed. `top_k = int(self.factor * math.log(length))` is the exact formula used, but we can treat k as a factor to control the top K selection.
- `dropout`: This is a float between 0 and 1, which determines the strength of the dropout in the network.



Notebook alert:

The complete code for training the Autoformer model can be found in the `03-Autoformer_NeuralForecast.ipynb` notebook in the Chapter16 folder.

Let's switch tracks and look at a family of simple linear models that were proposed to challenge Transformers in **Long-Term Time Series Forecasting (LTSF)**.

LTSF-Linear family of models

There has been a lot of debate on whether Transformers are right for forecasting problems, how popular Transformer papers haven't used strong baselines to show their superiority, how the order-agnostic attention mechanism may not be the best way to approach strongly ordered time series, and so on. The criticism was more pronounced for Long-Term Time Series Forecasting as it relies more on the extraction of strong trends and seasonalities. In 2023, Ailing Zeng et al. decided to put the Transformer models to the test and conducted a wide study using 5 multivariate datasets, pitting five Transformer models (FEDFormer, Autoformer, Informer, Pyraformer, and LogTrans) against a set of simple linear models that they proposed. Surprisingly, the simple linear models they proposed beat all the Transformer models comfortably.



Reference check:

The research papers by Ailing Zeng et al. and the different Transformer models, FEDFormer, Autoformer, Informer, Pyraformer, and LogTrans, are cited in the *References* section as 14, 16, 9, 8, 15, and 17 respectively.

There are three models in the family of LTSF models that the authors proposed:

1. Linear
2. D-Linear
3. N-Linear

These models are so simple that it's almost embarrassing that they outperformed the Transformer models. But once you get to know them a bit more, you might appreciate the simple but effective inductive biases that have been built into the model. Let's look at them one by one.

Linear

Just as the name suggests, this is a simple linear model. It takes the context window and applies a linear layer to predict the forecast horizon. It also considers different time series as separate channels and applies different linear layers to each of them. In PyTorch, all we need to have is an `nn.Linear` layer for each of the channels:

```
# Declare nn.Linear for each channel
layers = nn.ModuleList([nn.Linear(context_window, forecast_horizon) for _ in
range(n_timeseries)])

## Forward Method ##

# Now use these Layers once you get the input (Batch, Context Length, Channel)
forecast = [layers[i](input[:, :, i]) for i in range(n_timeseries)]
```

This embarrassingly simple model was able to outperform a few Transformer models like the Informer, LogTrans, and so on.

D-Linear

D-Linear took the simple linear model and injected a decomposition prior into it. We saw in *Chapter 3* how we can decompose a time series into trend, seasonality, and residual. D-Linear does exactly that and uses a moving average (the window or the kernel size is a hyperparameter) and separates the input time series, x , into trend, t (the moving average), and the rest, r (seasonality + residual). Now, it proceeds to apply separate linear layers to t and r separately, and finally add them back together for the final forecast. Let's look at a simplified PyTorch implementation:

```
# Declare nn.Linear for each channel, trend and seasonality separately
trend_layers = nn.ModuleList([nn.Linear(context_window, forecast_horizon) for _
in range(n_timeseries)])
seasonality_layers = nn.ModuleList([nn.Linear(context_window, forecast_horizon)
for _ in range(n_timeseries)])

## Forward Method ##

# Now use these layers once you get the input (Batch, Context Length, Channel)
# series_decomp is a function extracting trend using moving averages
trend, seasonality = series_decomp(input)
trend_forecast = [trend_layers[i]( trend[:, :, i]) for i in range(n_timeseries)]
seasonality_forecast = [seasonality_layers[i]( seasonality[:, :, i]) for i in
range(n_timeseries)]
forecast = [trend_forecast[i] + seasonality_forecast[i] for i in range(n_
timeseries)]
```

The decomposition prior in the model helps it perform better than a simple linear model consistently and it also outperforms all the Transformer models in the study in almost all the datasets used.

N-Linear

The authors also proposed another model, which added another very simple modification to the linear model. This modification was to handle the distributional shifts in data that are inherent in time series data. In N-Linear, we just extract the last value in the input context and subtract it from the entire series (in a sort of normalization) and then use the linear layer for prediction. Now, once the output from the linear layer is available, we add back the last value that we subtracted earlier. In PyTorch, a simple implementation would look like this:

```
# Declare nn.Linear for each channel
layers = nn.ModuleList([nn.Linear(context_window, forecast_horizon) for _ in
range(n_timeseries)])
```

```

## Forward Method ##

# Extract the last value once you get the input (Batch, Context Length,
Channel)
# Get the Last value of time series
last_value = sample_data[:, -1:, :]
# Normalize the time series
norm_ts = sample_data - last_value
# Use the Linear layers
output = [layers[i](norm_ts[:, :, i]) for i in range(n_timeseries)]
# Add back the Last value
forecast = [o + last_value[:, :, i] for i, o in enumerate(output)]

```

N-Linear models also perform quite well in comparison to the other Transformer models in the study. In most of the datasets that were part of the study, N-Linear or D-Linear came out to be the top-performing model, which is quite telling.

This paper exposed some major flaws in the way we were using Transformer models for time series forecasting, especially for multivariate time series problems. A typical input to a transformer is of the form (*Batch x Time steps x Embedding*). The most common way to forecast multivariate time series is to pass in all the time series or other features in a time step as the embedding. This results in seemingly unrelated values being embedded in a single token and mixed together in the attention mechanism (which itself isn't strongly ordered). This leads to a "muddled" representation and thereby Transformers might be struggling to wean out the real patterns from the data.



This paper had such an impact that many newer models, including PatchTST and iTransformer, which we will be seeing later in the chapter, have used these models as benchmarks and showed that they perform better than them. This underlines the need for strong and simple methods to be reserved as strong baselines so that we aren't misled by the "coolness" of any algorithm.

Now let's see how we can also use these simple linear models and get good long-term forecasts.

Forecasting with the LTSF-Linear family

NLinear and DLinear are implemented in NIXTLA forecasting with the same framework we have seen in the prior models.

Let's look at the initialization parameters of the implementation.

The `DLinear` class has similar parameters to many of the other models. Some callouts are the following major parameters:

- `moving_avg_window`: This is an integer value of the window size used for trend-seasonality decomposition. This value should be an odd integer.
- `exclude_insample_y`: This is a boolean value to skip the autoregressive features.

The `NLinear` class has no additional parameters because it is just an input window to the output window map.



Notebook alert:

The complete code for training the D-Linear model can be found in the `04-DLinear_NeuralForecast.ipynb` notebook, and for the N-Linear model, in the `05-NLinear_NeuralForecast.ipynb` notebook in the `Chapter16` folder.



Practitioner's tip:

The jury is out on this debate as Transformers are modified more and more to suit time series forecasting. There might always be datasets where using a Transformer-based model gives you better performance than some other class of models. As practitioners, we should be able to suspend disbelief and try different classes of models to see which one fits well for our use case. After all, we only care about the dataset we are trying to forecast.

Now, let's look at a modification of how Transformers can be used for time series that learned from the insights of the LTSF-Linear paper and showed that it can outperform the simple linear models we just saw.

Patch Time Series Transformer (PatchTST)

In 2021, Alexey Dosovitskiy et al. proposed Vision Transformer, which introduced the Transformer architecture which was widely successful in Natural Language Processing to Vision. Although not the first to introduce patching, they applied it in a way that works really well for vision. The design broke up an image into patches and fed the transformer each patch in sequence.



Reference check:

The research paper by Alexey Dosovitskiy et al. on Vision Transformers and Yuqi Nie et al. on PatchTST are cited in the *References* section as 12 and 13, respectively.

Fast-forward to 2023, and we have the same patching design applied to time series forecasting. Yuqi Nie et al. proposed **Patch Time Series Transformer (PatchTST)** by adopting the patching design for time series. They were motivated by the apparent ineffectiveness of more complicated Transformer designs (like Autoformer and Informer) on time series forecasting.

In 2023, Zheng et al. showed up many Transformer models by comparing them with a simple linear model which outperformed most of the Transformer models on common benchmarks. One of the key insights from the paper was that the point-wise application of time series to Transformer architecture doesn't capture the locality information and strong ordering in time series data. Therefore, the authors proposed a simpler alternative that performs better than the linear models and solves the problem of including long context windows to Transformers without blowing up the memory and compute requirements.

The architecture of the PatchTST model

The PatchTST model is a modification of Transformers. The following are its major contributions:

- **Patching:** A methodical way to include the history of the series along with other information, which will help in capturing long-term signals such as the week, month, holidays, and so on.
- **Channel-independence:** A conceptual way to process multi-variate time series as separate, independent time series. Although I wouldn't call this a major contribution, this is indeed something we need to be aware of.

Let's take a look at these in a bit more detail.

Patching

We saw some adaptations of Transformers for time series forecasting earlier in the chapter. All of them focused on making attention mechanisms adapt to time series forecasting and longer context windows. But all of them used attention in a pointwise manner. Let's use a diagram to make the point clearer and introduce patching.

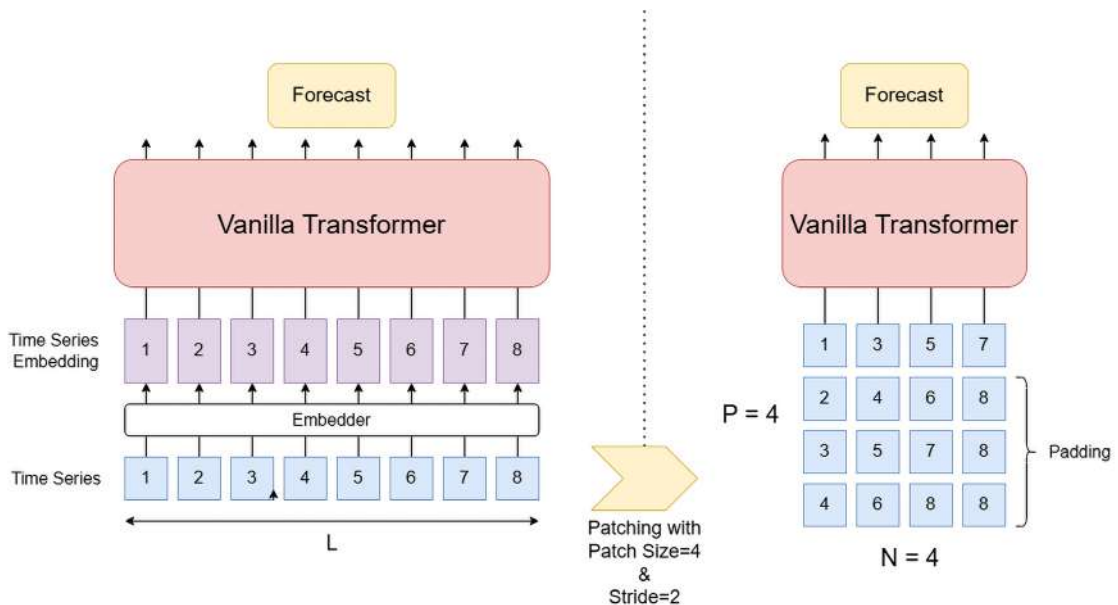


Figure 16.5: Patched vs non-patched time series inputs to Transformers

In *Figure 16.5*, we are considering a time series with 8 time steps as an example. On the left-hand side, we can see how all the other transformer architectures we have discussed handle the time series. They use some mechanism, like the Uniform Representation in AutoFormer, to convert a time series point into a k -dimensional embedding and then feed it to the Transformer architecture point by point. The attention mechanism for each point is calculated by looking at all the other points in the context window.

The PatchTST paper claims that this kind of point-wise attention for time series doesn't capture the locality effectively and proposes converting the time series into patches and feeding those patches to the Transformer instead. Patching is nothing but making the time series into shorter time series in a process very similar (or almost identical) to the sliding window operation we saw earlier in the book. The major difference is that this patching is done after we have already sampled a window from the larger time series.

Patching is typically defined by a couple of parameters:

- Patch Length (P) is the length of each sub-time series or patch.
- Stride (S) is the length of the non-overlapping region between two consecutive patches. More intuitively, this is the number of time steps we move in each iteration of patching. This holds the exact same meaning as stride in convolutions.

With these two parameters fixed, a time series of length L would result in $N = ((L - P)/S) + 2$ patches. Here, we also pad repeated numbers of the last value to the end of the original sequence to ensure each patch is of the same size.

In *Figure 16.5*, we can see that we have illustrated the patching process of a time series with length $L = 8$, with $P = 4$, and $S = 2$. Using the formula we saw just now, we can calculate $N = 4$. We can also see that the last value, 8, has been repeated at the end as a padding to make the last patch length also 4.

Now, each of these patches is considered as the embedding, of sorts, and passed into and processed by the Transformer architecture. With this kind of input patching, for a given context of L , the number of input tokens to the Transformer can be reduced to, approximately, L/S . This means that the computational complexity and memory usage are also reduced by a factor of S . This enables the model to process longer context windows with the same hardware constraints, thus possibly enhancing the forecasting performance of the model.

Now, let's look at channel independence.

Channel independence

A multivariate time series can be thought of as a multi-channel signal. Transformer inputs can either be a single channel or multiple. Most of the other Transformer-based models capable of multivariate forecasting take the approach where the channels are mixed together and processed. Or, in other words, input tokens take in information from all time series and project it to a shared embedding space, mixing information. But other simpler approaches process each channel separately, and the authors of PatchTST bring that independence to Transformers.

In practice, this is very simple. Let's try to understand it with an example. Consider a dataset where there are M time series, making it a multi-variate time series. So, the input to the PatchTST would be $B \times M \times C$, where B is the batch size and C is the length of the context window. After patching, it becomes $B \times M \times N \times P$, where N is the number of patches and P is the patch length. Now, to process this multi-variate signal in a channel-independent way, we just reshape the tensor such that each of the M time series becomes another sample in the batch, i.e., $\mathbb{B} \times N \times P$, where $\mathbb{B} = B \times M$.

While this independence brings some desirable properties to the model, it also means that any interaction between different time series is ignored as they are treated as completely independent. The model is still trained in a global model paradigm and will benefit from cross-learning, but any explicit interaction between different time series (like two time series varying together) is not captured.

Apart from these major components, the architecture is pretty similar to vanilla Transformer architecture. Now, let's look at how we can practically forecast using PatchTST.

Forecasting with PatchTST

PatchTST is implemented in NIXTLA forecasting. The same framework as used previously can be used here with PatchTST as well.

Let's look at the initialization parameters of the implementation.

The PatchTST class has the following major parameters:

- **encoder_layers:** This is an integer representing the number of encoder layers.
- **hidden_size:** This parameter sets the size of the embeddings and the encoders, directly influencing the model's capacity and its ability to capture information from the data. This is the activation to be used in the encoder and decoder layers.
- **patch_len & stride:** These parameters define how the input sequence is divided into patches, which affects how the model perceives temporal dependencies. **patch_len** controls the length of each segment, while **stride** affects the overlap between these segments.
- **stride:** This parameter sets the size of the embeddings and the encoders, directly influencing the model's capacity and its ability to capture information from the data. This is the activation to be used in the encoder and decoder layers.

Regularization parameters:

- **dropout:** This is a float between 0 and 1, which determines the strength of the dropout in the network.
- **fc_dropout:** This is a float value that is the linear layer dropout.
- **head_dropout:** This is a float value that is the flatten layer dropout.
- **attn_dropout:** This is a float value that is the attention layer dropout.



Notebook alert:

The complete code for training the PatchTST model can be found in the `06-PatchTST_NeuralForecast.ipynb` notebook in the `Chapter16` folder.

Now, let's look at another Transformer-based model that took the innovation from PatchTST and turned it on its head for good effect, outperforming the LTSF-Linear models.

iTransformer

We have already talked at length about the inadequacies of Transformer architectures in handling multivariate time series, namely the inefficient capture of locality, the order-agnostic attention mechanism muddling up information across time steps, and so on. In 2024, Yong Liu et al. took a slightly different view of this problem and, in their own words, “an extreme case of patching.”

The architecture of iTransformer

They proposed that it is not that the Transformer architecture is ineffective for time series forecasting, but rather it is improperly used. The authors suggested that we flip the inputs to the Transformer architecture so that the attention isn't applied across time steps but rather across variates or different series/features on the time series. Figure 16.6 shows the difference clearly.

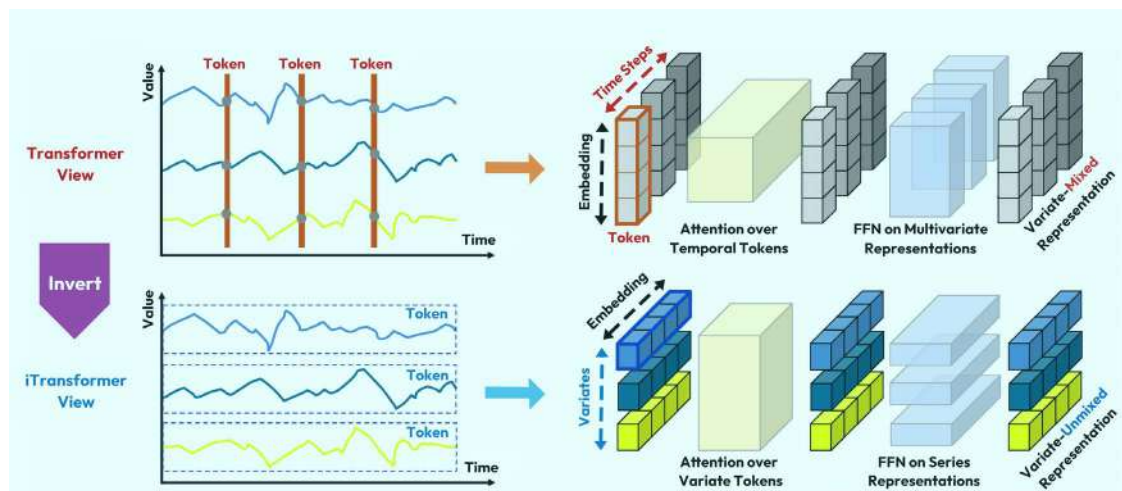


Figure 16.6: Transformers vs iTransformers—the difference

In vanilla Transformers, we use the input as (*Batch x Time steps x Embeddings (features)*), the attention gets applied across the time steps, and eventually, the Position-Wise Feed Forward Network mixes the different features into a Variate-Mixed Representation. But when you flip the input to (*Batch x Embeddings (features) x Timesteps*), the attention gets calculated across the variables and the Position-Wise Feed Forward Network mixes the time leaving variates separate in a Variate-Unmixed Representation.

This “flipping” comes with some more benefits. Now that attention isn't calculated across time, we can include very large context windows with minimal computational and memory constraints (remember that the computational and memory complexity comes from the $O(N^2)$ of the attention mechanism). In fact, the paper suggests including the entire time series history as the context window. On the other hand, we need to be mindful of the number of features or concurrent time series we include in the model.

A typical Transformer architecture has these major components:

- Attention mechanism
- Feed forward network
- Layer normalization

In the inverted version, we already saw that attention is applied across variates and the **Feed Forward Network (FFN)** learns generalizable representations of the lookback window for the final prediction of the forecast. The layer normalization also works out well in the inverted version. In standard Transformers, layer normalization is typically used to normalize the multivariate representation of each time step. But in the inverted version, we normalize each variate separately across time. This is similar to the normalization we were doing in the N-Linear model and has been proven to work well on non-stationary time series problems.



Reference check:

The research paper by Yong Liu et al. on iTransformers is cited in the *References* section as 18.

Forecasting with iTransformer

iTransformer is implemented in NIXTLA forecasting. The same framework as was used previously can be used here with iTransformer as well.

The iTransformer class has the following major parameters:

- `n_series`: This is an integer representing the number of time series.
- `e_layers`: This is an integer representing the number of encoder layers.
- `d_layers`: This is an integer representing the number of decoder layers.
- `d_ff`: This is an integer representing the number of kernels in the 1-dimensional convolutional layers used in the encoder and decoder layers.



Notebook alert:

The complete code for training the iTransformer model can be found in the `07-iTransformer_NeuralForecast.ipynb` notebook in the Chapter16 folder.

Now, let's look at one more, very successful, architecture that is well-designed to utilize all kinds of information in a global context.

Temporal Fusion Transformer (TFT)

TFT is a model that is thoughtfully designed from the ground up to make the most efficient use of all the different kinds of information in a global modeling context—static and dynamic variables. TFT also has interpretability at the heart of all design decisions. The result is a high-performing, interpretable, and global DL model.



Reference check:

The research paper by Lim et al. on TFT is cited in the *References* section as 10.

At first glance, the model architecture looks complicated and daunting. But once you peel the onion, it is quite simple and ingenious. We will take this one level of abstraction at a time to ease you into the full model. Along the way, there will be many black boxes I'm going to ask you to take for granted, but don't worry—we will open every one of them as we dive deeper.

The architecture of TFT

Let's establish some notations and a setting before we start. We have a dataset with I unique time series and each entity, i , has some static variables (s_i). The collection of all static variables of all entities can be represented by S . We also have the context window of length k . Along with this, we have the time-varying variables, which have one distinction—for some variables, we do not have the future data (unknown), and for other variables, we know the future (known). Let's denote all the time-varying information (the context window, known, and unknown time-varying variables) from the context window's input, $x_{t-k} \dots x_t$. The known time-varying variables for the future are denoted using $x_{t+1} \dots x_{t+\tau}$, where τ is the forecast horizon. With these notations, we are ready to look at the first level of abstraction:

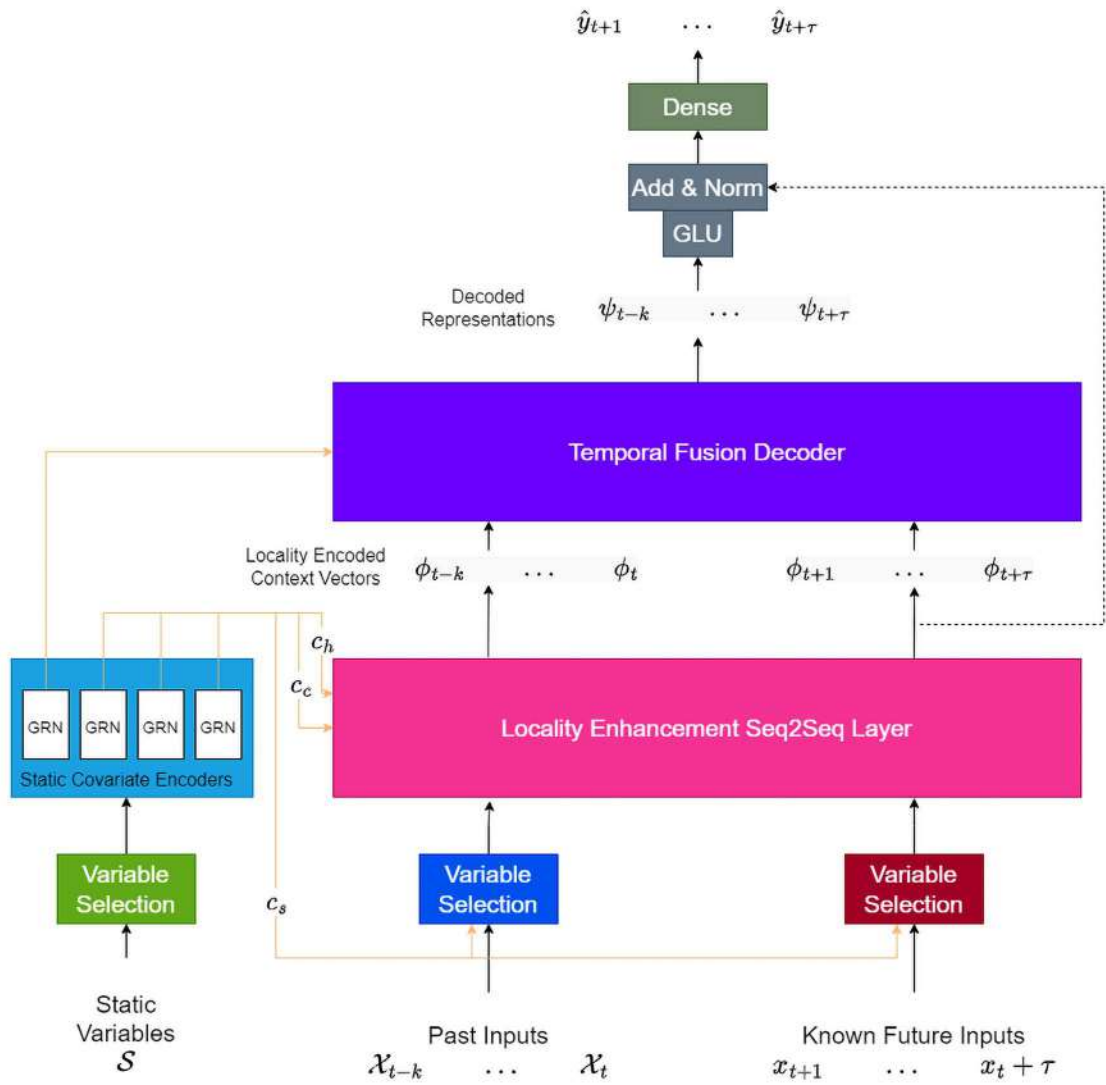


Figure 16.7: TFT—a high-level overview

There is a lot to unpack here. Let's start with the static variables, S . First, the static variables are passed through a **Variable Selection Network** (VSN). The VSN does instance-wise feature selection and performs some non-linear processing on the inputs. This processed input is fed into a bunch of **Static Covariate Encoders** (SEs). The SE block is designed to integrate the static metadata in a principled way.

If you follow the arrows from the SE block in *Figure 16.6*, you will see that the static covariates are used in three (four distinct outputs) different places in the architecture. We will see how these are used in each of these places when we talk about them. But all these different places may be looking at different aspects of the static information. To allow the model this flexibility, the processed and variable-selected output is fed into four different **Gated Residual Networks (GRNs)**, which, in turn, generate four outputs— c_s , c_e , c_c , and c_h . We will explain what a GRN is later, but for now, just understand that it is a block capable of non-linear processing, along with a residual connection, which enables it to bypass the non-linear processing if needed.

The past inputs, $x_{t-k} \dots x_t$, and the future known inputs, $x_{t+1} \dots x_{t+\tau}$, are also passed through separate VSNs and these processed outputs are fed into a **Locality Enhancement (LE)** Seq2Seq layer. We can think of LE as a way to encode the local context and temporal ordering into the embeddings of each timestep. This is similar to the positional embeddings in vanilla Transformers. We can also see similar attempts in the Conv1d layers that were used to encode the history in the uniform representation in the Autoformer models. We will see what is happening inside the LE later, but for now, just understand it captures the local context conditioned on other observed variables and static information. Let's call the output of the block **Locality Encoded Context Vectors** ($\phi_{t-k} \dots \phi_t$, and $\phi_{t+1} \dots \phi_{t+\tau}$).



The terminology, notation, and grouping of major blocks are not the same as in the original paper. I have changed these to make them more accessible and understandable.

Now, these LE context vectors are fed into a **Temporal Fusion Decoder (TFD)**. The TFD applies a slight variation of multi-head self-attention in a Transformer model-like manner and produces the **Decoded Representation** ($\psi_{t-k} \dots \psi_{t+\tau}$). Finally, this decoded representation is passed through a **Gated Linear Unit (GLU)** and an **Add & Norm** block that adds the LE context vectors as a residual connection.

A GLU is a unit that helps the model decide how much information it needs to allow to flow through. We can think of it as a learned information throttle that is widely used in **Natural Language Processing (NLP)** architectures. The formula is really simple:

$$GLU(X) = (X * W * b) \otimes \sigma(X * V + c)$$

Here, W and V are learnable weight matrices, b and c are learnable biases, σ is an activation function, and $\otimes$ is the Hadamard product operator (element-wise multiplication).

The **Add & Norm** block is the same as in the vanilla Transformer; we discussed this back in *Chapter 14, Attention and Transformers for Time Series*.

Now, to top it all off, we have a Dense layer (linear layer with bias) that projects the output of the Add & Norm block to the desired output dimensions.

And with that, it is time for us to step one level down in our abstraction.

Locality Enhancement Seq2Seq layer

Let's peel back the onion and see what's happening inside the LE Seq2Seq layer. Let's start with a figure:

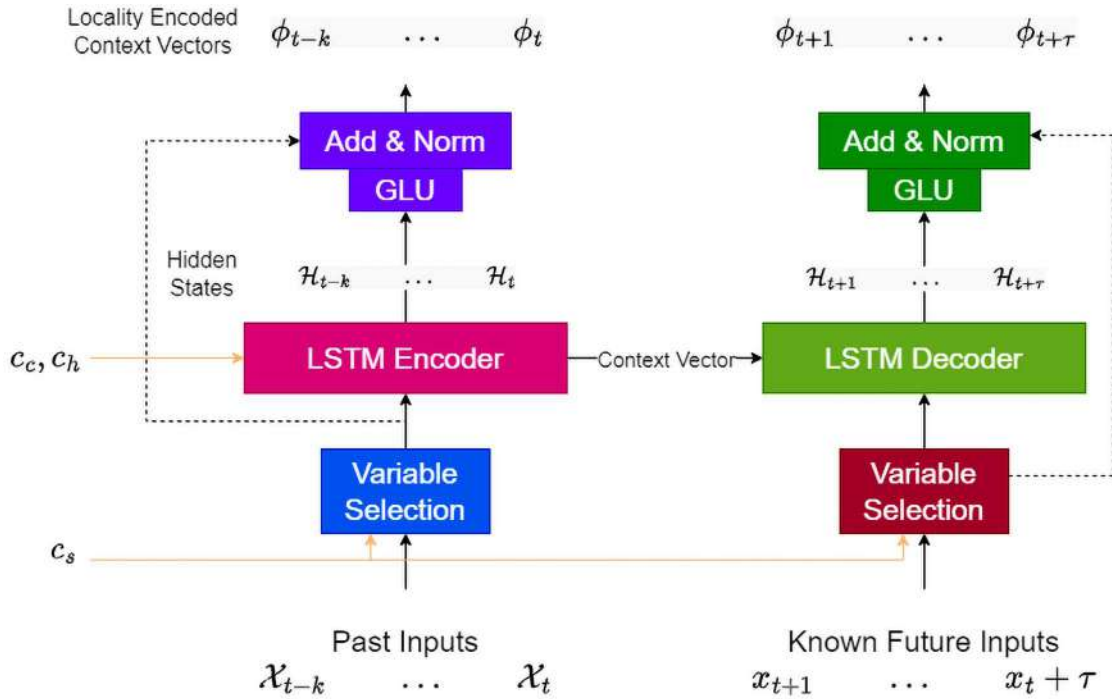


Figure 16.8: TFT-LE Seq2Seq layer

The LE uses a Seq2Seq architecture to capture the local context. The process starts with the processed past inputs. The LSTM encoder takes in these past inputs, $x_{t-k} \dots x_t$. c_h, c_c from the static covariate encoder acts as the initial hidden states of the LSTM. The encoder processes each timestep at a time, producing hidden states at each time step, $H_{t-k} \dots H_t$. The last hidden states (context vector) are now passed on to the LSTM decoder, which processes the known future inputs, $x_{t+1} \dots x_{t+\tau}$, and produces the hidden states at each of the future timesteps, $H_{t+1} \dots H_{t+\tau}$. Finally, all these hidden states are passed through a **GLU + AddNorm** block with the residual connection from before the LSTM processing. The outputs are the LE context vectors ($\phi_{t-k} \dots \phi_t$ and $\phi_{t+1} \dots \phi_{t+\tau}$).

Now, let's look at the next block: the TFD.

Temporal fusion decoder

Let's start this discussion with another figure:

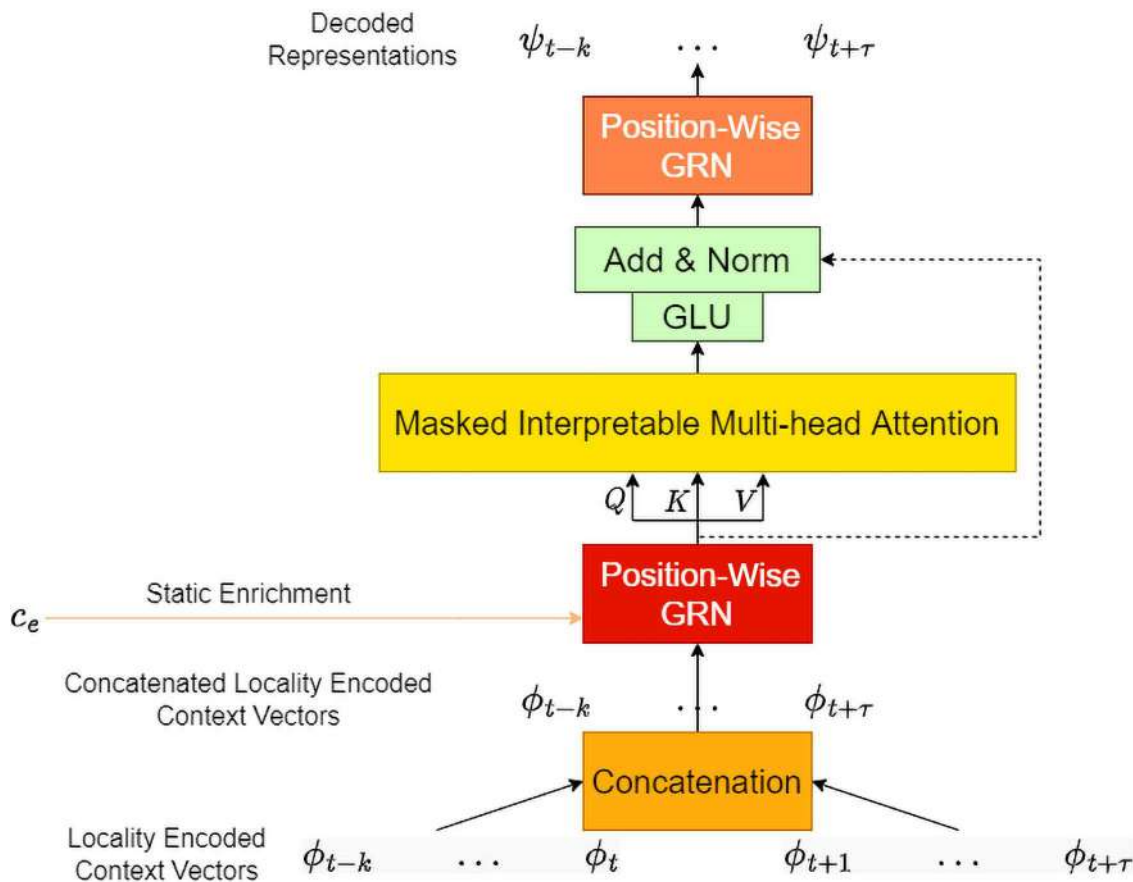


Figure 16.9: Temporal Fusion Transformer—Temporal Fusion Decoder

The LE context vectors from both the past input and known future input are concatenated into a single LE context vector. Now, this can be thought of as the position-encoded tokens in the Transformer paradigm. The first thing the TFD does is enrich these encodings with static information, c_e , that was created from the static covariate encoder. This was concatenated with the embeddings. A position-wise GRN is used to enrich the embeddings. These enriched embeddings are now used as the query, key, and values for the **Masked Interpretable Multi-Head Attention** block.

The paper posits that the **Masked Interpretable Multi-Head Attention** block learns long-term dependencies across time steps. The local dependencies are already captured by the LE Seq2Seq layer in the embeddings, but the point-wise long-term dependencies are captured by **Masked Interpretable Multi-Head Attention**. This block also enhances the interpretability of the architecture. The attention weights that are generated in the process give us some indication of the major timesteps involved in the process. However, the multi-head attention has one drawback from the interpretability perspective.

In vanilla multi-head attention, we use separate projection weights for the values, which means that the values for each head are different and hence the attention weights are not straightforward to interpret.

TFT gets over this limitation by employing a *single shared weight matrix* for projecting the values into the attention dimension. Even with the shared value projection weights, because of the individual query and key projection weights, each head can learn different temporal patterns. In addition to this, TFT also employs masking to make sure information from the future is not used in operations. We discussed this type of causal masking in *Chapter 14, Attention and Transformers for Time Series*. With these two modifications, TFT names this layer **Masked Interpretable Multi-Head Attention**.

And with that, it's time to open the last and most granular level of abstraction we have been using.

Gated residual networks

We have been talking about GRNs for some time now; so far, we have just taken them at face value. Let's understand what is happening inside a GRN—one of the most basic building blocks of a TFT.

Let's look at a schematic diagram of a GRN to understand it better:

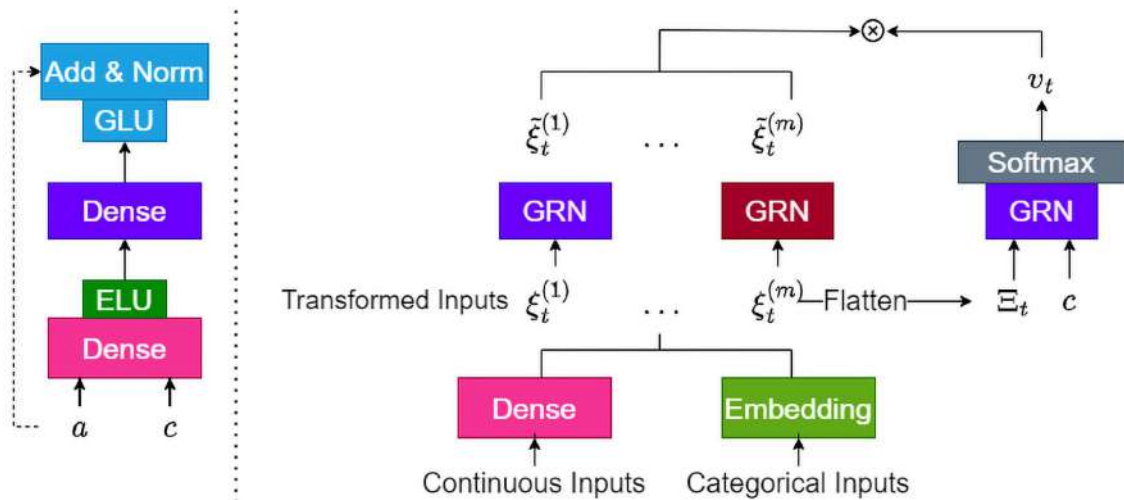


Figure 16.10: TFT—GRN (left) and VSN (right)

The GRN takes in two inputs: the primary input, a , and the external context, c . The context, c , is an optional input and is treated as zero if it's not present. First, both the inputs, a and c , are transformed by separate dense layers and a subsequent activation function—the **Exponential Linear Unit (ELU)** (<https://pytorch.org/docs/stable/generated/torch.nn.ELU.html>).

Now, the transformed a and c inputs are added together and then transformed again using another Dense layer. Finally, this is passed through a **GLU+Add & Norm** layer with residual connections from the original a . This structure bakes in enough non-linearity to learn complex interactions between the inputs, but at the same time lets the model ignore those non-linearities through a residual connection. Therefore, such a block allows the model to scale the computation required up or down based on the data.

Variable selection networks

The last building block of the TFT is the VSN. VSNs enable TFT to do instance-wise variable selection. Most real-world time series datasets have many variables that do not have a lot of predictive power, so being able to select the ones that do have predictive power automatically will help the model pick out relevant patterns. *Figure 16.9* (right) shows this VSN.

These additional variables can be categorical or continuous. TFT uses entity embeddings to convert the categorical features into numerical vectors of the dimension that we desire (d_{model}). We talked about this in *Chapter 15, Strategies for Global Deep Learning Forecasting Models*. The continuous features are linearly transformed (independently) into the same dimension, d_{model} . This gives us the transformed inputs, $\xi_t^{(1)} \dots \xi_t^{(m)}$, where m is the number of features and t is the timestep. We can concatenate all these embeddings (flatten them) and that flattened representation can be represented as Ξ_t .

Now, there are two parallel streams in which these embeddings are processed—one for non-linear processing of the embeddings and another to do feature selection. Each of these embeddings is processed by separate GRNs (but shared for all timesteps) to give us the non-linearly processed ones, $\xi_t^{(1)} \dots \xi_t^{(m)}$. In another stream, the VSN processes the flattened representation, Ξ_t , along with optional context information, c , and processes it through a GRN with a softmax activation. This gives us a weight, v_t , which is a vector of length m . This v_t is now used in a weighted sum of all the non-linearly processed feature embeddings, $\xi_t^{(1)} \dots \xi_t^{(m)}$, which is calculated as follows:

$$\tilde{\xi}_t = \sum_{j=1}^m v_t^{(j)} \xi_t^{(j)}$$

Forecasting with TFT

TFT is implemented in NIXTLA forecasting. We can use the same framework we were working with for NBEATS and extend it to train *TFT* on our data. Additionally, NIXTLA supports exogenous variables, the same way N-BEATSx handles exogenous variables. First, let's look at the initialization parameters of the implementation.

The TFT class in NIXTLA has the following major parameters:

- **hidden_size:** This is an integer representing the hidden dimension across the model. This is the dimension in which all the GRNs work, the VSN, the LSTM hidden sizes, the self-attention hidden sizes, and so on. Arguably, this is the most important hyperparameter in the model.
- **n_head:** This is an integer representing the number of attention heads.
- **dropout:** This is a float between 0 and 1, which determines the strength of the dropout in the Variable Selection Networks.
- **attn_dropout:** This is a float between 0 and 1, which determines the strength of the dropout in the decoder's attention layer.

**Notebook alert:**

The complete code for training TFT can be found in the `08-TFT_NeuralForecast.ipynb` notebook in the `Chapter16` folder.

Interpreting TFT

TFT approaches interpretability from a slightly different perspective than N-BEATS. While N-BEATS gives us a decomposed output for interpretability, TFT gives us visibility into how the model has interpreted the variables it has used. On account of the VSNs, we have ready access to feature weights. Like the feature importance we get from tree-based models, TFT gives us access to similar scores. Because of the self-attention layer, the attention weights can also be interpreted to help us understand which time steps hold a large enough weightage in the attention mechanism.

PyTorch Forecasting makes this possible by performing a few steps. First, we get the raw predictions using `mode="raw"` in the `predict` function. Then, we use those raw predictions in the `interpret_output` function. There is a parameter called `reduction` in the `interpret_output` function that decides how to aggregate the weights across different instances. We know that TFT does instance-wise feature selection in VSNs and attention is also done instance-wise. `'mean'` is a good option for looking at the global interpretability:

```
raw_predictions, x = best_model.predict(val_dataloader, mode="raw", return_
x=True)
interpretation = best_model.interpret_output(raw_predictions, reduction="sum")
```

This `interpretation` variable is a dictionary with weights for different aspects of the model, such as `attention`, `static_variables`, `encoder_variables`, and `decoder_variables`. PyTorch Forecasting also provides us with an easy way to visualize this importance:

```
best_model.plot_interpretation(interpretation)
```

This generates four plots:

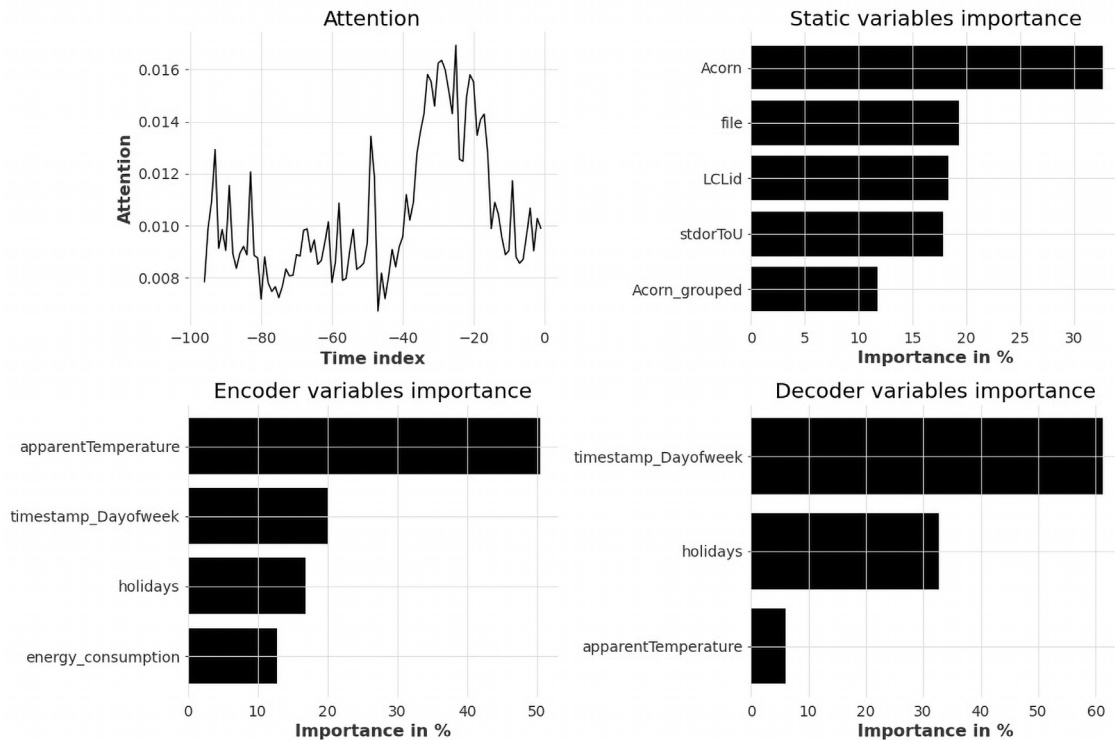


Figure 16.11: Interpreting TFT

We can also look at each instance and plot similar visualizations for each prediction we make. All we need to do is use `reduction="none"` and then plot it ourselves. The accompanying notebook explores how to do that and more.

Now, let's switch tracks and look at some models that proved that simple MLPs are also more than capable of matching or beating Transformer-based models.

TSMixer

While the Transformer-based models were forging ahead with steam, a parallel track of research started by using **Multi-Layer Perceptrons (MLPs)** instead of Transformers as the key learning unit. The trend kicked off in 2021 when MLP-Mixer showed that one can attain state-of-the-art performance in vision problems by using just MLPs, replacing Convolutional Neural Networks. And so, similar mixer architectures using MLPs as the key learning component started popping up in all domains. In 2023, Si-An Chen et al. from Google brought mixing MLPs into time series forecasting.



Reference check:

The research paper by Si-An et al. on TSMixer is cited in the *References* section as 19.

The architecture of the TSMixer model

TSMixer really took inspiration from the Transformer model but tried to replicate similar processes with an MLP. Let's use *Figure 16.10* to understand the similarities and differences.

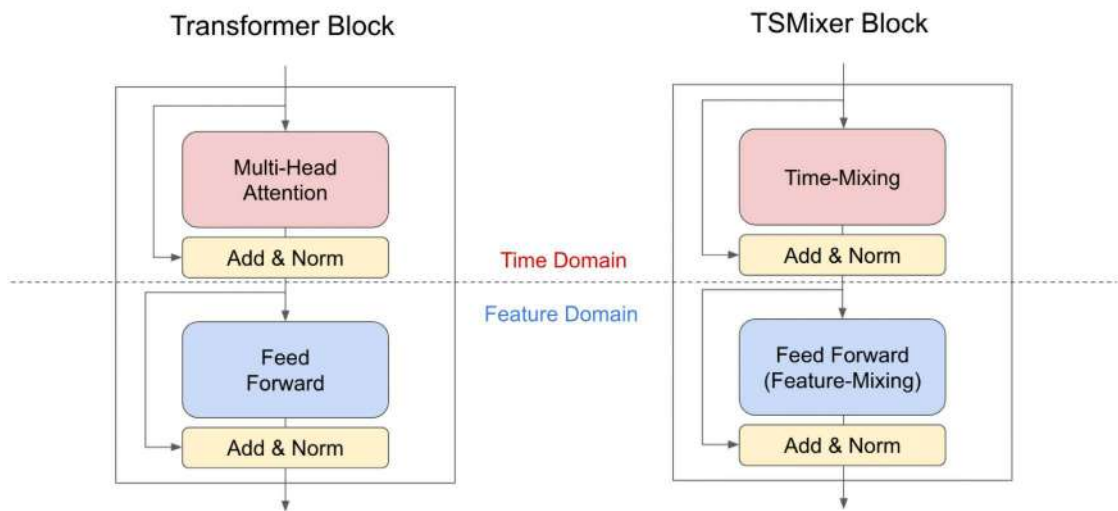


Figure 16.12: Transformer vs TSMixer

If you look at the Transformer block, we can see that there is a Multi-Head Attention that looks across timesteps, and “mixes” them together using attention. Then those outputs are passed on to the Position-Wise Feed Forward networks, which “mix” the different features together. Drawing inspiration from these, the TSMixer also has a Time-Mixing component and a Feature-Mixing component in a Mixer block. There is a Temporal Projection that takes the output from the Mixer block and projects it to the output space.

Let's take this one level of explanation at a time. *Figure 16.11* shows the entire architecture.

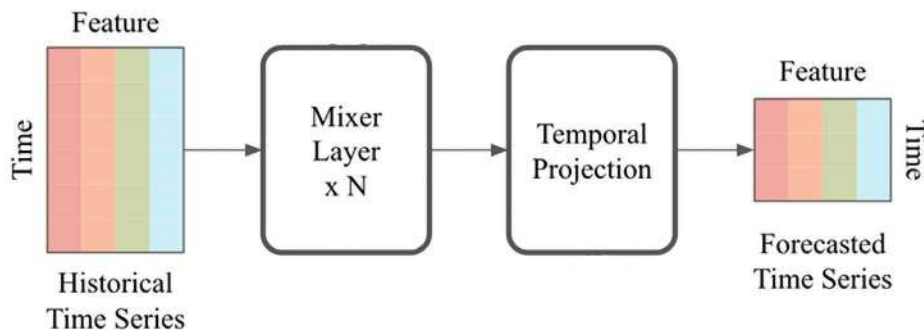


Figure 16.13: TSMixer Architecture

The input, which is a multivariate time series is fed into N mixer layers, which process it sequentially and the output from the final mixer layer is fed into the temporal projection layer, which converts the learned representation into the actual forecast. Although the figure and the paper refer to “features,” they aren’t features in the way we have been discussing in this book. Here, “features” means other time series in a multi-variate setting.

Now, let's double-click on Mixer Layer and see the Time Mixing and Feature Mixing inside a block.

Mixer Layer

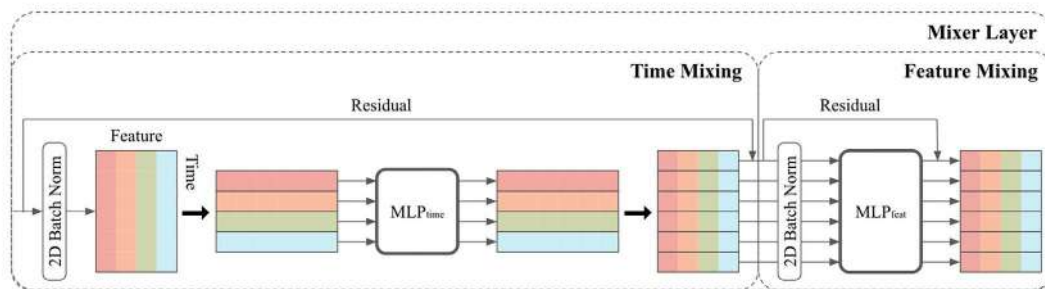


Figure 16.14: TSMixer—Mixer block

The input is of the form ($Batch\ Size \times Features \times Time\ Steps$) and is first passed through the Time Mixing block. The input is first transposed into the form ($Batch\ Size \times Time\ Steps \times Features$) such that the weights in the Time Mixing MLP are mixing timesteps. Now, this “time-mixed” output is passed to the Feature Mixing MLP, which uses its weights to mix the different features to give the final learned representation. Batch Normalization layers and residual connections are added in between to make the model more robust and learn deeper and smoother connections.

Given an input matrix $X \in \mathbb{R}^{L \times C}$, Time Mixing can be represented mathematically as:

$$\text{Time Mixing}(X) = 2D\text{Norm}\left(X + \text{Drop}\left(\sigma\left(\text{FC}_{\{L \rightarrow L(X)\}}\right)\right)\right)$$

Feature mixing is actually a two-layer MLP, one projecting to a hidden dimension, H_{inner} , and the next projecting from H_{inner} to the output dimension, H . If not specified explicitly, this defaults to the original number of features (or number of time series), C .

$$U = \text{Drop}\left(\sigma\left(\text{FC}_{\{C \rightarrow H_{\text{inner}}\}}\right)\right)$$

$$\text{Feature Mixing}(X) = 2d\text{Norm}\left(X + \text{Drop}\left(\text{FC}_{\{H_{\text{inner}} \rightarrow H\}}(U)\right)\right)$$

Therefore the entire Mixer layer can be represented as:

$$\text{Mix}_{\{C \rightarrow H\}} = \text{Feature Mixing}_{\{C \rightarrow H\}}\left(\text{Time Mixing}_{\{L \rightarrow L\}}(X)\right)$$

Now, this output is passed through the Temporal Projection layer to get the forecast.

Temporal Projection Layer

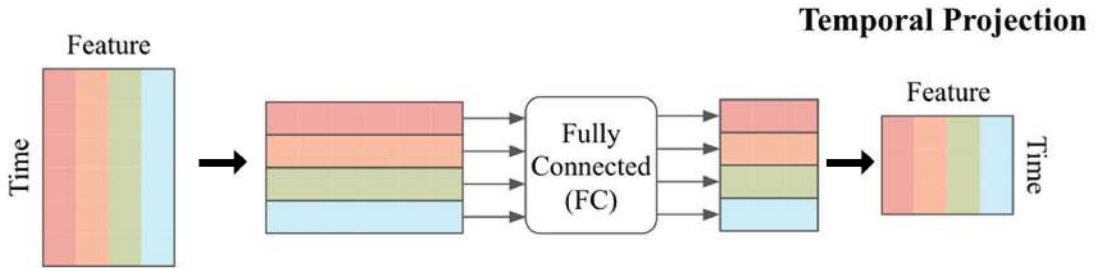


Figure 16.15: TSMixer—Temporal Projection Layer

The temporal projection layer is nothing but a fully connected layer applied to the time domain. This is identical to the simple linear model we saw earlier, where we apply a fully connected layer to the input context to get the forecast. Instead of applying the layer to the input, TSMixer applies this layer to the “mixed” output from the Mixer Layer.

The output from the previous layer is in the form (*Batch Size* $\times$ *Time Steps* $\times$ *Features*). This is transposed to (*Batch Size* $\times$ *Features* $\times$ *Time Steps*) and then passed through a fully connected layer, which projects the input into (*Batch Size* $\times$ *Features* $\times$ *Forecast Horizon*) to get the final forecast.

Given $O_k \in \mathbb{R}^{L \times H}$ as the output of the k -th Mixer Layer and forecast horizon, T :

$$\text{Temporal Projection}(O_k) = \text{FC}_{\{H \times L \rightarrow C \times T\}}$$

But how do we include additional features? Many time series problems have static features and dynamic (future-looking) features, which adds quite a bit of information to the problem. The architecture we have discussed so far doesn't let you include them. For this reason, the authors proposed a slight tweak to include this additional information, TSMixerx.

TSMixerx—TSMixer with auxiliary information

Following the same notation as before, consider we have the input time series (or collection of time series), $X \in \mathbb{R}^{L \times C}$. Now, we would have a few historical features, $\hat{X} \in \mathbb{R}^{L \times C_x}$, some future-looking features, $Z \in \mathbb{R}^{T \times C_z}$, and some static features, $S \in \mathbb{R}^{1 \times C_s}$. To effectively include all this additional information, the authors defined another unit of learning called the Conditional Feature Mixing layer and then used it in a way that assimilates all the information.

The **Conditional Feature Mixing (CFM)** layer is almost identical to the Feature Mixing layer, except for an additional layer to process the static information along with the features. The static information is first repeated across time steps and projected into the output dimension using a linear layer. This is then concatenated with the input features and the concatenated input is then “mixed” together and projected to the output dimension.

Mathematically, it can be represented as:

$$V = FC_{\{C_s \rightarrow H\}}(\text{Expand}(S))$$

$$\text{Conditional Feature Mixing}(X, S) = FC_{\{C+H \rightarrow H\}}(X \oplus V)$$

where $\oplus$ means concatenation and *Expand* means repeating the static information for all the time steps.

Now, let's see how the CFM layer is used in the overall TSMixerx architecture.

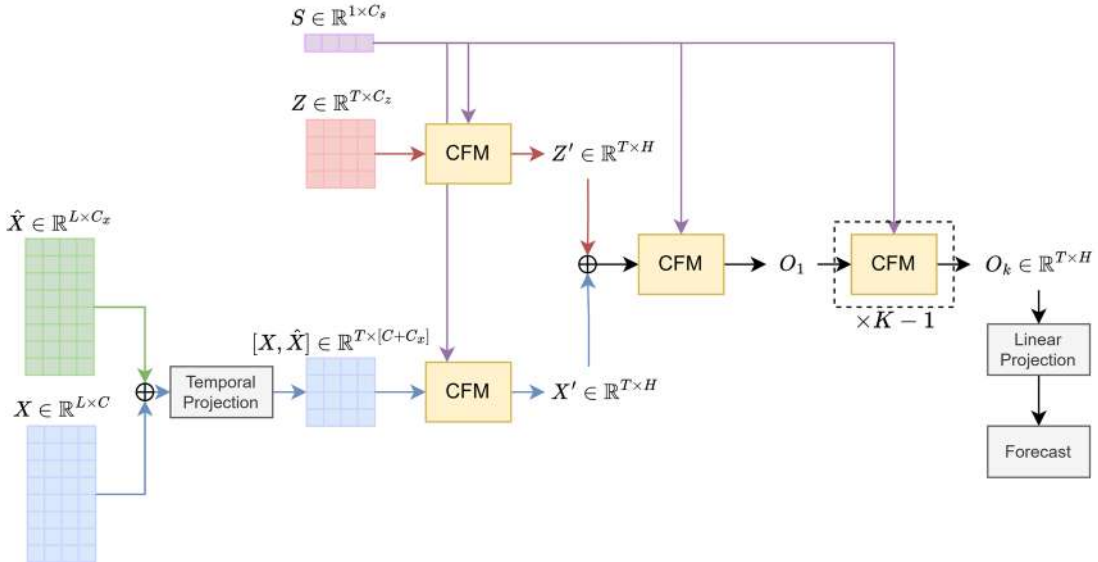


Figure 16.16: TSMixerx—architecture using exogenous variables

First off, we have X and $\hat{X}$, which have L timesteps, which is the length of the context window. Therefore, we concatenate both and use a simple temporal projection layer to project the combined tensor into $T \times (C + C_x)$, where T is the length of the forecast horizon. This is also the length of the future-looking features, Z . Now, we combine this with the static information using a CFM layer, which projects them into a hidden dimension, H . Formally, this step is represented as:

$$X' = CFM_{C+C_x \rightarrow H}(FC_{L \rightarrow T}(X \oplus \hat{X}), S)$$

Now, we want to conditionally mix the future-looking features, Z , as well with the static information, S . Therefore, we use a CFM layer to do that and project this combined information into a hidden dimension, H .

$$Z' = CFM_{C_z \rightarrow H}(Z, S)$$

At this point, we have X' and Z' , which are both in $T \times H$ dimensions. So, we use another CFM layer to mix these features further conditioned on S . This gives us the first feature mixed latent representation, O_1 .

$$O_1 = CFM_{2H \rightarrow H}(X' \oplus Z', S)$$

Now, this latent representation is passed through $K - 1$ subsequent CFMs (similar to regular TSMixer architecture), where K is the total number of Mixer layers, to give us O_k , the final latent representation. There are $K - 1$ layers because the first Mixer layer is already defined and is different from the rest in just the input dimensions.

$$O_k = CFM_{H \rightarrow H}(O_{k-1}, S), \forall k = 2, \dots, K$$

Now, we can use a simple linear layer to project this output into the desired output dimension. If it is a point prediction for a single time series, we can project it to $T \times 1$. In case we are predicting the M time series, then we can project it to $T \times M$.

Forecasting with TSMixer and TSMixerx

TSMixer is implemented in NIXTLA forecasting with the same framework we have seen in the prior models.

Let's look at the initialization parameters of the implementation.

The TSMixer class has the following major parameters:

- `n_series`: This is an integer value indicating the number of time series.
- `n_block`: This is an integer value indicating the number of mixing layers used in the model.
- `ff_dim`: This is an integer value indicating the number of units to use for the second feed forward layer.
- `revin`: This is a Boolean value that, if True, uses Reversible instance Normalization to process inputs and outputs (ICLR 2022 paper: <https://openreview.net/forum?id=cGDAkQo1C0p>).

Similar to NBEATX, there is a `TSMixerx` class that can take exogenous information. To forecast with exogenous information, you would add appropriate information into the parameters below:

- `futr_exog_list`: This takes a list of future exogenous columns.
- `hist_exog_list`: This takes a list of historical exogenous columns.
- `stag_exog_list`: This is a list of exogenous columns.



Notebook alert:

The complete code for training the TSMixer model can be found in the `09-TSMixer_NeuralForecast.ipynb` notebook in the `Chapter16` folder.

Let's now look at one more MLP-based architecture which has shown that it performs better than PatchTST and the Linear family of models we saw earlier.

Time Series Dense Encoder (TiDE)

We saw earlier in the chapter that a linear family of models outperformed quite a lot of Transformer models. In 2023, Das et al. from Google proposed a model that extends that idea into non-linearity. They argued that the linear models will fall short where there are inherent non-linearities in the dependence between the future and the past. The inclusion of covariates compounds this problem.



Reference check:

The research paper by Das et al. on TiDE is cited in the *References* section as 20.

Therefore, they introduced a simple and efficient **Multi-Layer Perceptron (MLP)** based architecture for long-term time series forecasting. The model essentially encodes the past of the time series, along with the covariates using dense MLPs, and then decodes this latent representation into a forecast. The model assumes channel independence (similar to PatchTST) and considers different related time series in a multivariate problem as separate time series.

The architecture of the TiDE model

The architecture has two main components—an encoder and a decoder. But all through the architecture, one learning component they call a Residual block is reused. Let's take a look at the Residual block first.

Residual block

The residual block is an MLP with a ReLU and a subsequent linear projection enabling a residual connection. *Figure 16.14* shows a residual block.

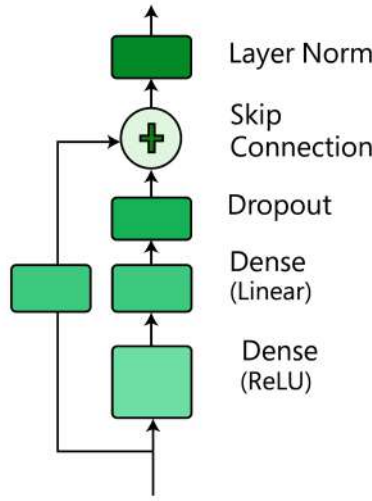


Figure 16.17: TiDE: residual block

We define the layer by setting a hidden dimension and an output dimension. The first Dense layer transforms the input to the hidden dimension and then ReLU non-linearity is applied to the output. This output is then linearly projected to the output dimension and a dropout layer is stacked on top of that. The residual connection is then added to the output by projecting the input into the output dimension using another linear projection. And to top it all off, the output is passed through Layer Normalization.

Let $X \in \mathbb{R}^f$ be the input to the block, h be the hidden dimension, and o be the output dimension. Then, the Residual block can be represented as:

$$\mathcal{O} \in \mathbb{R}^o = \text{LayerNorm} \left(FC_{f \rightarrow o}(X) + \text{Dropout} \left(FC_{h \rightarrow o} \left(\text{ReLU} \left(FC_{f \rightarrow h}(X) \right) \right) \right) \right)$$

Let's establish some notation to help us with the rest of the explanation. There are N time series in the dataset, L is the length of the lookback window, and H is the length of the forecast. So, the lookback of the i^{th} time series can be represented as $y_{1:L}^{(i)}$, and its forecast is $y_{L+1:L+H}^{(i)}$. The r -dimensional dynamic covariates at time t are represented by $x_t^{(i)} \in \mathbb{R}^r$. Static features of the i^{th} time series are $a^{(i)} \in \mathbb{R}^s$.

Now, let's look at the larger architecture in Figure 16.15.

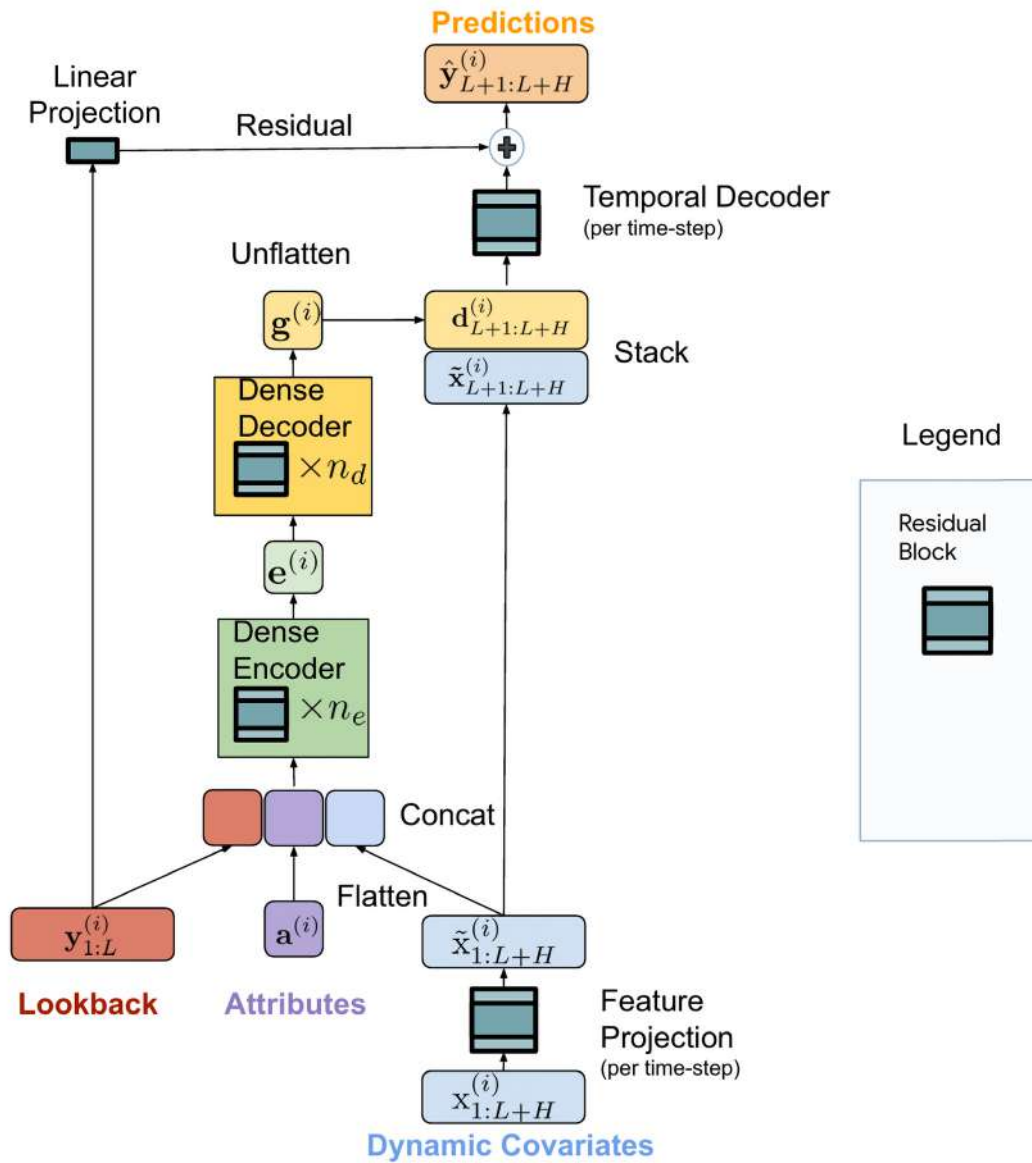


Figure 16.18: TiDE: overall architecture

Encoder

The encoder is tasked with mapping the lookback window and the corresponding covariates into a dense latent representation. The first step is a **Linear Projection** of the dynamic covariates, $x \in \mathbb{R}^r$, into $\tilde{x} \in \mathbb{R}^{\tilde{r}}$, where $\tilde{r}$, called *temporal width*, is much smaller than r . We use the Residual block for this projection.

$$\tilde{x}_t^{(i)} = \text{ResidualBlock}(x_t^{(i)})$$

From a programmatic perspective (where B is the batch size), if the input dimension of the dynamic covariates is $(B \times L \times r)$, we project it to $(B \times L \times \tilde{r})$.

This is done so that when we flatten the time series and its covariates before feeding it through the encoder, the dimension of the resulting tensor doesn't explode. That brings us to the next step, which is the flattening of the tensors and concatenating them. The flattening and concatenation operation looks something like this:

1. Lookback window: $B \times L \times 1 \rightarrow B \times L$
2. Dynamic covariates: $B \times L + H \times \tilde{r} \rightarrow B \times (L + H) \cdot \tilde{r}$
3. Static information: $B \times S$
4. Concatenated representation: $B \times (L + (L + H) \cdot \tilde{r} + S)$

Now, this concatenated representation is passed through a stack of n_e Residual blocks to encode them into a dense latent representation.

$$e^{(i)} = \text{Encoder}(y_{1:L}^{(i)}; \tilde{x}_{1:L+H}^{(i)}; a^{(i)})$$

In the programmatic perspective, the dimensions get transformed from $B \times (L + (L + H) \cdot \tilde{r} + S)$ to $B \times H$, where H is the hidden size of the latent representation.

Now that we have the latent representation, let's look at how we can decode the forecast from this.

Decoder

Just like the encoder, the decoder also has two separate steps. In the first step, we use a stack of n_d of Residual Blocks to decode the latent representation into a decoded vector of dimension, $p \cdot H$, where p is the *decoder output dimension*. This decoded vector is reshaped into a $p \times H$ dimensional vector.

$$g^{(i)} \in \mathbb{R}^{p \cdot H} = \text{Decoder}(e^{(i)})$$

$$D^{(i)} \in \mathbb{R}^{p \times H} = \text{Reshape}(g^{(i)})$$

Now, we use a **Temporal Decoder** to convert this decoded vector into predictions. The temporal decoder is just a Residual block that takes in the concatenated decoded vector, $D^{(i)}$ and the encoded future exogenous vector, $\tilde{x}_{L+1:L+H}^{(i)}$. The authors argue that this residual connection allows some future covariates to affect the forecast in a stronger way. For instance, if one of the future covariates is holidays in a retail forecasting problem, then you want that variable to have a strong influence on the forecast.

This residual connection helps the model enable that “highway” if needed.

$$\hat{y}_{L+t}^{(i)} = \text{TemporalDecoder}(d_t^{(i)}; \tilde{x}_{L+t}^{(i)}) \forall t \in [H]$$

Finally, we add a Global Residual Connection that linearly maps the lookback window to the prediction $\hat{y}_{L+1:L+H}^{(i)}$, after a linear mapping to the right dimension. This ensures that the linear model that we saw earlier in the chapter becomes a subclass of the TiDE model.

Forecasting with TiDE

TiDE is implemented in NIXTLA forecasting with the same framework we have seen in the prior models.

Let’s look at the initialization parameters of the implementation.

The TIDE class has the following major parameters:

- `decoder_output_dim`: An integer that controls the number of units in the output of the decoder (p). It directly impacts the dimensionality of the decoded sequence and can influence the model’s ability to reconstruct the target series.
- `temporal_decoder_dim`: An integer that defines the output size of the temporal decoder. Although we discussed that the output of the temporal decoder is the final forecast, `NeuralForecast` has a uniform map from network output to desired output dimension. Therefore, `temporal_decoder_dim` denotes the dimension of the penultimate layer, which will finally be transformed into the final output. The size of the dimension determines how much information you are allowing to pass on to the final forecasting layer.
- `num_encoder_layers`: The number of encoder layers stacked on top of each other.
- `num_decoder_layers`: The number of decoder layers stacked on top of each other
- `temporal_width`: An integer that affects the lower temporal projected dimension ($\tilde{r}$), influencing how exogenous data is projected and processed. It plays a role in how the model incorporates and learns from exogenous information.
- `layernorm`: This Boolean flag determines whether Layer Normalization is applied. Layer Normalization can stabilize and accelerate training, which might lead to better performance, especially in deeper networks.

Additionally, TIDE can handle exogenous information, which can be included in the below parameters:

- `futr_exog_list`: This takes a list of future exogenous columns
- `hist_exog_list`: This takes a list of historical exogenous columns
- `stag_exog_list`: This is a list of exogenous columns



Notebook alert:

The complete code for training the TIDE model can be found in the `10-TIDE_NeuralForecast.ipynb` notebook in the `Chapter16` folder.

We have covered a few popular specialized architectures for time series forecasting, but this is in no way a complete list. There are so many model architectures and techniques out there. I have included a few in the *Further reading* section to get you started.

Congratulations on making it through probably one of the toughest and densest chapters in this book. Give yourself a pat on the back, sit back, and relax.

Summary

Our journey with deep learning for time series has finally reached a conclusion with us reviewing a few specialized architectures for time series forecasting. We got an understanding of why it makes sense to have specialized architectures for time series and forecasting and went on to understand how different models such as *N-BEATS*, *N-BEATSx*, *N-HiTS*, *Autoformer*, *TFT*, *PatchTST*, *TiDE*, and *TSMixer* work. In addition to covering the architecture and theory behind it, we also looked at how we can use these models on real datasets using `neuralforecast` by NIXTLA. We never know which model will work better with our dataset, but as practitioners, we need to develop intuition that will guide us in the experimentation. Knowing how these models work behind the scenes is essential in developing that intuition and will help us experiment more efficiently.

This brings this part of this book to a close. At this point, you should be much more comfortable with using DL for time series forecasting problems.

In the next chapter, let's look at another important topic in forecasting—probabilistic forecasting.

References

The following is a list of the references that we used throughout this chapter:

1. Spyros Makridakis, Evangelos Spiliotis, and Vassilios Assimakopoulos. (2020). *The M4 Competition: 100,000 time series and 61 forecasting methods*. International Journal of Forecasting, Volume 36, Issue 1. Pages 54–74. <https://doi.org/10.1016/j.ijforecast.2019.04.014>.
2. Slawek Smyl. (2018). *M4 Forecasting Competition: Introducing a New Hybrid ES-RNN Model*. <https://www.uber.com/blog/m4-forecasting-competition/>.
3. Boris N. Oreshkin, Dmitri Carpov, Nicolas Chapados, and Yoshua Bengio. (2020). *N-BEATS: Neural basis expansion analysis for interpretable time series forecasting*. 8th International Conference on Learning Representations, (ICLR). <https://openreview.net/forum?id=r1ecqn4YwB>.
4. Kin G. Olivares and Cristian Challu and Grzegorz Marcjasz and R. Weron and A. Dubrawski. (2022). *Neural basis expansion analysis with exogenous variables: Forecasting electricity prices with NBEATSx*. International Journal of Forecasting, 2022. <https://www.sciencedirect.com/science/article/pii/S0169207022000413>.
5. Cristian Challu and Kin G. Olivares and Boris N. Oreshkin and Federico Garza and Max Mergenthaler-Canseco and Artur Dubrawski. (2022). *N-HiTS: Neural Hierarchical Interpolation for Time Series Forecasting*. arXiv preprint arXiv: Arxiv-2201.12886. <https://arxiv.org/abs/2201.12886>.

6. Vaswani, Ashish, Shazeer, Noam, Parmar, Niki, Uszkoreit, Jakob, Jones, Llion, Gomez, Aidan N, Kaiser, Lukasz, and Polosukhin, Illia. (2017). *Attention is All you Need*. Advances in Neural Information Processing Systems. <https://papers.nips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html>.
7. Haoyi Zhou, Shanghang Zhang, Jieqi Peng, Shuai Zhang, Jianxin Li, Hui Xiong, and Wancai Zhang. (2021). *Informer: Beyond Efficient Transformer for Long Sequence Time-Series Forecasting*. Thirty-Fifth {AAAI} Conference on Artificial Intelligence, {AAAI} 2021. <https://ojs.aaai.org/index.php/AAAI/article/view/17325>.
8. Haixu Wu, Jiehui Xu, Jianmin Wang, and Mingsheng Long. (2021). *Autoformer: Decomposition Transformers with Auto-Correlation for Long-Term Series Forecasting*. Advances in Neural Information Processing Systems 34: Annual Conference on Neural Information Processing Systems 2021, NeurIPS 2021, December 6–14, 2021. <https://proceedings.neurips.cc/paper/2021/hash/bcc0d400288793e8bdcd7c19a8ac0c2b-Abstract.html>.
9. Bryan Lim, Serkan Ö. Arik, Nicolas Loeff, and Tomas Pfister. (2019). *Temporal Fusion Transformers for Interpretable Multi-horizon Time Series Forecasting*. International Journal of Forecasting, Volume 37, Issue 4, 2021, Pages 1,748–1,764. <https://www.sciencedirect.com/science/article/pii/S0169207021000637>.
10. Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. (2021). *An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale*. 9th International Conference on Learning Representations, ICLR 2021. <https://openreview.net/forum?id=YicbFdNTTy>.
11. Yuqi Nie, Nam H. Nguyen, and Phanwadee Sinthong and J. Kalagnanam. (2022). *A Time Series is Worth 64 Words: Long-term Forecasting with Transformers*. 10th International Conference on Learning Representations, ICLR 2022. <https://openreview.net/forum?id=Jbdc0vT0col>.
12. Ailing Zeng and Mu-Hwa Chen, L. Zhang, and Qiang Xu. (2023). *Are Transformers Effective for Time Series Forecasting?* AAAI Conference on Artificial Intelligence. <https://ojs.aaai.org/index.php/AAAI/article/view/26317>.
13. Liu, Shizhan and Yu, Hang and Liao, Cong and Li, Jianguo and Lin, Weiyao and Liu, Alex X and Dustdar, Schahram. (2022). *Pyraformer: Low-Complexity Pyramidal Attention for Long-Range Time Series Modeling and Forecasting*. International Conference on Learning Representations. <https://openreview.net/pdf?id=0EXmFzUn5I>.
14. Zhou, Tian and Ma, Ziqing and Wen, Qingsong and Wang, Xue and Sun, Liang and Jin, Rong. (2022). *{FEDformer}: Frequency enhanced decomposed transformer for long-term series forecasting*. Proc. 39th International Conference on Machine Learning (ICML 2022). <https://proceedings.mlr.press/v162/zhou22g.html>.
15. Shiyang Li, Xiaoyong Jin, Yao Xuan, Xiyong Zhou, Wenhui Chen, Yu-Xiang Wang, and Xifeng Yan. (2019). *Enhancing the locality and breaking the memory bottle neck of transformer on time series forecasting*. Advances in Neural Information Processing Systems. https://proceedings.neurips.cc/paper_files/paper/2019/file/6775a0635c302542da2c32aa19d86be0-Paper.pdf.

16. Yong Liu, Tengge Hu, Haoran Zhang, Haixu Wu, Shiyu Wang, Lintao Ma, and Mingsheng Long. (2024). *iTransformer: Inverted Transformers Are Effective for Time Series Forecasting*. 12th International Conference on Learning Representations, ICLR 2024. <https://openreview.net/forum?id=JePfAI8fah>.
17. Si-An Chen and Chun-Liang Li and Sercan O Arik and Nathanael Christian Yoder and Tomas Pfister. (2023). *TSMixer: An All-MLP Architecture for Time Series Forecasting*. Transactions on Machine Learning Research. <https://openreview.net/forum?id=wbpxTuXgm0>.
18. Abhimanyu Das, Weihao Kong, Andrew Leach, Shaan Mathur, Rajat Sen, and Rose Yu. (2023). *Long-term Forecasting with TiDE: Time-series Dense Encoder*. Transactions on Machine Learning Research. <https://openreview.net/forum?id=pCbC3aQB5W>.

Further reading

You can check out the following resources for further reading:

- *Fast ES-RNN: A GPU Implementation of the ES-RNN Algorithm*: <https://arxiv.org/abs/1907.03329> and <https://github.com/damitkwr/ESRNN-GPU>
- *Functions as Vector Spaces*: <https://www.youtube.com/watch?v=NvEZol2Q8rs>
- *Forecast with N-BEATS*, by Gaetan Dubuc: <https://www.kaggle.com/code/gatandubuc/forecast-with-n-beats-interpretable-model/notebook>
- *WaveNet: A Generative Model for Audio*, by DeepMind: <https://www.deepmind.com/blog/wavenet-a-generative-model-for-raw-audio>
- *What is Residual Connection?*, by Wanshun Wong: <https://towardsdatascience.com/what-is-residual-connection-efb07cab0d55>
- *Efficient Transformers: A Survey*, by Tay et al.: <https://arxiv.org/abs/2009.06732>
- *Autocorrelation and the Wiener-Khinchin theorem*: https://www.itp.tu-berlin.de/fileadmin/a3233/grk/pototskyLectures2012/pototsky_lectures_part1.pdf
- *Modelling Long- and Short-Term Temporal Patterns with Deep Neural Networks*, by Lai et al.: <https://dl.acm.org/doi/10.1145/3209978.3210006> and <https://github.com/cure-lab/SCINet>
- *Think Globally, Act Locally: A Deep Neural Network Approach to High-Dimensional Time Series Forecasting*, by Sen et al.: <https://proceedings.neurips.cc/paper/2019/hash/3a0844cee4fcf57de0c71e9ad3035478-Abstract.html> and <https://github.com/rajatsen91/deepglo>

**Unlock this book's exclusive
benefits now**

This book comes with additional benefits
designed to elevate your learning experience.



Note: Have your purchase invoice ready before you begin. <https://www.packtpub.com/unlock/9781835883181>

17

Probabilistic Forecasting and More

Throughout the book, we have learned different techniques to generate a forecast, including some classical methods, using machine learning, and some deep learning architectures as well. But we have been focusing on one typical type of forecasting problem—generating a point forecast for continuous time series with no hierarchy and a good amount of history. We have been doing that because this is the most popular kind of problem you will face. But in this chapter, we will take some time to look at a few niche topics that, although less popular, are no less important.

In this chapter, we will focus on these topics:

- Probabilistic forecasting
 - Probability Density Functions
 - Quantile functions
 - Monte Carlo Dropout
 - Conformal Prediction
- Intermittent/sparse time series forecasting
- Interpretability
- Cold-start forecasting
 - Pre-trained models like TimeGPT
 - Similarity-based forecasting
- Hierarchical forecasting

Probabilistic forecasting

So far, we have been talking about the forecast as a single number. We have been projecting our DL models to a single dimension or training our machine learning models to output a single number. Subsequently, we were training the model using a loss, such as mean squared loss. This paradigm is what we call a *point forecast*. But we are not considering one important aspect. We are using the history to train our model to make the best guess. But how sure is the model about its prediction? Those of you who are aware of machine learning and classification problems would recognize that for classification problems, besides getting a prediction of which class the sample belongs to, we also get a notion of the uncertainty of the model. But our forecasting is a regression problem and we don't get the uncertainty for free.

But why is quantifying uncertainty important in forecasting? Any forecast is created for some purpose, some downstream task for which the forecasted information is being used. In other words, there is some decision that has to be taken using the forecast we generate. And when making a decision, we usually would like to have the maximum amount of information available to us.

Let's look at an example to really drive home the point. You have recorded your monthly grocery consumption for the last 5 years and, using the techniques in the book, created a super accurate forecast and an app that tells you how much to shop for in any month. You open up the app and it tells you that you need to buy two bread loaves this month. You head out to the supermarket, bag two loaves of bread, and return home. And a week before the end of the month, the bread loaves ran out, and you starved the rest of the time. At that peak of starvation, you started questioning your decisions and the forecast. You analyzed the data to figure out where you went wrong and realized that your consumption of bread per month varied a lot. In some months you consumed 4 loaves and, in some other months, it was just 1. So, there is a good chance that this forecast will leave you with no bread for a few months and with excess bread for a few other months.

Then, you read this chapter and convert your forecast to a probabilistic forecast and now it tells you that 50% of the time your next month's consumption of bread is 2 loaves. But now, there is an additional feature in the app, which asks you whether you prefer to starve or have excess bread at the end of the day. So, depending on your appetite to starve or save money, you decide on an option. Let's say you don't want to starve, but are okay if bread runs out like 10% of the time. Once you enter this preference into the app, it revises its forecast and tells you that you should get 3 loaves of bread, and you never starve again (also because you smartened up and bought other stuff from the supermarket).

The ability to use the uncertainty in the forecast and revise it according to our appetite for risk is one of the main utilities for probabilistic forecasting. It also helps our forecast to be more transparent and trustworthy to the users.

Now, let's quickly look at the types of uncertainty one would have in a prediction problem using learned models.

Types of Predictive Uncertainty

We saw in *Chapter 5* that supervised machine learning is nothing but learning a function, $\hat{y} = h(X, \phi)$, where h , along with ϕ , is the model that we learn and is the input data. So, if we think about the sources of uncertainty, it can be from two of these components.

The model, h , we learned is an approximation using a dataset, X , which may or may not cover all the cases completely and some uncertainty can be introduced to the system from this. We call this **Epistemic Uncertainty**. In the context of machine learning, epistemic uncertainty can occur when the model has not been exposed to enough data, the model itself is insufficient to learn the complexity of the problem, or the data it has been trained on does not represent all possible scenarios. This is also known as systemic or reducible uncertainty because this is the portion of the total predictive uncertainty that can be actively reduced by having better models, better data, and so on; in other words, by gaining more knowledge about the system. Let's see a few examples to make the concept clearer:

- If a weather forecast model has less data from a particular region (maybe because of a faulty sensor), there is going to be less knowledge about that region, and this increases uncertainty.
- If a linear model was used for a non-linear problem, we would be introducing some uncertainty because of the simpler model having less knowledge about the system.
- If an economic forecasting model is not trained using a few key influencing factors like change in economic policies, climate change influencing the decisions, and so on, this also creates some uncertainty.

The good thing about this kind of uncertainty is that this is something that we can actively reduce by collecting better data, training better models, and so on.

Now there is another kind of uncertainty in the total predictive uncertainty—**Aleatoric Uncertainty**. This refers to the inherent randomness in the data which cannot be explained away. This is also known as statistical or irreducible uncertainty. Although our universe appears deterministic to us, there is an ever-prevalent layer of uncertainty just beneath the surface.

For example, the motion of the celestial objects can be calculated accurately (thanks to General Relativity and Einstein), but still, a random asteroid can hit any of the bodies and cause alterations to the calculated trajectory. This kind of irreducible and unavoidable uncertainty is referred to as aleatoric uncertainty. Let's see a few examples:

- Weather predictions, no matter how accurate the measurements and the models are, are still variable. There is an inherent randomness in weather that we might never be able to completely explain.
- The performance of an athlete, no matter how much they have trained and followed the rules, is still not completely deterministic. There are a lot of factors, like weather, health, and other random events during or before the game, which will affect the performance of the athlete.

Now that we understand the different types of uncertainty, and why we need uncertainty quantification, let's see what it means in the context of forecasting.

What are probabilistic forecasts and Prediction Intervals?

A probabilistic forecast is when the forecast, instead of having a single-point prediction, captures the uncertainty of that forecast as well. Probabilistic forecasting is a method of predicting future events or outcomes by providing a range of possible values along with associated probabilities or confidence levels. This approach captures the uncertainty in the prediction process.

In the econometrics and classical time series world, the prediction intervals were already baked into the formulation. The statistical grounding and strong assumptions of those methods made sure that the output of those models was readily interpreted in a probabilistic way as well (so long as you could satisfy the assumptions that were stipulated by those models). But in the modern machine learning/deep learning world, probabilistic forecasting is not an afterthought. A combination of factors, such as fewer rigid assumptions and the way we train the models, leads to this predicament.

There are different methods with which we can add the probabilistic dimension to our forecast, and we will go through a few of them in this chapter. But before that, let's also understand one of the most useful manifestations of probabilistic forecasts—**Prediction Intervals**.

A Prediction Interval is a range within which a future observation is expected to fall with a specified probability. For instance, if we have a 95% prediction interval for a time step of $[5, 8]$, we say that 95% of the time, the actual value will lie between 5 and 8. Let's take an example of a normal distribution with mean, μ_t , and variance, σ_t^2 , as the forecast at timestep, t (one of the techniques we will talk about gives us just that). So, the prediction interval at time, t , with a significance level (the probability with which the forecast can fall outside the interval), α , can be written as:

$$[\mu_t - z\sigma, \mu_t + z\sigma]$$

where z is the z-score of the normal distribution.

Prediction Interval (PI) vs Confidence Interval (CI)



One of the most confusing topics is prediction intervals and confidence intervals. Let's demystify them here. These are both ways we quantify uncertainty, but they serve different purposes and are interpreted differently in the context of forecasting. A confidence interval also provides a range, but for a population parameter (like the mean) of the sample data, whereas a prediction interval focuses on providing a range for a future observation. One of the key differences between the two is that CIs are, typically, narrower than PIs because PIs also account for the uncertainty of the new point. On the other hand, CIs only account for the uncertainty accounted to the model parameters. So, we use PIs when we need to give a range for when a future observation is likely to fall, like the next month's sales. CIs are used when we need to provide a range for an estimated parameter, like the estimated average demand over a year.

There are a few terms and concepts we need to clarify before we begin the discussion in detail.

Confidence levels, error rates, and quantiles

When working with prediction intervals, it's crucial to understand the relationship between confidence levels, error rates, and quantiles. These concepts help in determining the range within which future observations are expected to fall with a certain probability.

Error Rate (α) is the allowable probability that the prediction interval will not contain the future observation. It is typically expressed in percentages or as a decimal between 0 and 1. If we say $\alpha = 10\%$ or $\alpha = 0.1$, it means that there is a 10% chance that the future observation will not be in the prediction interval.

Confidence Level (CL) is the complement of error rate and is the probability that the prediction interval contains the future observation. $CL = 1 - \alpha$. If we say that the error rate is 10%, the confidence level would be 90%.

Quantiles are points that divide the data into intervals with equal probabilities. In simpler terms, a quantile shows the value below which a certain percentage of data falls. For instance, the 5th percentile or 0.05th quantile marks the point where 5% of data lies below it. Therefore, we can use quantiles as well to define the prediction intervals where we don't have an analytical way to get the prediction intervals based on distributional assumption.

In *Figure 17.1*, we show the prediction intervals for a standard normal distribution.

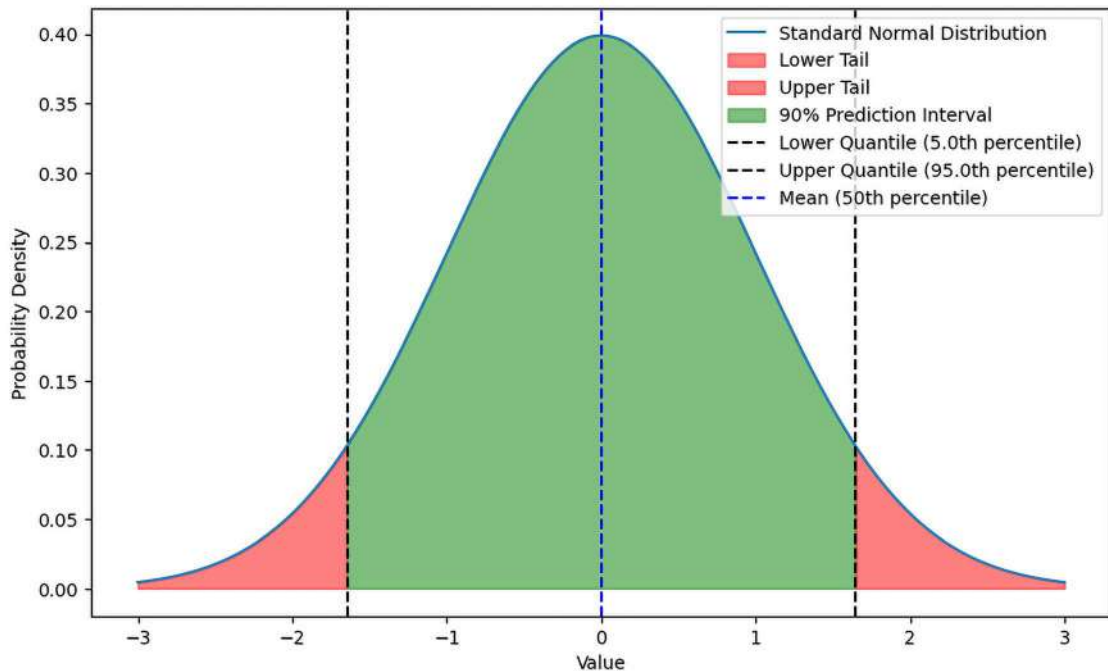


Figure 17.1: Prediction intervals for a standard normal distribution

There is a strong link between error rates, confidence levels, and quantiles. Error rates and confidence levels are direct complements to each other and can be used interchangeably to define the confidence that we want our prediction intervals to have or the error rate we are okay with from the prediction intervals. Another way to look at it is by the area under the curve. In *Figure 17.1*, the area of the green-shaded region denotes the confidence level, and the area of the red area denotes the error rate.

For a standard normal distribution, we can directly get the prediction intervals by using the analytical formula:

$$[\mu - z_{\alpha/2} \cdot \sigma, \mu + z_{\alpha/2} \cdot \sigma]$$

where μ is the mean of the distribution, σ is the standard deviation of the distribution, and $z_{\alpha/2}$ is the critical value from the standard normal distribution corresponding to the desired confidence level, $1 - \alpha$. $\alpha/2$ is taken because we are allowing for the error rate to be spread on both sides (red shaded area on both sides of the curve in *Figure 17.1*).

Now let's look at how error rates and confidence levels are linked with quantiles because if we don't know what the distribution is (and we don't want to assume any distribution), we can't go by the analytical formula to get prediction intervals. In such cases, we can use quantiles to get the same. Just like we did with the analytical formula, the error rate, α , should be equally divided across both sides. And therefore, the prediction interval would be:

$$[q_{\alpha/2}, q_{1-(\alpha/2)}]$$

where q_t is the t^{th} quantile. So, from the definition of quantile, we know $q_{\alpha/2}$ would have $\alpha/2$ % of data below it and $q_{1-(\alpha/2)}$ would have $\alpha/2$ % of data above it, thus making the area outside the intervals to be α .

Using this relation, we can go from error rates to quantiles or confidence levels to quantiles. If the error rate is α we have already seen what the corresponding quantiles denoting the prediction interval are. Let's also see one more quick formula to go from confidence levels (declared as percentages) to quantiles:

$$[q_{50-(CL/2)}, q_{50+(CL/2)}]$$

In Python code, this is simply:

```
level = 95 # Confidence Levels
qs = [50 - level / 2, 50 + level / 2] # Quantiles
```

Now, let's look at how to measure the goodness of prediction intervals.

Measuring the goodness of prediction intervals

We know what a probabilistic forecast is and what prediction intervals are. But before we look at techniques to generate prediction intervals, we need a way to measure the goodness of such an interval. Standard metrics like Mean Absolute Error or Mean Squared Error hold no more because they are point forecast measuring metrics.

What do we want from a prediction interval? If we have a prediction interval with 90% confidence, we would expect the data points to lie between the interval at least 90% of the time. This can easily be obtained by having very wide prediction intervals, but that again becomes a useless prediction interval. So, we want our prediction interval to be as narrow as possible and still have the 90% confidence criteria respected. To measure these two distinct aspects, we can use two metrics—**Coverage** and **Average Length of Prediction Intervals**.

Coverage is the proportion of true values that fall within the prediction intervals. Mathematically, if we denote the Prediction Interval by $[L_i, U_i]$ for each observation i , and true value by y_i , coverage can be defined as:

$$\text{Coverage} = \frac{1}{N} \sum_{i=1}^N \mathbb{I}(L_i \leq y_i \leq U_i)$$

where $\mathbb{I}$ is an indicator function that equals 1 if the condition inside is true and 0 otherwise, and N is the total number of observations. A coverage metric close to the desired confidence level (e.g., 95% for a 95% prediction interval) indicates that the model's uncertainty estimates are well-calibrated.

Average Length of Prediction Intervals is calculated by averaging the lengths of the prediction intervals across all observations. Using the same notations as above, it can be mathematically written as:

$$\text{Average Length} = \frac{1}{N} \sum_{i=1}^N (U_i - L_i)$$

This metric helps in understanding the trade-off between the coverage of the intervals and their precision. Let's also look at Python functions for both of these metrics:

Coverage:

```
import numpy as np

def coverage(y_true, lower_bounds, upper_bounds):
    """
    Calculate the coverage of prediction intervals.

    Parameters:
    y_true (array-like): True values.
    lower_bounds (array-like): Lower bounds of prediction intervals.
    upper_bounds (array-like): Upper bounds of prediction intervals.

    Returns:
    float: Coverage metric.
    """
    y_true = np.array(y_true)
    lower_bounds = np.array(lower_bounds)
```

```

upper_bounds = np.array(upper_bounds)
# Check if true values fall within the prediction intervals
coverage = np.mean((y_true >= lower_bounds) & (y_true <= upper_bounds))
return coverage

```

Average Length:

```

def average_length(lower_bounds, upper_bounds):
    """
    Calculate the average length of prediction intervals.

    Parameters:
    lower_bounds (array-like): Lower bounds of prediction intervals.
    upper_bounds (array-like): Upper bounds of prediction intervals.

    Returns:
    float: Average length of prediction intervals.
    """
    lower_bounds = np.array(lower_bounds)
    upper_bounds = np.array(upper_bounds)

    # Calculate the length of each prediction interval
    lengths = upper_bounds - lower_bounds
    # Calculate the average length
    average_length = np.mean(lengths)
    return average_length

```

Both of these Python functions can be found in `src/Utils/ts_utils.py` and we will be using them to measure the quality of prediction intervals generated in the chapter.

Now, let's look at different techniques we can use to get probabilistic forecasts and how we can use them practically.

Probability Density Function (PDF)

This is one of the most common techniques for probabilistic forecasting, especially in the deep learning space, because of the ease of implementation. The forecast at time t , $\hat{y}_t$, can be seen as the realization of a probability distribution, $p(\hat{y}_t)$. And instead of estimating $\hat{y}_t$, we can estimate $p(\hat{y}_t)$. If we assume $p(\hat{y}_t)$ is one of the parametrized distributions, $\mathcal{D}(\theta_t)$, with parameters θ_t , then we can estimate the parameters θ_t , instead of $\hat{y}_t$, directly.

For instance, if we assume the forecast is drawn from normal distribution, we can model

$$\hat{y}_t \sim \mathcal{N}(\mu_t, \sigma_t)$$

So, instead of getting our model to output $\hat{y}_t$, we can get it to output μ_t and σ_t . And with μ_t and σ_t , we can easily calculate the prediction intervals at the given α :

$$\text{Lower Bound} = \mu - Z \cdot \sigma$$

$$\text{Upper Bound} = \mu + Z \cdot \sigma$$

where Z is the critical value from the standard normal distribution corresponding to the desired confidence level. For a 90% confidence level, $Z \approx 1.645$. Simple enough, right? Not so fast!

Now that we are modeling the parameters of a distribution, how do we train the model? We still have the actual point forecast as the target. In the normal distribution case, the targets are still actual y_t and not the means and standard deviations. We get over this problem by using a loss function like **log likelihood**, instead of losses like Squared Error.

For instance, let's say we have a set of *i.i.d* observations (in our case, the target), $y_1, y_2, \dots, y_n$. With the predicted parameters of the assumed distribution ($\mathcal{D}(\theta)$), we will be able to calculate the probability of each of the target, $p(y_t)$. The *i.i.d* assumption means each sample is independent of each other. And high-school mathematics tells us that when two independent events happen, we can calculate their joint probability by multiplying the two independent probabilities together. Using the same logic, we can calculate the joint probability or likelihood of all n *i.i.d* observations (probability that all these events occur) by just multiplying all the individual probabilities together.

$$\text{Likelihood} = \prod_{i=1}^n p(y_i)$$

Maximizing the likelihood helps the model learn the right parameters for each sample such that the probability under the assumed distribution maximizes. We can intuitively think about this as follows. For an assumed distribution like normal distribution, maximizing the likelihood makes sure that the target falls in the center of the distribution defined by the predicted parameters for each sample.

However, this operation is not numerically stable. Since probabilities are $0 < p < 1$, multiplying them together makes the result progressively smaller and can soon lead to numerical underflow issues. Therefore, we use the log likelihood, which is nothing but the likelihood but log transformed. We do it because:

- Being a strictly monotonic transformation, optimizing a function is equivalent to optimizing the log transform of the function. Therefore, optimizing likelihood and log likelihood is the same thing.
- Log transformation converts the multiplication into an addition, which is a much more numerically stable operation.

$$\text{Log Likelihood} = \sum_{i=1}^n \log(p(y_t))$$

The major disadvantage of this approach is that this relies on parametrized probability distributions whose log likelihood computation is tractable. Therefore, we are forced to make assumptions about the output and pick a distribution that might fit ahead of time.

This is a double-edged sword. On one hand, we can inject some domain knowledge into the problem and regularize the model training, but on the other hand, if we aren't clear on whether choosing a normal distribution is the right choice, it can lead to an unnecessarily constrained model.

Many popular distributions, such as Normal, Poisson, Negative Binomial, Exponential, LogNormal, Tweedie, and so on, can be used for generating probabilistic forecasts with this technique.

Now, we have all the components for training and learning a model and can make it predict a full probabilistic distribution instead of a point forecast. With all that theory set, let's switch gears and see how we can use this technique.

Forecasting with PDF—machine learning models

We have seen how to use standard machine learning models for forecasting in *Part 2, Machine Learning for Time Series*. But all of that was for point forecast. Can we easily convert all of that into probabilistic forecasts using the PDF approach? In theory, yes. But practically, it's not that easy. All the popular implementations of machine learning models like `sci-kit learn`, `xgboost`, `lightgbm`, and so on take a point prediction paradigm. And as users of such open-source libraries, it isn't easy for us to tweak and re-write the code to make it optimize the log likelihood as a regression loss. But fear not, it is not impossible. NGBoost is a distant cousin of the very popular gradient boosting models, like `xgboost` and `lightgbm`, and it is implemented such that it predicts the PDF, instead of the point prediction.



There are other techniques like Quantile Forecast or Conformal Prediction, which are more widely applicable (and recommended) to the machine learning models we discussed in the book if the end goal is to have a prediction interval. NGBoost is discussed for completeness and for the cases where a full probability distribution is needed as an output.

We aren't going too deep into what NGBoost is and how it differs from the regular gradient boosting models here, but just know that it is a model that predicts probability distributions instead of point predictions.



Reference check:

The research paper by Duan et.al., which proposed NGBoost, is cited in *References* under reference 1.

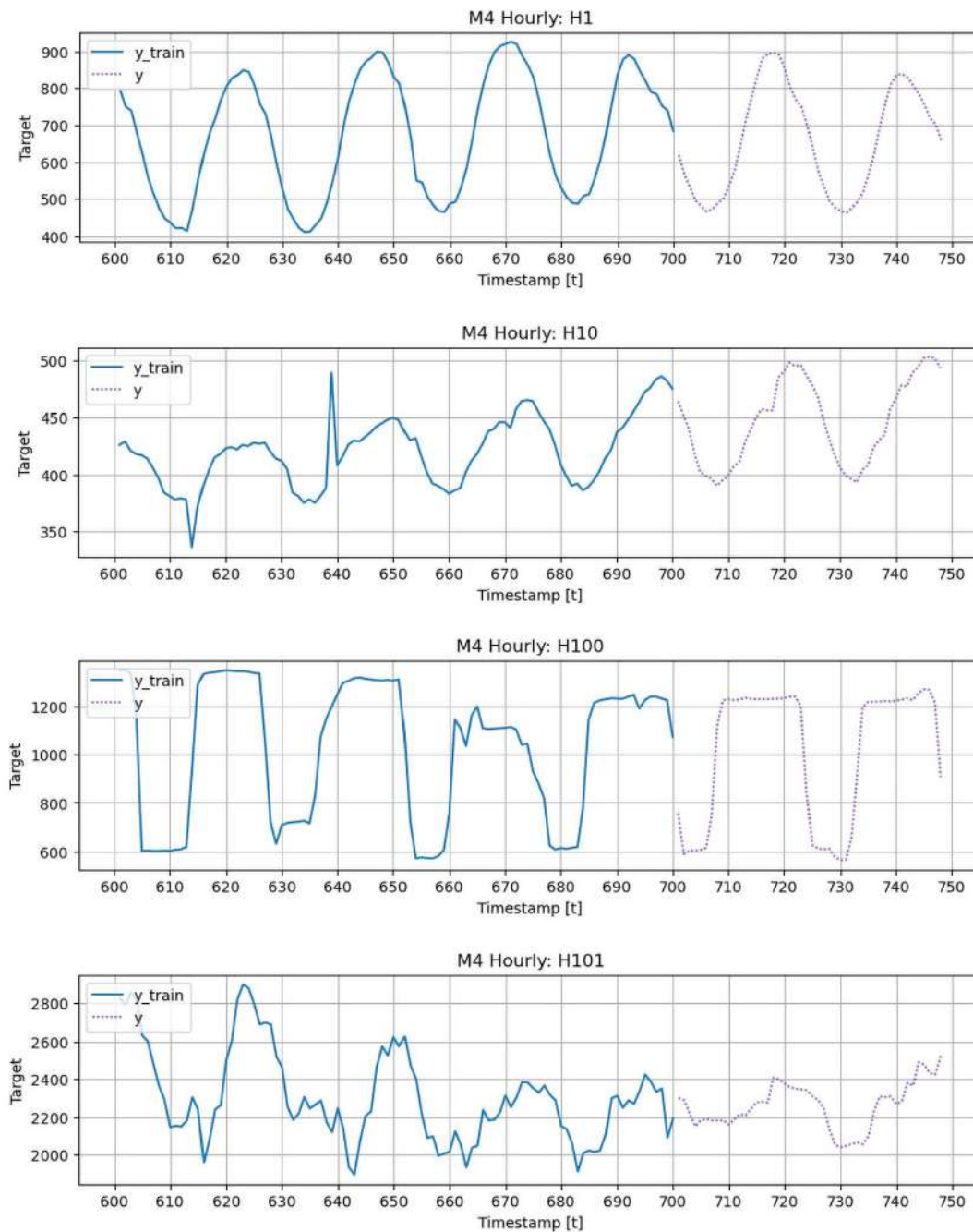
Further reading has a link to a blog about NGBoost, which goes into a bit more depth on what the model is.



Notebook alert:

To follow along with the complete code, use the notebook named `01-NGBoost_prediction_intervals.ipynb` in the `Chapter17` folder.

Let's use a sample of eight time series from the M4 competition (which has 100,000 time series, references 5) for the probabilistic forecasts. The data is easily available online and the download script is included in the notebook. We are using this simpler dataset than the one we have been working on because I want to avoid complicating the narrative with exogenous variables and such. Here is the plot of the last 100 time steps of the eight sampled time series.



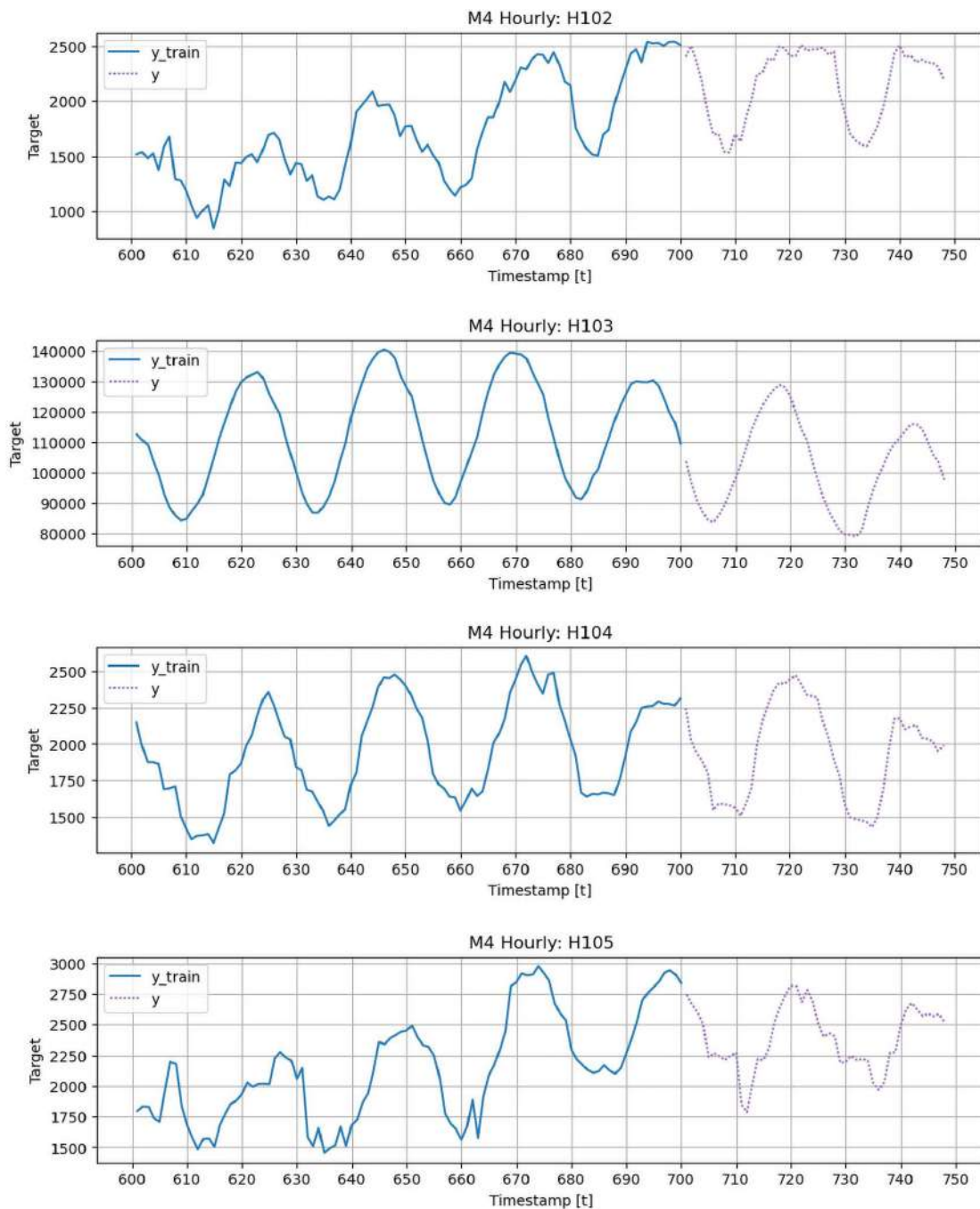


Figure 17.2: Last 100 timesteps of 8 sampled time series from the M4 Competition. Test period is drawn in a dotted purple line.

Let's use `mlforecast` to create some features quickly in order to convert this into a regression problem. We had a bonus notebook back in *Chapter 6*, which showed how to use `mlforecast` as an alternative for the feature engineering that is included in the book's repository. For the detailed code, refer to the full notebook, but for now, let's assume that we have a dataframe called `data`, which has all the features necessary to run a machine learning model. We have split it into `train` and `val` and then subsequently to `X_train`, `y_train`, `X_val`, and `y_val`. Now, let's see how we can train a model, assuming the output is a Normal distribution.

```
from ngboost import NGBRegressor
from ngboost.distsns import Normal
# Training the model
ngb = NGBRegressor(Dist=Normal).fit(X_train, Y_train)
```

NGBoost doesn't have a lot of parameters to tune and therefore isn't as flexible as other **gradient-boosting decision trees (GBDT)**. And it's also not as fast as the other GBDTs. This is a model that you use only for special use cases when you need probabilistic outputs.

These are a few parameters that NGBoost has:

- **Dist:** This is the assumed distributional form of the output. The package currently supports Normal, LogNormal, and Exponential—all of which can be imported from `ngboost.distsns`
- **Score:** This is any valid scoring function that is used to compare the predicted distributions to observations. The log likelihood score that we discussed earlier is called `LogScore` in the package and is the default value. All scoring functions can be imported from `ngboost.scores`.
- **n_estimators:** This is the number of estimators that are used in the boosted trees.
- **learning_rate:** This is the learning rate used for combining the boosted trees.
- **mini_batch_frac:** The percent of rows which is sub-sampled for each iteration. This is set to 1.0 as the default value.

Now that we have a trained NGBoost model, let's see how we can use it to generate predictions and prediction intervals.

To get point predictions, the syntax is exactly the same as the sci-kit learn API.

```
y_pred = ngb.predict(X_val)
```

This is nothing but a wrapper method that calculates the location parameter of the assumed distribution. For instance, for the Normal distribution, the mean of the predicted distribution is the point prediction.

Now to get the underlying probabilistic prediction and subsequently the prediction intervals, we need to use a different method:

```
y_pred_dists = ngb.pred_dist(X_val)
```

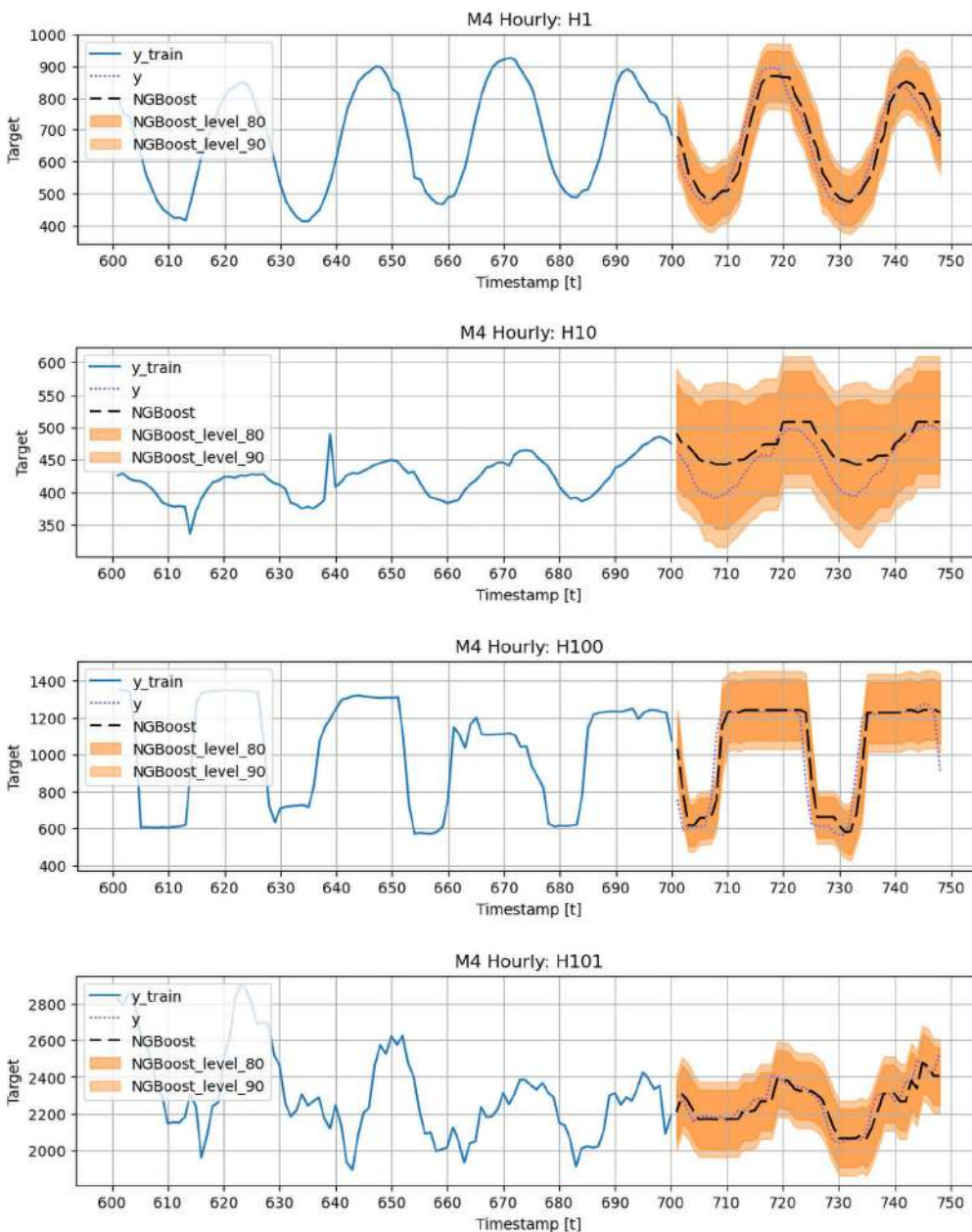
Each point in `y_pred_dists` is a complete distribution. If we want to instead the first five predicted points, we can do the following:

```
y_pred_dists[0:5].params
```

Now, to get the prediction interval, we can use `y_pred_dist` and call a method, giving the level of confidence we expect. This, in turn, calls the `scipy` distribution (like `scipy.stats.norm`), which has a method, `interval`, to get the intervals given the confidence level.

```
y_pred_lower, y_pred_upper = y_pred_dists.dist.interval(0.95)
```

Now, we have a window around `y_pred` wide enough to envelope the uncertainty expected at each data point—the prediction interval. Let's look at the forecast and the metrics.



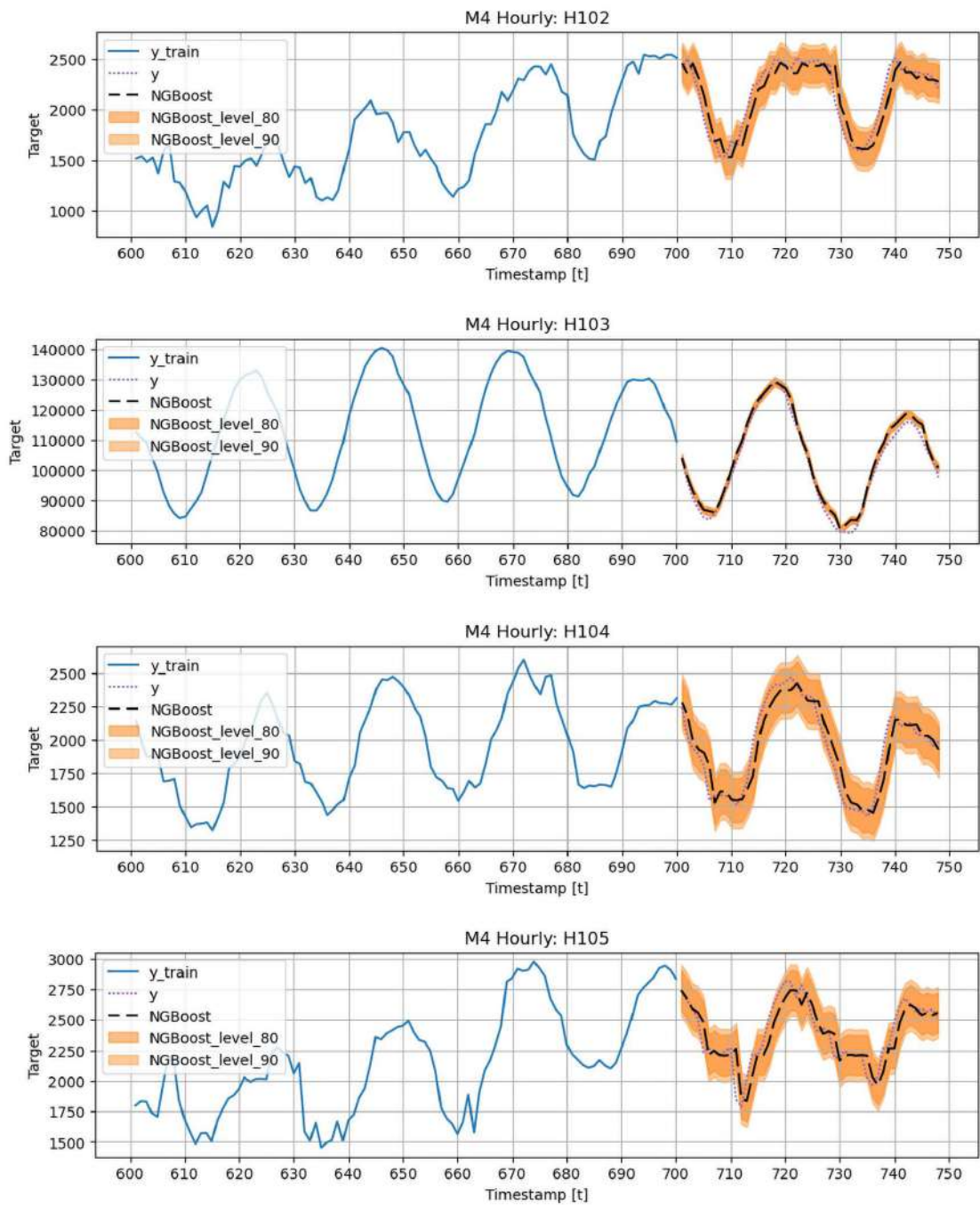


Figure 17.3: Forecast with prediction intervals from NGBoost

And below are the metrics we calculated for these eight time series (Mean Absolute Error, Coverage, and Average Length):

unique_id	NGBoost-mae	NGBoost-coverage-80	NGBoost-average_length-80	NGBoost-coverage-90	NGBoost-average_length-90
H1	33.446126	1.000000	167.524616	1.000000	215.015517
H10	26.330457	1.000000	166.351375	1.000000	213.509679
H100	67.937080	0.833333	302.439255	0.833333	388.176581
H101	37.364603	1.000000	318.001374	1.000000	408.150345
H102	102.209607	0.750000	328.523550	0.895833	421.655412
H103	1932.872853	0.291667	2326.816546	0.395833	2986.436705
H104	80.912530	0.875000	329.550064	0.916667	422.972928
H105	85.117678	0.812500	329.116238	0.937500	422.416119

Figure 17.4: Metrics for NGBoost

Although for some time series, the intervals seem to be good, some others (like time series H103) seem to have way too narrow a prediction interval, which is also evident in the low coverage as well.

Forecasting with PDF—deep learning models

Unlike machine learning models, it's very easy to convert all the deep learning models we have learned about in the book to their PDF version. Remember the discussion we had before we started the practical application? The major changes we needed to do were these:

- Instead of predicting point forecast (single number), we predict the parameters of a probability distribution (one or more numbers).
- Instead of using a point loss like Mean Squared Error, use a probabilistic scoring function like log likelihood.

In the deep learning paradigm, these are very simple changes to make, aren't they? In almost all the deep learning models we have learned to use in the book, there is a linear projection at the end, which projects the output into the required dimensions. Conceptually, it's simple enough to make these output multiple numbers (parameters of the assumed distribution) by changing the linear projection. Similarly, it's simple enough to change the loss function as well. Notably, *DeepAR* (Reference 3) is a well-known deep learning model that uses this technique for probabilistic forecasting.



Notebook alert:

To follow along with the complete code, use the notebook named `02-NeuralForecast_prediction_intervals_PDF.ipynb` in the `Chapter17` folder.

Let's see how we can do this in `neuralforecast` (the library we were using *Chapter 16*). In our example here, we will take a simple model like an LSTM, but we can do the same with any model because all we are doing is switching the loss function to `DistributionLoss`.

And, for this example, we are going to use the M4 competition dataset, which is freely available (the code to download the dataset is included in the notebook).

Let's start from the point where we have the data formatted the way `neuralforecast` expects in `Y_train_df` and `Y_test_df`. The first thing we need to do is import the necessary classes.

```
from neuralforecast import NeuralForecast
from neuralforecast.models import LSTM
from neuralforecast.losses.pytorch import DistributionLoss
```

The only class that we haven't looked at before in the book is `DistributionLoss`. This is a class that wraps `torch.distribution` classes and implements the negative log likelihood loss we discussed earlier. At the time of writing this book, the `DistributionLoss` class supports these underlying distributions:

- Poisson
- Normal
- StudentT
- NegativeBinomial
- Tweedie
- Bernoulli (Temporal Classifiers)
- ISQF (Incremental Spline Quantile Function)

The choice between these different distributions is totally up to the modeler and is a key assumption in the model. If the output we are modeling is expected to be in a normal distribution, then we can choose `Normal`. These are the major parameters of `DistributionLoss`:

- `distribution`: This is a string that identifies which distribution we are assuming. It can be any one from the list we saw before.
- `level`: This is a list of floats that defines the different confidence levels we are interested in modeling. For instance, if we want to model 80% and 90% confidence, we should give the values as `[80, 90]`.
- `quantiles`: This is an alternate way of defining the level. Instead of 95% confidence, you can define them in quantiles $\rightarrow [0.1, 0.9]$.

Practitioner's tip:



Making a distributional assumption requires a deep study of the domain and data. But if you are not completely familiar with these distributions, `Normal` or `StudentT` is a good starting point as a lot of data resembles normal distribution. But before you jump into using `Normal` distributions everywhere, you should do a bit of literature study in the domain of the problem and choose the distribution accordingly. For instance, intermittent demand or sparse demand, which is very common in retail, is better modeled using a `Poisson` distribution. If the forecast is a count data (positive integers), `Negative Binomial` is a good option.

Now let's set a horizon, the levels we need, and a few hyperparameters for LSTM (we've chosen some small and simple hyperparameters to make the training faster. In real-world problems, it is advisable to do a hyperparameter search to find the best parameters).

```
horizon = 48
levels = [80, 90]
lstm_config = dict(input_size=3*horizon, encoder_hidden_size=8, decoder_hidden_size=8)
```

Now we need to define the models we are going to use and the NeuralForecast class. Let's define two models—one using Normal and another using StudentT.

```
models = [
    LSTM(
        h=horizon,
        loss=DistributionLoss(distribution="StudentT", level=levels),
        alias="LSTM_StudentT",
        **lstm_config
    ),
    LSTM(
        h=horizon,
        loss=DistributionLoss(distribution="Normal", level=levels),
        alias="LSTM_Normal",
        **lstm_config
    ),
]
# Setting freq=1 because the ds column is not date, but instead a sequentially
# increasing number
nf = NeuralForecast(models=models, freq=1)
```

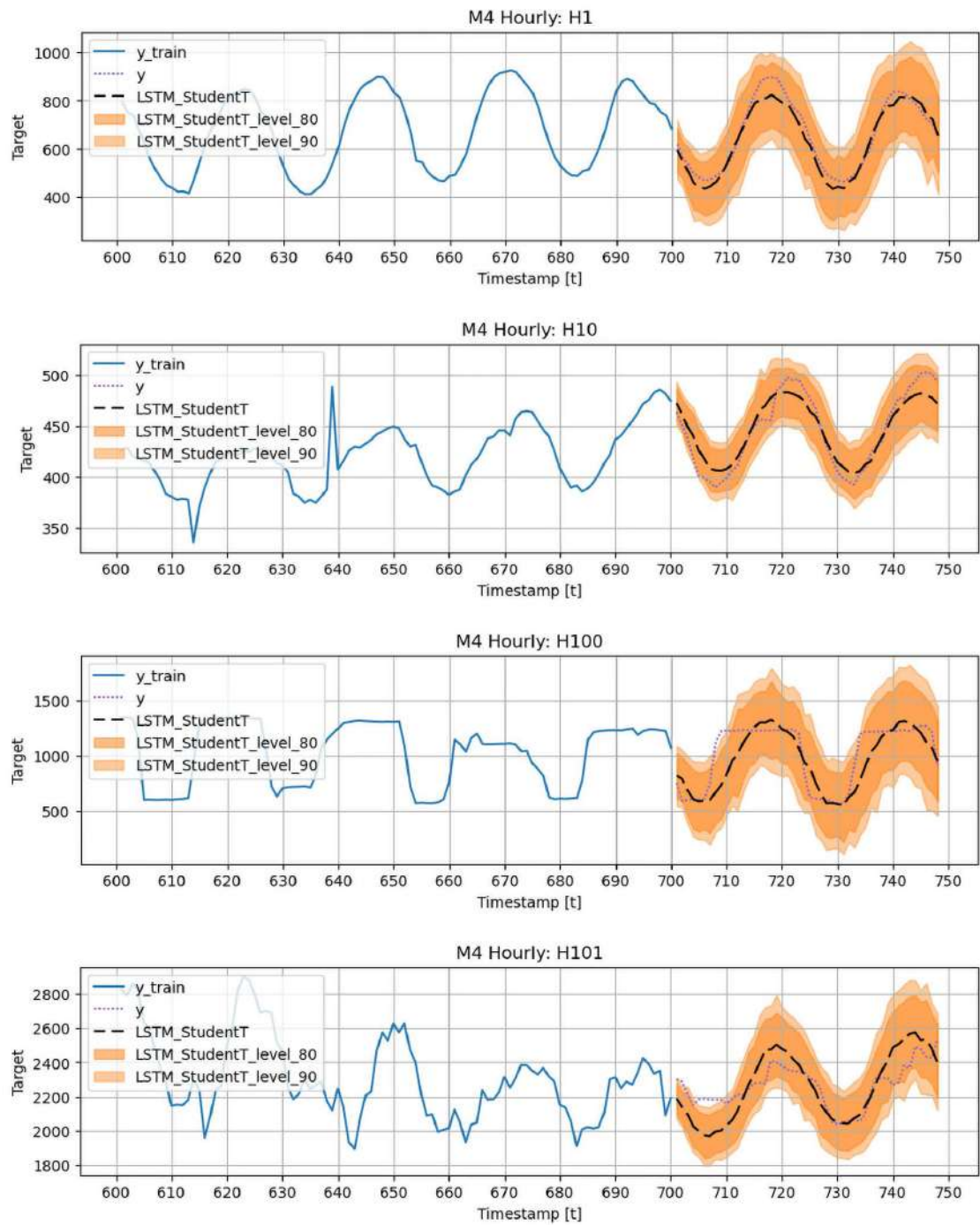
Notice how the syntax is exactly the same as for point forecast, except for the Distribution Loss we chose. Now all that's left is to train the models.

```
nf.fit(df=Y_train_df)
```

Once the model is trained, we can predict using the predict method. This output will have the point forecast under the alias we have defined and the high and low intervals for all the levels we have defined.

```
Y_hat_df = nf.predict()
```

Now that we have the probabilistic forecasts, let's take a look at them and also calculate the metrics.



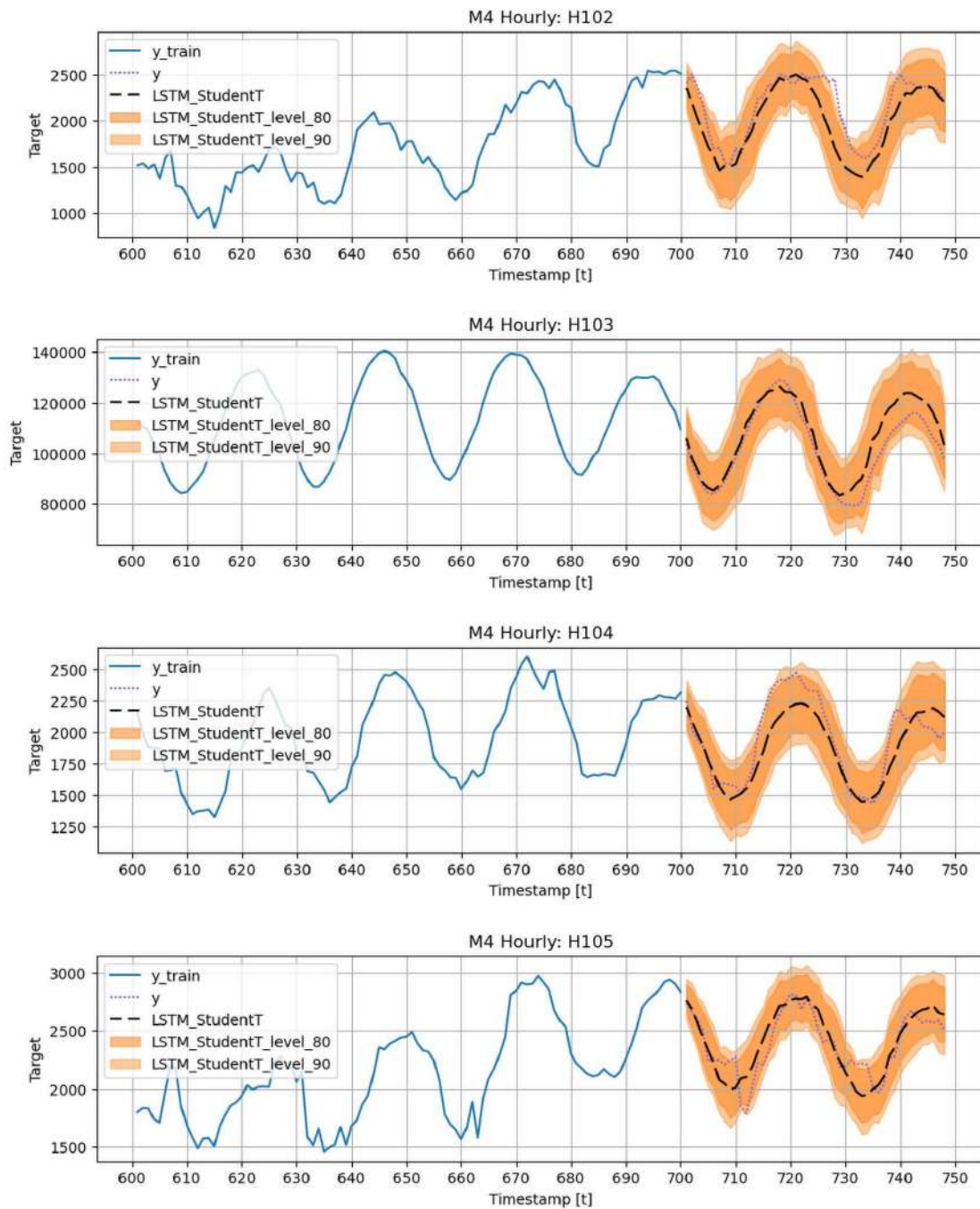


Figure 17.5: Forecast with prediction intervals from LSTM with StudentT distribution as output

unique_id	LSTM_StudentT-mae	LSTM_Normal-mae	LSTM_Normal-coverage-80	LSTM_Normal-average length-80	LSTM_StudentT-coverage-80	LSTM_StudentT-average length-80	LSTM_Normal-coverage-90	LSTM_Normal-average length-90	LSTM_StudentT-coverage-90	LSTM_StudentT-average length-90
H1	30.236069	25.624772	1.000000	283.496979	1.000000	277.002154	1.000000	364.234564	1.000000	379.727213
H10	10.624161	10.421406	0.979167	51.680541	0.958333	49.274955	1.000000	66.303233	1.000000	67.187339
H100	117.840120	121.935446	0.875000	650.567130	0.895833	641.255395	0.979167	830.685015	0.937500	879.152472
H101	92.860057	90.329717	0.958333	398.062027	0.750000	340.822159	0.958333	512.908348	0.875000	464.493647
H102	169.452031	173.042023	0.833333	633.416743	0.833333	619.196047	0.916667	813.279742	0.895833	847.938540
H103	4646.264323	4322.925293	1.000000	25270.319499	1.000000	24907.359212	1.000000	32457.823730	1.000000	33950.600911
H104	122.986824	119.372470	0.916667	510.660988	0.854167	493.365873	1.000000	655.989634	0.979167	666.488528
H105	110.559415	113.744522	0.854167	471.914998	0.854167	449.105464	0.979167	603.949382	1.000000	613.617622

Figure 17.6: Metrics for the LSTM with Normal and StudentT distribution outputs

If we compare the coverages with the ones with NGBoost, we can see that the deep learning approach increased the coverage, but also has wider than necessary intervals in most cases (as evidenced by larger average widths).

The biggest disadvantage of this method is that we restrict the output to one of the parametrized distributions. In many real-world cases, the data might not conform to any parametric distributions.

Now, let's see a method that doesn't require the assumption of any parametric distribution, but still gets the prediction intervals.

Quantile function

If our only aim is to get prediction intervals, we can also do the same using quantiles. Let's look at the PDF method in a slightly different light. In the PDF method, we have a full probability distribution as the output at each timestep and we use that distribution to get the quantiles, which are the prediction intervals. Although for most parametric distributions, there are analytical formulae to get the quantiles, we can also find the quantiles numerically. We just draw N samples, where N is sufficiently large, and then calculate the quantiles of the drawn samples. The point is that even though we have a full probability distribution, for prediction intervals, all we need are the quantiles. And calculating quantiles given N samples is not dependent on the kind of distribution.

So, what if we can train our models to predict the specified quantiles directly, having no assumption on the underlying probability distribution? This is exactly what we do with quantile functions.

Before talking about quantile functions, let's spend a minute on the **Cumulative Distribution Function (CDF)**. This, again, is high school probability. In simple words, a CDF returns the probability of some random variable, X being less than or equal to some value, x :

$$F(x) = P(X \leq x)$$

where F is the CDF. This function takes in an input, x , and returns a value between 0 and 1. Let's call this value p .

A *quantile function* is an inverse of CDF. This function tells you what value of x would make $F(x)$ return a particular value, p .

$$F^{-1}(p) = x$$

This function, F^{-1} , is the *quantile function*.

From the implementation perspective, for models which are capable of multi-output prediction (like the deep learning models), we can use a single model and predict all the quantiles we want to by changing the output layer. And for models that are restricted to a single output (like machine learning models), we can learn separate quantile models for each quantile.

Now, just like before, we can't use point losses like the mean squared error. We got over this problem in PDFs by using the log likelihood function. And here, we can use quantile loss or pinball loss.



Reference check:

The paper that proposed quantile loss and regression is cited in the *References* under reference 4.

The quantile loss can be defined as below:

$$L_q(y_t, \hat{y}_t^q) = \begin{cases} (y_t - \hat{y}_t^q) \times q, & \text{if } y_t \geq \hat{y}_t^q \\ (y_t^q - \hat{y}_t) \times (1 - q), & \text{if } y_t < \hat{y}_t^q \end{cases}$$

where y_t is the target value at time t , $\hat{y}_t^q$ is the quantile forecast, and q is the quantile we are forecasting. The formula looks daunting, but bear with me for a second; it's quite simple. Let's try and get some intuitions about the loss. We know the median is the 0.5 quantile, and that is a measure of central tendency. But if we want our predictions to approximate the 75th percentile or 0.75 quantile, then we would have to urge the model to overestimate, right? And if we want the model to overestimate, we need to penalize the model more if it's underestimating. And vice versa, if we want to predict the 0.25 quantile, we need to underestimate. Quantile loss does exactly that:

- For $y_t \geq \hat{y}_t^q$ (under estimation), the loss is $q \times (y_t - \hat{y}_t^q)$
- For $y_t < \hat{y}_t^q$ (over estimation), the loss is $(1 - q) \times (\hat{y}_t^q - y_t)$

The asymmetry is derived from the term q or $1 - q$. The other term is just the difference between the actual and predicted values.

Let's try to understand this with an example. Suppose we have the true value $y_t = 100$, and we want to estimate 0.75 quantile ($q = 0.75$).

- **Case 1: Overestimation:** $\hat{y}_t^q = 110$. Since $y_t < \hat{y}_t^q$, our quantile loss would be:

$$L_{q(100,110)} = (1 - 0.75) \times (110 - 100) = 2.5$$

- **Case 2: Underestimation:** $\hat{y}_t^q = 90$. Since $y_t \geq \hat{y}_t^q$, our quantile loss would be:

$$L_{q(100,110)} = 0.75 \times (110 - 100) = 7.5$$

**Notebook alert:**

To follow along with the complete code, use the notebook named 03-Understanding_Quantile_Loss.ipynb in the Chapter17 folder.

Therefore, by varying the value of q , we can make the loss more or less asymmetric and toward either side. Figure 17.7 below shows the loss curves for $q = 0.5$ and $q = 0.75$ with these example predictions marked on it.

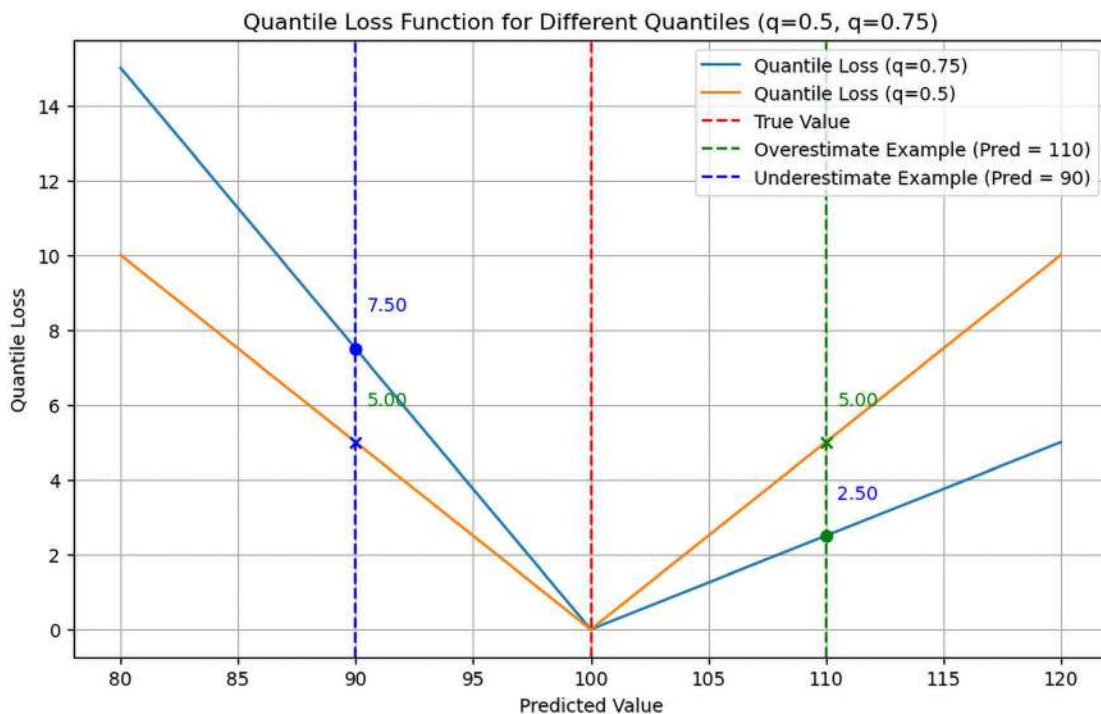


Figure 17.7: Quantile loss curves for $q=0.5$ and $q=0.75$

We can see that the quantile loss for $q = 0.5$ is symmetric as it represents the median, whereas the loss curve for $q = 0.75$ is asymmetric, penalizing underestimation a lot more than overestimation.

Although the formula has a branching structure, a maximum operation can be easily avoided when implementing it in code. The quantile loss in Python looks like this:

```
def quantile_loss(q, y, y_hat_q):
    """ Calculate the quantile loss for a given quantile.

    Args:
    q (float): The quantile to be evaluated, e.g., 0.5 for median.
    y (float): The target value.
    y_hat_q (float): The quantile forecast.
    """
    error = y - y_hat_q
    return np.maximum(q * error, (q - 1) * error)
```

Now, let's get into the practical side of things and learn how to use quantile loss with the different techniques (both machine learning and deep learning) we have covered in this book.

Forecasting with quantile loss (machine learning)

Regular machine learning models are typically capable of modeling one output. Therefore, we will need to train a different model for each of the quantiles we are interested in using quantile loss. So, if we want to predict the 0.5, 0.05, and 0.95 quantiles for a problem, we will have to train three separate models, one for each quantile.



Notebook alert:

To follow along with the complete code, use the notebook named `04-LightGBM_Prediction_Interval_Quantile_Loss.ipynb` in the `Chapter17` folder.

Let's see how we can do that. Just like in the PDF section, we are using `mlforecast` to quickly whip up a synthetic problem and create some features. For the detailed code, refer to the full notebook, but for now, let's assume that we have a dataframe `data`, which has all the features necessary to run a machine learning model. We have split it into `train` and `val` and then subsequently into `X_train`, `y_train`, `X_val`, and `y_val`.

The first step is to import the **LGBMRegressor** and set some parameters and quantiles we want to train.

```
params = {
    'objective': 'quantile',
    'metric': 'quantile',
    'max_depth': 4,
    'num_leaves': 15,
    'learning_rate': 0.1,
    'n_estimators': 100,
    'boosting_type': 'gbdt'
}
# converting levels to quantiles
# For 90% Confidence - 0.05 for lower, 0.5 for median, and 0.95 for upper
quantiles = [0.5] + sum([level_to_quantiles(l) for l in levels], [])
```

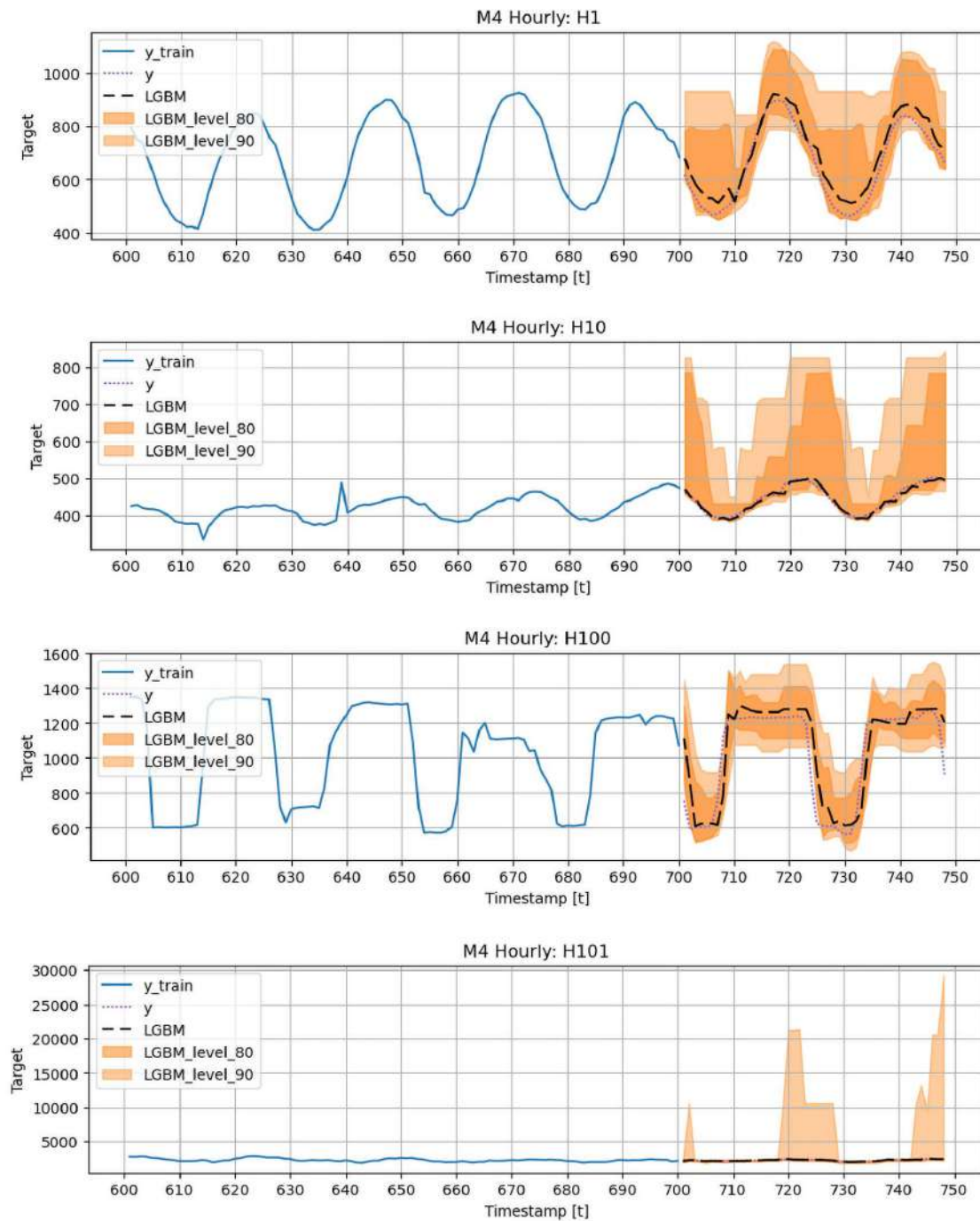
The key parameter to note here is that the objective is set as quantile and the metric is also set as quantile. The rest of the LightGBM parameters can be tuned and tweaked for each use case. Now, let's train all the quantile models.

```
# Training a model each for the quantiles
quantile_models = {}
for q in quantiles:
    model = LGBMRegressor(alpha=q, **params)
    model = model.fit(X_train, Y_train)
    quantile_models[q] = model
```

Now that the model is trained, we can get the point prediction and prediction intervals from the quantile models.

```
# Point Forecast using the 0.5 quantile model
y_pred = quantile_models[0.5].predict(X_val)
# Prediction Intervals using the 0.1 and 0.9 quantile models
y_pred_lower = quantile_models[0.1].predict(X_val)
y_pred_upper = quantile_models[0.9].predict(X_val)
```

Now, let's see what the forecast and metrics look like.



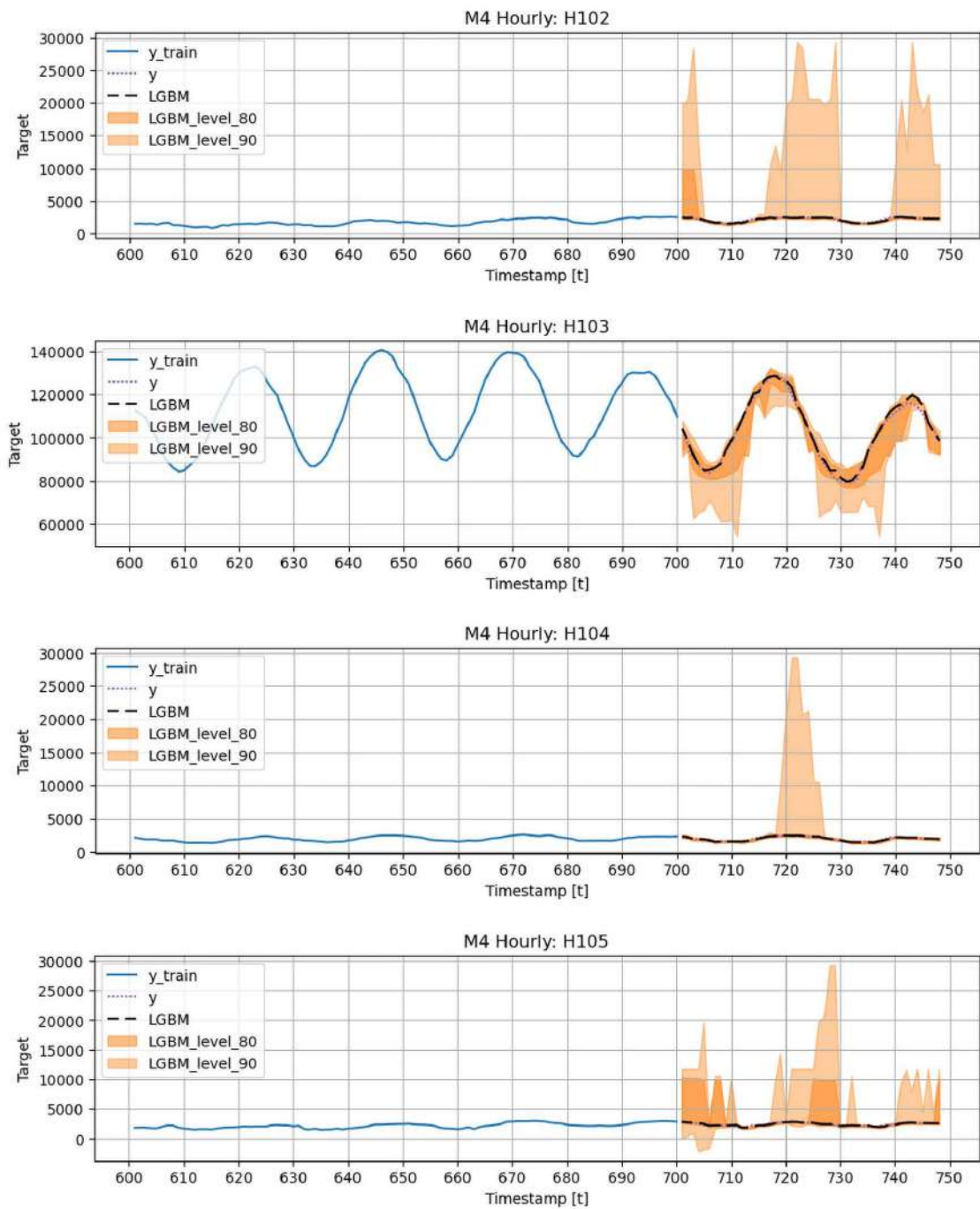


Figure 17.8: Forecast with prediction intervals using Quantile Regression with LightGBM

unique_id	LGBM-mae	LGBM-coverage-80	LGBM-average_length-80	LGBM-coverage-90	LGBM-average_length-90
H1	42.003218	0.895833	237.391189	1.000000	365.452160
H10	6.378531	0.812500	151.917693	0.895833	280.675432
H100	80.271020	0.812500	262.200214	0.875000	439.862966
H101	43.714527	0.958333	249.450173	1.000000	4679.752953
H102	98.038727	0.729167	738.286167	0.875000	9401.480156
H103	1277.223668	0.833333	10382.047638	0.979167	20295.171957
H104	72.351522	0.875000	288.520722	0.916667	3171.172246
H105	92.217800	0.666667	2458.989914	0.895833	6652.189939

Figure 17.9: Metrics for Quantile Regression with LightGBM

The disadvantage of this is that we are training a model for each quantile. This can become unwieldy quickly. Instead of training one model, training three models would make the total training time increase. Another issue is that since all three models are trained differently, they might have different properties, and the way they have learned to solve the problem can also be very different. And because of this inconsistency, the prediction intervals can also suffer from some issues. We can see it clearly in the jagged prediction intervals in many of the time series in *Figure 17.7*; they seem disconnected from the median prediction. This is a problem that we don't have in the deep learning world.

Forecasting with quantile loss (deep learning)

In the deep learning models, we use a common learning structure and just use different linear projections on the shared projection for the different quantiles. This ensures that the underlying representation and learning are common across all the quantiles and can cause a more coherent set of quantile predictions. So, for all the deep learning models we have learned about in the book, we can make them into a Quantile Forecast model by doing two things:

1. Instead of predicting point forecast (single number), we predict parameters of a probability distribution (one or more numbers).
2. Instead of using a point loss like Mean Squared Error, use a probabilistic scoring function like log likelihood.

And just like we did in the PDF section, all we need to do is to switch out the loss function in `neuralforecast`.



Notebook alert:

To follow along with the complete code, use the notebook named `05-NeuralForecast_prediction_intervals_Quantile_Loss.ipynb` in the `Chapter17` folder.

Let's see how we can do this in `neuralforecast` (the library we were using *Chapter 16*). Just like before, we will take a simple model like an LSTM and the M4 competition dataset, but we can do the same with any model or any dataset because all we are doing is switching the loss function to `MQLoss` (multi-quantile loss).

Let's start from the point where we have the data formatted the way `neuralforecast` expects in a `Y_train_df` and `Y_test_df`. The first thing we need to do is import the necessary classes.

```
from neuralforecast import NeuralForecast
from neuralforecast.models import LSTM
from neuralforecast.losses.pytorch import MQLoss
```

The only class that we haven't looked at before is `MQLoss`. This class just calculates the quantile loss we discussed just now for multiple quantiles (which is how you would want to typically train the model). These are the major parameters of `MQLoss`:

- `level`: This is a list of floats that defines the different confidence levels we are interested in modeling. For instance, if we want to model 80% and 90% confidence, we should give the values as `[80, 90]`.
- `quantiles`: This is an alternate way of defining the level. Instead of 95% confidence, you can define them in quantiles $\rightarrow [0.1, 0.9]$.

Now, let's set a horizon, the levels we need, and a few hyperparameters for LSTM.

```
horizon = 48
levels = [80, 90]
lstm_config = dict(input_size=3*horizon)
```

Now, we need to define the models we are going to use and the `NeuralForecast` class. Let's define just one model.

```
models = [LSTM(h=horizon, loss=MQLoss(level=levels), **lstm_config)]
# Setting freq=1 because the ds column is not date, but instead a sequentially
# increasing number
nf = NeuralForecast(models=models, freq=1)
```

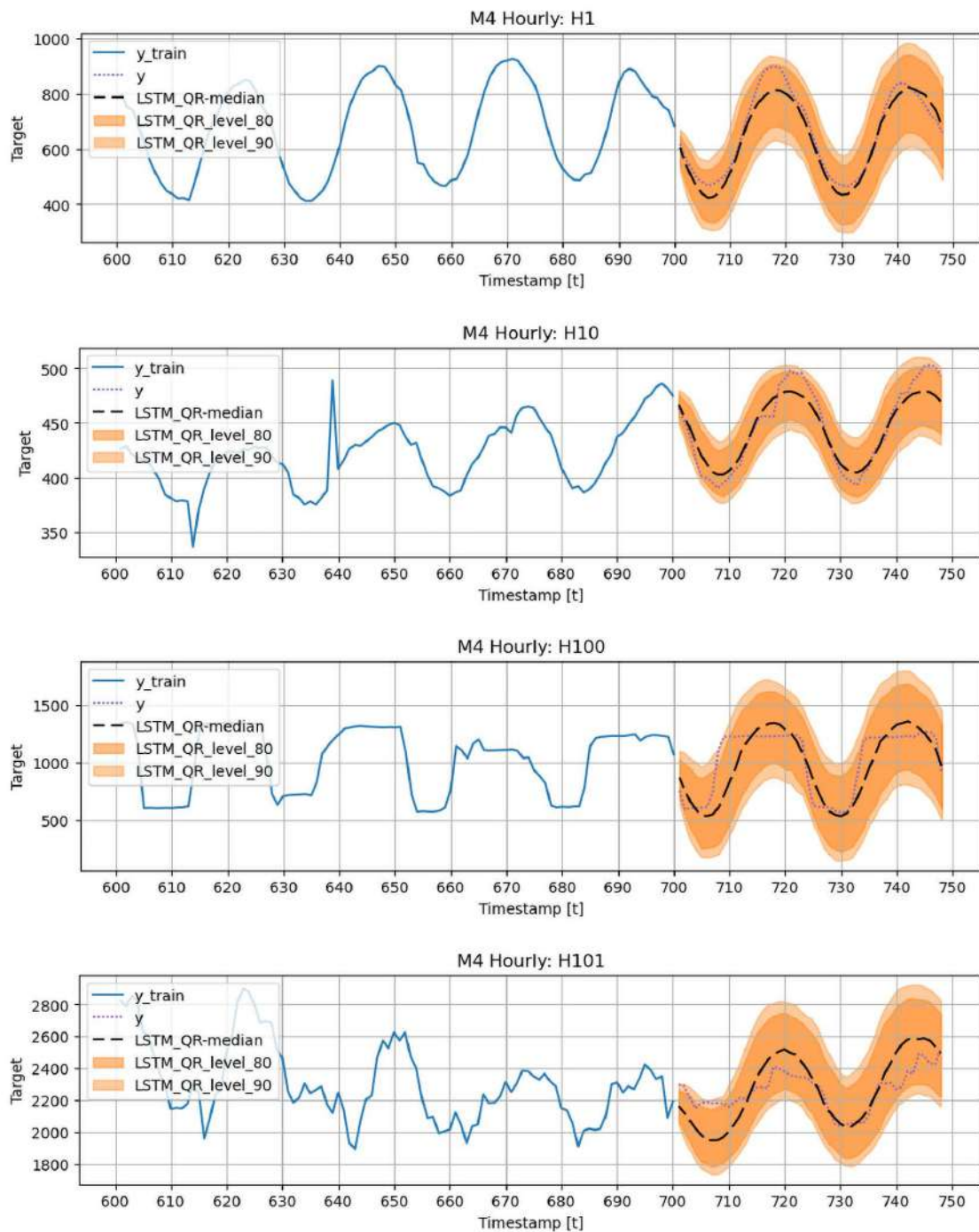
Notice how the syntax is exactly the same as the point forecast, except for the multi-quantile loss we chose. Now, all that's left is to train the models.

```
nf.fit(df=Y_train_df)
```

Once the model is trained, we can predict using the `predict` method. This output will have the point forecast under the alias we have defined and the high and low intervals for all the levels we have defined.

```
Y_hat_df = nf.predict()
```

Now, we have a way of getting prediction intervals without making an assumption about the output distribution and that is valuable in real-world cases where we are not sure what the underlying output distribution would be. Let's look at the generated probabilistic forecasts and its metrics.



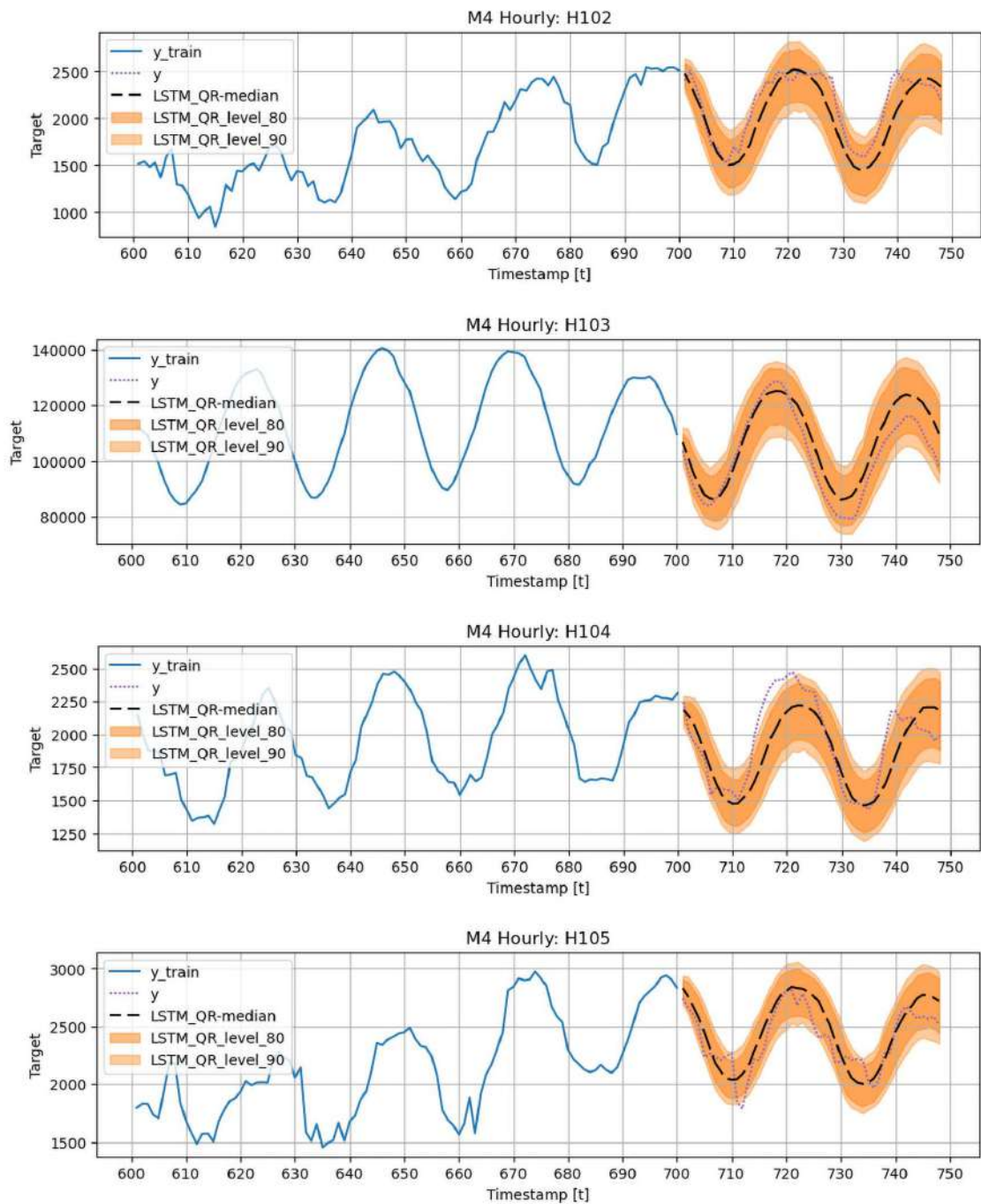


Figure 17.10: Forecast with prediction intervals using Quantile Regression (deep learning)

unique_id	LSTM_QR-mae	LSTM_QR-coverage-80	LSTM_QR-average_length-80	LSTM_QR-coverage-90	LSTM_QR-average_length-90
H1	31.440021	1.000000	238.284898	1.000000	318.520397
H10	9.935539	0.916667	48.285530	1.000000	64.168379
H100	122.138493	0.958333	681.996127	0.979167	895.010818
H101	107.808431	0.854167	464.218946	0.979167	596.402059
H102	152.887700	0.875000	564.524239	0.937500	756.065956
H103	5632.528158	1.000000	20626.433919	1.000000	27760.620931
H104	152.796626	0.729167	426.430529	0.812500	574.738497
H105	124.098546	0.812500	393.371297	0.979167	528.172567

Figure 17.11: Metrics for the Quantile Regression (deep learning)

We can see that the prediction intervals are quite in sync with each other and the median prediction and not disconnected like separate models with LightGBM. This is because the same learning is happening for all the quantiles, just the final projection heads are different. The coverage, in general, is also better.

There is another way to get prediction intervals that are dead simple, but not that easy to implement because of the way PyTorch is typically used. That's what we are going to see next.

Monte Carlo Dropout

Dropout is a hugely popular regularization layer in deep learning. Without going into details, dropout regularization is when we randomly make some part of the weights of the network while training (dropout is turned off during inference). Intuitively, this forces the model to not rely on a few weights but rather to distribute the relevance of the weights across the network. From another perspective, we are applying a sort of regularization (very similar to Ridge regularization), which makes sure none of the weights are single-handedly too high to influence the output drastically.

Technically, besides making a random part of the weights zero, we also debias each layer by normalizing by the fraction of nodes/weights that were retained (not zeroed out). Let's formalize this layer now. If the probability of dropout is p , and is applied on an intermediate activation, h then the activation after dropout, h' will be:

$$h' = \begin{cases} 0 & \text{with } p \text{ probability} \\ \frac{h}{1-p} & \text{otherwise} \end{cases}$$



Why do we need to scale/normalize the output when dropout is applied? The intuitive answer is to make sure the output has the same scale during training (when dropout is active) and during inference (when dropout is turned off). The long answer is as follows.

Let's say without dropout, the output of a node is h . Now, with dropout, with a probability of p , this output becomes 0 and with a probability of $1 - p$, it is h . Therefore, the expected value of the node would be: $\mathbb{E}[\text{output}] = (1 - p) \cdot h + p \cdot 0$. This means the average value of the output is reduced by a factor of $1 - p$, which is not desirable as this would change the scale of the values during training and inference. Therefore, the solution is to scale the output of the nodes that are retained by dropout by $1 - p$.

Now, we know what dropout is. But remember that we were using dropout only during training as a regularization. In 2015, Yarín Gal et al. proposed that the good old dropout also doubles as a *Bayesian Approximation of Gaussian Processes*. That's a lot of terms that we haven't come across in a single phrase. Let's take a short detour to understand these terms at a high level, and I'll include other links in *Further reading*.



Reference check:

The paper by Yarín Gal et al. about Monte Carlo Dropout is cited in *References* under reference 2.

Bayesian inference is a statistical method that updates the probability of a hypothesis as more evidence or information becomes available. It is based on Bayes' theorem, which mathematically expresses the relationship between the prior probability, the likelihood, and the posterior probability. Formally, Bayes' theorem is given by:

$$p(\theta|D) = \frac{p(D|\theta) \cdot p(\theta)}{p(D)}$$

where $p(\theta|D)$ is the *posterior probability* of hypothesis θ given the data D , $p(D|\theta)$ is the *likelihood* (probability of observing the data D given hypothesis θ), $p(\theta)$ is the *prior probability* of the hypothesis θ before observing the data, and $p(D)$ is the marginal likelihood or *evidence* (the total probability of observing the data under all possible hypotheses).

Although it has some specific terminology, this is very intuitive and provides a structured way to update our prior beliefs in the face of evidence or data. We start with a prior distribution $p(\theta)$ that represents our initial beliefs about the parameters. As we observe data D , we update our beliefs getting the posterior distribution $p(\theta|D)$. This posterior distribution combines the prior information and the likelihood of the observed data, providing a new, updated belief about the parameters. *Further reading* has a more detailed explanation for those who are interested. It also has a page from *Seeing Theory* that helps you visualize these in an intuitive way.

Now, let's move on to the **Gaussian Process (GP)**. As we have seen in *Chapter 5*, supervised learning is learning a function $\hat{y} = h(X, \phi)$ where $\hat{y}$ is the quantity we are interested in predicting, h , is the function we learn, X is the input data, and ϕ represents the model parameters. So, GPs assume this function as a probability distribution and use Bayesian inference to update the posterior of the function using the data we have available for training. It's a drastically different way of learning from data and is inherently probabilistic.

There is one more term left, which is approximation. In many cases, the complex posterior distributions in Bayesian models make them intractable. So, we have a technique called *Variational Inference* in which we use a known parametric distribution family, q , and find a member that is closest to the true posterior. Getting into details of Variational Inference and GPs is out of the scope of the book, but I have added a few links in *Further reading* for those of you who are interested.

So, coming back to dropouts, Yarin Gal et al. showed that a neural network defined with dropout layers before each of the weight layers is in fact a Bayesian approximation of a GP. So, if the model with dropout is a GP and GP is a posterior over functions, this should give us a probabilistic output, right? But here, we don't have a well-defined parametric probability distribution like the Normal distribution for us to analytically calculate the properties (like the mean, standard deviation, or quantiles) of the distribution. How do we do that?

Remember the discussion at the start of the Quantile Function section where we said that if we have N samples drawn from a distribution and if that N is sufficiently large, we can approximate the properties of the distribution? We have a name for that, and it's called *Monte Carlo sampling*. *Monte Carlo sampling* is a computational technique used to estimate the statistical properties of a distribution by generating many random samples from that distribution. Bringing this idea to the dropout-enabled neural network, we can assess the properties of the posterior probability distribution of the function using Monte Carlo sampling, which means we need to keep dropout turned on during inference and sample from the posterior by executing the forward pass N times.

Theoretical justification allows us to apply dropout to any neural network, getting uncertainty estimates through a simple operation. Isn't the simplicity of that beautiful?

So, all of that boils down to these simple steps to get prediction intervals for our forecasts:

1. Pick any deep learning architecture.
2. Insert Dropout Layers before every major operation and set them to a value p , where $p > 0$.
3. After training the model, with dropout enabled, do N forward passes.
4. Using the N samples, estimate the median (for the point prediction), and the quantiles corresponding to the defined confidence levels for prediction intervals.

This sounds simple enough, but it's complicated for just one reason. Both PyTorch and Tensorflow are designed in a way that dropouts are turned off during inference. In PyTorch, we can indicate if the model is in the training phase or inference phase by doing `model.train()` or `model.eval()`, respectively. And most popular implementations that wrap PyTorch to make training easy and automated (like PyTorch Lightning) do this `model.eval()` step in the backend before predicting. So, when using libraries like `neuralforecast` (which uses PyTorch Lightning in the background), turning on dropout during prediction isn't easy.

Before we learn how to implement MC Dropout for `neuralforecast` models, let's take a slight detour and learn how to define custom models in `neuralforecast`. It is useful when you want to tweak any model for your use case. And we are doing it here because of two reasons:

1. I wanted to show you how to define a new model in `neuralforecast`.
2. I wanted a model that is ideal for the MC Dropout technique, i.e., the model needs to have dropouts before every weight/layer.

Creating a custom model in neuralforecast

Now it's time to have some fun and define a custom PyTorch model that works with `neuralforecast`. To keep things simple, let's use a model that is a small tweak on **D-Linear** we learned about in *Chapter 16*. Besides the linear trend and seasonality, we also add a component for non-linear trend. Let's give it a wacky name as well—**D-NonLinear**. The architecture would be something like this:

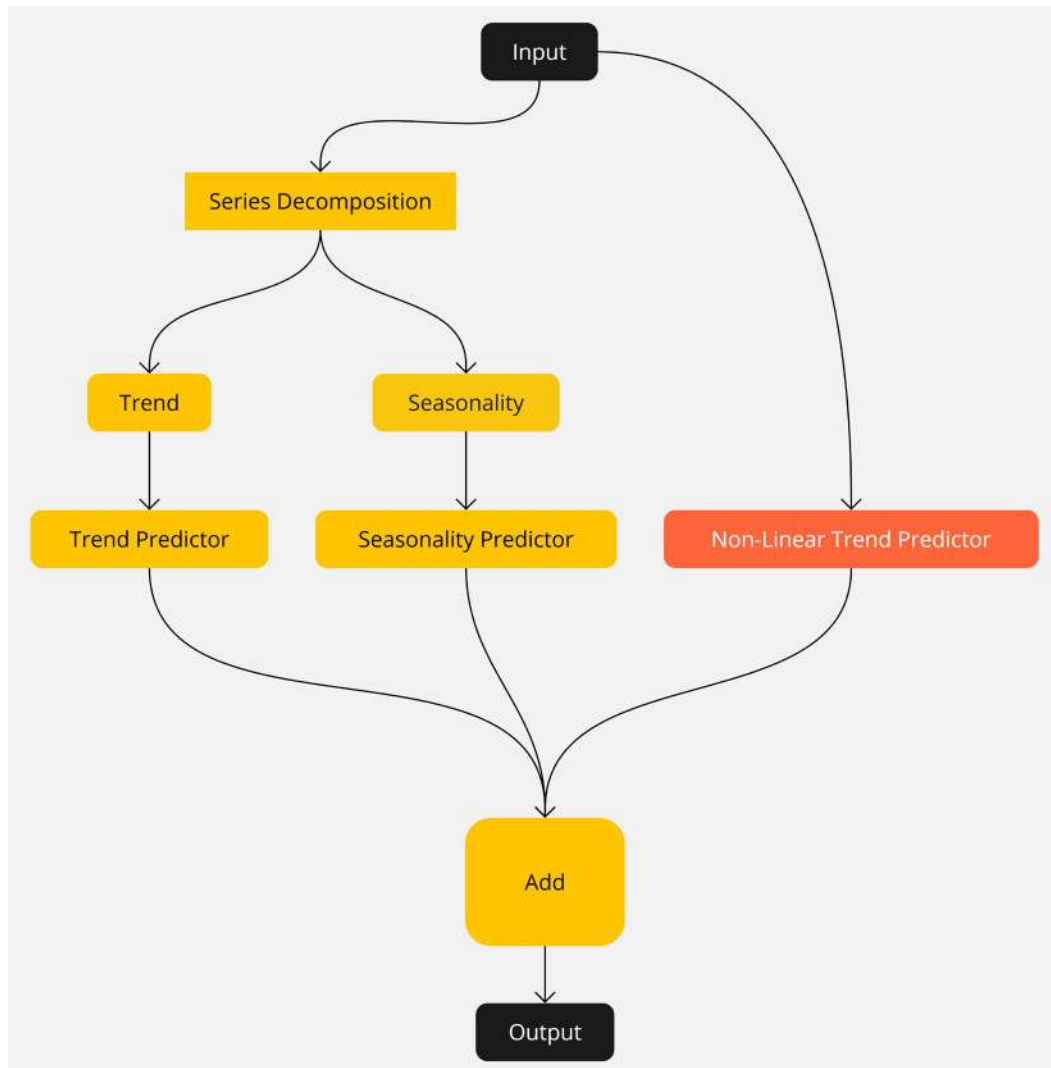


Figure 17.12: D-NonLinear model architecture

Now, let's understand how to write a model that would work with `neuralforecast`. All models in `neuralforecast` are inherited from one of three classes—`BaseWindows`, `BaseRecurrent`, or `BaseMultivariate`. The documentation clearly explains the purpose of `BaseWindows`, which is exactly what we require for our use case. We need to sample windows from a time series while training.

One more thing we need to keep in mind is that `neuralforecast` uses `PyTorch Lightning` under the hood for training. This link has more details on how to define a new model for `neuralforecast`: https://nixtlaverse.nixtla.io/neuralforecast/docs/tutorials/adding_models.html.



If you aren't aware of **Object-Oriented Programming (OOP)** and **inheritance**, then it might be difficult for you to understand what we are doing here. Inheritance allows a child class to inherit all the attributes and methods defined in a parent class. This allows developers to define common functionalities in a base class and then inherit that class to get all the functionality and then add on top of that any specific functionality you want to add to a class. It is highly recommended that you understand inheritance, not only for this example but also to become a better developer in general. There are hundreds of tutorials on the internet, and I'm linking one here: <https://ioflood.com/blog/python-inheritance>.

The full code for the model can be found in `src/dl/nf_models.py`, but we will look at key portions of the model definition right here.

Let's start by defining the `__init__` function (only including relevant portions here; refer to the Python file for the full class definition).

```
class DNonLinear(BaseWindows):
    def __init__(
        self,
        # Inherited hyperparameters with no defaults
        h,
        input_size,
        # Model specific hyperparameters
        # Window over which the moving average operates for trend extraction
        moving_avg_window=3,
        dropout=0.1,
        # Inherited hyperparameters with defaults
        ...
        **trainer_kwargs,
    ):
        super(DropoutDNonLinear, self).__init__(
            h=h,
            ...
            **trainer_kwargs,
        )
        # Model specific hyperparameters
        self.moving_avg_window = moving_avg_window
        self.dropout = dropout
        # Model initialization to follow
```

We have now defined part of the `__init__` function. Now, let's initialize the different layers needed in the rest of the method. We have a series decomposition layer that uses a moving average to split the input into a trend and seasonal component, a linear trend predictor and seasonality predictor that takes the linear trend and seasonality and projects it into the future, and a non-linear predictor that takes in the original input and projects into the future.

```
# Defining a decomposition Layer
self.decomp = SeriesDecomp(self.moving_avg_window)
# Defining a non-linear trend predictor with dropout
self.non_linear_block = nn.Sequential(
    nn.Dropout(self.dropout),
    nn.Linear(self.input_size, 100),
    nn.ReLU(),
    nn.Dropout(self.dropout),
    nn.Linear(100, 100),
    nn.ReLU(),
    nn.Dropout(self.dropout),
    nn.Linear(100, self.h),
)
# Defining a linear trend predictor with dropout
self.linear_trend = nn.Sequential(
    nn.Dropout(self.dropout),
    nn.Linear(self.input_size, self.h),
)
# Defining a seasonality predictor with dropout
self.seasonality = nn.Sequential(
    nn.Dropout(self.dropout),
    nn.Linear(self.input_size, self.h),
)
```

Now, let's define the forward method. The forward method should have just one argument that is a dictionary of different inputs:

- `insample_y`: The context window of the target time series we have to predict
- `futr_exog`: The exogenous variables for the future
- `hist_exog`: The exogenous variables for the context window
- `stat_exog`: The static variables

For this use case, we only need the `insample_y` since our model doesn't use any other information. So, this is the forward method implementation:

```
def forward(self, windows_batch):
    # Parse windows_batch
    insample_y = windows_batch[
```

```

        "insample_y"
    ].clone() # --> (batch_size, input_size)
    seasonal_init, trend_init = self.decomp(
        insample_y
    ) # --> (batch_size, input_size)
    # Non-linear block
    non_linear_part = self.non_linear_block(
        insample_y
    ) # --> (batch_size, horizon)
    # Linear trend block
    trend_part = self.linear_trend(trend_init) # --> (batch_size, horizon)
    # Seasonality block
    seasonal_part = self.seasonality(
        seasonal_init
    ) # --> (batch_size, horizon)
    # Combine the components
    forecast = (
        trend_part + seasonal_part + non_linear_part
    ) # --> (batch_size, horizon)
    # Map the forecast to the domain of the target
    forecast = self.loss.domain_map(forecast)
    return forecast

```

The code is pretty straightforward. The only thing we need to ensure to align to `neuralforecast` models is to take the data we need from the input dictionary and call `self.loss.domain_map` at the end so that it is mapped to the right output size depending on the loss. Now, this model will function just like any other model in the `neuralforecast` library.

Now, let's get back to MC Dropout and its implementation.

Forecasting with MC Dropout (`neuralforecast`)

We said it wasn't easy to implement MC Dropout in frameworks like `neuralforecast` and PyTorch Lightning, but just because something isn't easy shouldn't stop us from doing it. All we need to do is to make sure the dropouts are enabled during prediction and take multiple samples. If you are writing your own PyTorch training code, then it's as simple as not calling `model.eval()` before predicting. But the best practice is to just make the dropouts into train mode and not the whole model. There may be layers like batch normalization, which also behave differently during inference, which might be affected. Let's see a handy method that makes all dropout layers into train mode.

```

def enable_dropout(model):
    """Function to enable the dropout layers during test-time"""
    for m in model.modules():
        if m.__class__.__name__.startswith("Dropout"):
            m.train()

```

For `neuralforecast`, we have put together a recipe with which we can use MC Dropout for any of their models (provided they have enough dropouts). Now, we are going to use the custom `DNonLinear` model we just defined. Note that each component in the definition starts with a dropout layer so that we can apply MC Dropout with no qualms.



Notebook alert:

To follow along with the complete code, use the notebook named `06-Prediction_Intervals_MCDropout.ipynb` in the `Chapter17` folder.

If you remember, we used `PyTorch Lightning` back in *Chapter 13* and explained that it's pretty much standard `PyTorch` code, but organized in a specified form—`training_step`, `validation_step`, `predict_step`, `configure_optimizers`, etc. For a refresher, head back to *Chapter 13* and *Further reading* in the chapter to learn more about how to migrate from `PyTorch` to `PyTorch Lightning`. Since `neuralforecast` is already using `PyTorch Lightning` in the backend, the `BaseWindows` that we are inheriting is already a `PyTorch Lightning` model. This information is essential because we need to essentially modify the `predict_step` method to implement our MC Dropout.

Using the same inheritance we used to inherit `BaseWindows`, we can inherit the `DNonLinear` class we defined earlier and make a few changes so that it becomes an MC Dropout model. And for that, all we need to re-define is the `predict_step` method. The `predict_step` method is the method `PyTorch Lightning` calls every time it has to get a prediction for a batch. So, instead of taking the predictions as is, we need to keep the dropout enabled, take N samples from N forward passes, calculate the prediction intervals and median (point forecast), and return it.

```
class MCDropoutDNonLinear(DNonLinear):
    def predict_step(self, batch, batch_idx):
        enable_dropout(self)
        pred_samples = []
        # num_samples and levels will be saved to the model in MCNeuralForecast
        predict method
        for i in range(self.num_samples):
            y_hat = super().predict_step(batch, batch_idx)
            pred_samples.append(y_hat)
        # Stack the samples
        pred_samples = torch.stack(pred_samples, dim=0)
        # Calculate the median and the quantiles
        y_hat = [pred_samples.quantile(0.5, dim=0)]
        if self.levels is not None:
            for l in self.levels:
                lo, hi = level_to_quantiles(l)
                y_hat_lo = pred_samples.quantile(lo, dim=0)
                y_hat_hi = pred_samples.quantile(hi, dim=0)
                y_hat.extend([y_hat_lo, y_hat_hi])
```

```

# Stack the results
y_hat = torch.stack(y_hat, dim=-1)
return y_hat

```

Are we done yet? Not quite. There is just one little thing to be done. In *Chapter 16*, we used a class called `NeuralForecast` for all the fitting and predicting of neuralforecast models. This is like a wrapper class that does the heavy lifting of preparing the inputs and outputs in the right way before calling the underlying models. This class has to be aware that we have tweaked the `predict_step` and therefore, we need to make a small change there. The solution is more of a hack than a principled way of editing, but a hack is just as good if it achieves the purpose. I have done the snooping around the implementation to figure out the best way to hack `NeuralForecast` to enable our `MCDropout` inference. There is no short way of explaining the hack, but just understand that I have misused the way `neuralforecast` flexibly produces point forecasts and prediction intervals based on different losses. So, here is the re-defined `NeuralForecast` class with a hack in the `predict` method.

```

class MCNeuralForecast(NeuralForecast):
    def __init__(self, num_samples, levels=None, **kwargs):
        super().__init__(**kwargs)
        self.num_samples = num_samples
        self.levels = levels

    def predict(
        self,
        df=None,
        static_df=None,
        futr_df=None,
        sort_df=True,
        verbose=False,
        engine=None,
        **data_kwargs,
    ):
        # Adding model columns to loss output names
        # Necessary hack to get the quantiles and format it correctly
        for model in self.models:
            model.loss.output_names = ["-median"]
            for l in list(self.levels):

```

```

        model.loss.output_names.append(f"-lo-{l}")
        model.loss.output_names.append(f"-hi-{l}")
        # Setting the number of samples and levels in the model
        model.num_samples = self.num_samples
        model.levels = self.levels
    return super().predict(
        df, static_df, futr_df, sort_df, verbose, engine, **data_kwargs
    )

```

That's it. We have successfully “hacked” the library to do our bidding. In addition to this being an MC Dropout tutorial, it's also a tutorial on how to hack a library to do what you want it to do. It is important to note that this doesn't make you a “hacker,” so stop before you update your LinkedIn title.

Now, on to training the model. This is pretty much the same as you train other models in neuralforecast, but instead of the NeuralForecast class, you need to use the new MCNeuralForecast class we defined.

```

horizon = len(Y_test_df.ds.unique()) # 48
levels = [80, 90]
model = MCDropoutDNonLinear (
    h= horizon,
    input_size=WINDOW,
    moving_avg_window=horizon*3,
    dropout=0.1,
    max_steps=500,
    early_stop_patience_steps=5,
)

mcnf = MCNeuralForecast(models=[model], freq=1, num_samples=100, levels=levels)
mcnf.fit(Y_train_df, val_size= horizon, verbose=True)

```

Once the training is finished, we can generate the predictions like this:

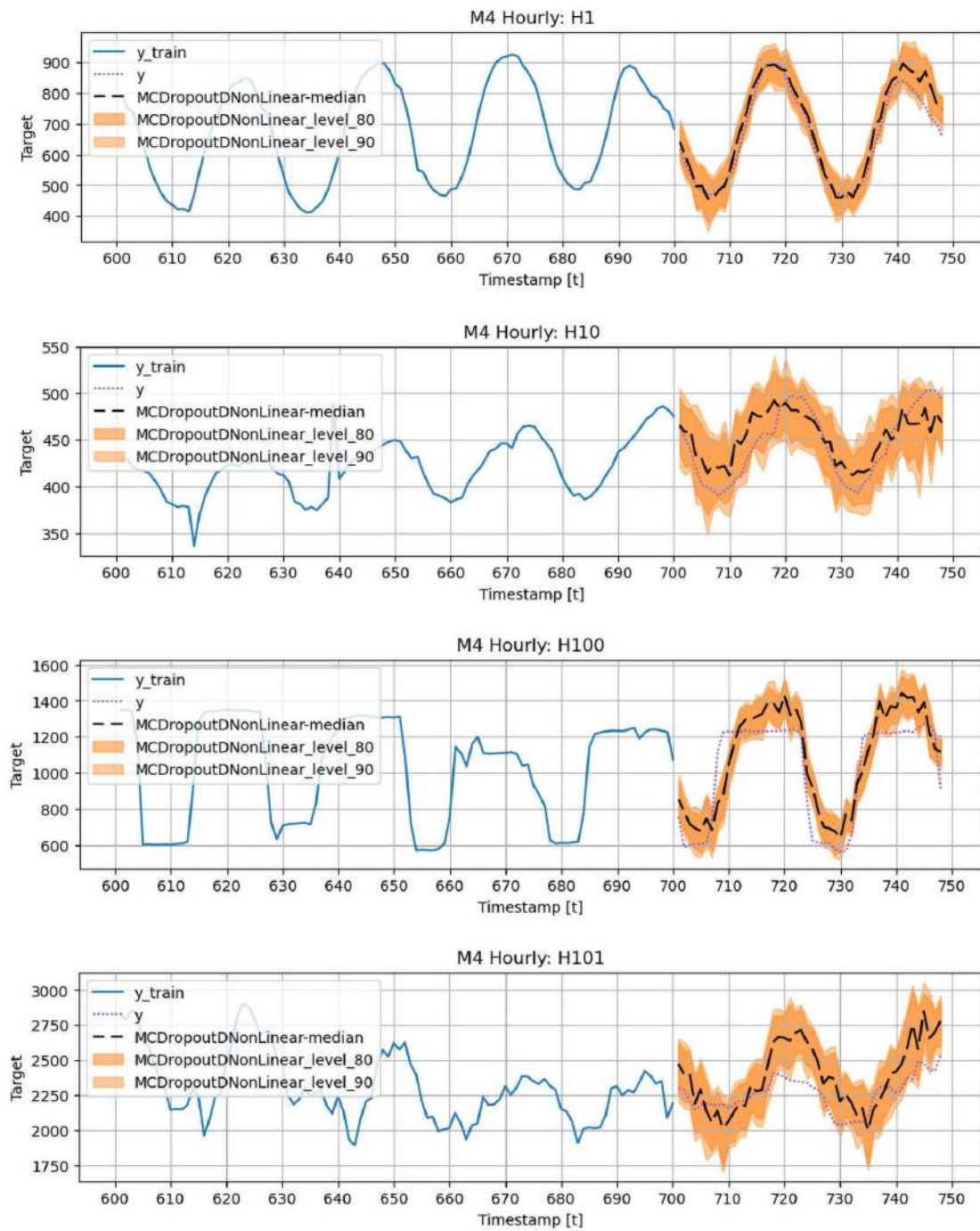
```

Y_hat_df = mcnf.predict()
Y_hat_df = Y_hat_df.reset_index()

```

The output is exactly like any other model from neuralforecast with the prediction intervals formatted as <ModelName>-lo-<level> and <ModelName>-hi-<level>. The point forecast can be found under <ModelName>-median. In this case, <ModelName> would be MCDropoutDNonLinear.

Let's look at the plot of the forecasts and the metrics.



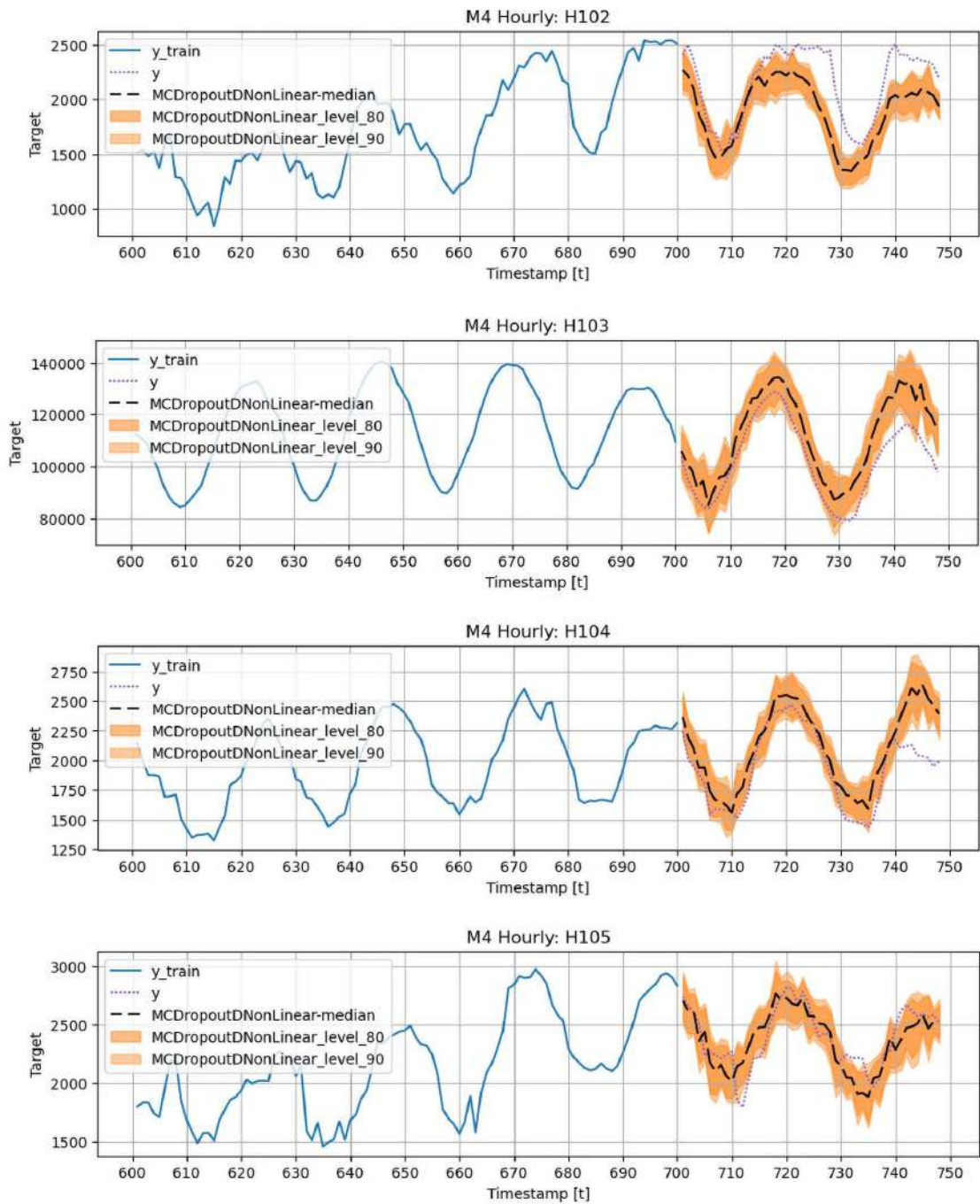


Figure 17.13: Forecast with prediction intervals using MC Dropout

unique_id	MCDropoutDNonLinear-mae	MCDropoutDNonLinear-coverage-80	MCDropoutDNonLinear-average_length-80	MCDropoutDNonLinear-coverage-90	MCDropoutDNonLinear-average_length-90
H1	29.088626	0.854167	107.338516	0.916667	136.682404
H10	18.265211	0.895833	64.447991	0.958333	82.978088
H100	127.643550	0.354167	181.783834	0.541667	229.839363
H101	163.722567	0.625000	368.342176	0.729167	462.756533
H102	266.991793	0.270833	289.874980	0.291667	364.814293
H103	9219.924805	0.583333	16096.631022	0.708333	20437.640462
H104	171.700793	0.583333	306.364553	0.666667	386.775024
H105	128.752263	0.729167	346.371218	0.770833	439.413638

Figure 17.14: Metrics for MC Dropout

Our forecasting model does decent enough on the data; it's nothing to write home about, but decent. If we do an ablation study, we might even realize that the non-linear component we added does absolutely nothing. But, as long as we had fun doing it and learned something from it, I'm happy. Now, look at the prediction intervals. They aren't really smooth and have quite a bit of "noise" when you look at them, right? This is because of the inherent randomness in the methodology and may be because of insufficient learning. When we do MC Dropout, we are essentially relying on N sub-models or sub-networks and calculating the quantiles based on these N forecasts. Maybe a few of these sub-networks haven't learned very well, and those outputs can skew the quantiles and thereby the prediction intervals.

There are many criticisms of the MC Dropout method. Many in the Bayesian community don't consider MC Dropout as Bayesian and consider the Variational Approximation that was proposed such a poor approximation that we can't refer to what it measures as Bayesian uncertainty. There is an unpublished Arxiv paper by Loic Le Folgoc et al. called "Is MC Dropout Bayesian?" (Reference 6), which claims that it isn't. But it still doesn't take away the fact that MC Dropout is a cheap way of getting uncertainty quantified. But when used in fields like medical studies, where uncertainty quantification is of paramount importance, we may want to take on something more principled.

We can also notice that the coverage is quite bad across all time series. And this is, again, something that is exhibited across different studies. In 2023, Nicolas Dewolf et al. published a study comparing different ways of uncertainty quantification for regression problems (Reference 7). They found that MC Dropout has one of the worst performances in both coverage and average length, underlying the claim that MC Dropout is a very dirty approximation of the uncertainty.

Now, let's look at another technique for probabilistic forecasting that promises theoretical guarantees for perfect coverage and has become quite the rage in the last few years.

Conformal Prediction

What if I tell you that there is a technique for generating prediction intervals that statistically guarantees perfect coverage, can work on any model, and doesn't require us to make any assumptions about the output distribution? Conformal Prediction is just that. Conformal Prediction is a method that helps machine learning models make reliable predictions by estimating how uncertain the model is. Conformal Prediction provides robust, statistically valid measures of uncertainty for any machine learning model, ensuring reliable and trustworthy predictions in critical applications.

Although it was proposed as early as 2005 by Vladimir Vovk (Reference 8), it picked up interest in the last couple of years. Let's first understand the basic principles of Conformal Prediction using a classification example and then see how we can do it for regression and time series examples.

Conformal Prediction for classification

Let's start with a trained model, $\hat{f}$, which outputs estimated probabilities (*softmax* scores) for K output classes ($\hat{f}(x) \in [0,1]^K$). It doesn't matter what this model is; it can be a machine learning model, a deep learning model, or even a rule-based model. We have training data, (X_{train}, Y_{train}) , and test data, (X_{test}, Y_{test}) . Now, we need a small amount of additional data (other than training and test) called *calibration data*, $(X_{calib}, Y_{calib}) = \{(x_1, y_1), \dots, (x_n, y_n)\}$. Now, what do we want from this? Using $\hat{f}$ and X_{calib} , we want to create a prediction set of possible labels, $\mathcal{C}(X_{test})$ that makes sure that the probability that a test data point is part of that set is almost exactly the user-defined error rate, α (we will be talking about error rates throughout this discussion. A 10% error rate is a 90% confidence level). This is exactly what Conformal Prediction guarantees. It's called the *marginal coverage* guarantee and it can be written more formally as:

$$1 - \alpha \leq \mathbb{P}(Y_{test} \in \mathcal{C}(X_{test})) \leq 1 - \alpha + \frac{1}{n+1}$$

Naturally, you may have this question in your mind. What is this $\frac{1}{n+1}$? This term signifies that the coverage guarantee is derived from a finite sample of size n . Since n is in the denominator, we know as n increases, this term becomes smaller and smaller. Extending this to the limit, we know that if $n = \infty$, this term would be zero and the coverage would be exactly $1 - \alpha$.

We know what we want, but how do we get it? The core idea in conformal prediction is very simple and can be laid out in four steps:

1. Identify a heuristic notion of uncertainty using the trained model. In our classification example, this can be the softmax scores.
2. Define a score function, $s(\hat{y}, y) \in \mathbb{R}$, which is also called *Non-Conformity Scores*. This can be a score that takes in the prediction, $\hat{y}$, and actual value, y , and gives a score that encodes the disagreement between them. The higher the score is, the larger the disagreement is. In the classification example, this would be something as simple as $1 - \hat{f}(x)_{y_t}$. In simple English, this means taking the softmax score of the correct class and doing $1 - \text{score}$.
3. Compute $\hat{q}$ as the $\frac{(1-\alpha)(n+1)}{n}$ quantile of the calibration scores. We use the calibration data and score function to calculate calibration scores and calculate the quantile on that data. The $\frac{n+1}{n}$ is the quantile calculation is again derived from the finite sample correction. As n tends to infinity, the term tends to zero.
4. Use this quantile to form prediction sets for new examples: $\mathcal{C}(x) = \{y: s(x, y) \leq \hat{q}\}$. This means selecting all the items from the output set that has a score (according to the score function) greater than the threshold, $\hat{q}$.

This simple technique will give us prediction sets that guarantee to satisfy the marginal coverage, no matter what model is used or what the distribution of the data is. Let's see how simple this is using Python code and assuming that the model we are talking about is a `scikit-learn` classifier.

We have a trained model, `model`, calibration data, `X_calib`, and test data, `X_test`. For full code and some visualizations, check the notebook.



Notebook alert:

To follow along with the complete code, use the notebook named `07-Understanding_Conformal_Prediction.ipynb` in the `Chapter17` folder.

```
# 1: Get conformal scores
n = calib_y.shape[0]
cal_smx = model.predict_proba(calib_x) # shape (n, n_classes)
# scores from the softmax score for the correct class
cal_scores = 1 - cal_smx[np.arange(n), calib_y] # shape (n,)
# 2: Get adjusted quantile
alpha = 0.1 # Confidence Level (1 - alpha)
q_level = np.ceil((n + 1) * (1 - alpha)) / n
qhat = np.quantile(cal_scores, q_level, method='higher')
# 3: Form prediction sets
val_smx = model.predict_proba(test_x)
prediction_sets = val_smx >= (1 - qhat)
```

Now, let's think about the prediction set, $\mathcal{C}$. We have been defining it as set-valued with discrete classes for the classification scenario. This set becomes larger or smaller based on how confident the initial heuristic estimate of uncertainty is.

Conformal Prediction for regression

Let's extend this notion of prediction sets to regression. In regression, the output space is continuous rather than discrete, and we aim to construct continuous prediction sets, which are typically a continuous interval in $\mathbb{R}$. The idea is to maintain the same principle of coverage: the prediction interval should contain the true value with high probability. So, now the prediction set, $\mathcal{C}$, that we saw earlier is also the prediction interval in the regression context. But along with the change in the interpretation of prediction sets, we will also need to change the score function, which calculates non-conformity scores. A common score function that is used is the distance to the conditional mean, $s(x, y) = |y - \mu(x)|$ (Reference 10). When we have a trained model, $\hat{f}$, we can consider the output of the model as the conditional mean, which will make this the absolute residual value for each point.

$$s(x, y) = |y - \hat{f}(x)|$$

Note that this score satisfies the condition. The larger the deviation, the larger the “heuristic” measure of uncertainty is. The rest of the procedure remains almost the same—calculating the quantile, $\hat{q}$, and forming the prediction intervals, $\mathcal{C}(x) = [\hat{f}(x) - \hat{q}, \hat{f}(x) + \hat{q}]$

Let's check how the Python code changes (full code is in the notebook).

```
# 1: Get conformal scores
calib_preds = model.predict(calib_x)
cal_scores = np.abs(calib_y - calib_preds)
# 2: Get adjusted quantile
qhat = ... # Exactly the same as classification
# 3: Form prediction intervals
test_preds = model.predict(test_x)
lower_bounds = test_preds - qhat
upper_bounds = test_preds + qhat
```

It's as simple as that. We can check coverage and see that it will be greater than 90%, which is the error rate we defined with $\alpha = 0.1$.



Practitioner's Note

In many use cases, we will be training a single model for multiple entities or groups we care about. For instance, for the global forecasting models we talked about in *Chapter 10*, we use a single regression model for multiple time series. In such cases, we can also run Conformal Prediction on each time series or groups of time series separately to be more adaptive to the errors in that subset. This would allow for coverage guarantees at the group/time series level.

But you might have noticed something. In this method, we have the same width of the interval all throughout. But we would expect the intervals to be tighter when the model is more confident and wider when it isn't (let's call it *adaptive prediction intervals* from now on). Let's look at another technique that has this property.

Conformalized Quantile Regression

We learned about Quantile Regression as one of the methods for probabilistic forecast (or regression in a general case). Quantile Regression is powerful in the sense that it does not require us to have any prior assumption about the underlying output distribution. But it does not enjoy the coverage guarantees that Conformal Prediction offers. In 2019, Yaniv Roano et al. (Reference 11) brought the best of both worlds into **Conformalized Quantile Regression (CQR)**. They proposed a way to take the quantile forecasts and conformalize them such that they have the coverage guarantees that conformal prediction assures.

In this case, the model to be used has a restriction. It should be a model that outputs quantile predictions. And, from our earlier discussion, we know that if the error rate is α , then the quantiles we need for the prediction intervals are $\hat{y}_t^{\alpha/2}$ and $\hat{y}_t^{1-(\alpha/2)}$.

So, the quantile model predicts $\hat{y}_t^{\alpha/2}$ and $\hat{y}_t^{1-(\alpha/2)}$. By definition, if $\hat{y}_t^{\alpha/2}$ and $\hat{y}_t^{1-(\alpha/2)}$ are true estimations of the real quantiles, a quantile regression alone will have perfect coverage. But the model fit may not be perfect and that would result in sub-par coverage. We will use conformal prediction to correct the quantiles based on calibration data such that we get the perfect coverage promised by conformal prediction.

Yaniv Roano et al. proposed to use a new non-conformity score function for Quantile Regression.

$$s(x, y) = \max \{ \hat{y}_t^{\alpha/2} - y, y - \hat{y}_t^{1-(\alpha/2)} \}$$

Let's take a beat and explore the score function using the diagram in Figure 17.15.

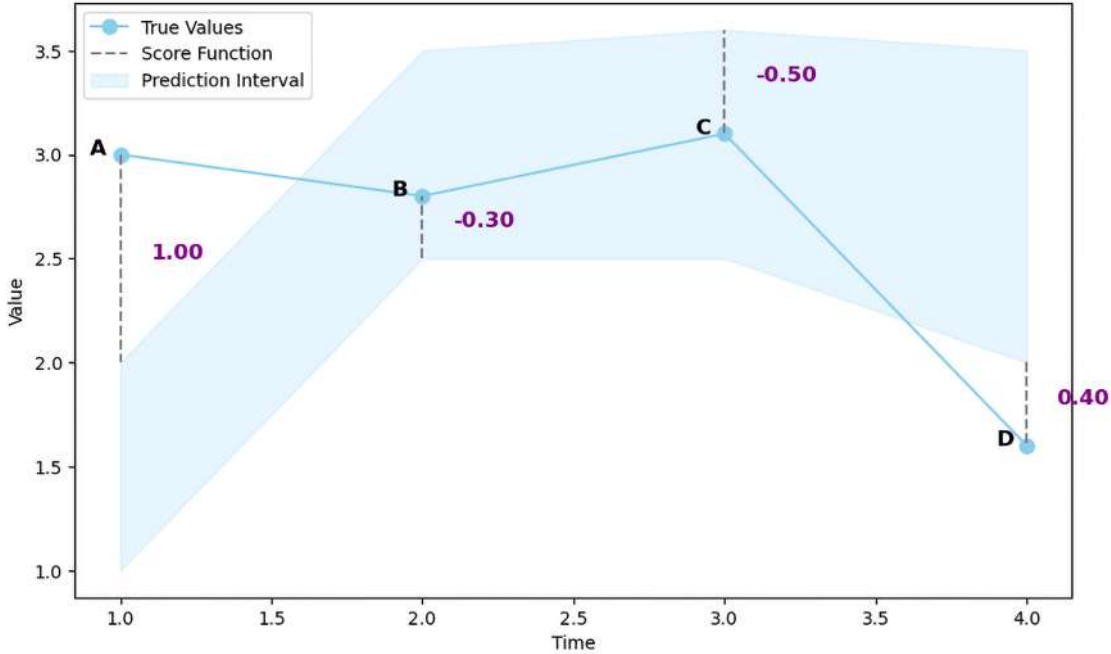


Figure 17.15: Illustration of score function for Conformalized Quantile Regression

There are two terms in the max operator. If the true value, y , is between the two quantiles, $\hat{y}_t^{\alpha/2}$ and $\hat{y}_t^{1-(\alpha/2)}$, both the terms inside will be negative and will be the distance to the nearest prediction interval (see points B and C in Figure 17.15).

Now, let's look at point A (3), which is above the higher quantile. The quantiles are [1, 2]. The two terms in the max operator would be:

$$\hat{y}_t^{\alpha/2} - y = 1 - 3 = -2$$

$$y - \hat{y}_t^{1-(\alpha/2)} = 3 - 2 = 1$$

This makes the score function 3 because of the max operator. And now, we see point D (1.6), which is below the lower quantile (quantiles are [3.5, 2]). This makes the score function $\max\{0.4, -1.9\}$, which would be 0.4. So, the max operator ensures that the score is positive if it falls outside the quantiles.

Therefore, what we have is a score function that assigns positive values to points where the actual value falls outside the intervals and negative to points within. And for the points that fall outside the intervals, the way the score function is constructed will choose the worse error. This satisfies our requirements from the score function. A larger score shows larger uncertainty, and it also encodes a heuristic notion of uncertainty.

Now that we have the scores, the rest of the steps are almost identical:

1. Compute $\hat{q}$ as the $\frac{(1-\alpha)(n+1)}{n}$ quantile of the calibration scores.
2. Use this quantile to form prediction sets for new examples: $\mathcal{C}(X_{test}) = [\hat{y}_t^{\alpha/2} - \hat{q}, \hat{y}_t^{1-(\alpha/2)} + \hat{q}]$, i.e., we widen the existing quantiles by $\hat{q}$ and get coverage guarantees.

Quantile regression is one of the better ways of getting *adaptive prediction intervals*. Let's look at one such technique now.

Conformalizing uncertainty estimates

If we think about Conformalized Quantile Regression (from the last section) a bit deeply, we can realize that what the underlying quantile regression is doing is capturing the *uncertainty estimate* at each point in our prediction. And we conformalize those estimates for better coverage. If we can capture this *uncertainty estimate*, $u(x)$, we have some hope of conformalizing this to get better *adaptive prediction intervals*.

Let's say that we have a trained model, $\hat{f}$, and some uncertainty scalar, $u(x)$, that has high values when uncertainty is high, and vice versa. We can define our non-conformity score as:

$$s(x, y) = \frac{|y - \hat{f}(x)|}{u(x)}$$

The natural interpretation of this score is that we are multiplying a correction factor to the standard $|y - \hat{f}(x)|$. Once we have this new score, the rest of the process is exactly the same—taking $\hat{q}$ from the scores and forming the prediction intervals as $\mathcal{C}(x) = [\hat{f}(x) - u(x) \cdot \hat{q}, \hat{f}(x) + u(x) \cdot \hat{q}]$.

So, what are some ways to capture this uncertainty? (Note that this uncertainty measure should be capturing it at a data point level.)

- Assume a probability distribution and modeling their parameters (**Probability Density Function**). In the case of Gaussian Distribution, we would have an estimate of uncertainty as the standard deviation, $\hat{\sigma}(x)$, and we consider that as $u(x)$. $u(x) = \hat{\sigma}(x)$.
- Use the MC Dropout technique to generate samples and calculate the standard deviation of $\hat{f}(x)$ from the samples: $u(x) = SD_{samples}[\hat{f}(x)]$.

- Along with the main model prediction $y|X$, train another model to predict the residuals, $\hat{r} = (\hat{y} - y)|X$, and set $u(x) = \hat{r}(x)$.
- Use an ensemble of models to generate multiple predictions for each data point and take the standard deviation of different predictions at each data point: $u(x) = SD_{ensemble}[\hat{f}(x)]$.

Although the list above is not exhaustive, it does show that we can apply conformal prediction to almost any uncertainty estimates (including the ones we have already seen in the chapter). This makes the conformal prediction paradigm a very flexible toolkit to get coverage guarantees with a wide variety of problems. Even with this flexible nature, there are cases that mess with the coverage guarantees that the framework promises. And for our time series context, this is important to understand.

Exchangeability in Conformal Prediction and time series forecasting

Exchangeability is a fundamental assumption in Conformal Prediction. Exchangeability means that the data points are identically distributed and their joint probability distribution does not change when the order of the data points is changed. This concept ensures that past data points can reliably predict future data points.

Imagine a chocolate factory producing chocolates with consistent weights. If the production process is highly controlled, with the same conditions and ingredients, the weights of the chocolates are exchangeable because their order of production does not affect their weight. You can sample 100 chocolates, measure their weights, and calculate nonconformity scores based on deviations from the predicted weight. Using these scores, you can form prediction intervals for future chocolates. Because the chocolates are exchangeable, the sample distribution represents the future distribution, making prediction intervals reliable.

However, if the production process changes over time—due to machinery wear or different ingredient batches—the weights become non-exchangeable. The order of production affects the weights, making the sample distribution unrepresentative of future weights, leading to unreliable prediction intervals.

In time series data, observations are typically dependent on previous observations, violating the exchangeability assumption. For example, in a sales forecast, today's sales may influence tomorrow's sales due to trends, or the sales a year ago may influence tomorrow's sales due to seasonal effects. This dependency means that the distribution of past data does not represent the distribution of future data accurately.

But what does this mean for us? The most obvious answer is that our coverage guarantees will suffer. But can we still apply these techniques for time series? Of course we can. Empirically, the community has seen that this framework works for time series data as well, but with some loss in coverage guarantees. For most practical purposes, there shouldn't be an issue in using regular conformal prediction for time series data. In 2023, Barber et al. (Reference 12) studied this issue and derived theoretical coverage guarantees for non-exchangeable data (like time series). They defined the Coverage Gap as the difference between expected coverage $(1 - \alpha)$ and actual coverage and derived an upper bound on this gap to show how much the exchangeability assumption on the scores is violated. For this bound, they considered the scores of the calibration data with our original model, $s(z)$, and an alternate model, which was trained on the same data but after swapping one randomly selected datapoint in the training data with the test datapoint, $s(z^i)$.

The bound was shown to be directly proportional to $d_{TV}(s(z), s(z^i))$, which is the distributional distance between these two scores. In most algorithms we use, swapping one data point may not change the model drastically and therefore, we can still use conformal prediction for time series data with minimal loss in coverage.

But on the other hand, if we want to be really accurate with the prediction intervals, or if we are using a model that is particularly impacted by the swapping of a datapoint, then we would need some techniques to overcome this degradation due to a shift in distributions. There are many ways to deal with this, and it is an active area of research at the time of writing the book. There are two very simple methods that are worth mentioning here.

Weighted conformal prediction

Suppose we have slowly varying changes in the data distribution in a time series, $(x_1, x_2, \dots, x_N)$, and we are using a calibration set, $X_c = (x_{N-k}, \dots, x_N)$, taking the last k timesteps. And we are interested in predicting the test set, $X_{test} = (x_{N+1}, \dots, x_{N+H})$, where H is the horizon of forecast.

So, it stands to reason that the most recent timestep in X_c would be closest to the distribution of values we would observe in the test time period. So, what if we assign weights to the non-conformity scores in the calibration data such that the most recent time step gets higher weights, and calculates a weighted quantile instead of a regular quantile? Apparently, that's a very good idea, with some solid theoretical backing as well.

And weighing the calibration data using recency is just one of the ways we can use the weights to tackle the distribution shift. More generally, any weight schedule, $w_1, \dots, w_k, w_i \in [0,1]$ can be used here. Maybe for a strongly seasonal time series, it makes sense to use seasonal periods to define the weights, or there may be some other known criteria that makes different instances of the calibration data more or less relevant to future prediction.

Before we get into the actual mechanics of this, we need to understand what weighed quantiles are. If you are already comfortable with the concept, feel free to go ahead. If you need some intuition about what it is, I strongly advise you to check out the notebook in the chapter folder named `08-Quantiles_and_Weighted_Quantiles.ipynb`.

Now, let's come back to our Weighted Conformal Prediction method.

So, as we discussed earlier, for any normalized weight schedule, $w_i \in [0,1]$ and $\sum_{i=1}^k w_i = 1$, and calibration scores, s_i the weighted quantile can be formally defined as:

$$\hat{q} = \inf \left\{ q: \sum_{i=1}^n w_i \mathbb{I}\{s_i < q\} \geq 1 - \alpha \right\}$$

where $\inf$ is the infimum and $\mathbb{I}$ is an indicator function, which is 1 when the condition is true and 0 otherwise. In this context, the *infimum* is the smallest value of q such that the inequality holds true. This is just a more rigorous way of defining the weighted quantile that we saw in the aforementioned notebook. The rest of the process is exactly the same as before.

Practically, there are a few different ways we can use this for time series problems. For instance:

1. We can consider a moving window of length K and have a fixed weight vector of length K . Under this scheme, we will apply the weights for each point in the time series to the last K points and calculate the prediction interval for that point. These weights can be equal weights or even decayed weights, capturing the temporal element.
2. When we model multiple time series together, we can make sure the weights reflect how close another time series is to the time series we are generating the intervals for, along with the temporal context.

The bottom line is that we can be as creative as we can when coming up with the weights. The guideline is that the weight should reflect how different the calibration datapoints are from the datapoint you are generating the prediction interval for. Remember, we talked about the upper bound earlier, $d_{TV}(s(z), s(z^i))$. The weight we choose would counteract this term. When we have smaller weights for datapoints that are “farther” away from the datapoint we care about, this brings down the upper bound of the coverage gap and makes it tighter.

Now, let’s learn about another very simple modification to account for distribution shift.

Adaptive Conformal Inference (ACI)

In 2021, Gibbs et al. (Reference 13) proposed another way to deal with distribution shift (especially in time series) in an online setting. Time series data usually comes in one datapoint at a time, and this way to deal with distribution shift relies on this online aspect as it proposes to keep adjusting the prediction intervals based on the data that keeps trickling in, thus making the prediction intervals adapt to changing distributions. This method, called **Adaptive Conformal Inference (ACI)**, can be integrated with any prediction algorithm to provide robust prediction sets under non-stationary conditions.

In traditional Conformal Prediction, we have a score function, $s(x, y)$, and a quantile function, $\hat{q}$, which gives us the prediction intervals, $\hat{C}$. Note that the underlying uncertainty model, which was conformalized as $\hat{q}$, can be any way of estimating uncertainties, like quantile regression, PDF, MC Dropouts, and so on. When the data is exchangeable, the $\hat{q}$ we calculated on calibration data will hold good on future test datapoints. But when the distribution shifts, this $\hat{q}$ will also start to become less and less relevant. To address this, the authors propose regularly re-estimating these functions to align with the most recent data observations. Specifically, at each time point, t , a new score function, $s_t(\cdot)$, and a new quantile function, $\hat{q}_t(\cdot)$, are fitted based on the most recent data.

For this, they defined the *miscoverage rate*, $M_t(\alpha)$, as the probability that the true label, Y_t , lies outside the prediction intervals, $\hat{C}$, where probability is calculated over the calibration data and the test datapoint. We want the *Miscoverage rate*, $M_t(\alpha)$, to be equal to α (expected error rate). But since the data distribution is shifting, $M_t(\alpha)$ is not expected to remain constant over time, and it may not equal the target level, α . The authors hypothesize that for each time, t , there may exist an optimal coverage level, α_t^* such that the miscoverage rate, $M_t(\alpha_t^*)$, is approximately α .

To estimate this α_t^* , the authors propose a simple online update equation. This update takes into consideration the empirical miscoverage rate of the previous observations and then decreases or increases our estimate of α_t^* . Concretely, if we set $\alpha_1 = \alpha$, we can define error as:

$$err_t = \begin{cases} 1, & \text{if } Y_t \notin \hat{C}(\alpha_t), \\ 0, & \text{otherwise,} \end{cases}$$

Now, we can define the update step recursively as:

$$\alpha_{t+1} = \alpha_t + \gamma(\alpha - err_t)$$

Here, err_t , serves as an estimate of the historical miscoverage rate and $\gamma > 0$ is the step-size (a hyper-parameter; more on this later). So, when $err_t = 0$ (prediction was within the interval), $\alpha - err_t$ will be positive and thus updating α_{t+1} to be higher than α_t . This, in turn, makes the prediction interval narrower (according to the γ we have defined). With the same logic, when $err_t = 1$ (prediction was outside the interval), the prediction interval becomes wider.

A natural alternative to this update, which also takes into account the history a bit more, is using a weighted average of past timesteps:

$$\alpha_{t+1} = \alpha_t + \gamma \left(\alpha - \sum_{i=1}^t w_i \cdot err_i \right)$$

where $\{w_i\}_{1 \leq i \leq t} \in [0,1]$ is a sequence of increasing weights with $\sum_{i=1}^t w_i = 1$. Instead of looking at just the last time step for the estimate of miscoverage, this update looks at the recent history. This makes it slightly more robust, at least in theory. The paper reported no significant difference between the two strategies. One of the strategies they have used to decide the weights is:

$$w_i = \frac{0.95^{t-s}}{\sum_{s=1}^t 0.95^{t-s}}$$

They reported that the trajectories of prediction intervals that they obtained from the simple update and weighted update were almost the same, but the weighted one was considerably smoother with less local variation in α_t .

Now, let's also spend some time understanding the effect of the step-size parameter, γ . The intuition is very similar to the learning rate in deep learning models. γ decides the magnitude with which we update the α . The larger the value, the quicker the update, and vice versa. The paper also gives us an intuition that the greater the distributional shift greater the value of γ . For all their experiments, they used $\gamma = 0.005$ with a justification that they found this value to make the trajectories relatively smooth while still being large enough to allow α_t to adapt to distributional shifts. We can see this as a parameter controlling the strength of "adapting" and letting us move in the spectrum between non-adaptive intervals and strongly adaptive intervals.

Now, let's see how to apply these in practice.

Forecasting with Conformal Prediction

We didn't find any ready-to-use implementations of all the techniques we wanted to show here, especially one that has these properties:

- Complete separation of model layer and conformal prediction layer (being model-agnostic is one of the most exciting features of Conformal Prediction)

- Out-of-the box compatibility with `neuralforecast` predictions
- Time series focus
- Pedagogical ease

Therefore, we have included a file (`src/conformal/conformal_predictions.py`) with the necessary implementations that would work with `neuralforecast` forecasts and have a unified API. It is also simple enough to understand. We will go through major parts of the code, but to see how it all fits together, you should just take a look at the file.

All the methods we discussed, like Conformal Prediction for Regression, Conformalized Quantile Regression, and Conformalizing uncertainty estimates, have been coded out in the same API. Let's look at the most basic Conformal Prediction for Regression to understand the API. It can be found in the `ConformalPrediction` class in the file. The rest of the techniques inherit this class and make slight tweaks. And all these classes are coded in such a way that they take in the prediction dataframe from `neuralforecast` (or `statsforecast`) and use the same naming conventions to conformalize those predictions. In theory, any forecast that can be made into the expected format can be used with these classes. The expected columns in the format are:

- `ds`: This column should contain dates or the numerical equivalent of time.
- `y`: For train and calibration datasets, this column is necessary and it represents the actual value of that time series.
- `unique_id`: This column is the unique identifier for different time series.

In addition to these columns, we would also have a column (or multiple columns) of forecast, named accordingly.

Before we start generating conformal predictions, we also need some data and forecasts. We are using the same data that we have been using in this chapter (M4), with one additional split (calibration) created. And using the new train data, we have created these three forecasts with `level = 90`:

1. LSTM point forecast (LSTM)
2. LSTM with Quantile Regression (LSTM_QR)
3. LSTM with PDF (normal distribution) (LSTM_PDF)



Notebook alert

To follow along with the complete code, use the notebook named `09-Conformal_Techniques.ipynb` in the `Chapter17` folder.

The notebook has the entire code, but we can start from the point where we have already split the data into `Y_train_df`, `Y_calib_df`, and `Y_test_df`, and generated and stored forecasts in a dictionary, `prediction_dict`. Let's take a look at the top five rows of the prepared data frame to see what kind of data we are working with.

	unique_id	ds	y	LSTM
0	H1	653	664.0	691.085999
1	H1	654	550.0	623.127563
2	H1	655	544.0	564.952698
3	H1	656	505.0	501.971893
4	H1	657	483.0	463.498230

Figure 17.16: Top five rows of the `Y_calib_df` we are working with. This is the format that the conformal prediction classes we have coded expect

Now, let's get down to business and start creating prediction intervals.

Conformal prediction for regression

The `ConformalPrediction` class provides a structured way to calculate prediction intervals based on a chosen model's predictions from a calibration dataset. It includes the following input parameters:

- `model` (str): The name of the column with the forecast you want to conformalize. This is a required parameter.
- `level` (float): The confidence level for the prediction intervals, expressed as a percentage (e.g., 95 for a 95% confidence interval). The level must be between 1 and 100. This is a required parameter.
- `alias` (str, optional): An optional string to provide an alias for the model. This can be useful when working with multiple models or versions and you want to call the output something else other than the model. If not provided, `model` is used.

The major functions for using the class are:

- `fit(Y_calib_df)`: This method orchestrates the entire calibration process. It first calculates the calibration scores using `calculate_scores` and then determines the quantiles for each `unique_id` using `get_quantile`. The resulting quantiles (`q_hat`) are stored as an attribute of the class, making them available for subsequent prediction intervals.
- `predict(Y_test_df)`: This method applies the prediction intervals to the test data. It uses the `calc_prediction_interval` method to compute the intervals and then adds them to the `DataFrame` as new columns. The new columns are created in this format: `f"{self.alias or self.model}-{self._mthd}-lo-{self.level}"`. For example, the higher interval for Conformal Prediction using LSTM would have `LSTM-CP-hi-90` as the column name in the dataframe.

These classes are the external API. Internally, there are some methods that actually define how it's done. Let's look at the major methods in the context of regular Conformal Prediction.

The `calculate_scores` method is defined below, where we just calculate the absolute residuals using the calibration dataset as the scores:

```
def calculate_scores(self, Y_calib_df):
    Y_calib_df = Y_calib_df.copy()
    Y_calib_df["calib_scores"] = np.abs(Y_calib_df["y"] - Y_calib_df[self.
model])
    return Y_calib_df
```

The `get_quantile` method calculates the quantile using the defined α for each `unique_id`:

```
def get_quantile(self, Y_calib_df):
    def get_qhat(Y_calib_df):
        n_cal = len(Y_calib_df)
        q_level = np.ceil((n_cal + 1) * (1 - self.alpha)) / n_cal
        return np.quantile(
            Y_calib_df["calib_scores"].values, q_level, method="higher"
        )

    return Y_calib_df.groupby(
        "unique_id").apply(get_qhat).to_dict()
```

The `calc_prediction_interval` method uses the calculated `q_hat` and mean prediction to generate the prediction intervals. For regular Conformal Prediction, it goes like this:

```
def calc_prediction_interval(self, Y_test_df, q_hat):
    return (
        Y_test_df[self.model] - Y_test_df["unique_id"].map(q_hat),
        Y_test_df[self.model] + Y_test_df["unique_id"].map(q_hat),
    )
```

Now, let's use it for forecasting.

```
from src.conformal.conformal_predictions import ConformalPrediction
Y_calib_df, Y_test_df = prediction_dict['LSTM']
# Y_calib_df & Y_test_df have forecasts in column named "LSTM"
cp = ConformalPrediction(model="LSTM", level=level)
# Calibrating the model
cp.fit(Y_calib_df=Y_calib_df)
# Generating Prediction intervals
Y_test_df_cp = cp.predict(Y_test_df=Y_test_df)
```

The generated dataframe (Figure 17.15) with prediction intervals will have two columns—`LSTM-CP-lo-90` and `LSTM-CP-hi-90` for the lower and upper prediction interval, respectively. CP is the method tag we have assigned the conformal prediction class.

We can check the method name of any object as follows:

```
>> cp.method_name
'Vanilla Conformal Prediction (CP)'
```

Let's see what the forecast dataframe looks like (CP is the method tag we have assigned the conformal prediction class. We can check the method name of any object by doing `cp.method_name`):

unique_id	ds	y	LSTM	LSTM-CP-lo-90	LSTM-CP-hi-90
H1	701	619.0	617.814331	544.686768	690.941895
H1	702	565.0	554.244019	481.116455	627.371582
H1	703	532.0	507.415466	434.287903	580.543030
H1	704	495.0	459.968445	386.840881	533.096008
H1	705	481.0	435.267883	362.140320	508.395447

Figure 17.17: The generated dataframe with prediction intervals

Now, we also have to calculate the coverage and average length of the intervals to assess how these prediction intervals are. We use the same methods we used earlier to do that. Instead of looking at the performance of each method, let's save the discussion for the end and look at creating the prediction intervals for now.

So, let's move on to the next technique.

Conformalized Quantile Regression

The first condition of applying this method of conformal prediction is that there should already be a set of prediction intervals from the underlying Quantile Regression. Therefore, we use the LSTM_QR model we trained here.

The main difference between vanilla Conformal Prediction and CQR is the way the scores are calculated and the prediction intervals. So, we can inherit `ConformalPrediction` and just redefine these two methods.

Let's look at the `calculate_scores` method.

```
def calculate_scores(self, Y_calib_df):
    Y_calib_df = Y_calib_df.copy()
    lower_bounds = Y_calib_df[self.lower_quantile_model]
    upper_bounds = Y_calib_df[self.upper_quantile_model]
    Y_calib_df["calib_scores"] = np.maximum(
        lower_bounds - Y_calib_df["y"], Y_calib_df["y"] - upper_bounds
    )
    return Y_calib_df
```

We have just implemented the formula we saw earlier. `self.lower_quantile_model` and `self.upper_quantile_model` are the column names of the already-generated intervals from CQR.

Now, we also need to define the `calc_prediction_interval` method.

```
def calc_prediction_interval(self, Y_test_df, q_hat):
    return (
        Y_test_df[self.lower_quantile_model] - Y_test_df["unique_id"].
    map(q_hat),
        Y_test_df[self.upper_quantile_model] + Y_test_df["unique_id"].
    map(q_hat),
    )
```

`q_hat` is a dictionary with the quantiles calculated for each `unique_id`. So, all we do here is take the existing prediction interval from CQR and adjust it by mapping the quantile using the `unique_id` in the input dataframe.

Now, let's use this for forecasting. The API is exactly the same as before.

```
from src.conformal.conformal_predictions import ConformalizedQuantileRegression
Y_calib_df, Y_test_df = prediction_dict['LSTM_QR']
# Forecast in column "LSTM_QR"
cp = ConformalizedQuantileRegression(model="LSTM_QR", level=level)
cp.fit(Y_calib_df=Y_calib_df)
Y_test_df_cqr = cp.predict(Y_test_df=Y_test_df)
```

Now, let's look at the third technique we discussed.

Conformalizing uncertainty estimates

If you remember the discussion we had earlier, to use this technique, we need an estimate of uncertainty that can be further conformalized. This is why we picked one of the other techniques we used earlier, PDF. But we can also do this with the MC Dropout just as easily. All we need is the standard deviation or something similar that captures the uncertainty at each data point.

We are using the `LSTM_PDF` forecast that we generated earlier for this purpose. Although the model predicts the mean and standard deviation of the normal distribution, it is used internally to generate the prediction intervals. So, the output from the PDF model we defined earlier would be the prediction intervals, but we want the standard deviation. Fear not. We know the prediction intervals were created with the normal distribution. So, it's not hard to re-engineer the standard deviation from the prediction intervals.

$$\text{Lower Bound} = \mu - Z \cdot \sigma$$

$$\text{Upper Bound} = \mu + Z \cdot \sigma$$

Using basic math, we can derive:

$$\sigma = \frac{\text{Upper Bound} - \text{Lower Bound}}{Z}$$

And Z is very straightforward to get. We can use `scipy.stats.norm` for this. Below is a method that will get you the standard deviation from the prediction intervals (do keep in mind that this is only for PDFs created using a normal distribution).

```
from scipy.stats import norm

def calculate_standard_deviation(upper_bound, point_prediction, confidence_
level):
    # Calculate the Z-value from the confidence level
    z_value = norm.ppf((1 + confidence_level) / 2)

    # Calculate the standard deviation
    sigma = (upper_bound - point_prediction) / z_value

    return sigma

def reverse_engineer_sd(X, model_tag, level):
    X["std"] = calculate_standard_deviation(
        X[f"{model_tag}-hi-{level}"], X[model_tag], level / 100
    )
    return X
```

Now, we add this to our `Y_calib_df` and `Y_test_df`.

```
Y_calib_df = reverse_engineer_sd(Y_calib_df, "LSTM_Normal", level)
Y_test_df = reverse_engineer_sd(Y_test_df, "LSTM_Normal", level)
```

Now, let's look at how we can define the class. We need additional information in here that we didn't need earlier—the column name of the uncertainty estimate. So, we define our new class (still inheriting `ConformalPrediction`) as below:

```
class ConformalizedUncertaintyEstimates(ConformalPrediction):
    def __init__(
        self,
        model: str,
        uncertainty_model: str,
        level: Optional[float] = None,
        alias: str = None,
    ):
        super().__init__(model, level, alias)
```



```

self.method = "Conformalized Uncertainty Intervals"
self._mthd = "CUE"
self.uncertainty_model = uncertainty_model

```

We define an additional parameter, `uncertainty_model`, and pass on the other parameters to the parent class.

Now, it's pretty straightforward. We need to define how the scores are calculated:

```

def calculate_scores(self, Y_calib_df):
    Y_calib_df = Y_calib_df.copy()
    uncertainty = Y_calib_df[self.uncertainty_model]
    Y_calib_df["calib_scores"] = (
        np.abs(Y_calib_df["y"] - Y_calib_df[self.model]) / uncertainty
    )
    return Y_calib_df

```

And the `calc_prediction_interval` method:

```

def calc_prediction_interval(self, Y_test_df, q_hat):
    uncertainty = Y_test_df[self.uncertainty_model]
    return (
        Y_test_df[self.model] - uncertainty * Y_test_df["unique_id"].map(q_hat),
        Y_test_df[self.model] + uncertainty * Y_test_df["unique_id"].map(q_hat),
    )

```

That's it. Now, we have a new class that conformalizes uncertainty estimates. Let's use it to get the forecast for the dataset we have been working with.

```

from src.conformal.conformal_predictions import
ConformalizedUncertaintyEstimates
# We have saved uncertainty estimates in "std"
cp = ConformalizedUncertaintyEstimates(model="LSTM_Normal", uncertainty_
model="std", level=level)
cp.fit(Y_calib_df=Y_calib_df)
Y_test_df_pdf = cp.predict(Y_test_df=Y_test_df)

```

Now, let's also look at the two techniques we saw, which were more suited for time series problems where there is a distribution shift.

Weighted conformal prediction

We saw earlier that weighted conformal prediction was just about applying the right kind of weights to the calibration data such that the points similar to the test point get more weight than the dissimilar ones. And the key difference occurs only in the way the quantiles are calculated.

What this means is that we can use any conformal prediction technique underneath, but instead of calculating a simple quantile, we need to calculate a weighted one. So, from an implementation perspective, we can look at this class as a wrapper class around other techniques we have defined and convert them into weighted conformal predictions.

Although the weighted conformal prediction can be implemented in many ways, taking in different kinds of weights (across time, across unique_id, and so on), we are going to implement a simpler look-back window-based weighted conformal prediction. We choose the last K timesteps and calculate the weighted quantile on these K scores using the weights given. The weights can either be simple uniform weights across all K steps, or have decayed weights giving the highest weightage to the most recent score. They can even have a completely custom weight.

So, let's define the `__init__` of the class as follows:

```
class WeightedConformalPredictor:
    def __init__(
        self,
        conformal_predictor: ConformalPrediction,
        K: int,
        weight_strategy: str,
        custom_weights: list = None,
        decay_factor: float = 0.5,
    ):
        ...
```

Here, K is the window, and `conformal_predictor` is the underlying conformal prediction class we should be using (this should be one of the three classes we have defined). We can define the weight strategy to be either uniform, decay, or custom for uniform weights, decayed weights, or custom weights, respectively. `decay_factor` decides how fast the decay is applied to the weights for the decayed weighting strategy, and `custom_weights` lets you specify exactly the weight on these K timesteps.

While we won't look at the entire code in the text here, we will go through the usual suspects so that you can understand what's happening. But I would definitely urge you to take some time to digest the code in the file.

First up, we have our `fit` method. In this method, we just use the score calculation of the underlying conformal predictor and store the calibration dataframe.

```
def fit(self, Y_calib_df):
    self.calib_df = self.conformal_predictor.calculate_scores(
        Y_calib_df.sort_values(["unique_id", "ds"])
    )
```

Now, let's look at the main parts of the `predict` method.

```
def predict(self, Y_test_df):
    # Groupby unique_id
```

```

...
# Calculate quantiles for each unique_id
self.q_hat = {}
for unique_id, group in grouped_calib:
    # Take the last K timesteps
    group = group.iloc[-self.K :]
    scores = group["calib_scores"].values
    # Calculate weights based on the last K timesteps
    total_timesteps = len(scores)
    weights = self._calculate_weight(total_timesteps)
    normalized_weights = weights / weights.sum()
    # Calculate quantile for the current unique_id
    quantile = self.get_weighted_quantile(
        scores, normalized_weights, self.conformal_predictor.alpha
    )
    self.q_hat[unique_id] = quantile

# Calculate prediction intervals using the underlying conformal predictor's
method
lo, hi = self.conformal_predictor.get_prediction_interval_names()
Y_test_df[lo], Y_test_df[hi] = (
    self.conformal_predictor.calc_prediction_interval(Y_test_df, self.q_hat)
)
return Y_test_df

```

Let's now take one of the methods we saw earlier and apply the weighted conformal predictor wrapper on it. For our example, let's choose the simple ConformalPrediction. Let's see how we can use this class:

```

from src.conformal.conformal_predictions import WeightedConformalPredictor
Y_calib_df, Y_test_df = prediction_dict['LSTM']
# Defining an underlying conformal predictor
cp = ConformalPrediction(model="LSTM", level=level)
# using the defined conformal predictor in weighted version
weighted_cp = WeightedConformalPredictor(
    conformal_predictor=cp,
    K=50,
    weight_strategy="uniform",
)
weighted_cp.fit(Y_calib_df=Y_calib_df)
Y_test_df_wcp = weighted_cp.predict(Y_test_df=Y_test_df)

```

This would create the prediction intervals with the tag CP_wtd. We can always check the tag by doing `weighted_cp.method_name`.

Now, there is one small shortcoming with the implementation. Although we are considering the temporal order in the scores, we still have a fixed calibration set. So, until we “re-fit” or calibrate with the latest data points, we will still be working on the same calibrated dataset. So, if you think about it, this should ideally be applied in an online way where each time we predict the new timestep, the previous timestep (with actual value) should be added to the calibration data. We have also provided an alternative implementation that is able to do this in an online fashion. We won’t go into details of the implementation because the core logic is the same, but the API is different, making it possible to update the calibration data. The full implementation can be found in the `OnlineWeightedConformalLPredictor` class in the file.

Let’s see how it can be used. First, we define the setup, initialize the class, and fit the calibration.

```
from src.conformal.conformal_predictions import
OnlineWeightedConformalPredictor
cp = ConformalPrediction(model="LSTM", level=level)
online_weighted_cp = OnlineWeightedConformalPredictor(
    conformal_predictor=cp,
    K=50,
    weight_strategy="uniform",
)
online_weighted_cp.fit(Y_calib_df=Y_calib_df)
joblib.dump(online_weighted_cp, "path/to/saved/file.pkl")
```

Now, during inference, we can do something like this for each timestep:

```
# Loading the saved model
online_weighted_cp = joblib.load("path/to/saved/file.pkl")
# current timestep data = current
# past timestep actuals = last_timestep_actuals
prediction = online_weighted_cp.predict_one(current_test)
# updating the calibration data using the last timestep actuals
online_weighted_cp.update(last_timestep_actuals)
```

For our special case where we are evaluating on test data where we know the actuals, there is another method that does similar online prediction for the data: `offline_predict`.

```
Y_test_df_wcpo = online_weighted_cp.offline_predict(Y_test_df=Y_test_df)
```

Now, let’s look at one last method.

Adaptive Conformal Inference

Lastly, we look at the Adaptive Conformal Inference. This can also be implemented as a wrapper over other conformal prediction methods because this technique involves updating α such that coverage is maintained in spite of the shift in distribution. And because of the nature of the technique, we can only apply this in an online manner, i.e., updating alpha at every time step with the available data. So, this will have the same API as the `OnlineWeightedConformalPredictor` we saw earlier.

The full class is available in `src/conformal/conformal_predictions.py`, but here, we will look at some main parts so that you understand it. Let's take a look at the `__init__` function first:

```
class OnlineAdaptiveConformalInference:
    def __init__(
        self,
        conformal_predictor: ConformalPrediction,
        gamma: float = 0.005,
        update_method: str = "simple",
        momentum_bw: float = 0.95,
        per_unique_id: bool = True,
    ):
        ...
```

Similar to `WeightedConformalPredictor`, we take in an underlying conformal predictor (`conformal_predictor`). In addition, we have `gamma`, which is the step size (γ), `update_method`, which can either be `simple` (taking only the last timestep for update) or `momentum` (taking the running average of the trajectory of errors). Finally, we can also define `momentum_bw`, which is the momentum backweight that is used to calculate the weighted average of past errors. A higher momentum (e.g., 0.95) makes the trajectory smoother, having the effect of past miscoverage rates decay slower. Going to the other extreme (0.05) makes the weighted average more reactive and closer to the “simple” method. Lastly, we also have a parameter to do the error calculation for each `unique_id` separately or pool all errors together.

As usual, we have a `fit` method that uses a calibration dataset to calculate the scores and keep them ready. We can also see this as a warm-up period in an online implementation. The α update will use the calibration data as an initial history to start off.

```
def fit(self, Y_calib_df):
    """
    Fit the conformal predictor model with calibration data.
    """
    self.calib_df = self.conformal_predictor.calculate_scores(Y_calib_df)
    self.scores_by_id = (
        self.calib_df.groupby("unique_id")["calib_scores"].apply(list).to_
dict()
    )
    # Some more code to initialize necessary data structures
    ...
    return self
```

Now, let's see the `predict_one` method, which predicts one timestep.

```
def predict_one(self, current_test):
    unique_ids = current_test["unique_id"].unique()
```

```

predictions = []

for unique_id in unique_ids:
    group_scores = self.scores_by_id.get(unique_id, [])
    if group_scores:
        # Determine the appropriate alpha to use
        alpha = (
            self.alphat[unique_id]
            if self.per_unique_id
            else self.alphat_global
        )

        # Calculate quantile for the current unique_id
        self.q_hat = {
            unique_id: np.quantile(group_scores, 1 - alpha,
method="higher")
        }

        # Calculating prediction intervals using conformal_predictor
        ...
        current_test[lo] = lower.values
        current_test[hi] = upper.values
        # Storing most recent prediction
        ...
    # Collecting and returning concatenated predictions
    ...

```

The code is well-commented and you should be able to understand it. For each `unique_id`, we get the scores, get the appropriate α , calculate the quantile, use this information to calculate the prediction interval using the underlying conformal predictor, and store the prediction for later use.

Now, we look at the update method, which we can use once the actual value for the timestep becomes available.

```

def update(self, new_data):
    new_scores = self.conformal_predictor.calculate_scores(new_data)
    for unique_id, score in zip(
        new_scores["unique_id"], new_scores["calib_scores"]
    ):
        # Updating score trajectory with new score
        ...
        # Retrieve stored predictions and calculate adapt_err
        if self.per_unique_id:

```

```

        lower, upper = self.predictions[unique_id]
        actual_y = new_data.loc[new_data["unique_id"] == unique_id, "y"].
values[0]
        adapt_err = int(actual_y < lower or actual_y > upper)
        # Update alpha updates the alpha using simple or momentum method
        self.update_alpha(unique_id, adapt_err)
    else:
        # Do the same update at a global error-pooled way

```

Let's see how it can be used. First, we define the setup, initialize the class, and fit the calibration.

```

from src.conformal.conformal_predictions import
OnlineAdaptiveConformalInference
cp = ConformalPrediction(model="LSTM", level=level)
aci_cp = OnlineAdaptiveConformalInference(
    conformal_predictor=cp,
    gamma=0.005,
    update_method="simple",
)
aci_cp.fit(Y_calib_df=Y_calib_df)
joblib.dump(aci_cp, "path/to/saved/file.pkl")

```

Now, during inference, we can do something like below for each timestep:

```

# Loading the saved model
aci_cp = joblib.load("path/to/saved/file.pkl")
# current timestep data = current
# past timestep actuals = last_timestep_actuals
prediction = aci_cp.predict_one(current_test)
# updating the calibration data using the last timestep actuals
aci_cp.update(last_timestep_actuals)

```

For our special case where we are evaluating on test data where we know the actuals, there is another method that does similar online prediction for the data: `offline_predict`.

```
Y_test_df_aci = aci_cp.offline_predict(Y_test_df=Y_test_df)
```

Now that we have seen all the techniques in action, let's take a look at how they have been performing.

Evaluating the results

If you have been following the notebooks, you would know that we have been calculating coverage and average length for each of these techniques. To recap, coverage measures the percentage of time the actual value fell between the intervals we predicted and average length measures the average width of the intervals we predicted. For `level=90`, we expect the coverage to be around 90% or 0.9 and the average length to be as small as possible, without compromising on coverage.

Let's take a look at the following figures summarizing the coverage and average length:

	LSTM-CP_Cov	LSTM_QR_Cov	LSTM_QR-CQR_Cov	LSTM_Normal_Cov	LSTM_Normal-CUE_Cov	LSTM-CP_Wtd_Cov	LSTM-CP_Wtd_O_Cov	LSTM-CP_ACI_Cov
H1	0.96	1.00	1.00	1.00	1.00	0.92	0.92	0.92
H10	0.94	0.96	0.98	1.00	0.88	0.92	0.90	0.90
H100	0.94	1.00	0.96	0.94	0.94	0.90	0.92	0.90
H101	1.00	1.00	1.00	1.00	1.00	1.00	0.96	1.00
H102	1.00	0.81	1.00	0.85	1.00	1.00	0.98	0.98
H103	0.96	0.98	1.00	1.00	1.00	0.94	0.83	0.88
H104	0.96	0.90	1.00	1.00	1.00	0.92	0.90	0.90
H105	1.00	0.94	1.00	0.94	1.00	1.00	1.00	1.00

Figure 17.18: Coverage for all the conformal techniques (for each unique_id), colored according to how close they are to 0.9 (we had set level=90)

	LSTM-CP_AvgL	LSTM_QR_AvgL	LSTM_QR-CQR_AvgL	LSTM_Normal_AvgL	LSTM_Normal-CUE_AvgL	LSTM-CP_Wtd_AvgL	LSTM-CP_Wtd_O_AvgL	LSTM-CP_ACI_AvgL
H1	146.26	608.68	565.17	366.44	183.43	121.92	122.27	122.88
H10	45.08	113.68	115.47	68.14	42.18	41.29	39.94	38.92
H100	932.07	1385.58	787.55	841.77	979.93	691.05	752.04	700.08
H101	846.41	800.34	870.21	520.64	881.61	746.22	586.37	675.34
H102	1707.86	1427.43	1660.76	826.26	1694.48	1524.95	1213.60	1224.89
H103	18879.27	55556.50	76969.47	32530.45	30236.44	17624.53	16599.94	17041.39
H104	678.20	1147.59	1433.89	667.14	688.96	615.05	587.81	587.54
H105	1008.87	1077.13	1416.38	614.02	932.22	931.00	801.13	858.25

Figure 17.19: Average length for all the conformal techniques (for each unique_id), colored according to how small they are

Here's a quick recap of the legend to understand the different columns, which are just a combination of these tags based on the application:

- LSTM: Underlying model that we trained to get point prediction
- LSTM_QR: Underlying Quantile Regression model we trained
- LSTM_Normal: Underlying PDF model we trained using Normal distribution assumption
- CP: Regular Conformal Prediction
- CQR: Conformalized Quantile Regression
- CUE: Conformalizing uncertainty Estimates
- CP_Wtd: Weighted corrections over Conformal Prediction
- CP_Wtd_O: Online Weighted corrections over Conformal Prediction
- ACI: Adaptive Conformal Inference

Right off the bat, we can see that the techniques that corrected for distribution shift are performing the best. There are more “greens” on the right side of both tables (closer to 0.9 coverage and lower average lengths). Remember that we started with regular conformal predictions and then corrected them for distributional shifts. We can note that for most `unique_id`, regular **Conformal Prediction** has wider than necessary intervals for `level = 90`. The coverages are greater than 0.9, in most cases 1.0. But both **Weighted Correction (CP_Wtd)** and **Adaptive Conformal Inference (ACI)** made the prediction intervals narrower and got closer to our expected level. There is no clear winner between the two and that has to be evaluated on your dataset.

And this concludes our discussion about Probabilistic Forecasts. We hope that with this, you can steer into this lesser-known territory with confidence and deliver more value to whoever you are forecasting for.

Now, let’s take a very high-level look at some niche topics in time series forecasting that get very little attention, but are quite relevant in many domains.

Road less traveled in time series forecasting

In the spirit of Robert Frost’s *The Road Not Taken*, this section explores the lesser-known, yet highly impactful, techniques in time series forecasting. Just as Frost chooses a path that is less traveled, we delve into niche methods that, while not mainstream, offer unique insights and potential breakthroughs in various domains.

Intermittent or sparse demand forecasting

Intermittent time series forecasting handles data with sporadic, irregular events, often seen in retail, where products may have infrequent sales. It’s crucial for managing inventory and avoiding stockouts or overstocking, especially for slow-moving items. Traditional methods struggle with these patterns because of the fact that for most of these items, the expectation of demand falls to zero, but specialized techniques are needed to improve accuracy, making them essential for retail forecasting.

Here, we will quickly name-drop a few alternative algorithms that were designed for intermittent forecasting and where you can find the implementations for it.

- **Croston and its friends:** The Croston model, developed in 1972, is used to forecast intermittent demand by separately estimating two components: the demand rate (when sales occur) and the time interval between sales, ignoring periods with zero sales. These estimates are combined to forecast future demand, making it useful for industries with infrequent sales. However, the model doesn’t account for external factors or changes in demand patterns, which can affect its accuracy in more complex situations. Although this was a method developed all the way back in 1972, it has stayed with us in many forms and led to many modifications and extensions. In *Nixtla’s statsforecast*, we can find `CrostonClassic`, `CrostonOptimized`, `CrostonSBA`, and `TSB` as four models that can be used just like the other models we saw in *Chapter 4, Setting a Strong Baseline Forecast*.

- **ADIDA:** The **A**ggregate-**D**isaggregate **I**ntermittent **D**emand **A**pproach (ADIDA) uses **S**imple **E**xponential **S**moother (SES) combined with temporal aggregation to forecast intermittent demand. The method starts by aggregating demand data into non-overlapping time buckets of a size equal to the **mean inter-demand interval (MI)**. SES is then applied to these aggregated values to generate forecasts, which are subsequently disaggregated back to the original time scale, providing demand predictions for each time period. This model is available in statsforecast as ADIDA.
- **IMAPA:** The **I**ntermittent **M**ultiple **A**ggregation **P**rediction **A**lgorithm (IMAPA) forecasts intermittent time series by aggregating values at regular intervals and applying optimized **S**imple **E**xponential **S**moother (SES) to predict future values. IMAPA is efficient, robust to missing data, and effective across various intermittent time series, making it a practical and easy-to-implement choice. This model is available in statsforecast as IMAPA.

Interpretability

We directed you toward a few interpretability techniques for machine learning models back in *Chapter 10, Global Forecasting Models*. While some of them, such as SHAP and LIME, can still be applied to deep learning models, none of them consider the temporal aspect by design. This is because all those techniques were developed for more general purposes, such as classification and regression. That being said, there has been some work in the interpretability of DL models and time series models. Here, I'll list a few promising papers that tackle the temporal aspect head-on:

- **TimeSHAP:** This is a model-agnostic recurrent explainer that builds upon **KernelSHAP** and extends it to the time series domain. Research paper: <https://dl.acm.org/doi/10.1145/3447548.3467166>. GitHub: <https://github.com/feedzai/timeshap>.
- **SHAPTime:** This technique uses the Shapley Value to provide stable explanations in the temporal dimension, improving forecasting performance. ShapTime's model-agnostic nature allows it to be deployed on any forecasting model at a lower cost, demonstrating significant performance improvements across various datasets. Research paper: https://link.springer.com/chapter/10.1007/978-3-031-47721-8_45. GitHub: <https://github.com/Zhangyuyi-0825/ShapTime>.
- **Instance-wise Feature Importance in Time (FIT):** This is an interpretability technique that relies on the distribution shift between the predictive distribution and a counterfactual, where all but the feature under inspection is unobserved. Research paper: <https://proceedings.neurips.cc/paper/2020/file/08fa43588c2571ade19bc0fa5936e028-Paper.pdf>. GitHub: https://github.com/sanatonek/time_series_explainability.
- **Time Interpret:** `time_interpret` is a library extending Captum, focused on temporal data, providing feature attribution methods to explain predictions from any PyTorch model. It includes synthetic and real-world time series datasets, various PyTorch models, and evaluation methods, with some components also applicable to language model predictions. Research paper: <https://arxiv.org/abs/2306.02968>. GitHub: https://github.com/josephenguehard/time_interpret.

While this is not an exhaustive list, these are a few works that we feel are important and promising. This is an area of active research, and new techniques will come up as time goes on.

Cold-start forecasting

Cold-start forecasting addresses the challenge of predicting demand for products with no historical data, a common issue in industries like retail, manufacturing, and consumer goods. It arises when launching new products, onboarding brands, or expanding into new regions. Traditional statistical forecasting models like ARIMA or exponential smoothing will not be able to tackle this as they require historical data, a good amount of it as well.

But all hope is not lost. We do have some ways to handle such cases:

- **Manual Substitution Mapping:** If we know that a new product is substituting another product, we can do manual alignment, which moves the history of the other product to the new product and uses regular techniques to forecast.
- **Global Machine Learning Models:** We saw how we can model multiple time series using global models in *Chapter 10*. There, we used some constructed features like lags, rolling aggregations, and so on. But if we train such a model without such features that rely on history and with features characterizing the item (item features that will enable us to cross-learn with other longer time series), we can utilize cross-learning to learn the forecast from similar items. But this works best for new launches that substitute another product.
- **Launch Profile Models:** If the new product is brand-new, we can use a very similar setup to Global Machine Learning Models but with a small tweak. We can convert our time series into a date/time agnostic manner and consider each time series to start from the date of launch. For instance, the first timestep after the launch of every product can be 1, the next can be 2, and so on. Once we convert all our time series in this manner, we have a dataset that considers how a new item is launched and ramps up. And this can be used to train models that consider the initial ramp-up during launch.
- **Foundational Time Series Models:** Foundation models for time series leverage large-scale pre-trained models to capture generalized patterns across various time series tasks. These models are highly effective in applications like cold start forecasting, where little or no historical data is available. By using foundational models, practitioners can apply pre-trained knowledge to new scenarios, improving accuracy in forecasting tasks across industries such as retail, healthcare, and finance. The models offer flexibility, allowing fine-tuning or zero-shot applications, making them particularly useful for complex, sparse, or intermittent data. Many foundational models for time series are already available for use—some are commercial and some others are open source. This is a nice survey of the foundational models at the time of writing the book: *Foundation Models for Time Series Analysis: A Tutorial and Survey*: <https://arxiv.org/abs/2403.14735>. The performance of such models is average at best, but in the absence of any data (cold-start), this presents as a good option to try out. A few popular ways of doing this practically are:
 - *TimeGPT* (Commercial) from Nixtla: <https://nixtlaverse.nixtla.io/nixtla/docs/getting-started/introduction.html>
 - *Chronos* from Amazon: <https://auto.gluon.ai/stable/tutorials/timeseries/forecasting-chronos.html>

- *Moirai* from Salesforce: <https://huggingface.co/Salesforce/moirai-1.0-R-large>
- *TimesFM* from Google: <https://github.com/google-research/timesfm>

Hierarchical forecasting

Hierarchical forecasting deals with time series that can be disaggregated into nested levels, such as product categories or geographic regions. These structures require forecasts that maintain coherence, meaning predictions at lower levels should sum up to higher levels, reflecting the aggregation. Grouped time series, which combine different hierarchies (e.g., product type and geography), add complexity. The goal is to produce consistent, accurate forecasts that align with the data's natural aggregation, making hierarchical forecasting essential for businesses managing large collections of time series across multiple dimensions.

There are some techniques of disaggregating, aggregating, or reconciling all forecasts so that they add up in a logical manner. *Chapter 10* from Rob Hyndman's free Bible of time series forecasting (*Forecasting: Principles and Practice*) talks about this at length and is a very good resource to get up to speed (fair warning: it's quite math-intensive). You can find the chapter here: <https://otexts.com/fpp2/hierarchical.html>. For a more practical approach, you can check out Nixtla's hierarchicalforecast library here: <https://nixtlaverse.nixtla.io/hierarchicalforecast/index.html>.

And with that, one of the longest chapters of the book comes to an end. Congrats on making it and digesting all the information. Do feel free to use the notebooks and play around with the code, different options, and so on to get a better grasp of what's happening.

Summary

In this chapter, we explored different techniques for generating probabilistic forecasts like Probability Density Functions, Quantile functions, Monte Carlo Dropout, and Conformal Prediction. We deep-dived into each of them and learned how to apply them to real data. For Conformal Predictions, an actively researched field, we learned different ways to conformalize prediction intervals with different underlying mechanisms, like Conformalized Quantile Regression, conformalizing uncertainty estimates, and so on. And finally, we saw a few tweaks to make conformal prediction work even better for time series problems.

To wrap it up, we also looked at a few topics that are on the road less traveled, like intermittent demand forecasting, interpretability, cold-start forecasting, and hierarchical forecasting.

In the next part of this book, we will look at a few mechanics of forecasting, such as multi-step forecasting, cross-validation, and evaluation.

References

The following are the references for this chapter:

1. Tony Duan, A. Avati, D. Ding, S. Basu, A. Ng, and Alejandro Schuler. (2019). *NGBoost: Natural Gradient Boosting for Probabilistic Prediction*. International Conference on Machine Learning. <https://proceedings.mlr.press/v119/duan20a/duan20a.pdf>.

2. Y. Gal and Zoubin Ghahramani. (2015). *Dropout as a Bayesian Approximation: Representing Model Uncertainty in Deep Learning*. International Conference on Machine Learning. <https://proceedings.mlr.press/v48/gal16.html>.
3. Valentin Flunkert, David Salinas, and Jan Gasthaus. (2017). *DeepAR: Probabilistic Forecasting with Autoregressive Recurrent Networks*. International Journal of Forecasting. <https://www.sciencedirect.com/science/article/pii/S0169207019301888>.
4. Koenker, Roger. (2005). *Quantile Regression*. Cambridge University Press. pp. 146–7. ISBN 978-0-521-60827-5. <http://www.econ.uiuc.edu/~roger/research/rq/QRJEP.pdf>.
5. Spyros Makridakis, Evangelos Spiliotis, Vassilios Assimakopoulos. (2020). *The M4 Competition: 100,000 time series and 61 forecasting methods*. International Journal of Forecasting. <https://www.sciencedirect.com/science/article/pii/S0169207019301128>.
6. Loic Le Folgoc and Vasileios Baltatzis and Sujal Desai and Anand Devaraj and Sam Ellis and Octavio E. Martinez Manzanera and Arjun Nair and Huaqi Qiu and Julia Schnabel and Ben Glocker. (2021). *Is MC Dropout Bayesian?*. arXiv preprint arXiv: Arxiv-2110.04286. <https://arxiv.org/abs/2110.04286>.
7. Nicolas Dewolf and Bernard De Baets and Willem Waegeman. (2023). *Valid prediction intervals for regression problems*. Artif. Intell. Rev. <https://doi.org/10.1007/s10462-022-10178-5>.
8. V. Vovk, A. Gammerman, and G. Shafer. (2005). *Algorithmic learning in a random world*. Springer. <https://link.springer.com/book/10.1007/b106715>.
9. Anastasios N. Angelopoulos and Stephen Bates. (2021). *A Gentle Introduction to Conformal Prediction and Distribution-Free Uncertainty Quantification*. arXiv preprint arXiv: Arxiv-2107.07511. <https://arxiv.org/abs/2107.07511>.
10. Harris Papadopoulos, Kostas Proedrou, Volodya Vovk, and Alex Gammerman. (2002). *Inductive confidence machines for regression*. Machine Learning: ECML 2002. ECML 2002. Lecture Notes in Computer Science(), vol 2430. Springer, Berlin, Heidelberg. https://doi.org/10.1007/3-540-36755-1_29.
11. Romano, Yaniv and Patterson, Evan and Candes, Emmanuel. (2019). *Conformalized Quantile Regression*. Advances in Neural Information Processing Systems. https://proceedings.neurips.cc/paper_files/paper/2019/file/5103c3584b063c431bd1268e9b5e76fb-Paper.pdf.
12. R. Barber, E. Candès, Aaditya Ramdas, and R. Tibshirani. (2022). *Conformal prediction beyond exchangeability*. Annals of Statistics. <https://projecteuclid.org/journals/annals-of-statistics/volume-51/issue-2/Conformal-prediction-beyond-exchangeability/10.1214/23-AOS2276.full>.
13. Gibbs, Isaac and Candes, Emmanuel. (2021). *Adaptive Conformal Inference Under Distribution Shift*. Advances in Neural Information Processing Systems. https://proceedings.neurips.cc/paper_files/paper/2021/file/0d441de75945e5acbc865406fc9a2559-Paper.pdf.

Further reading

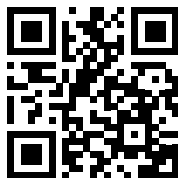
To learn more about the topics that were covered in this chapter, take a look at the following resources.

- *The Gradient Boosters VI(B): NGBoost*: <https://deep-and-shallow.com/2020/06/27/the-gradient-boosters-vib-ngboost/>
- *Dive into Deep Learning, Chapter 5.6*: https://d2l.ai/chapter_multilayer-perceptrons/dropout.html
- *Bayesian Inference* by Marco Taboga: <https://www.statlect.com/fundamentals-of-statistics/Bayesian-inference>
- *Seeing Theory: Bayesian Inference*: <https://seeing-theory.brown.edu/bayesian-inference/index.html>
- *A Tutorial on Sparse Gaussian Processes and Variational Inference* by Felix Leibfried et al.: <https://arxiv.org/pdf/2012.13962>
- *A Gentle Introduction to Conformal Prediction and Distribution-Free Uncertainty Quantification* by Anastasios N. Angelopoulos and Stephen Bates. (2021): <https://arxiv.org/abs/2107.07511>

Join our community on Discord

Join our community's Discord space for discussions with authors and other readers:

<https://packt.link/mts>



Part 4

Mechanics of Forecasting

In this last part, we cover a few concepts that are essential for creating an industry-ready forecasting system. We discuss rarely talked about concepts such as multi-step forecasting and delve into the details of evaluating a forecast.

This part comprises the following chapters:

- *Chapter 18, Multi-Step Forecasting*
- *Chapter 19, Evaluating Forecast Errors—A Survey of Forecast Metrics*
- *Chapter 20, Evaluating Forecasts—Validation Strategies*

18

Multi-Step Forecasting

In the previous parts, we covered some basics of forecasting and different types of modeling techniques for time series forecasting. However, a complete forecasting system is not just the model. There are a few mechanics of time series forecasting that make a lot of difference. These topics cannot be called *basics* because they require a nuanced understanding of the forecasting paradigm, and that is why we didn't cover these upfront.

Now that you have worked on some forecasting models and are familiar with time series, it's time to get more nuanced in our approach. Most of the forecasting exercises we have done throughout the book focus on forecasting the next timestep. In this chapter, we will look at strategies to generate multi-step forecasting—in other words, how to forecast the next H timesteps. In most practical applications of forecasting, we have to forecast multiple timesteps ahead, and being able to handle such cases is an essential skill.

In this chapter, we will cover these main topics:

- Why multi-step forecasting?
- Standard notation
- Recursive strategy
- Direct strategy
- Joint strategy
- Hybrid strategies
- How to choose a multi-step forecasting strategy

Why multi-step forecasting?

A multi-step forecasting task consists of forecasting the next H timesteps, $y_{t+1}, \dots, y_{t+H}$, of a time series, $y_1, \dots, y_t$, where $H > 1$. Most real-world applications of time series forecasting demand multi-step forecasting, whether it is the energy consumption of a household or the sales of a product. This is because forecasts are never created to know what will happen in the future but, rather, to enable us to take action using the visibility we get.

To effectively take any action, we would want to know the forecast a little ahead of time. For instance, the dataset we have used throughout the book is about the energy consumption of households, logged every half an hour. If the energy provider wants to plan its energy production to meet customer demand, the next half an hour doesn't help at all. Similarly, if we look at the retail scenario, where we want to forecast the sales of a product, we will want to forecast a few days ahead so that we can purchase necessary goods, ship them to the store, and so on, in time for the demand.

Despite being a more prevalent use case, multi-step forecasting has not received the attention it deserves. One of the reasons for that is the existence of classical statistical models or econometrics models, such as the *ARIMA* and *exponential smoothing* methods, which include the multi-step strategy bundled within what we call a model; because of that, these models can generate multiple timesteps without breaking a sweat (although, as we will see in the chapter, they rely on one specific multi-step strategy to generate their forecast). Because these models were the most popular models used, practitioners didn't need to worry about multi-step forecasting strategies. However, the advent of **machine learning (ML)** and **deep learning (DL)** methods for time series forecasting has opened up the need for a more focused study of multi-step forecasting strategies once again.

Another reason for the lower popularity of multi-step forecasting is that it is simply harder than single-step forecasting. This is because the more steps we extrapolate into the future, the more uncertainty there is in the predictions, due to complex interactions between the different steps ahead. Depending on the strategy we choose, we will have to manage the dependencies on previous forecasts, the propagation and magnification of errors, and so on.

There are many strategies that can be used to generate multi-step forecasting, and the following figure summarizes them neatly:

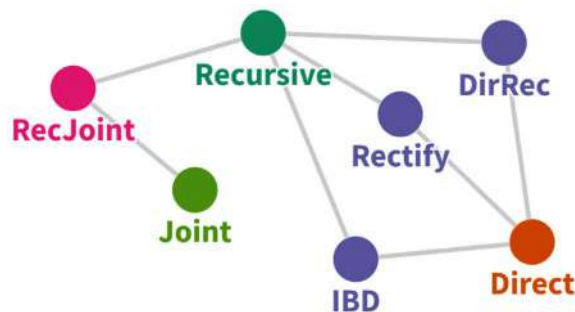


Figure 18.1: Multi-step forecasting strategies

Each node of the graph in Figure 18.1 is a strategy, and different strategies that have common elements have been linked together with edges in the graph. In the rest of the chapter, we will cover each of these nodes (strategies) and explain them in detail.

Standard notation

Let's establish a few basic notations to help us understand these strategies. We have a time series, Y_T , of T timesteps, $y_1, \dots, y_T$. Y_t denotes the same series but ending at timestep t . We also consider a function, W , which generates a window of size $k > 0$ from a time series.

This function is a proxy for how we prepare the input for the different models we have seen throughout the book. So if we see $W(Y_t)$, it means the function will draw a window from Y_T that ends at timestep t . We will also consider H to be the forecast horizon, where $H > 1$. We will also use $;$ as an operator, which denotes concatenation.

Now, let's look at the different strategies (Reference 1 is a good survey paper for different strategies). The discussion about merits and where we can use each of them is bundled in another upcoming section.

Recursive strategy

The recursive strategy is the oldest, most intuitive, and most popular technique to generate multi-step forecasts. To understand a strategy, there are two major regimes we have to understand:

- **Training regime:** How is the training of the models done?
- **Forecasting regime:** How are the trained models used to generate forecasts?

Let's take the help of a diagram to understand the recursive strategy:

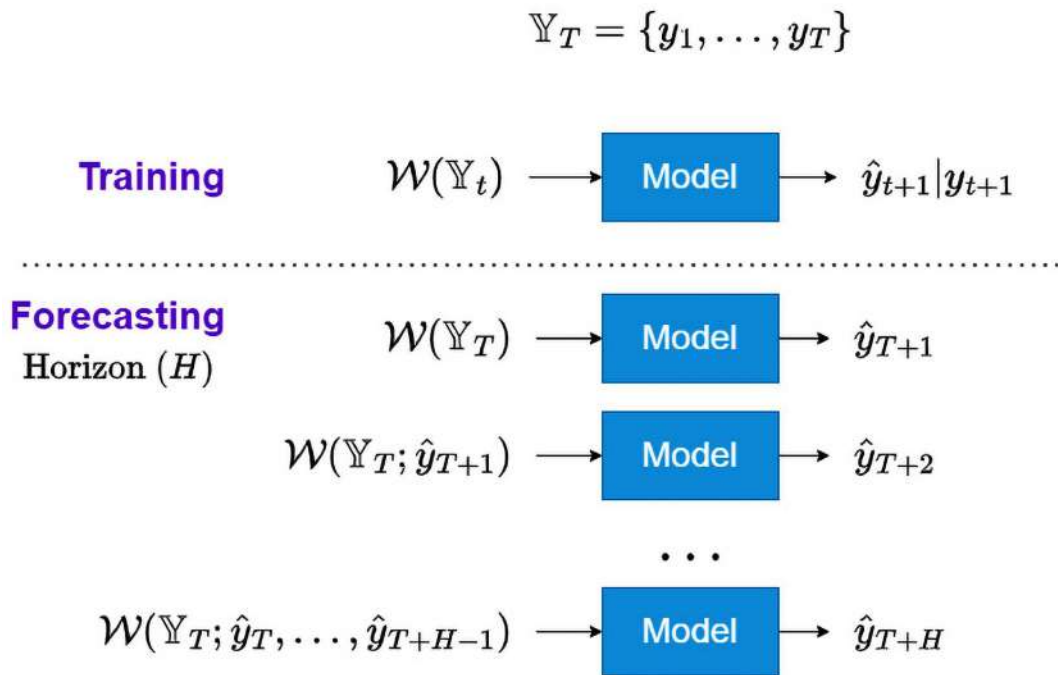


Figure 18.2: Recursive strategy for multi-step forecasting

Let's discuss these regimes in detail.

Training regime

The recursive strategy involves training a single model to perform a *one-step-ahead* forecast. We can see in Figure 18.2 that we use the window function, $W(Y_t)$, to draw a window from Y_t and train the model to predict Y_{t+1} .

During training, a loss function (which measures the divergence between the output of the model, $\hat{y}_{t+1}$, and the actual value, Y_{t+1}) is used to optimize the parameters of the model.

Forecasting regime

We have trained a model to do *one-step-ahead* predictions. Now, we use this model in a recursive fashion to generate forecasts H timesteps ahead. For the first step, we use $W(Y_t)$, the window using the latest timestamp in training data, and generate the forecast one step ahead, $\hat{y}_{T+1}$. Now, this generated forecast is added to the history, and a new window is drawn from this history, $W(Y_T; \hat{y}_{T+1})$. This window is given as input to the same *one-step-ahead* model, and the forecast for the next timestep, $\hat{y}_{T+2}$, is generated. This process is repeated until we get forecasts for all H timesteps.

This is the strategy that classical models that have stood the test of time (such as *ARIMA* and *exponential smoothing*) use internally when they generate multi-step forecasts. In an ML context, this means that we will train a model to predict one step ahead (as we have done all through this book) and then do a recursive operation, where we forecast one step ahead, use the new forecast to recalculate all the features such as lags, rolling windows, and so on, and forecast the next step. The pseudocode for the method would be:

```
# Function to create features (e.g., lags, rolling windows, external features
# like holidays or item category)
def create_features(df, **kwargs):
    ## Feature Pipeline goes here ##
    # Return features DataFrame
    return features

# Function to train the model
def train_model(train_df, **kwargs):
    # Create features from the training data
    features = create_features(train_df, **kwargs)

    ## Training code goes here ##

    # Return the trained model
    return model

def recursive_forecast(model, train_df, forecast_steps, **kwargs):
    """
    Perform recursive forecasting using the trained one-step model.
    - model: trained one-step-ahead model
```

```

- train_df: DataFrame with time series data
- forecast_steps: number of steps ahead to forecast
- kwargs: other parameters necessary like lag size, rolling size etc.
"""

forecasts = []
for step in range(forecast_steps):
    input_features = create_features(train_df, **kwargs)
    ## Replace with actual model.predict() code ##
    next_forecast = model.predict(input_features)
    forecasts.append(next_forecast)
    train_df = train_df.append({'target': next_forecast, "other_features":
other_features}, ignore_index=True)

return forecasts

```

In the context of the DL models, we can think of this as adding the forecast to the context window and using the trained model to generate the next step. The pseudocode for this would be:

```

def recursive_dl_forecast(dl_model, train_df, forecast_steps, **kwargs):
    """
    - dl_model: trained DL model (e.g., LSTM, Transformer)
    - train_df: DataFrame with time series data (context window)
    - forecast_steps: number of steps ahead to forecast
    - kwargs: other parameters like window size, etc.
    """

    forecasts = []
    # Extract initial context window from the end of the training data
    context_window = train_df['target'].values[-kwargs['window_size']:]
    for step in range(forecast_steps):
        ## Replace with actual dl_model.predict() code ##
        next_forecast = dl_model.predict(context_window)
        forecasts.append(next_forecast)
        # Update the context window by removing the oldest value and adding the
new forecast
        context_window = np.append(context_window[1:], next_forecast)

    return forecasts

```

Do note that this pseudocode is not ready-to-run code but more like a skeleton that you can adapt to your use case. Now, let's look at another strategy for multi-step forecasting.

Direct strategy

The **direct strategy**, also called the independent strategy, is a popular strategy in forecasting that uses ML. This involves forecasting each horizon independently of each other. Let's look at a diagram first:

$$\mathbb{Y}_T = \{y_1, \dots, y_T\}$$

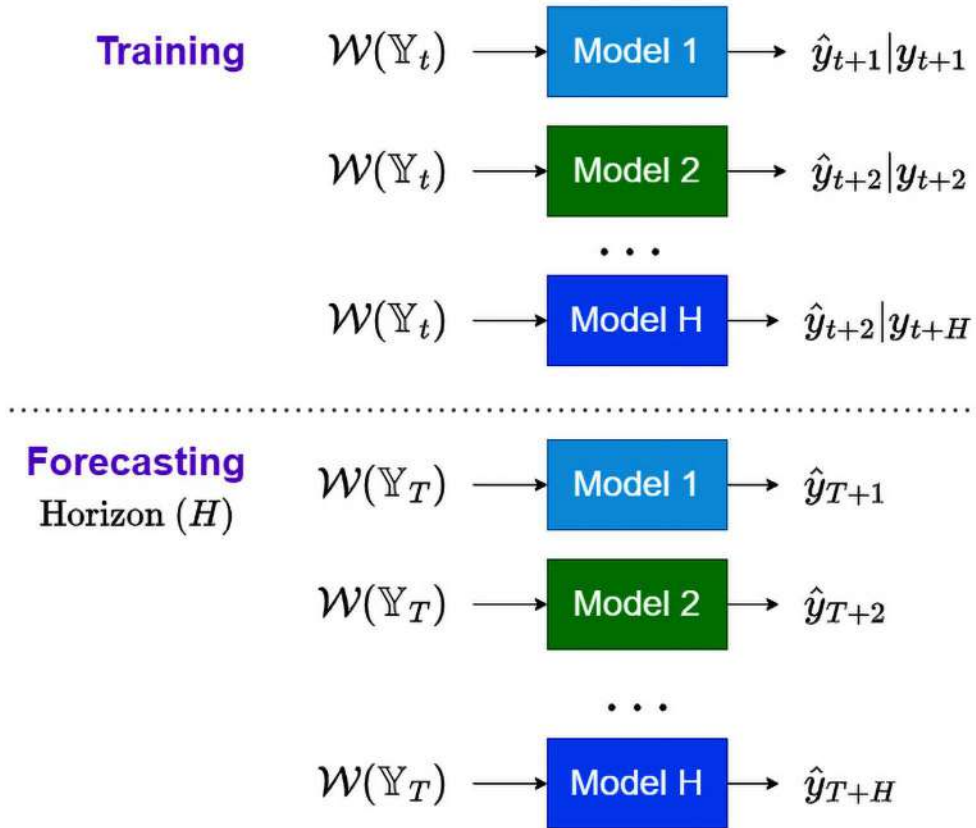


Figure 18.3: Direct strategy for multi-step forecasting

Next, let's discuss the regimes in detail.

Training regime

Under the direct strategy (Figure 18.3), we train H different models, which take in the same window function but are trained to predict different timesteps in the forecast horizon. Therefore, we learn a separate set of parameters, one for each timestep in the horizon, such that all the models combined learn a direct and independent mapping from the window, $\mathcal{W}(Y_t)$, to the forecast horizon, H .

This strategy has gained ground along with the popularity of ML-based time series forecasting. From the ML context, we can practically implement it in two ways:

- **Shifting targets:** Each model in the horizon is trained by shifting the target by as many steps as the horizon we train the model to forecast.
- **Eliminating features:** Each model in the horizon is trained by using only the allowable features, according to the rules. For instance, when predicting $H = 2$, we can't use lag 1 (because to predict $H = 2$, we would not have actuals for $H = 1$).



The two ways mentioned in the preceding list work nicely if we only have lags as features. For instance, to eliminate features, we can just drop the offending lags and train the model. But in cases where we use rolling features and other more sophisticated features, simple dropping doesn't work because lag 1 is already used to calculate the rolling features. This leads to data leakage. In such scenarios, we can make a dynamic function that calculates these features, taking in a parameter to specify the horizon we create these features for. All the helper methods we used in *Chapter 6, Feature Engineering for Time Series Forecasting* (`add_rolling_features`, `add_seasonal_rolling_features`, and `add_ewma`), have a parameter called `n_shift`, which handles this condition. If we train a model for $H = 2$, we need to pass `n_shift=2`, and then the method will take care of the rest. Now, while training the models, we use this dynamic method to recalculate these features for each horizon separately.

Forecasting regime

The forecasting regime is also fairly straightforward. We have the H -trained models, one for each timestep in the horizon, and we use $W(Y_t)$ to forecast each of them independently.

For ML models, this requires us to train separate models for each timestep, but `MultiOutputRegressor` from `scikit-learn` makes that a bit more manageable. Let's look at some pseudocode:

```
# Function to create shifted targets for direct strategy
def create_shifted_targets(df, horizon, **kwargs):
    ## Add one step ahead, 2 step ahead etc targets to the feature dataframe ##
    return dataframe, target_cols

def train_direct_ml_model(train_df, horizon, **kwargs):
    # Create shifted target columns for the horizon
    train_df, target_cols = create_shifted_targets(train_df, horizon, **kwargs)
    # Prepare features (X) and shifted targets (y) for training
    X = train_df.loc[:, [c for c in train_df.columns if c not in target_cols]]
    y = train_df.loc[:, target_cols]
    # Initialize a base model (e.g., Linear Regression) and
    MultiOutputRegressor
```



```

base_model = LinearRegression() # Example: can use any other model
multioutput_model = MultiOutputRegressor(base_model)
# Train the MultiOutputRegressor on the features and shifted targets
multioutput_model.fit(X, y)
return multioutput_model

def direct_ml_forecast(multioutput_model, test_df, horizon, **kwargs):
    # Adjust based on how test_df is structured
    X_test = test_df.loc[:, features]
    # (array with H steps)
    forecasts = multioutput_model.predict(X_test)
    return forecasts

```

Now, it's time to look at another strategy.

The Joint strategy

The previous two strategies consider a model to have a single output. This is the case with most ML models; we formulate the model to predict a single scalar value after taking in an array of inputs: **multiple input, single output (MISO)**. But there are some models, such as the DL models, which can be configured to give us multiple output. Therefore, the joint strategy, also called **multiple input, multiple output (MIMO)**, aims to learn a single model that produces the entire forecasting horizon as output:

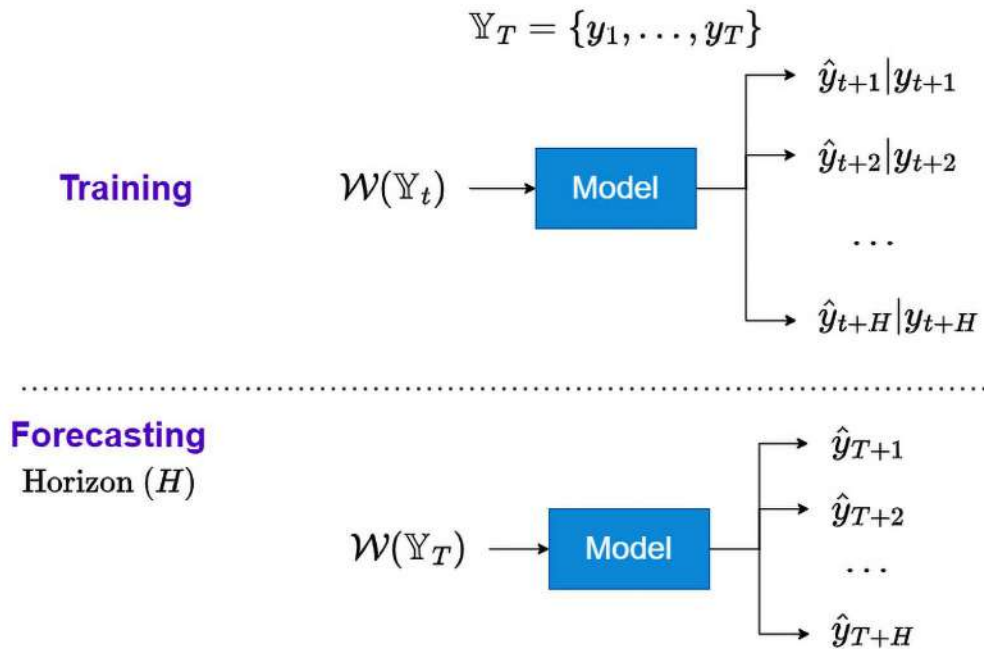


Figure 18.4: Joint strategy for multi-step forecasting

Let's see how these regimes work.

Training regime

The joint strategy involves training a single multi-output model to forecast all the timesteps in the horizon at once. We can see in *Figure 18.4* that we use the window function, $W(Y_t)$, to draw a window from Y_t and train the model to predict $y_{t+1}, \dots, y_{t+H}$. During training, a loss function that measures the divergence between all the output of the model, $\hat{y}_{t+1}, \dots, \hat{y}_{t+H}$, and the actual values, $y_{t+1}, \dots, y_{t+H}$, is used to optimize the parameters of the model.

Forecasting regime

The forecasting regime is also very simple. We have a trained model that is able to forecast all the timesteps in the horizon, and we use $W(Y_t)$ to forecast them at once.

This strategy is typically used in DL models where we configure the last layer to output H scalars instead of 1.

We have already seen this strategy in action at multiple places in the book:

- The tabular regression (*Chapter 13, Common Modeling Patterns for Time Series*) paradigm can easily be extended to output the whole horizon.
- We have seen *Sequence-to-Sequence* models with a *fully connected* decoder (*Chapter 13, Common Modeling Patterns for Time Series*) using this strategy for multi-step forecasting.
- In *Chapter 14, Attention and Transformers for Time Series*, we used this strategy to forecast using transformers.
- In *Chapter 16, Specialized Deep Learning Architectures for Forecasting*, we saw models such as *N-BEATS*, *N-HiTS*, and *Temporal Fusion Transformer*, which used this strategy to generate multi-step forecasts.

Hybrid strategies

The three strategies we have already covered are the three basic strategies for multi-step forecasting, each with its own merits and demerits. Over the years, researchers have tried to combine these as hybrid strategies that try to capture the good parts of each strategy. Let's go through a few of them here. This is not a comprehensive list because there is none. Anyone with enough creativity can come up with alternate strategies, but we will just cover a few that have received some attention and deep study from the forecasting community.

DirRec strategy

As the name suggests, the **DirRec** strategy is a combination of *direct* and *recursive* strategies for multi-step forecasting. One of the disadvantages of the direct method is that it forecasts each timestep independently and, therefore, loses out on some context when predicting far into the future. To rectify this shortcoming, we combine the direct and recursive methods by using the forecast generated by the n -step-ahead model as a feature in the $n+1$ -step-ahead model.

Let's look at the following diagram and solidify that understanding:

$$\mathbb{Y}_T = \{y_1, \dots, y_T\}$$

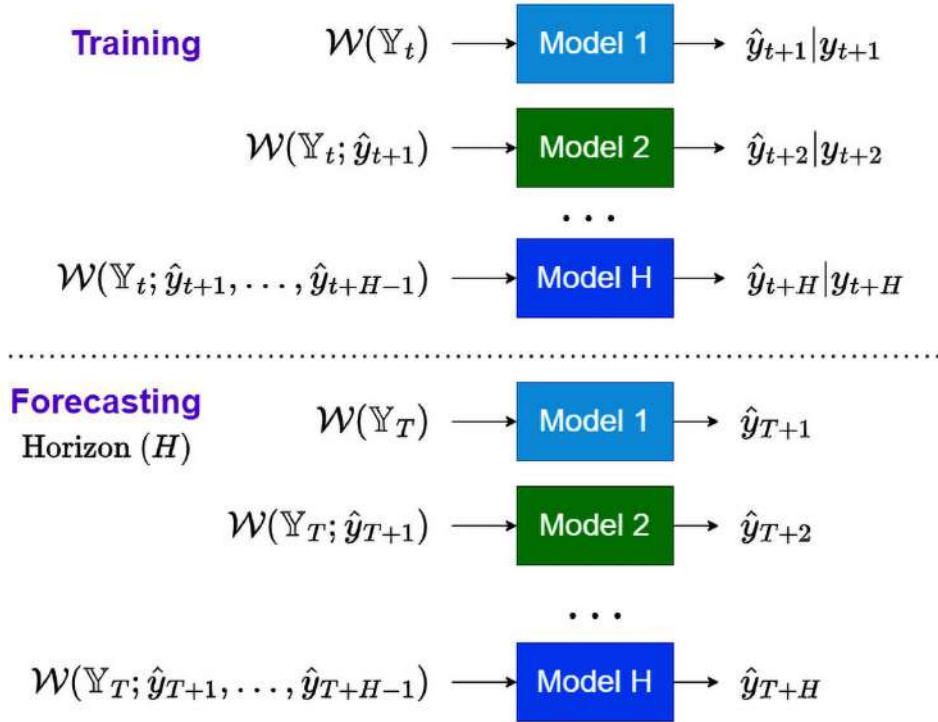


Figure 18.5: DirRec strategy for multi-step forecasting

Now, let's see how these regimes work for the DirRec strategy.

Training regime

Similar to the direct strategy, the DirRec strategy (Figure 18.5) also has H models for a forecasting horizon of H , but with a twist. We start the process by using $\mathcal{W}(Y_t)$ and train a model to predict one step ahead. In the recursive strategy, we used this forecasted timestep in the same model to predict the next timestep. But in DirRec, we train a separate model for $H=2$, using the forecast we generated in $H=1$. To generalize at timestep $h < H$, in addition to $\mathcal{W}(Y_t)$, we include all the forecasts generated by different models at timesteps 1 to h .

Forecasting regime

The forecasting regime is just like the training regime, but instead of training the models, we use the H -trained models to generate the forecasts recursively.

Let's take a look at some high-level pseudocode to solidify our understanding:

```
def train_dirrec_models(train_data, horizon, **kwargs):
    models = [] # To store the trained models for each timestep
    # Train the first model to predict the first step ahead (t+1)
    model_t1 = train_model(train_data) # Train model for t+1
    models.append(model_t1)
    for step in range(2, horizon + 1):
        previous_forecasts = []
        for prev_model in models:
            # Recursive prediction
            previous_forecasts.append(prev_model.predict(train_data))
        # Use the forecasts as features for the next model
        augmented_train_data = add_forecasts_as_features(train_data, previous_
forecasts)
        # Train the next model (e.g., for t+2, t+3, ...)
        model = train_model(augmented_train_data)
        models.append(model)
    return models

def dirrec_forecast(models, input_data, horizon, **kwargs):
    forecasts = []
    # Generate the first forecast (t+1)
    forecast_t1 = models[0].predict(input_data)
    forecasts.append(forecast_t1)
    # Generate subsequent forecasts recursively
    for step in range(1, horizon):
        augmented_input_data = add_forecasts_as_features(input_data, forecasts)
        next_forecast = models[step].predict(augmented_input_data)
        forecasts.append(next_forecast)
    return forecasts
```

Now, let's learn about another innovative way of multi-step forecasting.

Iterative block-wise direct strategy

The **iterative block-wise direct (IBD)** strategy is also called the **iterative multi-SVR strategy**, paying homage to the research paper that suggested this (Reference 2). The direct strategy requires H different models to train, and that makes it difficult to scale for long-horizon forecasting.

The IBD strategy tries to tackle that shortcoming by using a block-wise iterative style of forecasting:

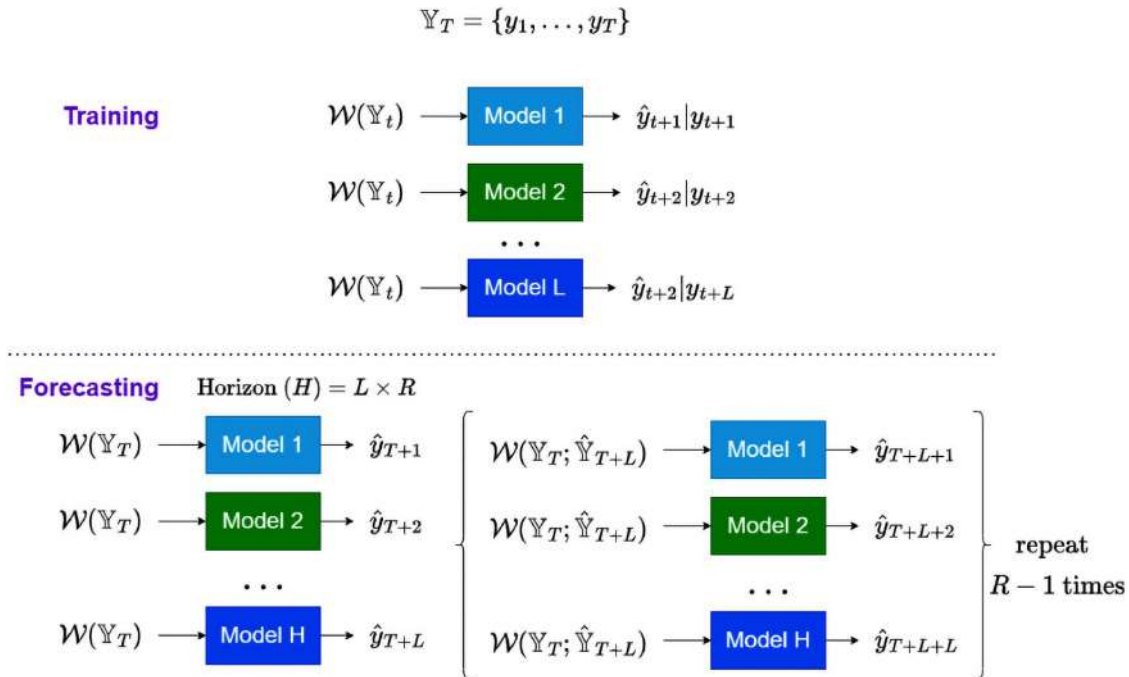


Figure 18.6: IBD strategy for multi-step forecasting

Let's understand the training and forecasting regimes for this strategy.

Training regime

In the IBD strategy, we split the forecast horizon, H , into R blocks of length L , such that $H = L \times R$. Instead of training H direct models, we train L direct models.

Forecasting regime

While forecasting (Figure 18.6), we use the L -trained models to generate the forecast for the first L timesteps ($T+1$ to $T+L$) in H , using the window, $\mathcal{W}(\mathbb{Y}_T)$. Let's denote this L forecast as $\mathbb{Y}_{T+L}$. Now, we will use $\mathbb{Y}_{T+L}$, along with $\mathbb{Y}_T$, in the window function to draw a new window, $\mathcal{W}(\mathbb{Y}_T; \mathbb{Y}_{T+L})$. This new window is used to generate the forecast for the next L timesteps ($T+L$ to $T+2L$). This process is repeated many times to complete the full horizon forecast.

Let's also see some high-level pseudocode for this process:

```
def train_ibd_models(train_data, horizon, block_size, **kwargs):
    # Calculate the number of models (L)
    n_models = horizon // block_size
    models = []
    # Train a model for each block
    for n in range(n_models):
```

```

        block_model = train_direct_model(train_data, n)
        models.append(block_model)
    return models

def ibd_forecast(models, input_data, horizon, block_size, **kwargs):
    forecasts = []
    window = input_data # Initial window from the time series data
    num_blocks = horizon // block_size
    # Generate forecasts block by block
    for _ in range(num_blocks):
        # Predict the next block of size L using direct models
        block_forecast = []
        for model in models:
            block_forecast.append(model.predict(window))
        # Append the block forecast to the overall forecast
        forecasts.extend(block_forecast)
        # Update the window by including the new block of predictions
        window = update_window(window, block_forecast)
    return forecasts

```

 **Quick tip:** Enhance your coding experience with the **AI Code Explainer** and **Quick Copy** features. Open this book in the next-gen Packt Reader. Click the **Copy** button (1) to quickly copy code into your coding environment, or click the **Explain** button (2) to get the AI assistant to explain a block of code to you.

```

function calculate(a, b) {
    return {sum: a + b};
};

```

Copy

Explain

1

2



 **The next-gen Packt Reader** is included for free with the purchase of this book. Unlock it by scanning the QR code below or visiting <https://www.packtpub.com/unlock/9781835883181>.



Now, let's move on to another creative way to hybridize different strategies.

Rectify strategy

The **rectify strategy** is another way we can combine direct and recursive strategies. It strikes a middle ground between the two by forming a two-stage training and inferencing methodology. We can see this as a model stacking approach (*Chapter 9, Ensembling and Stacking*) but between different multi-step forecasting strategies. In stage 1, we train a one-step-ahead model and generate recursive forecasts using that model. Then, in stage 2, we train direct models for the horizon using the original window and features, along with the recursive prediction.

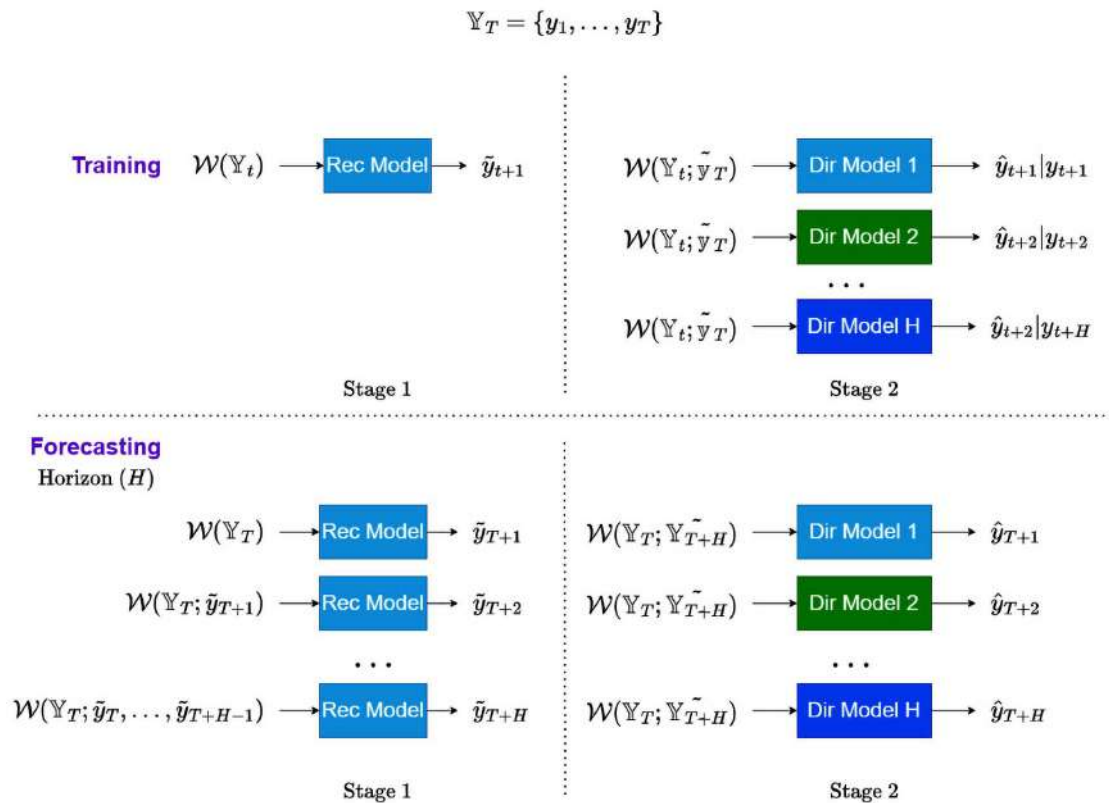


Figure 18.7: Rectify strategy for multi-step forecasting

Let's understand how this strategy works in detail.

Training regime

The training happens in two steps. The recursive strategy is applied to the horizon, and the forecast for all H timesteps is generated. Let's call this $\tilde{Y}_{t+H}$. Now, we train direct models for each horizon using the original history, Y_t , and the recursive forecasts, $\tilde{Y}_{t+H}$, as input.

Forecasting regime

The forecasting regime is similar to the training, where the recursive forecasts are generated first, and they, along with the original history, are used to generate the final forecasts.

Let's see some high-level pseudocode for this:

```
# Stage 1: Train recursive models
recursive_model, recursive_forecasts = train_one_step_ahead_model(train_data,
                                                                    horizon=horizon)

# Stage 2: Train direct models
direct_models = train_direct_models(train_data, recursive_forecasts,
                                     horizon=horizon)

def rectify_forecast(recursive_model, direct_models, input_data, horizon,
                    **kwargs):
    # Generate recursive forecasts using the recursive model
    recursive_forecasts = generate_recursive_forecasts(recursive_model, input_
data, horizon)
    # Generate final direct forecasts using original data and recursive
forecasts
    direct_forecasts = generate_direct_forecasts(direct_models, input_data,
recursive_forecasts, horizon)
    return direct_forecasts

forecast = rectify_forecast(recursive_model, direct_models, train_data,
                            horizon)
```

Now, let's move on to the last strategy we will cover here.

RecJoint

True to its name, **RecJoint** is a mashup between the recursive and joint strategies, but it is applicable for multi-output models. It aims to balance the benefits of both by leveraging recursive forecasting, while also considering dependencies between multiple timesteps in the forecast horizon.

$$\mathbb{Y}_T = \{y_1, \dots, y_T\}$$

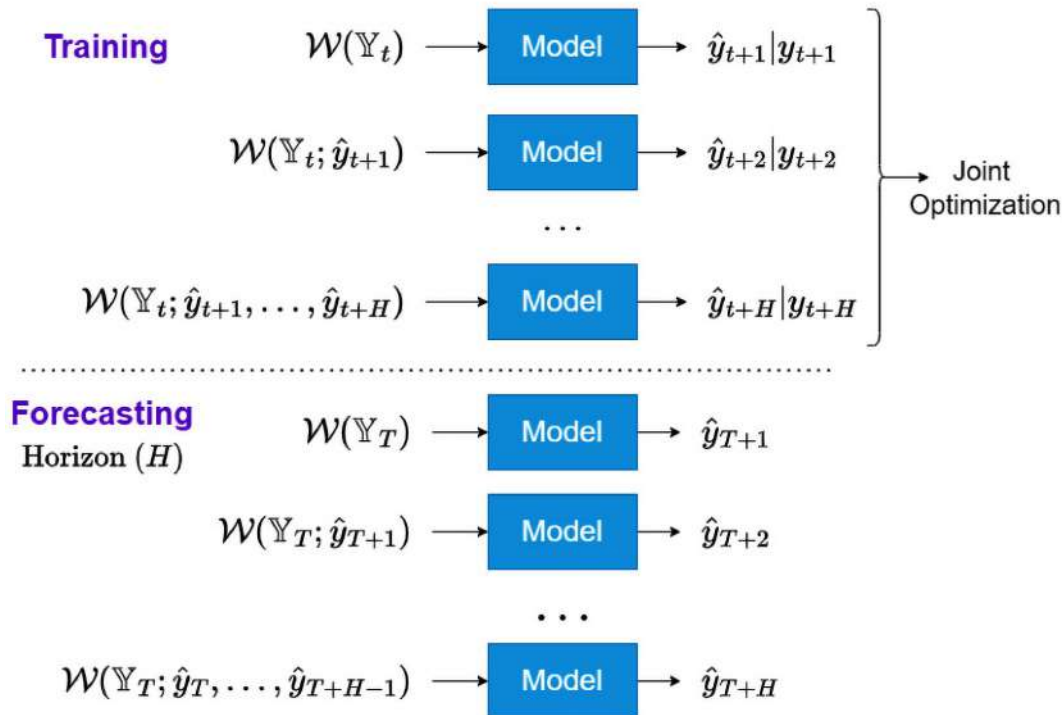


Figure 18.8: RecJoint strategy for multi-step forecasting

The following sections detail how this strategy works.

Training regime

The training regime (Figure 18.8) in the RecJoint strategy is very similar to the recursive strategy, in the way it trains a single model and recursively uses prediction at $t + 1$ as input to train $t + 2$, and so on. But the recursive strategy trains the model on just the next timestep, whereas RecJoint generates the predictions for the entire horizon and jointly optimizes the entire horizon forecasts while training. This forces the model to look at the next H timesteps and jointly optimize the entire horizon, instead of the myopic one-step-ahead objective. We saw this strategy at play when we trained Seq2Seq models using an RNN encoder and decoder (Chapter 13, *Common Modeling Patterns for Time Series*).

Forecasting regime

The forecasting regime for RecJoint is exactly the same as for the recursive strategy.

Now that we understand a few strategies, let's discuss their merits and demerits.

How to choose a multi-step forecasting strategy

Let's summarize all the different strategies that we have learned in a table:

Strategy	# of Models	Type	Output Size	Training Time	Prediction Time
Recursive	1	S.O	1	$1 \times T_{so}$	$H \times I_{so}$
Direct	H	S.O	1	$H \times T_{so}$	$H \times I_{so}$
DirRec	H	S.O	1	$H \times T_{so}$	$H \times I_{so}$
IBD	L	S.O	1	$L \times T_{so}$	$H \times I_{so}$
Rectify	H+1	S.O	1	$(H + 1) \times T_{so}$	$2 \times H \times I_{so}$
Joint	1	M.O	H	$1 \times T_{mo}$	$1 \times I_{mo}$
RecJoint	1	M.O	1	$1 \times (T_{mo} + \delta)$	$H \times I_{mo}$

Figure 18.9: Multi-step forecasting strategies—a summary

Here, the following apply:

- S.O: Single output
- M.O: Multi-output
- T_{so} and I_{so} : Training and inferencing the time of a single-output model
- T_{mo} and I_{mo} : Training and inferencing the time of a multi-output model (practically, T_{mo} is larger than T_{so} mostly because multi-output models are typically DL models, and their training time is higher than standard ML models)
- H: The horizon
- $L = H/R$, where R is the number of blocks in the IBD strategy
- δ is some positive real number

The table helps us understand and decide which strategy is better from multiple perspectives:

- **Engineering complexity:** *Recursive, Joint, RecJoint* << *IBD* << *Direct*, and *DirRec* << *Rectify*
- **Training time:** *Recursive* << *Joint* (typically $T_{mo} > T_{so}$) << *RecJoint* << *IBD* << *Direct*, and *DirRec* << *Rectify*
- **Inference time:** *Joint* << *Direct, Recursive, DirRec, IBD*, and *RecJoint* << *Rectify*

It also helps us to decide the kind of model we can use for each strategy. For instance, a joint strategy can only be implemented with a model that supports multi-output, such as a DL model. However, we have yet to discuss how these strategies affect accuracies.

Although, in ML, the final word goes to empirical evidence, there are ways we can analyze the different methods to provide us with some guidelines. *Taieb et al.* analyzed the bias and variance of these multi-step forecasting strategies, both theoretically and using simulated data.

With this analysis, along with other empirical findings over the years, we have an understanding of the strengths and weaknesses of these strategies, and some guidelines have emerged from these findings.



Reference check:

The research paper by Taieb et al. is cited in Reference 3.

Taieb et al. point out several disadvantages of the recursive strategy, contrasting with the direct strategy, based on the bias and variance components of error analysis. They further corroborated these observations through an empirical study.

The key points that elucidate the difference in performance are as follows:

- For the recursive strategy, the bias and variance components of error in step $h = 1$ affect step $h = 2$. Because of this phenomenon, the errors that a recursive model makes tend to accumulate as we move further in the forecast horizon. But for the direct strategy, this dependence is not explicit and, therefore, doesn't suffer the same deterioration that we see in the recursive strategy. This was also seen in the empirical study, where the recursive strategy was very erratic and had the highest variance, which increased significantly as we moved further in the horizon.
- For the direct strategy, the bias and variance components of error in step $h = 1$ do not affect $h = 2$. This is because each horizon, h , is forecasted in isolation. A downside of this approach is the fact that this strategy can produce completely unrelated forecasts across the horizon, leading to unrealistic forecasts. The complex dependencies that may exist between the forecast in the horizon are not captured in the direct strategy. For instance, a direct strategy on a time series with a non-linear trend may result in a broken curve because of the independence of each timestep in the horizon.
- Practically, in most cases, a direct strategy produces coherent forecasts.
- The bias for the recursive strategy is also amplified when the forecasting model produces forecasts that have large variations. Highly complex models are known to have low bias but a high amount of variations, and these high variations seem to amplify the bias for recursive strategy models.
- When we have very large datasets, the bias term of the direct strategy becomes zero, but the recursive strategy bias is still non-zero. This was further demonstrated in experiments—for long time series, the direct strategy almost always outperformed the recursive strategy. From a learning theory perspective, we learn H functions using the data for the direct strategy, whereas for recursive, we just learn one. So with the same amount of data, it is harder to learn H true functions than one. This is amplified in low-data situations.
- Although the recursive strategy seems inferior to the direct strategy theoretically and empirically, it is not without some advantages:
 - For highly non-linear and noisy time series, learning direct functions for all the horizons can be hard. In such situations, recursive can work better.

- If the underlying **data-generating process (DGP)** is very smooth and can be easily approximated, the recursive strategy can work better.
- When the time series is shorter, the recursive strategy can work better.
- We talked about the direct strategy generating possible unrelated forecasts for the horizon, but this is exactly the part that the joint strategy takes care of. The joint strategy can be thought of as an extension of the direct strategy, but instead of having H different models, we have a single model that produces H output. We learn a single function instead of H functions from the given data. Therefore, the joint strategy doesn't have the same weakness as the direct strategy in short time series.
- One of the weaknesses of the joint strategy (and RecJoint) is the high bias on very short horizons (such as $H = 2$, $H = 3$, and so on). We learn a model that optimizes across all the H timesteps in the horizon using a standard loss function, such as the mean squared error. But these errors are at different scales. The errors that can occur further down the horizon are larger than the immediate ones, and this implicitly puts more weight on the longer horizons; thus, the model learns a function that is skewed toward getting the longer horizons right.
- The joint and RecJoint strategies are comparable from a variance perspective. However, the joint strategy can give us a lower bias because the RecJoint strategy learns a recursive function, and it may not be flexible enough to capture the pattern. The joint strategy uses the full power of the forecasting model to directly forecast the horizon.

Hybrid strategies, such as DirRec, IBD, and so on, try to balance the merits and demerits of fundamental strategies, such as direct, recursive, and joint. With these merits and demerits, we can create an informed experimentation framework to come up with the best strategy for the problem at hand.

Summary

In this chapter, we touched upon a particular aspect of forecasting that is highly relevant for real-world use cases but rarely talked about and studied. We saw why we needed multi-step forecasting and then went on to review a few popular strategies we can use. We explored the popular and fundamental strategies, such as direct, recursive, and joint, and then went on to look at a few hybrid strategies, such as DirRec, rectify, and so on. Finally, we looked at the merits and demerits of these strategies and discussed a few guidelines for selecting the right strategy for your problem.

In the next chapter, we will look at another important aspect of forecasting—evaluation.

References

The following is the list of the references that we used throughout the chapter:

1. Taieb, S.B., Bontempi, G., Atiya, A.F., and Sorjamaa, A. (2012). *A review and comparison of strategies for multi-step ahead time series forecasting based on the NN5 forecasting competition*. Expert Syst. Appl., 39, 7067–7083: <https://arxiv.org/pdf/1108.3259.pdf>
2. Li Zhang, Wei-Da Zhou, Pei-Chann Chang, Ji-Wen Yang, and Fan-Zhang Li. (2013). *Iterated time series prediction with multiple support vector regression models*. Neurocomputing, Volume 99, 2013: <https://www.sciencedirect.com/science/article/pii/S0925231212005863>

3. Taieb, S.B. and Atiya, A.F. (2016). *A Bias and Variance Analysis for Multistep-Ahead Time Series Forecasting*. in IEEE Transactions on Neural Networks and Learning Systems, vol. 27, no. 1, pp. 62–76, Jan. 2016: <https://ieeexplore.ieee.org/document/7064712>

**Unlock this book's exclusive
benefits now**

This book comes with additional benefits
designed to elevate your learning experience.



Note: Have your purchase invoice ready before you begin. <https://www.packtpub.com/unlock/9781835883181>

19

Evaluating Forecast Errors—A Survey of Forecast Metrics

We started getting into the nuances of forecasting in the previous chapter where we saw how to generate multi-step forecasts. While that covers one of the aspects, there is another aspect of forecasting that is as important as it is confusing—*how to evaluate forecasts*.

In the real world, we generate forecasts to enable some downstream processes to plan better and take relevant actions. For instance, the operations manager at a bike rental company should decide how many bikes he should make available at the metro station the next day at 4 p.m. However, instead of using the forecasts blindly, he may want to know which forecasts he should trust and which ones he shouldn't. This can only be done by measuring how good a forecast is.

We have been using a few metrics throughout the book and it is now time to get down into the details to understand those metrics, when to use them, and when to not use them. We will also elucidate a few aspects of these metrics experimentally.

In this chapter, we will be covering these main topics:

- Taxonomy of forecast error measures
- Investigating error measures
- Experimental study of error measures
- Guidelines for choosing a metric

Technical requirements

You will need to set up the Anaconda environment following the instructions in the *Preface* of the book to get a working environment with all the packages and datasets required for the code in this book.

The associated code for the chapter can be found here: <https://github.com/PacktPublishing/Modern-Time-Series-Forecasting-with-Python/tree/main/notebooks/Chapter19>.

For this chapter, you need to run the notebooks in the Chapters02 and Chapter04 folders from the book's GitHub repository.

Taxonomy of forecast error measures



Measurement is the first step that leads to control and eventually improvement.

– H. James Harrington

Traditionally, in regression problems, we have very few general loss functions, such as the mean squared error or the mean absolute error, but when you step into the world of time series forecasting, you will be hit with a myriad of different metrics.



Since the focus of the book is on point predictions (and not probabilistic predictions), we will stick to reviewing point forecast metrics.

There are a few key factors that distinguish the metrics in time series forecasting:

- **Temporal relevance:** The temporal aspect of the prediction we make is an essential aspect of a forecasting paradigm. Metrics such as Forecast Bias and the tracking signal take this aspect into account.
- **Aggregate metrics:** In most business use cases, we would not be forecasting a single time series but, rather, a set of time series, related or unrelated. In these situations, looking at the metrics of individual time series becomes infeasible. Therefore, there should be metrics that capture the idiosyncrasies of this mix of time series.
- **Over- or under-forecasting:** Another key concept in time series forecasting is over- and under-forecasting. In a traditional regression problem, we do not really worry whether the predictions are more than or less than expected, but in the forecasting paradigm, we must be careful about structural biases that always over- or under-forecast. This, when combined with the temporal aspect of time series, accumulates errors and leads to problems in downstream planning.

These aforementioned factors, along with a few others, have led to an explosion in the number of forecast metrics. In a recent survey paper by Hewamalage et al. (Reference 1), the number of metrics that were covered stood at 38. Let's try and unify these metrics under some structure. *Figure 19.1* depicts a taxonomy of forecast error measures:

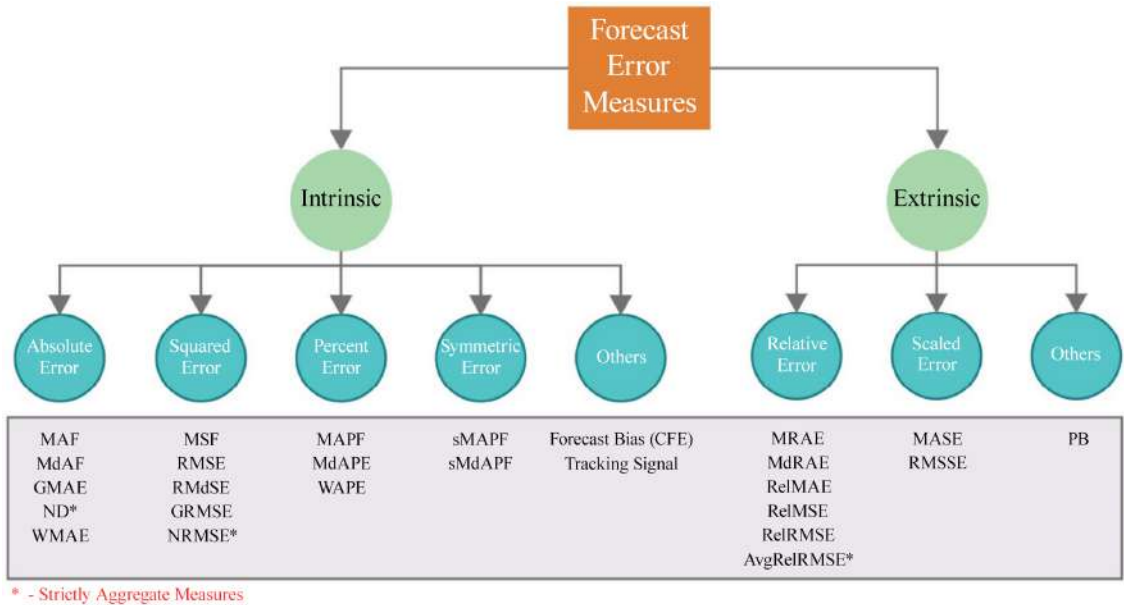


Figure 19.1: Taxonomy of forecast error measures

We can semantically separate the different forecast metrics into two buckets—**intrinsic** and **extrinsic**. *Intrinsic* metrics measure the generated forecast using nothing but the generated forecast and the corresponding actuals. As the name suggests, it is a very inward-looking metric. *Extrinsic* metrics, on the other hand, use an external reference or benchmark in addition to the generated forecast and ground truth to measure the quality of the forecast.

Before we start with the metrics, let's establish some notation to help us understand. y_t and $\hat{y}_t$ are the actual observation and the forecast at time t , respectively. The forecast horizon is denoted by H . In cases where we have a dataset of time series, we assume there are M time series, indexed by m , and finally, $e_t = y_t - \hat{y}_t$ denotes the error at timestep t . Now, let's start with the intrinsic metrics.

Intrinsic metrics

Intrinsic metrics are used to assess a forecast without any external information. These are ideal candidates for model development, hyperparameter tuning, and so on. That doesn't mean we cannot use this type of metric to report performances to non-technical people, but it'll have to be qualified with some other benchmark to show how well we are doing.

There are four major base errors—absolute error, squared error, percent error, and symmetric error—that are aggregated or summarized in different ways in a variety of metrics. Therefore, any property of these base errors also applies to the aggregate ones, so let's look at these base errors first.

Absolute error

The error, e_t , can be positive or negative, depending on whether $y_t < \hat{y}_t$ or not, but then when we are calculating and adding this error over the horizon, the positive and negative errors may cancel each other out and that paints a rosier picture. Therefore, we include a function on top of e_t to ensure that the errors do not cancel each other out.

The absolute function is one of these functions: *Absolute Error* (AE) = $|e_t|$. The absolute error is a scale-dependent error. This means that the magnitude of the error depends on the scale of the time series. For instance, if you have an AE of 10, it doesn't mean anything until you put it in context. For a time series with values of around 500 to 1,000, an AE of 10 may be a very good number, but if the time series has values around 50 to 70, then it is bad.



Scale dependence is not a deal breaker when we are looking at individual time series, but when we are aggregating or comparing across multiple time series, scale-dependent errors skew the metric in favor of the large-scale time series. The interesting thing to note here is that this is not necessarily bad. Sometimes, the scale in the time series is meaningful and it makes sense from the business perspective to focus more on the large-scale time series than the smaller ones. For instance, in a retail scenario, one would be more interested in getting the high-selling product forecast right than those of the low-selling ones. In these cases, using a scale-dependent error automatically favors the high-selling products.

You can see this by carrying out an experiment on your own. Generate a random time series, A . Now, similarly, generate a random forecast for the time series, F . Now, we multiply the forecast, F , and time series, A , by 100 to get two new time series and their forecasts, A_{scaled} and F_{scaled} , respectively. If we calculate the forecast metric for both these sets of time series and forecasts, the scaled-dependent metrics will give very different values, whereas the scale-independent ones will give the same values.

Many metrics are based on this error:

$$MAE = \frac{1}{H} \sum_{t=1}^H |e_t|$$

- **Mean Absolute Error (MAE):**
 - **Median Absolute Error:** $MdAE = median(|e_t|)$
 - **Geometric Mean Absolute Error:** $GMAE = \sqrt[H]{\prod_{t=1}^H |e_t|}$
- **Weighted Mean Absolute Error:** This is a more esoteric method that lets you put more weight on a particular timestep in the horizon:

$$WMAE = \frac{\sum_{t=1}^H w_t |e_t|}{\sum_{t=1}^H w_t}$$

Here, w_t is the weight of a particular timestep. This can be used to assign more weight to special days (such as weekends or promotion days).

- **Normalized Deviation (ND):** This is a metric that is strictly used to calculate aggregate measures across a dataset of time series. This is also one of the popular metrics used in the industry to measure aggregate performance across different time series. This is not scale-free and will be skewed toward large-scale time series. This metric has strong connections with another metric called the **Weighted Average Percent Error (WAPE)**. We will discuss these connections when we talk about the WAPE in the following sections.

To calculate ND, we just sum all the absolute errors across the horizons and time series and scale it by the actual observations across the horizons and time series:

$$ND = \frac{\sum_{m=1}^M \sum_{t=1}^H |e_{(t,m)}|}{\sum_{m=1}^M \sum_{t=1}^H |y_{t,m}|}$$

Squared error

Squaring is another function that makes the error positive and thereby prevents the errors from canceling each other out:

$$\text{Squared Error (SE)} = e_t^2$$

There are many metrics that are based on this error:

$$MSE = \frac{1}{H} \sum_{t=1}^H (e_t^2)$$

- **Mean Squared Error:**
 - **Root Mean Squared Error (RMSE):**

$$RMSE = \sqrt{\frac{1}{H} \sum_{t=1}^H (e_t^2)}$$

- **Root Median Squared Error:**

$$RMdSE = \text{median}(\sqrt{e_t^2})$$

- **Geometric Root Mean Squared Error:**

$$GRMSE = \sqrt[2n]{\prod_{t=1}^H (e_t^2)}$$

- **Normalized Root Mean Squared Error (NRMSE):** This is a metric that is very similar to ND in spirit. The only difference is that we take the square root of the squared errors in the numerator rather than the absolute error:

$$NRMSE = \frac{\sqrt{\frac{1}{MH} \sum_{m=1}^M \sum_{t=1}^H (e_{(t,m)}^2)}}{\frac{1}{MH} \sum_{m=1}^M \sum_{t=1}^H |y_t, m|}$$

Percent error

While absolute error and squared error are scale-dependent, percent error is a scale-free error measure. In percent error, we scale the error using the actual time series observations: $Percent\ Error(PE) = \frac{100e_t}{y_t}$. Some of the metrics that use percent error are as follows:

- **Mean Absolute Percent Error (MAPE):**

$$MAPE = \sum_{t=1}^H \frac{100|e_t|}{y_t}$$

- **Median Absolute Percent Error:**

$$MdAPE = median\left(\frac{100|e_t|}{y_t}\right)$$

- **WAPE:** WAPE is a metric that embraces scale dependency and explicitly weights the errors with the scale of the timestep. If we want to give more focus to high values on the horizon, we can weigh those timesteps more than the others. Instead of taking a simple mean, we use a weighted mean on the absolute percent error. We can choose the weight to be anything but, more often than not, it is chosen as the quantity of the observation itself. And, in that special case, the math (with some assumptions) works out to be a simple formula that reminds us of ND. The difference is that ND is a metric that aggregates across multiple time series, and WAPE is a metric that weighs across timesteps:

$$WAPE = \frac{\sum_{t=1}^H |e_t|}{\sum_{t=1}^H |y_t|}$$

Symmetric error

Percent error has a few problems—it is asymmetrical (we will see this in detail later in the chapter), and it breaks down when the actual observation is zero (due to division by zero). Symmetric error was proposed as an alternative to avoid this asymmetry, but as it turns out, symmetric error is itself asymmetric—more on that later, but for now, let's see what symmetric error is:

$$Symmetric\ Error(SE) = \frac{200|e_t|}{|y_t| + |\hat{y}_t|}$$

There are only two metrics that are popularly used under this base error:

- **Symmetric Mean Absolute Percent Error (sMAPE):**

$$sMAPE = \frac{1}{H} \sum_{t=1}^H \frac{200|e_t|}{|y_t| + |\hat{y}_t|}$$

- **Symmetric Median Absolute Percent Error:**

$$sMsAPE = \text{median} \left(\frac{200|e_t|}{|y_t| + |\hat{y}_t|} \right)$$

Other intrinsic metrics

There are a few other metrics that are intrinsic in nature but don't conform to the other metrics. Notable among those are three metrics that measure the over- or under-forecasting aspect of forecasts:

- **Cumulative Forecast Error (CFE):** CFE is simply the sum of all the errors, including the sign of the error. Here, we want the positives and negatives to cancel each other out so that we understand whether a forecast is consistently over- or under-forecasting in a given horizon. A CFE close to zero means the forecasting model is neither over- nor under-forecasting:

$$CFE = \sum_{t=1}^H e_t$$

- **Forecast Bias (FB):** While CFE measures the degree of over- and under-forecasting, it is still scale-dependent. When we want to compare across time series or have an intuitive understanding of the degree of over- or under-forecasting, we can scale CFE by the actual observations. This is called Forecast Bias:

$$FB = \frac{\sum_{t=1}^H e_t}{\sum_{t=1}^H y_t}$$

- **Tracking Signal (TS):** The Tracking Signal is another metric that is used to measure the same over- and under-forecasting in forecasts. While CFE and Forecast Bias are used more offline, the Tracking Signal finds its place in an online setting where we are tracking over- and under-forecasting over periodic time intervals, such as every hour or every week. It helps us detect structural biases in the forecasting model. Typically, the Tracking Signal is used along with a threshold value so that going above or below it throws a warning. Although a thumb rule is to use ± 3.75 , it is totally up to you to decide the right threshold for your problem. The value 3.75 has its roots in the properties of normal distribution where it corresponds to a 99% confidence interval, which means this value is sensitive enough to trigger when there is a structural bias while avoiding false positives.

But at the end of the day, this value should just be a starting point to start backdated tests to figure out what is the threshold that raises the right kind of alarms in your data:

$$TS_w = \frac{\sum_{t=0}^w e_t}{\frac{1}{w} \sum_{t=0}^w |e_t|}$$

Here, w is the past window over which TS is calculated.

Now, let's turn our attention to a few extrinsic metrics.

Extrinsic metrics

Extrinsic metrics evaluate the forecast quality by comparing it not only to the actuals but also to some external reference or benchmark. This could be a baseline model, an industry standard, or a competitor's forecast. These metrics are more suitable for reporting the performance of the model to non-technical people who can instantly get a sense of how good the model is doing. Suppose you have an existing forecast that you are trying to improve; using that forecast as a reference will instantly make your metrics interpretable.

There are two major buckets of metrics under the extrinsic umbrella—relative error and scaled error.

Relative error

One of the problems of intrinsic metrics is that they don't mean a lot unless a benchmark score exists. For instance, if we hear that the MAPE is 5%, it doesn't mean a lot because we don't know how forecastable that time series is. Maybe 5% is a bad error rate. Relative error solves this by including a benchmark forecast in the calculation so that the errors of the forecast we are measuring are measured against the benchmark and thus show the relative gains of the forecast. Therefore, in addition to the notation that we have established, we need to add some more.

Let y_t^* be the forecast from the benchmark and $e_t^* = y_t - y_t^*$ be the benchmark error. There are two ways we can include the benchmark in the metric:

- Using errors from the benchmark forecast to scale the error of the forecast
- Using forecast measures from the benchmark forecast to scale the forecast measure of the forecast we are measuring

Let's look at a few relative errors:

- **Mean Relative Absolute Error (MRAE):**

$$MRAE = \frac{1}{H} \sum_{t=1}^H \frac{|e_t|}{|e_t^*|}$$

- **Median Relative Absolute Error:**

$$MdRAE = \text{median} \left(\frac{|e_t|}{|e_t^*|} \right)$$

- **Geometric Mean Relative Absolute Error:**

$$GMRAE = \sqrt[H]{\prod_{t=1}^H \frac{|e_t|}{|e_t^*|}}$$

- **Relative Mean Absolute Error (RelMAE):**

$RelMAE = \frac{MAE}{MAE^*}$, where MAE^* is the MAE of the benchmark forecast.

- **Relative Root Mean Squared Error (RelRMSE):**

$RelRMSE = \frac{RMSE}{RMSE^*}$, where $RMSE^*$ is the RMSE of the benchmark forecast.

- **Average Relative Mean Absolute Error:** Davydenko and Fildes (Reference 2) proposed another metric that is strictly for calculating aggregate scores across time series. They argued that using a geometric mean over the RelMAEs of individual time series is better than an arithmetic mean, so they defined the Average Relative Mean Absolute Error as follows:

$$AvgRelMAE = \left(\prod_{m=1}^M \left(\frac{MAE_m}{MAE_m^*} \right)^{h_m} \right)^{\frac{1}{\sum_{m=1}^M h_m}}$$

Scaled error

Hyndman and Koehler introduced the idea of scaled error in 2006. This was an alternative to relative error and measures and tries to get over some of the drawbacks and subjectivity of choosing the benchmark forecast. Scaled error scales the forecast error using an in-sample MAE of a benchmark method such as naïve forecasting. Let the entire training history be of T timesteps, indexed by i .

So, the scaled error is defined as follows:

$$SE = \frac{|e_t|}{\frac{1}{T-1} \sum_{i=2}^T |y_t - y_{t-1}|}$$

There are a couple of metrics that adopt this principle:

- **Mean Absolute Scaled Error (MASE):**

$$MASE = \frac{1}{H} \sum_{t=1}^H |SE|$$

- **Root Mean Squared Scaled Error (RMSSE):** A similar scaled error was developed for the squared error and was used in the M5 Forecasting Competition in 2020:

$$RMSSE = \frac{1}{H} \sum_{t=1}^H \frac{e_t^2}{\frac{1}{T-1} \sum_{i=2}^T (y_t - y_{t-1})^2}$$

Other extrinsic metrics

There are other extrinsic metrics that don't fall into the categorization of errors we have made. One such error measure is the following.

Percent Better (PB) is a method that is based on counts and can be applied to individual time series as well as a dataset of time series. The idea here is to use a benchmark method and count how many times a given method is better than the benchmark and report it as a percentage. Formally, we can define it using MAE as the reference error, as follows:

$$PB_{MAE} = 100 \times \text{mean}(\mathbb{I}\{MAE < MAE^*\})$$

Here, $\mathbb{I}$ is an indicator function that returns 1 if the condition is true and 0 otherwise.

We have seen a lot of metrics in the previous sections, but now it's time to understand a bit more about the way they work and what they are suited for.

Investigating the error measures

It's not enough to just know the different metrics; we also need to understand how these work, what are they good for, and what are they not good for. We can start with the basic errors and work our way up because understanding the properties of basic errors such as *absolute error*, *squared error*, *percent error*, and *symmetric error* will help us understand the others as well, as most of the other metrics are derivatives of these primary errors, either aggregating them or using relative benchmarks.

Let's do this investigation using a few experiments and understand them through the results.



Notebook alert:

The notebook for running these experiments on your own is `01-Loss_Curves_and_Symmetry.ipynb` in the Chapter19 folder.

Loss curves and complementarity

All these base errors depend on two factors—forecasts and actual observations. We can examine the behavior of these metrics if we fix one and alter the other in a symmetric range of potential errors. The expectation is that the metric will behave the same way on both sides because deviation from the actual observation on either side should be equally penalized in an unbiased metric. We can also swap the forecasts and actual observations; that also should not affect the metric.

In the notebook, we did exactly these experiments—loss curves and complementary pairs.

Absolute error

When we plot these for absolute error, we get *Figure 19.2*:

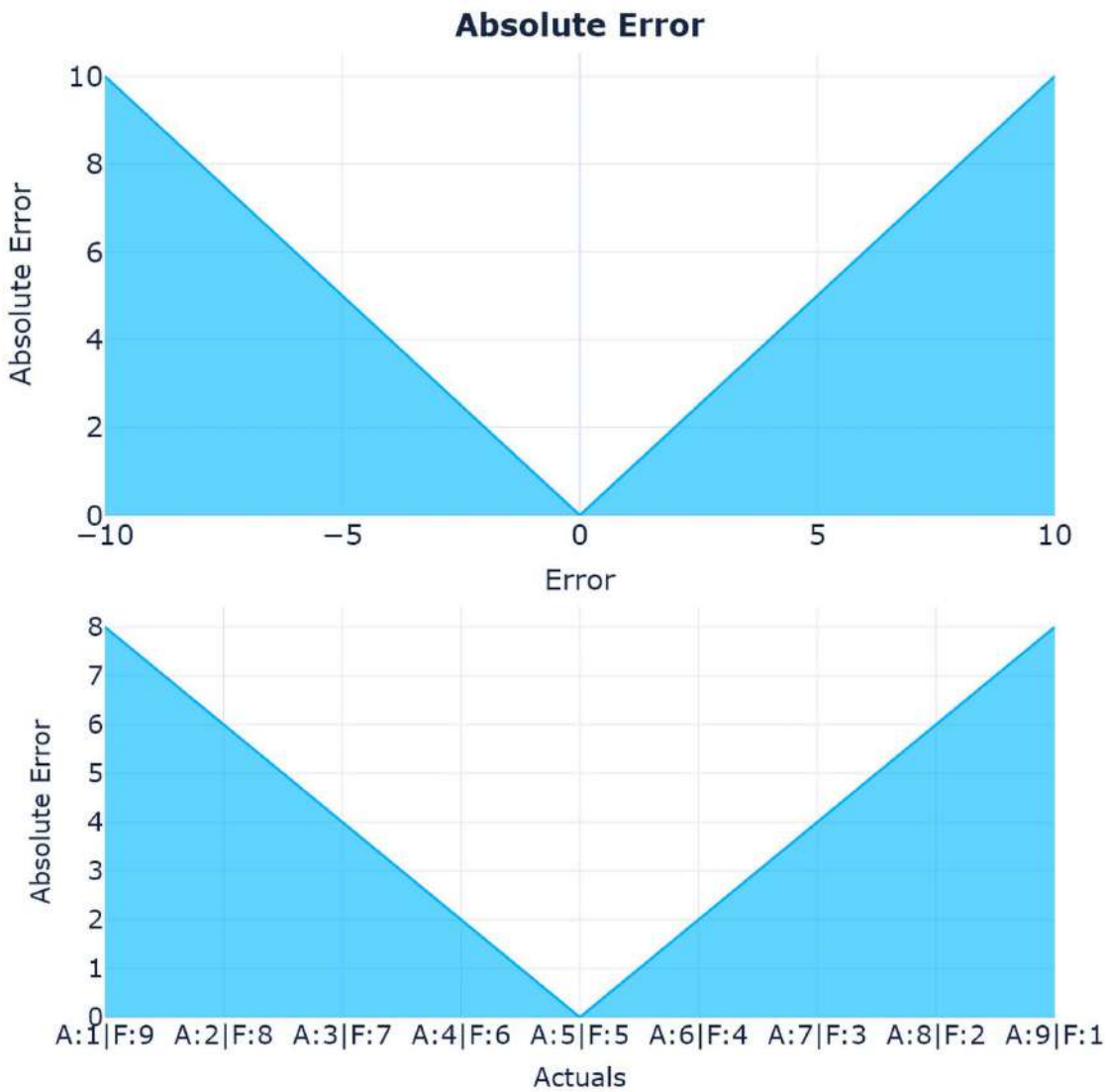


Figure 19.2: The loss curves and complementary pairs for absolute error

The first chart plots the signed error against the absolute error and the second one plots the absolute error with all the combinations of actuals and forecast, which add up to 10. The two charts are obviously symmetrical, which means that an equal deviation from the actual observed on either side is penalized equally, and if we swap the actual observation and the forecast, the metric remains unchanged.

Squared error

Now, let's look at the squared error:

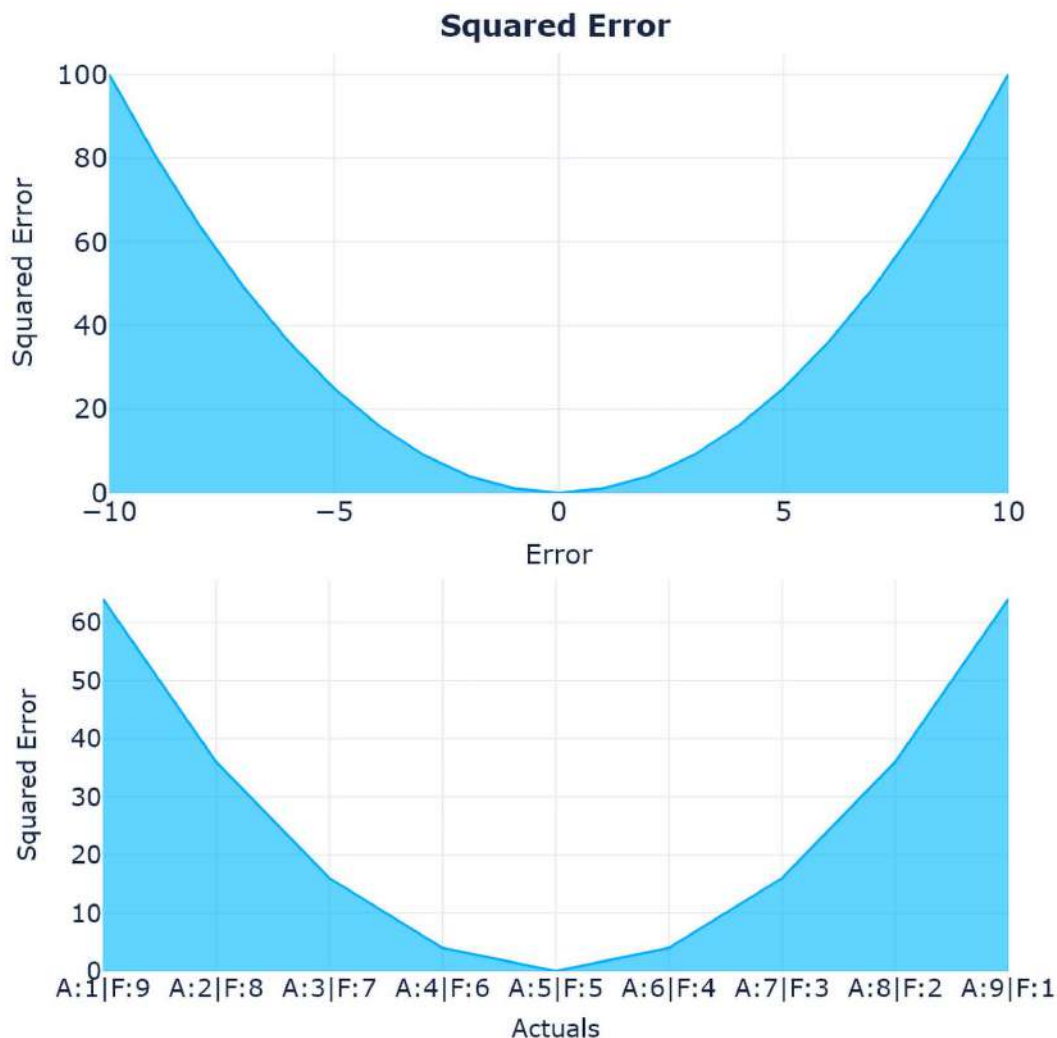


Figure 19.3: The loss curves and complementary pairs for squared error

These charts also look symmetrical, so the squared error also doesn't have an issue with asymmetric error distribution—but we can notice one thing here. The squared error increases exponentially as the error increases. This points to a property of the squared error—it gives undue weightage to outliers. If there are a few timesteps for which the forecast is really bad and excellent at all other points, the squared error inflates the impact of those outlying errors.

Percent error

Now, let's look at the percent error:

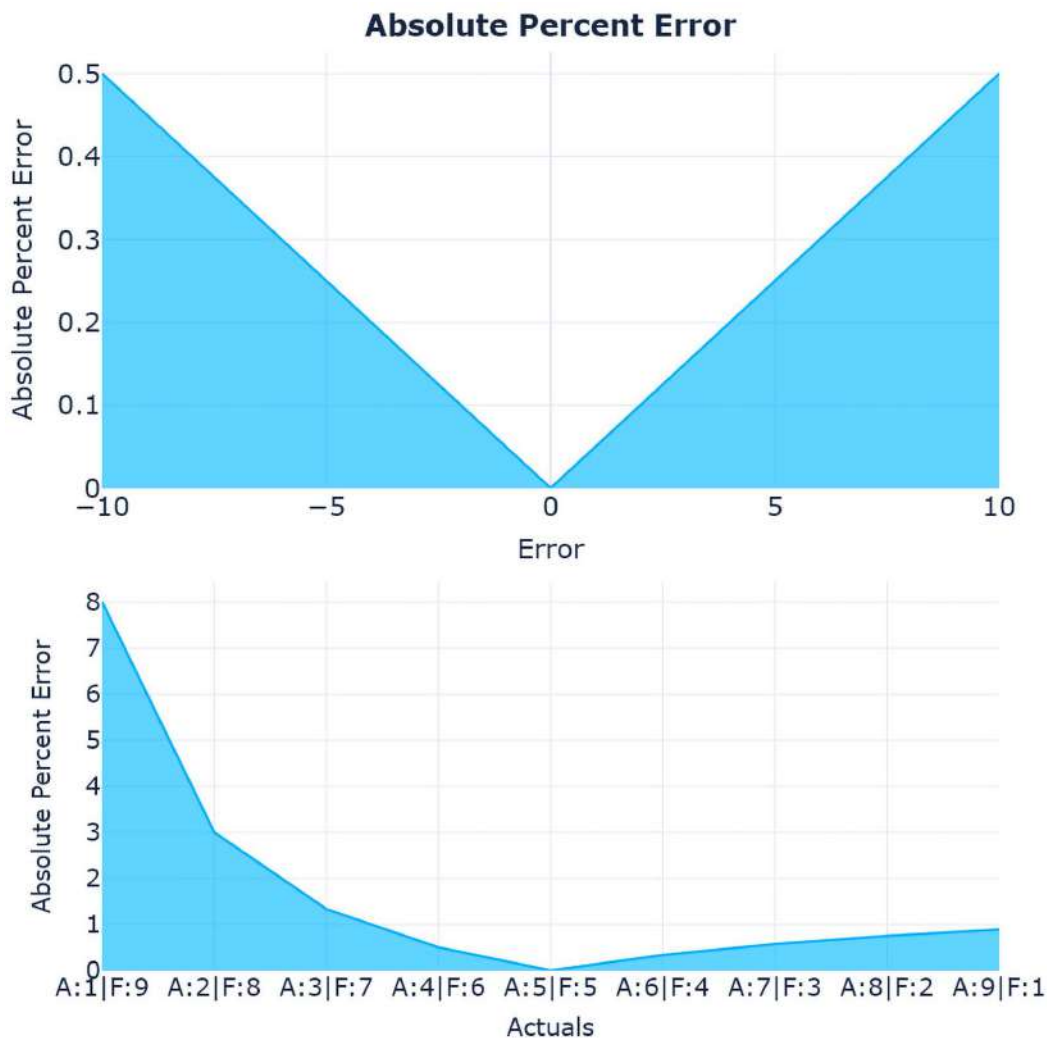


Figure 19.4: The loss curves and complementary pairs for percent error

There goes our symmetry. The percent error is symmetrical when you move away from the actuals on both sides (mostly because we are keeping the actuals constant), but the complementary pairs tell us a whole different story. When the actual is 1 and the forecast is 9, the percent error is 8, but when we swap them, the percent error drops to 1. This kind of asymmetry can cause the metric to favor under-forecasting. The right half of the second chart in Figure 19.4 are all cases where we are under-forecasting and we can see that the error is very low there when compared to the left half.

We will look at under- and over-forecasting in detail in another experiment.

Symmetric error

For now, let's move on and look at the last error we had—the symmetric error:

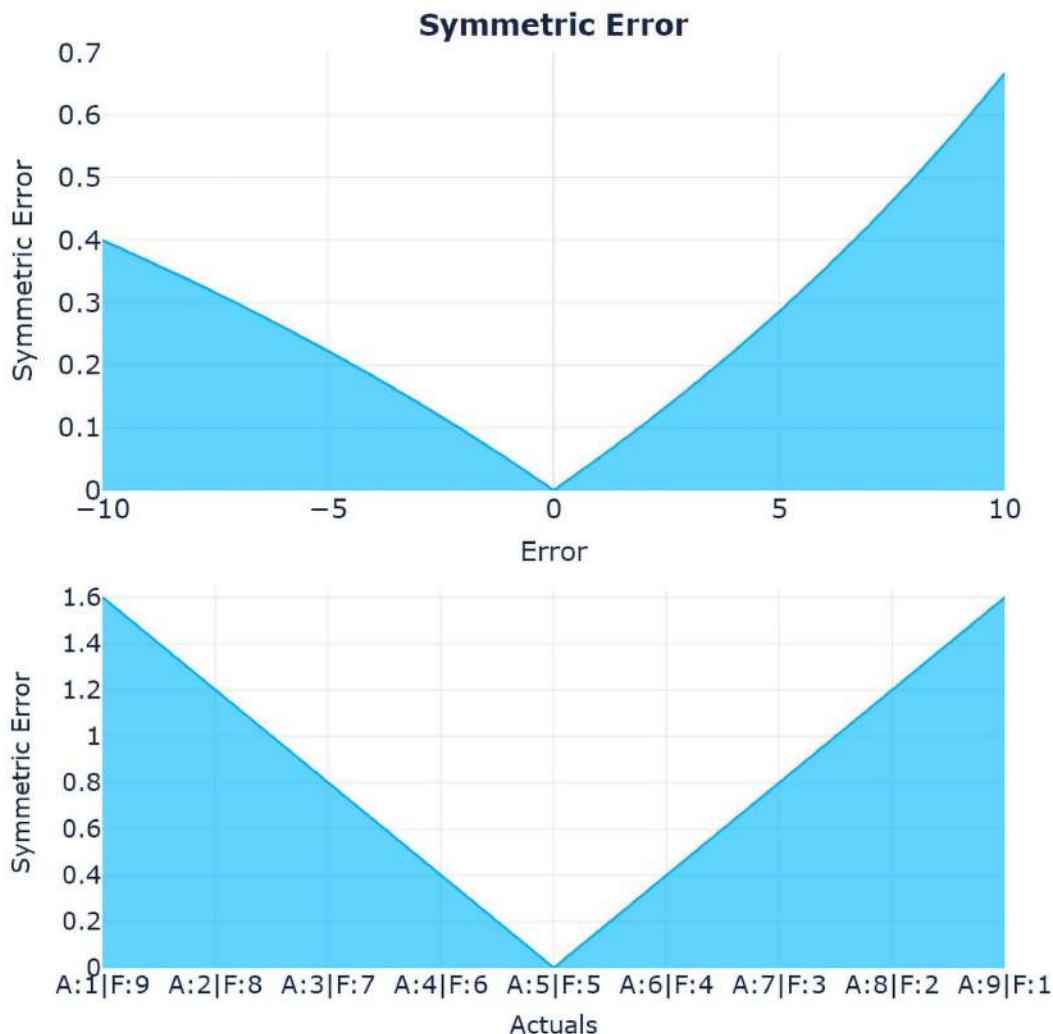


Figure 19.5: The loss curves and complementary pairs for symmetric error

Symmetric error was proposed mainly because of the asymmetry we saw in the percent error. MAPE, which uses percent error, is one of the most popular metrics used, and sMAPE was proposed to directly challenge and replace MAPE. True to its claim, it did resolve the asymmetry that was present in percent error. However, it introduced its own asymmetry. In the first chart, we can see that for a particular actual value, if the forecast moves on either side, it is penalized differently, so in effect, this metric favors over-forecasting (which is in direct contrast to percent error, which favors under-forecasting).

Extrinsic errors

With all the intrinsic measures done, we can also take a look at the extrinsic ones. With extrinsic measures, plotting the loss curves and checking symmetry is not as easy. Instead of two variables, we now have three—the actual observation, the forecast, and the reference forecast. The value of the measure can vary with any of these. We can use a contour plot for this, as shown in *Figure 19.6*:

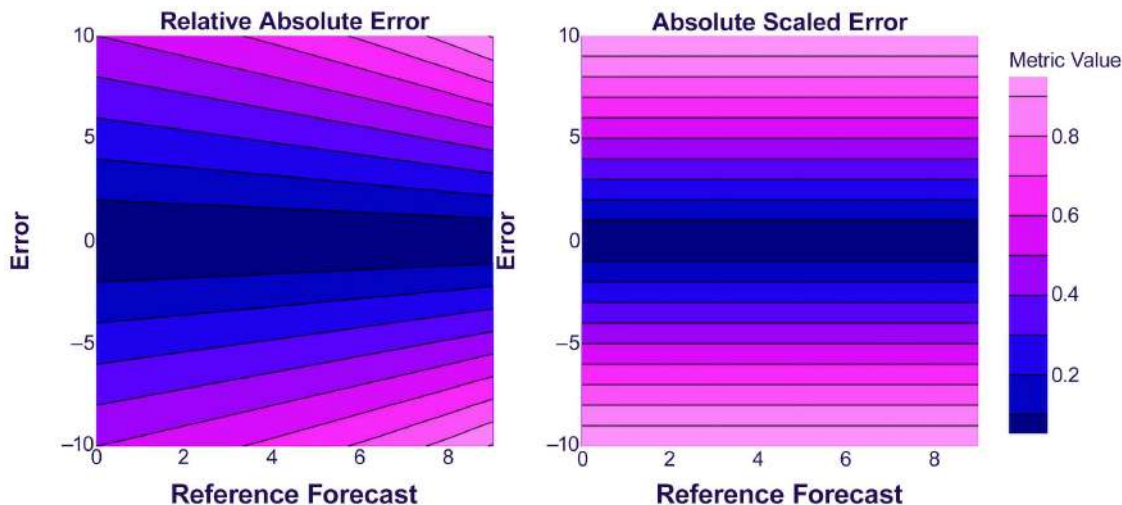


Figure 19.6: Contour plot of the loss surface—relative absolute error and absolute scaled error

 **Quick tip:** Need to see a high-resolution version of this image? Open this book in the next-gen Packt Reader or view it in the PDF/ePub copy.

 The next-gen Packt Reader and a free PDF/ePub copy of this book are included with your purchase. Unlock them by scanning the QR code below or visiting <https://www.packtpub.com/unlock/9781835883181>.



The contour plot enables us to plot three dimensions in a 2D plot. The two dimensions (error and reference forecast) are on the x - and y -axes. The third dimension (the relative absolute error and absolute scaled error values) is represented as color (refer to the color images file: <https://packt.link/gbp/9781835883181>), with contour lines bordering same-colored areas. The errors are symmetric around the error (horizontal) axis. This means that if we keep the reference forecast constant and vary the error, both measures vary equally on both sides of the errors. This is not surprising since both these errors have their base in absolute error, which we know was symmetric.

The interesting observation is the dependency on the reference forecast. We can see that for the same error, a *relative absolute error* has different values for different reference forecasts, but the *scaled error* doesn't have this problem. This is because it is not directly dependent on the reference forecast and rather uses the MAE of a naïve forecast. This value is fixed for a time series and eliminates the task of choosing a reference forecast. Therefore, scaled error has good symmetry for absolute error and very little or fixed dependency on the reference forecast.

Bias toward over- or under-forecasting

We have seen indications of bias toward over- or under-forecasting in a few metrics that we looked at. In fact, it looked like the popular metric, MAPE, favors under-forecasting. To finally put that to the test, we can perform another experiment with synthetically generated time series; we included a lot more metrics in this experiment so that we know which are safe to use and which need to be looked at carefully.



Notebook alert:

The notebook to run these experiments on your own is `02-Over_and_Under_Forecasting.ipynb` in the `Chapter19` folder.

The experiment is simple and detailed as follows:

1. We randomly sample a count time series of integers with a length of 100 from a uniform distribution between 2 and 5:

```
np.random.randint(2,5,n)
```

2. We use the same process to generate a forecast, which is also drawn from a uniform distribution between 2 and 5:

```
np.random.randint(2,5,n)
```

3. Now, we generate two additional forecasts, one from the uniform distribution between 0 to 4 and another between 3 and 7. The former predominantly under-forecasts and the latter over-forecasts:

```
np.random.randint(0,4,n) # Underforecast
np.random.randint(3,7,n) # Overforecast
```

4. We calculate all the measures we want to investigate using all three forecasts.
5. We repeat it 10,000 times to average out the effect of random draws.

After the experiment is done, we can plot a box plot of different metrics so that it shows the distribution of each metric for each of those three forecasts over these 10,000 runs of the experiment. Let's see the box plot in *Figure 19.7*:

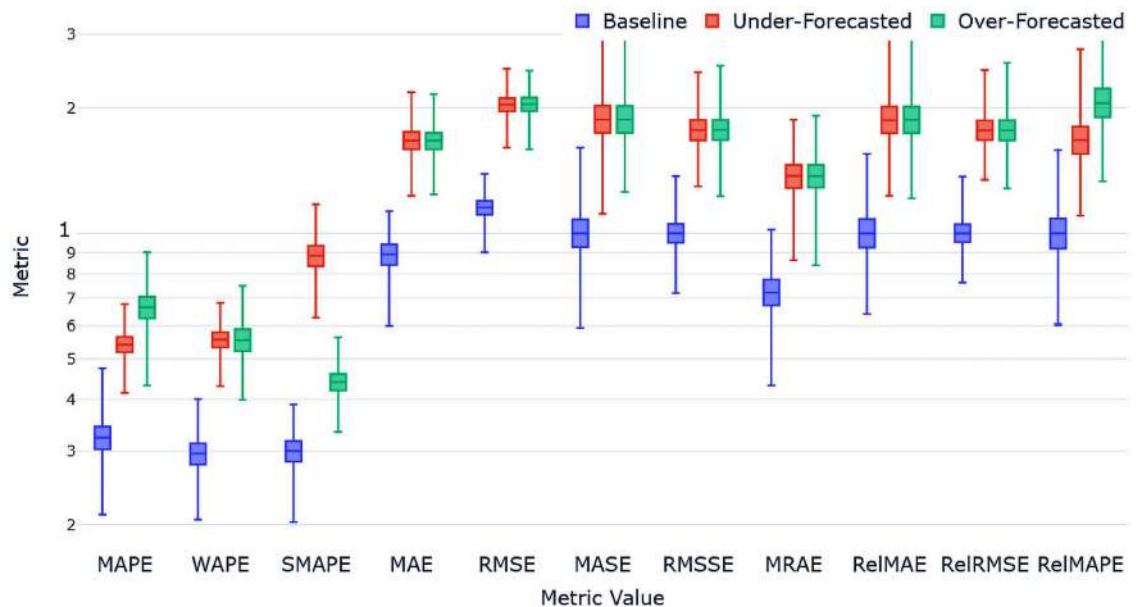


Figure 19.7: Over- and under-forecasting experiment

Let's go over what we would expect from this experiment first. The over- (green) and under- (red) forecasted forecasts would have a higher error than the baseline (blue). The over- and under-forecasted errors would be similar.

With that, let's summarize our major findings:

- MAPE clearly favors the under-forecasted with a lower MAPE than the over-forecasted.
- WAPE, although based on percent error, managed to get over the problem by having explicit weighting. This may be counteracting the bias that percent error has.
- sMAPE, in its attempt to fix MAPE, does a worse job in the opposite direction. sMAPE highly favors over-forecasting.
- Metrics such as MAE and RMSE, which are based on absolute error and squared error, respectively, don't show any preference for either over- or under-forecasting.
- MASE and RMSSE (both using versions of scaled error) are also fine.
- MRAE, in spite of some asymmetry regarding the reference forecast, turns out to be unbiased from the over- and under-forecasting perspective.
- The relative measures with absolute and squared error bases (RelMAE and RelRMSE) also do not have any bias toward over- or under-forecasting.
- The relative measure of mean absolute percentage error, RelMAPE, carries MAPE's bias toward under-forecasting.

We have investigated a few properties of different error measures and understood the basic properties of some of them. To further that understanding and move closer to helping us select the right measure for our problem, let's do one more experiment using the London Smart Meters dataset we have been using through this book.

Experimental study of the error measures

As we discussed earlier, there are a lot of metrics for forecasting that people have come up with over the years. Although there are many different formulations of these metrics, there can be similarities in what they are measuring. Therefore, if we are going to choose a primary and secondary metric while modeling, we should pick some metrics that are diverse and measure different aspects of the forecast.

Through this experiment, we are going to try and figure out which of these metrics are similar to each other. We are going to use the subset of the *London Smart Meters* dataset we have been using all through the book and generate some forecasts for each household. I chose to do this exercise with the *dart*s library because I wanted multi-step forecasting. I used five different forecasting methods—seasonal naïve, exponential smoothing, Theta, FFT, and LightGBM (local)—and generated forecasts. On top of that, I also calculated the following metrics on all of these forecasts: MAPE, WAPE, sMAPE, MAE, MdAE, MSE, RMSE, MRAE, MASE, RMSSE, RelMAE, RelRMSE, RelMAPE, CFE, Forecast Bias, and PB(MAE). In addition to this, we also calculated a few aggregate metrics: meanMASE, meanRMSSE, meanWAPE, meanMRAE, AvgRelRMSE, ND, and NRMSE.

Using Spearman's rank correlation

The basis of the experiment is that if different metrics measure the same underlying factor, then they will also rank forecasts on different households similarly. For instance, if we say that MAE and MASE are measuring one latent property of the forecast, then those two metrics would give similar rankings to different households. At the aggregated level, there are five different models and aggregate metrics that measure the same underlying latent factor and should also rank them in similar ways.

Let's look at the aggregate metrics first. We ranked the different forecast methods at the aggregate level using each of the metrics and then we calculated the Pearson correlation of the ranks. This gives us Spearman's rank correlation between the forecasting methods and metrics. The heatmap (refer to the color images file:<https://packt.link/gbp/9781835883181>) of the correlation matrix is in *Figure 19.8*:

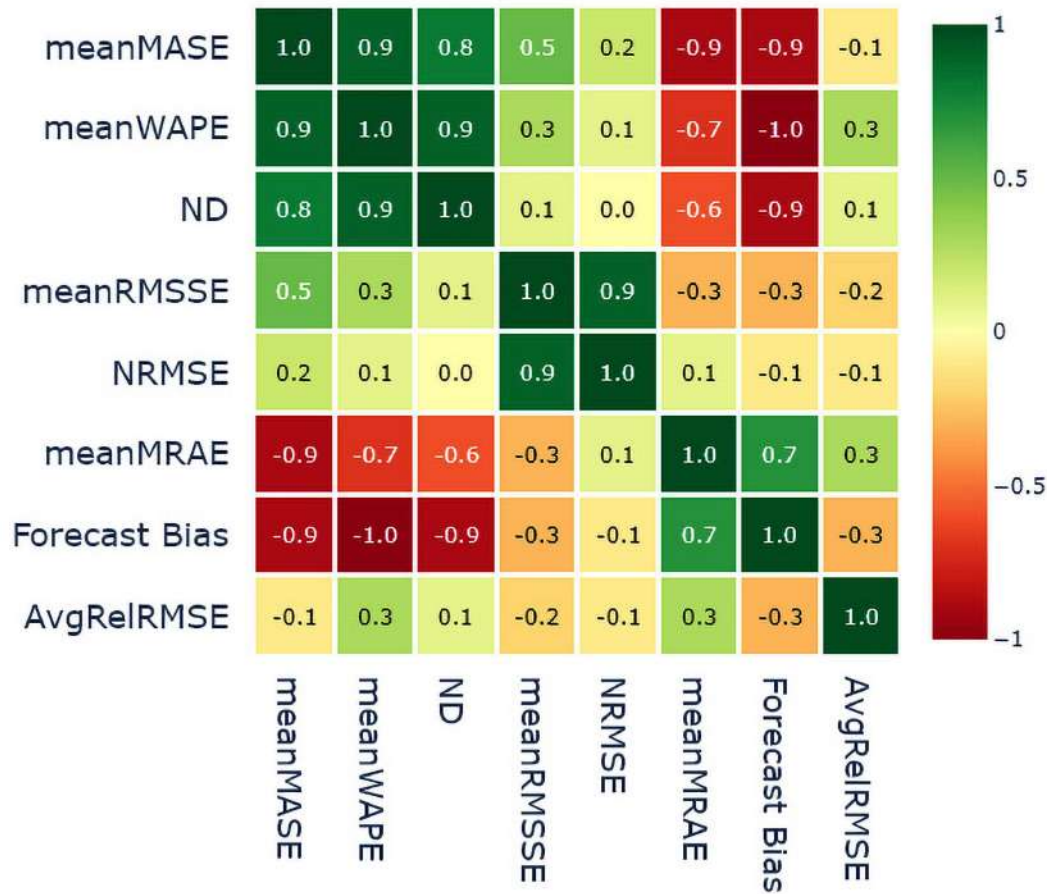


Figure 19.8: Spearman's rank correlation between the forecast methods and aggregate metrics

These are the major observations:

- We can see that *meanMASE*, *meanWAPE*, and *ND* (all based on absolute error) are highly correlated, indicating that they might be measuring similar latent factors of the forecast.
- The other pair that is highly correlated is *meanRMSSE* and *NRMSE*, which are both based on squared error.
- There is a weak correlation between *meanMASE* and *meanRMSSE*, maybe because they are both using scaled error.
- *meanMRAE* and *Forecast Bias* seem to be highly correlated, although there is no strong basis for that shared behavior. Some correlations can be because of chance and this needs to be validated further on more datasets.
- *meanMRAE* and *AvgRelRMSE* seem to be measuring very different latent factors from the rest of the metrics and each other.

Similarly, we calculated Spearman’s rank correlation between the forecast methods and metrics across all the households (Figure 19.9). This enables us to have the same kind of comparison as before at the item level:

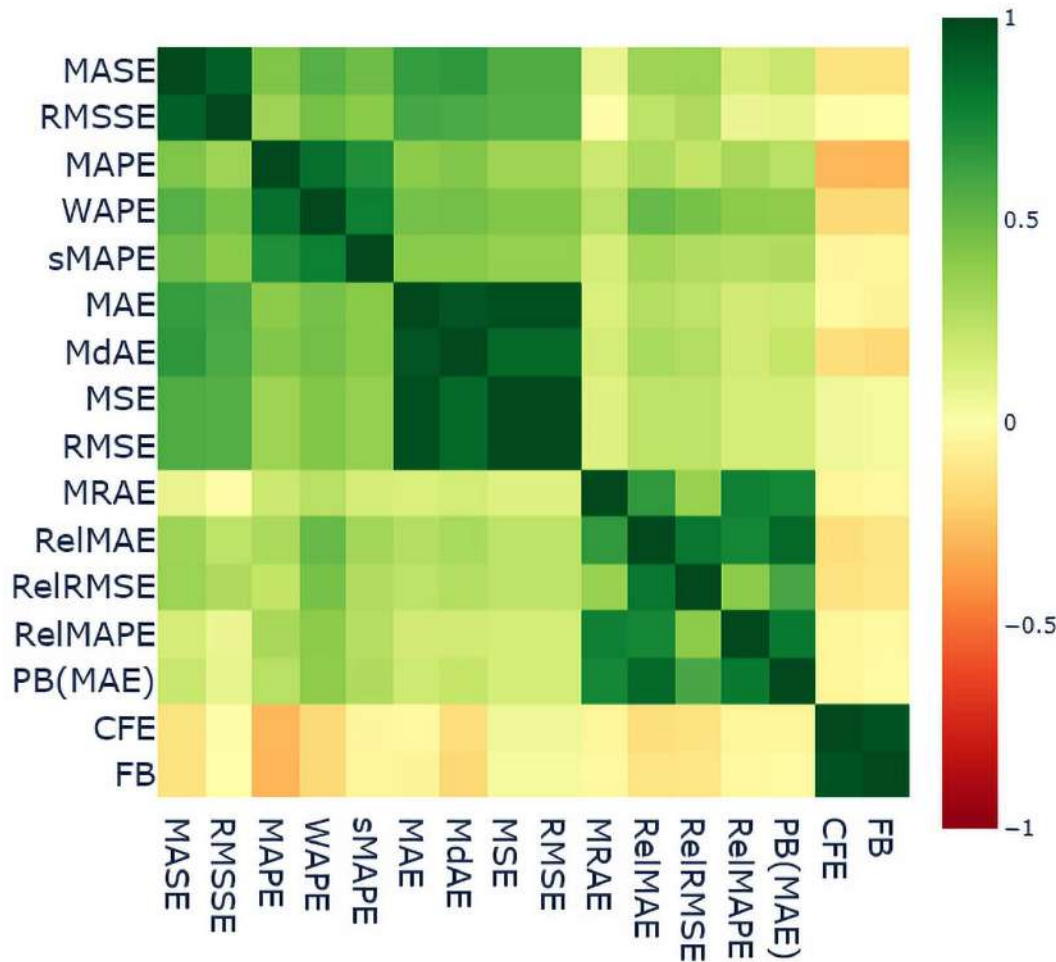


Figure 19.9: Spearman’s rank correlation between the forecast methods and item-level metrics

The major observations are as follows:

- We can see there are five clusters of highly correlated metrics (the five darker green boxes).
- The first group is *MASE* and *RMSSE*, which are highly correlated. This can be because of the scaled error formulation of both metrics.
- *WAPE*, *MAPE*, and *sMAPE* are the second group. Frankly, this is a bit confusing because I would have expected *MAPE* and *sMAPE* to have less correlated results. They do behave in the opposite way from an over- and under-forecasting perspective. Maybe all the forecasts we have used to check this correlation don’t over- or under-forecast and, therefore, the similarity came out through the shared percent error base. This needs to be investigated further.

- *MAE*, *MdAE*, *MSE*, and *RMSE* form the third group of highly similar metrics. *MAE* and *MdAE* are both absolute error metrics and *MSE* and *RMSE* are both squared error metrics. The similarity between these two could be because of the lack of outlying errors in the forecasts. The only difference between these two base errors is that the squared error puts a much greater weight on outlying errors.
- The next group of similar metrics is the motley crew of relative measures—*MRAE*, *RelMAE*, *RelRMSE*, *RelMAPE*, and *PB(MAE)*—but the intercorrelation among this group is not as strong as the other groups. The pairs of metrics that stand out in terms of having low inter-correlations are *MRAE* and *RelRMSE* and *RelMAPE* and *RelRMSE*.
- The last group that stands totally apart with much less correlation with any other metric but a higher correlation with each other is *Forecast Bias* and *CFE*. Both are calculated on unsigned errors and measure the amount of over- or under-forecasting.
- If we look at intergroup similarities, the only thing that stands out is the similarity between the scaled error group and absolute error and squared error group.



Spearman's rank correlation on aggregate metrics is done using a single dataset and has to be taken with a grain of salt. The item-level correlation has a bit more significance because it is made across many households, but there are still a few things in there that warrant further investigation. I urge you to repeat this experiment on some other datasets and check whether you see the same patterns repeated before adopting them as rules.

Now that we have explored the different metrics, it is time to summarize and leave you with a few guidelines for choosing a metric.

Guidelines for choosing a metric

Throughout this chapter, we have come to understand that it is difficult to choose one forecast metric and apply it universally. There are advantages and disadvantages for each metric, and being cognizant of these while selecting a metric is the only rational way to go about it.

Let's summarize and note a few points we have seen through different experiments in the chapter:

- Absolute error and squared error are both symmetric losses and are unbiased from the under- or over-forecasting perspective.
- Squared error does have a tendency to magnify the outlying error because of the square term in it. Therefore, if we use a squared-error-based metric, we will be penalizing high errors much more than small errors.
- *RMSE* is generally preferred over *MSE* because *RMSE* is on the same scale as the original input and, therefore, is a bit more interpretable.

- Percent error and symmetric error are not symmetric in the complete sense and favor under-forecasting and over-forecasting, respectively. MAPE, which is a very popular metric, is plagued by this shortcoming. For instance, if we are forecasting demand, optimizing for MAPE will lead you to select a forecast that is conservative and, therefore, under-forecast. This will lead to an inventory shortage and out-of-stock situations. sMAPE, with all its shortcomings, has fallen out of favor with practitioners.
- Relative measures are a good alternative to percent-error-based metrics because they are also inherently interpretable, but relative measures depend on the quality of the benchmark method. If the benchmark method performs poorly, the relative measures will tend to dampen the impact of errors from the model under evaluation. On the other hand, if the benchmark forecast is close to an oracle forecast with close to zero errors, the relative measure will exaggerate the errors of the model. Therefore, you have to be careful when choosing the benchmark forecast, which is an additional thing to worry about.
- Although a geometric mean offers a few advantages over an arithmetic mean (such as resistance to outliers and better approximation when there is high variation in data), it is not without its own problems. Geometric mean-based measures mean that even if a single series (when aggregating across time series) or a single timestep (when aggregating across timesteps) performs really well, it will make the overall error come down drastically due to the multiplication.
- PB, although an intuitive metric, has one disadvantage. We are simply counting the instances in which we perform better. However, it doesn't assess how well or poorly we are doing. The effect on the PB score is the same, whether our error is 50% less than the reference error or 1% less.

Hewamalage et al. (Reference 1) have proposed a very detailed flowchart to aid in decision-making, but that is also more of a guideline as to what not to use. The selection of a single metric is a very debatable task. There are a lot of conflicting opinions out there and I'm just adding another to that noise. Here are a few guidelines I propose to help you pick a forecasting metric:

- Avoid *MAPE*. In any situation, there is always a better metric to measure what you want. At the very least, stick to *WAPE* for single time series datasets.
- For a single time series dataset, the best metrics to choose are *MAE* or *RMSE* (depending on whether you want to penalize large errors more or not).
- For multiple time series datasets, use *ND* or *NRMSSE* (depending on whether you want to penalize large errors more or not). As a second choice, *meanMASE* or *meanRMSSE* can also be used.
- If there are large changes in the time series (in the horizon we are measuring, there is a huge shift in time series levels), then something such as *PB* or *MRAE* can be used.
- Whichever metric you choose, always make sure to use *Forecast Bias*, *CFE*, or *Tracking Signal* to keep an eye on structural over- or under-forecasting problems.
- If the time series you are forecasting is intermittent (as in, has a lot of time steps with zero values), use *RMSE* and avoid *MAE*. *MAE* favors forecasts that generate all zeros. Avoid all percent-error-based metrics because intermittency brings to light another one of their shortcomings—it is undefined when actual observations are zero (the *Further reading* section has a link to a blog that explores other metrics for intermittent series).

Congratulations on finishing a chapter full of new terms and metrics. I hope you have gained the necessary intuition to intelligently select the metric to focus on for your next forecasting assignment!

Summary

In this chapter, we looked at the thickly populated and highly controversial area of forecast metrics. We started with a basic taxonomy of forecast measures to help you categorize and organize all the metrics in the field.

Then, we launched a few experiments through which we learned about the different properties of these metrics, slowly approaching a better understanding of what these metrics are measuring; looking at synthetic time series experiments, we learned how *MAPE* and *sMAPE* favor under- and over-forecasting, respectively.

We also analyzed the rank correlations between these metrics on real data to see how similar the different metrics are, and finally, rounded off by laying out a few guidelines that can help you pick a forecasting metric for your problem.

In the next chapter (our last chapter), we will look at cross-validation strategies for time series.

References

The following are the references that we used throughout the chapter:

1. Hewamalage, Hansika; Ackermann, Klaus; and Bergmeir, Christoph. (2022). *Forecast Evaluation for Data Scientists: Common Pitfalls and Best Practices*. arXiv preprint arXiv: Arxiv-2203.10716: <https://arxiv.org/abs/2203.10716v2>.
2. Davydenko, Andrey and Fildes, Robert. (2013). *Measuring forecasting accuracy: the case of judgmental adjustments to SKU-level demand forecasts*. In *International Journal of Forecasting*, Vol. 29, No. 3., 2013, pp. 510-522: <https://doi.org/10.1016/j.ijforecast.2012.09.002>.
3. Hyndman, Rob J. and Koehler, Anne B. (2006). *Another look at measures of forecast accuracy*. In *International Journal of Forecasting*, Vol. 22, Issue 4, 2006, pp. 679-688: <https://robjhyndman.com/publications/another-look-at-measures-of-forecast-accuracy/>.

Further reading

- If you wish to read further about forecast metrics, you can check out the blog post *Forecast Error Measures: Intermittent Demand* by Manu Joseph: <https://deep-and-shallow.com/2020/10/07/forecast-error-measures-intermittent-demand/>

Join our community on Discord

Join our community's Discord space for discussions with authors and other readers:

<https://packt.link/mts>



20

Evaluating Forecasts—Validation Strategies

Throughout the last few chapters, we have been looking at a few relevant, but seldom discussed, aspects of time series forecasting. While we learned about different forecasting metrics in the previous chapter, we now move on to the final piece of the puzzle—validation strategies. This is another integral part of evaluating forecasts.

In this chapter, we try to answer the question *How do we choose the validation strategy to evaluate models from a time series forecasting perspective?* We will look at different strategies and their merits and demerits so that, by the end of the chapter, you can make an informed decision to set up the validation strategy for your time series problem.

In this chapter, we will be covering these main topics:

- Model validation
- Holdout strategies
- Cross-validation strategies
- Choosing a validation strategy
- Validation strategies for datasets with multiple time series

Technical requirements

You will need to set up the Anaconda environment by following the instructions in the *Preface* of the book to get a working environment with all the packages and datasets required for the code in this book.

The associated code for the chapter can be found at <https://github.com/PacktPublishing/Modern-Time-Series-Forecasting-with-Python-/tree/main/notebooks/Chapter20>.

Model validation

In *Chapter 19, Evaluating Forecast Errors—A Survey of Forecast Metrics*, we learned about different forecast metrics that can be used to measure the quality of a forecast. One of the main uses for this is to measure how well our forecast is doing on test data (new and unseen data), but this comes after we train a model, tweak it, and tinker with it until we are happy with it. How do we know whether a model we are training or tweaking is good enough?

Model validation is the process of evaluating a trained model using data to assess how good the model is. We use the metrics we learned about in *Chapter 19, Evaluating Forecast Errors—A Survey of Forecast Metrics*, to calculate the goodness of the forecast. But there is one question we haven't answered. Which part of the data do we use to evaluate? In a standard machine learning setup (classification or regression), we randomly sample a portion of the training data and call it validation data, and it is based on this data that all the modeling decisions are taken. The best practice in the field is to use cross-validation. **Cross-validation** is a resampling procedure where we sample different portions of the training dataset to train and test in multiple iterations. In addition to repeated evaluation, cross-validation also makes the most efficient use of the data.

However, in the field of time series forecasting, such a consensus on best practice does not exist. This is mainly because of the temporal nature and the sheer variety of ways we can go about it. Different time series might have different lengths of history and we may choose different ways to model it, or there might be different horizons to forecast for, and so on. Because of the temporal dependence in the data, standard assumptions of i.i.d. don't hold true; therefore, techniques such as cross-validation have their own complications. When randomly chosen, the validation and training datasets may not be independent, which will lead to an optimistic and misleading estimate of error.

There are two main paradigms of validation:

- **In-sample validation:** As the name suggests, the model is evaluated on the same or a subset of the same data that was used to train it.
- **Out-of-sample validation:** Under this paradigm, the data we use to evaluate the model has no intersection with the data used to train the model.

In-sample validation helps you understand how well your model fits the data you have. This was very popular in the era of statistics, where the models were meticulously designed and primarily used for inferencing and not predicting. In such cases, the in-sample error shows how well the specified model fits the data and how valid the inferences we make from that model are. But in a predictive paradigm, like most machine learning, the in-sample error is not the right measure of the *goodness* of a model. Complex models can easily fit the training data, memorize it, and not work well on new and unseen data. Therefore, out-of-sample validations are almost exclusively used in today's predictive model evaluations. Since this book is solely concerned with forecasting, which is a predictive task, we will be sticking to out-of-sample evaluations only.

As discussed earlier, deciding on a validation strategy for forecasting problems is not as trivial as standard machine learning. There are two major schools of thought here:

- Holdout-based strategies, which respect the temporal integrity of the problem
- Cross-validation-based strategies, which sample validation splits with no or a very loose sense of temporal ordering

Let's discuss the major ones in each category. What we have to keep in mind is that all the validation strategies that we discuss in the book are not exhaustive. They are merely a few popular ones. In the explanations that follow, the length of the validation period is L_v and the length of the training period is L_t .

Now, let's look at the first school of thought.

Holdout strategies

There are three aspects of a holdout strategy, and they can be mixed and matched to create many variations of the strategy. For instance, we might have a sampling strategy with a fixed split, a rolling window for the training data, and a recalibration of the model for each iteration. The three aspects are as follows:

- **Sampling strategy:** A sampling strategy is how we sample the validation split(s) from training data.
- **Window strategy:** A window strategy decides how we sample the window of training split(s) from training data.
- **Calibration strategy:** A calibration strategy decides whether a model should be recalibrated or not.

That said, designing a holdout validation strategy for a time series problem includes making decisions on these three aspects.

Sampling strategies are ways to pick one or more origins in the training data. These **origins** are points in time that determine the starting point of the validation split and the ending point of the training split. The exact length of the validation split is governed by a parameter L_v , which is the horizon chosen for validation. The length of the training split depends on the window strategy.

Window strategy

There are two ways we can draw the windows of training split—expanding window and rolling window. *Figure 20.1* shows the difference between the two setups:

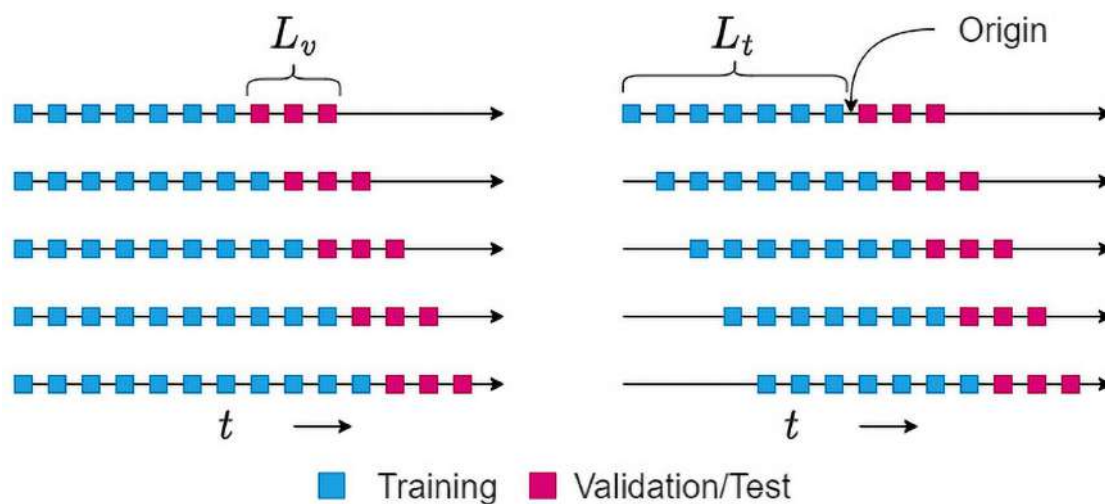


Figure 20.1: Expanding (left) versus rolling (right) window strategies

Under the expanding window strategy, the training split expands as the origin moves forward in time. In other words, under the expanding window strategy, we choose all the data that is available before the origin as the training split. This effectively increases the training length every time the origin moves forward in time.

In the rolling window strategy, we keep the length of the training split constant (L_t). Therefore, when we move the origin forward by three timesteps, the training split drops three timesteps from the start of the time series.



Although the expanding and rolling window concept may remind you of the window we use for feature engineering or use as the context in deep learning models, this window is not the same. The window we talk about in this chapter is the window of training data that we chose to train our model. For instance, the features of a machine learning model may only extend to the 5 days before, and we can have the training split using the last 5 years of data.

There are merits and demerits to both of these window strategies. Let's summarize them in a few key points:

- The expanding window is a good setup for a short time series, where the expanding window leads to more data being available for the models.

- The rolling window removes the oldest data from training. If the time series is non-stationary and the behavior is bound to change as time passes, having a rolling window will be beneficial to keep the model up to date.
- When we use the expanding window strategy for repeated evaluation, such as in cross-validation, the increase in time series length used for training can introduce some bias toward windows with a longer history. The rolling window strategy takes care of that bias by maintaining the same length of the series.

Now, let's look at another aspect of a validation strategy.

Calibration strategy

The calibration strategy is only valid in cases where we do multiple evaluations with different origins. There are two ways we can do evaluations with different origins—recalibrate every origin or update every origin (terminology from Tashman, reference 1).

Under the *recalibrate* strategy, the model is retrained with the new training split for every origin. This retrained model is used to evaluate the validation split. For the *update* strategy, we do not retrain the model but use the trained model to evaluate the new validation split.

Let's summarize a couple of key points to be considered for choosing a strategy here:

- The golden standard is to recalibrate every new origin, but many times, this may not be feasible. In the econometrics/classical statistical models, the norm was to recalibrate every origin. That was feasible because those models are relatively less compute-intensive and the datasets at the time were also small. So, one could refit a model in a very short time. Nowadays, the datasets have grown in size, and so have the models. Retraining a deep learning model every time we move the origin may not be as easy.

Therefore, if you are using modern, complex models with long training times, an update strategy might be better.

- For classical models that run fast, we can explore the recalibration strategy. However, if the time series you are forecasting is so dynamic that the behavior changes very frequently, then the recalibration strategy might be the way to go.

Now, let's get on to the third part of the validation strategy.

Sampling strategy

In the holdout strategy, we sample a point (*origin*) on the time series, preferably toward the end, such that the portion of the time series after the origin is shorter than the portion of the time series before. From this origin, we can use either the *expanding* or *rolling window* strategy to generate training and validation splits. The model is trained on the training split and tested on the held-out validation split. This strategy is simply called the **holdout** strategy. The calibration strategy is fixed at *recalibrate* because we are only testing and evaluating the model once.

The simple holdout strategy has one disadvantage—the forecast measure we have calculated on the held-out data may not be robust enough because of the single evaluation paradigm. We are relying on a single split of data to calculate the predictive performance of the model. For non-stationary series, this can be a problem because we might be selecting a model that captures the idiosyncrasies of the split that we have chosen.

We can get over this problem by repeating the holdout evaluation multiple times. We can either hand-tailor the different origins using business domain knowledge, such as taking into account seasonality or some other factor, or we could sample the origin points randomly. If we repeat this n times, there will be n validation splits, and they may or may not overlap with each other. The performance metric from these repeated trials can be aggregated using a function such as the mean, maximum, and minimum. This is called the **repeated holdout (Rep-Holdout)** strategy.

A note on implementation:



The simple holdout strategy is very simple to implement because we decide the size of the validation split and keep that much from the end of the time series aside for the validation. The **Rep-Holdout** strategy involves sampling multiple windows randomly or using predefined windows as validation splits. We can make use of the `PredefinedSplit` class from `scikit-learn` to this effect.

Figure 20.2 shows the two holdout strategies using an expanding window approach:

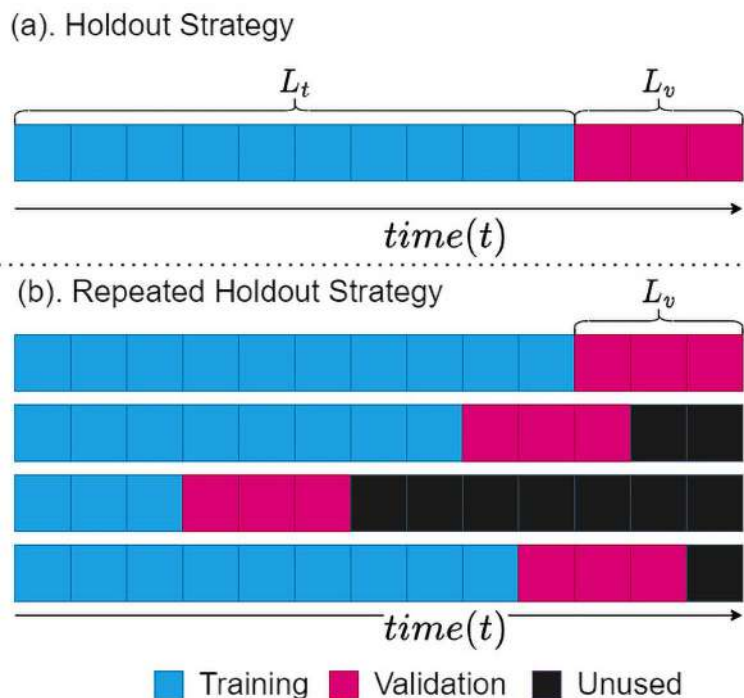


Figure 20.2: Holdout strategy (a) and Rep-Holdout strategy (b)

The Rep-Holdout strategy has a few more variants. The vanilla *Rep-Holdout* strategy evaluates multiple validation datasets, is mostly hand-crafted, and can have overlapping validation datasets. A variation of the Rep-Holdout strategy that insists that multiple validation splits should have no overlap is a more popular option. We call this **Repeated Holdout (No Overlap) (Rep-Holdout-O)**. This has some properties from the cross-validation family and tries to use more data systematically. Figure 20.3 (a) shows this strategy:

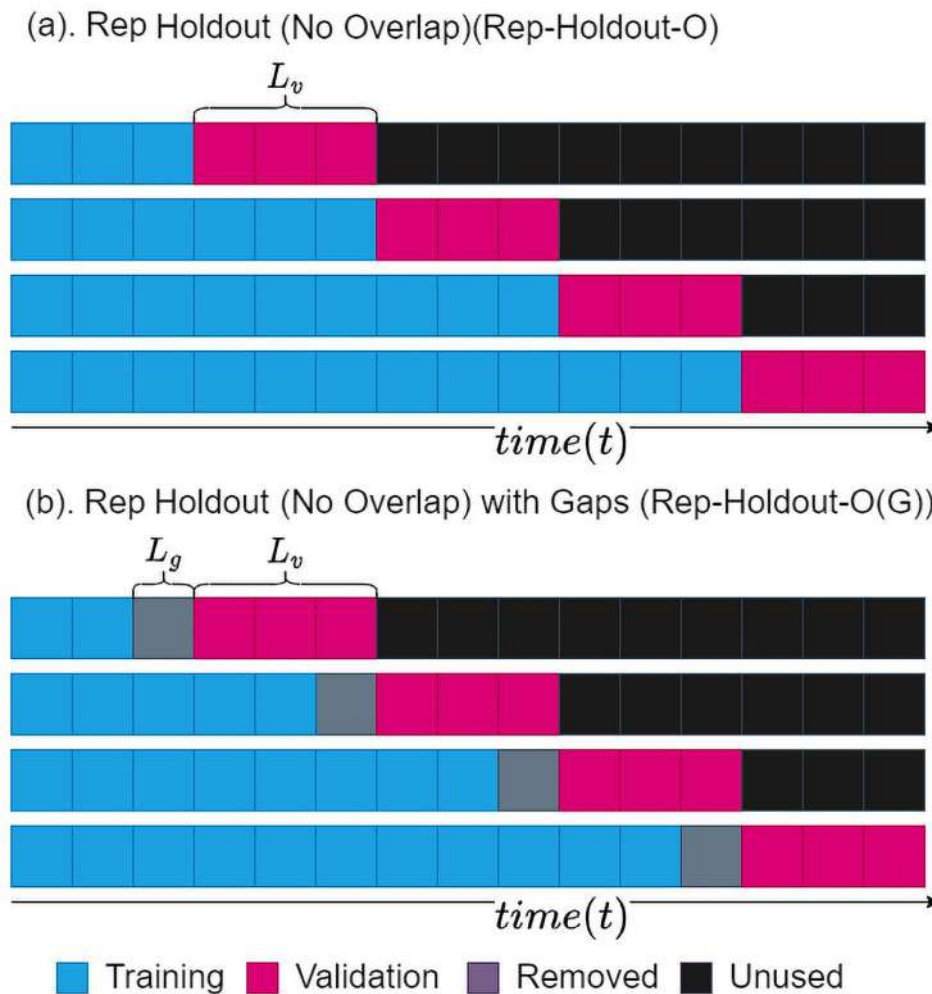


Figure 20.3: Variations of the Rep-Holdout strategy

The *Rep-Holdout-O* strategy is easy to implement in scikit-learn using the `TimeSeriesSplit` class for single time series datasets.

**Notebook alert:**

The associated notebook that shows how to implement different validation strategies can be found in the `Chapter20` folder under the name `01-Validation_Strategies.ipynb`.

The `TimeSeriesSplit` class from `sklearn.model_selection` implements the Rep-Holdout validation strategy and even supports expanding or rolling window variants. The main parameter is `n_splits`. This determines how many splits you want from the data, and the validation split size is decided automatically according to this formula:

$$\text{round}\left(\frac{n_{\text{samples}}}{n_{\text{splits}} + 1}\right)$$

In the default configuration, this implements an expanding window Rep-Holdout-O strategy. But there is a `max_train_size` parameter. If we set this parameter, then it will use a window of `max_train_size` in a rolling window manner.

Yet another variant of the Rep-Holdout strategy introduces a gap of length L_g between the train and validation splits. This is to increase the independence between the train and validation splits, hence getting a better error estimate through the procedure. We call this strategy **Repeated Holdout (No Overlap) with Gaps (Rep-Holdout-O(G))**. This strategy is depicted in *Figure 20.3 (b)*.

We can implement this using the `TimeSeriesSplit` class as well. All we need to do is use a parameter called `gap`. By default, the gap is set to 0. But if we change to a non-zero number, it inserts that much timestep gap between the end of the training and the beginning of validation.

Before we move on to the next set of strategies, let's summarize and discuss some key points about the holdout strategies:

- Holdout strategies respect the temporal integrity of the problem and have been the preferred way of evaluating forecasting models for a long time. However, they do have a weakness in the inefficient use of available data. For short time series, holdout or Rep-Holdout may not have enough training data to train a model.
- A simple holdout depends on a single evaluation, and the error estimate is not robust. Even in a stationary series, this procedure does not guarantee a good estimate of the error. In non-stationary time series, such as a seasonal time series, this problem is exacerbated. But Rep-Holdout and its variants take care of that issue.

Now, let's look at the other major school of thought.

Cross-validation strategies

Cross-validation is one of the most important tools when evaluating standard regression and classification methods. This is because of two reasons:

- A simple holdout approach doesn't use all the data available and, in cases where data is scarce, cross-validation makes the best use of the available data.
- Theoretically, the time series we have observed is one realization of a stochastic process, and so the acquired error measure of the data is also a stochastic variable. Therefore, it is essential to sample multiple error estimates to get an idea about the distribution of the stochastic variable. Intuitively, we can think of this as a “lack of reliability” on the error measure derived from a single slice of data.

The most common strategy that is used in standard machine learning is called **k-fold cross-validation**. Under this strategy, we randomly shuffle and partition the training data into k equal parts. Now, the whole training of the model and calculating the error is repeated k times such that every k subset we have kept aside is used as a test set once, and only once. When we use a particular subset as testing data, we use all the other subsets as the training data. After we acquire k different estimates of the error measure, we aggregate it using a function such as an average. This mean will typically be more robust than a single error measure.

However, there is one assumption that is central to the validity procedure: *i.i.d samples*. This is one assumption that is invalid in time series problems because, by definition, the different samples in time series are dependent on each other through autocorrelation.



Some argue that when we use time delay embedding to convert time series to a regression problem, we can start to use k-fold cross-validation on time series problems. While there are obvious theoretical problems, *Bergmeir et al.* (Reference 2) showed that, empirically, the k-fold cross-validation is not a bad option, but the caveat is that the time series needs to be stationary. We will talk about this in more detail in the next section, where we will discuss the merits and demerits of these strategies.

However, there have been modifications to the k -fold strategy, specifically aimed at sequential data.

Snijders et al. (Reference 4) proposed a modification we call the **Blocked Cross-Validation (BI-CV)** strategy. It is similar to the standard k -fold strategy, but we do not randomly shuffle the dataset before partitioning it into k subsets of length L_n . So, this partitioning strategy results in k contiguous blocks of observations. Then, like a standard k -fold strategy, we train and test each of these k blocks and aggregate the error measure over these multiple evaluations, so the temporal integrity of the problem is satisfied partially.

In other words, temporal integrity is maintained within each of the blocks, but not between the blocks. *Figure 20.4 (a)* shows this strategy:

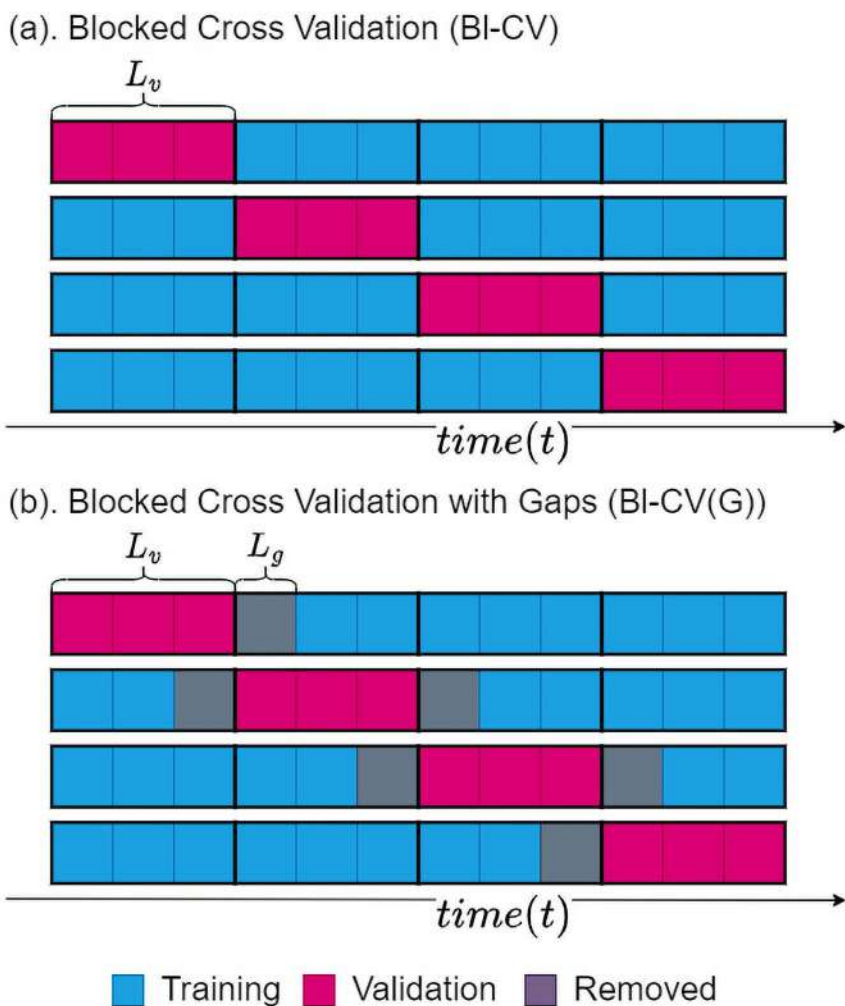


Figure 20.4: BI-CV strategies

To implement the **BI-CV** strategy, we can use the same `Kfold` class from `scikit-learn`. As we saw earlier, the main parameter the cross-validation classes in `scikit-learn` takes in is `n_splits`. Here, `n_splits` also defines the number of equally sized folds it selects. There is another parameter, `shuffle`, which is set to `True` by default. If we make sure our data is sorted according to time and then use the `Kfold` class with `shuffle=False`, it will imitate the **BI-CV** strategy. The associated notebook shows this usage. I urge you to check the notebook for a better understanding of how this is implemented.

In the previous section, we talked about introducing gaps between train and validation splits, to increase independence between them. Another variant of the BI-CV is a version that uses these gaps. We call it **Blocked Cross-Validation with Gaps (BI-CV(G))**. We can see this in action in *Figure 20.4 (b)*.

Unfortunately, the `Kfold` implementation in `scikit-learn` does not support this variant, but it's simple to extend the `Kfold` implementation to include gaps as well. The associated notebook has an implementation of this. It has an additional parameter, `gap`, that lets us set the gap between the train and validation splits.

We saw many different strategies for validation; now let's try and lay down a few points that will help you in deciding the right strategy for your problem.

Choosing a validation strategy

Choosing the right validation strategy is one of the most important but overlooked tasks in the machine learning workflow. A good validation setup will go a long way in all the different steps in the modeling process, such as feature engineering, feature selection, model selection, and hyperparameter tuning. Although there are no hard and fast rules in setting up a validation strategy, there are a few guidelines we can follow. Some of them are from experience (both mine and others) and some of them are from empirical and theoretical studies that have been published as research papers:

- One guiding principle in the design is that we try to make the validation strategy replicate the real use of the model as much as possible. For instance, if the model is going to be used to predict the next 24 timesteps, we make the length of the validation split 24 timesteps. Of course, it's not as simple as that because other practical constraints such as the availability of enough data, time, and computers have to be kept in mind while designing a validation strategy.
- Rep-Holdout strategies that respect the temporal order of the time series problem are the preferred option, especially in cases where there is sufficient data available.
- For purely autoregressive formulations of stationary time series, regular `Kfold` can also be used, and Bergmeir et al. (Reference 2) empirically show that they perform better than holdout strategies. But BI-CV is a better alternative among cross-validated strategies. *Cerqueira et al.* (Reference 3) corroborated the findings in their empirical study for stationary time series.
- If the time series is non-stationary, then *Cerqueira et al.* showed empirically that the holdout strategies (especially Rep-Holdout strategies) are the best ones to choose.
- If the time series is short, using BI-CV after making the time series stationary is a good strategy for autoregressive formulations, such as time delay embedding. However, for models that use some kind of memory of the history to forecast, such as exponential smoothing or deep learning models such as RNN, cross-validation strategies may not be safe to use.
- If we have exogenous variables in addition to the autoregressive part, it may not be safe to use cross-validation strategies. It is best to stick to holdout-based strategies.
- For a strongly seasonal time series, it is beneficial to use validation periods that mimic the forecast horizon. For instance, if we are forecasting for October, November, and December, it is beneficial to check the performance of the model in October, November, and December of last year.

Up until now, we have been talking about validation strategies for a single time series. But in the context of global models, we are at a point where we need to think about validation strategies for such cases as well.

Validation strategies for datasets with multiple time series

All the strategies we have seen until now are perfectly valid for datasets with multiple time series, such as the London Smart Meters dataset we have been working with in this book. The insights we discussed in the last section are also valid. The implementation of such strategies can be slightly tricky because the scikit-learn classes we discussed work for a single time series. Those implementations assume that we have a single time series, sorted according to the temporal order. If there are multiple time series, the splits will be haphazard and messy.

There are a couple of options we can adopt for datasets with multiple time series:

- We can loop over the different time series and use the methods we discussed to do the train-validation split, and then concatenate the resulting sets across all the time series. But that is not going to be so efficient.
- We can write some code and design the validation strategies to use datetime or a time index (such as the one we saw in PyTorch forecasting in *Chapter 15, Strategies for Global Deep Learning Forecasting Models*). I have provided a link to a brilliant notebook from *Konrad Banachewicz* in the *Further reading* section of this chapter, where he uses a custom `GroupSplit` class that uses the time index as the group. This is one way to use Rep-Holdout strategies on a dataset with multiple time series.

There are a few points that we need to keep in mind for datasets with multiple time series:

- Do not use different time windows for different time series. This is because different windows in time would have different errors, and that would skew the aggregate error metric we are tracking.
- If different time series have different lengths, align the length of the validation period across all the series. Training length can be different, but validation windows should be the same so that every time series equally contributes to the aggregate error metric.
- It is easy to get carried away by complicated validation schemes, but always keep the technical debt you incur by choosing a specific technique in mind.

With that, we have come to the end of a short but important chapter.

Summary

We have come to the end of our journey through the world of time series forecasting. In the last couple of chapters, we addressed a few mechanics of forecasting, such as how to do multi-step forecasting and how to evaluate forecasts. Different validation strategies for evaluating forecasts and forecasting models were the topics of the current chapter.

We started by enlightening you as to why model validation is an important task. Then, we looked at a few different validation strategies, such as the holdout strategies, and navigated the controversial use of cross-validation for time series. We spent some time summarizing and laying down a few guidelines to be used to select a validation strategy. To top it all off, we looked at how these validation strategies are applicable to datasets with multiple time series and talked about how to adapt them to such scenarios.

With that, we have come to the end of the book. Congratulations on making it all the way through, and I hope you have gained enough skills from the book to tackle the next time series problem that comes your way. I strongly urge you to start putting into practice the skills that you have gained from the book because, as Richard Feynman rightly put it, “*You do not know anything until you have practiced.*”

References

The following are the references used in this chapter:

1. Tashman, Len. (2000). *Out-of-sample tests of forecasting accuracy: An analysis and review*. In: International Journal of Forecasting. 16. 437–450. 10.1016/S0169-2070(00)00065-0: https://www.researchgate.net/publication/223319987_Out-of-sample_tests_of_forecasting_accuracy_An_analysis_and_review.
2. Bergmeir, Christoph and Benítez, José M. (2012). *On the use of cross-validation for time series predictor evaluation*. In Information Sciences, Volume 191, 2012, Pages 192–213: <https://www.sciencedirect.com/science/article/abs/pii/S0020025511006773>.
3. Cerqueira, V., Torgo, L., and Mozetič, I. (2020). *Evaluating time series forecasting models: an empirical study on performance estimation methods*. Mach Learn 109, 1997–2028 (2020): <https://doi.org/10.1007/s10994-020-05910-7>.
4. Snijders, T.A.B. (1988). *On Cross-Validation for Predictor Evaluation in Time Series*. In: Dijkstra, T.K. (eds) *On Model Uncertainty and its Statistical Implications*. Lecture Notes in Economics and Mathematical Systems, vol 307. Springer, Berlin, Heidelberg. https://doi.org/10.1007/978-3-642-61564-1_4.

Further reading

- *TS-10: Validation methods for time series* by Konrad Banachewicz: <https://www.kaggle.com/code/konradb/ts-10-validation-methods-for-time-series>

**Unlock this book's exclusive
benefits now**

This book comes with additional benefits
designed to elevate your learning experience.



Note: Have your purchase invoice ready before you begin. <https://www.packtpub.com/unlock/9781835883181>



packt.com

Subscribe to our online digital library for full access to over 7,000 books and videos, as well as industry leading tools to help you plan your personal development and advance your career. For more information, please visit our website.

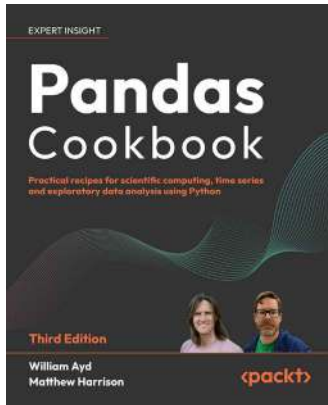
Why subscribe?

- Spend less time learning and more time coding with practical eBooks and Videos from over 4,000 industry professionals
- Improve your learning with Skill Plans built especially for you
- Get a free eBook or video every month
- Fully searchable for easy access to vital information
- Copy and paste, print, and bookmark content

At www.packt.com, you can also read a collection of free technical articles, sign up for a range of free newsletters, and receive exclusive discounts and offers on Packt books and eBooks.

Other Books You May Enjoy

If you enjoyed this book, you may be interested in these other books by Packt:

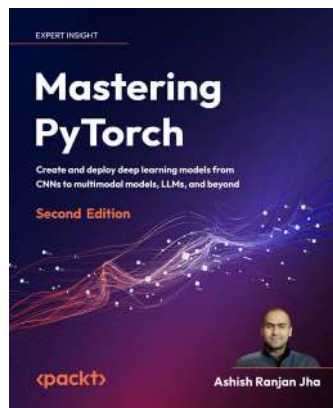


Pandas Cookbook

William Ayd, Matthew Harrison

ISBN: 9781836205876

- The pandas type system and how to best navigate it
- Import/export DataFrames to/from common data formats
- Data exploration in pandas through dozens of practice problems
- Grouping, aggregation, transformation, reshaping, and filtering data
- Merge data from different sources through pandas SQL-like operations
- Leverage the robust pandas time series functionality in advanced analyses
- Scale pandas operations to get the most out of your system
- The large ecosystem that pandas can coordinate with and supplement



Mastering PyTorch

Ashish Ranjan Jha

ISBN: 9781801074308

- Implement text, vision, and music generation models using PyTorch
- Build a deep Q-network (DQN) model in PyTorch
- Deploy PyTorch models on mobile devices (Android and iOS)
- Become well versed in rapid prototyping using PyTorch with fastai
- Perform neural architecture search effectively using AutoML
- Easily interpret machine learning models using Captum
- Design ResNets, LSTMs, and graph neural networks (GNNs)
- Create language and vision transformer models using Hugging Face

Packt is searching for authors like you

If you're interested in becoming an author for Packt, please visit authors.packtpub.com and apply today. We have worked with thousands of developers and tech professionals, just like you, to help them share their insight with the global tech community. You can make a general application, apply for a specific hot topic that we are recruiting an author for, or submit your own idea.

Share your thoughts

Now you've finished *Modern Time Series Forecasting with Python, Second Edition*, we'd love to hear your thoughts! If you purchased the book from Amazon, please [click here](#) to go straight to the Amazon review page for this book and share your feedback or leave a review on the site that you purchased it from.

Your review is important to us and the tech community and will help us make sure we're delivering excellent quality content.

Index

A

absolute error

- Geometric Mean Absolute Error 570
- Mean Absolute Error (MAE) 570
- Median Absolute Error 570
- Weighted Mean Absolute Error 570

activation functions 284

- hyperbolic tangent (tanh) 286
- rectified linear units (ReLUs) 287
- sigmoid 285

Adaptive Conformal Inference (ACI) 522, 538

Add and Norm block 450

Aggregate-Disaggregate Intermittent Demand Approach (ADIDA) 539

aggregate metrics 208, 568

Akaike Information Criterion (AIC) 98

Aleatoric Uncertainty 473

algorithmic partitioning 260-264

alignment functions 360

- additive/concat attention 362, 363
- dot product 360, 361
- general attention 362
- scaled dot product attention 361

Anaconda environment 147

attention 355-358

- Bahdanau, versus Luong 366
- forecasting 364-367

Augmented Dickey-Fuller (ADF) test 152

AutoARIMA 89

autocorrelation 88

Auto-Correlation block 434

autocorrelation function (ACF) 161

auto-correlation mechanism, Autoformer 436

- period-based dependencies 436, 437
- time delay aggregation 437

autocorrelation plots 59, 60

Autoformer 431

- architecture 432
- auto-correlation mechanism 432, 436
- decomposition architecture 432-434
- forecasting with 438
- generative-style decoder 432, 433
- parameters 438
- Uniform Input Representation 432

AutoML approach

- for transforming target 169-172

autoregressive (AR) component 98

autoregressive (AR) signal 12

AutoRegressive Integrated Moving Average (ARIMA) models 88-90

AutoRegressive Moving Average (ARMA) 88, 97, 98

AutoStationaryTransformer

- using 209, 210

B

Back Propagation Through Time (BPTT) 309
backtesting 18
backward fill 30
backward propagation 291
bagging 198
base learners 198
baseline 77
baseline forecasts
 ARIMA models 88-90
 evaluating 101-103
 exponential smoothing 85-88
 generating 80-82
 moving average forecast 83
 MSTL 99-101
 naïve forecast 82, 83
 seasonal naïve forecast 84
 TBATS 93, 94
 Theta forecast 91, 92
batch gradient descent 297
 advantages 297
 disadvantages 297
batch normalization 378
best fit forecast 215
bidirectional RNNs 310
biological neuron 277
blending 229, 230
Blocked Cross-Validation (BI-CV) strategy 599
 implementing 600
Blocked Cross-Validation with Gaps (BI-CV(G))
 strategy 601
block shuffling 108
boosting 172
Box-Cox transformation 94, 95, 167-169

C

calendar heatmaps 58
calibration strategy 593-595
classical statistical models 80
classification 118
Classification and Regression Trees (CART) 195
Coefficient of Variation (CoV) 103, 104
cold-start forecasting 540
 Foundational Time Series Models 540
 Global Machine Learning Models 540
 Launch Profile Models 540
 Manual Substitution Mapping 540
compact data 34
concept drift 148
Conditional Feature Mixing (CFM) layer 460
Confidence Interval (CI) 474
confidence levels 475
config (SingleStepRNNConfig) parameters 336
Conformal Prediction 514
 Adaptive Conformal Inference (ACI) 522, 523
 Conformalized Quantile Regression 517-519
 exchangeability 520, 521
 for classification 515, 516
 forecasting with 523, 524
 for regression 516
 uncertainty estimates, conformalizing 519
 Weighted Conformal Prediction 521
constraints 251
context 356
Conv1d layer 435
convolution 318
convolutional neural networks (CNNs) 318
 causal convolutions 322
 convolution 318-320
 dilations 321, 322

- in PyTorch 323, 324
- padding 320
- stride 321, 322
- cross-entropy loss** 290
- cross-validation** 122, 592
 - strategies 599
- Croston model** 538
- Cumulative Distribution Function (CDF)** 491
- Cumulative Forecast Error (CFE)** 573
- cyclical component** 51
- cyclical/seasonal signals** 10-12
- D**
- data-generating process**
 - (DGP) 5-7, 29, 71, 116, 125, 565
- data leakage**
 - avoiding 131, 132
 - target leakage 131
 - train-test contamination 131
- dataloader** 334
 - creating, from TimeSeriesDataset 396
 - working 397
- data model**
 - preparing 23, 24
- datasets, with multiple time series**
 - validation strategies 602
- date columns**
 - converting,
 - into pd.Timestamp/DatetimeIndex 25, 26
- date offsets**
 - managing 28, 29
- date sequences**
 - creating 28
- datetime property**
 - using 27
- decision trees** 193-196
 - anatomy 194
 - branch 194
 - decision node 194
 - features 197, 198
 - forecast 197
 - leaf node 194
 - root node 194
 - splitting 194
- decoder, Transformer model** 379
 - masked self-attention 379
- decomposition architecture, Autoformer** 434
 - decoder 435, 436
 - encoder 434
- deep learning** 271, 275, 276
 - features 272
 - increase, in compute availability 272
 - increase, in data availability 273-275
- deep learning system, components** 281, 282
 - activation functions 284
 - backward propagation 291
 - forward propagation 291
 - gradient descent 291-296
 - linear transformation 284
 - loss function 290
 - output activation functions 289
 - representation learning 282, 283
- deep neural networks (DNNs)** 378
- deseasonalizing** 60, 61
 - Fourier series 61-63
 - period adjusted averages 61
- deseasonalizing transform** 163-165
- deterministic trend** 154
- detrended time series** 60
- detrending** 60
 - LOESS algorithm 60, 61
 - moving averages 60
- detrending transform** 160
- differencing** 89
- differencing transform** 152, 153
- dilation** 321

direct strategy 552
 forecasting regime 553
 training regime 552, 553
DirRec strategy 555, 556
 forecasting regime 556, 557
 training regime 556
distribution function 363
D-NonLinear model architecture 505
dot product 360, 361
double exponential smoothing (DES) 85, 86
drop column importance 265
.dt accessor property
 using 27
dubbed ES-RNN 420
dummy variable encoding 245
dynamic time-of-use (dToU) 22
Dynamic Time Warping (DTW) distance 260

E

econometrics models 80
efficient market hypothesis (EMH) 17
electrocardiogram (EKG) 5
electroencephalogram (EEG) 5
encoder-decoder architecture 302, 303
encoder, Transformer model 376, 377
 layer normalization 378, 379
 residual connections 377
endogenous variables 19
end-to-end (E2E) model 332
ensemble learning 198
entmax 364
entropy-based measures 105-107
 spectral entropy 106, 107
Epistemic Uncertainty 473
error measures
 experimental study 584

 Spearman's rank correlation, using 584-587
error measures investigation 576
 bias toward over- or under-forecasting 582, 584
 loss curves and complementarity 576
error rates 475
Euclidean distance 188
evaluation metric
 selecting 79, 80
exogenous variables 19
expanded data 34
explanatory forecasting 18
exploding gradients 313
Exploratory Data Analysis (EDA) 22, 49, 52, 77
Exponential Linear Unit (ELU) 453
exponentially weighted moving average (EWMA) 139-141
exponential smoothing 85-88
expressiveness ratio 430
extreme studentized deviate (ESD) 73
extrinsic metrics, forecast error measures 574
 Percent Better (PB) 576
 relative error 574, 575
 scaled error 575

F

Facebook AI Research (FAIR) Lab 305
Fast Fourier Transform (FFT) 106, 436
Feature-Based Forecast Model Averaging (FFORMA) 232
FeatureConfig 177-179
feature engineering 130, 131
feature_importance function 182
feature space 302
feed-forward networks (FFNs) 303-307, 416
 hidden layers 304
 input layer 304
 output layer 304

files

saving, and loading to disk 38, 39

fit function 181**forecastability, of time series**

Coefficient of Variation (CoV) 103, 104

entropy-based measures 105, 106

Kaboudan metric 108, 109

residual variability (RV) 104

Forecast Bias (FB) 79, 80, 573**forecast combinations 19****forecast error measures**

extrinsic metrics 574

intrinsic metrics 569

taxonomy 568, 569

forecast horizons

setting 132

forecasting 18**forecasting, with Conformal****Prediction 523, 524**

Adaptive Conformal Inference 533-536

Conformalized Quantile Regression 527, 528

Conformal Prediction for regression 525-527

results, evaluating 536-538

uncertainty estimates, conformalizing 528-530

Weighted Conformal Prediction 530-533

forecast period 420**forecasts**

combining 214

strategies for combining 214

forward fill 30**forward propagation 290****Fourier decomposition 66-68****Fourier series 61-63**

using, for seasonal decomposition 96

Fourier terms 124, 143-145**frequency encoding 245, 246****fully connected networks 303, 304****functime 146****G****Gated Linear Unit (GLU) 376, 450****gated recurrent unit (GRU) 316, 332**

architecture 316, 317

in PyTorch 318

Gated Residual Networks (GRNs) 450**Gaussian Process (GP) 503****generalization capability 118, 119****generalized attention model 358-360**

alignment function 359, 360

distribution function 359-364

generative-style decoder 433**GFM(DL) 401****GFM(ML) 401****global deep learning forecasting models**

building 398

creating 390

data preprocessing 391-394

RNN model, defining 398

RNN model, initializing 399

RNN model, training 399, 400

TimeSeriesDataset 394

trained model, forecasting with 400, 401

Global Forecasting Models 124-126

advantages 236

creating 239, 240

cross-learning 237

engineering complexity 238, 239

features 236

improvement techniques 241, 242

interpretability 265

interpretation techniques 265

multi-task learning 238

permutation feature importance 265

post hoc interpretation techniques 265

sample size 236

Shapley values 265

transparency 265

global or cross-learning 125

GLU + AddNorm block 451

gradient boosting decision trees 201-207, 251, 483

gradient descent 291-296

batch gradient descent 297

mini-batch gradient descent 297

stochastic gradient descent (SGD) 297

gross domestic product (GDP) 18

Grubbs's Test 73

H

half-hourly block-level data (hhblock)

converting, into time series data 34

helper functions

for evaluating models 182

heteroscedasticity 16

correcting 165

detecting 165, 166

log transform 166, 167

hierarchical forecasting 541

hill climbing 218

holdout strategies 593

calibration strategy 593-595

sampling strategy 593-598

window strategy 593, 594

holdout (test)

creating 78, 79

Holt-Winters' seasonal smoothing 85

HuberRegressor 232

hybrid strategies, multi-step forecasting

DirRec strategy 555

iterative block-wise direct (IBD) strategy 557

RecJoint 562

rectify strategy 560

hyperparameters 121, 251

hyperparameters tuning 251

Bayesian optimization 254-257

grid search 252, 253

random search 253, 254

I

ideal target function 116

improvement techniques, GFMs 241, 242

hyperparameters tuning 251, 252

memory, increasing 242

partitioning 258

time series meta-features, using 243

independent and identically

distributed (iid) 123, 230

indexing 27, 28

Individual Conditional Expectation (ICE)

Plots 266

inheritance 506

input space 302

in-sample metrics 18

in-sample validation 592

Instance-wise Feature Importance

in Time (FIT) 539

Intermittent Multiple Aggregation Prediction

Algorithm (IMAPA) 539

intermittent time series forecasting 538

interpretation techniques, GFMs

drop column importance 265

Individual Conditional Expectation (ICE)

Plots 266

LIME 266

mean decrease in impurity 265

Partial Dependence Plots (PDP) 266

permutation Importance 265

SHAP 266

interquartile range (IQR) 57, 72

intrinsic metrics, forecast error measures 569

- absolute error 570, 571
- Cumulative Forecast Error (CFE) 573
- Forecast Bias (FB) 573
- percent error 572
- squared error 571, 572
- symmetric error 572
- Tracking Signal (TS) 573

irregular component 51, 52**irregular time series 4****Isolation Forest 72****iterative block-wise direct (IBD) strategy 557**

- forecasting regime 558
- training regime 558

iterative multi-SVR strategy 557**iTransformer 446**

- architecture 446, 447
- forecasting with 447

J**joint strategy 554**

- forecasting regime 555
- training regime 555

judgmental partitioning 259, 260**K****Kaboudan metric 108, 109****Kendall's Tau 156, 157****kernel 319****KernelSHAP 539****k-fold cross-validation 122, 599****L****Lagrangian Multiplier (LM) 166****lags/backshift 133, 134****lasso regression 186****layer normalization 378****leaky ReLU 288****learning rate 293****LGBMRegressor 495****LIME (Local Interpretable Model-agnostic Explanations) 266****linear interpolation 31****Linear Projection 465****linear regression 88, 183-186****linear transformation 284****line charts 52-55****Locality Encoded Context Vectors 450****Locality Enhancement (LE) Seq2Seq layer 450****Locally Estimated Scatterplot Smoothing (LOESS) 60, 64, 96****log transform 166, 167****London Smart Meters dataset**

- converting, into time series format 36, 37

longer periods, of missing data

- handling 39-43
- hourly average for each weekday 45, 46
- hourly average profile 44, 45
- inputting, with previous day 43, 44
- seasonal interpolation 46-48

long short-term memory (LSTM)**networks 313, 332, 377**

- architecture 313, 314
- in PyTorch 315, 316

Long-Term Time Series Forecasting (LTSF)-**Linear family 439**

- D-Linear 440
- forecasting with 441, 442
- linear layer 439
- N-Linear 440, 441

lookback period 420**loss curves and complementary pairs**

- for absolute error 576, 577
- for extrinsic errors 581, 582

- for percent error 579
- for squared error 578
- for symmetric error 580

loss function 290

M

machine learning (ML) 3, 115, 147

- hyperparameters 121, 122
- overfitting 118, 119
- supervised machine learning tasks 118
- underfitting 119, 121
- validation sets 122
- versus traditional programming 116-118

machine learning models

- predicting 176
- standardized code, for training and evaluation 177
- training 176

Manhattan distance 188

Mann-Kendall test (M-K test) 157-160

Markov models 123

Masked Interpretable Multi-Head Attention block 452

matrix 280

maximum likelihood estimation (MLE) 289

McCulloch-Pitts model

- limitations 277

Mean Absolute Error (MAE) 42, 79, 251, 290

Mean Absolute Scaled Error (MASE) 79

mean inter-demand interval (MI) 539

Mean Squared Error (MSE) 79, 183, 290

memory cell 313

memory increase

- EWMA features, adding 243
- lag features, adding 243
- rolling features, adding 243

meta model 229

metrics

- guidelines, for selection 587-589

mini-batch gradient descent 297

missing data

- handling 29, 30
- hhblock, converting into time series data 34
- longer periods, handling 39-43

MissingValueConfig 179, 180

MLForecast 181

ModelConfig 180, 181

model validation 592

- in-sample validation 592
- out-of-sample validation 592

modified Kaboudan metric 108

Monte Carlo Dropout 502-504

- custom model, creating with neuralforecast 505-508
- forecasting with 508-514

moving average 60, 139

moving average forecast 83

multi-headed attention (MHA) 370

Multi-Layer Perceptrons (MLPs) 456

multiple households

- predicting 207, 208
- training 207, 208

multiple input, multiple output (MIMO) 554

multiple input, single output (MISO) 554

Multiple Seasonal-Trend decomposition using LOESS (MSTL) 99-101

multi-rate signal sampling 429

multi-step forecasting 547, 548

- direct strategy 552
- hybrid strategies 555
- joint strategy 554
- recursive strategy 549
- standard notation 548
- strategy selection 563, 564, 565

multivariate forecasting 18

N

naïve forecast 82, 83

naïve forecasting method 79

natural language processing (NLP) 367, 450

N-BEATS model 420

architecture 420-422

basis functions 423

blocks 421

forecasting with 424

interpretability 423-426

stacks 422

N-BEATSx model 426

exogenous blocks 427

exogenous variables, handling 427

nearest interpolation 31

NeuralForecast 417

“auto” models 419

common parameters 417-419

exogenous features 419

neural network (NN) models 367

neuron 276

NGBoost

parameters 483

N-HiTS 428

architecture 428

forecasting with 431

hierarchical interpolation 429, 430

input sampling and output interpolation,
synchronizing 430

multi-rate data sampling 429

NIXTLA

URL 80

No Free Lunch Theorem (NFLT) 249

non-linear interpolation techniques

spline, polynomial, and other
interpolations 32, 33

non-stationary time series 14

change in mean over time 14-16

change in variance over time 16

detecting, in time series 148-150

normalization 378

Normalized Deviation (ND) 571

O

objective function 251

one-hot encoding 244, 245

optimal weighted ensemble 226-228

ordinal encoding 244

ordinary least squares (OLS) 183

outlier 71

treating 74

outlier detection, techniques 71

extreme studentized deviate (ESD) 73

IQR 72

Isolation Forest 72, 73

seasonal ESD (S-ESD) 73

standard deviation 71

out-of-sample validation 592

output activation functions 289

softmax 289, 290

out-sample metrics 18

overfitting 119

over- or under-forecasting 568

overparameterization 274

P

padding 320

pandas datetime operations 25

date columns, converting into pd.Timestamp/
DatetimeIndex 25, 26

date offsets, managing 28, 29

date sequences, creating 28, 29

- datetime property, using 27
 - .dt accessor property, using 27
 - indexing 27, 28
 - slicing 27, 28
 - parameter optimization** 98, 99
 - parameter sharing** 308
 - parametrized ReLU** 288
 - partial autocorrelation** 59
 - Partial Dependence Plots (PDP)** 266
 - partitioning** 258
 - algorithmic partitioning 260-264
 - judgmental partitioning 259, 260
 - random partition 259
 - Patch Time Series Transformer (PatchTST)** 442
 - architecture 443
 - channel independence 444, 445
 - forecasting with 445
 - patching 443, 444
 - Percent Better (PB)** 576
 - percent error** 572
 - Mean Absolute Percent Error (MAPE) 572
 - Median Absolute Percent Error 572
 - Weighted Average Percent Error (WAPE) 572
 - Perceptron** 276-278
 - components 278
 - mathematical form 279
 - period adjusted averages** 61
 - permutation Importance** 265
 - point-of-sale (POS)** 29
 - positional encoding** 372
 - power spectral density (PSD)** 106
 - predictability**
 - factors 17
 - predict function** 181
 - Prediction Intervals** 474
 - Average Length 477, 478
 - confidence levels 475
 - Coverage 477
 - error rates 475
 - for standard normal distribution 476
 - goodness, measuring 476, 477
 - quantiles 475
 - Predictive Uncertainty**
 - Aleatoric Uncertainty 473
 - Epistemic Uncertainty 473
 - pre-whitening** 158
 - Principal Component Analysis (PCA)** 303
 - probabilistic forecasting** 472
 - Conformal Prediction 514
 - Monte Carlo Dropout 502
 - Prediction Intervals 474
 - Predictive Uncertainty 473
 - Probability Density Function (PDF) 478-480
 - quantile function 491
 - Probability Density Function (PDF)** 478-480
 - deep learning models 486-491
 - machine learning models 480-486
 - pystacknet Python library**
 - reference link 230
 - PyTorch library** 305
 - PyTorch Lightning** 335
 - reference link 335
 - PyTorch Tabular** 328-331
 - reference link 328
- ## Q
- quantile function** 491-493
 - forecasting, with quantile loss (deep learning) 498-502
 - forecasting, with quantile loss (machine learning) 494-498
 - quantiles** 475

R

Random Forest 198

- feature importance 201
- forecast 200
- using 199

random partition 259

RecJoint 562

- forecasting regime 562
- training regime 562

rectified linear units (ReLU) 286

- advantages 287
- leaky ReLU 288
- parametrized ReLU 288

rectify strategy 560

- forecasting regime 561
- training regime 561

recurrent neural network (RNN) 308, 356

- architecture 308
- bidirectional RNNs 310
- inputs, processing 309, 310
- in PyTorch 310-313

recursive strategy 549

- forecasting regime 550, 551
- training regime 549

red noise process 9

regression 118

- time-series forecasting 122

regularization 120

- from geometric perspective 187-193

regularized linear regression 186, 187

regular time series 4

related time series 125

relative error

- Average Relative Root Mean Squared Error 575
- Geometric Mean Relative Absolute Error 575
- Mean Relative Absolute Error (MRAE) 574

Median Relative Absolute Error 574

Relative Mean Absolute Error (RelMAE) 575

Relative Root Mean Squared
Error (RelRMSE) 575

Repeated Holdout (No Overlap) (Rep-Holdout-O) strategy 597

Repeated Holdout (No Overlap) with Gaps (Rep-Holdout-O(G)) strategy 598

repeated holdout (Rep-Holdout) strategy 596, 597

representation learning 282, 283

residual spectral entropy 107

residual variability (RV) 104

ridge regression 186

rigorous mathematical procedure 165

RNN-to-fully connected network, Seq2Seq models 343-345

RNN-to-RNN, Seq2Seq models 345-353

rolling window aggregations 134-136

S

sampling procedure

- balancing 408, 409
- data distribution, visualizing 409, 410
- dataloader, using with
WeightedRandomSampler 411
- dataloader, visualizing with
WeightedRandomSampler 411, 412
- tweaking 410, 411

sampling strategy 593, 595

scaled dot product attention 361

scaled error 575

Mean Absolute Scaled Error (MASE) 575

Root Mean Squared Scaled Error (RMSSE) 575

search space subject 251

seasonal ARIMA 89

seasonal box plots 56, 57

- seasonal component** 51
- seasonal_decompose**
 - from statsmodel 63, 64
- seasonal decomposition** 60
- seasonal ESD (S-ESD)** 73
- seasonality**
 - correcting 161
 - detecting 161-163
- seasonal naive forecast** 84, 85
- seasonal plots** 55, 56
- seasonal rolling window aggregations** 136-138
- sequence-to-sequence (Seq2Seq) models** 343
 - forecasting 364-367
 - RNN-to-fully connected network 343-345
 - RNN-to-RNN 345-353
- Series Decomposition block** 434
- SHAP (SHapley Additive exPlanations)** 266
- SHAPTime** 539
- sigmoid** 285
- simple exponential smoothing (SES)** 86, 539
- simple hill climbing** 218
- simulated annealing** 222-225
- single exponential smoothing** 85
- single-step-ahead recurrent neural networks** 332-343
- single-step forecast baselines**
 - generating 176, 177
- skimming** 217
- slicing** 27, 28
- slope** 291
- softmax function** 289, 290, 361, 363
- span** 139, 140
- sparse demand forecasting** 538
- sparsemax** 364
- Spearman's rank correlation**
 - using, for error measures
 - experimental study 584-587
- specialized architectures**
 - need for 416
- squared error** 571
 - Geometric Root Mean Squared Error 572
 - Mean Squared Error 571
 - Root Mean Squared Error (RMSE) 572
 - Root Median Squared Error 572
- stacked model** 229
- stacking** 229
- standard deviation** 71, 103
- Static Covariate Encoders** 449
- static/meta information**
 - embedding vector 405
 - model, defining with categorical features 405-407
 - one-hot encoding 404
 - using 404
 - vector representation 405
- stationarity** 147, 148
- stationary time series** 14
- stochastic gradient descent (SGD)** 297
 - advantages 297
 - disadvantages 297
- stochastic hill climbing** 221, 222
- Stochastic Processes** 436
- stochastic trend** 154
- strategies, for combining forecasts** 214
 - best fit forecast 215
 - measures of central tendency 216, 217
 - optimal weighted ensemble 226-228
 - simple hill climbing 218-220
 - simulated annealing 222-225
 - stochastic hill climbing 221, 222
- strftime convention**
 - URL 25, 26
- strict stationarity** 148
- stride** 321
- sum of squared errors (SSE)** 108

supervised machine learning tasks 118

symmetric error 572

Symmetric Mean Absolute Percent Error (sMAPE) 573

Symmetric Median Absolute Percent Error 573

synthetic time series

autoregressive (AR) signal 12

cyclical or seasonal signals 10-12

generating 7

mix and match 13

red noise process 9

white noise process 7

T

tabular regression 328

tanh 286

target leakage 131

identifying 132

target mean encoding 247-249

target transformation

AutoML approach 169-172

TBATS 93, 94

ARMA 97, 98

Box-Cox transformation 94, 95

Fourier series, using to model seasonality 96

Locally Estimated Scatterplot Smoothing (LOESS) 96

parameter optimization 98, 99

Temporal Convolutional Network (TCN) 427

temporal embedding 130, 141, 433

calendar features 142

Fourier terms 143-145

time elapsed 142, 143

Temporal Fusion Decoder (TD) 450

Temporal Fusion Transformer (TFT) 448

architecture 448-450

forecasting with 454

gated residual networks 453

interpreting 455

Locality Enhancement Seq2Seq layer 451

temporal fusion decoder 452, 453

variable selection networks 454

temporal interpolation 430

temporal relevance 568

tensors 305

test harness

evaluation metric, selecting 79, 80

holdout (test), creating 78, 79

setting up 78

validation datasets, creating 78, 79

Theta forecast 91, 92

Theta lines 91

time delay embedding 123, 124, 130-132

exponentially weighted moving average (EWMA) 139-141

lags/backshift 133, 134

rolling window aggregations 134-136

seasonal rolling window aggregations 136-138

Time Elapsed column 101

Time Interpret 539

time series 4

forecastability, assessing 103

forecasting 17, 18

irregular time series 4

regular time series 4

scale, using 407

time series analysis 3

application areas 4, 5

interpretation and causality 5

outlier detection 5

time series classification 5

time series forecasting 4

time series components 50

cyclical 51

irregular 51, 52

seasonal 51

trend 50

time series data 77

- Acorn classes 22
- additional information, mapping 37, 38
- hhblock, converting into 34
- visualizing 52

time series dataset 22, 23

- regular intervals, enforcing 35, 36

TimeSeriesDataset 394

- dataloader, creating 396
- initializing 395
- parameters 333, 394

time series dataset, formatting

- compact data 34, 35
- expanded data 34, 35
- wide data 34, 35

time series, decomposing 60

- deseasonalizing 60, 61
- detrending 60
- Fourier decomposition 66-68
- multiple seasonality decomposition, with LOESS 68-70
- seasonal_decompose, from statsmodel 63, 64
- seasonality and trend decomposition, with LOESS 64, 66

Time Series Dense Encoder (TiDE) 462

- architecture 462
- decoder 465
- encoder 465
- forecasting with 466, 467
- Multi-Layer Perceptron (MLP) based architecture 462
- residual block 462, 463

Time Series Feature Extraction Library 261**time series forecasting 77, 80, 538**

- cold-start forecasting 540
- hierarchical forecasting 541
- intermittent or sparse demand forecasting 538, 539
- interpretability 539

- temporal embedding 124
- time delay embedding 123, 124
- using, as regression 122

time series format

- London Smart Meters dataset, converting into 36, 37

time series meta-features

- frequency encoding 245, 246
- LightGBM's native handling, of categorical features 249-251
- one-hot encoding 244
- ordinal encoding 244
- target mean encoding 247-249
- using 243

TimeSHAP 539**time-varying information**

- using 402-404

Tracking Signal (TS) 573**traditional programming**

- versus machine learning 116

training process 118**train-test contamination 131****Transformers 367**

- decoder 379
- encoder 376
- forecasting with 381-385
- in time series 380
- multi-headed attention 370-372
- positional encoding 372-374
- position-wise feed-forward layer 375
- self-attention 368-370

Tree Parzen Estimator (TPE) sampler 256**trend component 50, 60****trends**

- correcting 154
- detecting 154
- deterministic trend 154-156
- detrending transform 160

Kendall's Tau 156, 157
Mann-Kendall test (M-K test) 157-160
stochastic trend 154

trimming 217

triple exponential smoothing 86

TSMixer 456

architecture 457
forecasting with 461, 462
mixer layer 458, 459
temporal projection layer 459, 460

TSMixerx 460, 461

forecasting with 461, 462

U

Uniform Input Representation 432

unit roots 148-152

correcting 150
detecting 150

V

validation datasets

creating 78, 79

validation set 122

validation strategy

for datasets with multiple time series 602
selecting 601

value embedding 433

vanilla RNN 310

vanilla Transformer model 368

vanishing gradients 313

Variable Selection Network (VSN) 449

vector 280

vector space 280

visualization techniques, for time series datasets

autocorrelation plots 58-60

calendar heatmaps 58
line charts 52-55
seasonal box plots 56, 57
seasonal plots 55, 56

W

WaveNet 427

weak stationarity 148

Weighted Average Percent Error (WAPE) 571, 572

Weighted Correction (CP_Wtd) 538

Welch method 107

white noise process 7

White test 165

wide data 34

Wiener-Khinchin theorem 436

window strategy 593, 594

